

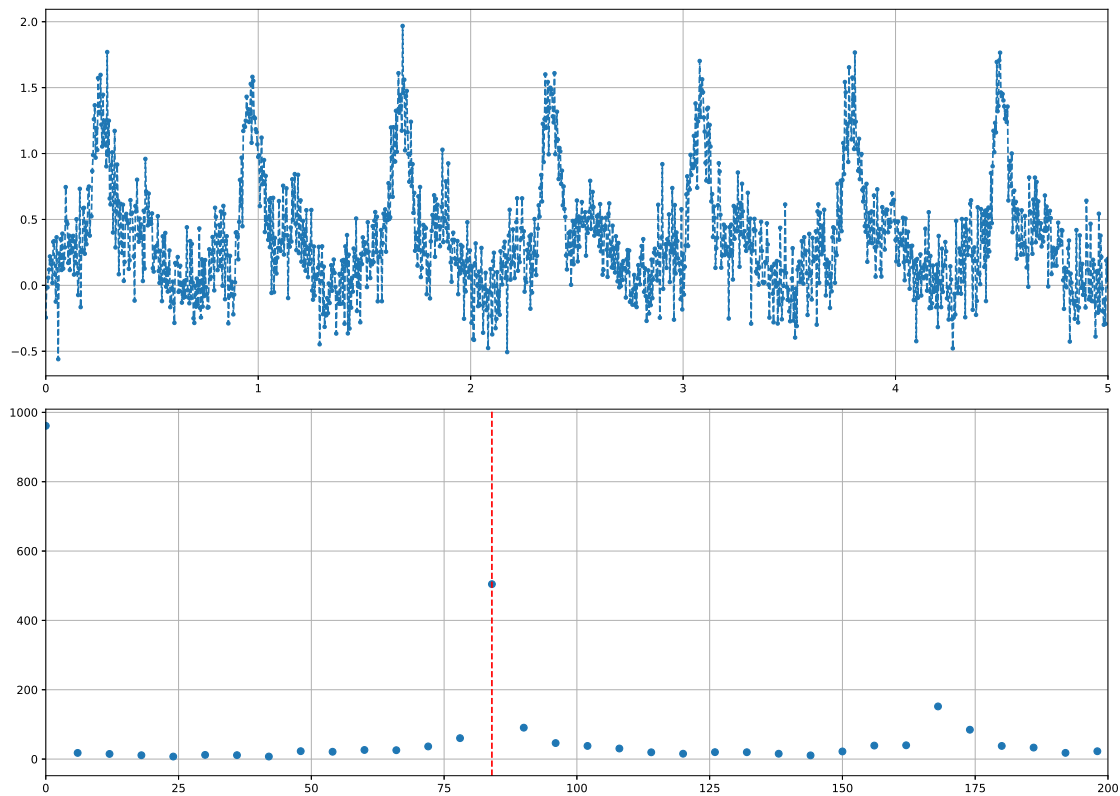
NUMERISCHE METHODEN

Vasile Gradinaru

Dokument zur Vorlesung
401-0663-00 Numerische Methoden D-INFK
an der ETH Zürich
2025-.

©Seminar für Angewandte Mathematik, ETH Zürich¹

Generiert am August 1, 2026



¹Alle Rechte vorbehalten; keine Vervielfältigung jeglicher Art oder Umfang, kein Upload bei einem genAI oder sonst ist ohne die Zustimmung des Autors erlaubt. Rechtschreibung wurde teilweise mit Gemini Pro korrigiert.

Denken vor Rechnen!

Ziele dieser Vorlesung

Die numerischen Methoden sind allgegenwärtig in der modernen Wissenschaft und Technik, vor allem als Komponenten der Computersimulationen, aber auch als Hilfswerkzeuge bei Experimenten oder bei der Entwicklung neuer Theorien. Der Experimentator sollte wissen, wie die **Ergebnisse**, die seine Geräte liefern, **zustande kommen**, wie zuverlässig sie sind, warum manche Artefakte entstehen und wie diese zu vermeiden sind. Beim Aufstellen neuer Theorien braucht man oft Berechnungen, die nicht analytisch durchführbar sind. Oder der Rechner ist in manchen Fällen einfach schneller ... Man baut oft ein neues mathematisches Modell für ein physikalisches Problem und testet es, ob die Ergebnisse etwas mit der Realität zu tun haben. Damit dies möglich ist, muss man numerische Methoden wählen, die zur Lösung des Problems geeignet sind. Wir werden sehen, dass dies nicht so einfach ist, sobald das mathematische Modell nicht trivial ist. Wir können numerischen Approximationen zur Lösung mathematischer Probleme nur trauen, wenn diese numerischen Lösungen sehr **sorgfältig** berechnet worden sind. Das genaue Wissen, wo die **Grenzen** eines numerischen Verfahrens liegen und was typischerweise passiert, wenn diese Grenzen nicht eingehalten werden, ist sogar wichtiger als das Verfahren selbst, das man einfach in einem Buch oder im Internet finden kann: Wir wollen die Eigenschaften eines Modells sicher nicht mit denen eines Verfahrens verwechseln! Das Aneignen dieses Wissens geht nur über Probieren und Experimentieren, wenn die Zeit für die fundierte mathematische Analyse verschiedener Verfahren fehlt. Deswegen ist der Inhalt dieser Vorlesung **implementierungs- und experimentierungsorientiert**: Die Studenten müssen **selbst** die Methoden in **verschiedenen Umgebungen** ausprobieren, um Erfahrungen zu sammeln.

Lernziele dieser Vorlesung sind:

- Übersicht über die wichtigsten Algorithmen zur Lösung der grundlegenden numerischen Probleme in der Physik und ihren Anwendungen;
- Übersicht über Softwarequellen (engl. *Software-Repositories*) zur Problemlösung;
- Fertigkeit, konkrete Probleme mit diesen Werkzeugen numerisch zu lösen;
- Fähigkeit, numerische Resultate zu interpretieren.

Der Schwerpunkt liegt auf dem Erwerb von Fertigkeiten in der Anwendung von numerischen Verfahren: Algorithmen (Prinzipien, Ziele und Grenzen) und Implementierungen (Effizienz und Stabilität) werden mittels relevanter numerischer Experimente (Design und Interpretation) vorgestellt. Wir werden Theorien und Beweise nur dann vorbringen, wenn es für das Verständnis der Algorithmen essenziell erscheint.

Achtung: Die hier eingebundenen Codes wurden über die Jahre von verschiedenen Personen entwickelt. Mit der Zeit ändern sich die Standards, sodass es sein kann, dass die aktuelle Syntax und die verwendeten Konventionen nicht immer übereinstimmen oder nicht ganz konsistent sind. Verschiedene Geräte können auch leicht unterschiedliche Versionen von Software haben; das ist eine andere Quelle für mögliche Abweichungen.

Materialien

Viele Ideen und Materialien, die in dieses Skript eingeflossen sind, sind aus den Diskussionen und Erfahrungen am Seminar für Angewandte Mathematik der ETH Zürich entstanden. Meinen Kollegen bin ich dafür sowie für deren Vertrauen, mich diese neue und aufregende Vorlesung entwickeln zu lassen, sehr dankbar. Der Autor ist von den vorhandenen Materialien der Folien *Numerical Methods for CSE* und *Numerische Mathematik*, die vor allem von Prof. Ralf Hiptmair entwickelt worden sind, ausgegangen. Ich danke auch meinen Assistierenden und den zahlreichen Studenten der Studiengänge Physik und Rechnergestützte Wissenschaften (RW/CSE), die mir Kommentare und Verbesserungsvorschläge zur Vorlesung und zum Skript gegeben haben.

Folgende Bücher wurden für die Vorbereitung dieses Skriptes verwendet und können teilweise als Begleitlektüre zur Vorlesung in Betracht gezogen werden.

- * M. HANKE-BOURGEOIS, *Grundlagen der Numerischen Mathematik und des Wissenschaftlichen Rechnens*, Mathematische Leitfäden, B.G. Teubner, Stuttgart, 2009, [NEBIS 009860519](#)
- * P. DEUFLHARD AND A. HOHMANN, *Numerische Mathematik. Eine algorithmisch orientierte Einführung*, DeGruyter, Berlin, 2008, [NEBIS 009855563](#)
- * J. STOER AND R. BULIRSCH, *Einführung in die Numerische Mathematik*, Springer Verlag, 2005, [NEBIS 009840490](#)
- * W. DAHMEN AND A. REUSKEN, *Numerik für Ingenieure und Naturwissenschaftler*, Springer 2008, [NEBIS 00985199](#)
- * A. QUARTERONI, R. SACCO AND F. SALERI, *Numerische Mathematik 1 + 2*, Springer 2007, [NEBIS 009842244](#)
- * L.N. TREFETHEN AND D. BAU, *Numerical Linear Algebra*, SIAM 1997, [NEBIS 009819168](#)
- * G. STRANG, *Wissenschaftliches Rechnen*, Springer 2010, [NEBIS 009860694](#)
- * P. DEVRIES AND J. HASBUN, *A First Course in Computational Physics*, Jones and Bartlett Publishers; 2 edition, 2010, [NEBIS 006032942](#)
- * W.L. DUNN, J.K. SHULTIS, *Exploring Monte Carlo Methods*, Amsterdam, Elsevier, 2012, [NEBIS 009882256](#)

- * T. MÜLLER-GRONBACH, E. NOVAK, K. RITTER, *Monte-Carlo-Algorithmen*, Berlin, Springer, 2012, [NEBIS 009889710](#)
- * H.P. LANGTANGEN, *A primer on scientific programming with Python*, Springer, 2012, [NEBIS 009893026](#)
- * H.P. LANGTANGEN, *Python scripting for computational science*, Springer, 2009, [NEBIS 009859422](#)
- * [Python Scientific Lecture Notes](#)

Erwartet werden solide Kenntnisse in Analysis (Approximation und Vektoranalysis) und linearer Algebra (Gauss-Elimination, Matrixzerlegungen sowie Algorithmen, Vektor- und Matrizenrechnung: Matrixmultiplikation, Determinante, LU-Zerlegung nicht singulärer Matrizen).

Die meistverwendeten Python-Pakete in diesem Dokument sind `numpy`, `scipy`, `matplotlib`, `sympy`. Das Paket `pandas` kann auch sehr nützlich sein, wurde hier aber nicht verwendet. Es wird ausdrücklich empfohlen, die Dokumentation zu den verwendeten Funktionen in jeder Sektion genau anzusehen.

Die Prüfung findet zurzeit am Computer mit **CodeExpert** statt, was auch für die Übungen in Verwendung ist; dort sind alle nötigen Pakete installiert. Einige arbeiten lieber mit Jupyter-Notebooks oder IDEs wie `code`, `PyCharm`, `Spyder`, `Atom`, die aber bei der Prüfung nicht zur Verfügung stehen. Eine minimale Methode, um für sich zu arbeiten, ist, nur einen einfachen Texteditor und `ipython` in der Shell zu verwenden. Wir gehen einfach davon aus, dass die Studierenden selbst einen Einstieg in (einfaches) Python mit wissenschaftlichen Paketen gefunden haben. Notfalls empfiehlt es sich, die [Python Scientific Lecture Notes](#) anzusehen.

Contents

1	Vor dem Start	6
1.1	Rundungsfehler und Fehlerpropagation	6
1.2	Rechenaufwand	21
1.3	Rechnen mit Matrizen	26
2	Polynominterpolation	41
2.1	Interpolation und Polynome	41
2.2	Newton-Basis und dividierte Differenzen	44
2.3	Lagrange- und baryzentrische Interpolationsformeln	49
2.4	Chebyshev-Interpolation	54
2.4.1	Fehlerbetrachtung	58
2.4.2	Interpolation und Auswertung	61
3	Trigonometrische Interpolation	65
3.1	Trigonometrische Polynome und Fourier-Reihen	65
3.2	Diskrete Fouriertransformation	71
3.2.1	Motivation: von der Approximation zur Interpolation	71
3.2.2	Konstruktion	72
3.2.3	<code>fftshift</code> und erste Anwendungen	75
3.2.4	Ein Blick aus der Perspektive der Linearen Algebra	82
3.3	FFT – Schnelle Fourier-Transformation	86
3.4	Trigonometrische Interpolation und rechentechnische Aspekte	91
3.4.1	Wiederholung: von der Approximation zur Interpolation	91
3.4.2	Auswertung: "Zero-Padding"	94
3.5	Trigonometrische Interpolation und Fehlerabschätzungen	95
3.6	DFT und Chebyshev-Interpolation	102
4	Stückweise Polynomiale Interpolation	110
4.1	Stückweise lineare Interpolation	110
4.2	Kubische Hermite-Interpolation	111
4.3	Splines	114

5	Numerische Quadratur	119
5.1	Motivation	119
5.2	Schneller Überblick	119
5.3	Grundbegriffe und -ideen	120
5.4	Äquidistante Stützstellen	126
5.5	Nicht äquidistante Stützstellen	139
5.5.1	Gauss Quadratur	139
5.5.2	Clenshaw-Curtis Quadraturformel	146
5.6	Adaptive Quadratur	151
5.7	Quadratur in $\mathbb{R}^d$ und dünne Gitter	154
5.8	Monte-Carlo Quadratur	164
5.9	Methoden zur Reduktion der Varianz	179
6	Nullstellensuche	194
6.1	Iterative Verfahren	194
6.2	Abbruchkriterien	201
6.3	Fixpunktiteration	203
6.4	Intervallhalbierungsverfahren (nur für $d = 1$)	209
6.5	Newton-Verfahren in 1 Dimension	210
6.6	Sekantenverfahren	212
6.7	Newton-Verfahren in d Dimensionen	214
6.8	Gedämpftes Newton-Verfahren	223
6.9	Das Quasi-Newton Verfahren	228
7	Intermezzo über (numerische) lineare Algebra	234
7.1.1	Grundlagen	234
7.1.2	Wichtige Zerlegungen	237
7.1.3	Ausgewählte Funktionen aus <code>numpy</code> , <code>scipy</code>	263
8	Ausgleichsrechnung	266
8.1	Lineare Ausgleichsrechnung	267
8.1.1	Normalengleichungen	269
8.1.2	Lösung mittels orthogonaler Transformationen	272
8.1.3	Totale Ausgleichsrechnung	276
8.2	Nichtlineare Ausgleichsrechnung	277
8.2.1	Newton-Verfahren im $\mathbb{R}^n$	278
8.2.2	Gauss-Newton Verfahren	279
8.2.3	Weitere Methoden: BFGS, GD, SGD, CG, LM, ADAM	289
8.3	DNNs	298

Chapter 1

Vor dem Start

1.1 Rundungsfehler und Fehlerpropagation

In der Mathematik haben Sie bisher meistens exakte Formeln (z. B. für die Lösung einer quadratischen Gleichung) oder Definitionen und Ergebnisse in Form von Grenzwerten (z. B. Definition der Ableitung einer Funktion, des Riemannschen Integrals, der Taylor-Reihe, der Exponentialfunktion) oder in Form von Existenzaussagen (z. B. Existenz von n Eigenwerten einer $n \times n$ -Matrix) kennengelernt. Oft kann man jedoch diese Größen nicht exakt (analytisch) berechnen, weshalb man mit numerischen Approximationen arbeiten muss: z. B. mit einer abgebrochenen Taylor-Reihe oder einem Glied einer konvergenten Folge. **Approximationen, die die Rechenzeit, den nötigen Speicher und den Fehler in möglichst allgemeinen Fällen gleichzeitig optimieren, sind das Ziel der Numerischen Mathematik.** Wichtig dabei ist, dass jede dieser Größen (Rechenzeit, Speicher, Fehler) präzise in Abhängigkeit von Diskretisierungsparametern (die dem Nutzer zur Verfügung stehen) quantifizierbar ist; der Fehler sollte systematisch durch eine entsprechende Wahl der Diskretisierungsparameter verkleinert werden können (auf Kosten der Rechenzeit und/oder des Speicheraufwands). Damit befassen wir uns in den nächsten Kapiteln.

Die **Rundungsfehleranalyse**, die wir hier kurz ansprechen, ist **nicht wesentlich!** Es ist zwar wichtig, die offensichtlich vermeidbaren Fehler zu vermeiden und auch immer einen kritischen Blick auf die Algorithmen und numerischen Ergebnisse zu behalten; die **Ideen, die zu Approximationen und effizienten Algorithmen führen, sind uns jedoch wichtiger.** Dort, wo sich die Rundungsfehler und die Fehlerfortpflanzung auf die Wahl eines Algorithmus auswirken, werden wir das auch betonen und vielleicht gelegentlich die Rundungsfehleranalyse durchführen, ansonsten nicht. Trotzdem ist es gut zu wissen, was bei der Rundung schiefgehen kann.

Definition 1.1.1. (relativer, absoluter Fehler) Sei $\tilde{\mathbf{x}} \in \mathbb{R}^n$ eine Approximation an $\mathbf{x} \in \mathbb{R}^n$. Der absolute Fehler ist $\|\tilde{\mathbf{x}} - \mathbf{x}\|$, wobei $\|\cdot\|$ eine Norm im $\mathbb{R}^n$ ist. Wenn $\tilde{\mathbf{x}} \neq 0$, dann betrachten wir auch den relativen Fehler $\|\tilde{\mathbf{x}} - \mathbf{x}\|/\|\mathbf{x}\|$.

Beispiel 1.1.2. Rundungsfehler sind bereits bei den einfachsten Operationen vorhanden.

Code 1.1.3: Rundungsfehler

```

1  a = 4/3; b = a - 1; c = 3*b; e = 1-c
   print(e) # exact should be 0, get 2.220446049250313e-16
3
   a = 1012/113; b = a-9; c = 113*b; e = 5+c
5   print(e) # exact should be 0, get 6.750155989720952e-14
7
   a = 83810206/6789
   b = a - 12345
9   c = 6789*b
   e = c-1
11  print(e) # exact should be 0, get -1.6079866327345371e-09

```

Meistens liefern selbst analytische Formeln nicht die exakte Lösung. Der Grund dafür ist, dass Computer nur endlich viele Zahlen exakt darstellen können; alle anderen Zahlen werden durch darstellbare Zahlen (Maschinenzahlen) approximiert. Die Menge der Maschinenzahlen ist eine diskrete Teilmenge von $\mathbb{R}$, die bezüglich der arithmetischen Operationen $+$, $-$, $\cdot$, $/$ nicht abgeschlossen ist, sodass Rundung und Rundungsfehler unvermeidbar sind. Die Maschinenzahlen werden als Gleitkommazahlen dargestellt. Gegeben seien eine Basis $B \in \mathbb{N}$, $B \geq 2$, ein Bereich von Exponenten $\{e_{\min}, \dots, e_{\max}\} \subset \mathbb{Z}$ und die Anzahl an Ziffern $m \in \mathbb{N}$ (für die Mantisse d). Dann ist die Menge der Maschinenzahlen

$$\mathcal{M} = \{d \cdot B^E, \text{ mit } d = i \cdot B^{-m}, i = B^{m-1}, \dots, B^m - 1, E \in \{e_{\min}, \dots, e_{\max}\}\}.$$

Diese Zahlen haben die Form

$$x = \pm 0.a_1 a_2 \dots a_m \cdot B^e$$

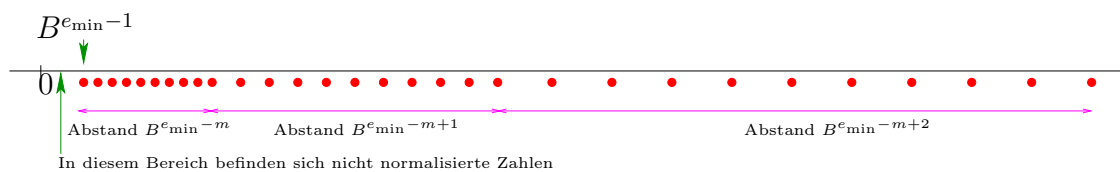
normalisiert mit der Bedingung $a_1 \neq 0$, falls $x \neq 0$; z. B. wird -1.473 als $-0.1473 \cdot 10^1$ und $14.7 \cdot 10^{-4}$ als $0.147 \cdot 10^{-2}$ dargestellt. Die wichtigsten Informationen über die zur Verfügung stehenden Maschinenzahlen (wie z. B. Extrema und Maschinengenauigkeit) erhält man mit `numpy.finfo`. Probieren Sie:

```

numpy.finfo(np.float64).resolution,
numpy.finfo(np.float64).tiny,
numpy.finfo(np.float64).max.

```

Die Abstände zwischen den Maschinenzahlen sind für grosse Zahlen grösser:



Manchmal trifft man auch die `eps`-Zahlen an; probieren Sie `numpy.finfo(float64).eps` für die Differenz zwischen 1.0 und der nächstgrösseren Maschinenzahl ($2.220446049250313 \cdot 10^{-16}$ auf meinem Rechner) und `numpy.finfo(float64).epsneg` für die Differenz

zwischen 1.0 und der nächstkleineren Maschinenzahl ($1.1102230246251565 \cdot 10^{-16}$ auf meinem Rechner). Bei der Approximation reeller Zahlen durch Maschinenzahlen entstehen Rundungsfehler. Wenn sich zwei positive Maschinenzahlen x und $\tilde{x}$ erst ab der $(\ell + 1)$ -ten Ziffer unterscheiden, dann ist $|1 - \frac{\tilde{x}}{x}| < 10^{-\ell}$. So möchte man immer einen relativen Fehler $\|\tilde{x} - x\|/\|x\| < 10^{-\ell}$ haben, um diese «Genauigkeitsgüte» zu erreichen.

Beispiel 1.1.4. (Unterlauf und Überlauf)

Code 1.1.5: Unterlauf und Überlauf

```

1  import numpy as np
   np.seterr(under="warn")
3
   numbers = np.array([0.5,0.1,0.03,1e-16])
5  print(np.sum(numbers**20))
7
   numbers = np.array([42.1,-42,-0.1])
   should_be_zero = np.sum(numbers)
9  print(should_be_zero) # 1.4155343563970746e-15
   assert should_be_zero**30 == 0 # don't compare floats via ==
11 # correct way to compare floats:
   tol = 1E-16 # np.finfo(float64).eps is better
13 s = 1. # compared to which |x| should be small
   x = should_be_zero
15 if( abs(x) < tol*s): print(x, " is 0 for us")
   else: print(x," is not 0 for us")
17 x = should_be_zero/100
   if( abs(x) < tol*s): print(x, " is 0 for us")
19 else: print(x," is not 0 for us")
21
   print( np.prod(10**np.arange(30.)) ) # np.float64(inf)
   print( np.uint8(23) - np.uint8(42) ) # np.uint8(237)
23
   print(np.array([1, 2]) / 0.0) # inf
   print(np.array([0, 0]) / 0.0) # NaN

```

Wenn das Ergebnis einer arithmetischen Operation über der grössten Maschinenzahl liegt, sprechen wir von Überlauf (Overflow); wenn das Ergebnis zwischen 0 und der kleinsten positiven Maschinenzahl liegt, sprechen wir von Unterlauf (Underflow). Unterlauf wird oft ignoriert (die entsprechenden Warnungen sind a priori ausgeschaltet), da er meistens harmlos ist, jedoch kann er zu einem Genauigkeitsverlust führen. Überläufe sind problematischer, darum gibt es meistens auch Warnungen dazu. Spezialfälle sind `inf` und `NaN`. Die Auswirkungen von Rundungsfehlern in einem komplizierten Algorithmus können unter Umständen sehr schwierig bis unmöglich vorauszusehen sein. Jedoch gibt es Fälle, in denen das klar ist, also vermeidbare Fälle.

Beispiel 1.1.6. Die Berechnung der Nullstellen eines quadratischen Polynoms mit der Standardformel (die Sie aus der Schule kennen) ist instabil. Wir beschränken uns auf den einfachen Fall $p(x) = x^2 + ax + b$. Der folgende Code implementiert direkt

die Formel mit der Diskriminante, die jede Schülerin und jeder Schüler auswendig lernt. Wir wenden dann diese Formel für $p(x) = (x-c)(x-1/c) = x^2 - (c+1/c)x + 1$ an, also ist $a = -(c+1/c)$ und $b = 1$. Wir testen mit 101 äquidistanten Werten für c zwischen 2 und 1000. Anschliessend plotten wir die relativen Fehler der beiden gefundenen Nullstellen von p .

Code 1.1.7: Instabile Berechnung der Lösungen einer quadratischen Gleichung

```

import numpy as np
2
def zerosquadpol(a,b):
4     """
    compute the real roots of p(x) = x**2 + a*x + b
6     with the standard discriminant formula
    caution: round-off!
8     """

    z = np.array([])
    D = a**2 - 4*b # vectorized
12    if D.any()<0: print('no real zeros') # one of them
    else:
14        wD = np.sqrt(D)
        z = np.array([ (-a-wD)/2 , (-a+wD)/2 ])
16
    return(z)
18
    """
20    Test this function for p(x) = (x-c)*(x-1/c)
    a and b are computed from c, which introduces a small relative roundoff error
22    """

    n = 100 # how many samples
24    cs, cL= 2, 1000
    c = np.linspace(cs, cL, num=n+1, endpoint=True) # all at once
26    a = -(c+1/c)
    b = 1. + 0*c
28    z = zerosquadpol(a,b) # vectorized: all at once
    z0 = z[0] # school book computed "smaller zero"
30    z1 = z[1] # school book computed "larger zero"
    x0 = 1/c # actual "smaller zero"
32    x1 = c # actual "larger zero"
    # plotting settings -----
34    import matplotlib.pyplot as plt
    params = {'axes.labelsize': 20,
36              'legend.fontsize': 20,
              'figure.figsize' : (18,12),
38              'xtick.labelsize': 16,
              'lines.markersize': 12,
40              'ytick.labelsize': 16
              }

42    #-----
    # plot the relative errors
44    plt.rcParams.update(params)
    plt.semilogy(c, abs(z0-x0)/x0, 'r+', label='small root')

```

```

46 plt.semilogy(c, abs(z1-x1)/x1, 'b.', label='large root')
   plt.ylabel('relative error')
48 plt.xlabel('c')
   plt.grid()
50 plt.legend()
   plt.savefig('qprootsunstable.eps')
52 plt.savefig('qprootsunstable.pdf')
   plt.savefig('qprootsunstable.png')
54 plt.show()

```

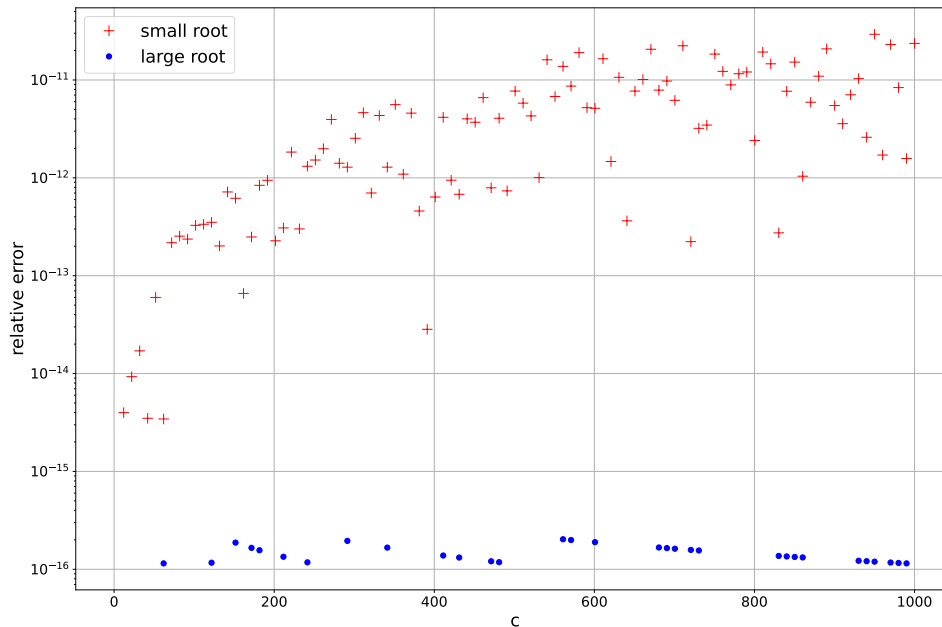


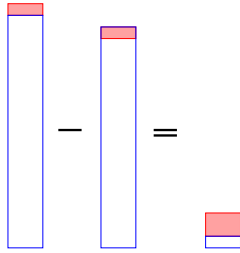
Abb. 1.1.8. Rundungsfehler in den mit der Standardformel berechneten zwei Nullstellen von $p(x) = (x - c)(x - 1/c)$.

Die Berechnung von a hat kleine Rundungsfehler in der Grössenordnung von `numpy.finfo(float64).eps` $\approx 10^{-16}$ produziert, was sich in den relativen Fehlern von $\approx 10^{-11}$ in der Approximation der kleinsten Nullstelle und $\approx 10^{-16}$ in der Approximation der grössten Nullstelle niederschlägt, vor allem für grössere Werte von c . Der wesentliche Unterschied in der Berechnung der beiden Nullstellen ist, dass wir bei der kleinsten Nullstelle eine Subtraktion statt einer Addition haben.

Bemerkung 1.1.9. Wenn wir zwei fast gleich grosse Zahlen, die beide kleine relative Fehler aufweisen, subtrahieren, erhalten wir einen kleinen Wert, doch die Fehler können sich addieren, sodass wir relativ zum kleinen Wert des Ergebnisses einen riesigen Fehler haben. Diese extreme Vergrösserung des relativen Fehlers bei der

Subtraktion zweier fast gleicher Zahlen heisst Auslöschung.

(absoluter) Fehler



Auslöschung

$\hat{=}$ Subtraktion zweier ungefähr gleich grosser Zahlen

($\Rightarrow$ extreme *Verstärkung* der *relativen* Fehler)

Der Rundungsfehler der Subtraktion selbst ist vernachlässigbar!

Beispiel 1.1.10. (Auslöschung im Dezimalsystem)

Wir betrachten zwei positive Zahlen x, y , die ungefähr gleich gross sind und die mit relativen Fehlern der Grössenordnung 10^{-7} behaftet sind. Das bedeutet, ab der 7. Stelle nach dem Komma sind die Ziffern verfälscht, was wir hier mit * bezeichnen. Bei der Subtraktion $x - y$ verschieben sich die verfälschten Ziffern nach links, sodass der relative Fehler des Ergebnisses $\approx 10^{-3}$ ist:

$$\begin{array}{rcl} x & = & 0.123467* \quad \leftarrow 7. \text{ Ziffer verfälscht} \\ y & = & 0.123456* \quad \leftarrow 7. \text{ Ziffer verfälscht} \\ \hline x - y & = & 0.000011* = 0.11*000 \cdot 10^{-4} \quad \leftarrow 3. \text{ Ziffer verfälscht} \end{array}$$

unwesentliche Nullen

Beispiel 1.1.11. (Auslöschung bei der Auswertung von Differenzenquotienten)

Aus der Analysis kennen wir die Definition der Ableitung einer differenzierbaren Funktion $f : I \subset \mathbb{R} \rightarrow \mathbb{R}$ in einem Punkt $x \in I$ als Grenzwert des Differenzenquotienten

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Daher kommt man auf die Idee, die Ableitung von f in x über den Differenzenquotienten für ein kleines h zu approximieren:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \text{ mit } |h| \ll 1.$$

Der Satz von Taylor um x sagt uns, dass der Fehler ungefähr wie h gegen 0 gehen sollte, wenn die Funktion mindestens zweimal stetig differenzierbar ist:

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(c)h^2 \text{ mit } c = c(x, h) \in [\min\{x, x+h\}, \max\{x, x+h\}],$$

d. h.

$$\frac{f(x+h) - f(x)}{h} - f'(x) = \frac{1}{2}f''(c)h.$$

Das lässt sich leicht implementieren. Wir probieren diese Idee für $f(x) = e^x$ und $x = 0$. Die vorherige Analyse sagt voraus, dass der relative Fehler gegen 0 zumindest

wie h sinken sollte.

Code 1.1.12: Ableitung via Differenzenquotient

```

from numpy import exp, sin, cos
f = exp; df = exp; x = 0
print(f.__name__, x)
h = 0.1
for k in range(1,21):
    ndf = (f(x+h) - f(x))/h
    print(
        'log(h) =', -k, 'rel. err. =',
        abs( (ndf-df(x))/df(x) )
    )
    h *= 0.1

```

$\log_{10}(h)$	relativer Fehler
-1	0.05170918075648
-2	0.00501670841679
-3	0.00050016670838
-4	0.00005000166714
-5	0.00000500000696
-6	0.00000049996218
-7	0.00000004943368
-8	0.00000000607747
-9	0.000000008274037
-10	0.000000008274037
-11	0.000000008274037
-12	0.00008890058234
-13	0.00079927783736
-14	0.00079927783736
-15	0.11022302462516
-16	1.00000000000000

Wir beobachten zuerst einen Abfall des relativen Fehlers, dann einen raschen Anstieg des relativen Fehlers, wenn $h < 10^{-8}$. Der Effekt der Auslöschung ist deutlich stärker als die Konvergenzgeschwindigkeit der Approximation mit dem Differenzenquotienten. Man kann die genaue Rundungsfehleranalyse in diesem Fall durchführen und erhält

$$\left| \frac{e^x - df}{e^x} \right| \approx |h| + \frac{2\text{eps}}{|h|} \rightarrow \min \quad \text{für } |h| = \sqrt{2 \text{eps}}.$$

`np.sqrt(2*np.finfo(np.float64).eps)` = $2.1073424255447017 \cdot 10^{-8}$.

Ist Auslöschung vermeidbar? Mit ein bisschen mehr Mathematik, ja!

Beispiel 1.1.13.

(Stabile Berechnung der Nullstellen eines quadratischen Polynoms)

Wir kehren zur Aufgabe der Nullstellen von $p(x) = x^2 + ax + b$ zurück. Um die Subtraktion zu vermeiden, kann man nur die Nullstelle ausrechnen, die keine Subtraktion erfordert, und die andere aufgrund des Satzes von Vieta ($x_1 x_2 = b$) als $x_2 = b/x_1$ berechnen. Eine stabile Implementierung ist:

Code 1.1.14: Stabile Lösung einer quadratischen Gleichung

```

def stabzerosquadpol(a,b):
    """
    compute the real roots of  $p(x) = x^2 + ax + b$ 
    with the standard discriminant formula
    avoiding cancellation
    """

    z = np.array([])
    D = a**2 - 4*b
    if D.any() < 0: print('no real zeros')
    else:
        wD = np.sqrt(D)
        z2 = np.where(a < 0, (wD-a)/2, (-wD-a)/2)
        z1 = b/z2
        z = np.array([z1, z2])
    return(z)

```

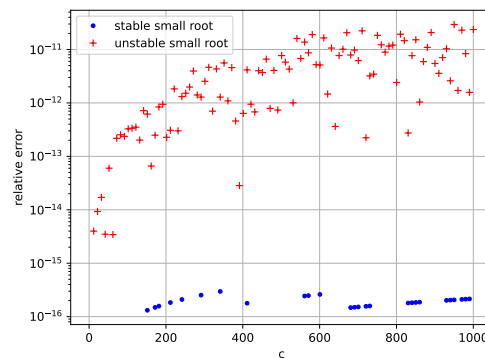
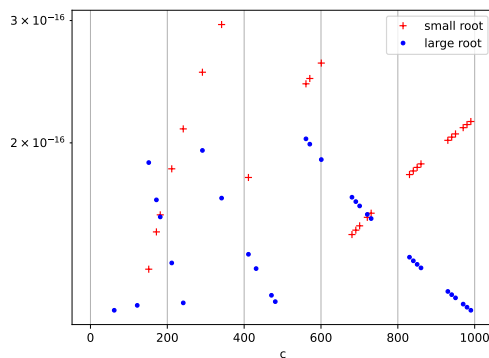


Abb. 1.1.15.

Links: stabile Berechnung der Nullstellen eines quadratischen Polynoms mit Vieta
 Rechts: Auslöschung bei der Berechnung der grösseren Nullstelle via Standardformel

Beispiel 1.1.16. (Summe von Potenzreihen)

Die Potenzreihe für e^x konvergiert überall, also könnte man e^x beliebig gut überall approximieren, indem man die Summe nach den ersten Termen der Potenzreihe abbricht.

Code 1.1.17: Summe der ersten Terme der Potenzreihe von e^x

```
import numpy as np
2 def expeval(x, tol=1e-8):
    y = 1.; term = 1. # first term
4    k = 1
    while abs(term) > tol*y:
6        term *= x/k # next summand
        y += term # summation
8        k += 1
    return y

10 xs = np.arange(-20,21,2)
12 ys = np.array([expeval(x) for x in xs])
zs = np.exp(xs)
14 A = np.array([xs, ys, zs, abs(ys-zs)/zs])
print(A.T)

16 def println(line):
18     opt = ""; opt += str( int(line[0]) ); opt += " & "
    opt += '{:.12e}'.format(line[1]); opt += " & "
20     opt += '{:.12e}'.format(line[2]); opt += " & "
    opt += '{:.16f}'.format(line[3])
22     return(opt)

24 print(" \\\n".join([println(line) for line in A.T]) )
```

x	Approximation $\widetilde{\exp}(x)$	$\exp(x)$	$\frac{ \exp(x) - \widetilde{\exp}(x) }{ \exp(x) }$
-20	6.147561824192e-09	2.061153622439e-09	1.9825830337278934
-18	1.598372035943e-08	1.522997974471e-08	0.0494905855000887
-16	1.124750330014e-07	1.125351747193e-07	0.0005344259515302
-14	8.315441787438e-07	8.315287191036e-07	0.0000185918296272
-12	6.144210514234e-06	6.144212353328e-06	0.0000002993214527
-10	4.539992960354e-05	4.539992976248e-05	0.0000000035010444
-8	3.354626281246e-04	3.354626279025e-04	0.0000000006620042
-6	2.478752175842e-03	2.478752176666e-03	0.0000000003325188
-4	1.831563887901e-02	1.831563888873e-02	0.0000000005307235
-2	1.353352831996e-01	1.353352832366e-01	0.0000000002736031
0	1.000000000000e+00	1.000000000000e+00	0.0000000000000000
2	7.389056095384e+00	7.389056098931e+00	0.0000000004799685
4	5.459814992815e+01	5.459815003314e+01	0.0000000019230582
6	4.034287929504e+02	4.034287934927e+02	0.0000000013442484
8	2.980957980774e+03	2.980957987042e+03	0.0000000021025843
10	2.202646574759e+04	2.202646579481e+04	0.0000000021437994
12	1.627547911384e+05	1.627547914190e+05	0.0000000017238449
14	1.202604279794e+06	1.202604284165e+06	0.0000000036341346
16	8.886110500976e+06	8.886110520508e+06	0.0000000021979897
18	6.565996891074e+07	6.565996913733e+07	0.0000000034509718
20	4.851651930671e+08	4.851651954098e+08	0.0000000048287371

Wir beobachten: Für grosse negative x ist die Approximation mit der Partialsumme der Potenzreihe katastrophal. In diesem Fall werden grosse Zahlen mit alternierenden Vorzeichen summiert, sodass die Auslöschung das Ergebnis bedeutungslos macht. Wir können dies vermeiden, indem wir $e^x = 1/e^{-x}$ für $x < 0$ verwenden.

Beispiel 1.1.18. (Ableitung via rein imaginärem Schritt) Um die Auslöschung in der Approximation der Ableitung $f'(x_0)$ mittels Differenzenquotienten zu vermeiden, bietet sich ein einfacher Trick an: ein rein imaginärer Schritt $ih \in \mathbb{C}$ mit $i^2 = -1$. Wir nehmen hier an, die Funktion f sei um x_0 sehr glatt, und zwar als Funktion einer komplexen Variable $z \in \mathbb{C}$. In der Sprache der Vorlesung *Mathematische Methoden* nehmen wir an, dass f in einer Umgebung von x_0 analytisch ist. Konkret bedeutet dies, dass f sich als die Summe einer Potenzreihe schreiben lässt, die in einem komplexen Kreis um x_0 konvergent ist:

$$f(z) = \sum_{j=0}^{\infty} a_j (z - x_0)^j \quad \text{für alle } z : |z - x_0| < \rho \quad \text{mit festem } \rho > 0, a_j \in \mathbb{R}.$$

Das bedeutet auch, dass f in einer Umgebung von x_0 unendlich oft differenzierbar ist und die Ableitungen in x_0 die Form $f^{(n)}(x_0) = n!a_n \in \mathbb{R}$ haben. Unter diesen Voraussetzungen können wir den Satz von Taylor mit einem rein imaginären Schritt schreiben:

$$f(x_0 + ih) = f(x_0) + f'(x_0)ih - \frac{1}{2}f''(x_0)h^2 - \frac{i}{6}f'''(x_0)h^3 + \mathcal{O}(h^4) \quad \text{für } \mathbb{R} \ni h \rightarrow 0.$$

Die Terme $f(x_0)$ und $-\frac{1}{2}f''(x_0)h^2$ sind reell, also verschwinden sie, wenn wir nur den Imaginärteil der obigen Gleichung betrachten:

$$\operatorname{Im}(f(x_0 + ih)) = hf'(x_0) - \frac{1}{6}f'''(x_0)h^3 + \mathcal{O}(h^5) \quad \text{für } \mathbb{R} \ni h \rightarrow 0.$$

Wir erhalten also eine Approximation der Ableitung, die so schnell wie h^2 konvergiert:

$$f'(x_0) = \frac{\operatorname{Im}(f(x_0 + ih))}{h} + \frac{1}{6}f'''(x_0)h^2 + \mathcal{O}(h^4) \quad \text{für } \mathbb{R} \ni h \rightarrow 0,$$

die auch keine Subtraktion beinhaltet:

$$f'(x_0) \approx \frac{\operatorname{Im}(f(x_0 + ih))}{h}.$$

Im Gegenzug brauchen wir die Auswertung der Funktion f für komplexwertige Argumente und deutlich mehr Glattheit der Funktion f in einer komplexen Umgebung von x_0 , was manchmal (aber nicht immer) gegeben ist. Mehr Glattheit (und mehr Mathematik) erlaubt eine genauere (Fehler verhält sich wie h^2 statt h) und auslöschungsfreie Formel, siehe Abbildung 1.1.19.

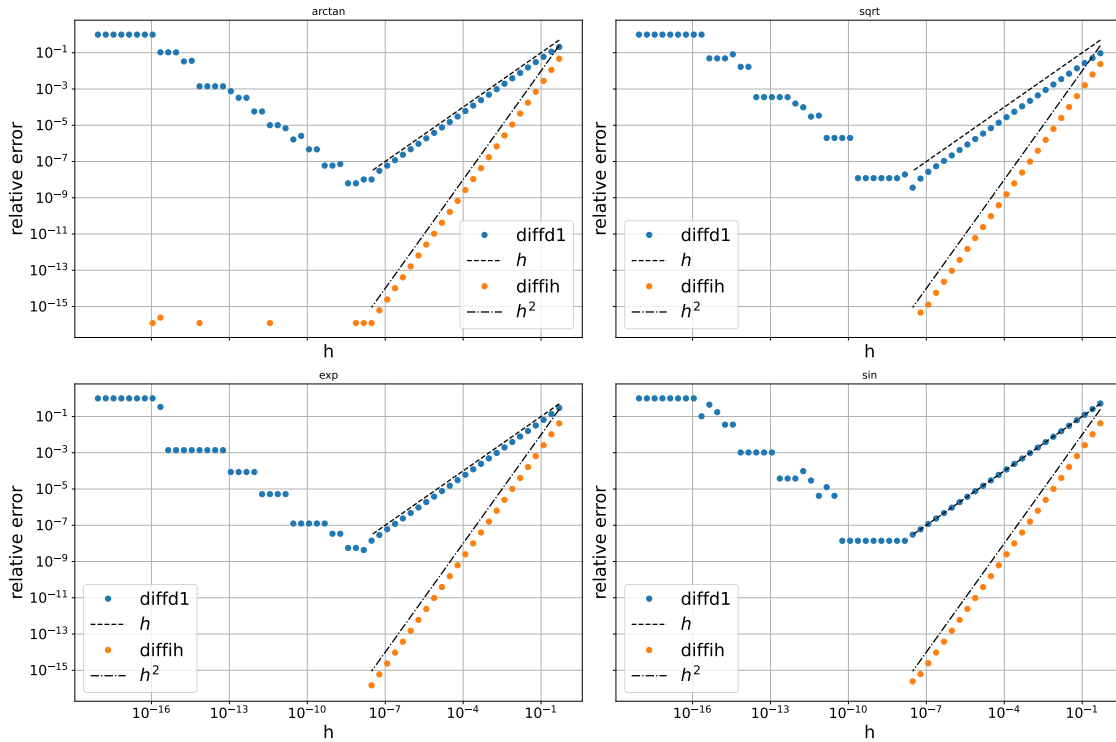


Abb. 1.1.19. Relativer Fehler bei der Berechnung der Ableitung in $x = 1.1$ einiger glatter Funktionen mit dem Differenzenquotienten und mit dem rein imaginären Schritt.

Die vorige Methode funktioniert perfekt, wenn die Funktion sehr glatt ist und die Auswertung von $\text{Im}(f(x + ih))$ exakt ist. Hätten wir bei der Auswertung von f erhebliche Fehler (z. B. wenn f nicht analytisch vorhanden ist, sondern nur Auswertungen davon über Experimente möglich sind), würden diese bei der Division durch ein (möglicherweise von den Genauigkeitsanforderungen an $f'(x)$ verlangtes) kleines h massiv vergrößert werden. Darum ist es wichtig, Alternativen zu haben. Das nächste Beispiel zeigt eine wichtige Technik zur Konvergenzbeschleunigung, die bei einem rechtzeitigen Abbruch die Auslöschung umgehen (jedoch nicht gänzlich verhindern) kann.

Beispiel 1.1.20. (Konvergenzbeschleunigung nach Richardson / Bogolyubov-Krylov) Eine Möglichkeit, die Auslöschung zu vermeiden, ist, eine Methode zu verwenden, die so schnell konvergiert, dass wir gar nicht erst in den Bereich kommen, wo die Auslöschung stattfindet. Als Beispiel betrachten wir wieder die Aufgabe der numerischen Approximation der Ableitung einer glatten Funktion f in einem Punkt x . Die ersten Terme im Satz von Taylor um x sind:

$$f(x + h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + \frac{1}{6}f'''(x)h^3 + \frac{1}{24}f^{(4)}(x)h^4 + \dots \quad (1.1.1)$$

Schreiben wir dasselbe auch für $-h$:

$$f(x - h) = f(x) - f'(x)h + \frac{1}{2}f''(x)h^2 - \frac{1}{6}f'''(x)h^3 + \frac{1}{24}f^{(4)}(x)h^4 - \dots \quad (1.1.2)$$

Subtrahieren wir die beiden letzten Gleichungen, heben sich die geraden Potenzen in h auf:

$$f(x + h) - f(x - h) = 2f'(x)h + \frac{1}{3}f'''(x)h^3 + \frac{1}{60}f^{(5)}(x)h^5 + \dots$$

Daraus folgt

$$f'(x) = \frac{f(x + h) - f(x - h)}{2h} - \frac{1}{6}f'''(x)h^2 - \frac{1}{120}f^{(5)}(x)h^4 - \dots$$

Das weist auf eine mit $\mathcal{O}(h^2)$ konvergierende Approximation von $f'(x)$ über den symmetrischen Differenzenquotienten $d(h) = \frac{f(x+h)-f(x-h)}{2h}$ hin. Leider leidet auch diese (bessere!) Methode unter der Auslöschung. Die wichtige Bemerkung ist nun, dass der Ausdruck des Fehlers

$$f'(x) - \frac{f(x + h) - f(x - h)}{2h} = - \sum_{k=1}^{\infty} \frac{f^{(2k+1)}(x)}{(2k+1)!} h^{2k}$$

eine Summe von **nur geraden Potenzen** von h ist:

$$f'(x) - d(h) = c_1 h^2 + c_2 h^4 + \dots + c_n h^{2n} + \mathcal{O}(h^{2n+2}), \quad (1.1.3)$$

wobei wir mit $d(h)$ den symmetrischen Differenzenquotienten und mit $c_k = -\frac{f^{(2k+1)}(x)}{(2k+1)!}$ die Koeffizienten der Potenzen in h bezeichnet haben. Die entscheidende Idee ist

nun, dasselbe für einen anderen Wert von h zu schreiben. Zum Beispiel können wir h halbieren:

$$f'(x) - d(h/2) = c_1 \frac{1}{2^2} h^2 + c_2 \frac{1}{2^4} h^4 + \dots + c_n \frac{1}{2^{2n}} h^{2n} + \mathcal{O}(h^{2n+2}), \quad (1.1.4)$$

Wir bilden nun die Linearkombination der letzten beiden Gleichungen, die den Term in h^2 eliminiert:

$$f'(x) - \frac{4d(h/2) - d(h)}{3} = \tilde{c}_2 h^4 + \tilde{c}_3 h^6 + \dots + \tilde{c}_n h^{2n} + \mathcal{O}(h^{2n+2}), \quad (1.1.5)$$

wobei $\tilde{c}_2, \dots$ einfach die neuen Koeffizienten der Potenzen von h bezeichnen. Diese neue Formel

$$d_1(h) = \frac{4d(h/2) - d(h)}{3}$$

liefert also eine Approximation an $f'(x)$, die wie h^4 konvergiert.

Wir können den Trick nun wiederholen!

Schreiben wir die Gleichung 1.1.5 nun für h und für $h/2$:

$$\begin{aligned} f'(x) - d_1(h) &= \tilde{c}_2 h^4 + \tilde{c}_3 h^6 + \dots + \tilde{c}_n h^{2n} + \mathcal{O}(h^{2n+2}) \\ f'(x) - d_1(h/2) &= \tilde{c}_2 \frac{1}{2^4} h^4 + \tilde{c}_3 \frac{1}{2^6} h^6 + \dots + \tilde{c}_n \frac{1}{2^{2n}} h^{2n} + \mathcal{O}(h^{2n+2}). \end{aligned}$$

Die Linearkombination der letzten beiden Gleichungen, die den Term in h^4 eliminiert, ergibt

$$f'(x) - \frac{16d_1(h/2) - d_1(h)}{15} = \tilde{c}'_3 h^6 + \dots + \tilde{c}'_n h^{2n} + \mathcal{O}(h^{2n+2}), \quad (1.1.6)$$

wobei $\tilde{c}'_3, \dots$ einfach die neuen Koeffizienten der Potenzen von h bezeichnen. Der Fehler für die neue Formel

$$d_2(h) = \frac{16d_1(h/2) - d_1(h)}{15}$$

konvergiert wie h^6 .

Dieses Vorgehen kann man beliebig weiterverfolgen, solange die Glattheit von f es erlaubt. Bezeichnen wir den symmetrischen Differenzenquotienten mit

$$d_{\ell,0} = d(h/2^\ell), \text{ für die Level } \ell = 0, 1, 2, \dots,$$

Die Approximationsformel nach einer Linearkombination dieser ist:

$$R_{\ell,1} = \frac{2^2 d(\frac{1}{2}h) - d(h)}{2^2 - 1} = \frac{4^1 R_{\ell,0} - R_{\ell-1,0}}{4^1 - 1},$$

die Approximationsformel nach einer Linearkombination der Letzteren ist:

$$R_{\ell,2} = \frac{2^4 d_1(\frac{1}{2}h) - d_1(h)}{2^4 - 1} = \frac{4^2 R_{\ell,1} - R_{\ell-1,1}}{4^2 - 1},$$

und so weiter. Allgemein haben wir also die Definition für $k = 1, 2, \dots, \ell$:

$$R_{\ell,k} = \frac{4^k R_{\ell,k-1} - R_{\ell-1,k-1}}{4^k - 1}, \quad (1.1.7)$$

was dem Richardson-Schema

$$\begin{array}{c|cccc}
 \ell & k=0 & k=1 & k=2 & k=3 \\
 \hline
 0 & d(h) = R_{0,0} & & & \\
 1 & d(h/2) = R_{1,0} \begin{array}{c} \nearrow \\ \searrow \end{array} & R_{1,1} \begin{array}{c} \nearrow \\ \searrow \end{array} & & \\
 2 & d(h/2^2) = R_{2,0} \begin{array}{c} \nearrow \\ \searrow \end{array} & R_{2,1} \begin{array}{c} \nearrow \\ \searrow \end{array} & R_{2,2} \begin{array}{c} \nearrow \\ \searrow \end{array} & \\
 3 & d(h/2^3) = R_{3,0} \begin{array}{c} \nearrow \\ \searrow \end{array} & R_{3,1} \begin{array}{c} \nearrow \\ \searrow \end{array} & R_{3,2} \begin{array}{c} \nearrow \\ \searrow \end{array} & R_{3,3} \\
 \vdots & \vdots & \vdots & \vdots & \vdots
 \end{array} \quad (1.1.8)$$

entspricht.

Die erste Spalte beinhaltet symmetrische Differenzenquotienten für verschiedene h . Die anderen Elemente ergeben sich durch die Linearkombinationen, welche die Konvergenzordnung erhöhen. Auf der Diagonalen entstehen also die Ergebnisse des Richardson-Schemas mit den verschiedenen verwendeten h , also immer bessere Approximationen. Der Fehler nimmt mit k und mit ℓ sehr schnell ab:

$$f'(x) - R_{\ell,k} = C \cdot (h/2^\ell)^{2k+2}.$$

Sollte das Ergebnis ganz rechts unten nicht genügen, kann man das kleinste h noch einmal verkleinern und lediglich eine weitere Zeile in der obigen Tabelle berechnen und unten anfügen. Mit einer Fehlerschätzung können wir das Richardson-Schema abbrechen, wenn der Fehler als klein genug eingeschätzt wird.

Code 1.1.21: Beschleunigung des symmetrischen Differenzenquotienten nach Richardson

```

1  import numpy as np

3  # Richardson extrapolation; fixed level for vectorisation
def diffRichardsonV(f,x, h0, rtol=1e-12, atol=1e-12):
5      nit = 30 # max depth of iterations
      # first column at once
7      h = h0/2**np.arange(nit)
      fp, fm = f(x+h), f(x-h)
9      y = (fp-fm)/2/h
      # go column by column in Richardson
11     # we prefer to store it in the last part of the vector
      for j in range(1,nit):
13         fact = 4**j
         y[j:]=(fact*y[j:] - y[j-1:-1])/(fact-1)
15         errest = abs(y[j]-y[j-1])
         print(j, h[j], errest)
17         if errest < rtol*abs(y[j]) or errest < atol:
             break
19     #return y[:j], h[:j]
     return y[:j+1], h[:j+1] # return the last computed io to show
cancellation

```

Diese

Methode zur Konvergenzbeschleunigung funktioniert immer, wenn sich der Fehler wie hier als Summe gerader Potenzen von h schreiben lässt. Bei der Quadratur heisst dies Romberg-Schema; der verwendete Algorithmus heisst Aitken-Neville-Algorithmus und die Idee ist als Richardson-Extrapolation bekannt, obwohl Richardson selbst Bogolyubov und N. M. Krylov zitiert. Selbstverständlich sind auch andere Sequenzen als die hier verwendeten $h, h/2, h/2^2, h/2^3, \dots$ möglich und unter Umständen (Bulirsch-Sequenz) nützlich.

Es ist jedoch wichtig zu betonen, dass die Konvergenzbeschleunigung das Problem der Auslöschung nicht behebt (im Unterschied zur Technik mit komplexen Zahlen). Wenn wir den Algorithmus nicht vorzeitig via Fehlerschätzung abbrechen, sehen wir bald auch die Auslöschung. In der Abbildung 1.1.22 sehen wir in 3 Fällen, wie die Auslöschung im letzten Level des Richardson-Schemas passiert.

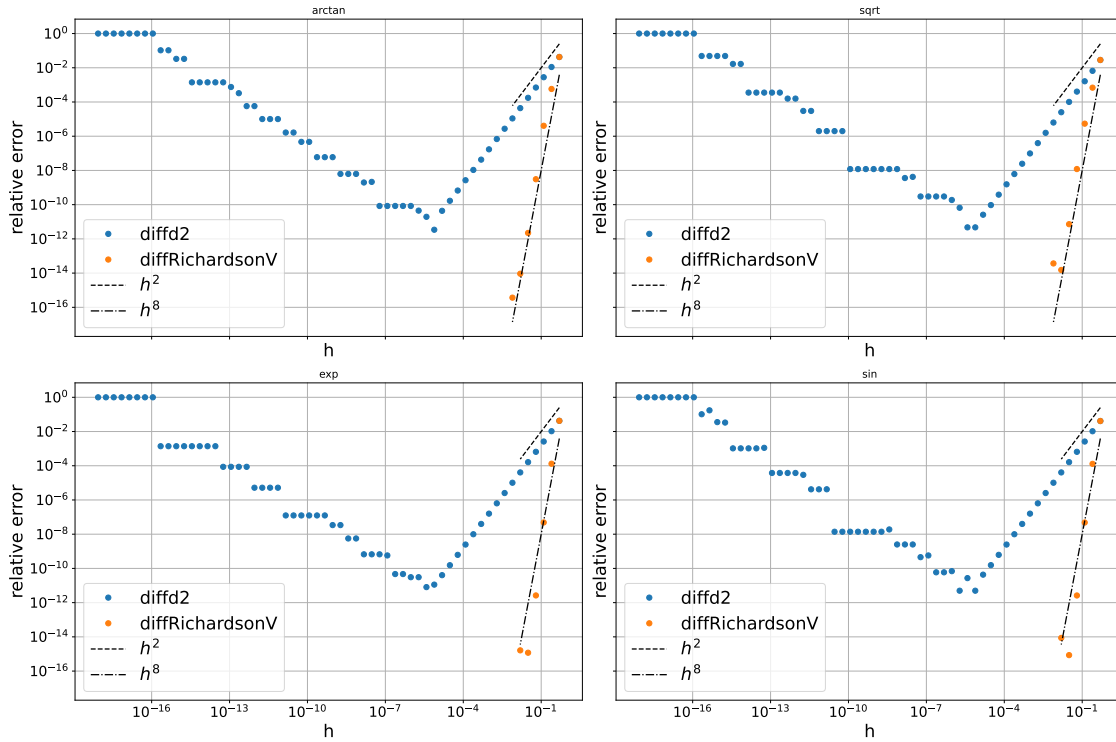


Abb. 1.1.22. Relativer Fehler bei der Berechnung der Ableitung in $x = 1.1$ einiger glatter Funktionen mit dem symmetrischen Differenzenquotienten und mit dem Richardson-Schema.

1.2 Rechenaufwand

Definition 1.2.1. Der Rechenaufwand (computational effort / computational cost) eines numerischen Algorithmus ist die Anzahl elementarer Operationen (Additionen, Subtraktionen, Multiplikationen, Divisionen). Manchmal werden auch Aufrufe elementarer Funktionen wie $\sin$, $\cos$, $\sqrt{\cdot}$, $\exp$, $\ln$ etc. dazu mitgezählt.

70 Jahre zuvor war die Anzahl elementarer Operationen ein guter Indikator für die zu erwartende Laufzeit eines Programms; heute jedoch nicht mehr, ausser auf eine eher ungenaue, asymptotische Weise. Heute ist die Bandbreite des Arbeitsspeichers und somit die Art und Weise, wie die Daten für die Berechnungen gelesen und geschrieben werden, entscheidend für die Laufzeit eines Programms. Moderne Rechenarchitekturen verwenden verschiedene Cache-Level: Daten, die sich in niedrigeren Leveln befinden, werden deutlich schneller und womöglich gleichzeitig (parallel) bearbeitet als Daten, die sich in oberen Cache-Leveln befinden. Bestimmte Operationen, wie z. B. implementierte grundlegende lineare Algebra-Operationen (BLAS = Basic Linear Algebra Subprograms), wurden bereits so weit optimiert, dass alle zur Verfügung stehenden Cores eines Prozessors die Daten im Cache parallel bearbeiten.

Definition 1.2.2. Die asymptotische (Rechen-)Komplexität eines Algorithmus charakterisiert die ungünstigste (Worst-Case-)Abhängigkeit seines Rechenaufwands von einem oder mehreren Problemgrößenparametern, wenn diese gegen ∞ wachsen.

„Ungünstigst“ bedeutet, dass der maximale Aufwand für einen Satz zulässiger Daten berücksichtigt wird. Ein Beispiel für einen Problemgrößenparameter ist die Länge $n \rightarrow \infty$ der Input-/Output-Vektoren in einem Problem der linearen Algebra.

Definition 1.2.3. Wir schreiben $F(n) = \mathcal{O}(G(n))$ für zwei Funktionen $F, G : \mathbb{N} \rightarrow \mathbb{R}$, falls es eine Konstante $C > 0$ und ein $n_* \in \mathbb{N}$ gibt, sodass

$$F(n) \leq C G(n) \quad \text{für alle } n \geq n_* .$$

Anschaulich: $F(n) = \mathcal{O}(G(n))$ bedeutet, F wächst höchstens genauso schnell wie G . Im Kontext der asymptotischen Komplexität versteht sich die Landau-Notation oft auch „scharf“ in dem Sinne, dass es keine Funktion gibt, die langsamer wächst als G , die in $\mathcal{O}(\cdot)$ verwendet werden kann. D. h.: $F(n) = \mathcal{O}(G(n))$ bedeutet in unserem Kontext immer etwas präziser: F wächst genauso schnell wie G . Zum Beispiel: Die Gauss-Elimination benötigt für allgemeine $n \times n$ -Matrizen $\mathcal{O}(n^3)$ elementare Operationen; für obere Dreiecksmatrizen benötigt sie $\mathcal{O}(n^2)$ Operationen und für Diagonalmatrizen $\mathcal{O}(n)$ Operationen. Die angegebenen Komplexitäten beziehen sich jeweils auf das Worst-Case-Szenario in der Klasse der betrachteten Matrizen und können jeweils nicht durch langsamer wachsende Funktionen in n ersetzt werden.

Die praktische Bedeutung der asymptotischen Komplexität ist eine (unpräzise) Vorhersage für die Laufzeit einer Implementierung des Algorithmus. Zum Beispiel könnte ein Algorithmus mit der Komplexität $\mathcal{O}(n^2)$ die vierfache Zeit benötigen, wenn sich die Problemgrösse verdoppelt, vorausgesetzt n ist so gross, dass die Cache-Level und sonstige Einflüsse der Hardware vernachlässigbar sind.

Die $\mathcal{O}(h)$ -Notation wird manchmal auch für das asymptotische Verhalten $h \rightarrow 0$ „missbraucht“. In den vorigen Beispielen haben wir gesehen, dass sich der Fehler in einem Fall theoretisch (bis die Auslöschung dominiert) wie $C \cdot h$ und im anderen Fall wie $C \cdot h^2$ verhielt. Korrekterweise müsste man $o(h)$ und $o(h^2)$ schreiben, um zu betonen, dass $h \rightarrow 0$ und nicht $h \rightarrow \infty$ betrachtet wird.

Damit wir die aktuelle Hardware optimal verwenden können, müssen wir die theoretisch optimalen Algorithmen mit Bedacht implementieren. BLAS ist eine Sammlung von Spezifikationen, die aus den 1970er-Jahren stammt, daher ist sie (immer noch) in FORTRAN77 implementiert. Wrapper für andere Programmiersprachen existieren, sowie professionelle Implementierungen: OpenBLAS, ATLAS (Automatically Tuned Linear Algebra Software – Standard in Linux), Intel MKL (Math Kernel Library).

In dieser Vorlesung verwenden wir Python, das für rasche, flexible und sichere Implementierungen konzipiert worden ist. Es lohnt sich, Einiges im Internet über die Entstehung und Prinzipien von Python zu lesen; einige dazugehörige Sprüche sind legendär (z. B.: “batteries included”, “we are all adults here”). Um diese Ziele zu erreichen, musste man gewisse Kompromisse eingehen, die sich negativ auf die Laufzeit von Python-Programmen auswirken (z. B. kostspielige Kopien, Ausschluss von parallelen Threads). Jedoch können langsame Teile von schnell entwickelten und sicher getesteten Python-Programmen in eine schnell ausführbare Sprache migriert und im Python-Programm leicht eingebunden werden. Die Verwendung von spezialisierten Bibliotheken wie numpy nimmt dem Programmierer sogar diesen Aufwand

ab. Die Python-Bibliothek `numpy` stellt die Verbindung zu optimierten BLAS-Implementierungen und zu optimierten FORTRAN-, C- oder C++-Implementierungen wichtiger numerischer Algorithmen her, sodass die Computer-Hardware aus Python heraus optimal verwendet wird. Wenn wir also `numpy` geschickt verwenden, können wir die bestmögliche Rechenzeit erreichen, ohne dass wir uns mühsam mit komplizierten FORTRAN-, C- oder C++-Aufrufen und Verlinkungen beschäftigen müssen.

Die Python-Bibliothek `scipy` baut auf `numpy` auf, bietet zum Teil andere, flexiblere Funktionssignaturen oder gar andere Implementierungen für manche Algorithmen (manche nur in Python mit `numpy` implementiert) und beinhaltet viele Sammlungen von Bibliotheken, die auf bestimmte Bereiche spezialisiert sind, z. B. `integrate`, `interpolate`, `optimisation`, `linalg`, `special`, `stats`, `fft`, `sparse`, `signal`, `io`. Manche Funktionen bauen auf Aufrufen von Funktionen auf, die in FORTRAN, C oder C++ implementiert, kompiliert und verlinkt sind; manche sind `numpy`-basierte Python-Implementierungen. Wir verwenden auch die Bibliothek `sympy`, die (noch bescheidene, aber gute) symbolische Berechnungen ermöglicht, und die sehr weitverbreitete und extrem flexible Bibliothek `matplotlib`, um leicht die nötigen (meist 2D-) Plots zu erzeugen. Der wichtigste Datentyp von `numpy` ist `ndarray`. Bitte lesen Sie die Tutorials und Dokumentationen dazu.

Wichtig, um die optimale Laufzeit zu erreichen, ist: die Daten soweit möglich auf einmal in Arrays (ohne Schleifen, d. h. vektorisiert) zu verwenden, wo möglich Kopien zu vermeiden (z. B. indem man `slices` und `views` verwendet) und genügend Speicher rechtzeitig zu belegen. Wichtig, um schwer auffindbare Bugs zu vermeiden, ist: **Speicher eines `ndarray` immer mit dem richtigen Datentyp initialisieren**. Das scheint zum Teil gegen einige Grundprinzipien von Python und gegen den Wunsch nach Flexibilität zu verstossen, ist jedoch nötig: Mit der Verwendung von `numpy` haben wir Laufgeschwindigkeit gegen ein bisschen Sicherheit eingetauscht. Wenn wir eine komplexe Zahl in ein Element eines `arrays` schreiben, das (automatisch) mit `floats` initialisiert worden ist, werden wir den vielleicht für den Algorithmus wichtigen Imaginärteil stillschweigend verlieren: viel Spass bei der zeitraubenden Bugsuche! Unvorhersehbare und oft nicht reproduzierbare Abstürze können beim Schreiben auf deklarierten, jedoch nicht initialisierten Speicher passieren - die C- und C++-Programmierer kennen das. Im Umgang mit `ndarrays` ist es auch wichtig, eine echte Kopie zu erzwingen, wenn man eine braucht, denn aufgrund von Effizienz machen `ndarrays` oft gar keine (echte) Kopie. Diese Mehrarbeit kann auch die Suche nach manchem üblen Bug ersparen, wenn ein `ndarray`, das nicht verändert werden soll, plötzlich wegen eines `slices` oder einer *shallow copy* neue Werte enthält. Probieren Sie:

Code 1.2.4: View ist schnell, aber Achtung bei Änderungen!

```

1  import numpy as np
   n = 4
3  A = np.diag(np.mgrid[1:n+1])
   A[:, -1] = A[-1, :] = np.mgrid[1:n+1]
5  print('A=\n', A)
   B = A[::-1, :] # B is not a copy, but another view on A
7  print('B=\n', B) # note: rows of A in reversed order
   A[2,2] = -2
9  print('A=\n', A) # changed an element on the diagonal of A
   print('B=\n', B) # the corresponding element "of B" changed!
11 B[2,2] = -10
   print('B=\n', B) # changed an element on the diagonal of B
13 print('A=\n', A) # the corresponding element "of A" changed!
   print('B.base=\n', B.base)

```

Noch eine grosse Warnung: Verwenden Sie **niemals** `numpy.vectorize`! Ironischerweise vektorisiert diese Funktion nicht so, wie man es sich wünscht, sondern schreibt eine in der Ausführung sehr langsame Schleife um den Code. Das steht zwar als Erstes in der Anmerkung der Dokumentation, wird aber sehr leicht und sehr gerne ignoriert, denn die Aufgabe, einen Code vektorisiert zu schreiben, ist zwar sehr wichtig, aber manchmal auch sehr schwierig...

Ein Beispiel für die immer wichtigere, echte **Vektorisierung** und gewisse Kompromisse für die Sicherheit sehen wir im vorigen Code 1.1.21: Wir setzen eine harte obere Schranke für die Iteration (`nit=30`) und verwenden im Folgenden `ndarrays` dieser Dimension, auch wenn die Iteration de facto viel früher (bei circa 7) abbricht. Ein `ndarray` namens `h` wird auf einmal erstellt, und die Funktionswerte für alle diese einzelnen `h` werden auf einmal (vektoriert) in den Aufrufen `fp = f(x+h)` und `fm = f(x-h)` erzeugt. `y` wird auch vektorisiert erzeugt (`y = (fp-fm)/2/h`) und via Slices (also keine Kopien!) `y[j:]` in jedem Schritt der unvermeidbaren `j`-Schleife verändert. Der Preis für diese sehr schnell laufende Implementierung ist: Ein grosser Teil des bereitgestellten Speichers wird in unseren Beispielen gar nicht verwendet; wenn in einem Fall mehr als `nit=30` nötig wären, wäre das nicht direkt möglich, sondern man müsste `nit` von Hand erhöhen und das Programm nochmals laufen lassen. Das Erweitern eines `ndarrays` ist sehr teuer und man sollte es, wenn möglich, vermeiden, genauso wie explizite Schleifen (z. B. `for`) über die Elemente eines `ndarrays`.

Schreiben Sie als Übung den Code 1.1.12 in eine vektorisierte Version (ohne Schleifen) um. Warum können wir nicht ohne Weiteres dasselbe mit dem Code 1.1.17 tun? Schlagen Sie trotzdem eine Umschreibung des Codes 1.1.17 in vektorisierter Form vor und besprechen Sie die Vor- und Nachteile Ihrer Implementierung.

Hier sind einige Übungen und Beispiele für die Indizierung, `slices` und `views`:

Code 1.2.5: Übungen mit Indizierungen

```

1  """https://numpy.org/doc/stable/user/basics.indexing.html#advanced-indexing"""
x = np.arange(10) # 0,1,2,...,9
3  print(x[0], x[-1], x[-2]) # 0, 9, 8
print(x.reshape(2,5)) # makes a copy
5  print(x) # no change
x.shape = (2,5) # modify the attribute shape of this x
7  print(x) # see: now x is 2-dimensional
print(x[0]) # 0 refers now to the first row in x
9  # x[0] is a view, not a copy:
x[0][2] = -2; print(x)
11 x[0,2] = 2; print(x) # more efficient than x[0][2]
# be caution with the data type!
13 x[0,2] = 2.5; print(x) # 2.5 was cast to the data type of x (int)
# basic slicing returns a view
15 y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
print(y[1:7:2]) # ([1, 3, 5]
17 print(y[-2:10], y[-2:]) # [8, 9]
print(y[-3:3:-1]) # [7 6 5 4]
19 print(y[2], y[2:3]) # watch the difference!
print(x)
21 print(x[1:10:5, ::-1]) # is the same as the more general:
obj = (slice(1, 10, 5), slice(None, None, -1)); print(x[obj])
23 x = np.array([[1],[2],[3]], [[4],[5],[6]])
print(x.shape); print(x[1:2])
25 print(x[..., 0]) # is the same as
print(x[:, :, 0])
27 print(x[:, np.newaxis, :, :].shape) # is synonym to:
print(x[:, None, :, :].shape)
29 # advanced indexing returns a copy
x = np.arange(10, 1, -1); print(x)
31 print(x[np.array([3, 3, 1, 8])]) # [7, 7, 9, 2]
y = np.arange(35).reshape(5, 7); print(y)
33 print(y[np.array([0, 2, 4]), np.array([0, 1, 2])])
print(y[np.array([0, 2, 4]), 1] ) # elements [0,1], [2,1], [4,1]
35 print(y[np.array([0, 2, 4])]) # copies of the rows 0,2,4 of y
# select corners of a 4x3 array:
37 x = np.arange(12).reshape(4,3); print(x)
rows = np.array([[0,0], [3,3]])
39 cols = np.array([[0,2], [0,2]])
print( x[rows, cols] )
41 # shorter via broadcasting:
rows = np.array([0, 3])
43 cols = np.array([0, 2])
print( rows[:, np.newaxis] )
45 print( x[rows[:, np.newaxis], cols] )
# even shorter:
47 print( x[np.ix_(rows, cols)] )
print( np.ix_(rows, cols) )
49 # note the difference:
print( x[rows, cols] )

```

Code 1.2.6: Weitere Übungen mit Indizierungen

```

# boolean array indexing
2 x = np.array([[1., 2.], [np.nan, 3.], [np.nan, np.nan]])
  print( x[~np.isnan(x)] ) # [1.,2.,3.]
4 x = np.array([1., -1., -2., 3])
  x[x < 0] += 20
6 print(x) # [ 1., 19., 18., 3.]
  x = np.arange(35).reshape(5, 7)
8 b = x > 20
  print( b[:, 5] ) # [False, False, False, True, True]
10 print( x[b[:, 5]] )
  x = np.array([[0, 1], [1, 1], [2, 2]])
12 rowsum = x.sum(-1)
  print( x[rowsum <= 2, :] )
14 # combining advanced and basic indexing creates concatenations
  y = np.arange(35).reshape(5,7)
16 print( y[np.array([0, 2, 4]), 1:3] )
  # assigning
18 x = np.arange(10)
  x[2:5] = 1; print(x)
20 x[2:5] = np.arange(3); print(x)
  # No surprise, because adv. indexing makes a copy, but confusing!
22 x = np.arange(0, 50, 10); print(x)
  x[np.array([1, 1, 3, 1])] += 1; print(x)
24 # variable number of indices
  z = np.arange(81).reshape(3, 3, 3, 3)
26 indices = (1, 1, 1, 1); print( z[indices] ) # 40
  indices = (1, 1, 1, slice(0, 2)) # same as [1, 1, 1, 0:2]
28 print( z[indices] ) # [39, 40]
  indices = (1, Ellipsis, 1) # same as [1, ..., 1]
30 print( z[indices] )

```

In dieser Vorlesung versuchen wir, ein gutes Beispiel dafür zu geben, wie man vektorisiert, wo es geht, ohne die Lesbarkeit und vor allem die Korrektheit aufs Spiel zu setzen. “Versuchen” ist das richtige Wort: Die Codes wurden im Laufe vieler Jahre und von verschiedenen Personen geschrieben, sodass manche Inkonsistenzen und Veralterungen unumgänglich sind; daher bitte ich schon jetzt um Verzeihung.

1.3 Rechnen mit Matrizen

Aus mathematischem Gesichtspunkt betrachten wir Vektoren immer als Spalten, also ist $\mathbf{x} \in \mathbb{C}^m$ immer eine $m \times 1$ -Matrix; wenn wir den Vektor als Zeile haben wollen, schreiben wir $\mathbf{x}^T$; wenn wir ihn als Zeile und komplex konjugiert brauchen, dann notieren wir $\mathbf{x}^H$. Wir kennen aus der linearen Algebra das Skalarprodukt zweier Vektoren $\mathbf{b}, \mathbf{a}$ aus $\mathbb{C}^m$:

$$\langle \mathbf{b}, \mathbf{a} \rangle = \bar{\mathbf{b}} \cdot \mathbf{a} = \mathbf{b}^H \mathbf{a} = \sum_{k=1}^m \bar{b}_k a_k \in \mathbb{C}$$

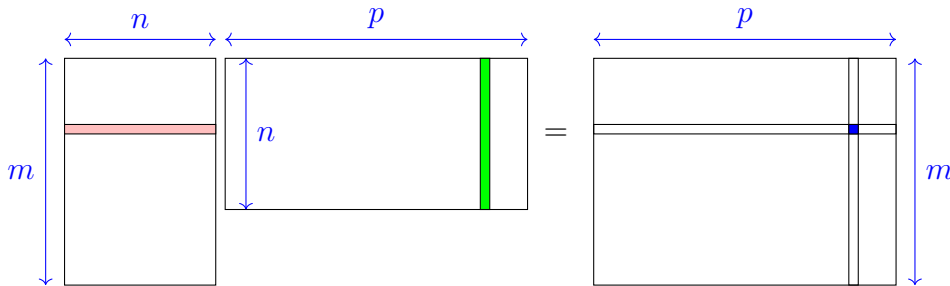
und das Tensorprodukt (dyadisches Produkt, manchmal irreführend äusseres Produkt genannt) der Vektoren $\mathbf{b} \in \mathbb{C}^m$ und $\mathbf{a} \in \mathbb{C}^n$:

$$\mathbf{b}\mathbf{a}^H = [b_i \overline{a_j}] \in \mathbb{C}^{m,n}$$

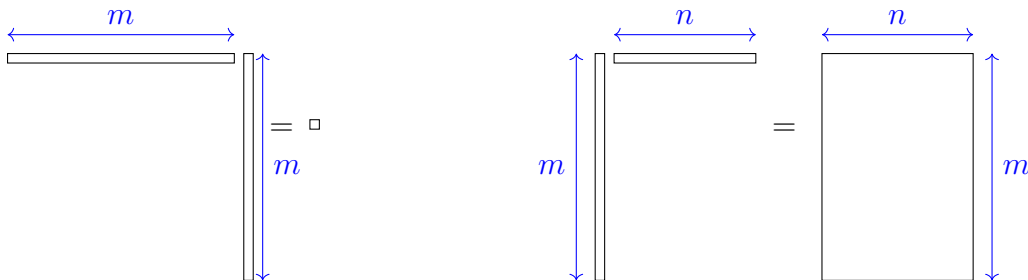
als spezielle Fälle des Produkts von Matrizen $\mathbf{A} \in \mathbb{C}^{m,n}$ und $\mathbf{B} \in \mathbb{C}^{n,p}$:

$$\mathbf{AB} = \left[\sum_{j=1}^n a_{ij} b_{jk} \right] \in \mathbb{C}^{m,p}.$$

Man kann diese Multiplikation auch so visualisieren:



und auch die zwei speziellen Fälle: Der erste Fall entspricht dem (euklidischen) Skalarprodukt, wenn das Ergebnis ein Skalar ist, der zweite Fall entspricht dem Tensorprodukt (dyadischen Produkt) zweier Vektoren, wenn das Ergebnis eine Matrix vom Rang 1 ist:



Da in `numpy` die 1-dimensionalen `arrays` immer Zeilen sind, muss man in den Implementierungen aufpassen:

Code 1.3.1: Verschiedene Arten, das Skalarprodukt zu berechnen

```

a = np.array([1., 2., 3.]) # 1. ensures data type is float
2 b = np.array([4.j, 5.j, 6.j]) # 1j = complex imaginary unit
print( a*b ) # use broadcasting: array as result, not a scalar product
4 print( (a*b).sum(), np.sum(a*b) ) # object functions should be faster
print( b @ a ) # no conjugation
6 print( np.dot(b,a) ) # no conjugation, better style
print( np.vdot(b,a) ) # conjugationn of first argument
8 print( np.dot( b.conj(), a ) ) # manual conjugation; b.conj() makes no copy
print( (b.conj()*a).sum(), np.sum(np.conjugate(b)*a) ) # last makes a copy
10 print( np.tensordot(b, a, axes = 1) ) # same as dot
print( np.tensordot(b.conj(), a, axes = 1) ) # same as vdot
12 # look up np.einsum
print( np.einsum('i,i',b,a) ) # np.dot(b,a)
14 print( np.einsum('i,i',b.conj(),a) ) # np.vdot(b,a)

16 # make them 2D ndarrays; caution on type and shape of output
A = a.reshape(1,3); print(A) # view, no copy
18 B = b.reshape(1,3); print(B)
print( A @ b )
20 # print( A @ B ) """mismatch"""
print( B @ A.T ) # real scalar product of b and a
22 print( B.conj() @ A.T ) # complex scalar product of b and a
print(A @ b, A @b.conj())

```

Weitere wichtige Funktionen sind `tensordot`, `outer`, `kron` und vor allem das sehr elegante und universelle `einsum` (allgemein bekannt als Einstein-Notation, erfunden eigentlich von Ricci). `einsum` kann viel schneller und RAM-sparsamer als `@`, `sum`, `transpose`, aber langsamer als `dot`, `inner`, `tensordot` (diese verwenden BLAS) sein, braucht aber mehr Zeit zum Erlernen; es gibt Einführungen, die besser als die `numpy`-Dokumentation sind, suchen Sie nach “einstein summation in numpy”:

Code 1.3.2: Tensor-, äusseres, Kronecker-Produkt und Einstein-Summation

```

a = np.array([1., 2., 3.]) # 1. ensures data type is float
2 b = np.array([4.j, 5.j, 6.j]) # 1j = complex imaginary unit
print( a*b ) # use broadcasting: array as result
4 print( np.tensordot(b, a, axes = 1) ) # same as dot

6 print( np.outer(b,a) )
print( np.outer(b.conj(),a) )
8 print( np.tensordot(b, a, axes = -1) ) # tensor product of b and a
print( np.tensordot(b.conj(), a, axes = -1) ) # tensor product
10 # look up np.einsum
print( np.einsum('i,i',b,a) ) # np.dot(b,a)
12 print( np.einsum('i,j',b,a) ) # outer product
print( np.einsum('i,i->i',b,a) ) # b*a
14 # make them 2D ndarrays
A = a.reshape(1,3); print(A)
16 B = b.reshape(1,3); print(B)
print( A @ b )
18 # print( A @ B ) """mismatch"""
print( B @ A.T ) # real scalar product of b and a
20 print( B.conj() @ A.T ) # complex scalar product of b and a
print( B.T @ A ) # tensor product of b and a
22 print( B.T.conj() @ A ) # tensor product of b and a and conjugation
print( np.asmatrix(b).H @ A )
24 print( np.asmatrix(b).H * a ) # broadcasting
print( b[:, np.newaxis] * a ) # broadcasting
26 print( b[:, np.newaxis].conj() * a ) # broadcasting

28 A, B = np.array([[1,2],[3,4]]), np.array([[10,11],[12,13]])
print(A, B)
30 print( np.outer(A,B) )
print( np.kron(A,B) ) # Kroneker product
32 print( np.einsum('ij,kl->ikjl',A,B).reshape(4,4) ) # Kroneker product
print( np.einsum('ik,kj->ij',A,B) ) # A @ B
34 print( np.einsum('ik,kj->ji',A,B) ) # (A @ B).T
print( np.einsum('ii->i',A) ) # diag(A)
36 print( np.einsum('ii->',A) ) # trace(A)
x = np.array([-1., -2.])
38 print( np.einsum('ij,j->i',A,x) ) # A @ x
print( np.einsum('i,ij->j',x,A) ) # x.dot(A)
40 print( np.einsum('s,st,t->',x,A,x) ) # x.dot(A @ x) quadratic form

```

Hier haben wir mit winzigen, klaren Arrays gearbeitet, um die Effekte jedes Befehls sehen zu können. Aufgabe: Bauen Sie sehr grosse (soweit der Arbeitsspeicher reicht) voll besetzte, zufällige (`np.random.rand`) Arrays und probieren Sie jeden Befehl in `ipython` oder `jupyter` mit dem Präfix `timeit`, z.B.:

```
timeit np.sum(np.conjugate(bb)*aa)
```

um die zu erwartenden Unterschiede in der Rechenzeit abzuschätzen. Wenn Sie auch die Lesbarkeit des Codes berücksichtigen, welche der vorigen Rechenarten sind zu bevorzugen? Nutzen Sie konsistent Ihre bevorzugte Variante!

Bemerkung 1.3.3. Das Ergebnis der Multiplikation einer $m \times n$ Matrix $\mathbf{A}$ mit einem Vektor $\mathbf{x}$ ist eine Linearkombination der Spalten von $\mathbf{A}$:

$$\mathbf{A} \mathbf{x} = \sum_{j=1}^n x_j \mathbf{A}_{:,j}.$$

Bemerkung 1.3.4. Der Rang des Ergebnisses der Matrixmultiplikation ist kleiner oder gleich dem Minimum der Ränge der Faktoren:

$$\text{Rank}(\mathbf{A}\mathbf{X}) \leq \min(\text{Rank}\mathbf{A}, \text{Rank}\mathbf{X}).$$

In der Tat ist jede Spalte im Ergebnis $\mathbf{B}$ der Matrixmultiplikation $\mathbf{A}\mathbf{X} = \mathbf{B}$ eine Linearkombination der Spalten von $\mathbf{A}$:

$$\mathbf{A} [\mathbf{X}_{:,1} \ \mathbf{X}_{:,2} \ \cdots \ \mathbf{X}_{:,p}] = [\mathbf{B}_{:,1} \ \mathbf{B}_{:,2} \ \cdots \ \mathbf{B}_{:,p}] \quad (1.3.9)$$

$$\mathbf{A}\mathbf{X}_{:,1} = \mathbf{B}_{:,1}$$

$$\mathbf{A}\mathbf{X}_{:,2} = \mathbf{B}_{:,2}$$

$$\vdots$$

$$\mathbf{A}\mathbf{X}_{:,p} = \mathbf{B}_{:,p}$$

$$\mathbf{A}\mathbf{X} = \mathbf{B}.$$

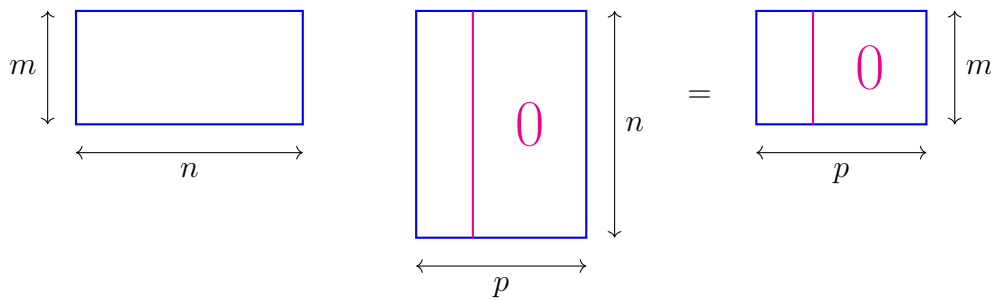
Die Koeffizienten in dieser Linearkombination sind die Einträge in den entsprechenden Spalten von $\mathbf{X}$, z.B. $\mathbf{B}_{:,1} = \sum_{j=1}^n \mathbf{X}_{j,1} \mathbf{A}_{:,j}$. Somit ist $\text{Rank}\mathbf{B} \leq \text{Rank}\mathbf{A}$. Jede Zeile im Ergebnis $\mathbf{B}$ der Matrixmultiplikation $\mathbf{A}\mathbf{X} = \mathbf{B}$ ist eine Linearkombination der Zeilen von $\mathbf{A}$:

$$\begin{bmatrix} \mathbf{A}_{1,:} \\ \vdots \\ \mathbf{A}_{m,:} \end{bmatrix} \mathbf{X} = \begin{bmatrix} \mathbf{B}_{1,:} \\ \vdots \\ \mathbf{B}_{m,:} \end{bmatrix}. \quad (1.3.10)$$

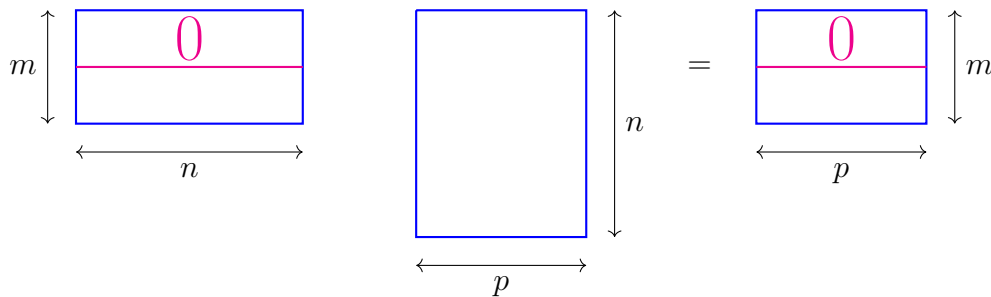
Die Koeffizienten in dieser Linearkombination sind die Einträge in den entsprechenden Zeilen von $\mathbf{A}$, z.B. bedeutet $\mathbf{B}_{1,:} = \mathbf{A}_{1,:}\mathbf{X}$, dass $\mathbf{B}_{1,:} = \sum_{k=1}^n \mathbf{A}_{1,k} \mathbf{X}_{k,:}$. Somit ist $\text{Rank}\mathbf{B} \leq \text{Rank}\mathbf{X}$.

Damit ist der Rang des Ergebnisses kleiner oder gleich dem Minimum der Ränge von $\mathbf{A}$ und $\mathbf{X}$.

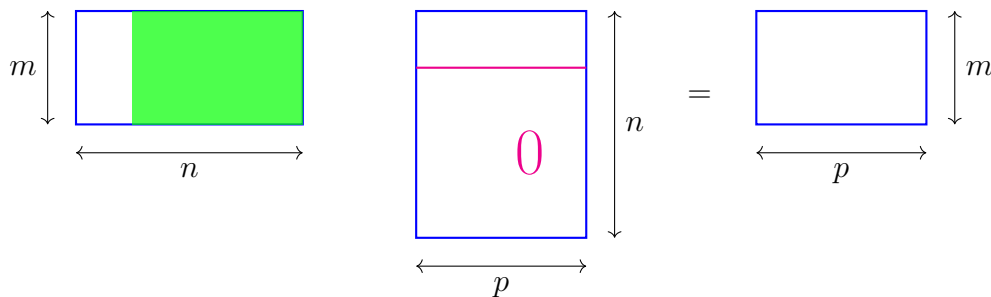
Bemerkung 1.3.5. Die Formel 1.3.9 zeigt das Ergebnis der Matrixmultiplikation als eine Matrix, die aus Spalten $\mathbf{B}_{:,j} = \mathbf{A}\mathbf{X}_{:,j}$ gebaut ist, während die Formel 1.3.10 es als eine Matrix zeigt, die aus Zeilen $\mathbf{B}_{i,:} = \mathbf{A}_{i,:}\mathbf{X}$ gebaut ist. Je nach Struktur der Matrizen $\mathbf{A}$ und $\mathbf{X}$ kann die eine oder die andere Sichtweise vorteilhaft sein, z.B:



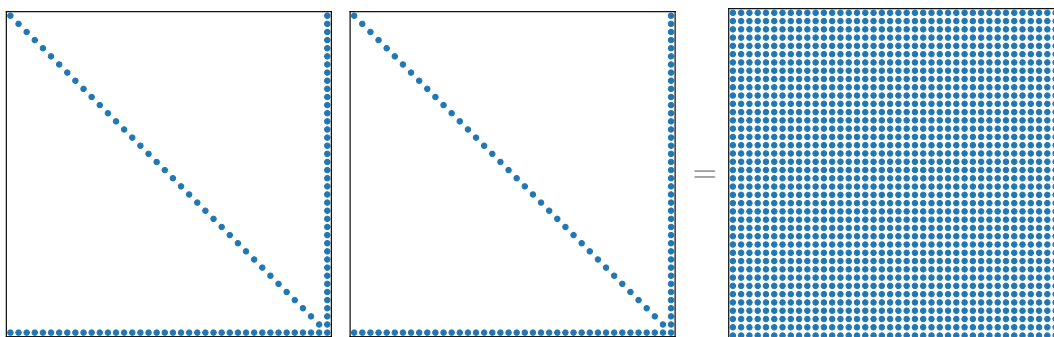
und

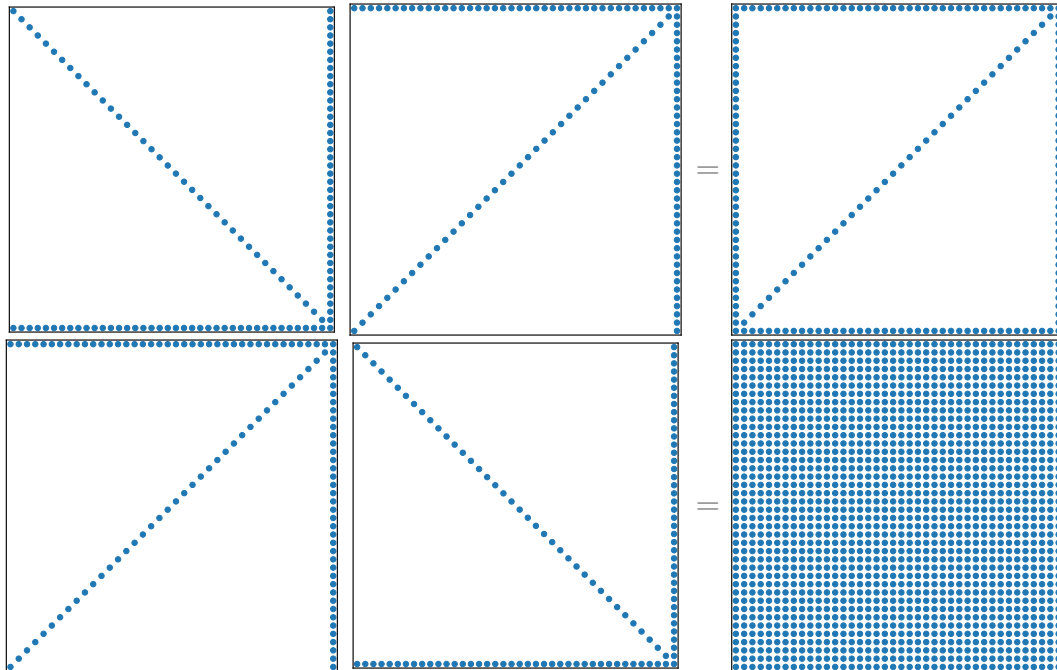


Die Teile, die einen Block aus Nullen multiplizieren, sind für das Ergebnis nicht relevant. Speicher und/oder Arbeit kann man sich dafür sparen:

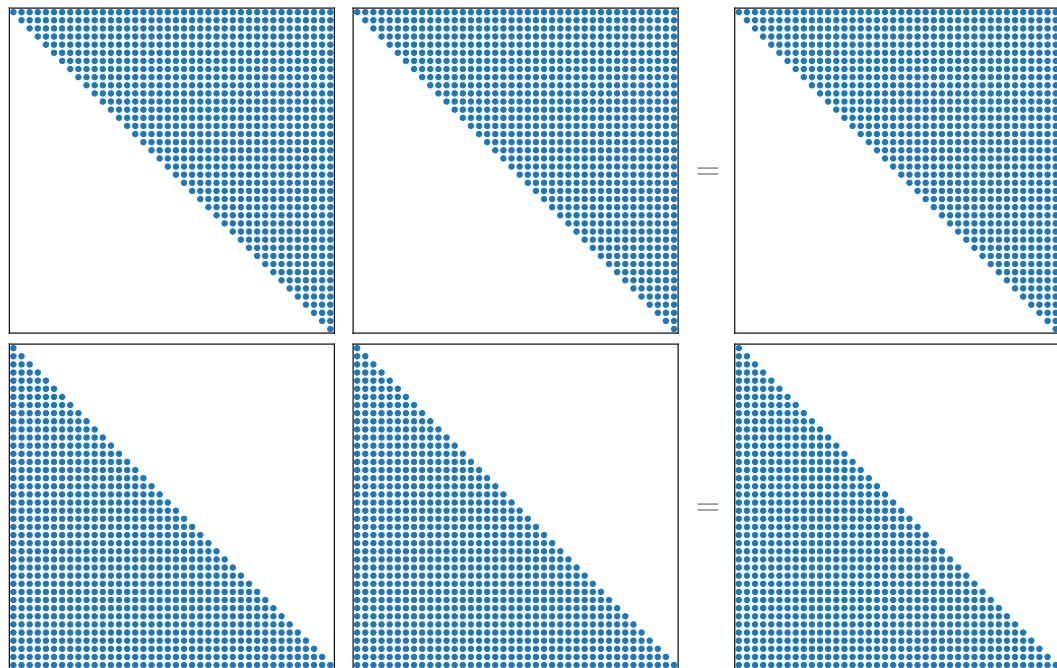


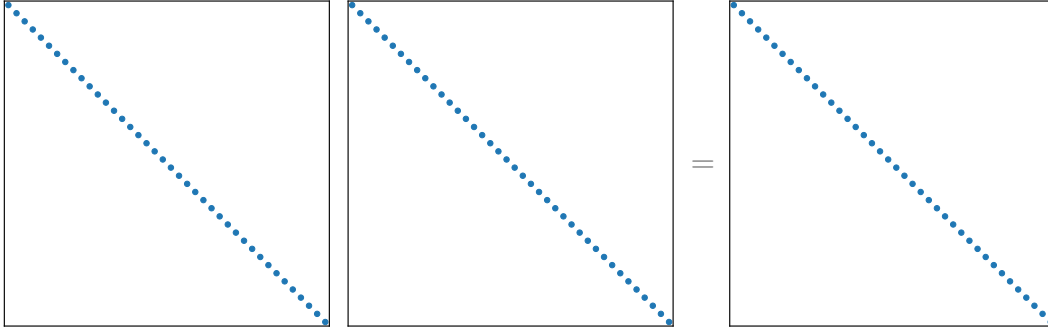
Die Besetzungsmuster und die Rechenregel sind sehr wichtig bei Berechnungen mit Matrizen, die vorwiegend 0-Einträge haben: dünnbesetzte (sparse) Matrizen. Z.B. die Ergebnisse der Multiplikation von Pfeil-Matrizen:





Bemerkung 1.3.6. Die Menge der Diagonalmatrizen, bzw. der unteren Dreiecksmatrizen, bzw. der oberen Dreiecksmatrizen sind jeweils abgeschlossen bezüglich der Matrixmultiplikation:





Vorwärts- bzw. Rückwärtssubstitution zeigt, dass die Inverse von nicht-singulären unteren bzw. oberen Dreiecksmatrizen auch untere bzw. obere Dreiecksmatrizen sind; dasselbe gilt natürlich auch für invertierbare Diagonalmatrizen.

Bemerkung 1.3.7. Es ist auch wichtig, sich den Unterschied im Ergebnis der Multiplikation einer Matrix mit einer Diagonalmatrix von links oder von rechts zu merken. Die Multiplikation mit einer Diagonalmatrix von links skaliert die Zeilen:

$$\begin{bmatrix} d_1 & 0 & 0 \\ 0 & d_2 & 0 \\ & & \ddots \\ 0 & 0 & d_n \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & & a_{2m} \\ \vdots & & & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 a_{11} & d_1 a_{12} & \dots & d_1 a_{1m} \\ d_2 a_{21} & d_2 a_{22} & \dots & d_2 a_{2m} \\ \vdots & & & \vdots \\ d_n a_{n1} & d_n a_{n2} & \dots & d_n a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 \mathbf{A}_{1,:} \\ \vdots \\ d_n \mathbf{A}_{n,:} \end{bmatrix}.$$

Multiplikation mit einer Diagonalmatrix von rechts skaliert die Spalten:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & & a_{2m} \\ \vdots & & & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} \begin{bmatrix} d_1 & 0 & 0 \\ 0 & d_2 & 0 \\ & & \ddots \\ 0 & 0 & d_m \end{bmatrix} = \begin{bmatrix} d_1 a_{11} & d_2 a_{12} & \dots & d_m a_{1m} \\ d_1 a_{21} & d_2 a_{22} & \dots & d_m a_{2m} \\ \vdots & & & \vdots \\ d_1 a_{n1} & d_2 a_{n2} & \dots & d_m a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 \mathbf{A}_{:,1} & \dots & d_m \mathbf{A}_{:,m} \end{bmatrix}.$$

Bemerkung 1.3.8. Die blinde Verwendung von Prozeduren aus `numpy` garantiert nicht die bestmögliche Rechenzeit. Denken vor Rechnen!

Beispiel 1.3.9. Sei $\mathbf{A}$ eine zufällige, voll besetzte $n \times n$ Matrix und $\mathbf{D}$ eine zufällige $n \times n$ Diagonalmatrix. Die direkte Multiplikation $\mathbf{DA}$ mittels des offensichtlichen

$$\mathbf{D} @ \mathbf{A}$$

kostet $\mathcal{O}(n^3)$ Operationen. Wenn wir die vorige Bemerkung zur Multiplikation mit Diagonalmatrizen verwenden, d.h. die Struktur der dünnbesetzten Matrix $\mathbf{D}$ ausnutzen, indem wir nur die Zeilen von $\mathbf{A}$ mit den jeweiligen Einträgen aus der Diagonale von $\mathbf{D}$ skalieren, kostet das nur $\mathcal{O}(n^2)$ Operationen. Dabei müssen wir natürlich darauf aufpassen, dass wir keine unnötigen Kopien erstellen und dass wir Schleifen vermeiden. Eine solche Implementierung ist:

$$\mathbf{D}.\text{diagonal}()[:, \text{np.newaxis}] * \mathbf{A}.$$

Hier nahmen wir an, die grosse Diagonalmatrix sei gegeben, so brauchen wir eine

View (keine Kopie) der Diagonale mit `D.diagonal()` und nutzen `np.newaxis`, um Broadcasting zu bewirken. Die Rechenzeiten auf meinem Laptop im August 2025 sind in Abbildung 1.3.10 dargestellt. Denken vor Rechnen zählt sich aus.

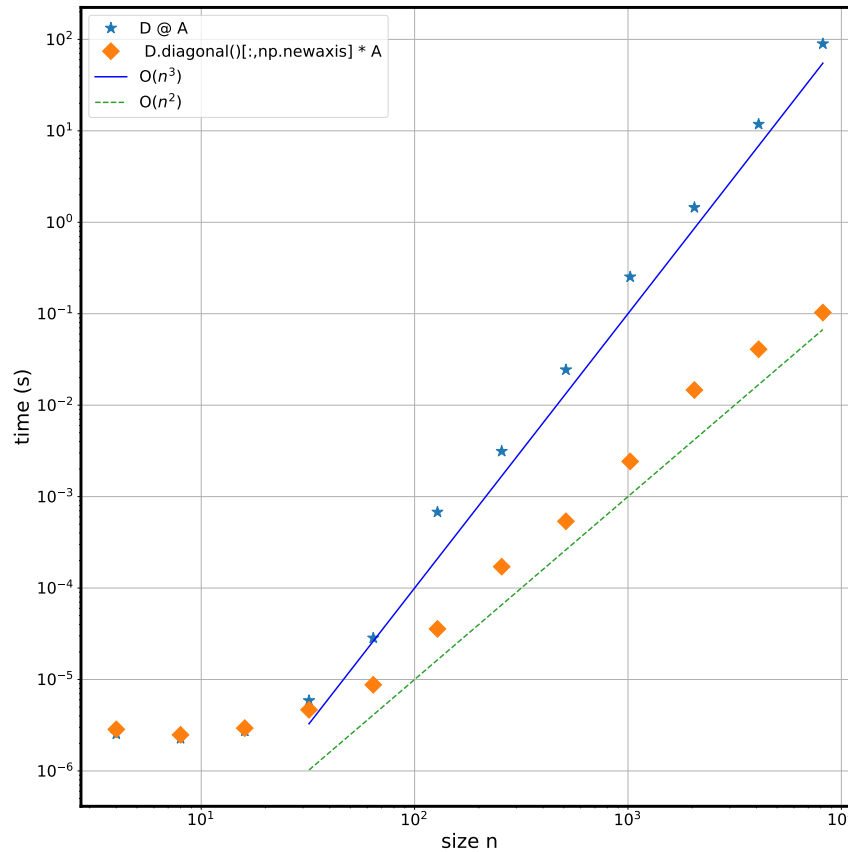


Abb. 1.3.10. Rechenzeiten für die zwei Arten, die Multiplikation mit einer Diagonalmatrix zu berechnen.

Bemerkung 1.3.11. Wenn die Verwendung von `timeit` in `ipython` aus welchen Gründen auch immer nicht möglich ist, kann man die Laufzeiten eines Teils des Codes so messen:

Code 1.3.12: Mögliche Laufzeitmessung für `func`

```

1  import time
3  nrEXP = 10 #number of experiments
   ts = [] # measured times
5  for k in range(nrEXP):
       start = time.process_time()
       result = func() # call the function/part we measure
       end = time.process_time()
9     ts.append(end - start)
   print(f'Min. Time taken: {min(ts):.6f} seconds')
```

Die Aufrufe messen die “wall time”, die stark von allen weiteren gleichzeitigen Prozessen auf dem Rechner beeinflusst wird, also müssen wir viele Messungen machen;

die kleinste gemessene Zeit ist das Beste, was wir mit diesem Experiment aussagen können. Für sehr lange Rechenzeiten kann es sein, dass man sich weniger Experimente leistet, man sollte jedoch so viele wie möglich, mindestens 3, machen, sonst sind die Angaben irrelevant und zu pessimistisch.

Man kann diese Funktion auch in einem `decorator` schreiben:

Code 1.3.13: Möglicher `decorator` für Laufzeitmessung

```

import time
2
def mytimeit(rep=1): # decorator with argument
4     def mytimeit_real_decorator(func): # actual decorator
        def wrapper(*args, **kwargs):
8             ts = [] # measured times
            for k in range(rep):
                start = time.process_time()
                result = func(*args, **kwargs)
                end = time.process_time()
                ts.append(end - start)
12             print(f'Min. Time taken: {min(ts):.6f} seconds in
',func.__name__)
                wrapper.ats.append(min(ts)) # record the minimal measured time
14             return result
            wrapper.ats = [] # declare attribute to use later
16             return wrapper
        return mytimeit_real_decorator

```

Vor der Definition einer Funktion, die wir messen wollen, schreiben wir `@mytimeit(rep=nrEXP)`, probieren Sie z.B.

```
nrEXP = 10
```

```

@mytimeit(rep=nrEXP)
def slowmultiply(D,A):
    X = D @ A
    return(X)

```

```

slowmultiply(D,A)
print(slowmultiply.ats)
slowmultiply(D,A)
print(slowmultiply.ats)

```

Bemerkung 1.3.14. Aus der Linearen Algebra wissen wir, dass es manchmal wichtig ist, Operationen mit Zeilen oder Spalten einer Matrix via Matrixmultiplikationen mathematisch darzustellen. Sei $i > j$. Die Operation mit Zeilen

$$\mathbf{B}_{i,:} = (-2)\mathbf{A}_{j,:} + \mathbf{A}_{i,:}$$

realisiert man via Matrixmultiplikation von links mit der Elementarmatrix $\mathbf{E}^{(ij)}$, deren Elemente denen der Einheitsmatrix entsprechen, ausser dem (i, j) -Element

(unterhalb der Hauptdiagonale, da $i > j$), das -2 ist, z.B.:

$$\mathbf{E}^{(2,1)} \mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ -2a_{11} + a_{21} & -2a_{12} + a_{22} & -2a_{13} + a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}.$$

Multiplikation von rechts mit einer solchen Elementarmatrix agiert mit den Spalten von $\mathbf{A}$:

$$\mathbf{A} \mathbf{E}^{(2,1)} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} a_{11} - 2a_{12} & a_{12} & a_{13} \\ a_{21} - 2a_{22} & a_{22} & a_{23} \\ a_{31} - 2a_{32} & a_{32} & a_{33} \end{bmatrix},$$

d.h. $\mathbf{B}_{:,j} = \mathbf{A}_{:,i} + (-2) \cdot \mathbf{A}_{:,j}$, da $i > j$. Für $i < j$ haben wir $\mathbf{B}_{:,j} = (-2) \cdot \mathbf{A}_{:,i} + \mathbf{A}_{:,j}$:

$$\mathbf{A} \mathbf{E}^{(1,2)} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} 1 & -2 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} a_{11} & -2a_{11} + a_{12} & a_{13} \\ a_{21} & -2a_{21} + a_{22} & a_{23} \\ a_{31} & -2a_{31} + a_{32} & a_{33} \end{bmatrix}.$$

Bemerkung 1.3.15. Das Produkt zweier Matrizen kann auch blockweise berechnet werden. Dabei muss man nur darauf achten, dass die Multiplikation von Matrizen/Blöcken nicht kommutativ ist.

Seien die Dimensionen $M, N, K \in \mathbb{N}$ und die Blocklängen $1 \leq n < N$ ($n' := N - n$), $1 \leq m < M$ ($m' := M - m$), $1 \leq k < K$ ($k' := K - k$); nehmen wir die Matrizen

$$\begin{array}{llll} \mathbf{A}_{11} \in \mathbb{R}^{m,n} & \mathbf{A}_{12} \in \mathbb{R}^{m,n'} & \mathbf{B}_{11} \in \mathbb{R}^{n,k} & \mathbf{B}_{12} \in \mathbb{R}^{n,k'} \\ \mathbf{A}_{21} \in \mathbb{R}^{m',n} & \mathbf{A}_{22} \in \mathbb{R}^{m',n'} & \mathbf{B}_{21} \in \mathbb{R}^{n',k} & \mathbf{B}_{22} \in \mathbb{R}^{n',k'} \end{array},$$

die wir als Blöcke verwenden, um grössere Matrizen zu formen:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \in \mathbb{R}^{M,N}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{bmatrix} \in \mathbb{R}^{N,K}.$$

Das Produkt $\mathbf{AB}$ kann man mit derselben Formel wie das Produkt einfacher 2×2 -Matrizen ausrechnen:

$$\begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21} & \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22} \\ \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21} & \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22} \end{bmatrix}. \quad (1.3.11)$$

Name	Operation	$\#\{*, /\}$	$\#\{+,-\}$	Komplexität
Skalarprodukt	$\mathbf{x} \in \mathbb{R}^n, \mathbf{y} \in \mathbb{R}^n \rightarrow \mathbf{x}^H \mathbf{y}$	n	n-1	$O(n)$
Tensorprodukt	$\mathbf{x} \in \mathbb{R}^m, \mathbf{y} \in \mathbb{R}^n \rightarrow \mathbf{xy}^H$	nm	0	$O(mn)$
Matrix $\times$ Vektor	$\mathbf{x} \in \mathbb{R}^n, \mathbf{A} \in \mathbb{R}^{m,n} \rightarrow \mathbf{Ax}$	mn	(n-1)m	$O(mn)$
Matrixprodukt	$\mathbf{A} \in \mathbb{R}^{m,n}, \mathbf{B} \in \mathbb{R}^{n,p} \rightarrow \mathbf{AB}$	mnp	(n-1)mp	$O(mnp)$

Bemerkung 1.3.16. Für $m = n = p$ kann man das Matrixprodukt mittels des Divide-and-Conquer-Algorithmus von Strassen dank Block-Partitionierung und Rekursion zu einer asymptotischen Komplexität von $O(n^{\log_2 7})$ und sogar $O(n^{2.36})$ reduzieren, was jedoch für Anwendungen bedeutungslos ist.

Bei der Multiplikation mit Diagonalmatrizen haben wir den Effekt von “Denken vor Rechnen” gesehen: statt $\mathcal{O}(n^3)$ kann man einfach $\mathcal{O}(n^2)$ erreichen, wenn man elementare lineare Algebra verwendet. Hier kommen weitere Beispiele dazu.

Beispiel 1.3.17. Matrizen vom Rang 1 kann man immer als Tensorprodukt zweier Vektoren schreiben, was für eine geschickte weitere Rechnung nützlich sein kann:

$$\mathbf{y} = (\mathbf{a}\mathbf{b}^T) \mathbf{x}$$

kostet $\mathcal{O}(mn)$ Operationen, denn die voll besetzte $m \times n$ -Matrix $(\mathbf{a}\mathbf{b}^T)$ multipliziert $\mathbf{x}$. Im Gegensatz dazu:

$$\mathbf{y} = \mathbf{a} (\mathbf{b}^T \mathbf{x})$$

kostet $\mathcal{O}(m + n)$ Operationen, denn $\mathbf{b}^T \mathbf{x}$ ist das Skalarprodukt.

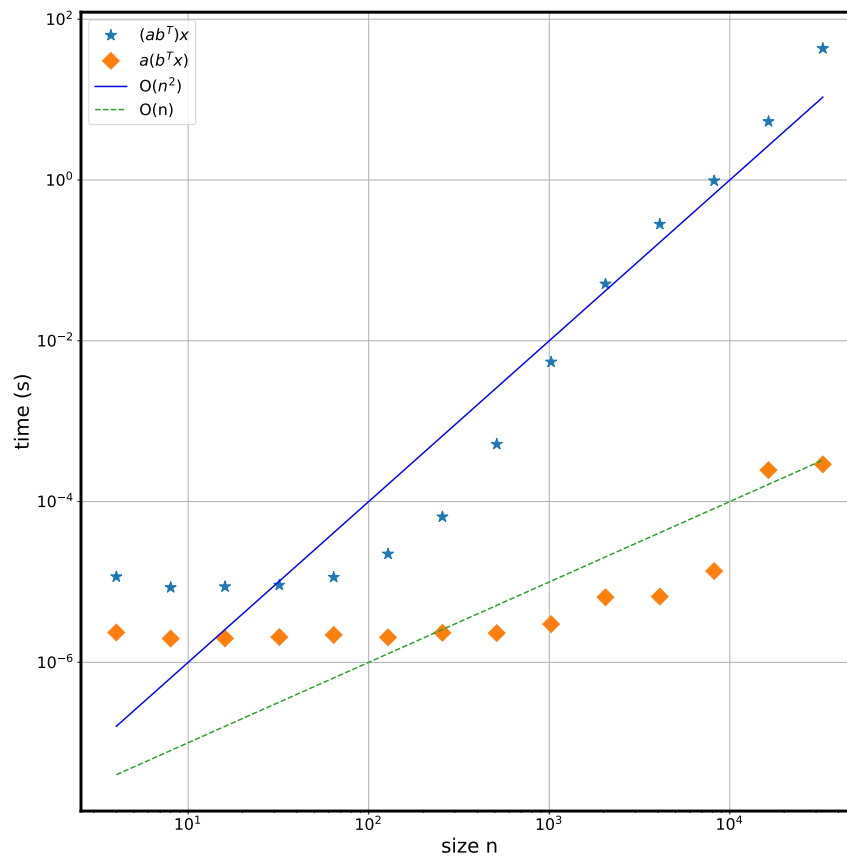


Abb. 1.3.18. Rechenzeiten für die zwei Arten, die Multiplikation mit einer Matrix vom Rang 1 zu berechnen.

Beispiel 1.3.19. Seien $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n,p}$ mit $p \ll n$, also mit kleinem Rang, und sei $\mathbf{R}$ der obere/rechte Teil des Produkts $\mathbf{A}\mathbf{B}^T$. Die direkte Multiplikation $\mathbf{R}\mathbf{x}$, einfach implementiert als `np.triu(A @ B.T) @ x`, kostet $\mathcal{O}(pn^2)$ Operationen. Schauen

wir, was diese Operation für $p = 1$ eigentlich ist:

$$\begin{aligned}
 \mathbf{y} = \text{triu}(\mathbf{a}\mathbf{b}^T)\mathbf{x} &= \begin{bmatrix} a_1b_1 & a_1b_2 & \dots & \dots & a_1b_n \\ 0 & a_2b_2 & a_2b_3 & \dots & a_2b_n \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & 0 & a_nb_n \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ \vdots \\ \vdots \\ x_n \end{bmatrix} \\
 &= \begin{bmatrix} a_1 & & & & \\ & \ddots & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & a_n \end{bmatrix} \underbrace{\left(\begin{bmatrix} 1 & 1 & \dots & \dots & 1 \\ 0 & 1 & 1 & \dots & 1 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & 0 & 1 \end{bmatrix} \right)}_{\mathbf{T}} \begin{bmatrix} b_1 & & & & \\ & \ddots & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ \vdots \\ \vdots \\ x_n \end{bmatrix} .
 \end{aligned}$$

Die Klammern zeigen die Reihenfolge der Operationen. Wir sehen, der teuerste Teil ist die Multiplikation mit der speziellen Matrix $\mathbf{T}$: eine obere Dreiecksmatrix nur mit 1-en. Das Ergebnis der Multiplikation dieser Matrix $\mathbf{T}$ mit einem Vektor $\mathbf{v}$ ist erstaunlich einfach:

$$\begin{bmatrix} 1 & 1 & \dots & \dots & 1 \\ 0 & 1 & 1 & \dots & 1 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & 0 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ \vdots \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} s_n \\ s_{n-1} \\ \vdots \\ \vdots \\ s_1 \end{bmatrix} , \quad s_j := \sum_{k=n-j+1}^n v_k ,$$

d.h. das Ergebnis enthält die Teilsummen der Elemente des (umgekehrten) Vektors $\mathbf{v}$. Teilsummen können mit `v.cumsum()` oder `np.cumsum(v)` effizient berechnet werden. In diesem Fall kann man also die Berechnung so organisieren, dass das Ganze nur $\mathcal{O}(n)$ statt $\mathcal{O}(n^2)$ Operationen braucht. Für p Spalten macht das entsprechend $\mathcal{O}(n \cdot p)$ Operationen via

$$\mathbf{A}\mathbf{B}^T = \sum_{\ell=1}^p \mathbf{A}_{:, \ell} (\mathbf{B}_{:, \ell})^T ,$$

was für $p \ll n$ deutlich günstiger als $\mathcal{O}(pn^2)$ der direkten Implementierung ist.

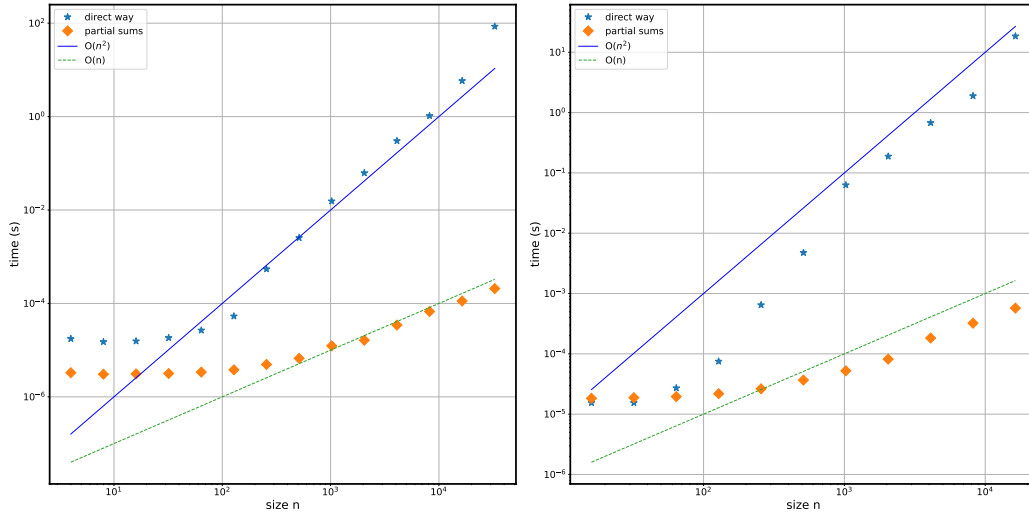


Abb. 1.3.20. Rechenzeiten für die zwei Arten, die Multiplikation mit dem oberen Teil einer Matrix vom Rang $p = 1$ (links) und $p = 5$ (rechts) zu berechnen.

Definition 1.3.21. Das Kronecker-Produkt $\mathbf{A} \otimes \mathbf{B}$ zweier beliebiger Matrizen $\mathbf{A} \in \mathbb{R}^{m,n}$ und $\mathbf{B} \in \mathbb{R}^{l,k}$ mit $m, n, l, k \in \mathbb{N}$ ist die $(ml) \times (nk)$ -Matrix

$$\mathbf{A} \otimes \mathbf{B} := \begin{bmatrix} (\mathbf{A})_{1,1}\mathbf{B} & (\mathbf{A})_{1,2}\mathbf{B} & \dots & \dots & (\mathbf{A})_{1,n}\mathbf{B} \\ (\mathbf{A})_{2,1}\mathbf{B} & (\mathbf{A})_{2,2}\mathbf{B} & & & \vdots \\ \vdots & \vdots & & & \vdots \\ \vdots & \vdots & & & \vdots \\ (\mathbf{A})_{m,1}\mathbf{B} & (\mathbf{A})_{m,2}\mathbf{B} & \dots & \dots & (\mathbf{A})_{m,n}\mathbf{B} \end{bmatrix} \in \mathbb{R}^{ml,nk}.$$

Beispiel 1.3.22 (Multiplikation des Kronecker-Produkts mit einem Vektor). Würden wir zuerst die Matrix $\mathbf{A} \otimes \mathbf{B}$ bauen und dann mit dem nk -langen Vektor $\mathbf{x}$ multiplizieren, hätten wir die Komplexität $\mathcal{O}(m \cdot n \cdot l \cdot k)$. Wenn wir den Vektor $\mathbf{x}$ in n gleich grosse Blöcke aufteilen

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}^1 \\ \mathbf{x}^2 \\ \vdots \\ \mathbf{x}^n \end{bmatrix} \quad \text{mit } \mathbf{x}^j \in \mathbb{R}^k,$$

können wir das Ergebnis direkt berechnen:

$$(\mathbf{A} \otimes \mathbf{B})\mathbf{x} = \begin{bmatrix} \mathbf{A}_{1,1}\mathbf{B}\mathbf{x}^1 + \mathbf{A}_{1,2}\mathbf{B}\mathbf{x}^2 + \dots + \mathbf{A}_{1,n}\mathbf{B}\mathbf{x}^n \\ \mathbf{A}_{2,1}\mathbf{B}\mathbf{x}^1 + \mathbf{A}_{2,2}\mathbf{B}\mathbf{x}^2 + \dots + \mathbf{A}_{2,n}\mathbf{B}\mathbf{x}^n \\ \vdots \\ \vdots \\ \mathbf{A}_{m,1}\mathbf{B}\mathbf{x}^1 + \mathbf{A}_{m,2}\mathbf{B}\mathbf{x}^2 + \dots + \mathbf{A}_{m,n}\mathbf{B}\mathbf{x}^n \end{bmatrix}.$$

Die Idee ist, die kleineren Produkte $\mathbf{B}\mathbf{x}^j$ für $j = 1, \dots, n$ auf einmal zu berechnen und diese dann mit den Koeffizienten aus den Spalten von $\mathbf{A}$ linear zu kombinieren:

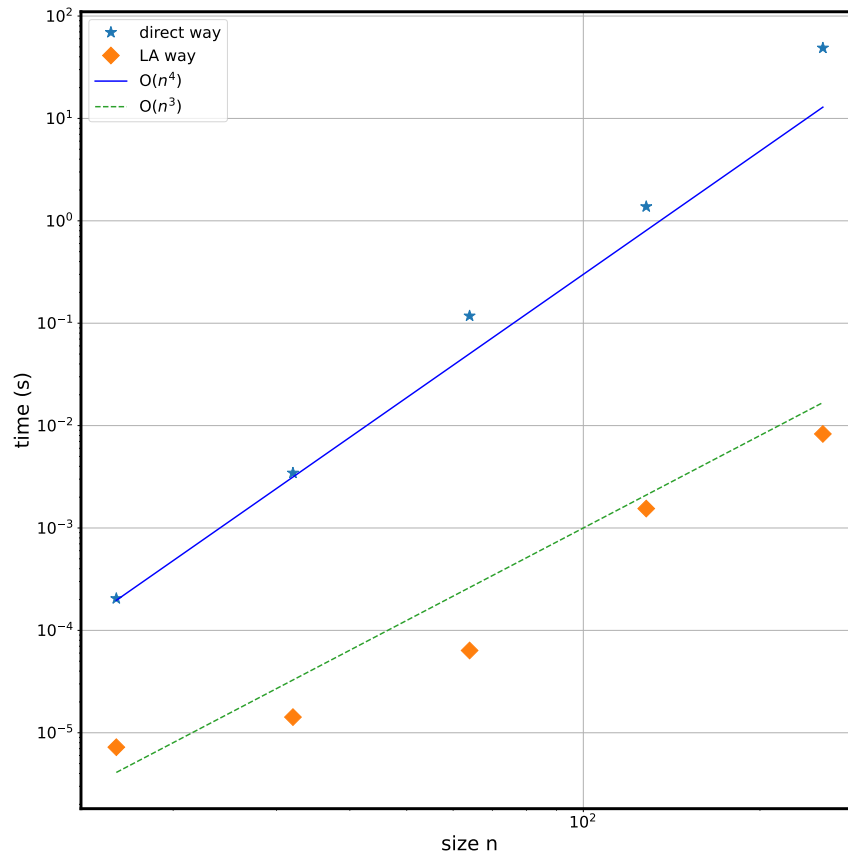


Abb. 1.3.23. Rechenzeiten für die zwei Arten, $(\mathbf{A} \otimes \mathbf{B})\mathbf{x}$ zu berechnen.

Chapter 2

Polynominterpolation

2.1 Interpolation und Polynome

In diesem und den nächsten Kapiteln wird es um die Interpolation von Funktionen gehen. Wir nehmen an, dass wir eine Funktion haben, die z.B. einen physikalischen Vorgang beschreibt, aber deren analytische Form sehr verschachtelt, unhandlich oder gar nicht bekannt ist. Meistens ist es aber nützlich, Extremstellen, Ableitungen oder Integrale davon zu bestimmen. Hier bietet es sich an, die komplizierte Funktion durch eine wesentlich einfachere Funktion zu approximieren. Je nach Interpolationsverfahren erhalten wir möglicherweise sogar andere Einsichten in die Funktion; z.B. können wir Informationen über die Hauptschwingung erhalten. Wir wollen also in diesem Kapitel einige Methoden zur Interpolation erarbeiten.

Definitionen

Das Ziel der Interpolation ist es, eine Funktion $\tilde{f}$ zu gegebenen Datenpunkten (bzw. Funktionswerten) zu finden:

$$\begin{bmatrix} x_0 & x_1 & \cdots & x_n \\ y_0 & y_1 & \cdots & y_n \end{bmatrix}, \quad x_i \in \mathbb{R}, y_i \in \mathbb{R}.$$

Dabei heissen x_i **Stützstellen** oder **Knoten** und $y_i = f(x_i)$ sind die dazugehörigen Datenpunkte (bzw. Funktionswerte). Die **Interpolationsbedingung**

$$\tilde{f}(x_i) = y_i \quad \text{für alle } i = 1, 2, \dots, n \quad (2.1.1)$$

sollte exakt (und nicht im Sinne der Ausgleichsrechnung) erfüllt werden.

Die interessante und vorgegebene Funktion f liegt normalerweise in einem unendlich dimensionalen Vektorraum $\mathcal{V}$ von Funktionen gewisser Glattheit. Wir suchen also zu $f \in \mathcal{V}$ eine Approximation $\tilde{f}$ in einem linearen Unterraum $\mathcal{V}_n$ (von $\mathcal{V}$) der endlichen Dimension $\dim \mathcal{V}_n = n$. Dabei werden wir die Elemente einer Basis des Raumes $\mathcal{V}_n$ verwenden, sodass wir schreiben können:

$$f(x) \approx \tilde{f}_n(x) = \sum_{j=1}^n \alpha_j b_j(x).$$

Hier bilden die $b_j(x)$ eine Basis von $\mathcal{V}_n$: $\text{span}\{b_1, \dots, b_n\} = \mathcal{V}_n$.

Bemerkung 2.1.1. Hier handelt es sich *nicht* um eine Ausgleichsrechnung. In der Tat bauen wir eine Interpolationsfunktion, die durch *alle* Datenpunkte $y_i = \tilde{f}(x_i)$ laufen soll.

Bemerkung 2.1.2. Für einen linearen Raum $\mathcal{V}$ kann man Unterräume $\mathcal{V}_n$ von verschiedenen Arten von Funktionen betrachten oder sogar für eine Art von Unterraum $\mathcal{V}_n$ verschiedene Basen verwenden. Zunächst wollen wir uns aber ausschliesslich mit Polynomen beschäftigen. Wir werden im Folgenden häufig $p(x)$ für $\tilde{f}(x)$ schreiben, um diesen Sachverhalt zu verdeutlichen.

Beispiel 2.1.3. Beispiele von möglichen $b_j(x)$ sind:

- | | | |
|-----|---|--------------------------------------|
| (1) | $b_j(x) = x^{j-1}$ | polynomielle Interpolation |
| (2) | $b_j(x) = \cos((j-1) \arccos(x))$ | Chebyshev ¹ Interpolation |
| (3) | $b_j(x) = e^{i2\pi jx} = \cos(2\pi jx) + i \sin(2\pi jx)$ | trigonometrische Interpolation. |

Beispiel 2.1.4. (Taylor-Formel)

Aus der Analysis ist die Taylor-Formel (um den Punkt a) bereits bekannt:

$$f(x) = p_n(x) + R_n(x) \quad \text{mit Restglied} \quad R_n(x) = \frac{(x-a)^{n+1}}{(n+1)} f^{(n+1)}(\xi) \quad (2.1.2)$$

mit ξ zwischen a und x . Die Taylor-Formel wird häufig in der Mathematik und Physik verwendet, um Funktionen lokal zu approximieren. Für numerische Berechnungen ist sie aber nicht gut genug, da sie nur lokal approximiert und auch Ableitungen benötigt werden, die wir eventuell nur schwer oder gar nicht ausrechnen können.

Existenz

Es stellt sich natürlich die Frage, ob unsere Interpolationsfunktionen überhaupt gute Approximationen sind oder ob es überhaupt möglich ist, jede Funktion zu approximieren. Dazu kennen wir aber den aus der Analysis bekannten Satz:

Theorem 2.1.5. (Satz von Peano)

Angenommen f ist stetig, dann gilt:

Es existiert ein Polynom p , welches die Funktion f in der $\|\cdot\|_\infty$ -Norm beliebig gut approximiert.

Bemerkung 2.1.6. Je glatter f ist, desto bessere Konvergenz erhalten wir im Allgemeinen mit den einzelnen Verfahren.

¹Die korrekte deutsche Schreibweise ist **Tschebyschow**; ich verwende jedoch die gebräuchliche englische Schreibweise, die in der Literatur verwendet wird, obwohl er selbst meist auf Französisch unter dem Namen Tchebychef publiziert hat.

Polynome und Monombasis

Definition 2.1.7. (Raum der Polynome)

Der Raum der Polynome vom Grad $\leq k, k \in \mathbb{N}$ ist gegeben durch:

$$\mathcal{P}_k := \{x \mapsto \alpha_k x^k + \alpha_{k-1} x^{k-1} + \cdots + \alpha_1 x + \alpha_0, \alpha_j \in \mathbb{K}\}. \quad (2.1.3)$$

Definition 2.1.8. (Monom)

Die Funktionen der Form $x \mapsto x^k, k \in (\mathbb{N} \cup \{0\})$ heissen *Monome*.

$x \mapsto \alpha_k x^k + \alpha_{k-1} x^{k-1} + \cdots + \alpha_0$ ist die *Monom-Darstellung* eines Polynoms.

Theorem 2.1.9. (Eigenschaften des Raumes der Polynome)

$\mathcal{P}_k$ ist ein Vektorraum der Dimension $k + 1$. Zusätzlich gilt $\mathcal{P}_k \subset C^\infty(\mathbb{R})$.

Proof. Die Monome vom Grad $\leq k$ bilden eine Basis des Vektorraums der Polynome, also gilt $\dim \mathcal{P}_k = k + 1$. Ausserdem sind alle Polynome ∞ -oft stetig differenzierbar und liegen somit in $C^\infty(\mathbb{R})$ (man schreibt $\mathcal{P}_k \subset C^\infty(\mathbb{R})$). $\square$

Berechnung mit der Monombasis

Theorem 2.1.10. (Eindeutigkeit)

Ein Polynom $p(x)$ vom k -ten Grad ist durch $k + 1$ Punkte $y_i = p(x_i)$ eindeutig bestimmt.

Damit haben wir bereits eine Methode zur Polynominterpolation kennengelernt. Wir müssen lediglich die Funktion, die wir approximieren wollen, an einer bestimmten Anzahl von Stützstellen auswerten. Danach lassen sich die Koeffizienten in der Monombasis durch Lösen eines Gleichungssystems leicht bestimmen.

Beispiel 2.1.11. (Berechnung der Koeffizienten in der Monombasis)

Wir möchten das Polynom durch die Punkte $y_0, \dots, y_n$ bestimmen:

Das gesuchte Polynom $p_n \in \mathcal{P}_n$ hat in der Monombasis die Form:

$$p_n(x) = \alpha_n x^n + \cdots + \alpha_1 x^1 + \alpha_0.$$

Wir finden also das lineare Gleichungssystem:

$$\begin{bmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

Bemerkung 2.1.12. Dieses Gleichungssystem weist die Vandermonde-Matrix auf; deswegen ist es sehr schlecht konditioniert (die Vandermonde-Matrix sehen wir auch in einem Beispiel bei der QR-Zerlegung, vgl. Abbildung 7.1.34). Würden die Koeffizienten $\alpha_0, \dots, \alpha_n$ daraus berechnet werden, würden diese sehr ungenau ausfallen. Die Funktion `polyfit` kann für die bequeme Berechnung von $\alpha_0, \dots, \alpha_n$ verwendet werden; sie gibt aber auch die Warnung `RankWarning: Polyfit may be poorly conditioned` aus.

Auswertung und Bemerkungen

Allgemein lassen sich Polynome, wie im Beispiel oben, über die Matrixdarstellung auswerten: Mit bekannten α_i lassen sich aus den Auswertungsstellen t_i die dazugehörigen y_i bestimmen. Eine effizientere Möglichkeit der Auswertung ist aber das sogenannte Horner-Schema:

Definition 2.1.13. (Horner-Schema)

$$p(x) = (x \dots x (x (\alpha_n x + \alpha_{n-1}) + \dots + \alpha_1) + \alpha_0). \quad (2.1.4)$$

Bemerkung 2.1.14. Das Polynom $p_n(x) = \alpha_n x^n + \dots + \alpha_1 x^1 + \alpha_0$ wird in `python` durch ein Array `p = [ $\alpha_n, \alpha_{n-1}, \dots, \alpha_1, \alpha_0$ ]` der Länge $n+1$ repräsentiert. Das Modul `numpy` stellt die Funktion `polyval(p, x)` zur Verfügung, um ein solches Polynom via Horner-Schema schnell auszuwerten. Mit unserer Notation haben wir `p[0] =  $\alpha_n$` , `p[1] =  $\alpha_{n-1}$` , $\dots$, `p[n] =  $\alpha_0$` und eine mögliche Implementierung ist:

Code 2.1.15: Horner-Schema

```

1  def horner(p, x):
2      y = p[0]
3      for i in range(1, len(p)):
4          y = x * y + p[i]
5      return y

```

`p` ist hier ein Array mit den Koeffizienten α_i des Interpolationspolynoms.

Bemerkung 2.1.16. Die Monombasis ist zwar sehr anschaulich, hat aber keine nützlichen numerischen Eigenschaften, sondern eher Nachteile. Daher wird diese auch nur sehr selten verwendet. Das Horner-Schema wirkt aber stabilisierend: Obwohl `polyfit` grosse Fehler bei der Berechnung der Koeffizienten $\alpha_0, \dots, \alpha_n$ produziert, liefert die Auswertung über `polyval` wieder relativ gute Ergebnisse.

2.2 Newton-Basis und dividierte Differenzen

Grundlagen

In der Monombasis können neu gewonnene Datenpunkte nur durch Neuberechnung des Interpolationspolynoms eingebunden werden. Die Newton-Basis erlaubt das Hinzufügen von Datenpunkten, ohne die bereits vorhandenen Informationen zu verwerfen. Betrachten wir hierzu zunächst den einfachen Fall, in dem wir nur eine Stützstelle haben und eine weitere hinzufügen möchten:

Beispiel 2.2.1. (Motivation der Newton-Basis)

Angenommen, wir haben zunächst nur eine Stützstelle (x_0, y_0) zur Verfügung. Daraus ergibt sich trivialerweise als Interpolationspolynom z.B. $p_0(x) = y_0$. Nun

möchten wir eine zweite Stützstelle (x_1, y_1) hinzufügen, ohne das bisher berechnete Polynom zu verwerfen. Mithilfe der Interpolationsbedingungen $p_0(x_0) = y_0$ und $p_1(x_1) = y_1$ finden wir folgendes Polynom:

$$p_0(x_0) = y_0 \xrightarrow{(x_1, y_1)} p_1(x) = p_0(x) + (x - x_0) \frac{(y_1 - y_0)}{(x_1 - x_0)},$$

Nun möchten wir noch eine weitere Stützstelle (x_2, y_2) hinzufügen. Wir wenden also dasselbe Schema an:

$$p_2(x) = p_1(x) + \frac{\left(\frac{y_2 - y_1}{x_2 - x_1}\right) - \left(\frac{y_1 - y_0}{x_1 - x_0}\right)}{x_2 - x_0} (x - x_0)(x - x_1).$$

Dieses Schema lässt sich beliebig fortführen.

Bemerkung 2.2.2. Nach Theorem 2.1.10 sind die im obigen Beispiel gefundenen Polynome eindeutig und entsprechen den Polynomen, die wir mit der "naiven Herangehensweise" erhalten würden.

Newton-Basis und dividierte Differenzen

Theorem 2.2.3. (Newton-Basis)

Die Newton-Basis:

$$N_0(x) := 1, \quad N_1(x) := x - x_0, \quad N_2(x) := (x - x_0)(x - x_1), \dots, \quad N_n(x) := \prod_{i=0}^{n-1} (x - x_i) \quad (2.2.5)$$

ist eine Basis des Raumes $\mathcal{P}_n$.

Wie erhalten wir im Allgemeinen die Koeffizienten β_j zur Darstellung eines Polynoms in der Newton-Basis?

$$p_n(x) = \beta_0 N_0(x) + \beta_1 N_1(x) + \dots + \beta_n N_n(x), \quad \text{mit } \beta_j \in \mathbb{R}.$$

Wir können diese Koeffizienten β_j durch das Lösen eines linearen Gleichungssystems erhalten:

$$\beta_0 N_0(x_j) + \beta_1 N_1(x_j) + \dots + \beta_n N_n(x_j) = y_j, \quad j = 0, \dots, n$$

$$\Leftrightarrow \begin{bmatrix} 1 & 0 & \dots & 0 \\ 1 & (x_1 - x_0) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x_0) & \dots & \prod_{i=0}^{n-1} (x_n - x_i) \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

Dabei haben wir verwendet:

$$N_n(x) = \prod_{i=0}^{n-1} (x - x_i) = 0 \quad \text{für } x = x_0, \dots, x_{n-1}.$$

Bemerkung 2.2.4. Beim Lösen dieses Gleichungssystems fällt auf, dass immer wieder dieselben Ausdrücke berechnet werden. Die dividierten Differenzen nutzen diesen Sachverhalt und stellen damit eine effizientere Methode zur Berechnung der Koeffizienten dar.

Definition 2.2.5. (Dividierte Differenzen)

Die dividierten Differenzen sind definiert durch:

$$y[x_i] = y_i \text{ und } y[x_i, \dots, x_{i+k}] = \frac{y[x_{i+1}, \dots, x_{i+k}] - y[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}. \quad (2.2.6)$$

Zur Berechnung verwendet man folgendes Schema:

$$\begin{array}{c|c} x_0 & y[x_0] \\ & > y[x_0, x_1] \\ x_1 & y[x_1] & > y[x_0, x_1, x_2] \\ & > y[x_1, x_2] & > y[x_0, x_1, x_2, x_3] \\ x_2 & y[x_2] & > y[x_1, x_2, x_3] \\ & > y[x_2, x_3] \\ x_3 & y[x_3], \end{array} \quad (2.2.7)$$

wobei jedes ">" für eine in (2.2.6) definierte Rekursion steht.

Bemerkung 2.2.6. (Äquidistante Stützstellen)

Falls die x_j äquidistant mit Abstand h sind, d.h. falls $x_j = x_0 + j \cdot h$, so können wir schreiben:

$$\begin{aligned} y[x_0, x_1] &= \frac{y_1 - y_0}{h} &:= \frac{1}{h} \Delta y_0 \\ y[x_0, x_1, x_2] &= \frac{\frac{1}{h} \Delta y_1 - \frac{1}{h} \Delta y_0}{2h} &:= \frac{1}{2! h^2} \Delta^2 y_0 \\ &\vdots &\vdots \\ y[x_0, \dots, x_n] &= \frac{1}{n! h^n} \Delta^n y_0. \end{aligned} \quad (2.2.8)$$

Beachte: Das n in Δ^n ist als Index zu verstehen, nicht als Potenz!

Code 2.2.7: Dividierte Differenzen in einer Matrix

```

1  def divdiff(x, y):
2      n = y.size
3      T = np.zeros((n, n))
4      T[:,0] = y
5      for level in range(1, n):
6          for i in range(n-level):
7              T[i, level] = (T[i+1, level-1] - T[i, level-1]) / (x[i+level]-x[i])
8
9      return T[0,:]
```

Theorem 2.2.8. (Newton, 1676)

Es gibt ein Polynom n -ten Grades, das durch $(x_0, y_0), \dots, (x_n, y_n)$ geht. Dieses lässt sich wie folgt berechnen:

$$p(x) = y[x_0] + y[x_0, x_1](x - x_0) + \dots + y[x_0, x_1, \dots, x_n](x - x_0) \cdots (x - x_{n-1}).$$

Die Koeffizienten β_j in der Newton-Basis sind also gegeben durch $\beta_j = y[x_0, x_1, \dots, x_j]$.

Auswertung, Kosten und Fehler

Für die Auswertung des Interpolationspolynoms bietet sich wieder ein abgewandeltes Horner-Schema an:

Wir verwenden p im folgenden Algorithmus als "Speichervariable":

$$\begin{aligned} p &:= \beta_n \\ p &:= (x - x_{n-1})p + \beta_{n-1} \\ p &:= (x - x_{n-2})p + \beta_{n-2}. \end{aligned}$$

Bemerkung 2.2.9. Der Aufwand (gegliedert nach den Teilschritten) ist gegeben durch:

- (1) $O(n^2)$ für die Berechnung der dividierten Differenzen;
- (2) $O(n)$ jeweils für jede Auswertung.

Bemerkung 2.2.10. Der obige naive Algorithmus zur Berechnung der dividierten Differenzen mit n Interpolationsknoten verwendet einen Teil des Speichers einer $n \times n$ -Matrix T . Für das Newton-Polynom brauchen wir aber nicht alle diese Werte, sondern nur die Elemente auf der Diagonale. Wenn wir die Definition der dividierten Differenzen anschauen, sehen wir, dass sich jedes Element auf der Diagonale von T auch in einer anderen Reihenfolge berechnen lässt. Das zeigt uns einerseits, wie das neue Newton-Polynom aus dem alten entsteht, wenn wir einen Interpolationsknoten dazunehmen, und andererseits ist das ein Hinweis, wie man das Polynom nur mit dem Speicher eines Vektors der Länge n aufbaut.

Wir nehmen an, wir haben bereits aus n Interpolationsbedingungen

$$(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$$

die Koeffizienten des Newton-Polynoms vom Grad $n - 1$ erhalten, die wir über den Speicher der Messungen geschrieben haben

$$y[k] := \beta_k = y[x_0, x_1, \dots, x_k] \text{ für } k = 0, 1, \dots, n - 1,$$

sodass die Messungen $y_1, \dots, y_{n-1}$ nicht mehr zur Verfügung stehen. Wenn wir die neue Information (x_n, y_n) erhalten, können wir die dividierten Differenzen bilden

$$y[x_0, x_n], y[x_0, x_1, x_n], y[x_0, x_1, x_2, x_n], \dots, y[x_0, x_1, \dots, x_{n-1}, x_n],$$

als ob wir (x_n, y_n) nicht unten, sondern oben in der Tabelle (2.2.7) hinzufügen würden. Dabei verwenden wir nur den Speicherort der neuen Messung y_n .

Aufgabe: Implementieren Sie einen sequenziellen Algorithmus, der die Koeffizienten des Newton-Interpolationspolynoms nur mit der Verwendung von $\mathcal{O}(n)$ Speicherplätzen und $\mathcal{O}(n^2)$ Operationen auskommt.

Aufgabe: Schreiben Sie die Implementierung in einer vektorisierten Form um.

Aufgabe: Implementieren Sie einen rekursiven Algorithmus mit denselben Eigenschaften. Entdecken Sie experimentell die Schranke, die Python auf die Rekursionstiefe setzt. Ist ein rekursiver Algorithmus hier überhaupt sinnvoll? Warum?

Bisher haben wir uns noch nicht mit dem Fehler $\|f(x) - p(x)\|$ des Interpolationspolynoms beschäftigt. Insbesondere zur Entwicklung besserer Interpolationsverfahren möchten wir den Fehler nun genauer abschätzen.

Theorem 2.2.11. Angenommen, f ist n -mal stetig differenzierbar und $y_i = f(x_i)$ für $i = 0, \dots, n$, dann existiert ein $\xi \in]\min_i x_i, \max_i x_i[$, sodass

$$y[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}. \quad (2.2.9)$$

Theorem 2.2.12. (Fehler)

Sei $f : [a, b] \rightarrow \mathbb{R}$ $(n+1)$ -mal stetig differenzierbar und p das Interpolationspolynom zu f in $x_0, x_1, \dots, x_n \in [a, b]$. Dann gilt:
für jedes $x \in [a, b]$ gibt es ein $\xi \in]a, b[$ mit

$$f(x) - p(x) = (x - x_0) \cdots (x - x_n) \frac{f^{(n+1)}(\xi)}{(n+1)!}. \quad (2.2.10)$$

Damit kann der Fehler maximal

$$(b - a)^{n+1} \frac{\|f^{(n+1)}\|_\infty}{(n+1)!} \quad (2.2.11)$$

sein.

Beispiel 2.2.13. Als Beispiel einer Anwendung dieses Satzes wollen wir uns die Funktion $f(x) = \log_{10} x = \frac{\ln x}{\ln 10}$ auf dem Intervall $[55, 58]$ anschauen. Als Stützstellen verwenden wir $\{55, 56, 57, 58\}$. Als Fehler an der Stelle $x = 56.5$ ergibt sich:

$$|\log_{10}(56.5) - p(56.5)| \leq 1.5 \cdot 0.5 \cdot 0.5 \cdot 1.5 \cdot \frac{6}{(55)^4 \ln(10)} \cdot \frac{1}{4!} \approx 6.7 \cdot 10^{-9}.$$

Der geringe Fehler, den wir erhalten haben, lässt sich vor allem auf das kleine Intervall und die geringe Oszillation der Funktion zurückführen.

Aus der Formel zur Abschätzung des Fehlers geht hervor, dass die Wahl der Stützstellen massgeblichen Einfluss auf den Fehler hat. Also stellt sich natürlich die Frage, welche Wahl von $x_1, \dots, x_n$ der Stützstellen ist am besten?

D.h. wann ist $\max_{x \in [a, b]} |(x - x_0) \cdots (x - x_n)|$ minimal?

Eine andere Frage ist, wie wirken sich Fehler in den Funktionswerten (z.B. Messfehler) auf das Interpolationspolynom aus?

$$\left. \begin{array}{ll} (x_i, y_i) & \text{für } i = 0, 1, \dots, n \rightarrow p(x) \\ (x_i, \tilde{y}_i) & \text{für } i = 0, 1, \dots, n \rightarrow \tilde{p}(x) \end{array} \right\} p(x) - \tilde{p}(x) = ?$$

Um diese Fragen zu beantworten, wechseln wir in eine andere Basis des Raumes der Polynome $\mathcal{P}$, die sogenannten Lagrange-Polynome. Die Lagrange-Basis hat einige nützliche Eigenschaften, die uns helfen werden, diese Fragen zu beantworten.

2.3 Lagrange- und baryzentrische Interpolationsformeln

Im letzten Abschnitt haben wir uns mit dem Fehler im Interpolationspolynom beschäftigt. Wir möchten hier eine Basis einführen, die uns die Untersuchung des Fehlers deutlich vereinfacht. Diese Basis ist zur praktischen Berechnung von Interpolationspolynomen ungeeignet. Stattdessen wird die baryzentrische Formel verwendet.

Definition 2.3.1. (Lagrange-Polynome) Für die Knoten $x_0, x_1, \dots, x_n \in \mathbb{R}$ definieren wir die (Familie von) Polynome(n)

$$\ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}. \quad (2.3.12)$$

Die ℓ_i werden auch die Lagrange-Polynome zu den Stützstellen $x_0, x_1, \dots, x_n$ genannt.

Beispiel 2.3.2. Gegeben seien die Stützstellen x_0, x_1 und x_2 . Wir erhalten die Lagrange-Polynome:

$$\begin{aligned} \ell_0(x) &= \frac{x - x_1}{x_0 - x_1} \cdot \frac{x - x_2}{x_0 - x_2} \\ \ell_1(x) &= \frac{x - x_0}{x_1 - x_0} \cdot \frac{x - x_2}{x_1 - x_2} \\ \ell_2(x) &= \frac{x - x_0}{x_2 - x_0} \cdot \frac{x - x_1}{x_2 - x_1}. \end{aligned}$$

Die Abbildung 2.1 zeigt die 5 Lagrange-Polynome zu den 5 Knoten $0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$.

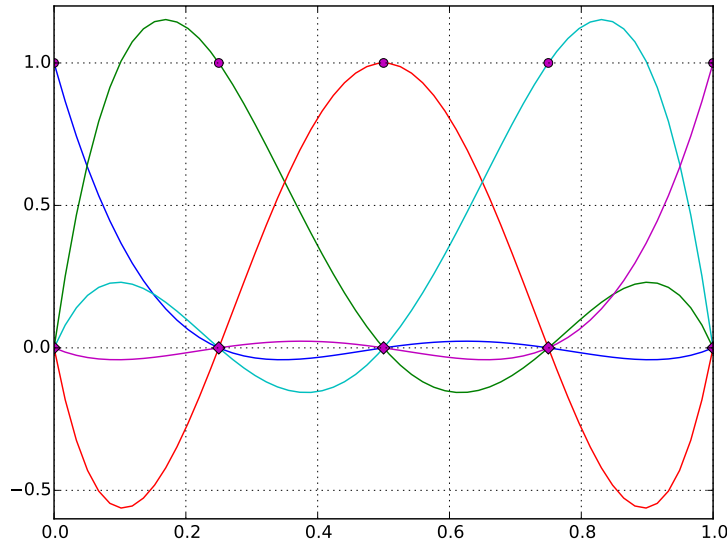


Figure 2.1: Die 5 Lagrange-Polynome zu den Knoten $0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$.

Theorem 2.3.3. (Lagrange-Interpolationsformel)

Die Lagrange-Polynome $\ell_i(x)$ zu den Stützstellen $(x_0, y_0), \dots, (x_n, y_n)$ bilden eine Basis des Raumes der Polynome $\mathcal{P}_n$, und wir können schreiben:

$$p(x) = \sum_{i=0}^n y_i \ell_i(x) \quad \text{mit} \quad \ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}. \quad (2.3.13)$$

Bemerkung 2.3.4. (Eigenschaften der Lagrange-Polynome)

1. $\ell_i(x_j) = 0$ für alle $j \neq i$,
2. $\ell_i(x_i) = 1$ für alle i ,
3. $\text{grad } \ell_i = n$ für alle i .
4. $\sum_{k=0}^n \ell_k(x) = 1$ und $\sum_{k=0}^n \ell_k^{(m)}(x) = 0$ für $m > 0$.

Wenn wir $L(x) = (x - x_0)(x - x_1) \cdots (x - x_n)$ und

$$\lambda_k = \prod_{j \neq k} \frac{1}{x_k - x_j} = \frac{1}{(x_k - x_0) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}$$

definieren, dann lässt sich p umschreiben zu

$$p(x) = L(x) \sum_{k=0}^n \frac{\lambda_k}{x - x_k} y_k.$$

Für das konstante Polynom 1 ergibt das

$$1 = L(x) \sum_{k=0}^n \frac{\lambda_k}{x - x_k},$$

was zur baryzentrischen Formel für das Interpolationspolynom führt:

$$p(x) = \frac{\sum_{k=0}^n \frac{\lambda_k}{x - x_k} y_k}{\sum_{k=0}^n \frac{\lambda_k}{x - x_k}}. \quad (2.3.14)$$

Diese Formel ist skalierungsinvariant bezüglich des Interpolationsintervalls $[a, b]$ mit $a = x_0 < \dots < x_n = b$; die Auswertung der baryzentrischen Formel ist extrem stabil für $x \in [a, b]$ und lässt sich mit $\mathcal{O}(n)$ Operationen durchführen, wenn man die λ_k im Voraus ausrechnet (in $\mathcal{O}(n^2)$ Operationen). Auch einige zusätzliche Stützstellen lassen sich leicht hinzufügen; die darauf basierte Neuauswertung des entstandenen Interpolationspolynoms kostet nur $\mathcal{O}(n)$ Operationen. Für eine spezielle Wahl der Stützstellen ($n+1$ Chebyshev-Abszissen $x_k = \cos(k\pi/n)$) sind die λ_k gegeben durch $\lambda_k = (-1)^k \delta_k$ mit $\delta_0 = \delta_n = 1/2$ und $\delta_k = 1$ sonst. Wenn wir andere λ_k zulassen, dann ergibt die baryzentrische Formel kein Polynom mehr, sodass sich der Weg zu einer Verallgemeinerung zur Interpolation mittels rationaler Funktionen öffnet.

Die baryzentrische Formel kann auch als Ausgangspunkt für die Spektralmethode für Differentialgleichungen genommen werden. Wenn wir die unbekannte Funktion u durch deren Interpolationspolynom ersetzen und die Ableitungen ausrechnen, dann brauchen wir $\ell'_j(x_i)$ und $\ell''_j(x_i)$, um die Differentialgleichung der Form $F(u'', u', u, x) = 0$ in ein lineares Gleichungssystem in den Unbekannten $y_i = u(x_i)$ umzuschreiben. Eine geschickte Rechnung ergibt für $i \neq j$:

$$\ell'_j(x_i) = \frac{\lambda_j/\lambda_i}{x_i - x_j} \quad (2.3.15)$$

$$\ell''_j(x_i) = -2 \frac{\lambda_j/\lambda_i}{x_i - x_j} \left(\sum_{k \neq i} \frac{\lambda_k/\lambda_i}{x_i - x_k} - \frac{1}{x_i - x_j} \right) \quad (2.3.16)$$

während für $i = j$ gilt

$$\begin{aligned} \ell'_j(x_j) &= - \sum_{k \neq j} \ell'_k(x_j) = - \sum_{k \neq j} \frac{\lambda_k/\lambda_j}{x_j - x_k} \\ \ell''_j(x_j) &= - \sum_{k \neq j} \ell''_k(x_j) = 2 \left(\sum_{k \neq j} \frac{\lambda_k/\lambda_j}{x_j - x_k} \right)^2 + 2 \sum_{k \neq j} \frac{\lambda_k/\lambda_j}{(x_j - x_k)^2}. \end{aligned}$$

Fehlerbetrachtung

Wenden wir uns zunächst dem Fall zu, dass, z.B. durch ungenaue Messungen, die Werte $\tilde{y}_i$ an den Stützstellen x_i mit Fehlern behaftet sind. Wir möchten nun den

Fehler zwischen dem korrekten Interpolationspolynom $p(x)$ und dem mit Fehlern behafteten Polynom $\tilde{p}(x)$ berechnen. Dazu schreiben wir p und $\tilde{p}$ in der Lagrange-Basis:

$$p(x) = \sum_{i=0}^n y_i \ell_i(x) \text{ und } \tilde{p}(x) = \sum_{i=0}^n \tilde{y}_i \ell_i(x).$$

Dann ergibt sich für den Fehler:

$$|p(x) - \tilde{p}(x)| = \left| \sum_{i=0}^n (y_i - \tilde{y}_i) \ell_i(x) \right| \leq \max_{i=0, \dots, n} |y_i - \tilde{y}_i| \cdot \sum_{i=0}^n |\ell_i(x)|.$$

Definition 2.3.5. (Lebesgue-Konstante)

Die Lebesgue-Konstante Λ_n zu den Stützstellen $x_0, \dots, x_n$ im Intervall $[a, b]$ ist definiert durch

$$\Lambda_n := \max_{x \in [a, b]} \sum_{i=0}^n |\ell_i(x)|. \quad (2.3.17)$$

Bemerkung 2.3.6. Die Lebesgue-Konstante hängt offensichtlich nur von der Wahl der Stützstellen ab.

Theorem 2.3.7. (Auswirkung von Messfehlern)

Es gilt:

$$\max_{x \in [a, b]} |p(x) - \tilde{p}(x)| \leq \Lambda_n \max_{i=0, \dots, n} |y_i - \tilde{y}_i|, \quad (2.3.18)$$

wobei die Lebesgue-Konstante Λ_n die bestmögliche Konstante in dieser Ungleichung ist.

Theorem 2.3.8. (Fehler)

Sei $f : [a, b] \rightarrow \mathbb{R}$ und p das Interpolationspolynom zu f . Seien $x_0, \dots, x_n$ die Stützstellen, dann gilt:

$$\|f(x) - p(x)\|_\infty = \max_{x \in [a, b]} |f(x) - p(x)| \leq (1 + \Lambda_n) \min_{q \in \mathcal{P}_n} \max_{x \in [a, b]} |f(x) - q(x)|.$$

Proof. Sei $q \in \mathcal{P}_n$ das Polynom bester Approximation, dann gilt:

$$|f(x) - p(x)| = |[f(x) - q(x)] + [q(x) - p(x)]| \leq |f(x) - q(x)| + |q(x) - p(x)|.$$

Seien $y_i = f(x_i)$ und $\tilde{y}_i = q(x_i)$. Wir wissen, dass p das Interpolationspolynom von f ist und dass q sein eigenes Interpolationspolynom ist. Dann:

$$\max_{x \in [a, b]} |p(x) - q(x)| \leq \Lambda_n \max_{i=0, 1, \dots, n} |f(x_i) - q(x_i)| \leq \Lambda_n \max_{x \in [a, b]} |f(x) - q(x)|.$$

Die letzten beiden Ungleichungen ergeben dann:

$$\max_{x \in [a, b]} |f(x) - p(x)| \leq \max_{x \in [a, b]} |f(x) - q(x)| + \Lambda_n \max_{x \in [a, b]} |f(x) - q(x)|.$$

□

Beispiel 2.3.9. (Lebesgue-Konstante)

Auf dem Intervall $[-1, 1]$ ist die Lebesgue-Konstante für eine äquidistante Verteilung der Stützstellen $x_j = -1 + \frac{2}{n}j$:

$$\Lambda_{10} \approx 40, \quad \Lambda_{20} \approx 30000, \quad \Lambda_{40} \approx 10^{10}.$$

Wir sehen also, dass schon für etwa 40 Stützstellen der Fehler sehr gross werden kann!

Bemerkung 2.3.10. Für äquidistant verteilte Stützstellen gilt:

$$\Lambda_n \approx \frac{2^{n+1}}{n \log n}.$$

Beispiel 2.3.11. (Runge-Funktion, Runge-Phänomen)

Als Beispiel betrachten wir die sogenannte Runge-Funktion auf dem Intervall $[-5, 5]$:

$$f(x) = \frac{1}{1+x^2}.$$

Zur Berechnung des Interpolationspolynoms wählen wir äquidistant verteilte Stützstellen. Das Interpolationspolynom ist in der Abbildung 2.2 dargestellt. Wir sehen, dass in den Randbereichen starke Oszillationen auftreten. Die Abbildung 2.3 zeigt, dass wir durch einen höheren Grad des Interpolationspolynoms sogar noch höhere Fehler erhalten. Wir sehen das Runge-Phänomen: weite Schwingungen am Enden des Intervalls, verursacht durch die grosse Lebesgue-Konstante.

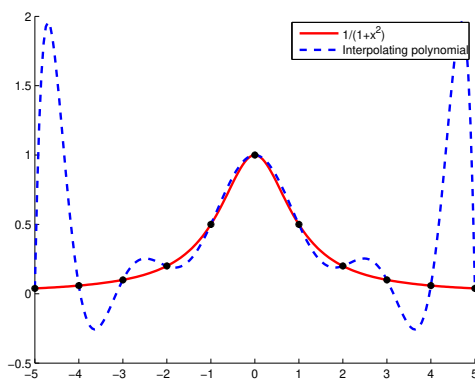
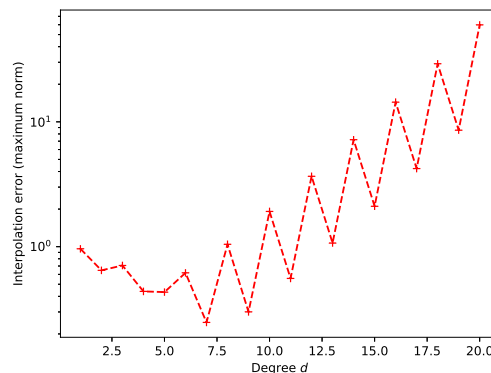


Figure 2.2: Interpolationspolynom


 Figure 2.3: $\|f - p\|_\infty$ auf $[-5, 5]$

Die Wahl der Stützstellen hat erheblichen Einfluss auf den Fehler. Aus dem obigen Beispiel lässt sich zudem eine einfache Grundregel für die Interpolation ableiten: **Verwende für eine Interpolation hohen Grades *niemals* äquidistant verteilte Stützstellen!**

Wie können wir genauere Interpolationen durch einen höheren Grad des Interpolationspolynoms erhalten?

Mögliche Ansatzpunkte könnten sein:

1. Das Intervall in kleinere Intervalle zu zerteilen und für jeden Teil ein eigenes Interpolationspolynom zu berechnen.
2. Die ideale Verteilung der Stützstellen zu wählen, die diesen Nachteil nicht aufweist.

Die erste Idee bringt uns zu Splines und finiten Elementen. Im nächsten Kapitel werden wir uns mit der zweiten Idee näher beschäftigen: Chebyshev-Knoten und -Abszissen, die mit den Chebyshev-Polynomen einhergehen.

2.4 Chebyshev-Interpolation

Hier möchten wir durch geschickte Wahl der Stützstellen den Interpolationsfehler verkleinern. Bisher haben wir immer äquidistant verteilte Stützstellen verwendet, in diesem Kapitel werden wir die Chebyshev-Knoten als die ideale Verteilung kennenlernen.

Definition 2.4.1. (Chebyshev-Polynome erster Art)

Die Chebyshev-Polynome (erster Art) sind:

$$T_n(x) = \cos(n \arccos(x)), \quad x \in [-1, 1]. \quad (2.4.19)$$

Definition 2.4.2. (Chebyshev-Polynome zweiter Art)

Die Chebyshev-Polynome zweiter Art sind:

$$U_n(x) = \frac{\sin((n+1) \arccos(x))}{\sin(\arccos(x))}, \quad x \in [-1, 1]. \quad (2.4.20)$$

Auf den ersten Blick ist es nicht sofort einzusehen, dass die $T_n(x)$ überhaupt Polynome sind! Im folgenden Satz wird die Aussage aber klar.

Theorem 2.4.3. (Eigenschaften)

Das n -te Chebyshev-Polynom ist ein Polynom vom Grad n und es gilt für $x \in [-1, 1]$:

1. $T_0(x) = 1, T_1(x) = x, T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x);$
2. $|T_n(x)| \leq 1;$
3. $T_n(\cos(\frac{k\pi}{n})) = (-1)^k$ für $k = 0, 1, \dots, n;$
4. $T_n(\cos(\frac{(2k+1)\pi}{2n})) = 0$ für $k = 0, 1, \dots, n-1.$

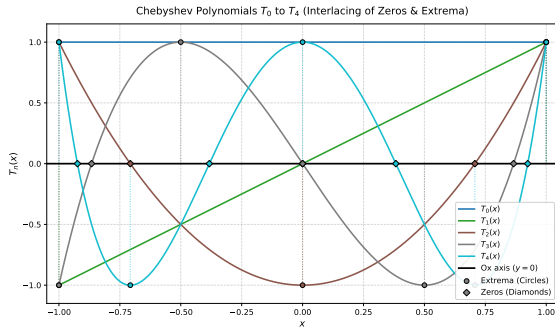
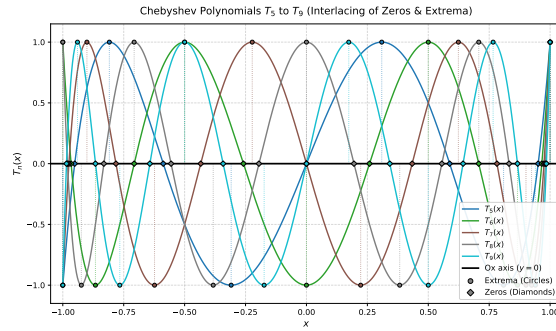
Proof. Es gilt:

$$\cos((n+1)t) = 2\cos(nt)\cos(t) - \cos((n-1)t).$$

Für $\cos(t) = x$ folgt die Rekursion.

Daraus folgt auch direkt, dass $T_n(x)$ ein Polynom ist, da $T_n(x)$ nach der Rekursion nur aus Polynomen besteht.

Die anderen Punkte ergeben sich durch einfaches Nachrechnen. □


 Figure 2.4: $T_0, \dots, T_4$

 Figure 2.5: $T_5, \dots, T_9$
Definition 2.4.4. (Chebyshev-Knoten)

Die $n + 1$ Chebyshev-Knoten $x_0, \dots, x_n$ im Intervall $[-1, 1]$ sind die Nullstellen von $T_{n+1}(x)$.

Bemerkung 2.4.5. Für ein beliebiges Intervall $[a, b]$ sind die Chebyshev-Knoten gegeben durch:

$$x_k = a + \frac{1}{2}(b - a) \left(\cos \left(\frac{2k + 1}{2(n + 1)} \pi \right) + 1 \right) \quad k = 0, \dots, n. \quad (2.4.21)$$

Definition 2.4.6. (Chebyshev-Abszissen)

Die $n - 1$ Chebyshev-Abszissen $x_0, \dots, x_{n-2}$ im Intervall $[-1, 1]$ sind die Extrema des Chebyshev-Polynoms $T_n(x)$. Gleichzeitig sind sie jedoch auch die Nullstellen von $U_{n-1}(x)$. Man nimmt je nach Kontext noch die Punkte -1 und 1 hinzu und erhält dann $n + 1$ Chebyshev-Abszissen.

Bemerkung 2.4.7. Für ein beliebiges Intervall $[a, b]$ sind die Chebyshev-Abszissen gegeben durch:

$$x_k = a + \frac{1}{2}(b - a) \left(\cos \left(\frac{k}{n} \pi \right) + 1 \right) \quad k = 0, \dots, n. \quad (2.4.22)$$

Schliesst man die Endpunkte a und b aus, so ist $k = 1, \dots, n - 1$.

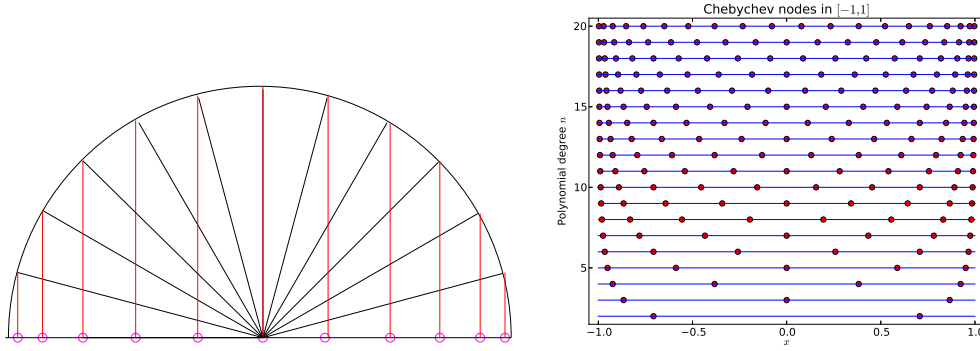
Bemerkung 2.4.8. In Abb. 2.6 sehen wir, dass die Chebyshev-Knoten zu den Rändern des Intervalls dichter werden.

Zusätzlich zu diesen Eigenschaften haben die Chebyshev-Polynome eine weitere wichtige Eigenschaft im Raum der Polynome:

Theorem 2.4.9. (Orthogonalität I)

Die Chebyshev-Polynome sind orthogonal bezüglich des Skalarprodukts

$$\langle f, g \rangle = \int_{-1}^1 f(x)g(x) \frac{1}{\sqrt{1 - x^2}} dx. \quad (2.4.23)$$


 Figure 2.6: Chebyshev-Knoten: links für $n = 11$, rechts für verschiedene n

Proof.

$$\begin{aligned}
 \langle T_k, T_j \rangle &= \int_{-1}^1 \frac{1}{\sqrt{1-x^2}} T_k(x) T_j(x) dx \\
 &\quad \text{Substitution } x = \cos(\phi) \Rightarrow dx = -\sin(\phi) d\phi = -\sqrt{1-x^2} d\phi \\
 &= - \int_0^\pi \cos(k\phi) \cos(j\phi) d\phi \\
 &= \int_0^\pi \frac{1}{2} [\cos((j-k)\phi) + \cos((j+k)\phi)] d\phi \\
 &= \begin{cases} 0 & \text{für } j \neq k \\ \frac{\pi}{2} & \text{für } j = k \neq 0 \\ \pi & \text{für } j = k = 0. \end{cases}
 \end{aligned}$$

□

Theorem 2.4.10. (Orthogonalität II)

Die Chebyshev-Polynome $T_0, T_1, \dots, T_n$ sind orthogonal bezüglich des diskreten Skalarprodukts im Raum der Polynome vom Grad $\leq n$:

$$(f, g) = \sum_{l=0}^n f(x_l) g(x_l), \quad (2.4.24)$$

wobei $(x_0, x_1, \dots, x_n)$ die Nullstellen von T_{n+1} sind.

Proof. Die $n+1$ Nullstellen von T_{n+1} sind:

$$x_\ell = \cos\left(\frac{2\ell+1}{2(n+1)}\pi\right), \quad \ell = 0, 1, \dots, n,$$

also

$$T_k(x_\ell) = \cos(k \arccos x_\ell) = \cos\left(k \left(\ell + \frac{1}{2}\right) \frac{\pi}{n+1}\right).$$

Mit der Abkürzung $\theta_\ell = \frac{2\ell+1}{2(n+1)}\pi$ schreiben wir den diskreten Skalarprodukt:

$$(T_k, T_j) = \sum_{\ell=0}^n T_k(x_\ell) T_j(x_\ell) = \sum_{\ell=0}^n \cos(k\theta_\ell) \cos(j\theta_\ell)$$

d.h.

$$(T_k, T_j) = \frac{1}{2} \sum_{\ell=0}^n (\cos((k+j)\theta_\ell) + \cos((k-j)\theta_\ell))$$

was in zwei ähnlich aussehende Summen zerfällt. Wir untersuchen nun die allgemeine Summe:

$$C(m) = \sum_{\ell=0}^n \cos(m\theta_\ell) = \sum_{\ell=0}^n \cos\left(m \frac{2\ell+1}{2(n+1)}\pi\right).$$

Unter Anwendung der Eulerschen Formel $\cos(\phi) = \operatorname{Re}(e^{i\phi})$ (wobei i die imaginäre Einheit bezeichnet) schreiben wir die Summe um:

$$C(m) = \operatorname{Re}\left(\sum_{\ell=0}^n e^{im \frac{2\ell+1}{2(n+1)}\pi}\right) = \operatorname{Re}\left(e^{i \frac{m}{2} \frac{\pi}{n+1}} \sum_{\ell=0}^n \left(e^{im \frac{\pi}{n+1}}\right)^\ell\right).$$

Dies ist eine einfache endliche geometrische Reihe mit dem Quotienten $q = e^{im \frac{\pi}{n+1}}$, es bedarf einer Diskussion nach q , d.h. nach dem Verhältnis zwischen m und $n+1$.

Falls m ein Vielfaches von $2(n+1)$ ist, d.h. $m = 2p(n+1)$ mit $p \in \mathbb{Z}$, dann haben wir $q = e^{i2p\pi} = 1$ und die Summe ist sehr einfach:

$$C(m) = \sum_{\ell=0}^n \cos(p(2\ell+1)\pi) = \sum_{\ell=0}^n (-1)^p = (n+1)(-1)^p$$

Falls m ist kein Vielfaches von $2(n+1)$, dann $q \neq 1$ und die Formel für die Partialsumme einer geometrischen Folge liefert:

$$\sum_{\ell=0}^n e^{im\theta_\ell} = e^{i \frac{m}{2} \frac{\pi}{n+1}} \cdot \frac{1 - e^{im\pi}}{1 - e^{im \frac{\pi}{n+1}}} = \frac{1 - e^{im\pi}}{e^{-i \frac{m}{2} \frac{\pi}{n+1}} - e^{i \frac{m}{2} \frac{\pi}{n+1}}} = \frac{1 - (-1)^m}{-2i \sin\left(\frac{m}{2} \frac{\pi}{n+1}\right)}.$$

Der Zähler $1 - (-1)^m$ ist reell und der Nenner rein imaginär. Der Gesamtausdruck ist somit rein imaginär (oder 0). Da $C(m)$ als Realteil definiert ist, folgt:

$$C(m) = 0 \quad \text{für alle } m \notin 2(n+1)\mathbb{Z}.$$

Das müssen wir jetzt auf die Formel in (T_k, T_j) anwenden. Da für die Polynomgrade $0 \leq k, j \leq n$ gilt, liegen die Summen- und Differenzindizes im Bereich:

- $0 \leq k+j \leq 2n < 2(n+1),$
- $-n \leq k-j \leq n.$

Daraus ergeben sich die drei Fälle:

Fall 1: $k \neq j$:

- $k - j \neq 0$ und $|k - j| \leq n < n + 1 \implies k - j \notin 2(n + 1)\mathbb{Z} \implies C(k - j) = 0$.
- $k + j > 0$ und $k + j \leq 2n < 2(n + 1) \implies k + j \notin 2(n + 1)\mathbb{Z} \implies C(k + j) = 0$.

$$(T_k, T_j) = \frac{1}{2}(0 + 0) = 0$$

Fall 2: $k = j \neq 0$:

- $k - j = 0 \implies C(0) = \sum_{\ell=0}^n \cos(0) = n + 1$.
- $k + j = 2k$. Da $0 < 2k \leq 2n < 2(n + 1)$, gilt $2k \notin 2(n + 1)\mathbb{Z} \implies C(2k) = 0$.

$$(T_k, T_k) = \frac{1}{2}(0 + n + 1) = \frac{n + 1}{2}$$

Fall 3: $k = j = 0$

- $k - j = 0 \implies C(0) = n + 1$.
- $k + j = 0 \implies C(0) = n + 1$.

$$(T_0, T_0) = \frac{1}{2}((n + 1) + (n + 1)) = n + 1$$

□

2.4.1 Fehlerbetrachtung

Wir wollen nun untersuchen, welche Vorteile wir durch die Verteilung der Stützstellen erhalten haben. Dazu betrachten wir nochmals die Fehlerabschätzung, die wir im letzten Kapitel kennengelernt haben. Der Fehler war gegeben durch:

$$f(x) - p(x) = (x - x_0) \cdots (x - x_n) \frac{f^{(n+1)}(\xi)}{(n + 1)!}.$$

Theorem 2.4.11. (Fehlerabschätzung)

Unter allen $(x_0, \dots, x_n)$ mit $x_i \in \mathbb{R}$ wird

$$\max_{x \in [-1, 1]} |(x - x_0) \cdots (x - x_n)| \quad \text{minimal für} \quad x_k = \cos \left(\frac{2k + 1}{2(n + 1)} \pi \right),$$

x_k sind die Nullstellen von T_{n+1} .

Bemerkung 2.4.12. Die Nullstellen der Chebyshev-Polynome T_n (d.h. Chebyshev-Knoten) sind also die bestmögliche Wahl für die Stützstellen. Die Extrema des Chebyshev-Polynoms T_{2n} (d.h. Chebyshev-Abszissen) beinhalten die Nullstellen von T_n . Zwischen je zwei nebeneinanderliegenden Chebyshev-Abszissen gibt es eine Nullstelle von T_{2n} . Aufgrund dieser Eigenschaften und der einfacheren Verbindung in der FFT werden eher die Chebyshev-Abszissen statt der Chebyshev-Knoten für praktische Berechnungen verwendet.

Theorem 2.4.13. (Lebesgue-Konstante)

Die Lebesgue-Konstante Λ_n für die Chebyshev-Interpolation ist gegeben durch

$$\Lambda_n \approx \frac{2}{\pi} \log n \quad \text{für } n \rightarrow \infty.$$

Beispiel 2.4.14. (Runge-Funktion)

Zur Veranschaulichung wollen wir die Runge-Funktion im Intervall $[-5, 5]$ nochmals betrachten. Zur Erinnerung, die Runge-Funktion war gegeben durch:

$$f(x) = \frac{1}{x^2 + 1}.$$

Diesmal wählen wir die Chebyshev-Knoten zur Interpolation. Zum Vergleich machen wir dieselbe Interpolation mit einer äquidistanten Verteilung der Stützstellen. Die beiden Interpolationspolynome sind in Abb. 2.7 bzw. Abb. 2.8 dargestellt.

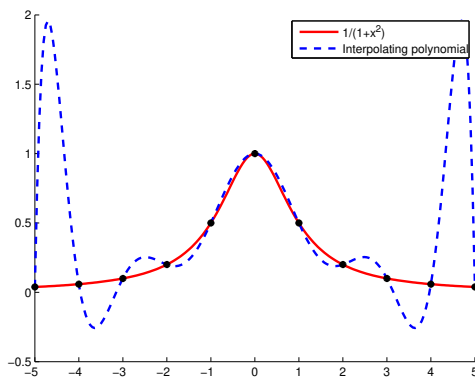


Figure 2.7: Äquidistante Stützstellen

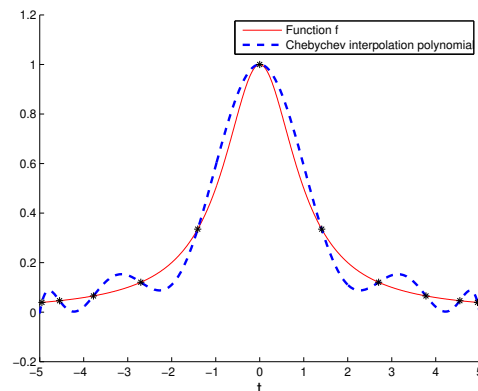


Figure 2.8: Chebyshev-Knoten

Der Interpolationsfehler der Chebyshev-Interpolation ist eindeutig geringer. Aber wie verhält sich der Fehler als Funktion des Grades der Interpolationsfunktion im Falle der Chebyshev-Interpolation?

Aus Abb. 2.10 wird deutlich, dass wir einen geringeren Fehler bei der Chebyshev-Interpolation erhalten, wenn wir den Grad des Interpolationspolynoms erhöhen. Die äquidistante Verteilung der Stützstellen hingegen verursacht immer stärker werdende Oszillationen an den Rändern des Intervalls und einen grösser werdenden Fehler (Abb. 2.9).

Code 2.4.15: Baryzentrische Interpolation in Chebyshev Abszissen

```
1 import numpy as np
2 from scipy.interpolate import BarycentricInterpolator
3 from plotsettings import *
```

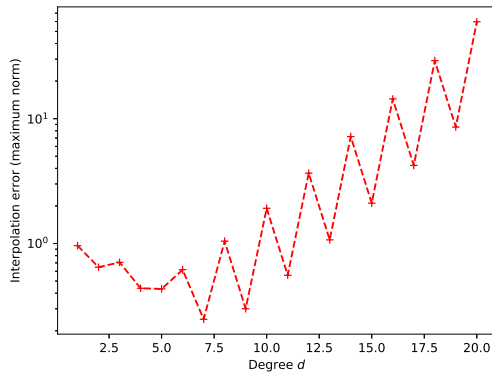



Figure 2.9: Äquidistante Stützstellen

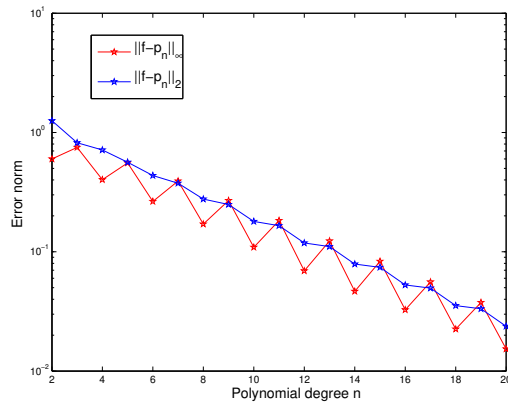


Figure 2.10: Chebyshev-Knoten

```

5  f = lambda x: 1./(1. + x**2) # Funktion
   a, b = -3., 3. # Interpolation domain I = [a, b]

7

9  n = 1+20 # number of interpolation points

   indofp = np.arange(n) # indices of the points
11  x_cheb = a+ 0.5*(b-a)*(1+np.cos(indofp*np.pi/(n - 1))) # Chebyshev abszissa on
   [a, b]
   w_cheb = (-1.)*indofp # Explicit formula for weights of Chebyshev abszissa
13  w_cheb[[0, -1]] *= 0.5
   y_cheb = f(x_cheb) # measurements
15  p_cheb = BarycentricInterpolator(x_cheb, y_cheb, wi=w_cheb)
   x_equi = np.linspace(a, b, endpoint=True, num=n)
17  y_equi = f(x_equi) # measurements
   p_equi = BarycentricInterpolator(x_equi, y_equi)
19  x_eval = np.linspace(a, b, endpoint=True, num=1+10**4) # where we evaluate

21  pcx = p_cheb(x_eval)
   pex = p_equi(x_eval)
23  fx = f(x_eval)

25  print('max absolute error in Chebyshev:', abs(pcx-fx).max() )
   print('max absolute error in equidistant:', abs(pex-fx).max() )

27

   # plot function and the two interpolants
29  fig, ax = plt.subplots()
   ax.plot(x_eval, fx, 'k-', label="Original Function")
31  ax.plot(x_cheb, y_cheb, 'ko', label='Measured Chebyshev')
   ax.plot(x_equi, y_equi, 'kx', label='Measured Equidistant')
33  ax.plot(x_eval, pcx, "b--", label="Chebshev Interpolant")
   ax.plot(x_eval, pex, "r-.", label="Equidistant Interpolant")
35  ax.set(xlabel="$x$", ylabel="$f(x)$", xlim=[a, b])
   ax.grid()
37  ax.legend()

39  plt.show()

```

testspbary.py

2.4.2 Interpolation und Auswertung

Neben den schon kennengelernten nützlichen Eigenschaften wollen wir uns nun mit der praktischen Anwendung der Chebyshev-Interpolation beschäftigen. Die erste Frage, die sich stellt, ist, wie wir die Darstellung in der Chebyshev Basis ausrechnen können, und ob das effizient und stabil ist.

Das Interpolationspolynom p zu f mit den Chebyshev-Knoten $x_0, x_1, \dots, x_n$, also den Nullstellen von T_{n+1} , ist gegeben durch:

$$p(x) = c_0 T_0(x) + c_1 T_1(x) + \dots + c_n T_n(x) \quad (2.4.25)$$

oder im linearen Raum der Polynome vom Grad kleiner gleich n :

$$p = c_0 T_0 + c_1 T_1 + \dots + c_n T_n.$$

Die Orthogonalität des diskreten Skalarprodukt hilft, wenn wir nun den Skalarprodukt mit einem beliebigen Element der Orthogonalbasis von Chebyshev Polynome bilden:

$$(T_k, p) = 0 + \dots + 0 + c_k \cdot (T_k, T_k) + 0 + \dots + 0.$$

Für $k \neq 0$ haben wir also $c_k = \frac{2}{n+1} (T_k, p)$ und $c_0 = \frac{1}{n+1} (T_0, p)$. Wir setzen diese in der Definition des diskreten Skalarprodukts und verwenden, dass p die unbekannte Funktion f an den Chebyshev Knoten x_ℓ interpoliert:

$$\begin{aligned} c_k &= \frac{2}{n+1} \sum_{l=0}^n \underbrace{f(x_\ell)}_{y_\ell} T_k(x_\ell) \text{ für } k = 1, 2, \dots, n, \\ c_0 &= \frac{1}{n+1} \sum_{l=0}^n \underbrace{f(x_\ell)}_{y_\ell} \cdot 1 \text{ für } k = 0. \end{aligned}$$

Somit haben wir bewiesen:

Theorem 2.4.16. (Interpolationspolynom)

Das Interpolationspolynom p zu f mit den Chebyshev-Knoten $x_0, x_1, \dots, x_n$, also den Nullstellen von T_{n+1} , ist gegeben durch:

$$p(x) = c_0 + c_1 T_1(x) + \dots + c_n T_n(x), \quad (2.4.26)$$

wobei für die c_k gilt:

$$\begin{aligned} c_k &= \frac{2}{n+1} \sum_{l=0}^n f \left(\underbrace{\cos \left(\frac{2l+1}{n+1} \frac{\pi}{2} \right)}_{=x_\ell(\text{Knoten})} \right) \underbrace{\cos \left(k \frac{2l+1}{n+1} \frac{\pi}{2} \right)}_{T_k(x_\ell)} \text{ für } k = 1, 2, \dots, n, \\ c_0 &= \frac{1}{n+1} \sum_{l=0}^n f \left(\underbrace{\cos \left(\frac{2l+1}{n+1} \frac{\pi}{2} \right)}_{=x_\ell(\text{Knoten})} \right) \cos \left(k \frac{2l+1}{n+1} \frac{\pi}{2} \right) \text{ für } k = 0. \end{aligned}$$

d.h

$$\begin{aligned} c_k &= \frac{2}{n+1} \sum_{l=0}^n \underbrace{f(x_l)}_{y_l} T_k(x_l) \text{ für } k = 1, 2, \dots, n, \\ c_0 &= \frac{1}{n+1} \sum_{l=0}^n \underbrace{f(x_l)}_{y_l} \cdot 1 \text{ für } k = 0. \end{aligned}$$

Bemerkung 2.4.17. Für grosse n ($n \geq 15$) berechnet man die c_k mithilfe der Schnellen Fourier-Transformation (FFT). Diese wird im nächsten Kapitel behandelt. Damit ergibt sich für den Aufwand zur Berechnung des Interpolationspolynoms:

Direkte Berechnung der c_k	$\mathcal{O}((n+1)^2)$ Operationen
Dividierte Differenzen	$\mathcal{O}(\frac{n(n+1)}{2})$ Operationen (zum Vergleich)
c_k mittels FFT	$\mathcal{O}(n \log n)$ Operationen.

Auch die Auswertung des Interpolationspolynoms an beliebigen Punkten gestaltet sich durch den sogenannten Clenshaw-Algorithmus sehr einfach:

Theorem 2.4.18. (Clenshaw-Algorithmus)

Das Interpolationspolynom sei, wie oben, gegeben durch

$$p(x) = c_0 T_0(x) + c_1 T_1(x) + \dots + c_n T_n(x).$$

Seien $d_{n+2} = d_{n+1} = 0$. Setze nun:

$$d_k = c_k + (2x)d_{k+1} - d_{k+2} \quad \text{für } k = n, n-1, \dots, 0. \quad (2.4.27)$$

Dann gilt: $p(x) = d_0 - xd_1 = \frac{1}{2}(d_0 - d_2)$.

Man kann also $p(x)$ über eine Rückwärtsrekursion berechnen.

Proof. Wir wissen bereits, dass $T_{k+1}(x) = 2xT_k(x) - T_{k-1}$ gilt, also folgt mit mehrmaligem Einsetzen:

$$\begin{aligned} p(x) &= c_0 T_0(x) + c_1 T_1(x) + \dots + \underbrace{c_n}_{= d_n} T_n(x) \\ &= c_0 + c_1 T_1(x) + \dots + (c_{n-2} - d_n) T_{n-2}(x) + \underbrace{(c_{n-1} + 2xd_n)}_{= d_{n-1}} T_{n-1}(x) \\ &= c_0 T_0(x) + c_1 T_1(x) + \dots + (c_{n-3} - d_{n-1}) T_{n-3}(x) + \underbrace{(c_{n-2} + 2xd_{n-1} - d_n)}_{= d_{n-2}} T_{n-2}(x) \\ &= \dots = c_0 T_0(x) + xd_1 - d_2 = \underbrace{(c_0 + 2xd_1 - d_2)}_{= d_0} - xd_1 \\ &= d_0 - xd_1. \end{aligned}$$

□

Bemerkung 2.4.19. Man kann beweisen, dass der Clenshaw-Algorithmus sehr stabil ist. Rekursionen sind nicht immer stabil, z.B.

$$x_{n+1} = 10x_n - 9$$

ist instabil. Wenn $x_0 = 1$, dann ist $x_n = 1$ für alle n . Bei einem Mess- oder Rundungsfehler am Anfang der Rekursion

$$x_0 = 1 + \varepsilon$$

erhalten wir nach n Schritten

$$x_n = 1 + 10^n \varepsilon.$$

Mit anderen Worten, der relative Fehler wird in jedem Schritt mit 10 multipliziert, sodass er nach n Schritten 10^n mal grösser ist.

Wir wollen nun das Konvergenzverhalten der polynomiellen Interpolation an den Chebyshev-Abszissen für einige Testfunktionen untersuchen. Wir wählen $[a, b] = [0, 2]$, $f(x) = \sin(\pi x)$ als Beispiel einer glatten, periodischen Funktion, $f(x) = |\sin(\pi x)|^3$ als Beispiel einer zweimal stetig differenzierbaren, periodischen Funktion, $f(x) = 1/(1 + 16(x - 1)^2)$ als Beispiel einer glatten, nicht-periodischen Funktion (Runge-Funktion),

$$f(x) = \begin{cases} \frac{1}{10(x-1)^2+1} & \text{für } x \leq 1 \\ (5x-4)e^{5-5x} & \text{sonst} \end{cases} \quad (2.4.28)$$

als Beispiel einer Funktion, die nur einmal stetig differenzierbar ist, und

$$f(x) = \begin{cases} \sin\left(\frac{\pi x}{2}\right) & \text{für } x \leq 1 \\ 2.5x + \left(-3.125 - \frac{\pi^2}{8}\right)(x-1)^2 - 2.5 + 12.1824939607035e^{-2.5x} & \text{sonst} \end{cases} \quad (2.4.29)$$

als Beispiel einer Funktion, die zweimal stetig differenzierbar ist. Die Konvergenzkurven befinden sich in Abbildung 2.4.20: links, berechnet nur mit dem baryzentrischen Interpolator an den Chebyshev-Abszissen mit den analytischen baryzentrischen Gewichten, und rechts mit `numpy.polynomial.chebyshev.chebfit`, das ein spezielles `polyfit` für die Chebyshev-Polynome ist, nicht gerade stabil, jedoch in diesen Fällen genügend. Die Berechnung der Ableitungen der Approximationen wurde mit der Funktion `derivative` des `numpy.polynomial`-Pakets berechnet, was deutlich stabiler als `numpy.gradient` ist, wie wir im Beispiel 1.1.11 gesehen haben. Wir beobachten: Für glatte Funktionen liefert die Chebyshev-Interpolation eine exponentielle Konvergenz (Fehler $E(n) = \mathcal{O}(q^n)$ mit $0 < q < 1$) und für weniger glatte Funktionen nur eine algebraische Konvergenz (Fehler $E(n) = \mathcal{O}(\frac{1}{n^p})$).

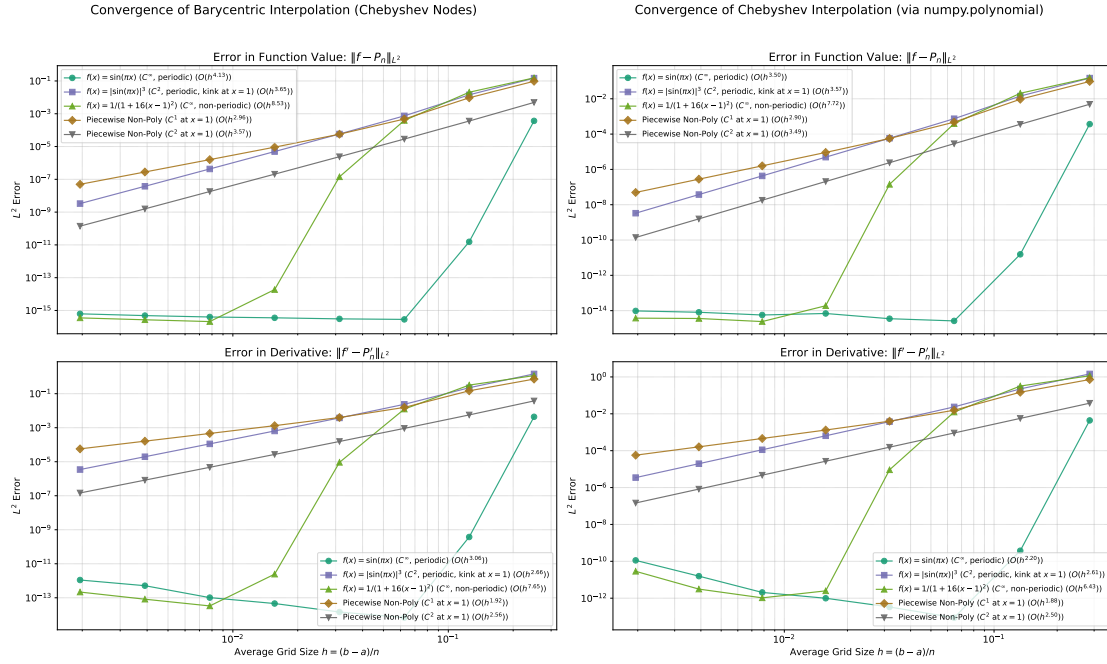


Abb. 2.4.20. Konvergenz für einige Testfunktionen

Showing a Fourier Transform to a physics student generally produces the same reaction as showing a crucifix to Count Dracula.

J.F. James - "A Student's Guide to Fourier Transforms"

Chapter 3

Trigonometrische Interpolation

3.1 Trigonometrische Polynome und Fourier-Reihen

Wir haben bereits gesehen, dass man Funktionen durch Polynome und die Taylor-Entwicklung approximieren kann. Allerdings gibt es auch weitere und bessere Möglichkeiten.

Der französische Mathematiker und Physiker Fourier¹ machte 1822 mit seinem Werk "Théorie analytique de la chaleur" die Idee bekannt, Funktionen durch trigonometrische Polynome zu approximieren. Er benutzte (bereits 1804–1807) Reihen von trigonometrischen Funktionen, um die Wärmeleitungsgleichung zu lösen. Weiterhin behauptete er, dass es für alle Funktionen eine solche Entwicklung nach trigonometrischen Funktionen gibt. Die Behauptung, jede Funktion durch eine periodische und unendlich glatte Funktion zu approximieren, war damals dermassen verblüffend, dass Lagrange, Laplace, Biot und Poisson dieser anfangs sehr kritisch gegenüberstanden. Die Mathematik war damals einfach noch nicht so weit entwickelt, um diese Ideen mit der Eleganz von heute zu erklären. Tatsache ist, dass wichtige wissenschaftliche und technologische Fortschritte aus dieser einfachen Idee entstanden sind. Hier werden wir einen eher intuitiven Zugang wählen und auch nur Teile dieser gigantischen Theorie betrachten. Mathematische Sätze, die wir verwenden werden, sind zum Beispiel in *Königsberger, Analysis I, Kapitel 16* und *Amann, Escher, Analysis II, Kapitel VI.7* zu finden.

Definition 3.1.1. Wir definieren als *trigonometrisches Polynom vom Grad $\leq m$* die Funktion

$$p_m(t) := t \mapsto \sum_{j=-m}^m \gamma_j e^{2\pi i j t} \text{ wobei } \gamma_j \in \mathbb{C} \text{ und } t \in \mathbb{R}. \quad (3.1.1)$$

Bemerkung 3.1.2. Die Funktion $p_m : \mathbb{R} \rightarrow \mathbb{C}$ ist periodisch mit Periode 1. Wenn $\gamma_{-j} = \overline{\gamma_j}$ für alle j , dann nimmt p_m nur reelle Werte an und man kann p_m auch folgendermassen darstellen:

$$p_m(t) = \frac{a_0}{2} + \sum_{j=1}^m (a_j \cos(2\pi j t) + b_j \sin(2\pi j t)) \quad (3.1.2)$$

¹Jean Baptiste Joseph Fourier (1768–1830)

mit $a_0 = 2\gamma_0$, $a_j = 2\operatorname{Re}(\gamma_j)$ und $b_j = -2\operatorname{Im}(\gamma_j)$.

Trigonometrische Polynome kommen in einer einführenden Physikvorlesung unter dem Namen “Überlagerung von Schwingungen” vor.

Definition 3.1.3. Wir definieren die L^2 -Funktionen auf dem Intervall $(0, 1)$ als

$$L^2(0, 1) := \{f : (0, 1) \rightarrow \mathbb{C} : \|f\|_{L^2(0,1)} < \infty\}. \quad (3.1.3)$$

Die L^2 -Norm auf $(0, 1)$ wird durch das Skalarprodukt

$$\langle g, f \rangle_{L^2(0,1)} := \int_0^1 \overline{g(x)} f(x) dx \quad (3.1.4)$$

über $\|f\|_{L^2(0,1)} = \sqrt{\langle f, f \rangle_{L^2(0,1)}}$ induziert.

Bemerkung 3.1.4. $L^2(a, b)$ lässt sich analog definieren mit

$\langle g, f \rangle_{L^2(a,b)} := \int_a^b \overline{g(x)} f(x) dx = (b-a) \int_0^1 \overline{g(a+(b-a)t)} f(a+(b-a)t) dt$. In den Anwendungen kommt oft das Intervall $[-\frac{T}{2}, \frac{T}{2}]$ vor, sodass die Integrale in der Form $\frac{1}{T} \int_{-\frac{T}{2}}^{\frac{T}{2}} \cdot dt$ vorkommen und man statt $\exp(2\pi i j t)$ mit $\exp(i \frac{2\pi j}{T} t)$ arbeitet. Einfachheitshalber arbeiten wir hier auf $[0, 1]$.

Bemerkung 3.1.5. $L^2(0, 1)$ ist ein linearer Raum mit dem Skalarprodukt $\langle f, g \rangle_{L^2(0,1)}$. Er enthält auch Funktionen, die nur stückweise stetig sind, und auch solche, die deutlich unglatter sind.

Bemerkung 3.1.6. Die Funktionen $\varphi_k(x) = \exp(2\pi i k x)$ sind orthogonal bezüglich des $L^2(0, 1)$ -Skalarprodukts

$$\langle \varphi_k, \varphi_j \rangle_{L^2(0,1)} = \begin{cases} 0, & \text{wenn } k \neq j \\ 1, & \text{wenn } k = j \end{cases}$$

und bilden somit eine Basis für den Unterraum der trigonometrischen Polynome.

Definition 3.1.7. Eine Funktion f ist der L^2 -Grenzwert einer Folge von Funktionen $f_n \in L^2(0, 1)$, wenn für $n \rightarrow \infty$ gilt, dass $\|f - f_n\|_{L^2(0,1)} \rightarrow 0$.

Bemerkung 3.1.8. Dieser Konvergenzbegriff ergänzt entsprechende Begriffe, die bereits aus der Analysis I bekannt sind. Es handelt sich um eine schwächere Form der Konvergenz als die punktweise und die gleichmässige Konvergenz, da es hier nur um eine Art Mittelwert geht.

Theorem 3.1.9. Jede Funktion $f \in L^2(0, 1)$ ist der L^2 -Grenzwert ihrer Fourier-Reihe:

$$f(t) = \sum_{k=-\infty}^{\infty} \widehat{f}(k) e^{2\pi i k t}, \quad (3.1.5)$$

wobei die Fourier-Koeffizienten $\widehat{f}(k)$ als

$$\widehat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt, \quad k \in \mathbb{Z}. \quad (3.1.6)$$

definiert sind. Dabei gilt die Parseval'sche Gleichung:

$$\sum_{k=-\infty}^{\infty} |\hat{f}(k)|^2 = \|f\|_{L^2(0,1)}^2. \quad (3.1.7)$$

Bemerkung 3.1.10. Wir können dieses Theorem auch kürzer formulieren: Die Funktionen $\varphi_k(t) = \exp(2\pi i k t)$ bilden eine **vollständige Orthonormalbasis** in $L^2(0, 1)$.

Bemerkung 3.1.11. Dieser Satz rechtfertigt, dass man sich eine Funktion entweder im Zeitbereich $t \mapsto f(t)$ oder im Frequenzbereich $k \mapsto \hat{f}(k)$ vorstellen kann. Es ist unter Umständen sinnvoll, zwischen den Darstellungen zu wechseln, wenn man in einer Darstellung gewisse Eigenschaften sehr einfach ablesen kann. Dafür gibt es unzählige Anwendungen: NMR, IR-Spektroskopie, etc.

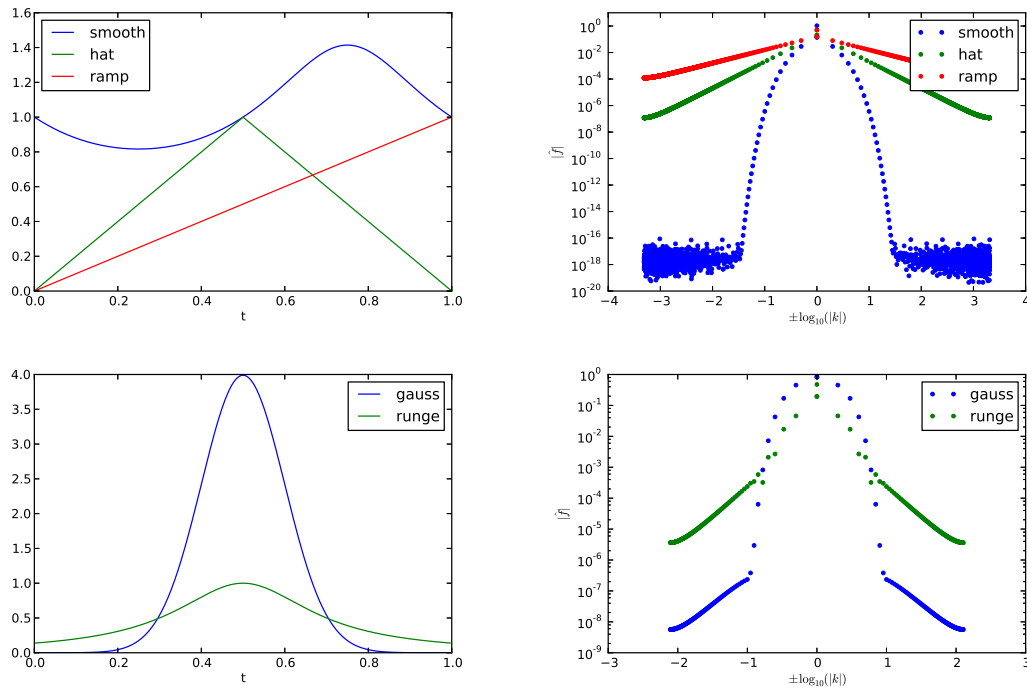


Abb. 3.1.12. Darstellungen im Zeitbereich (links) und im Frequenzbereich (rechts)

Bemerkung 3.1.13. Für reellwertige Funktionen können wir diese Ergebnisse auch noch anders darstellen:

$$f(t) = \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos(2\pi k t) + b_k \sin(2\pi k t))$$

$$a_k = 2 \int_0^1 f(t) \cos(2\pi k t) dt$$

$$b_k = 2 \int_0^1 f(t) \sin(2\pi k t) dt.$$

Bemerkung 3.1.14. Die Parseval'sche Gleichung besagt zum einen, dass die Fourier-Koeffizienten für $L^2(0, 1)$ schneller als $\frac{1}{\sqrt{k}}$ abklingen, da die rechte Seite einen definierten Wert hat und die linke Reihe damit konvergieren muss. Zum anderen besagt sie, dass die L^2 -Norm der Funktion (die manche als "Energie" bezeichnen) statt aus einem Integral aus einer Summe (der Betragsquadrate der Fourier-Koeffizienten) berechnet werden kann. In der Analysis wird dies verwendet, um gewisse Reihen zu summieren.

Leiten wir (3.1.5) nach t ab, erhalten wir

$$f'(t) = \sum_{k=-\infty}^{\infty} 2\pi i k \hat{f}(k) e^{2\pi i k t}.$$

Theorem 3.1.15. Seien f und f' integrierbar auf $(0, 1)$, dann gilt $\widehat{f'}(k) = 2\pi i k \hat{f}(k)$ für $k \in \mathbb{Z}$.

Wenn die Operationen erlaubt sind, dann gilt:

$$\widehat{f^{(n)}}(k) = (2\pi i k)^n \hat{f}(k) \text{ und } \|f^{(n)}\|_{L^2}^2 = (2\pi)^{2n} \sum_{k=-\infty}^{\infty} k^{2n} |\hat{f}(k)|^2,$$

was eine direkte Verbindung zwischen der Glattheit einer Funktion und dem Abfall ihrer Fourier-Koeffizienten zeigt: Je glatter die Funktion, desto schneller ist der Abfall der Fourier-Koeffizienten. Diese Tatsache wird verwendet, um die Glattheit über das Konvergenzverhalten der Fourier-Reihen zu definieren (was wir hier nicht tun).

Theorem 3.1.16. Wenn $\int_0^1 |f^{(n)}(t)| dt < \infty$, dann ist $\hat{f}(k) = \mathcal{O}(k^{-n})$.

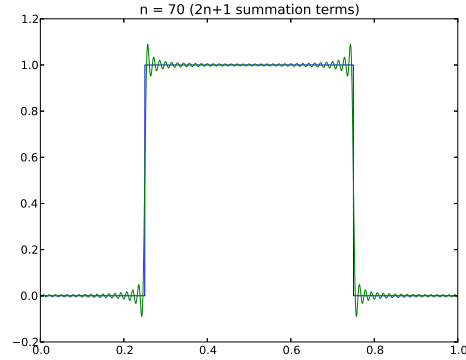
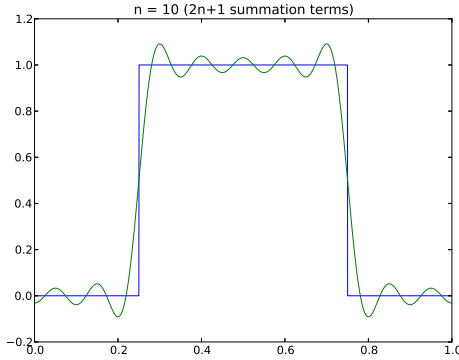
Interessant ist auch, was passiert, wenn die Funktion eben nicht so glatt ist. Was macht die Fourier-Reihe an einer Sprungstelle der Funktion? Dort kommt es zu Überschwüngen, die immer näher an die Sprünge kommen, aber nicht kleiner werden, wenn wir mehr Terme in der Fourier-Reihe aufsummieren: Das ist das Gibbs-Phänomen. In der Sprache der Konvergenz: Wir haben L^2 -Konvergenz, aber keine punktweise Konvergenz an der Sprungstelle.

Beispiel 3.1.17. Die Fourier-Reihe der charakteristischen Funktion des Intervalls $[a, b] \subset]0, 1[$ lässt sich analytisch berechnen:

$$b - a + \frac{1}{\pi} \sum_{k \neq 0} e^{-ikc} \frac{\sin kd}{k} e^{i2\pi kt}, \quad t \in [0, 1],$$

mit $c = \pi(a + b)$ und $d = \pi(b - a)$. Die Fourier-Koeffizienten fallen sehr langsam ab, sodass wir eine sehr langsame Konvergenz der Fourier-Reihe erwarten. In den Bildern beobachten wir auch das Gibbs-Phänomen; wir haben nur L^2 -Konvergenz

der Fourier-Reihe.



Bemerkung 3.1.18. Man kann beweisen: Wenn f stetig und stückweise stetig differenzierbar ist, dann konvergiert ihre Fourier-Reihe punktweise und gleichmässig. Wenn f von beschränkter Variation ist und der linke und rechte Grenzwert mit dem Mittelwert übereinstimmen, dann konvergiert sie punktweise.

Bemerkung 3.1.19. Normalerweise können wir die Fourier-Koeffizienten nicht analytisch ausrechnen; stattdessen verwenden wir die Funktionswerte an den Knoten $t_\ell = \frac{\ell}{N}$ für $\ell = 0, 1, \dots, N-1$ und die Trapezregel, um die exakten Fourier-Koeffizienten numerisch zu approximieren:

$$\hat{f}(k) \approx \frac{1}{N} \sum_{\ell=0}^{N-1} f(t_\ell) e^{-2\pi i k t_\ell} =: \hat{f}_N(k), \quad (3.1.8)$$

wobei wir hier $f(1) = f(0)$ vorausgesetzt haben (Periodizität).

Bevor wir zur diskreten Fourier-Transformation kommen, lassen Sie uns die wichtigsten Ideen dieses Abschnitts zusammenfassen.

Das Ziel unserer Untersuchungen sind (eventuell unglatte) Funktionen:

$$f : (0, 1) \longrightarrow \mathbb{C}, \text{ mit } \int_0^1 |f(t)|^2 dt < \infty.$$

Für solche Funktionen definieren wir die (kontinuierlichen) Fourier-Koeffizienten:

$$\hat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt, \text{ für } k = 0, \pm 1, \pm 2, \dots$$

Dann bilden wir die Summen

$$p_m(t) = \sum_{k=-m}^m \hat{f}(k) e^{2\pi i k t}$$

und wir lassen $m \rightarrow \infty$. Fourier besagt:

$$\lim_{m \rightarrow \infty} \|p_m(\cdot) - f(\cdot)\|_2 = \lim_{m \rightarrow \infty} \sqrt{\int_0^1 |p_m(x) - f(x)|^2 dx} = 0$$

und Parseval besagt:

$$\|f\|_2^2 = \int_0^1 |f(x)|^2 dx = \lim_{m \rightarrow \infty} \sum_{k=-m}^m |\hat{f}(k)|^2 = \|\hat{f}\|_{\ell^2}^2,$$

wobei $\hat{f} = \left(\hat{f}(k) \right)_{k \in \mathbb{Z}}$ als eine Folge mit entsprechender Norm interpretiert wird.

Die Konvergenz der Reihe $\sum_{k=-\infty}^{\infty} |\hat{f}(k)|^2$ steht in direktem Zusammenhang mit dem Abfall ihrer Glieder $|\hat{f}(k)|$. Es stellt sich heraus: Je glatter f , desto schneller ist der Abfall der Fourier-Koeffizienten $|\hat{f}(k)|$, also desto schneller ist die Konvergenz der Fourier-Reihe. Für die Approximation bedeutet das, dass sich glatte Funktionen sehr gut durch kurze Fourier-Summen approximieren lassen, während unglatte Funktionen einfach längere Fourier-Summen benötigen. Anwendungen dafür finden sich in der Kompression, Rauschunterdrückung, beim Autofokus etc. Z.B. ist Rauschen unglatt und entspricht somit grossen Fourier-Koeffizienten $|\hat{f}(k)|$ für grosse $|k|$.

Im Folgenden werden wir N als gerade annehmen und die trigonometrischen Polynome

$$e^{2\pi i(-\frac{N}{2})t}, e^{2\pi i(-\frac{N}{2}+1)t}, \dots, e^{2\pi i 0t}, e^{2\pi i 1t}, \dots, e^{2\pi i(\frac{N}{2}-1)t}$$

als eine orthogonale Basis des von diesen aufgespannten Raumes $\mathcal{T}_N$ der Dimension N betrachten. Die orthogonale Projektion von f auf diesen Raum ergibt eine trigonometrische Approximation:

$$f(t) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{2\pi i k t} \approx \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \hat{f}(k) e^{2\pi i k t} =: p_{N-1}(t).$$

Weiter approximieren wir jeden Fourier-Koeffizienten mit der Trapezregel (3.1.8):

$$\hat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt \approx \frac{1}{N} \sum_{\ell=0}^{N-1} f(t_\ell) e^{-2\pi i k t_\ell} =: \hat{f}_N(k)$$

mit den Punkten $t_\ell = \frac{\ell}{N}$, für $\ell = 0, 1, \dots, N-1$. Schreiben wir dasselbe mit dem Indexwechsel $\ell = j$:

$$\hat{f}(k) \approx \hat{f}_N(k) = \frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) e^{-2\pi i k \frac{j}{N}}. \quad (3.1.9)$$

So haben wir eine berechenbare weitere Approximation erreicht:

$$f(t) \approx \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \hat{f}_N(k) e^{2\pi i k t} =: \tilde{p}_{N-1}(t). \quad (3.1.10)$$

3.2 Diskrete Fouriertransformation

3.2.1 Motivation: von der Approximation zur Interpolation

Definition 3.2.1. Die N -te Einheitswurzel ist $\omega_N := \exp(\frac{-2\pi i}{N})$.

Bemerkung 3.2.2. Wir finden durch Nachrechnen und Ausnutzen der Eigenschaften der komplexen Exponentialfunktion:

$$\omega_N^{k+jN} = \omega_N^k, \text{ für alle } j, k \in \mathbb{Z}, \quad (3.2.11)$$

$$\omega_N^{t+kN} = \omega_N^t, \text{ für alle } k \in \mathbb{Z} \text{ und } t \in \mathbb{R}, \quad (3.2.12)$$

$$\omega_N^N = 1, \quad (3.2.13)$$

$$\omega_N^{N/2} = -1, \quad (3.2.14)$$

$$\sum_{k=0}^{N-1} \omega_N^{kj} = \begin{cases} N, & \text{wenn } j \equiv 0 \pmod{N} \\ 0, & \text{sonst.} \end{cases} \quad (3.2.15)$$

In der Tat erinnern wir uns:

$$\exp(z) = e^{\operatorname{Re}(z)} (\cos(\operatorname{Im}(z)) + i \sin(\operatorname{Im}(z))),$$

und

$$\omega_N = \cos \frac{2\pi}{N} - i \sin \frac{2\pi}{N}.$$

Die Potenzfunktion ist definiert als

$$z^\alpha = \exp(\alpha \ln z),$$

also

$$\omega_N^\alpha = \exp(\alpha \ln \omega_N) = \exp\left(-\frac{2\pi i}{N} \alpha\right) = \cos \frac{2\pi \alpha}{N} - i \sin \frac{2\pi \alpha}{N}.$$

Wir erhalten dann

$$\omega_N^{t+kN} = \exp\left(-\frac{2\pi i}{N}(t+kN)\right) = \cos \frac{2\pi t + 2\pi kN}{N} - i \sin \frac{2\pi t + 2\pi kN}{N} = \omega_N^t.$$

Ausserdem können wir für $j \not\equiv 0 \pmod{N}$ schreiben:

$$\sum_{k=0}^{N-1} \omega_N^{kj} = \sum_{k=0}^{N-1} (\omega_N^j)^k = \frac{1 - (\omega_N^j)^N}{1 - \omega_N^j} = \frac{1 - (\omega_N^N)^j}{1 - \omega_N^j} = \frac{1 - 1}{1 - \omega_N^j} = 0,$$

was die letzte Behauptung beweist.

Wir erinnern uns an die Formel (3.1.8): Die Trapezregel für die Approximation eines Fourier-Koeffizienten ergab:

$$\hat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt \approx \frac{1}{N} \sum_{\ell=0}^{N-1} f(t_\ell) e^{-2\pi i k t_\ell} =: \hat{f}_N(k), \quad (3.2.16)$$

mit $t_\ell = \frac{\ell}{N}$, d.h.

$$\hat{f}(k) \approx \hat{f}_N(k) = \frac{1}{N} \sum_{\ell=0}^{N-1} f\left(\frac{\ell}{N}\right) e^{-2\pi i k \frac{\ell}{N}} = \frac{1}{N} \sum_{\ell=0}^{N-1} f\left(\frac{\ell}{N}\right) \omega_N^{k\ell}. \quad (3.2.17)$$

Fixieren wir nun ℓ und werten die Fourier-Approximation p_{N-1} aus (3.1.10) mit den approximierten Fourier-Koeffizienten $\hat{f}_N(k)$ aus (3.1.9) an der Stelle t_ℓ aus:

$$\begin{aligned} p_{N-1}(t_\ell) &= p_{N-1}\left(\frac{\ell}{N}\right) = \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \hat{f}_N(k) e^{2\pi i k \frac{\ell}{N}} \\ &= \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \left(\frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) \omega_N^{kj} \right) \omega_N^{k\ell}. \end{aligned}$$

Vertauschen wir die beiden Summen:

$$p_{N-1}(t_\ell) = \frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \omega_N^{k(j-\ell)}.$$

Die Gleichung (3.2.15) garantiert, dass die innere (letzte) Summe N ist, falls $j = \ell$, und andernfalls 0 ist. Somit bleibt von der ersten Summe nur der Term für $j = \ell$:

$$p_{N-1}(t_\ell) = \frac{1}{N} \left(0 + \dots + 0 + f\left(\frac{\ell}{N}\right) N + 0 + \dots + 0 \right) = f(t_\ell).$$

Diese letzte Rechnung beweist das Verblüffende: die **Fourier-Approximation** zusammen mit der zusammengesetzten **Trapezregel auf äquidistanten Punkten** $(\frac{0}{N}, \frac{1}{N}, \dots, \frac{N-1}{N})$ liefert ein trigonometrisches Polynom, das die Funktionswerte $(f(\frac{0}{N}), f(\frac{1}{N}), \dots, f(\frac{N-1}{N}))$ in diesen Punkten exakt **interpoliert**. Anders gesagt: das trigonometrische Polynom

$$p_{N-1}(t) = \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \hat{f}_N(k) e^{2\pi i k t}$$

erfüllt die Interpolationsbedingungen

$$p_{N-1}(t_\ell) = f(t_\ell), \text{ für } t_\ell = \frac{\ell}{N} \text{ mit } \ell = 0, 1, \dots, N-1.$$

3.2.2 Konstruktion

Das motiviert nun den folgenden Ansatz: Sei $w_N = \exp(\frac{2\pi i}{N}) = \overline{\omega_N}$ und

$$\mathbf{v}_k = \begin{bmatrix} w_N^{0 \cdot k} \\ w_N^{1 \cdot k} \\ w_N^{2 \cdot k} \\ \vdots \\ w_N^{(N-1) \cdot k} \end{bmatrix} \in \mathbb{C}^N.$$

Definition 3.2.3. Wir definieren als *trigonometrische Basis* von $\mathbb{C}^N$ die Vektoren

$$\mathbf{v}_0 = \begin{bmatrix} w_N^{0 \cdot 0} \\ w_N^{1 \cdot 0} \\ w_N^{2 \cdot 0} \\ \vdots \\ w_N^{(N-1) \cdot 0} \end{bmatrix}, \dots, \mathbf{v}_k = \begin{bmatrix} w_N^{0 \cdot k} \\ w_N^{1 \cdot k} \\ w_N^{2 \cdot k} \\ \vdots \\ w_N^{(N-1) \cdot k} \end{bmatrix}, \dots, \mathbf{v}_{N-1} = \begin{bmatrix} w_N^{0 \cdot (N-1)} \\ w_N^{1 \cdot (N-1)} \\ w_N^{2 \cdot (N-1)} \\ \vdots \\ w_N^{(N-1) \cdot (N-1)} \end{bmatrix}. \quad (3.2.18)$$

Die vorherigen Eigenschaften zeigen, dass es sich um paarweise orthogonale Vektoren in $\mathbb{C}^N$ handelt. Setzen wir diese Vektoren in einer Matrix zusammen:

$$\mathbf{V} = [\mathbf{v}_0 \dots \mathbf{v}_{N-1}],$$

so berechnen wir leicht

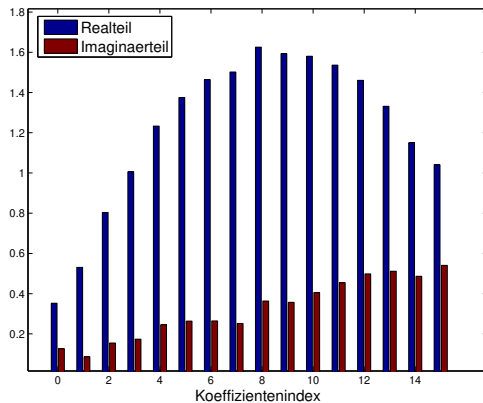
$$\mathbf{V}^H \mathbf{V} = N \mathbf{I}_N,$$

wiederum aufgrund von (3.2.15), was die Orthogonalität der trigonometrischen Basis sichert.

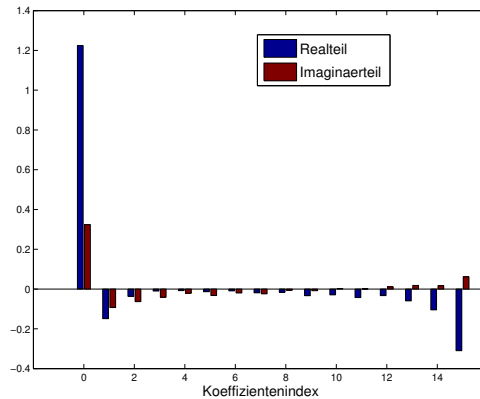
$\mathbf{V}$ ist eine **symmetrische, nicht hermitesche** $N \times N$ -Matrix mit komplexen Einträgen. Die Basiswechselmatrix von der Standardbasis zur trigonometrischen Basis ist $\mathbf{V}$, denn $\mathbf{v}_k = \mathbf{V} \mathbf{e}_{k+1}$. Für den Koordinatenwechsel haben wir also

$$\mathbf{y} = \sum_{k=0}^{N-1} y_k \mathbf{e}_{k+1} = \sum_{k=0}^{N-1} z_k \mathbf{v}_k = \sum_{k=0}^{N-1} z_k \begin{bmatrix} w_N^{0 \cdot k} \\ w_N^{1 \cdot k} \\ w_N^{2 \cdot k} \\ \vdots \\ w_N^{(N-1) \cdot k} \end{bmatrix}.$$

d.h. $\mathbf{y} = \mathbf{V} \mathbf{z}$ und $\mathbf{z} = \mathbf{V}^{-1} \mathbf{y} = \frac{1}{N} \mathbf{V}^H \mathbf{y} = \frac{1}{N} \mathbf{F}_N \mathbf{y}$ mit $\mathbf{F}_N := \mathbf{V}^H$. Hier ist $\mathbf{y}$ die Koordinatendarstellung in der Standardbasis und $\mathbf{z}$ die Koordinatendarstellung in der trigonometrischen Basis desselben Vektors. So können wir einen Vektor aus der trigonometrischen Basis durch Matrixmultiplikation mit $\mathbf{V}$ in der Standardbasis darstellen. Für die umgekehrte Darstellung müssen wir mit der Inversen von $\mathbf{V}$ multiplizieren. Eventuell ist die Darstellung eines Vektors in einer der beiden Basen wesentlich einfacher; wir können dann in dieser Basis Rechnungen vornehmen und am Ende transformieren wir zurück.



Vektor in der Standardbasis



Vektor in der trigonometrischen Basis

Der ℓ -te Eintrag in $\mathbf{y}$ ist

$$\begin{aligned} y_\ell &= \sum_{k=0}^{N-1} z_k w_N^{\ell \cdot k} = \sum_{k=0}^{N/2-1} z_k w_N^{\ell \cdot k} + \sum_{k=N/2}^{N-1} z_k w_N^{\ell \cdot k} = \sum_{k=0}^{N/2-1} z_k w_N^{\ell \cdot k} + \sum_{j=-N/2}^{-1} z_{N+j} w_N^{\ell(N+j)} \\ &= \sum_{k=-N/2}^{N/2-1} \gamma_k \exp\left(\frac{2\pi i}{N} \ell k\right), \end{aligned}$$

was einem Glied der Fourier-Reihe, ausgewertet am Punkt $\frac{\ell}{N} \in [0, 1[$, entspricht, wobei die (diskreten) Fourier-Koeffizienten durch eine Umsortierung der Koeffizienten in der trigonometrischen Basis gegeben sind:

$$\gamma_k = \begin{cases} z_k, & \text{für } 0 \leq k \leq \frac{N}{2} - 1 \\ z_{k+N}, & \text{für } -\frac{N}{2} \leq k < 0 \end{cases}. \quad (3.2.19)$$

Definition 3.2.4. Wir definieren die *Fouriermatrix* als die Matrix, die die Koeffizienten einer Darstellung in der Standardbasis auf die Koeffizienten der Darstellung in der trigonometrischen Basis abbildet (bis auf den Vorfaktor $1/N$); anders gesagt: die Fouriermatrix bildet jedes Element der Standardbasis auf ein Element der trigonometrischen Basis ab (bis auf den Faktor $1/N$):

$$\mathbf{F}_N = \mathbf{V}^H = \begin{bmatrix} \omega_N^0 & \omega_N^0 & \dots & \omega_N^0 \\ \omega_N^0 & \omega_N^1 & \dots & \omega_N^{N-1} \\ \omega_N^0 & \omega_N^2 & \dots & \omega_N^{2(N-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega_N^0 & \omega_N^{N-1} & \dots & \omega_N^{(N-1)^2} \end{bmatrix} = \left(\omega_N^{jk} \right)_{j,k=0}^{N-1} \in \mathbb{C}^{N \times N}. \quad (3.2.20)$$

Definition 3.2.5. Sei $\mathcal{F}_N : \mathbb{C}^N \rightarrow \mathbb{C}^N$ die lineare Abbildung $\mathcal{F}_N(\mathbf{y}) := \mathbf{F}_N \mathbf{y}$ für $\mathbf{y} \in \mathbb{C}^N$. Wir nennen diese Abbildung *Diskrete Fourier-Transformation*, kurz DFT. Äquivalent dazu ist die Formulierung, dass für $\mathbf{c} = \mathcal{F}_N(\mathbf{y})$ gilt:

$$c_k = \sum_{j=0}^{N-1} y_j \omega_N^{kj}, \quad k = 0, 1, \dots, N-1. \quad (3.2.21)$$

Theorem 3.2.6. Die skalierte Fouriermatrix $\frac{1}{\sqrt{N}} \mathbf{F}_N$ ist unitär:

$$\mathbf{F}_N^{-1} = \frac{1}{N} \mathbf{F}_N^H = \frac{1}{N} \overline{\mathbf{F}_N}.$$

Bemerkung 3.2.7. Wenn man zweimal die normierte Fourier-Transformation anwendet, erhält man die Ursprungsdaten in umgekehrter Reihenfolge bis auf das erste Element:

$$\frac{1}{N} \mathbf{F}_N \mathbf{F}_N \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-2} \\ y_{N-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_{N-1} \\ \vdots \\ y_2 \\ y_1 \end{bmatrix}$$

(Zeitumkehrung modulo N), also ist $\frac{1}{N^2}\mathbf{F}_N^4 = \mathbf{I}$, und somit liegen die Eigenwerte von $\frac{1}{\sqrt{N}}\mathbf{F}_N$ in der Menge $\{1, -1, i, -i\}$.

Bemerkung 3.2.8 (Zeit/Orts- und Frequenzbereich). In unserer Konstruktion enthält der Vektor $\mathbf{y}$ die Messwerte $f(t_\ell)$ zu den Zeit-/Ortspunkten $t_\ell = \ell/N$, während der Vektor $\mathbf{c}$ die (approximierten) Fourier-Koeffizienten enthält; entsprechend sehen wir $\mathbf{y}$ als eine Abtastung von f im Zeit-/Ortsbereich und $\mathbf{c}$ (eventuell nach `fftshift`) als eine Darstellung im Frequenzbereich. Wenn wir uns nur auf den diskreten Fall konzentrieren, dann gibt es nur den Vektor $\mathbf{y}$ mit Koeffizienten in der Standardbasis, während $\mathbf{c}$ seine Darstellung in der trigonometrischen Basis ist. Die Matrix $\mathbf{F}_N^{-1}$ lässt uns die Perspektive aber auch ändern: Wir können $\mathbf{c}$ als Koeffizienten in der Standardbasis betrachten und $\mathbf{y}$ als eine Darstellung in einer orthonormalen Basis, die nun mit den Potenzen von w_N gebaut ist. Es gibt also eine schöne Symmetrie zwischen den beiden Perspektiven: Zeit-/Ortsbereich und Frequenzbereich. Abhängig von der getroffenen Konvention kann man die (diskrete) Fourier-Transformation mit w_N oder mit ω_N definieren. Wir haben unsere Wahl getroffen, damit wir konsistent mit der Implementierung in `numpy` und `FFTW` sind.

3.2.3 `fftshift` und erste Anwendungen

Bemerkung 3.2.9. `numpy` stellt die Funktionen

$$\mathbf{c} = \text{fft}(\mathbf{y}) \text{ und } \mathbf{y} = \text{ifft}(\mathbf{c})$$

für

$$\mathbf{c} = \mathbf{F}_N \mathbf{y} \text{ und } \mathbf{y} = \frac{1}{N} \mathbf{F}_N^H \mathbf{c}$$

zur Verfügung.

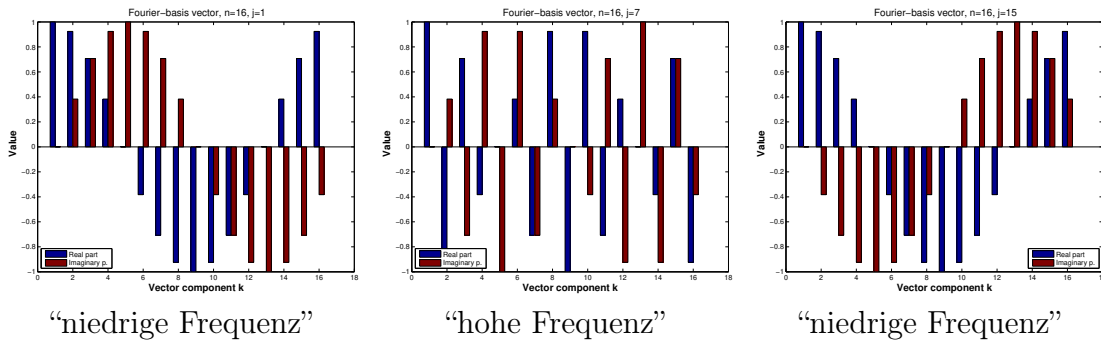


Abb. 3.2.10. Einige Vektoren der Fourier-Basis ($N = 16$)

In Abbildung 3.2.10 sind einige Vektoren der Fourier-Basis ($N = 16$) dargestellt. Die Real- und Imaginärteile oszillieren für die ersten ($j = 0, 1, 2$) und die letzten ($j = 13, 14, 15$) Vektoren der Basis weniger und für die mittleren (um $j = 8$) viel stärker. In diesem Sinne entsprechen die hohen Frequenzen der Mitte der Vektorindizes im Kontext der diskreten Fourier-Transformation, im Kontrast zur Formel (3.1.1), die ein trigonometrisches Polynom definiert.

Bemerkung 3.2.11. Wenn wir eine Darstellung analog zu (3.1.1) wollen, dann müssen wir die Basisfunktionen und die Koeffizienten umsortieren (zum Beispiel mit `fft.fftshift`). Hier ist das Argument und die Umsortierung für eine gerade Anzahl $N = 2n$ von Basisfunktionen. Für $k = 0, 1, 2, \dots, N - 1$ gilt:

$$\begin{aligned}
 y_k &= \sum_{j=-n}^{n-1} \gamma_j \omega_N^{jk} = \sum_{j=-n}^{-1} \gamma_j \omega_N^{jk} + \sum_{j=0}^{n-1} \gamma_j \omega_N^{jk} \\
 &= \sum_{p=n}^{2n-1} \gamma_{p-N} \omega_N^{(p-N)k} + \sum_{j=0}^{n-1} \gamma_j \omega_N^{jk} \\
 &= \sum_{p=n}^{N-1} \gamma_{p-N} \omega_N^{pk} + \sum_{j=0}^{n-1} \gamma_j \omega_N^{jk} = \sum_{j=0}^{N-1} z_j \omega_N^{jk}, \tag{3.2.22}
 \end{aligned}$$

wobei

$$z_j = \begin{cases} \gamma_j, & \text{für } 0 \leq j \leq n-1 = \frac{N}{2} - 1 \\ \gamma_{j-N}, & \text{für } n \leq j \leq 2n-1 = N-1 \end{cases}. \tag{3.2.23}$$

Somit erhalten wir $\mathbf{z} = \mathbf{F}_N \mathbf{y}$ und

$$\gamma_k = \begin{cases} z_k, & \text{für } 0 \leq k \leq n-1 = \frac{N}{2} - 1 \\ z_{k+N}, & \text{für } -\frac{N}{2} = -n \leq k < 0 \end{cases}.$$

Ausgehend von den Werten

$$y_k = f\left(\frac{k}{N}\right), \text{ für } k = 0, 1, \dots, N-1,$$

berechnen wir

$$\mathbf{z} = \frac{1}{N} \mathbf{F}_N \mathbf{y}$$

mittels `fft.fft` und erhalten laut (3.1.8) eine Approximation von f mittels eines trigonometrischen Polynoms

$$f(x) \approx \sum_{j=0}^{\frac{N}{2}-1} z_j e^{2\pi i j x} + \sum_{j=\frac{N}{2}}^{N-1} z_j e^{2\pi i (j-N)x},$$

oder

$$f(x) \approx \sum_{k=0}^{\frac{N}{2}-1} z_k e^{2\pi i k x} + \sum_{k=-\frac{N}{2}}^{-1} z_{k+N} e^{2\pi i k x},$$

oder mit $\zeta = \text{fft.fftshift}(\mathbf{z})$:

$$f(x) \approx \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \zeta_k e^{2\pi i k x}.$$

Beispiel 3.2.12. Wenn wir `fft.fft` auf einen Vektor y der Länge 8 anwenden, erhalten wir den Vektor $c = (c_0, \dots, c_7)$. `fft.fftshift` stellt diese Einträge in der Reihenfolge $(c_4, c_5, c_6, c_7, c_0, c_1, c_2, c_3)$ dar, was den Koeffizienten $(\gamma_{-4}, \gamma_{-3}, \gamma_{-2}, \gamma_{-1}, \gamma_0, \gamma_1, \gamma_2, \gamma_3)$ entspricht.

Bemerkung 3.2.13. Diese Approximation kann für die numerische Berechnung der L^2 -Norm der Funktion f oder ihrer Ableitungen verwendet werden, z.B.

$$\|f\|_{L^2}^2 \approx \left\| \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \zeta_k e^{2\pi i k t} \right\|_{L^2}^2 = \sum_{k=-N/2}^{N/2-1} |\zeta_k|^2 = \|\mathbf{z}\|_{\ell^2}^2,$$

oder auch für das numerische Lösen von Differentialgleichungen. In der Tat ist die Berechnung der Ableitung einer trigonometrischen Approximation sehr einfach:

$$f'(t) \approx \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} (2\pi i k) \zeta_k e^{2\pi i k t}$$

und alle Algorithmen, die für f geschrieben wurden, sind auch für f' direkt verwendbar. Man muss nur aufpassen, dass die Faktoren $(2\pi i k)$ die entsprechenden Fourier-Koeffizienten multiplizieren: ohne Shift sind diese in der Reihenfolge $k = 0, 1, \dots, \frac{N}{2}-1, -\frac{N}{2}, -\frac{N}{2}+1, \dots, -1$. Die Berechnung der Ableitung im Zeit- oder Ortsbereich reduziert sich also auf eine punktweise Multiplikation im Frequenzbereich. Darum ist das Lösen von Differentialgleichungen mit konstanten Koeffizienten mittels Fourier-Transformation eine sehr einfache Aufgabe.

Beispiel 3.2.14 (Frequenzanalyse mittels DFT). Nun nehmen wir ein periodisches Signal mit zwei Frequenzen (1 und 7); dazu addieren wir eine zufällige Störung. Wir transformieren dann dieses gestörte Signal in die trigonometrischen Basis und schauen uns das sogenannte “power spectrum” (die spektrale Leistungsdichte / das Energiespektrum) an. Die ursprünglichen zwei Frequenzen sind in Abbildung 3.2.16 eindeutig sichtbar. In diesem Beispiel sind die Periodizität und die Abtastpunkte des Signals allerdings wichtig, sie müssen gut zueinander passen. Wenn das der Fall ist, kann man die Frequenzen und die Perioden daraus ablesen. In Abbildung 3.2.16 unten haben wir das Signal mit den 7 kleinen “Perioden” dargestellt, was der gefundenen Frequenz 7 entspricht.

Im Frequenzbereich schauen wir uns auch nur die erste Hälfte der Koeffizienten an. Probieren Sie auch `bar(t-N/2,fft.fftshift(p))` und vergleichen Sie mit dem einfachen `bar(t,p[:N/2])`! Die Plots in Abbildung 3.1.12 kann man entweder mit den exakt (und eventuell mühsam) berechneten Fourier-Koeffizienten oder mit den mit der diskreten Fourier-Transformation approximierten Koeffizienten erhalten; hierzu verwendet man alle Koeffizienten und `fft.fftshift` ist unerlässlich.

Code 3.2.15: Frequenzanalyse: Das Energiespektrum / die spektrale Leistungsdichte / das power spectrum verrät die wichtigsten Frequenzen und Perioden im Signal

```

import numpy as np
import matplotlib.pyplot as plt

lev = 7
N = 2**lev
t = np.linspace(0,1,N, endpoint=False)

fqw = [1., 7.]
x = np.zeros(N)
for fq in fqw:
    x += np.sin(fq*2*np.pi*t)
y = x + np.random.randn(N) #distortion
c = np.fft.fft(y)
p = abs(c)**2; p /= N

k = len(fqw)
sorted_indices = np.argsort(p[:N//2])
dfq = sorted_indices[-1:-k-1:-1]
print('Dominant frequencies are:', dfq)
print('Dominant periods are:', 1/dfq)

plt.plot(t,y,'-+')
my = abs(y).max()
color = iter(plt.cm.rainbow(np.linspace(0, 1, k)))
for z in dfq:
    colz = next(color)
    for k in range(z):
        plt.axvline(x=(k+1)*1/z, color = colz, linestyle='--')

plt.grid(True); plt.savefig('fftfreqanal01.eps'); plt.show()
plt.bar(np.r_[N//2:],p[:N//2])
plt.grid(True); plt.savefig('fftfreqanal02.eps'); plt.show()

```

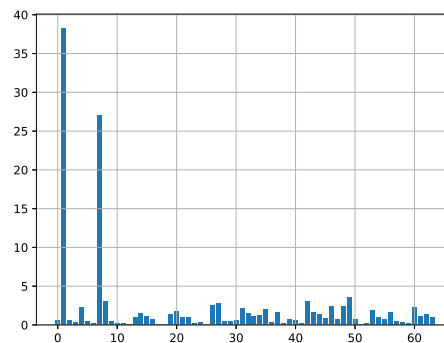
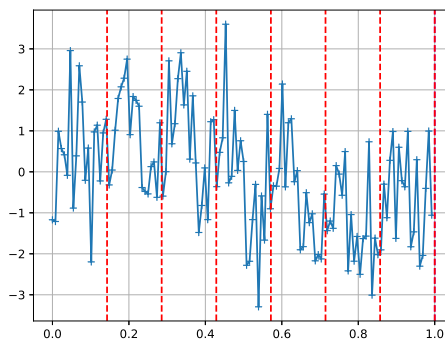


Abb. 3.2.16. Frequenzanalyse mittels DFT

Beispiel 3.2.17 (Herzschlag aus EKG-Daten ablesen). Dieselbe Idee können wir bei einem aus einem EKG gemessenen Signal anwenden, um den Herzschlag zu bestimmen. Hier wird die Frequenz 0 ignoriert, da diese einfach dem Grundsignal des Geräts entspricht (eine Konstante). Im oberen Teil von Abbildung 3.2.18 werden

Werte aus einer Simulation eines EKGs gezeigt: Das reine Signal hatte 85 Schläge pro Minute und wurde mit einer normalverteilten Störung additiv verändert. Es liegen 256 äquidistante Messungen pro Sekunde (256 Hz ist ein typischer Wert für EKGs) für 10 Sekunden vor, also insgesamt $N = 2560$ Messungen. Das Ziel ist, die wichtigste Frequenz zu bestimmen. Im unteren Teil der Abbildung werden die Beträge der ersten Hälfte der Fourier-Koeffizienten gezeigt. Die Frequenzen in Hz erhalten wir mit `fft.fftfreq(N, d=1/256)`. Der grösste Koeffizient ist in diesen Daten eindeutig; die entsprechende Frequenz ist 1.4 Hz, was einem Puls von $1.4 \cdot 60 = 84$ Schlägen pro Minute entspricht.

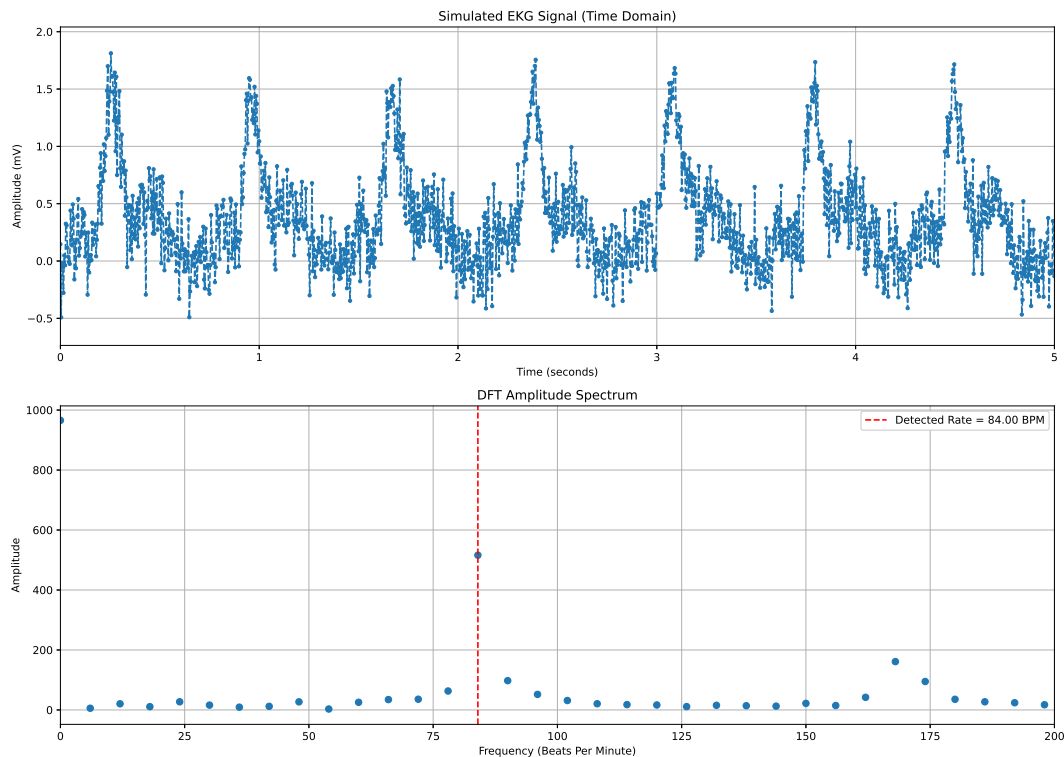


Abb. 3.2.18. Frequenzanalyse mittels DFT verrät den Herzschlag.

Beispiel 3.2.19 (Materialbestimmung aus einem MRI). In einem MRI werden hohe Frequenzen verwendet, in diesem Beispiel 5000 Hz für 0.5 Sekunden, also wir haben $N = 2500$ Messungen. Das simulierte Signal beinhaltet zwei wichtige Frequenzen (Wasser mit 1200 Hz und Fett mit 1050 Hz), klingt sehr schnell ab und ist mit normalverteilten Daten gestört. Alle Frequenzen in Hz erhalten wir mit `fft.fftfreq(N, d=1/5000)`. Wir sehen zwei grösste Koeffizienten; die entsprechenden Frequenzen sind 1050 Hz mit einer Periode von 0.952 ms und 1200 Hz mit einer Periode von 0.833 ms. (Die Perioden in Sekunden sind die Kehrwerte der Frequenzen in Hz.)

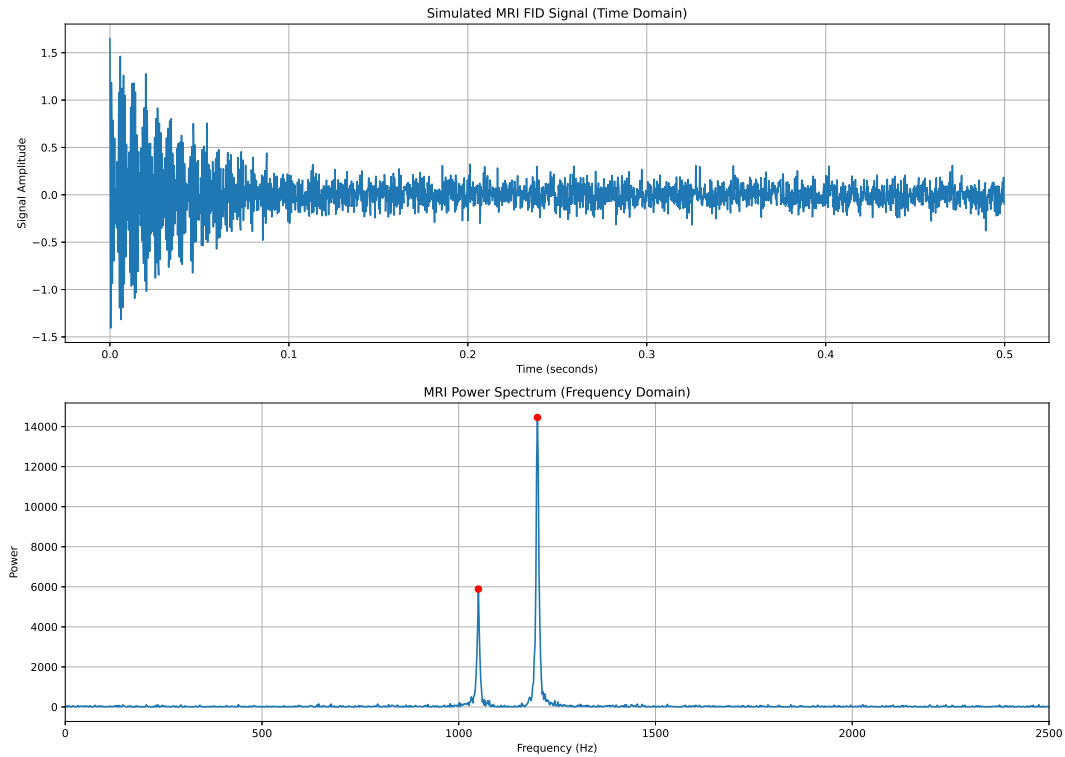


Abb. 3.2.20. Blick in eine Zellprobe dank Frequenzanalyse mittels DFT.

Beispiel 3.2.21 (Eigenfrequenzen aus Messungen). Die Vibration eines Motors wurde mit 2000 Hz (Messpunkte pro Sekunde) für 2 Sekunden aufgenommen ($N = 4000$ Messungen). Das Energiespektrum / die spektrale Leistungsdichte / das power spectrum verrät die Existenz von 7 Eigenfrequenzen:

Frequenz (Hz)	Periode (ms)
51.50	19.417
75.00	13.333
109.00	9.174
191.00	5.236
242.00	4.132
303.50	3.295
415.00	2.410

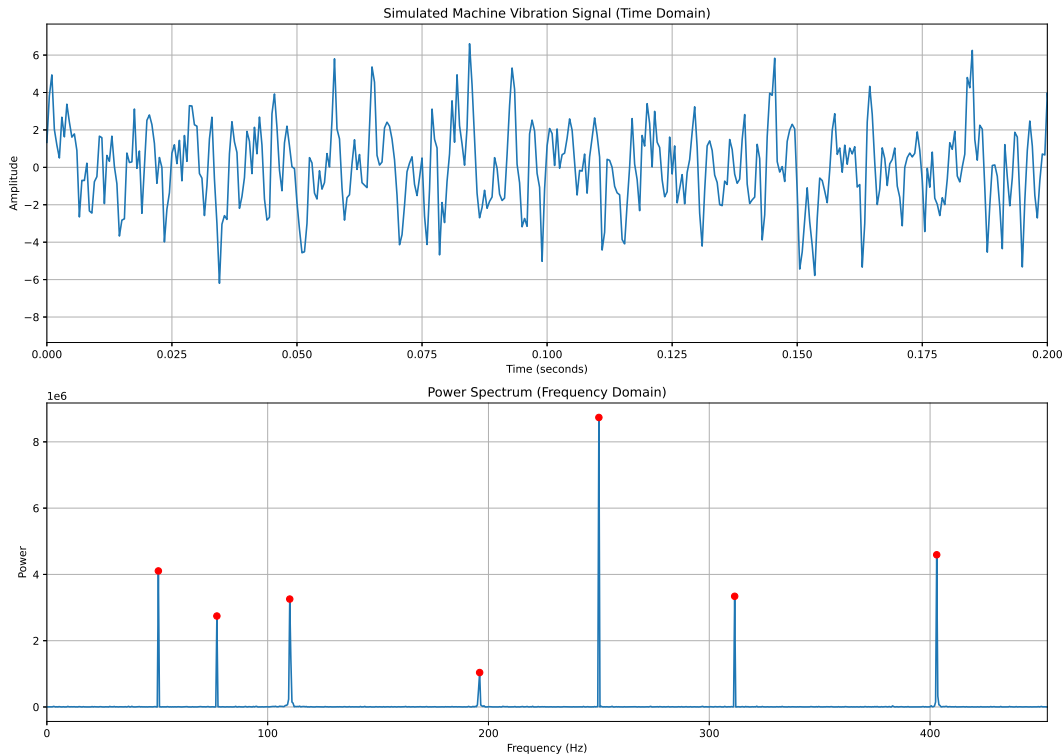
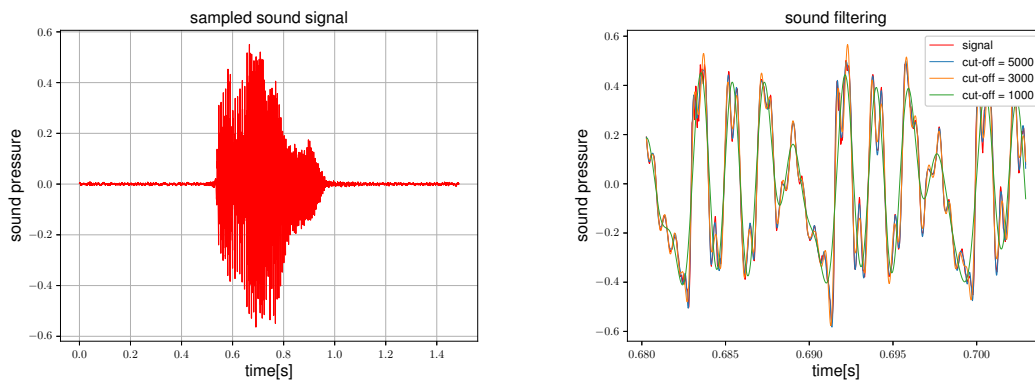


Abb. 3.2.22. Frequenzanalyse eines Motors zur Bestimmung der Eigenfrequenzen mittels DFT.

Beispiel 3.2.23 (Kompression). Ein Signal kann man komprimieren, indem man nur die Fourier-Koeffizienten behält, die wesentlich im Energiespektrum (Power Spectrum) erscheinen. In manchen Kontexten macht es Sinn, entweder nur die niedrigen oder nur die hohen Frequenzen zu behalten. Unten zeigen wir ein Sprachsignal (Luftdruck am Mikrofon) mit 70 Tausend Messungen und einem Ausschnitt, der rekonstruierte Signale aus nur den ersten 5, 3 bzw. 1 Tausend Fourier-Koeffizienten zeigt. Natürlich gibt es auch bessere Filter als diesen “Low-Pass-Filter”.



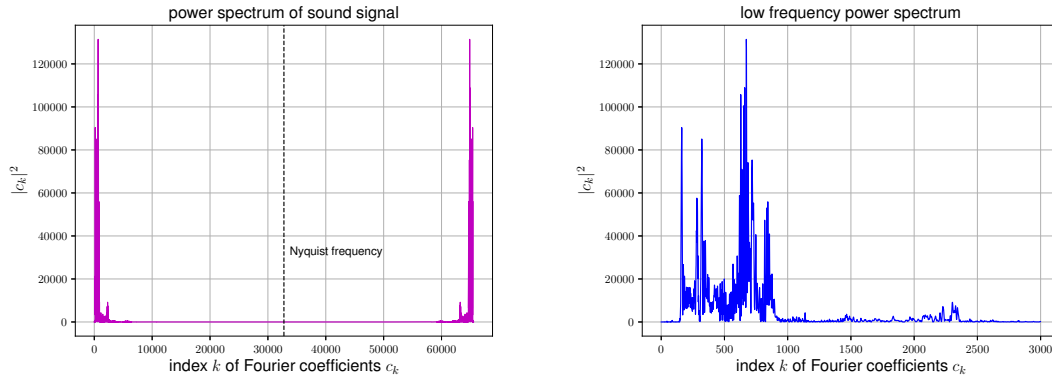


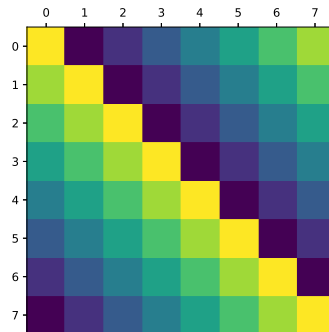
Abb. 3.2.24. Sprachkompression mit DFT.

3.2.4 Ein Blick aus der Perspektive der Linearen Algebra

Definition 3.2.25. Eine Matrix mit folgender Struktur:

$$\mathbf{C} = \text{circ}[\mathbf{c}] = \begin{bmatrix} c_0 & c_{N-1} & c_{N-2} & \cdots & c_3 & c_2 & c_1 \\ c_1 & c_0 & c_{N-1} & \ddots & \ddots & c_3 & c_2 \\ c_2 & \ddots & \ddots & \ddots & \ddots & \ddots & c_3 \\ \vdots & & & & & & \vdots \\ c_{N-3} & \ddots & & \ddots & \ddots & \ddots & c_{N-2} \\ c_{N-2} & c_{N-3} & \ddots & & \ddots & c_0 & c_{N-1} \\ c_{N-1} & c_{N-2} & c_{N-3} & \cdots & \cdots & c_{N-1} & c_0 \end{bmatrix}$$

heisst zirkulante Matrix (oder einfach Zirkulant). Hier wurde die erste Spalte als Startvektor $\mathbf{c}$ verwendet.



Notieren wir $\mathbf{c} = [c_0 \ c_1 \ \dots \ c_{N-1}]^T \in \mathbb{R}^N$, so ist die zirkulante Matrix mit dieser ersten Spalte wohldefiniert. Für $\mathbf{c} = \mathbf{e}_2 = [0 \ 1 \ 0 \ \dots \ 0]^T$ haben wir eine spezielle zirku-

lante Matrix, die folgende Shift-Matrix mit N Spalten:

$$\mathbf{S}_N = \begin{bmatrix} 0 & 0 & \dots & & 1 \\ 1 & 0 & \ddots & & 0 \\ \vdots & \ddots & \ddots & & \vdots \\ & & & \ddots & \\ 0 & 0 & & & 0 & 0 \\ 0 & 0 & \dots & & 1 & 0 \end{bmatrix}.$$

Diese Shift-Matrix $\mathbf{S}_N$ verschiebt die Einträge vorwärts periodisch und $\mathbf{S}_N^T$ verschiebt sie zurück:

$$\mathbf{S}_N \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{N-2} \\ x_{N-1} \end{bmatrix} = \begin{bmatrix} x_{N-1} \\ x_0 \\ x_1 \\ \vdots \\ x_{N-2} \end{bmatrix}, \quad \mathbf{S}_N^T \begin{bmatrix} x_{N-1} \\ x_0 \\ x_1 \\ \vdots \\ x_{N-2} \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{N-2} \\ x_{N-1} \end{bmatrix},$$

d.h. $\mathbf{S}_N$ ist eine Permutationsmatrix und somit unitär, also normal und diagonalisierbar. Die charakteristische Gleichung der Shift-Matrix ist (nach geschickter Gauss-Elimination) $\lambda^N = 1$, d.h. die N Eigenwerte von $\mathbf{S}_N$ sind die N -ten Einheitswurzeln

$$\lambda_k = e^{2\pi i \frac{-k}{N}}, \quad k = 0, 1, 2, \dots, N-1.$$

Für $j = 0, 1, \dots, N-1$ schreiben wir $w_N = e^{2\pi i/N}$. $(\mathbf{S}\mathbf{v}_k)_j = (\mathbf{v}_k)_{j-1} = w^{(j-1)k} = w^{-k}(\mathbf{v}_k)_j$, also $\mathbf{S}_N \mathbf{v}_k = \lambda_k \mathbf{v}_k$. Dies beweist:

Bemerkung 3.2.26. Der k -te Fourier-Vektor $\mathbf{v}_k$ ist ein Eigenvektor der Shift-Matrix $\mathbf{S}_N$ zum Eigenwert λ_k .

Somit ist es kein Wunder, dass die Fourier-Vektoren $\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{N-1}$ eine orthogonale Basis in $\mathbb{C}^N$, die trigonometrische Basis oder auch Fourier-Basis genannt, bilden. Wenn wir diese Vektoren auch normieren, $\mathbf{u}_k = \mathbf{v}_k/\sqrt{N}$, erhalten wir die unitäre Basis, die die Shift-Matrix diagonalisiert. Sammeln wir wie zuvor die Fourier-Vektoren in

$$\mathbf{V} = [\mathbf{v}_0 \mathbf{v}_1 \dots \mathbf{v}_{N-1}]$$

und ihre normierten Versionen in

$$\mathbf{U} = [\mathbf{u}_0 \mathbf{u}_1 \dots \mathbf{u}_{N-1}],$$

so haben wir:

$$\mathbf{V}^H \mathbf{V} = N \mathbf{I}_N, \quad \mathbf{U} = \frac{1}{\sqrt{N}} \mathbf{V}, \quad \mathbf{U}^H \mathbf{U} = \mathbf{I}_N$$

und die Diagonalisierung der Shift-Matrix $\mathbf{S}_N$ ist

$$\mathbf{S}_N = \mathbf{U}\mathbf{D}\mathbf{U}^H,$$

wobei $\mathbf{D}$ die Diagonalmatrix mit den Eigenwerten $\lambda_k = w_N^{-k}$ ist.

In der allgemeinen zirkulanten Matrix $\mathbf{C}$ ist jede Spalte ein solcher zyklischer Shift der vorherigen Spalte, d.h. die j -te Spalte in $\mathbf{C}$ ist $\mathbf{S}_N^j \mathbf{c}$. Darum betrachten wir nun

$$p(z) = c_0 z^0 + c_1 z^1 + c_2 z^2 + \dots + c_{N-1} z^{N-1},$$

sodass offensichtlich $p(\mathbf{S}_N) = \mathbf{C}$ gilt. Die Diagonalisierung von $\mathbf{S}_N$ liefert also die Diagonalisierung jeder beliebigen zirkulanten Matrix:

$$\mathbf{C} = p(\mathbf{S}_N) = \mathbf{U}p(\mathbf{D})\mathbf{U}^H,$$

wobei $p(\mathbf{D})$ ist die Diagonalmatrix mit den Einträgen $p(\lambda_k)$. Wir haben also bewiesen:

Theorem 3.2.27 (Gleichzeitige Diagonalisierung aller zirkulanten Matrizen). Die Elemente der trigonometrischen Basis sind die Eigenvektoren der Shift-Matrix $\mathbf{S}_N$ und sie diagonalisieren jede zirkulante Matrix $\mathbf{C}$ (sind also auch deren Eigenvektoren); die entsprechenden Eigenwerte sind in der Diagonalmatrix $p(\mathbf{D})$.

Wir bemerken auch, dass

$$(\mathbf{F}\mathbf{c})_k = p(\lambda_k),$$

und

$$(\mathbf{C}\mathbf{x})_k = \sum_{m=0}^{N-1} c_{(m-k) \bmod N} x_m.$$

Seien nun $g(x) = \sum_{k=-\infty}^{\infty} a_k x^k$ und $h(x) = \sum_{k=-\infty}^{\infty} b_k x^k$. Das Produkt dieser beiden Funktionen kann man auch als Potenzreihe schreiben:

$$f(x) = g(x)h(x) = \sum_{k=-\infty}^{\infty} c_k x^k = \sum_{k=-\infty}^{\infty} \left(\sum_{n=-\infty}^{\infty} a_n b_{k-n} \right) x^k.$$

Wenn wir nur endliche Summen haben, z.B. wenn g und h Polynome vom Grad maximal $N-1$ sind, dann sind auch alle Summen im Ergebnis endlich:

$$f(x) = g(x)h(x) = \sum_{k=0}^{2N-2} c_k x^k = \sum_{k=0}^{2N-2} \left(\sum_{n=0}^k a_n b_{k-n} \right) x^k.$$

Das begründet die folgende Definition:

Definition 3.2.28 (Faltung). Die Faltung zweier unendlicher Folgen $a = (a_k)_{k \in \mathbb{Z}}$ und $b = (b_k)_{k \in \mathbb{Z}}$ ist die unendliche Folge $c = (c_k)_{k \in \mathbb{Z}}$ mit $c_k = \sum_{n=-\infty}^{\infty} a_n b_{k-n}$. Man schreibt $c = a * b$.

Die Faltung von $\mathbf{a} = [a_0, \dots, a_{N-1}]^T$ und $\mathbf{b} = [b_0, \dots, b_{N-1}]^T$ erhält man automatisch aus dieser Definition, wenn wir $\mathbf{a}$ und $\mathbf{b}$ links und rechts mit Nullen zu unendlichen Folgen erweitern. Ein einfaches Beispiel der Faltung liefert die Multiplikation zweier grosser ganzer Zahlen; ein Beispiel (aus dem Buch “Wissenschaftliches Rechnen” von G. Strang) für 421×503 ist

$$(1 \cdot x^0 + 2 \cdot x^1 + 4 \cdot x^2) (3 \cdot x^0 + 0 \cdot x^1 + 5 \cdot x^2),$$

d.h. die Faltung von $(1, 2, 4)$ mit $(3, 0, 5)$ lässt sich via

$$3 \cdot x^0 + 6 \cdot x^1 + 17 \cdot x^2 + 10 \cdot x^3 + 20 \cdot x^4$$

berechnen, d.h. $(1, 2, 4) * (3, 0, 5) = (3, 6, 17, 10, 20)$.

Definition 3.2.29 (Zyklische Faltung). Ein spezieller und wichtiger Fall entsteht, wenn $x^N = 1$, d.h. wenn $x = \omega_N$, denn dann ist $x^{N+j} = x^j$. In diesem Fall erhalten wir die **zyklische Faltung** $c = a \circledast b$ der beiden periodischen Folgen mit Periode N oder der Vektoren der Länge N , die man periodisch auf unendliche periodische Folgen erweitert:

$$c = a \circledast b, \text{ mit } c_k = \sum_{j=0}^{N-1} a_j b_{(k-j) \bmod N} = \sum_{j=0}^{N-1} a_j b_{(k+j) \bmod N}.$$

Für das vorige Beispiel mit $x^3 = 1$ ist das leicht zu rechnen:

$$\begin{aligned} (1 \cdot x^0 + 2 \cdot x^1 + 4 \cdot x^2) (3 \cdot x^0 + 0 \cdot x^1 + 5 \cdot x^2) &= \\ (1 \cdot 3 + 2 \cdot 5 \cdot x^3 + 4 \cdot 0 \cdot x^3) x^0 + & \\ (1 \cdot 0 + 2 \cdot 3 + 4 \cdot 5 \cdot x^3) x^1 + & \\ (1 \cdot 5 + 2 \cdot 0 + 4 \cdot 3) x^2 &= \\ 13 \cdot x^0 + 26 \cdot x^1 + 17 \cdot x^2, & \end{aligned}$$

d.h. die zyklische Faltung in diesem Fall ist $(1, 2, 4) \circledast (3, 0, 5) = (13, 26, 17)$.

Bemerkung 3.2.30. Die Multiplikation im Zeit-/Ortsbereich $f(t) = g(t)h(t)$ bedeutet die Faltung im Frequenzbereich: $c = a * b$, denn $x = e^{2\pi i t}$. In anderen Worten: Die Fouriertransformierte des Produkts $f = g \cdot h$ ist die Faltung der Fouriertransformierten $c = a * b$. Wenn wir die Symmetrie zwischen Zeit/Ort und Frequenz berücksichtigen, ist es ersichtlich, dass die Multiplikation im Frequenzbereich die Faltung im Ortsbereich sein muss. Im diskreten Fall ist das offensichtlich (oder wir beweisen es gleich unten mittels Zirkulanten), im Fall von periodischen Funktionen der Periode T ist die Faltung als $(g * h)(x) = \frac{1}{T} \int_0^T g(t)h(x-t)dt$ definiert. Wenn das Signal g auf ein Messgerät mit dem Grundsignal h trifft, lesen wir eigentlich $f = g * h$ auf dem Gerät ab.

Bemerkung 3.2.31. Die Autokorrelation $f(t) * \bar{f}(-t)$ eines Signals $f(t)$ lässt sich im Frequenzbereich leicht berechnen: der k -te Fourier-Koeffizient der Autokorrelation ist $|\hat{f}_k|^2$. Das Energiespektrum / die spektrale Leistungsdichte / das Leistungsspektrum ist die Darstellung der Autokorrelation des Signals im Frequenzbereich.

Bemerkung 3.2.32. Die zyklische Faltung von $\mathbf{a} = [a_0, a_1, \dots, a_{N-1}]^T$ und $\mathbf{b} = [b_0, b_1, \dots, b_{N-1}]^T$ lässt sich einfach mit zirkulanten Matrizen schreiben. Sei $\mathbf{A}$ der mit der ersten Spalte $\mathbf{a} = [a_0, a_1, \dots, a_{N-1}]$ gebildete Zirkulant; dann ist die zyklische Faltung $\mathbf{c} = \mathbf{a} \circledast \mathbf{b}$ durch die Multiplikation mit dem Zirkulanten gegeben:

$$\mathbf{c} = \mathbf{A}\mathbf{b}.$$

Bemerkung 3.2.33. Da die Fourier-Basis jede zirkulante Matrix diagonalisiert, bedeutet die letzte Bemerkung, dass sich die Faltung $\mathbf{c} = \mathbf{a} \circledast \mathbf{b}$ als die folgende Matrix-Multiplikation schreiben lässt:

$$\mathbf{c} = \mathbf{A}\mathbf{b} = \mathbf{U}p(\mathbf{D})\mathbf{U}^H\mathbf{b} = \frac{1}{\sqrt{N}}\mathbf{V}p(\mathbf{D})\frac{1}{\sqrt{N}}\mathbf{V}^H\mathbf{b} = \mathbf{F}_N^{-1}p(\mathbf{D})\mathbf{F}_N\mathbf{b},$$

wobei, wie oben, $p(x) = a_0 + a_1x + \dots + a_{N-1}x^{N-1}$, sodass $p(\mathbf{S}_N) = \mathbf{A}$ und $\mathbf{D}$ die Diagonalmatrix mit den Einträgen $\lambda_k = w_N^k$ ist. Das ergibt dann:

$$\mathbf{F}_N\mathbf{c} = \text{diag}(\mathbf{F}_N\mathbf{a})\mathbf{F}_N\mathbf{b}, \quad (3.2.24)$$

d.h. die Fouriertransformierte der Faltung $\mathbf{F}_N\mathbf{c}$ ist die punktweise Multiplikation der Fouriertransformierten $\mathbf{F}_N\mathbf{a}$ und $\mathbf{F}_N\mathbf{b}$. Das ist ein sehr wichtiges Ergebnis für die Signalverarbeitung und trägt den Namen “Faltungstheorem”.

Bemerkung 3.2.34. Wir werden bald sehen, dass sich die diskrete Fourier-Transformation in $\mathcal{O}(N \log N)$ Operationen durchführen lässt. Aus der letzten Bemerkung folgt, dass sich die zyklische Faltung in $\mathcal{O}(N \log N)$ statt den erwarteten $\mathcal{O}(N^2)$ Operationen berechnen lässt.

Bemerkung 3.2.35. Die Faltung zweier Vektoren $\mathbf{x}, \mathbf{y} \in \mathbb{C}^N$ kann auf die zyklische Faltung der mit Nullen (nach unten) erweiterten Vektoren $\mathbf{a} = [\mathbf{x}^T, \mathbf{0}]^T$, $\mathbf{b} = [\mathbf{y}^T, \mathbf{0}]^T \in \mathbb{C}^{2N-1}$ zurückgeführt werden, die man dann zu $(2N-1)$ -periodischen Folgen erweitert.

3.3 FFT – Schnelle Fourier-Transformation

Nach *Science, Random Samples*, S. 799, 4.2.2000 gehört die schnelle Fourier-Transformation, kurz FFT, zu den 10 wichtigsten Algorithmen in Naturwissenschaft und Technik im 20. Jahrhundert.

Warum ist dieser Algorithmus aber so wichtig? Auf den ersten Blick scheint es, dass der DFT-Algorithmus für einen Vektor aus $\mathbb{C}^n$ eine Komplexität von $\mathcal{O}(N^2)$ hat, da es sich um die Multiplikation einer vollbesetzten Matrix mit einem Vektor handelt. Die Stärke der FFT liegt darin, dass sie eine asymptotische Komplexität von $\mathcal{O}(N \log N)$ hat, was deutlich besser ist.

Beispiel 3.3.1. Wir können die DFT mittels einer naiven, auf Schleifen basierten Implementierung, mittels Multiplikation mit der Matrix $\mathbf{F}_N$ oder mittels `fft.fft` realisieren. Hier sind die Ergebnisse, die für sich sprechen:

Code 3.3.2: Laufzeit verschiedener Implementierungen der DFT

```

from numpy import zeros, random, exp, pi, meshgrid, r_, dot,  fft
import timeit
from pylab import semilogy, show, plot, savefig, legend, grid, xlabel, ylabel

def naiveFT():
    global y, n

    c = 0.*1j*y
    # naive
    omega = exp(-2*pi*1j/n)
    c[0] = y.sum(); s = omega
    for jj in range(1,n):
        c[jj] = y[n-1]
        for kk in range(n-2,-1,-1): c[jj] = c[jj]*s+y[kk]
        s *= omega
    #return c

def matrixFT():
    global y, n

    # matrix based
    I, J = meshgrid(r_[0:n], r_[0:n])
    F = exp(-2*pi*1j*I*J/n)
    c = dot(F,y)
    #return c

def fftFT():
    global y,n
    c = fft.fft(y)
    #return c

nrexp = 5
N = 2**11 # how large the vector will be
res = zeros((N,4))
for n in range(1,N+1):
    y = random.rand(n)

    t = timeit.Timer('naiveFT()', 'from __main__ import naiveFT')
    tn = t.timeit(number=nrexp)
    t = timeit.Timer('matrixFT()', 'from __main__ import matrixFT')
    tm = t.timeit(number=nrexp)
    t = timeit.Timer('fftFT()', 'from __main__ import fftFT')
    tf = t.timeit(number=nrexp)
    #print n, tn, tm, tf
    res[n-1] = [n, tn, tm, tf]

print(res[:,3])

semilogy(res[:,0], res[:,1], 'b-', label='naive')
semilogy(res[:,0], res[:,2], 'k-', label='matrix')
semilogy(res[:,0], res[:,3], 'r-', label='fft')
legend()

```

```

grid()
54 xlabel(r'vector length $n$')
    ylabel('time (s)')
56 savefig('ffftime.pdf')
    savefig('ffftime.eps')
58 #show()

```

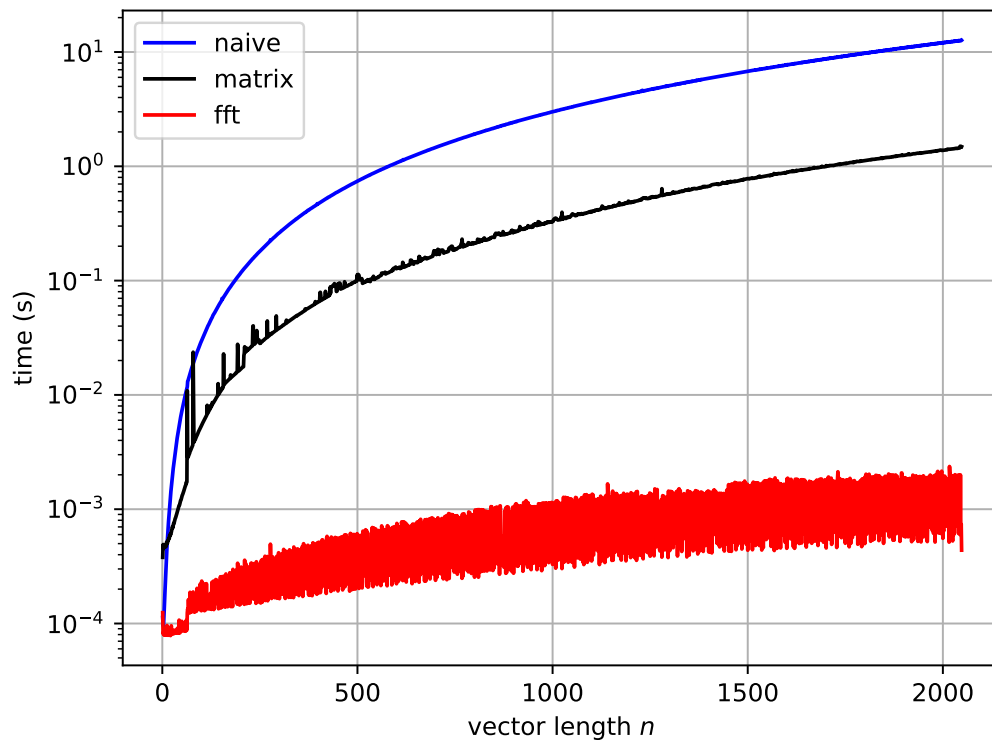


Abb. 3.3.3. Die Funktion `fft` ist viel besser als alle anderen Implementierungen.

Es gibt viele Algorithmen zur Durchführung einer FFT. Der heute am häufigsten verwendete Algorithmus wurde zuerst von Gauss² im Jahr 1805 entdeckt und dann 1965 von Cooley und Tukey³ wiederentdeckt.

Dieser Algorithmus heisst auch Cooley-Tukey-Algorithmus und beruht auf dem Prinzip “Divide and Conquer”. Die Idee dahinter ist, dass man das Problem, eine DFT der Länge n zu berechnen, auf die Berechnung vieler DFTs mit wesentlich kleineren Längen zurückführt und diese Ergebnisse dann wieder zusammenführt. Die Idee kann in vier Schritten beleuchtet werden:

²Nachlass: Theoria interpolationis methodo nova tractata, Werke, Band 3, 265–327 (Königliche Gesellschaft der Wissenschaften, Göttingen, 1866)

³An algorithm for the machine calculation of complex Fourier series, Math. Comput. 19, 297–301 (1965)

1. Vektoren der Länge $N = 2m$
2. Vektoren der Länge $N = 2^L$
3. Vektoren der Länge $N = pq$ mit p, q ganzen Zahlen
4. Vektoren der Länge N , wobei N eine Primzahl ist

Für den ersten Schritt führen wir eine Umformung der diskreten Fourier-Transformation durch:

$$\begin{aligned}
 c_k &= \sum_{j=0}^{N-1} y_j e^{-\frac{2\pi i}{N} jk} \\
 &= \sum_{j=0}^{m-1} y_{2j} e^{-\frac{2\pi i}{N} 2jk} + \sum_{j=0}^{m-1} y_{2j+1} e^{-\frac{2\pi i}{N} (2j+1)k} \\
 &= \sum_{j=0}^{m-1} y_{2j} e^{-\frac{2\pi i}{N/2} jk} + e^{-\frac{2\pi i}{N} k} \sum_{j=0}^{m-1} y_{2j+1} e^{-\frac{2\pi i}{N/2} jk}
 \end{aligned}$$

Die Summen stellen nun die geraden und ungeraden Koeffizienten der diskreten Fourier-Transformation dar. Diese bekommt man nun wieder durch eine diskrete Fourier-Transformation. Dies kann man prinzipiell so lange wiederholen, bis man die DFT eines Vektors der Länge eins berechnen muss, was dann einfach ist, da die entsprechende Transformationsmatrix eine 1×1 -Matrix mit dem Eintrag $\omega_1^0 = 1$ ist. Damit ist das Problem für Vektoren der Länge $N = 2^L$ eigentlich gelöst. Der folgende Python-Code zeigt, wie man dies in Python realisieren kann.

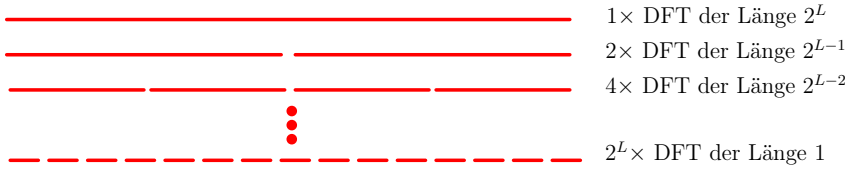
Code 3.3.4: FFT-Implementierung für Vektoren der Länge $N = 2^L$

```

1 from numpy import linspace, random, hstack, pi, exp
2 def fftrec(y):
3     n = len(y)
4     if n == 1:
5         c = y
6         return c
7     else:
8         c0 = fftrec(y[::2])
9         c1 = fftrec(y[1::2])
10        r = (-2.j*pi/n) * linspace(0,n-1,n)
11        c = hstack((c0,c0)) + exp(r) * hstack((c1,c1))
12    return c

```

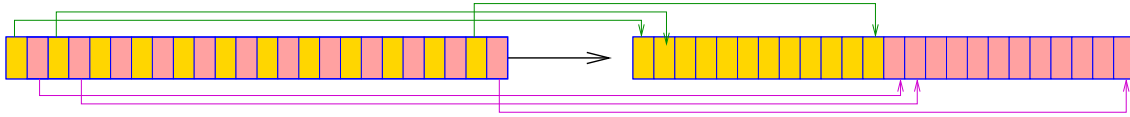
Die Kosten dieser Rekursion sind:



d.h. es sind L Stufen, und jede Stufe der Rekursion braucht $\mathcal{O}(2^L)$ Operationen.

Bemerkung 3.3.5. Wir können die FFT auch als Matrix-Zerlegung betrachten. Für $N = 2m$ sei die Permutation

$$P_m^{\text{OE}}(1, \dots, N) = (1, 3, \dots, N-1, 2, 4, \dots, N) .$$



Da aber $\omega_N^{2j} = \omega_m^j$, haben wir für diese Permutation der Spalten:

$$P_m^{\text{OE}} \mathbf{F}_N = \begin{bmatrix} \mathbf{F}_m & \mathbf{D}_m \mathbf{F}_m \\ \mathbf{F}_m & -\mathbf{D}_m \mathbf{F}_m \end{bmatrix} = \begin{bmatrix} \mathbf{F}_m & \\ & \mathbf{F}_m \end{bmatrix} \begin{bmatrix} \mathbf{I}_m & \mathbf{D}_m \\ \mathbf{I}_m & -\mathbf{D}_m \end{bmatrix}$$

wobei $\mathbf{D}_m = \text{diag}(\omega_N^0, \omega_N^1, \dots, \omega_N^{m-1})$.

Nun ist es allerdings so, dass man in der Numerik nicht nur Vektoren mit einer solchen Länge schnell transformieren möchte. Was macht man nun, wenn man einen Vektor der Länge $N = pq$ hat? Dann verwendet man den Cooley-Tukey-Algorithmus.

Zusammenfassend lässt sich sagen, dass die FFT auf dem Prinzip beruht, dass es effizienter ist, viele einfache, kleine Probleme zu lösen, als eine Matrixmultiplikation mit $\mathcal{O}(N^2)$ auszuführen. Man beobachtet in der Praxis, dass die FFT für

Zweierpotenzen am schnellsten ist. Zahlen mit kleinen Primfaktoren sind auch noch relativ schnell, für Zahlen mit grossen Primfaktoren oder Primzahlen ist sie jedoch deutlich langsamer.

3.4 Trigonometrische Interpolation und rechentechische Aspekte

3.4.1 Wiederholung: von der Approximation zur Interpolation

Unser Zugang zur Fourier-Approximation war: Gegeben $f \in L^2(0, 1)$, können wir f als eine 1-periodische Funktion auf $\mathbb{R}$ ansehen:

$$f(t) = \sum_{k=-\infty}^{\infty} \widehat{f}(k) e^{2\pi i k t} \text{ in } L^2(0, 1),$$

wobei die exakten Fourier-Koeffizienten durch ein Integral gegeben sind:

$$\widehat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt.$$

Die Fourier-Approximation erhält man, wenn man die unendliche Fourier-Reihe abbricht; analog zur Taylor-Reihe (deren Abbruch zur Approximation durch ein Taylor-Polynom führt) erhalten wir eine Approximation mit trigonometrischen Polynomen:

$$f_{2n}(t) = \sum_{k=-n}^n \widehat{f}(k) e^{2\pi i k t}, \text{ was } 2n + 1 \text{ Koeffizienten braucht,}$$

bzw.

$$f_{2n-1}(t) = \sum_{k=-n}^{n-1} \widehat{f}(k) e^{2\pi i k t}, \text{ was } 2n \text{ Koeffizienten braucht.}$$

Diese sind 1-periodische Funktionen, obwohl f nicht zwingend periodisch sein muss. Die Approximation gilt im Sinne der $L^2(0, 1)$ -Konvergenz, und die Konvergenzgeschwindigkeit hängt entscheidend von der Glattheit der Periodisierung von f ab. Die Orthogonalität der $e^{2\pi i k t}$ -Funktionen bezüglich des $L^2(0, 1)$ -Skalarprodukts macht diese Approximation auch zu einer orthogonalen Projektion im Raum der trigonometrischen Polynome vom Grad $\leq n$.

Wenn wir die Fourier-Koeffizienten $\widehat{f}(k)$ nicht exakt kennen, müssen wir diese ebenfalls approximieren. Eine Option, die wir bereits gesehen haben, ist, das Integral durch die Trapezregel (siehe dazu das Kapitel über die Quadratur) anzunähern:

$$\widehat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt \approx \frac{1}{N} \sum_{\ell=0}^{N-1} f\left(\frac{\ell}{N}\right) e^{-2\pi i k \frac{\ell}{N}} =: \widehat{f}_N(k)$$

für eine 1-periodische Funktion f (d.h. $f(0) = f(1)$). Somit sind die diskreten Fourier-Koeffizienten $\hat{f}_N(k)$ für $k = 0, 1, \dots, N-1$ durch die Anwendung der Fourier-Matrix auf den Vektor der Funktionswerte an den Punkten $\frac{\ell}{N}$ zu berechnen. Die richtige Zuordnung zu den Frequenzen im trigonometrischen Polynom geschieht mit `fftshift` wie in Theorem 3.2.11 beschrieben; hier ist sie nochmals (für gerades $N = 2n$):

$$\begin{aligned}\hat{f}_N(k) &= \gamma_k \approx \hat{f}(k) \text{ für } k = 0, \dots, n-1, \\ \hat{f}_N(n) &= \gamma_{-n} \approx \hat{f}(-n), \\ \hat{f}_N(k) &= \gamma_{k-N} \approx \hat{f}(k-N) \text{ für } k = n+1, \dots, N-1.\end{aligned}$$

Bemerkung 3.4.1. Lassen Sie uns diese diskrete Fourier-Approximation mit $N = 2n$ Koeffizienten an den Punkten $\frac{\ell}{N}$ für $\ell = 0, 1, \dots, N-1$ auswerten. Zuerst wollen wir uns um den Shift kümmern:

$$f_{N-1}(x) = \sum_{k=-n}^{n-1} \gamma_k e^{2\pi i k x} = \sum_{k=0}^{n-1} \gamma_k e^{2\pi i k x} + \sum_{k=-n}^{-1} \gamma_k e^{2\pi i k x}.$$

Für $k \geq 0$ ist $\gamma_k = \hat{f}_N(k) = \frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) e^{-2\pi i k \frac{j}{N}}$ und für $k < 0$ ist $\gamma_k = \hat{f}_N(N+k) = \frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) e^{-2\pi i (N+k) \frac{j}{N}}$. Da $e^{-2\pi i (N+k) \frac{j}{N}} = e^{-2\pi i k \frac{j}{N}}$, erhalten wir

$$f_{N-1}(x) = \frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) \sum_{k=-n}^{n-1} e^{2\pi i k \left(x - \frac{j}{N}\right)},$$

welche wir nun an der Stelle $\frac{\ell}{N}$ auswerten:

$$\begin{aligned}f_{N-1}\left(\frac{\ell}{N}\right) &= \frac{1}{N} \sum_{j=0}^{N-1} \sum_{k=-n}^{n-1} f\left(\frac{j}{N}\right) \omega_N^{-k(j-\ell)} = \frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) \sum_{k=-n}^{n-1} \omega_N^{k(\ell-j)} \\ &= f\left(\frac{\ell}{N}\right),\end{aligned}$$

denn die Orthogonalität der diskreten Fourier-Vektoren besagt, dass $\sum_{k=-n}^{n-1} \omega_N^{k(j-\ell)} = 0$, ausser für $j = \ell$, wo die Summe N ergibt.

Somit erfüllt unsere diskrete Fourier-Approximation die Interpolationsbedingung an den Punkten $\frac{\ell}{N}$:

$$f_{N-1}\left(\frac{\ell}{N}\right) = f\left(\frac{\ell}{N}\right) \text{ für } \ell = 0, 1, \dots, N-1,$$

das heisst, sie löst auch die Interpolationsaufgabe:

Finde ein trigonometrisches Polynom p_{N-1} , sodass

$$p_{N-1}\left(\frac{\ell}{N}\right) = f\left(\frac{\ell}{N}\right) \text{ für } \ell = 0, 1, \dots, N-1.$$

Für allgemeines $N \in \mathbb{N}$ definieren wir nun die Räume der trigonometrischen Polynome

$$\mathcal{T}_N = \text{span}\{e^{2\pi i j t}; j = -\lfloor \frac{N-1}{2} \rfloor, \dots, \lfloor \frac{N}{2} \rfloor\}.$$

Für die Lösung der Interpolationsaufgabe im Raum $\mathcal{T}_N$ an den Punkten $t_\ell \in [0, 1[$ haben wir zwei Möglichkeiten:

- (1) mittels Lösung eines linearen Gleichungssystems:

$$\sum_j \gamma_j e^{2\pi i j t_\ell} = f(t_\ell) \text{ für } \ell = 0, 1, \dots, N-1.$$

Das kostet $\mathcal{O}(N^3)$ Operationen, funktioniert aber für beliebige $t_\ell \in [0, 1[$.

- (2) mittels FFT in $\mathcal{O}(N \log N)$ Operationen, falls die Punkte äquidistant sind, $t_\ell = \frac{\ell}{N}$, denn dann ist die Matrix des obigen Gleichungssystems $\mathbf{F}_N^{-1}$.

Code 3.4.2: Berechnung der Koeffizienten des trigonometrischen Polynoms

```

1  from numpy import hstack, arange, sin, cos, pi, outer, exp, linspace, fft,
   random, conj
2  from numpy.linalg import solve
3  import time
4
5  def trigpolycoeff(t, y):
6      N = y.shape[0]
7
8      if N%2 == 1:
9          n = (N-1)/2.
10         M = exp(2*pi*1j*outer(t, arange(-n, n+1)))
11     else:
12         n = N/2.
13         M = exp(2*pi*1j*outer(t, arange(-n, n)))
14     c = solve(M, y)
15     return c
16
17 N = 2**12
18 t = linspace(0, 1, N, endpoint=False)
19 y = random.rand(N) #+ 1j*random.rand(N)
20 tic = time.time()
21 direkt = trigpolycoeff(t, y)
22 toc = time.time() - tic
23 print('time for a direkt transform', toc, ' s')
24 tic = time.time()
25 viafft = fft.fftshift(fft.fft(y)/N)
   toc = time.time() - tic

```

```

27 print('time for a transform via fft',toc, ' s')
    tic = time.time()
29 viaIfft = conj(fft.fftshift(fft.ifft(y)))
    toc = time.time() - tic
31 print('time for a transform via ifft',toc, ' s')
    print(abs(direkt-viafft).max(), abs(direkt-viaIfft).max())

```

3.4.2 Auswertung: "Zero-Padding"

Eine andere Frage ist, wie wir ein solches trigonometrisches Polynom

$$p_{N-1}(t) = \sum_{j=-n}^{n-1} \gamma_j e^{2\pi i j t}$$

effizient an den äquidistanten Punkten $\frac{k}{M}$ mit $M > N$ für $k = 0, 1, 2, \dots, M-1$ auswerten können. Wir schreiben dafür $p_{N-1} \in \mathcal{T}_N \subset \mathcal{T}_M$ in der trigonometrischen Basis von $\mathcal{T}_M$, was einfach das Einfügen von Nullen im Koeffizientenvektor an den Stellen der höheren Frequenzen bedeutet. Wir müssen die Nullen direkt nach der Anwendung der Funktion `fft` oder `ifft` in der Mitte des Koeffizientenvektors einfügen. Man nennt dieses Verfahren auch *Zero-Padding*.

Die folgenden beiden Codes geben zwei Varianten, wie man dies implementieren kann. Dabei benötigt der zweite Code einige Multiplikationen mehr. Der erste Code ist somit ein bisschen schneller, jedoch nur für reelle Werte der Funktion f korrekt.

Code 3.4.3: Schnelle Auswertung des trigonometrischen Interpolationspolynoms auf äquidistanten Punkten: mit `ifft` nur für reelle Werte korrekt

```

1  from numpy import zeros, fft, sqrt, pi, real, linalg, linspace, sin
3  def evaliptrig(y,N):
    n = len(y)
5     if (n%2) == 0:
        c = fft.ifft(y)
7         a = zeros(N, dtype=complex)
        a[:n//2] = c[:n//2]
9         a[N-n//2:] = c[n//2:]
        v = fft.fft(a);
11        return v
    else: raise TypeError('odd length')
13
14  f = lambda t: 1./sqrt(1.+0.5*sin(2*pi*t))
15
16  vlinf = []; vl2 = []; vn = []
17  N = 4096; n = 2
18  while n < 1+2**7:
19      t = linspace(0,1,n, endpoint=False)
20      y = f(t)
21      v = real(evaliptrig(y,N))

```

```

23     t = linspace(0,1,N, endpoint=False)
    fv = f(t)
    d = abs(v-fv); linf = d.max()
25     l2 = linalg.norm(d)/sqrt(N)
    vlinf += [linf]; vl2 += [l2]; vn += [n]
27     n *= 2

29 from pylab import semilogy, show
    semilogy(vn,vl2,'-+'); show()

```

Code 3.4.4: Schnelle Auswertung des trigonometrischen Interpolationspolynoms auf äquidistanten Punkten: mit `fft` immer korrekt

```

1  from numpy import zeros, fft, sqrt, pi, real, linalg, linspace, sin

3  def evaliptrig(y,N):
    n = len(y)
5     if (n%2) == 0:
        c = fft.fft(y)*1./n
        a = zeros(N, dtype=complex)
        a[:n//2] = c[:n//2]
        a[N-n//2:] = c[n//2:]
        v = fft.ifft(a)*N;
11     return v
    else: raise TypeError('odd length')

13 f = lambda t: 1./sqrt(1.+0.5*sin(2*pi*t))

15 vlinf = []; vl2 = []; vn = []
17 N = 4096; n = 2
    while n < 1+2**7:
19         t = linspace(0,1,n, endpoint=False)
        y = f(t)
21         v = real(evaliptrig(y,N))
        t = linspace(0,1,N, endpoint=False)
23         fv = f(t)
        d = abs(v-fv); linf = d.max()
25         l2 = linalg.norm(d)/sqrt(N)
        vlinf += [linf]; vl2 += [l2]; vn += [n]
27         n *= 2

29 from pylab import semilogy, show
    semilogy(vn,vl2,'-+'); show()

```

3.5 Trigonometrische Interpolation und Fehlerabschätzungen

Definition 3.5.1. Wir sprechen von

algebraischer Konvergenz, wenn der Fehler $E(n) = O(\frac{1}{n^p})$ mit $p > 0$ ist,

und von

exponentieller Konvergenz, wenn der Fehler $E(n) = O(q^n)$ mit $0 \leq q < 1$ ist.

Wir wollen nun den Fehler betrachten, wenn wir $f : [0, 1] \rightarrow \mathbb{R}$ mittels trigonometrischer Interpolation an den äquidistanten Punkten $\frac{k}{N}$, für $k = 0, 1, \dots, N-1$, approximieren. Wir motivieren diese Fehlerbetrachtung durch drei Beispiele:

(1) Stufenfunktion: die periodische Fortsetzung der Funktion f :

$$f : [0, 1] \rightarrow \mathbb{R}$$

$$f(t) = \begin{cases} 0 & \text{für } |t - \frac{1}{2}| > 1/4 \\ 1 & \text{für } |t - \frac{1}{2}| \leq 1/4. \end{cases} \quad (3.5.25)$$

(2) Periodische glatte Funktion:

$$h : \mathbb{R} \rightarrow \mathbb{R}$$

$$h(t) = \frac{1}{\sqrt{1 + \frac{1}{2} \sin(2\pi t)}}. \quad (3.5.26)$$

(3) Hutfunktion: die periodische Fortsetzung der Funktion g :

$$g : [0, 1] \rightarrow \mathbb{R}$$

$$g(t) = |t - \frac{1}{2}|. \quad (3.5.27)$$

In den folgenden Plots sehen wir die Konvergenz dieser Funktionen in Abhängigkeit vom Grad des interpolierenden trigonometrischen Polynoms.

```
1  t = linspace(0,1,n+1); y = f(t[:-1])
2  v = real(evaliptrig(y,N))
3  t = linspace(0,1,N+1); fv = f(t)
4  d = abs(v-fv[:-1]); linf = d.max()
5  l2 = linalg.norm(d) / sqrt(N)
```

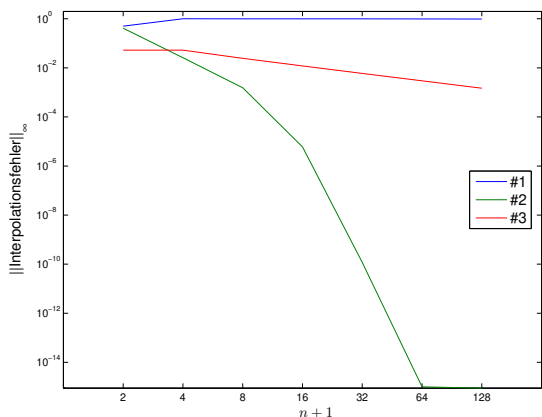


Abb. 3.5.2. Fall (1) und (3): algebraische Konvergenz, Fall (2): exponentielle Konvergenz

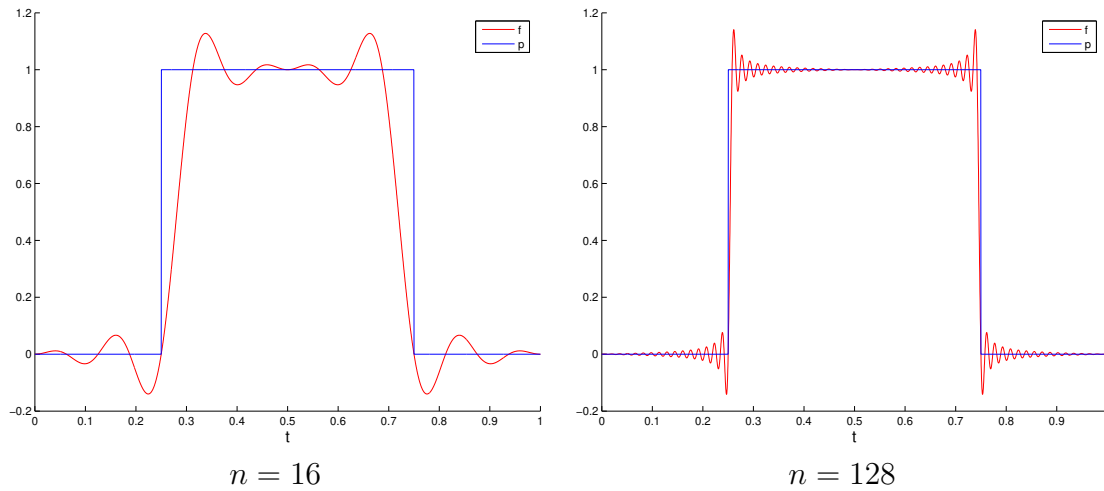


Abb. 3.5.3. Gibbs-Phänomen an Sprungstellen

In diesem Beispiel ist eindeutig zu erkennen, dass die Stellen, an denen die Funktion unglatt ist, für die Verlangsamung der Konvergenz zuständig sind. Noch interessanter ist das Beispiel, in dem wir eine sehr glatte Funktion mit einem zusätzlichen Parameter haben:

Beispiel 3.5.4. Für $\alpha \in [0, 1)$ definieren wir:

$$f(t) = \frac{1}{\sqrt{1 - \alpha \sin(2\pi t)}} \text{ auf } I = [0, 1],$$

und wir beobachten eine exponentielle Konvergenz in n , die für kleineres α schneller ist.

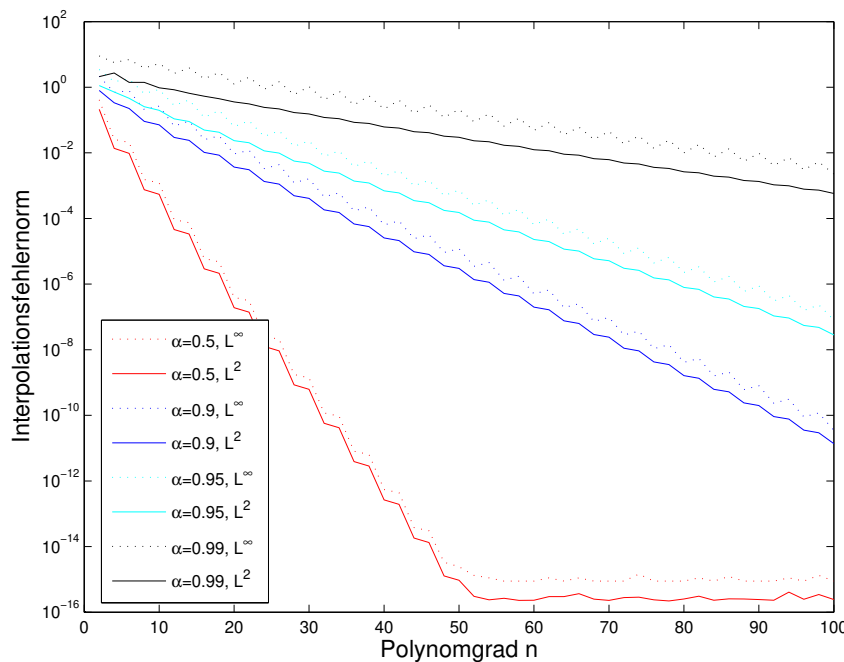


Abb. 3.5.5. Fehler bei der trigonometrischen Interpolation

Theorem 3.5.6 (Aliasing). Der k -te Fourier-Koeffizient des N -ten trigonometrischen Interpolationspolynoms unterscheidet sich vom k -ten Fourier-Koeffizienten von f gerade um die Summe aller Fourier-Koeffizienten, die um ganze Vielfache von N vom k -ten Fourier-Koeffizienten verschoben sind:

$$\hat{p}_N(k) - \hat{f}(k) = \sum_{j \in \mathbb{Z}, j \neq 0} \hat{f}(k + jN). \quad (3.5.28)$$

Die erste Aussage des folgenden Korollars folgt aus der Bemerkung über die Fourier-Koeffizienten der Ableitung. Wenden wir dieses Lemma p -mal an und benutzen dann auf beiden Seiten die Identität von Parseval, so folgt:

$$\|f^{(p)}\|_{L_2(0,1)}^2 = (2\pi)^{2p} \sum_{k=-\infty}^{\infty} k^{2p} |\hat{f}(k)|^2. \quad (3.5.29)$$

Falls $\int_0^1 |f^{(p)}(t)| dt < \infty$ erfüllt ist, gilt $|\hat{f}(k)| = \mathcal{O}(k^{-p})$ für $|k| \rightarrow \infty$.

Korollar 3.5.7. Für $f \in C^p([0, 1])$ mit $p \geq 1$ und f 1-periodisch, gilt:

$$|\hat{p}_N(k) - \hat{f}(k)| = \mathcal{O}(N^{-p}) \quad \text{für } |k| \leq \frac{N}{2}. \quad (3.5.30)$$

Als Folgerung kann man aus diesem Satz ziehen, dass die Approximation der Fourier-Koeffizienten von f bei kleinen Frequenzen, d.h. hier konkret $|k| < \frac{N}{2}$, gut durch die Fourier-Koeffizienten des trigonometrischen Interpolationspolynoms approximiert wird.

Der folgende Satz stellt einen interessanten Zusammenhang zwischen den Fourier-Koeffizienten einer Funktion f und der Güte der Approximation durch trigonometrische Polynome her.

Theorem 3.5.8 (Fehler der trigonometrischen Interpolation). Sei f eine 1-periodische Funktion und konvergiere die Reihe $\sum_{k \in \mathbb{Z}} |\hat{f}(k)|$ absolut, dann kann man

$$|p_N(x) - f(x)| \leq 2 \sum_{|k| \geq N/2} |\hat{f}(k)|, \quad \forall x \in \mathbb{R} \quad (3.5.31)$$

für den Approximationsfehler schreiben.

Der Fehler bei der Interpolation mit trigonometrischen Polynomen wird also durch die Beträge der schlecht approximierbaren Fourier-Koeffizienten $\hat{f}(k)$ nach oben beschränkt. Es stellt sich die Frage, wann man Funktionen mittels trigonometrischer Polynome perfekt approximieren kann. Mit Polynomen n -ten Grades kann man ja alle Polynome von kleinerem oder gleichem Grad perfekt approximieren. Bei periodischen Funktionen ist nicht der Grad ausschlaggebend, sondern die "Frequenzen" der Funktion.

Korollar 3.5.9 (Perfekte Approximation, Abtasttheorem). Sei f eine 1-periodische Funktion mit der maximalen Frequenz m , das heisst $\hat{f}(k) = 0$ für alle $|k| > m$. Falls $N > 2m$, gilt dann $p_N(x) = f(x)$ für alle x .

Man sieht also: Falls die Fourier-Koeffizienten ab einem gewissen $|k|$ identisch verschwinden, dann wird die Funktion perfekt approximiert. Was bedeutet dies anschaulich? Wenn in unserem trigonometrischen Interpolationspolynom Summanden vorkommen, die mehr als doppelt so schnell wie die zu interpolierende Funktion oszillieren, ist eine perfekte Interpolation möglich.

Diese Sätze haben aber eine erstaunliche Konsequenz für die Anwendung. Stellen wir uns vor, wir haben ein analoges Musiksinal und wollen dieses digitalisieren. Wir messen also das Signal an einer diskreten Menge von Zeitpunkten:

$$\mathcal{T} = \left\{ \frac{j}{n} \right\}_{j=0}^{n-1} \quad (3.5.32)$$

Sei M nun die maximale Frequenz, mit der die Funktion schwingt (d.h. $\hat{f}(k) = 0$ für alle $|k| > M$), und n unsere Abtastfrequenz mit $n \leq 2M$. Dann gilt für alle Punkte aus unserer Messung:

$$\exp(2\pi i M t) = \exp(2\pi i (M - n) t) \quad \forall t \in \mathcal{T} \quad (3.5.33)$$

Interpolieren wir diese beiden Funktionen, so erhält man, dass folgende Gleichheit gilt:

$$p_n(e^{2\pi i M t}) = p_n(e^{2\pi i (M - n) t}) \quad (3.5.34)$$

Die interpolierenden Polynome sind also für die beiden unterschiedlichen Funktionen gleich, d.h. die wesentlich langsamer schwingende Funktion erzeugt eine genau gleich gute Interpolation. Wenn man also ein Signal mit zu wenig Punkten abtastet, bekommt man durch diesen Effekt den sog. "Aliasing-Effekt". Dasselbe interpolierende Polynom für zwei klar unterscheidbare Funktionen, die in ihrer Periode

klar voneinander abweichen. Daraus folgt, dass viele Details der zu interpolierenden Funktion verloren gehen. Für $f(t) = \sin(2\pi \cdot 19t)$ und $N = 38$ entstehen bei $n = 4, 8, 16$ Funktionen, die wenig mit dem Original zu tun haben, wie in Abbildung 3.5.10 zu sehen ist.

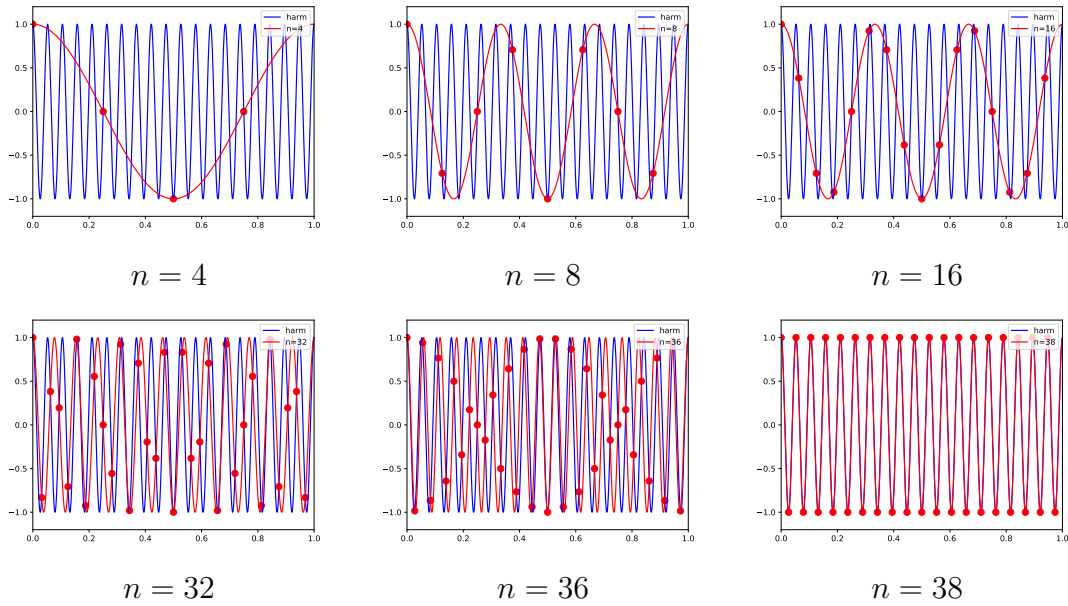


Abb. 3.5.10. Aliasing für $f(t) = \cos(2\pi \cdot 19t)$

Auf unser Musikstück übertragen bedeutet dies, dass es nicht mehr so wie vorher klingt. Ein weiteres interessantes Beispiel für diesen Effekt ist die scheinbare Rückwärtsbewegung von Autorädern, die in alten Filmen bei zu niedriger Bildrate pro Sekunde zu sehen ist.

Man kann mit einer genauen Analyse der Situation den folgenden Satz beweisen, der die Regularität der Funktion direkt mit der Güte der Approximation durch die trigonometrische Interpolation in Verbindung setzt.

Theorem 3.5.11 (Fehlerabschätzung für trigonometrische Interpolation). Sei für $k \in \mathbb{N}$ $f^{(k)} \in L^2(0, 1)$, dann gilt:

$$\|f - p_N(f)\|_{L^2(0,1)} \leq \sqrt{1 + c_k} N^{-k} \|f^{(k)}\|_{L^2(0,1)} \quad (3.5.35)$$

mit

$$c_k = 2 \sum_{l=1}^{\infty} (2l - 1)^{-2k}. \quad (3.5.36)$$

Je mehr Ableitungen in $L^2(0, 1)$ liegen, desto kleiner ist der Fehler. Man beachte, dass man hier eine Konvergenz im Sinne der L^2 -Norm hat; eine solche Konvergenz ist schwächer als eine Konvergenz im Sinne der Supremumsnorm. Zur Illustration des Konvergenzverhaltens schauen wir uns die folgenden Figuren an.

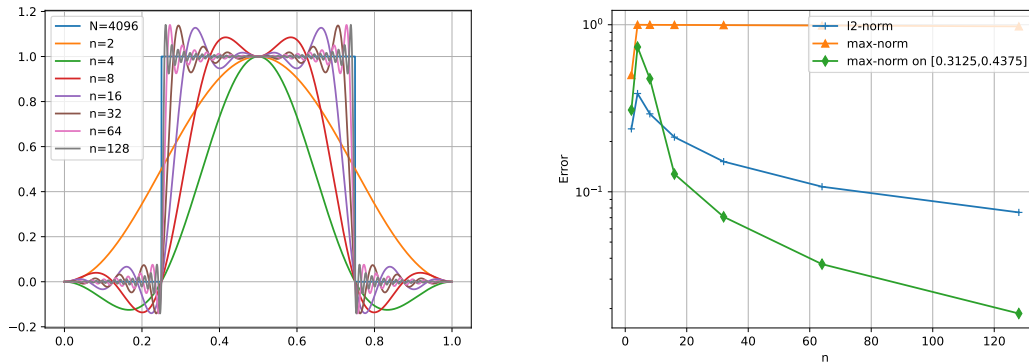


Abb. 3.5.12. Trigonometrische Approximation der Stufenfunktion (3.5.25)

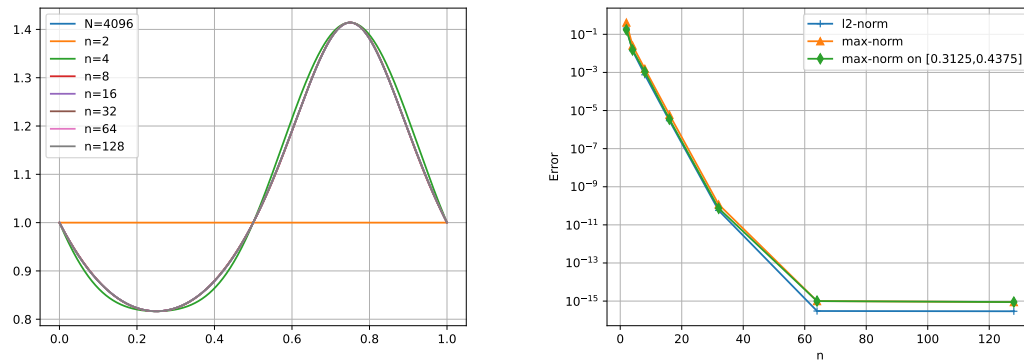
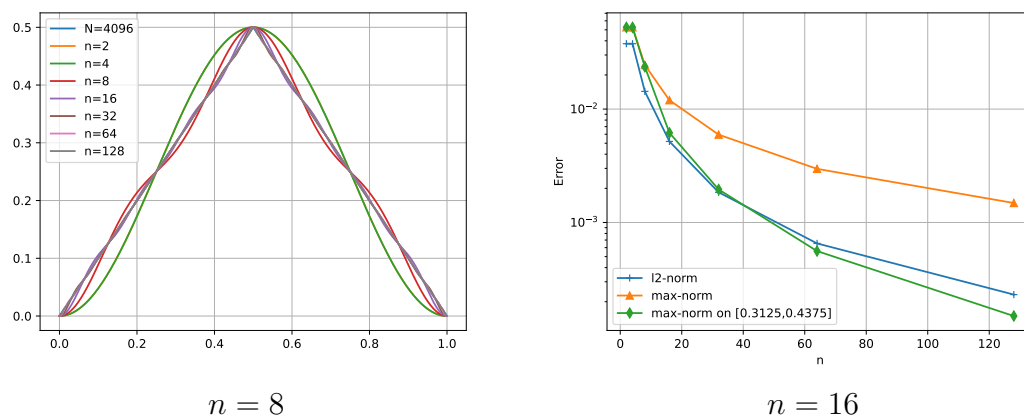


Abb. 3.5.13. Trigonometrische Approximation der glatten Funktion (3.5.26)



$n = 8$

$n = 16$

Abb. 3.5.14. Trigonometrische Approximation der Hutfunktion (3.5.27)

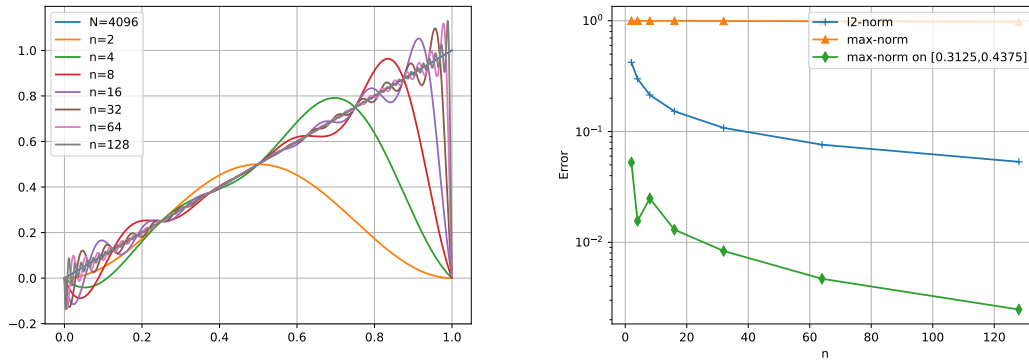


Abb. 3.5.15. Trigonometrische Approximation einer Rampenfunktion; durch Periodisierung entstehen Sprünge bei 0 und 1.

3.6 DFT und Chebyshev-Interpolation

Wir haben gesehen, dass man glatte Funktionen numerisch gut durch Chebyshev-Polynome interpolieren kann. Wie können wir die **Chebyshev-Koeffizienten effizient aus Interpolationswerten berechnen**? Die naive Aufsummierung aus Theorem 2.4.16 kostet $\mathcal{O}(n^2)$ Operationen und kann etwas beschleunigt werden, wenn wir den gesamten Koeffizientenvektor mittels einer Matrixmultiplikation erhalten. Kommt das irgendwie bekannt vor?

Die Formeln aus Theorem 2.4.16 verraten bereits, dass es eine Verbindung zwischen den Chebyshev-Koeffizienten und den Fourier-Koeffizienten geben muss. Mit der FFT bekommt man einen sehr effizienten Weg, diese Interpolation durchzuführen. Betrachten wir nun ein Chebyshev-Interpolationspolynom $p \in \mathcal{P}_n$ auf $[-1, 1]$ zu einer Funktion $f : [-1, 1] \rightarrow \mathbb{C}$, das auf der Grundlage von Interpolationswerten an den **Chebyshev-Knoten** t_k konstruiert wird, d.h. $f(t_k) = p(t_k)$, wobei die Chebyshev-Knoten wie folgt definiert sind:

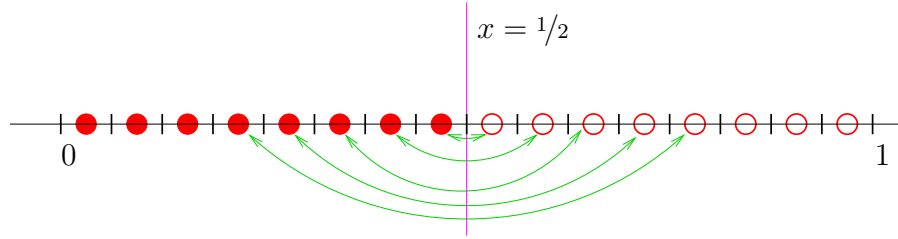
$$t_k := \cos \left(\frac{2k+1}{2(n+1)} \pi \right), \quad k = 0, \dots, n.$$

Wir definieren die beiden Hilfsfunktionen g und q :

$$\begin{aligned} g : [-1, 1] &\rightarrow \mathbb{C} : & g(s) &:= f(\cos 2\pi s) \\ q : [-1, 1] &\rightarrow \mathbb{C} : & q(s) &:= p(\cos 2\pi s). \end{aligned}$$

Dann übersetzt sich unsere Interpolationsbedingung in den Chebyshev-Knoten wie folgt:

$$p(t_k) = f(t_k) \Leftrightarrow q \left(\frac{2k+1}{4(n+1)} \right) = g \left(\frac{2k+1}{4(n+1)} \right)$$



Wir wenden nun die Translation $s^* = s + \frac{1}{4(n+1)}$ an und definieren erneut Hilfsfunktionen:

$$\begin{aligned} g^*(s) &:= g(s^*) = g\left(s + \frac{1}{4(n+1)}\right) \\ q^*(s) &:= q(s^*) = q\left(s + \frac{1}{4(n+1)}\right) \end{aligned}$$

Wir zeigen nun, dass $q(s)$ ein trigonometrisches Polynom ist, genauer zeigen wir, dass $q(s) \in \mathcal{T}_n$ ist:

$$\begin{aligned} q(s) &= p(\cos 2\pi s) \\ &= \sum_{j=0}^n a_j T_j(\cos(2\pi s)) \\ &= \sum_{j=0}^n a_j \cos(2\pi j s) \\ &= \sum_{j=0}^n \frac{1}{2} a_j (e^{2\pi i j s} + e^{-2\pi i j s}) \\ &= \sum_{j=-n}^n c_j e^{2\pi i j s}, \end{aligned}$$

wobei

$$c_j := \begin{cases} \frac{1}{2} a_j & , \text{ für } j = 1, \dots, n, \\ a_0 & , \text{ für } j = 0, \\ \frac{1}{2} a_{-j} & , \text{ für } j = -n, \dots, -1. \end{cases}$$

Die Symmetrie $q(s) = q(1-s)$ und $q\left(\frac{2k+1}{4(n+1)}\right) = f(t_k)$ für $k = 0, \dots, n$ ergeben zusammen die Interpolationsbedingungen

$$q\left(\frac{k}{2(n+1)} + \frac{1}{4(n+1)}\right) = z_k := \begin{cases} y_k & , \text{ für } k = 0, \dots, n, \\ y_{2n+1-k} & , \text{ für } k = n+1, \dots, 2n+1, \end{cases} \quad (3.6.37)$$

und somit ist q^* das trigonometrische Interpolationspolynom von g^* . Man kann also die Chebyshev-Interpolation durch eine DFT durchführen. Weiterhin übertragen

sich die Fehlerabschätzungen von der trigonometrischen Interpolation direkt auf die Chebyshev-Interpolation.

Für die konkrete Implementierung ist es nützlich, die Rechnungen anders zu gestalten. Die Nullstellen von T_{n+1} , d.h. die Chebyshev-Knoten t_k , lassen sich auch schreiben als

$$t_k = \cos(2\pi\xi_k) \text{ mit } \xi_k = \frac{2k+1}{4(n+1)}.$$

Wir sahen bereits

$$q(s) = p(\cos 2\pi s) = \sum_{j=-n}^n c_j \exp(2\pi i j s).$$

Nun ist

$$y_k = p(t_k) = p(\cos 2\pi\xi_k) = q(\xi_k) = q\left(\frac{2k+1}{4(n+1)}\right) = \sum_{j=-n}^n c_j \exp\left(2\pi i j \frac{2k+1}{4(n+1)}\right).$$

Jetzt unterscheidet sich die Rechnung von der vorherigen, denn wir isolieren $\exp\left(\frac{2\pi i j}{4(n+1)}\right)$:

$$y_k = \sum_{j=-n}^n c_j \exp\left(\frac{2\pi i j}{4(n+1)}\right) \exp\left(\frac{2\pi i j k}{2n+2}\right) \text{ für } k = 0, 1, \dots, n$$

und mit der Notation

$$\zeta_j = c_j \exp\left(\frac{2\pi i j}{4(n+1)}\right) \text{ für } j = -n, \dots, -1, 0, 1, \dots, n$$

erhalten wir

$$y_k = \sum_{j=-n}^n \zeta_j \exp\left(\frac{2\pi i j k}{2n+2}\right) \text{ für } k = 0, 1, \dots, n.$$

Der Nenner $2n+2$ in dieser Formel verleitet zu einer Fourier-Transformation doppelter Länge. Wie in (3.6.37) definieren wir

$$z_j = y_j \text{ für } j = 0, 1, \dots, n$$

und per Symmetrie

$$z_j = z_{2n+1-j} \text{ für } j = n+1, \dots, 2n+1.$$

Somit haben wir

$$z_\ell = \sum_{j=-n}^n \zeta_j \exp\left(2\pi i j \tilde{\xi}_\ell\right) \text{ mit } \tilde{\xi}_\ell = \frac{\ell}{2n+2} \text{ für } \ell = 0, 1, \dots, 2n+1.$$

Jetzt kommt $\omega_{2n+2} = \exp(\frac{2\pi i}{2n+2})$ ins Spiel:

$$z_\ell = \sum_{j=-n}^n \zeta_j \omega_{2n+2}^{j\ell} = \sum_{k=0}^{2n} \zeta_{k-n} \omega_{2n+2}^{(k-n)\ell} = \omega_{2n+2}^{-n\ell} \sum_{k=0}^{2n} \zeta_{k-n} \omega_{2n+2}^{k\ell}$$

oder

$$\omega_{2n+2}^{n\ell} z_\ell = \sum_{k=0}^{2n} \zeta_{k-n} \omega_{2n+2}^{k\ell}.$$

Somit

$$\mathbf{F}_{2n+2}^{-1}[\exp(\frac{\pi i n \ell}{n+1}) z_\ell] = [\zeta_{k-n}].$$

Code 3.6.1: Effiziente Interpolation in den Chebyshev-Knoten

```

1  from numpy import exp, pi, real, hstack, arange
2  from scipy.fft import ifft
3
4  def chebexp(y):
5      """Efficiently compute coefficients  $\alpha_j$  in the Chebychev expansion
6       $p = \sum_{j=0}^n \alpha_j T_j$  of  $p \in \mathbb{C}_n$  based on values  $y_k$ ,
7       $k=0, \dots, n$ , in Chebychev nodes  $t_k$ ,  $k=0, \dots, n$ . These values are
8      passed in the row vector  $y$ .
9      """
10
11     # degree of polynomial
12     n = y.shape[0] - 1
13
14     # create vector z by wrapping and componentwise scaling
15
16     # r.h.s. vector
17     t = arange(0, 2*n+2)
18     z = exp(-pi*1.0j*n/(n+1.0)*t) * hstack([y, y[::-1]])
19
20     # Solve linear system for  $\zeta$  with effort  $O(n \log n)$ 
21     c = ifft(z)
22
23     # recover  $\gamma_j$ 
24     t = arange(-n, n+2)
25     b = real(exp(0.5j*pi/(n+1.0)*t) * c)
26
27     # recover coefficients  $c$  of Chebyshev expansion
28     a = hstack([ b[n], 2*b[n+1:2*n+1] ])
29
30     return a

```

Bemerkung 3.6.2. Bei einem genaueren Blick sind die Formeln in Theorem 2.4.16 und in dieser Implementierung nichts anderes als eine Version der Diskreten Cosinus-Transformation (DCT: eine für reelle, gerade fortgesetzte Funktionen entwickelte, dafür etwas günstiger als die volle FFT implementierbare Version der Fourier-Transformation).

Um die DCT korrekt zu verwenden, muss man in den Spezifikationen der Software nachsehen, was genau implementiert ist. In `scipy 1.14.1` gibt es `scipy.fft.dct`; wenn wir an den Chebyshev-Knoten interpolieren, dann brauchen wir die DCT vom Typ II. Die Messungen müssen den Punkten $x_j = \cos((j + 0.5) * \pi/N)$ für $j = 0, \dots, N - 1$ in dieser Reihenfolge entsprechen. Ohne Normierung müssen wir durch die Anzahl der Punkte N teilen und den ersten Koeffizienten noch halbieren.

Bemerkung 3.6.3. Die Chebyshev-Abszissen sind als die Extrema des Chebyshev-Polynoms T_n definiert:

$$\zeta_k = \cos\left(\frac{k\pi}{n}\right) \text{ für } k = 0, 1, \dots, n.$$

Die Chebyshev-Abszissen enthalten sowohl die Extrema als auch die Nullstellen von T_n und können daher genauso effektiv für die Chebyshev-Interpolation verwendet werden, da sie ebenfalls eine schnelle Berechnung der Chebyshev-Koeffizienten erlauben.

Die **Berechnung der Chebyshev-Koeffizienten** aufgrund der Funktionswerte an den Chebyshev-Knoten wurde mit Theorem 2.4.16 begründet, was eine einfache Konsequenz der Orthogonalität der Chebyshev-Polynome (Theorem 2.4.10) bezüglich des diskreten Skalarprodukts (2.4.24) war. Wie können wir nun die Chebyshev-Koeffizienten aufgrund der Funktionswerte an den **Chebyshev-Abszissen** berechnen? Faszinierenderweise liefert die Orthogonalität der Chebyshev-Polynome bezüglich des kontinuierlichen Skalarprodukts (2.4.23), d.h. mit dem $1/\sqrt{1-x^2}$ -gewichteten Integral (Theorem 2.4.9), die Antwort. Derselbe Variablenwechsel, der beim Beweis der Orthogonalität verwendet wurde, ergibt für $j \neq 0, n$ (diese erfordern noch einen Faktor $1/2$):

$$\frac{\pi}{2} c_j = \int_0^\pi f(\cos(\varphi)) \cos(j\varphi) d\varphi.$$

Wir wollen das Integral auf $[0, 2\pi]$ einer um die Mitte des Intervalls π geraden Funktion haben, damit wir wieder die DCT anwenden können. Wir nehmen die Quantität also zweimal, was einer Spiegelung der Daten am rechten Rand entspricht:

$$2 \cdot \frac{\pi}{2} c_j = \int_0^\pi f(\cos(\varphi)) \cos(j\varphi) d\varphi + \int_\pi^{2\pi} f(\cos(\varphi)) \cos(j\varphi) d\varphi,$$

d.h.

$$c_j = \frac{1}{\pi} \int_0^{2\pi} f(\cos(\varphi)) \cos(j\varphi) d\varphi,$$

was zum folgenden, deutlich einfacheren Code mit FFT führt. Somit können wir auch an den Chebyshev-Abszissen sehr effizient und stabil interpolieren.

Code 3.6.4: Effiziente Interpolation an den Chebyshev-Abszissen

```
from numpy import exp, pi, real, hstack, arange, zeros, flipud, cos, linspace,
real
2 from scipy.fft import ifft, fft
```

```

4 def chebexp(y):
6     r"""Efficiently compute coefficients  $\alpha_j$  in the Chebyshev expansion
         $p = \sum_{j=0}^n \alpha_j T_j$  of  $p$  in  $C_p$  based on values  $y_k$ ,
         $k=0, \dots, n$ , in Chebyshev abscissas  $t_k$ ,  $k=0, \dots, n$ . These values
    are
8     passed in the row vector y.
        """

10     # degree of polynomial
12     n = y.shape[0]-1
        # create vector z by wrapping: complex values!
14     z = zeros(2*n, dtype=complex)
        z[:n+1] = 1.*y
16     z[n+1:] = flipud(y)[1:-1]
        c = fft(z)/(2.*n)

18     a = 1.*c[:n+1]
20     a[0] *= 0.5
        a[-1] *= 0.5

22     return a

24 def evalchebexp(a,N):
26     n = a.shape[0]-1
        c = zeros(2*N, dtype=complex)
28     c[:n+1] = 1.*a
        c[0] *= 2
30     c[n] *= 2
        c[2*N-n+1:] = c[n-1:0:-1]
32     z = ifft(c)*2*N
        y = 1.*z[:N+1]
34     x = cos(arange(N+1)*pi/N)
        return x,y

36 def rungef(x):
38     """Compute the value of the function of Runge's counterexample.
        """

40     return 1.0 / (1.0 + x**2)

42 def step(x):
44     return where(abs(x-0.5)<0.25, 1., 0.)

46 def ramp(x):
48     return where(x<1., x, 0.)

50 def hat(x):
52     return where(abs(x)<=1., 1-abs(x), 0.)

54 def morf(x):
56     return where(abs(x)<=1, 0.5*(1+cos(pi*x)), 0.)

58 def smooth(x):
60     return 1./sqrt(1+0.5*sin(2*pi*x))

62 if __name__ == "__main__":

```



```

56 f = rungef
57 n = 10
58 x = 5*cos(arange(n+1)*pi/n) # Chebyshev points
59 y = f(x)
60 c = real(chebexp(y))

62 t = linspace(-5.0, 5.0, 201)
63 N = 200
64 xN, yN = real(evalchebexp(c,N))

66 from pylab import plot, legend, xlabel, show, grid
67 plot(t, f(t), "r")
68 plot(5*xN,yN,"b--")
69 plot(x, y, "k*")
70 grid(True)
71 legend(["Function","Chebychev"])
72 xlabel("t")
73 show()

```

Das ist nichts anderes als die DCT vom Typ I mit den entsprechenden Skalierungen und der Behandlung der Endkoeffizienten.

Die **Auswertung** kann wie in den oberen Codes via FFT und "Zero-Padding" erfolgen, allerdings dann nur auf feineren Chebyshev-Gittern (Knoten oder Abszissen). Die Kosten betragen in diesem Fall $\mathcal{O}(n \log(n))$ für die Berechnung der Koeffizienten und $\mathcal{O}(M \log(M))$ für die Auswertung auf einem feineren Chebyshev-Gitter mit M Punkten. Wenn wir Auswertungen auf beliebigen Gittern der Länge M brauchen, müssen wir auf den Clenshaw-Algorithmus zurückgreifen, was $\mathcal{O}(nM)$ Operationen benötigt. Die Konvergenzkurven in Abbildung 3.6.5 wurden mit den Diskreten Cosinus-Transformationen und dem Clenshaw-Algorithmus für dieselben Testfunktionen wie am Ende von Abschnitt 2.4.2 berechnet; sie sehen nicht viel anders aus als die in Abbildung 2.4.20.

Convergence of Chebyshev Interpolation (Extrema, DCT-I) on [0.0,2.0]

Convergence of Chebyshev Interpolation (Zeros, DCT-II) on [0.0,2.0]

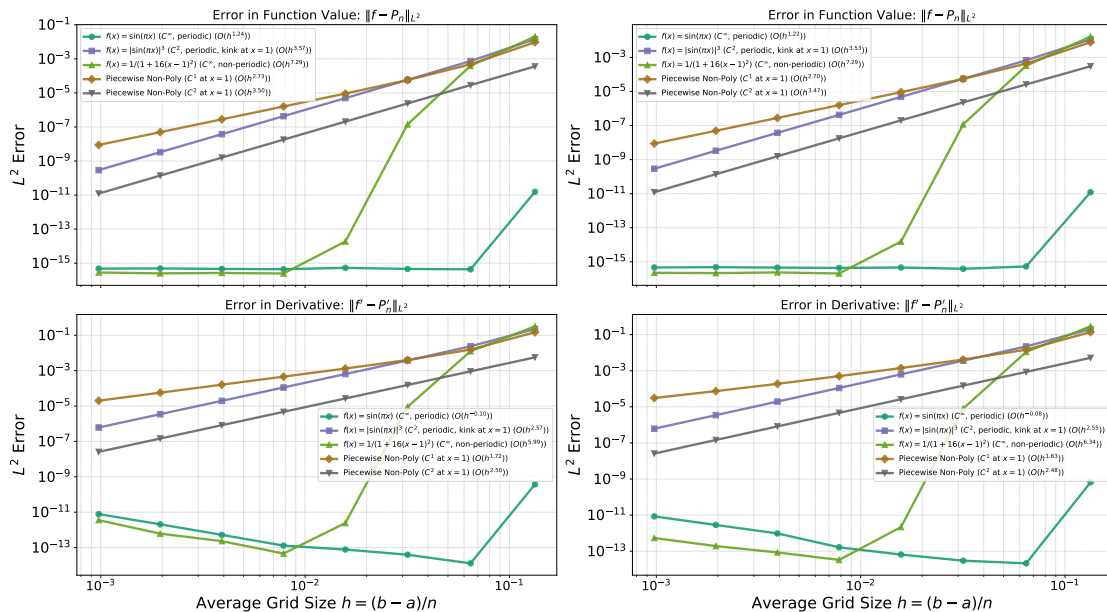


Abb. 3.6.5. Konvergenz für einige Testfunktionen

Chapter 4

Stückweise Polynomiale Interpolation

Globale Interpolation mit Polynomen funktioniert nur dann gut, wenn wir die Interpolationspunkte als die Chebyshev-Knoten oder -Abszissen verwenden und wenn die Funktion sehr glatt ist. Die Fehlerabschätzung (2.2.10) aus Theorem 2.2.12 weist auf eine neue Idee hin: Wir zerlegen das ganze Intervall $[a, b]$ in viele kleine Intervalle (möglicherweise von verschiedener Länge) mit Knoten ("breakpoints") $a = x_0 < x_1 < \dots < x_N = b$, sodass die maximale Subintervalllänge $h = \max_{j=1, \dots, N} (x_j - x_{j-1})$ als zusätzlicher Parameter zur Verfügung steht. h ist ein Parameter des Gitters von Knoten ("breakpoints") $\{x_0, x_1, \dots, x_N\}$. Wenn wir auf jedem kleinen Intervall die Polynominterpolation mit $n+1$ Punkten anwenden, erhalten wir einen Interpolanten s , der stückweise (d.h. auf jedem Intervall $[x_{j-1}, x_j]$) ein Polynom vom Grad n ist; die Fehlerabschätzung (2.2.11) ergibt

$$|f(x) - s(x)| < \frac{h^{n+1}}{(n+1)!} \|f^{(n+1)}\|_{\infty}. \quad (4.0.1)$$

Das ist zwar keine exponentielle Konvergenz in h , jedoch ermöglicht eine flexible Wahl des Gitters und den praktischen Umständen angepasste Interpolationen. Im Folgenden beschreiben wir einige konkrete Ansätze, die man oft verwendet.

4.1 Stückweise lineare Interpolation

Seien die $N+1$ Messungen $(x_j, y_j)_{j=0, \dots, N}$ gegeben. Wir verwenden die Messpunkte als Knoten ("breakpoints"). Wir notieren die Subintervalllängen $h_j = x_j - x_{j-1}$ und konstruieren den linearen Interpolanten für jedes Intervall:

$$s_j(x) = y_{j-1} \frac{x_j - x}{h_j} + y_j \frac{x - x_{j-1}}{h_j}, \text{ für } x \in [x_{j-1}, x_j]. \quad (4.1.2)$$

Diese Formel ist zwar offensichtlich, es ist aber nützlich, diese Konstruktion langsamer anzugehen. Wir begründen diese Formel nun, indem wir der folgenden Konstruktion folgen:

Sei für $t \in [0, 1]$ die Funktion $\chi(t) = t$. Da $\chi(0) = 0$ und $\chi(1) = 1$ gilt, hat die Funktion $f(t) = y_0\chi(1-t) + y_1\chi(t)$ die Interpolationseigenschaften $f(0) = y_0$ und $f(1) = y_1$ und ist linear in t . Die entsprechende lineare Interpolation $s_j(x)$ auf $[x_{j-1}, x_j]$ entsteht aus f mit dem Variablenwechsel

$$\begin{aligned} t \in [0, 1] &\longleftrightarrow x \in [x_{j-1}, x_j] \\ t = \frac{x - x_{j-1}}{h_j} &\longleftrightarrow x = x_{j-1} + h_j t, \end{aligned}$$

d.h.

$$s_j(x) = y_{j-1}\chi\left(\frac{x_j - x}{h_j}\right) + y_j\chi\left(\frac{x - x_{j-1}}{h_j}\right), \text{ für } x \in [x_{j-1}, x_j]. \quad (4.1.3)$$

Diese Interpolation ist lokal: Wenn $y_k = \delta_{kj}$ für $k = 0, \dots, N$, dann ist der Interpolant s überall 0 ausser im Intervall $[x_{j-1}, x_{j+1}]$. Die so konstruierte Funktion s ist auf dem ganzen Intervall $[a, b]$ stetig, aber an den Knoten nicht differenzierbar.

4.2 Kubische Hermite-Interpolation

Wir wollen nun eine Funktion s konstruieren, die auf $[a, b]$ stetig differenzierbar ist und auf den Subintervallen $[x_{j-1}, x_j]$ stückweise ein Polynom vom Grad 3 ist. Nehmen wir an, wir kennen die Funktionswerte y_j und die Werte der Ableitung der Funktion, c_j , an den Messpunkten $x_0, \dots, x_N$. Dann stellen die Bedingungen

$$\begin{aligned} s(x_j) &= y_j, j = 0, 1, \dots, N \\ s'(x_j) &= c_j, j = 0, 1, \dots, N \end{aligned}$$

automatisch sicher, dass s auf $[a, b]$ stetig differenzierbar ist. Gleichzeitig haben wir vier Bedingungen für jedes Subintervall $[x_{j-1}, x_j]$, auf dem wir ein Polynom vom Grad 3, also mit vier Parametern, konstruieren müssen. Nun konstruieren wir dieses Polynom ähnlich wie im linearen Fall.

Für $t \in [0, 1]$ betrachten wir die Funktion $\varphi(t) = t^2(3-2t)$; sie hat die Eigenschaften $\varphi(0) = 0$ und $\varphi(1) = 1$, somit erfüllt $f(t) = y_0\varphi(1-t) + y_1\varphi(t)$ die Interpolationseigenschaften $f(0) = y_0$ und $f(1) = y_1$. Der Variablenwechsel

$$\begin{aligned} t \in [0, 1] &\longleftrightarrow x \in [x_{j-1}, x_j] \\ t = \frac{x - x_{j-1}}{h_j} &\longleftrightarrow x = x_{j-1} + h_j t \end{aligned}$$

liefert

$$p_j(x) = y_{j-1}\varphi\left(\frac{x_j - x}{h_j}\right) + y_j\varphi\left(\frac{x - x_{j-1}}{h_j}\right), \text{ für } x \in [x_{j-1}, x_j]. \quad (4.2.4)$$

Jetzt müssen wir die Ableitungen ins Spiel bringen. Aus $\varphi(t) = 3t^2 - 2t^3$ folgt $\varphi'(t) = 6t - 6t^2 = 6t(1-t)$, d.h. φ' verschwindet an den Stellen $t = 0$ und $t = 1$.

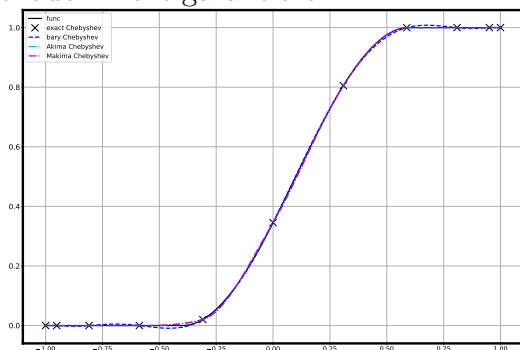
Somit verschwindet auch die Ableitung von p_j an den Stellen x_{j-1} und x_j , d.h. p_j leistet keinen Beitrag zu den Ableitungen von s an den Knoten ("breakpoints") x_{j-1} und x_j . Für die Ableitungen brauchen wir eine zweite Funktion $\psi(t) = t^2(t-1) = t^3 - t^2$. Diese verschwindet an den Stellen $t = 0$ und $t = 1$ und hat die Ableitung $\psi'(t) = 3t^2 - 2t = t(3t-2)$, sodass $\psi'(0) = 0$ und $\psi'(1) = 1$ gilt. Somit erfüllt $\psi_j(t) = c_j\psi(t) + c_{j-1}\psi(1-t)$ die Bedingungen $\psi'_j(0) = c_{j-1}$ und $\psi'_j(1) = c_j$. Wir verwenden nun den Variablenwechsel von vorhin, um die Funktion ψ vom Referenzintervall $[0, 1]$ auf das jeweilige Intervall $[x_{j-1}, x_j]$ zu übertragen. Dabei müssen wir nun die Kettenregel beachten: $\frac{d}{dx}\psi(t(x)) = \psi'(t) \cdot t'(x)$, d.h. für $x \in [x_{j-1}, x_j]$: $\frac{d}{dx}\psi(t(x)) = \psi'(t)/h_j$. Das ergibt den Ansatz:

$$q_j(x) = c_j h_j \psi\left(\frac{x - x_{j-1}}{h_j}\right) - c_{j-1} h_j \psi\left(\frac{x_j - x}{h_j}\right), \text{ für } x \in [x_{j-1}, x_j]. \quad (4.2.5)$$

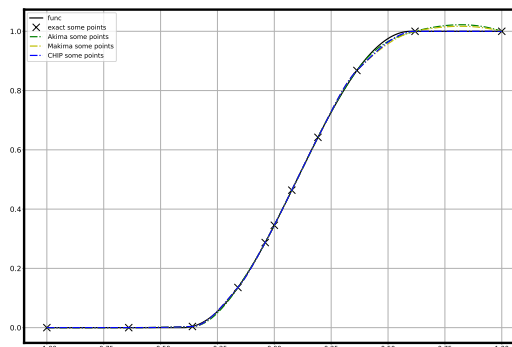
Die Interpolationsfunktion ist dann einfach die Summe der beiden Teile:

$$s_j(x) = p_j(x) + q_j(x), \text{ für } x \in [x_{j-1}, x_j].$$

Wenn wir also die Werte $(x_j, y_j, c_j)_{j=0,1,\dots,N}$ gegeben haben, dann können wir den Interpolanten s eindeutig konstruieren, der stückweise ein Polynom vom Grad 3 und global stetig differenzierbar ist. Wenn uns nur die Werte $(x_j, y_j)_{j=0,1,\dots,N}$ vorliegen, dann haben wir noch die Freiheit, die c_j zu wählen. Das ergibt einige mögliche solche kubische Hermite-Interpolanten. In `scipy.interpolate` stehen die meistverwendeten kubischen Hermite-Interpolanten zur Verfügung: `Akima1DInterpolator` (und mit `method='makima'` auch eine neuere, modifizierte Variante davon) und `PchipInterpolator`. Man kann beweisen, dass Letzterer ein "formerhaltender" Interpolator ist: Wenn die ursprüngliche Funktion lokal monoton ist, ist der Interpolant, den `PchipInterpolator` liefert, ebenfalls lokal monoton. Das ist in mancher Anwendungen eine wichtige, erwünschte Eigenschaft, die die anderen Interpolationssmethoden nicht garantieren.



Baryzentrische Interpolation und Akima auf Chebyshev-Punkten sind nicht formerhaltend.



CHIP ist formerhaltend.

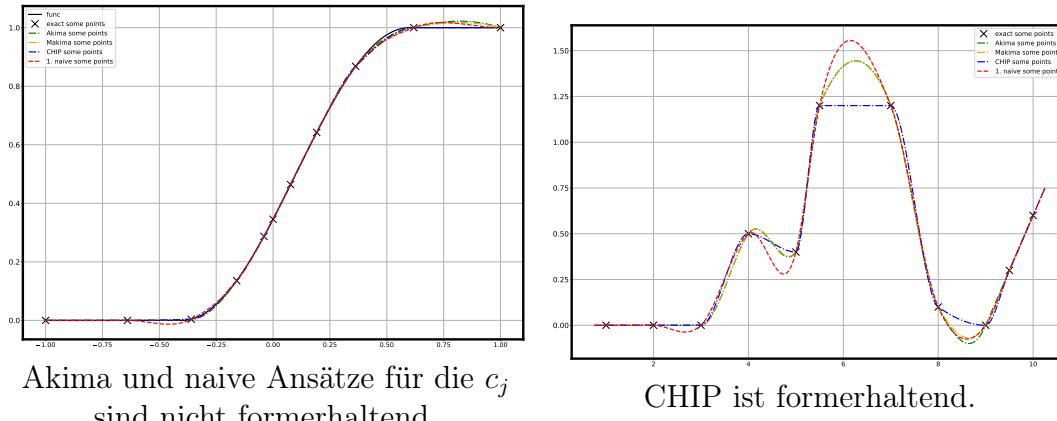


Abb. 4.2.1. Die Monotonieerhaltung wird nur von der CHIP-Interpolationsmethode garantiert.

Für den Fehler kann man beweisen:

Theorem 4.2.2. Sei $f \in C^4[a, b]$ und sei s der stückweise kubische Hermite-Interpolant mit exakten Werten der Ableitungen $s'(x_j) = f'(x_j)$, $s(x_j) = f(x_j)$, $j = 0, 1, \dots, N$, und $s|_{[x_{j-1}, x_j]}$ ist ein Polynom vom Grad 3, $j = 1, \dots, N$. Dann gilt:

$$\|f^{(k)} - s^{(k)}\|_{L^\infty} \leq \frac{1}{384} h^{4-k} \|f^{(4)}\|_{L^\infty},$$

mit $h = \max_{j=1, \dots, N} (x_j - x_{j-1})$ und $k = 0, 1$.

Man beobachtet nur eine deutlich niedrigere Konvergenzordnung von für die CHIP-Methode, bei der die Ableitungen nicht exakt gegeben sind, sondern mittels einer Mittelung (welche die Monotonieerhaltung sicherstellt) konstruiert werden. In den numerischen Experimente, dessen Ergebnisse unten vorliegen, haben wir dieselben Funktionen, die wir in (2.4.28) und (2.4.29) am Ende der Sektion 2.4.2 gesehen haben.

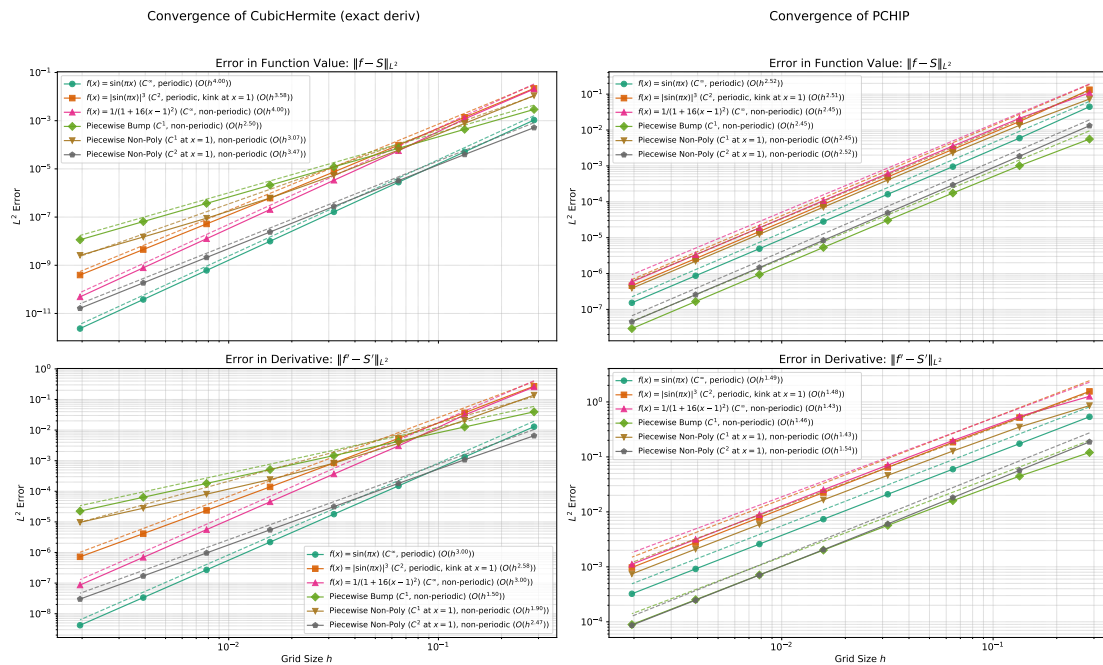


Abb. 4.2.3. Konvergenz für einige Testfunktionen: Kubische Hermite Interpolation mit gegebenen exakten Ableitungen (links) und CHIP (rechts)

4.3 Splines

Definition 4.3.1. (Raum der Splines) Sei das Intervall $[a, b] \subset \mathbb{R}$ und ein Gitter $\mathcal{G} = \{a = x_0 < x_1 < \dots < x_N = b\}$ von Knoten (“breakpoints”) gegeben, und sei $d \in \mathbb{N}, d \geq 1$. Die Menge der auf $[a, b]$ $(d - 1)$ -mal stetig differenzierbaren Funktionen, die auf dem Gitter $\mathcal{G}$ aus stückweisen Polynomen vom Grad höchstens d bestehen,

$$\mathcal{S}_{d,\mathcal{G}} = \{s \in C^{d-1}[a, b], \quad s_j := s|_{[x_{j-1}, x_j]} \text{ ist ein Polynom vom Grad höchstens } d\},$$

heisst der Raum der Splines vom Grad d (von der Ordnung $d + 1$).

Bemerkung 4.3.2. Für $d = 0$ funktioniert diese Definition nicht, aber man kann $\mathcal{S}_{0,\mathcal{G}}$ als die Menge der stückweise konstanten Funktionen ansehen. Für $d = 1$ ist $\mathcal{S}_{1,\mathcal{G}}$ die Menge der stückweise linearen Funktionen, die auf $[a, b]$ stetig sind. Für $d = 3$ (kubische Splines) enthält $\mathcal{S}_{3,\mathcal{G}}$ Funktionen, die *zweimal* stetig differenzierbar sind und stückweise Polynome vom Grad 3 sind. Dadurch unterscheiden sich diese Funktionen von den kubischen Hermite-Interpolanten aus Abschnitt 4.2, die nur einmal stetig differenzierbar waren.

Bemerkung 4.3.3. Offensichtlich ist $\mathcal{S}_{d,\mathcal{G}}$ ein linearer Raum, und ein formales Zählargument ergibt die Anzahl der Freiheitsgrade (Anzahl der Polynomkoeffizienten minus Anzahl der Stetigkeitsbedingungen):

$$\dim \mathcal{S}_{d,\mathcal{G}} = N(d + 1) - (N - 1)d = N + d.$$

Meistens werden in den Anwendungen kubische Splines ($d = 3$) verwendet. In diesem Fall haben wir $\dim \mathcal{S}_{3,\mathcal{G}} = N + 3$, aber nur $N + 1$ vorgegebene Funktionswerte $(y_0, y_1, \dots, y_N)$, so dass noch zwei Freiheitsgrade übrig bleiben, die wir je nach Anwendung wählen müssen.

Notieren wir wie zuvor die lokale Schrittweite $h_j = x_j - x_{j-1}$ und die Variable $t = (x - x_{j-1})/h_j$ im Referenzintervall $[0, 1]$ für $x \in [x_{j-1}, x_j]$. Die Konstruktion aus Abschnitt 4.2 liefert

$$\begin{aligned} s_j(x) = s|_{[x_{j-1}, x_j]}(x) &= s(x_{j-1})(1 - 3t^2 + 2t^3) + s(x_j)(3t^2 - 2t^3) + \\ &\quad h_j s'(x_{j-1})(t - 2t^2 + t^3) + h_j s'(x_j)(-t^2 + t^3). \end{aligned}$$

Damit ist s (per Konstruktion) einmal stetig differenzierbar, und wir müssen noch die Steigungen $s'(x_j)$, $j = 0, \dots, N$, an den Knoten (“breakpoints”) des Gitters $\mathcal{G}$ bestimmen. Die Bedingung, dass s'' an allen inneren Knoten x_j stetig ist, wird die Wahl der Steigungen bestimmen:

$$s''|_{[x_{j-1}, x_j]}(x_j) = s''|_{[x_j, x_{j+1}]}(x_j), \quad \text{für } j = 1, \dots, N - 1.$$

Eine längere Rechnung, bei der die Kettenregel beim Ableiten eine wichtige Rolle spielt, ergibt ein lineares Gleichungssystem mit $N + 1$ Unbekannten $c_j = s'(x_j)$ für $j = 0, \dots, N$ und $N - 1$ Gleichungen; die j -te Gleichung ($j = 1, \dots, N - 1$) ist:

$$\frac{1}{h_j}c_{j-1} + \left(\frac{2}{h_j} + \frac{2}{h_{j+1}}\right)c_j + \frac{1}{h_{j+1}}c_{j+1} = 3\left(\frac{y_j - y_{j-1}}{h_j^2} + \frac{y_{j+1} - y_j}{h_{j+1}^2}\right).$$

Das entstandene lineare Gleichungssystem sieht so aus:

$$\begin{bmatrix} b_0 & a_1 & b_1 & 0 & \cdots & \cdots & 0 \\ 0 & b_1 & a_2 & b_2 & & & \\ & 0 & \ddots & \ddots & \ddots & & \\ \vdots & & & \ddots & \ddots & \ddots & \\ & & & & \ddots & a_{N-2} & b_{N-2} & 0 \\ 0 & \cdots & \cdots & 0 & b_{N-2} & a_{N-1} & b_{N-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \\ c_N \end{bmatrix} = \begin{bmatrix} 3\left(\frac{y_1 - y_0}{h_1^2} + \frac{y_2 - y_1}{h_2^2}\right) \\ \vdots \\ 3\left(\frac{y_{N-1} - y_{N-2}}{h_{N-1}^2} + \frac{y_N - y_{N-1}}{h_N^2}\right) \end{bmatrix}. \quad (4.3.6)$$

mit

$$b_j := \frac{1}{h_{j+1}}, \quad a_j := \frac{2}{h_j} + \frac{2}{h_{j+1}}, \quad j = 0, 1, \dots, N - 1, \quad [\quad b_j, a_j > 0 \quad , \quad a_j = 2(b_j + b_{j-1}) \quad].$$

Wir verändern das lineare Gleichungssystem so, dass wir eine eindeutige Lösung erhalten. Meistens wird einer der folgenden Ansätze verwendet; alle Ansätze beinhalten nur lineare Bedingungen in den Daten.

Definition 4.3.4. (Vollständige kubische Spline-Interpolation) Wir sprechen von vollständiger kubischer Spline-Interpolation, wenn wir die zusätzlichen Bedingungen

$$s'(x_0) = c_0, \quad s'(x_N) = c_N$$

mit vorgegebenen c_0 und c_N stellen. Ein alternativer Name ist “eingespannte” (clamped) kubische Spline.

Im Fall der vollständigen kubischen Spline-Interpolation können wir die erste und die letzte Spalte aus der Systemmatrix (4.3.6) weglassen, indem wir die Terme mit c_0 und c_N auf die rechte Seite verschieben:

$$\begin{bmatrix} a_1 & b_1 & 0 & \cdots & \cdots & 0 \\ b_1 & a_2 & b_2 & & & \\ 0 & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ & & & \ddots & a_{N-2} & b_{N-2} \\ 0 & \cdots & \cdots & 0 & b_{N-2} & a_{N-1} \end{bmatrix} \begin{bmatrix} c_1 \\ \vdots \\ c_{N-1} \end{bmatrix} = \begin{bmatrix} 3\left(\frac{y_1 - y_0}{h_1^2} + \frac{y_2 - y_1}{h_2^2}\right) - c_0 b_0 \\ \vdots \\ 3\left(\frac{y_{N-1} - y_{N-2}}{h_{N-1}^2} + \frac{y_N - y_{N-1}}{h_N^2}\right) - c_N b_{N-1} \end{bmatrix}. \quad (4.3.7)$$

Definition 4.3.5. (Natürliche kubische Spline-Interpolation) Wir sprechen von natürlicher kubischer Spline-Interpolation, wenn wir die zusätzlichen Bedingungen

$$s''(x_0) = 0, \quad s''(x_N) = 0$$

stellen.

Im Fall der natürlichen kubischen Spline-Interpolation fügen wir also zwei Gleichungen hinzu:

$$\frac{2}{h_1}c_0 + \frac{1}{h_1}c_1 = 3 \frac{y_1 - y_0}{h_1^2}, \quad \frac{1}{h_n}c_{n-1} + \frac{2}{h_n}c_n = 3 \frac{y_n - y_{n-1}}{h_n^2}.$$

Das führt wieder zu einem linearen Gleichungssystem mit einer tridiagonalen, symmetrischen, positiv definiten Systemmatrix:

$$\begin{bmatrix} 2 & 1 & 0 & \cdots & & \cdots & 0 \\ b_0 & a_1 & b_1 & 0 & \cdots & \cdots & 0 \\ 0 & b_1 & a_2 & b_2 & & & \\ & 0 & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & & \ddots & \ddots & \ddots & \\ & & & & \ddots & a_{N-2} & b_{N-2} & 0 \\ 0 & \cdots & \cdots & 0 & b_{N-2} & a_{N-1} & b_{N-1} \\ 0 & \cdots & & \cdots & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} c_0 \\ \vdots \\ c_N \end{bmatrix} = 3 \begin{bmatrix} \frac{y_1 - y_0}{h_1} \\ \frac{y_1 - y_0}{h_1^2} + \frac{y_2 - y_1}{h_2^2} \\ \vdots \\ \frac{y_{N-1} - y_{N-2}}{h_{N-1}^2} + \frac{y_N - y_{N-1}}{h_N^2} \\ \frac{y_N - y_{N-1}}{h_N} \end{bmatrix}. \quad (4.3.8)$$

Definition 4.3.6. (Periodische kubische Spline-Interpolation) Wir sprechen von periodischer kubischer Spline-Interpolation, wenn wir die zusätzlichen Bedingungen

$$s'(x_0) = s'(x_N), \quad s''(x_0) = s''(x_N)$$

stellen.

Periodische kubische Spline-Interpolation ist nur sinnvoll, wenn $y_0 = y_N$; wir nehmen hier also an, dass $y_0 = y_N$ gilt. Im Fall der periodischen kubischen Spline-Interpolation haben wir also eine Unbekannte weniger, $c_0 = c_N$, und eine Gleichung mehr, so dass sich die folgende Systemmatrix ergibt:

$$\mathbf{A} := \begin{bmatrix} a_1 & b_1 & 0 & \cdots & 0 & b_0 \\ b_1 & a_2 & b_2 & & & 0 \\ 0 & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ 0 & & & \ddots & a_{N-1} & b_{N-1} \\ b_0 & 0 & \cdots & 0 & b_{N-1} & a_0 \end{bmatrix}, \quad \begin{aligned} b_j &:= \frac{1}{h_{j+1}}, & j &= 0, 1, \dots, N-1, \\ a_j &:= \frac{2}{h_j} + \frac{2}{h_{j+1}}, & i &= 0, 1, \dots, N-1. \end{aligned}$$

Bemerkung 4.3.7. In all diesen Fällen kann man das erhaltene lineare Gleichungssystem mit $\mathcal{O}(N)$ -Operationen lösen.

Bemerkung 4.3.8. Ein anderer Ansatz ist die “not-a-knot”-Bedingung:

$$s''' \text{ ist stetig in } x_1 \text{ und } x_{N-1};$$

dabei braucht man mindestens $N = 3$, d.h. 4 Knoten. Da es sich um Polynome vom Grad höchstens 3 handelt, erzwingt diese Bedingung, dass nur ein Polynom in den ersten beiden und ein anderes in den letzten beiden Subintervallen erscheint: $s_1 = s_2$ und $s_{N-1} = s_N$.

Bemerkung 4.3.9. Der natürliche Spline ist diejenige Interpolationsfunktion, welche die Gesamtkrümmung des Funktionsgraphen minimiert:

$$\int_a^b |s''(x)|^2 dx \leq \int_a^b |g''(x)|^2 dx$$

für alle Funktionen g , die zweimal stetig differenzierbar sind und $g(x_j) = y_j$ für $j = 0, \dots, N$ erfüllen.

Theorem 4.3.10. (Interpolationsfehler vollständiger kubischer Splines) Wenn $f \in C^4[a, b]$ und s der vollständige kubische Spline-Interpolant von f auf einem äquidistanten Gitter mit Gitterweite h ist, dann gilt für den Fehler in allen Ableitungen $k = 0, 1, 2, 3$:

$$\|f^{(k)} - s^{(k)}\|_{L^\infty} \leq \frac{5}{384} h^{4-k} \|f^{(4)}\|_{L^\infty}.$$

In `scipy.interpolate` finden wir `CubicSpline`, das implizit zurzeit die “not-a-knot”-Bedingung verwendet; die anderen kubischen Splines kann man auch konstruieren, wenn man die Variable `bc_type` dem Konstruktor entsprechend übergibt. In den numerischen Experimenten, deren Ergebnisse unten vorliegen, haben wir dieselben Funktionen verwendet, die wir in (2.4.28) und (2.4.29) am Ende des Abschnitts 2.4.2 gesehen haben.

Convergence of CubicSpline (clamped)

Convergence of CubicSpline (not-a-knot)

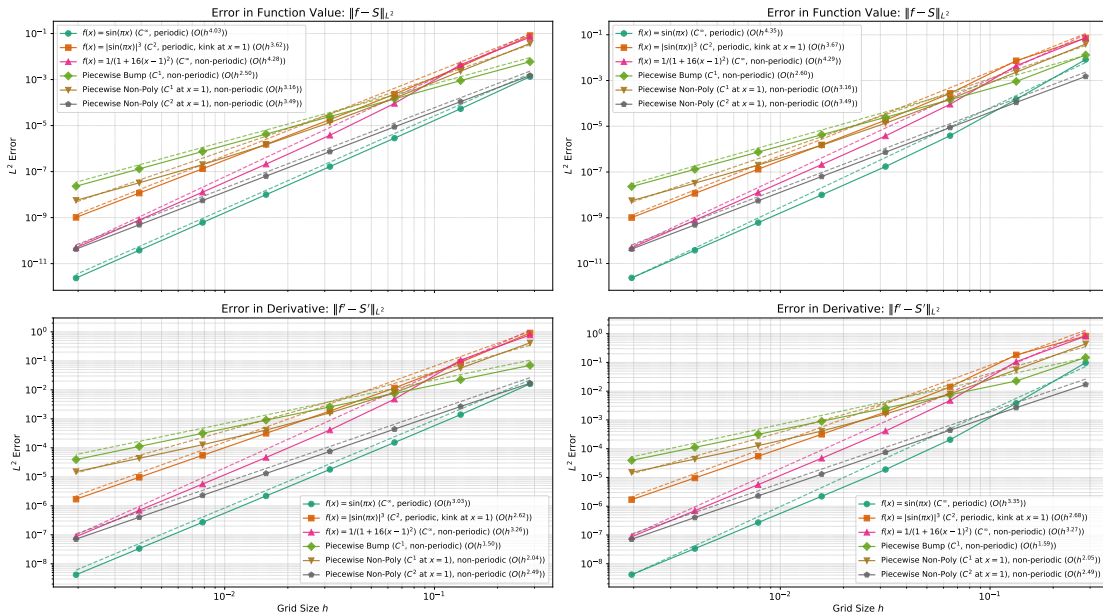


Abb. 4.3.11. Konvergenz der Interpolation mit kubischen Splines für einige Testfunktionen: vollständige (eingespannte) Splines (links) und “not-a-knot”-Ansatz (rechts)

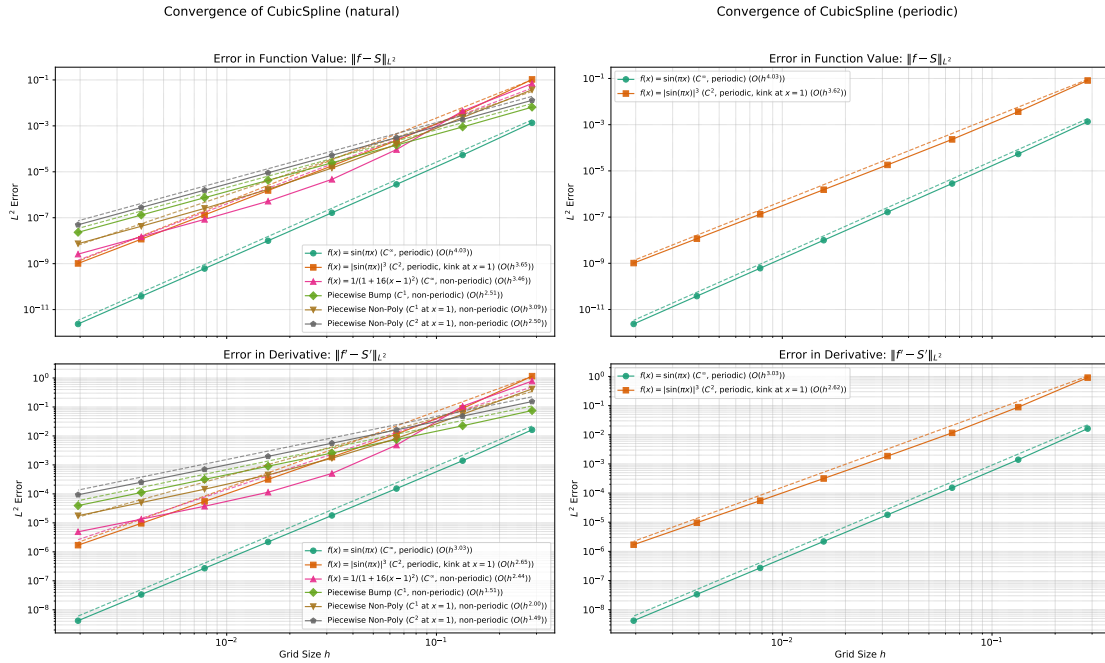


Abb. 4.3.12. Konvergenz der Interpolation mit kubischen Splines für einige Testfunktionen: natürliche (links) und periodische Splines (rechts)

Chapter 5

Numerische Quadratur

5.1 Motivation

In der Vorlesung Physik I lernt man: das mathematische Pendel mit Masse m , Länge ℓ und Anfangsauslenkungswinkel α_0 im Gravitationsfeld mit g hat die Periode $T = 4\sqrt{\frac{\ell}{g}}K(\sin \frac{\alpha_0}{2})$ mit der Spezialfunktion

$$K(a) = \int_0^{\frac{\pi}{2}} \frac{1}{\sqrt{1 - a^2 \sin(s)^2}} ds,$$

deren Werte nicht analytisch, d.h. exakt berechenbar sind. Um diese Formel in den Anwendungen verwenden zu können, muss man in der Lage sein, schnell sehr gute Approximationen der Werte vom Integral zu berechnen. Viele Integrale sind mit analytischen Mitteln nicht oder nur schwer berechenbar. In solchen Fällen bedient man sich der in diesem Kapitel erläuterten Verfahren, um die Integrale näherungsweise zu bestimmen.

Quadratur in einer Dimension ist eine sehr einfache mathematische Aufgabe, die uns erlaubt, viele Ideen und Ansätze kennenzulernen, die von zentraler Bedeutung für andere kompliziertere Probleme sind. Beispiele sind: Elemente von Interpolation und Approximation, Teile und Herrsche, Referenzintervall und Folgen von Variablenwechseln, spektrale Verfahren, polynomiale und exponentielle Konvergenz, Adaptivität.

5.2 Schneller Überblick

In Python kann man die Funktion `quad` aus der Bibliothek `scipy.integrate` verwenden. Wenn man eine eigene Implementierung braucht, dann werden die Trapez- und die Simpson-Regeln oft gebraucht, weil sie sehr einfach und effizient sind. Für bestimmte Anwendungen sind die Gauss-Quadratur-Formeln wichtig. Weniger bekannt, aber viel effizienter als die Gauss-Quadratur sind die Spektralmethoden, die die FFT (Fast Fourier Transform) verwenden, wie z.B. die Clenshaw-Curtis-Regel.

5.3 Grundbegriffe und -ideen

Wir betrachten zuerst Integrale auf einem beschränkten Intervall in $\mathbb{R}$, d.h. Integrale der Form

$$\int_a^b f(x)dx, \text{ mit } f : [a, b] \longrightarrow \mathbb{R}.$$

Unbeschränkte Integrationsintervalle brauchen weitere Techniken, wie Variablenwechsel oder Abschneiden des Intervalls - was wir aus Zeitgründen hier nicht besprechen. Integrale auf Gebieten in $\mathbb{R}^D$ mit $D > 3$ brauchen spezielle Techniken wie dünne Gitter oder (Quasi-) Monte Carlo Methoden - Grundlagen dazu befinden sich am Ende dieses Kapitels, werden aber aus Zeitgründen eventuell nicht besprochen. In den folgenden Beispielen wird f meistens eine einfache Funktion sein, aber es hilft, sich vorzustellen, dass die Auswertung der Funktion f an einer Stelle kostspielig ist, z.B. eine Auswertung kann einen Tag Laborarbeit kosten. Man möchte also sehr genaue Ergebnisse mit wenigen Funktionsauswertungen haben und die verwendeten Algorithmen sollten universell sein, d.h. sie sollten keine speziellen Eigenschaften der Funktion f verwenden. Nur die jeweils minimal nötige Glattheit (integrierbar, stetig oder k -mal stetig differenzierbar - je nach Methode) von f wird angenommen.

Definition 5.3.1. (Quadratur) Das Integral wird durch eine gewichtete Summe von durch die Funktion f an verschiedenen Stellen c_i^n angenommenen Werten approximiert:

$$\int_a^b f(x)dx \approx Q_n(f; a, b) := \sum_{i=1}^n \omega_i^n f(c_i^n). \quad (5.3.1)$$

Hierbei sind die ω_i^n die *Gewichte* und die $c_i^n \in [a, b]$ die *Knoten* der Quadraturformel.

Das Ziel ist also: wähle Knoten $c_i^n \in [a, b]$ und Gewichte ω_i^n , so dass der Fehler

$$E(n) = \left| \int_a^b f(x)dx - Q_n(f; a, b) \right|$$

systematisch klein ist, ohne dass die Rechenkosten (d.h. Anzahl Funktionsauswertungen) zu gross sind. Systematisch bedeutet: die Methoden sollten genauere Ergebnisse für grössere n liefern, d.h. $E(n) \rightarrow 0$ für $n \rightarrow \infty$, und die Verbesserung bei der Verdoppelung der Zahl der Funktionsauswertungen sollte quantifizierbar sein. Man redet oft von der "Geschwindigkeit" der Konvergenz.

Definition 5.3.2. (Fehler)

Der Fehler einer Quadraturformel $Q_n(f)$ ist

$$E(n) := \left| \int_a^b f(x)dx - Q_n(f; a, b) \right|.$$

Wir reden von

algebraischer Konvergenz wenn $E(n) = O(\frac{1}{n^p})$ mit $p > 0$,

und von

exponentieller Konvergenz wenn $E(n) = O(q^n)$ mit $0 \leq q < 1$.

Aufgabe: plotten Sie im selben Bild n^p und q^n für verschiedene $p > 0$ und $0 \leq q < 1$ für $n = 1, \dots, 12$ (eine lin-log Skala kann dabei nützlich sein).

Wie kann man solche Methoden bauen? Hier ist eine mögliche Idee:

Idee: ersetze die komplizierte Funktion durch eine einfachere!

Wir approximieren die Funktion f durch eine einfachere Funktion, deren Integral analytisch berechenbar ist. In der Sprache der linearen Algebra: Wir ersetzen f durch $f_n \in \text{span}\{b_0, b_1, \dots, b_n\}$, wobei die b_j eine Basis eines gewissen Unterraums des Raums der Funktionen auf $[a, b]$ bilden. Daraus resultiert:

$$\int_a^b f(x)dx \approx \int_a^b f_n(x)dx = \int_a^b \left(\sum_{k=0}^n \alpha_k b_k(x) \right) dx = \sum_{k=0}^n \alpha_k \int_a^b b_k(x)dx.$$

Falls die Basisfunktionen die Monome $b_k(x) = x^k$ sind, dann ergibt dieser Ansatz eine Approximation von f durch ein Polynom vom Grad maximal n und die letzten Integrale sind ganz einfach

$$\int_a^b b_k(x)dx = \frac{b^{k+1} - a^{k+1}}{k+1}.$$

Die approximierenden Funktionen sind meistens Polynome. Bei spezifischen Problemen kann jedoch die Wahl anderer Funktionen (z.B. rationaler Funktionen oder Schwingungen verschiedener Frequenzen) von Vorteil sein. Wenn die Basisfunktionen trigonometrische Polynome e^{ikx} mit $k \in \mathbb{Z}$ sind, dann ergibt dieser Ansatz die Fourier-Approximation und man kann die FFT verwenden, um sehr effiziente Algorithmen zu bekommen. In der Analysis haben Sie Riemannsche Summen und eventuell auch die Trapezregel kennengelernt. Die Riemannschen Summen sind nur für analytische Zwecke geeignet, für konkrete Berechnungen sind sie leider nicht gut genug. Wichtige Ergebnisse in der Analysis gab es auch zur Approximation von Funktionen durch Polynome. Wir verwenden hier ein Approximationsergebnis, um mögliche Wege zu zeigen, wie man Algorithmen und Methoden entwickelt, die eine systematische Verbesserung erlauben.

Gegeben die Punkte $x_0, x_1, \dots, x_n$ (z.B. äquidistant – eine sehr einschränkende und wie wir gleich sehen, schlechte, aber vereinfachende Idee, zugegeben), kann man Koeffizienten $\alpha_0, \dots, \alpha_n$ berechnen, so dass

$$p_n(x) = \alpha_0 + \alpha_1 x + \dots + \alpha_n x^n$$

erfüllt

$$p_n(x_j) = f(x_j) \text{ für } j = 0, 1, \dots, n.$$

Das ist ein lineares Gleichungssystem mit einer $(n+1) \times (n+1)$ -Vandermonde-Matrix in den Unbekannten $\alpha_0, \dots, \alpha_n$

$$\begin{bmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}, \quad (5.3.2)$$

was numerisch sehr schlecht konditioniert (d.h. die Rundungsfehler machen das Ergebnis sinnlos) für grosses n ist. Darum ist das kein echter Weg zu einem nützlichen Algorithmus, aber zumindest ergibt dieser Ansatz einen universellen Zugang: f wird durch p_n approximiert und der exakte Wert von $\int_a^b p_n(x)dx$ kann als Approximation von $\int_a^b f(x)dx$ verwendet werden.

Interpolationspolynome können auch einfach via die Lagrange-Polynome erhalten werden.

Definition 5.3.3. (Lagrange-Polynome) Für die Knoten $x_0, x_1, \dots, x_n \in \mathbb{R}$ definieren wir die (Familie von) Polynome(n)

$$\ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}. \quad (5.3.3)$$

Die ℓ_i werden auch die Lagrange-Polynome zu den Stützstellen $x_0, x_1, \dots, x_n$ genannt.

Beispiel 5.3.4. Gegeben seien die Stützstellen x_0, x_1 und x_2 . Wir erhalten die Lagrange-Polynome:

$$\begin{aligned} \ell_0(x) &= \frac{x - x_1}{x_0 - x_1} \cdot \frac{x - x_2}{x_0 - x_2} \\ \ell_1(x) &= \frac{x - x_0}{x_1 - x_0} \cdot \frac{x - x_2}{x_1 - x_2} \\ \ell_2(x) &= \frac{x - x_0}{x_2 - x_0} \cdot \frac{x - x_1}{x_2 - x_1}. \end{aligned}$$

Die Abbildung 5.3.5 zeigt die 5 Lagrange-Polynome zu den 5 Knoten $0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$.

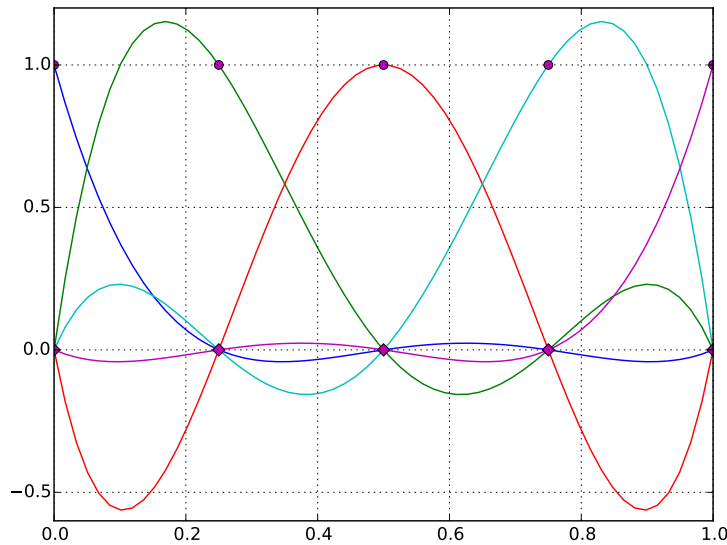


Abb. 5.3.5. Die 5 Lagrange-Polynome zu den Knoten $0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$.

Bemerkung 5.3.6. (Eigenschaften der Lagrange-Polynome)

- $\ell_i(x_j) = 0$ für $j \neq i$,
- $\ell_i(x_i) = 1$,
- $\text{Grad}(\ell_i) = n$,
- $\sum_{i=0}^n \ell_i(x) = 1$ für alle $x \in \mathbb{R}$,
- $\sum_{i=0}^n \ell_i^{(m)}(x) = 0$ für $m \geq 1$,
- $\ell_0, \ell_1, \dots, \ell_n$ bilden eine Basis in $\mathcal{P}_{n+1}$ (dem Raum der Polynome vom Grad maximal n).

Bemerkung 5.3.7. (Quadraturgewichte aus den Lagrange-Polynomen) Gegeben die Funktionswerte $f(x_j)$ an den $n + 1$ Knoten (x_j) lässt sich das Interpolationsspolynom mit den Lagrange-Polynomen sehr einfach bauen:

$$p(x) = \sum_{j=0}^n f(x_j) \ell_j(x).$$

Die Eigenschaften der Lagrange-Polynome sichern nicht nur $p(x_j) = f(x_j)$ für alle $j = 0, 1, \dots, n$, sondern auch die Existenz und Eindeutigkeit der Konstruktion in $\mathcal{P}_{n+1}$. Wenn wir dieses p als Approximation für f verwenden, erhalten wir also eine Quadraturformel mit den Knoten $x_0, x_1, \dots, x_n$ und mit den Gewichten

$$w_j = \int_a^b \ell_j(x), j = 0, 1, \dots, n.$$

Auf Grund der Konstruktion ist diese Quadraturformel exakt für alle Polynome aus $\mathcal{P}_{n+1}$.

Der Quadraturfehler hängt direkt vom Approximationsfehler ab. Angenommen, es gibt keine Rundungsfehler, kann man sich auf analytische Approximationsergebnisse berufen. Man kann beweisen:

Falls f eine n -mal stetig differenzierbare Funktion auf $[a, b]$ ist (kurz geschrieben $f \in C^n[a, b]$), dann liefert diese "Interpolations"-Methode (d.h. via Polynom p_n vom Grad n mit $p_n(x_j) = f(x_j)$, für $j = 0, 1, \dots, n$):

$$\left| \int_a^b f(x) dx - \int_a^b p_n(x) dx \right| \leq \frac{1}{n!} (b-a)^{n+1} \max |f^{(n)}(z)|$$

Wir lesen daraus zwei wichtige Informationen: die Glattheit von f ist wichtig und die Länge des Integrationsintervalls ist wichtig. Probleme mit unglattem f werden schlecht mit diesem Ansatz (Polynominterpolation) integriert. Ein Integral auf einem kurzen Intervall (z.B. $b-a < 1$) lässt sich besser approximieren. So kommen wir auf eine wichtige und praktische

Idee:

Zerlege das Problem in eine Summe ähnlicher, aber einfacher Probleme.

Wir nehmen ein *Gitter* von Punkten im Intervall $[a, b]$. Zur Einfachheit nehmen wir $x_0 = a$ und ein äquidistantes Gitter mit $N+1$ Punkten:

$$x_k = x_0 + kh \text{ mit } h = \frac{b-a}{N} \text{ klein für } N \text{ gross.}$$

Dann zerlegen wir das Problem:

$$\int_a^b f(x) dx = \sum_{k=0}^{N-1} \int_{x_k}^{x_{k+1}} f(x) dx,$$

so dass es reicht, eine Quadraturformel auf jedem kleinen Intervall der Länge h anzuwenden (und dann die Ergebnisse zu addieren). Das ergibt *summierte* Quadraturformeln. Der Parameter h steht uns nun zur Verfügung; mit dem können wir die Approximation kontrollieren. Man spricht über "h-Methoden".

Wir schauen uns jetzt eine Abschätzung des Fehlers an, um eine wichtige Tatsache zu sehen, die in vielen verschiedenen Kontexten in der Numerik nützlich ist. Der Fehler ist:

$$\begin{aligned} & \left| \int_a^b f(x) dx - \sum_{k=0}^{N-1} Q(f, x_k, x_{k+1}) \right| \leq \\ & \sum_{k=0}^{N-1} \left| \int_{x_k}^{x_{k+1}} f(x) dx - Q(f, x_k, x_{k+1}) \right| \leq \\ & \sum_{k=0}^{N-1} \frac{1}{n!} (x_{k+1} - x_k)^{n+1} \max_{z \in [x_k, x_{k+1}]} |f^{(n)}(z)| \leq \\ & C \frac{h^{n+1}}{n!} \sum_{k=0}^{N-1} 1 = C \frac{h^{n+1}}{n!} N = C \frac{h^{n+1}}{n!} \frac{b-a}{h} = C \frac{h^n}{n!} (b-a), \end{aligned}$$

wobei $C = \max_{z \in [a,b]} |f^{(n)}(z)| = \|f^{(n)}\|_{\max}$.

Wir lernen daraus ein wichtiges Prinzip: “Teile und Herrsche” ergibt den lokalen Fehler von der Ordnung h^{n+1} und $N = (b-a)/h$ solche lokalen Fehler addieren sich, so erhalten wir einen globalen Fehler der Ordnung h^n . Ein Faktor h geht also durch die Addition verloren. Auch sehr nützlich in verschiedenen Kontexten ist: die Potenz h^{n+1} kommt aus der Glattheit n auf einem Intervall der Länge h . Der Grad dieser Potenz ist 1 grösser als die Glattheit.

Ausserdem haben wir in diesem Kontext: wir profitieren bei einem glatteren f , aber die Glattheit von f ist vorgegeben. Wir können genauere Ergebnisse erhalten, wenn wir ein kleineres h verwenden, d.h., wenn wir feinere Gitter verwenden. Diese Konvergenz (in h) kann für glattere Funktionen schneller sein. Aus den vorigen Überlegungen sehen wir auch: die Entwicklung und der Begriff der Quadraturformel sollten eigentlich unabhängig vom tatsächlichen Integrationsintervall sein. Mit anderen Worten: wir sollten die Quadraturformel auf einem festen Intervall definieren und deren Eigenschaften untersuchen; ein Variablenwechsel bildet automatisch die Quadraturformel auf jedes beliebige und benötigte Intervall $[x_k, x_{k+1}]$. Wir arbeiten oft auf dem Intervall $[-1, 1]$. Soll eine Funktion auf einem anderen Intervall $[a, b]$ integriert werden, so muss dieses vorher durch eine affine Transformation auf $[-1, 1]$ transformiert werden.

$$\int_a^b f(t) dt = \frac{1}{2}(b-a) \int_{-1}^1 \hat{f}(\tau) d\tau, \quad \hat{f}(\tau) := f\left(\frac{1}{2}(1-\tau)a + \frac{1}{2}(\tau+1)b\right),$$

Für das Referenzintervall $[-1, 1]$ können die Gewichte $\hat{w}_j$ und Knoten $\hat{c}_j$ für eine Quadraturformel bestimmt und tabelliert werden. Diese werden automatisch in Gewichte w_j und Knoten c_j auf dem Intervall $[a, b]$ abgebildet:

$$\int_a^b f(t) dt \approx \frac{1}{2}(b-a) \sum_{j=1}^n \hat{w}_j \hat{f}(\hat{c}_j) = \sum_{j=1}^n w_j f(c_j) \quad \text{mit} \quad \begin{aligned} c_j &= \frac{1}{2}(1-\hat{c}_j)a + \frac{1}{2}(1+\hat{c}_j)b, \\ w_j &= \frac{1}{2}(b-a)\hat{w}_j. \end{aligned}$$

Die Quadraturformeln werden oft verglichen, indem man deren Wirkung auf Polynome misst. Natürlich ist die Integration von Polynomen nicht interessant, aber die eben vorgestellte Idee zur Lösung via Approximation begründet die Verwendung folgender Definition.

Definition 5.3.8. Eine Quadraturformel besitzt Ordnung $n+1$, wenn sie Polynome vom Grad n exakt integriert.

Zu beachten: ein Polynom vom effektiven Grad n wird durch genau $n+1$ Koeffizienten dargestellt.

Anders gesagt: die Quadraturformel besitzt die Ordnung $n + 1$, wenn sie die Integrale aller Monome bis zum Grad n exakt berechnet und das erste falsche Ergebnis bei x^{n+1} vorkommt.

Definition 5.3.9. Eine Quadraturformel auf $[-1; 1]$ mit Knoten $c_1, \dots, c_n$ und Gewichten $\omega_1, \dots, \omega_n$ heisst symmetrisch, falls:

$$\omega_i = \omega_{n+1-i} \text{ und } c_i = -c_{n+1-i} .$$

Bemerkung 5.3.10. Die Mittelpunkts-, Trapez- und Simpson-Regel aus Abschnitt 5.4 sind symmetrisch.

Es gilt:

Theorem 5.3.11. Die Ordnung einer symmetrischen Quadraturformel ist gerade.

Proof. Wir beweisen diesen Satz mittels Induktion.

Induktionsannahme: die Quadraturformel sei exakt für $p \in \mathcal{P}_{2m-2}$.

Induktionsschritt: Sei $f \in \mathcal{P}_{2m-1}$, dann gilt $f(x) = ax^{2m-1} + g(x)$ mit $g(x) \in \mathcal{P}_{2m-2}$, woraus folgt:

$$\begin{aligned} \int_{-1}^1 f(x) dx &= a \int_{-1}^1 x^{2m-1} dx + \int_{-1}^1 g(x) dx = \sum_{j=1}^n \omega_j g(c_j) \\ Q_n(f) &= \sum_{j=1}^n \omega_j f(c_j) = a \sum_{j=1}^n \omega_j c_j^{2m-1} + \sum_{j=1}^n \omega_j g(c_j) . \end{aligned}$$

Es bleibt noch zu zeigen, dass $\sum_{j=1}^n \omega_j c_j^{2m-1} = 0$ für eine symmetrische Quadraturformel. In der Tat:

$$\sum_{j=1}^n \omega_j c_j^{2m-1} = \sum_{j=1}^n \omega_{n+1-j} \underbrace{(-c_{n+1-j})}_{c_k}^{2m-1} = - \sum_{k=1}^n \omega_k c_k^{2m-1} .$$

□

5.4 Äquidistante Stützstellen

Hier betrachten wir die wichtigsten Quadraturformeln mit äquidistanten Knoten.

Mittelpunkt-Regel

Die Mittelpunktsregel ist nichts anderes, als die aus der Analysis bekannte Näherung des bestimmten Integrals:

$$Q^M(f; a, b) = (b - a) f \left(\frac{a+b}{2} \right) . \quad (5.4.4)$$

Die Mittelpunktsregel besitzt Ordnung 2, denn sie ist exakt für lineare Funktionen.

Trapez-Regel

Die nächste Formel der Ordnung 2 ist die aus der Schule bekannte Trapez-Regel:

$$Q^T(f; a, b) = \frac{b-a}{2} (f(a) + f(b)) . \quad (5.4.5)$$

Für den Fehler der Trapez-Regel gilt:

$$E(n) = \left| -\frac{1}{12} (b-a)^3 f^{(2)}(\xi) \right|, \text{ mit einem } \xi \in [a, b] .$$

Simpson-Regel

Die Simpson-Regel ist eine Quadraturformel der Ordnung 4 und bedarf dreier Stützstellen:

$$Q^S(f; a, b) = \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right) . \quad (5.4.6)$$

Während die Trapez-Regel auf linearer Interpolation auf $[a, b]$ basiert, approximieren wir bei der Simpson-Regel die Funktion mit einer Parabel; somit ist die Ordnung der Simpson-Regel mindestens 3, aber da die Quadraturformel symmetrisch ist, ist die Ordnung 4, nach Theorem 5.3.11¹. Für den Fehler der Simpson-Regel gilt:

$$E(n) = \left| -\frac{1}{90} \left(\frac{b-a}{2} \right)^5 f^{(4)}(\xi) \right|, \text{ mit einem } \xi \in [a, b] .$$

Es lohnt sich anzumerken, dass ab Ordnung 8 negative Gewichte auftreten, was die Quadraturformeln aufgrund von Auslöschung numerisch instabil werden lässt. Es handelt sich um dasselbe Phänomen wie bei der Interpolation mittels Polynomen höheren Grades auf äquidistanten Punkten.

Bemerkung 5.4.1. Schranken für den Quadraturfehler bekommt man direkt aus den entsprechenden Resultaten für die Interpolation mittels Lagrange-Polynomen vom Grad $n-1$:

$$f \in C^n([a, b]) \implies \left| \int_a^b f(t) dt - Q_n(f) \right| \leq \frac{1}{n!} (b-a)^{n+1} \|f^{(n)}\|_{L^\infty([a, b])} .$$

Summierte Quadraturformeln

Eine äusserst einfache Möglichkeit, die Genauigkeit der Integration zu verbessern, beruht auf dem “zerlege und herrsche” (“divide and conquer”)-Prinzip. Man unterteilt das Intervall in Subintervalle und wendet auf jedes Subintervall die Quadraturformel an. Die Ergebnisse werden summiert.² Formal lautet diese Aussage:

$$\int_a^b f(x) dx = \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} f(x) dx = \sum_{i=0}^{N-1} Q_n(f; x_i, x_{i+1}) ,$$

¹Aus demselben Grund hat die Mittelpunktsregel auch Ordnung 2 und nicht 1

²Dies ist analog zu den Riemannschen Summen in der Analysis

wobei hier das Intervall $[a, b]$ in N Subintervalle unterteilt wurde. Konkret gilt für $h = \frac{b-a}{N}$ und $x_0 = a$, $x_i = x_0 + ih$, $x_N = b$, folgendes:

Summierte Mittelpunkt-Regel:

$$I(f; a, b) \approx \sum_{i=0}^{N-1} h f\left(\frac{x_i + x_{i+1}}{2}\right), \quad (5.4.7)$$

Summierte Trapez-Regel:

$$I(f; a, b) \approx \sum_{i=0}^{N-1} \frac{h}{2} (f(x_i) + f(x_{i+1})) = \frac{h}{2} \left(f(a) + 2 \sum_{i=1}^{N-1} f(x_i) + f(b) \right). \quad (5.4.8)$$

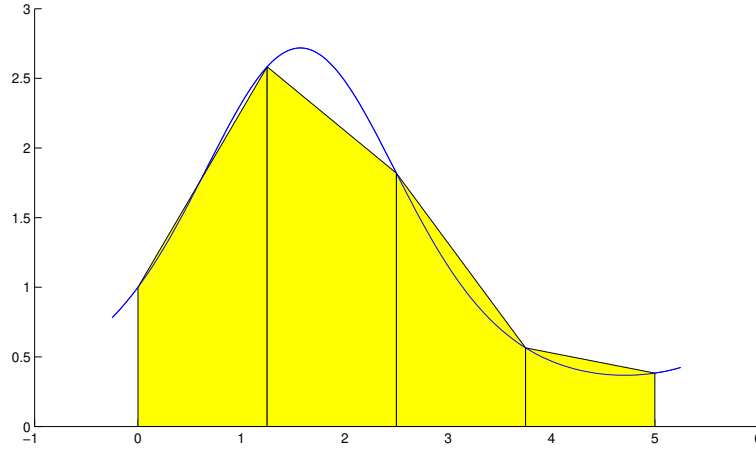


Abb. 5.4.2. Summierte Trapezregel: die Funktion wird durch eine stückweise lineare, aber global stetige Funktion approximiert.

Für den Fehler gilt:

$$E(n) \leq \frac{h^2}{12} (b - a) \max_{x \in [a, b]} |f''(x)|.$$

Summierte Simpson-Regel

$$\begin{aligned} I(f; a, b) &\approx \sum_{i=0}^{N-1} \frac{h}{6} \left(f(x_i) + 4f\left(\frac{x_i + x_{i+1}}{2}\right) + f(x_{i+1}) \right) \\ &= \frac{h}{6} \left(f(a) + 2 \sum_{i=1}^{N-1} f(x_i) + 4 \sum_{i=1}^N f\left(\frac{x_{i-1} + x_i}{2}\right) + f(b) \right) \\ &= \frac{\tilde{h}}{3} \left(f(\tilde{x}_0) + 2 \sum_{i=1}^{N-1} f(\tilde{x}_{2i}) + 4 \sum_{i=1}^N f(\tilde{x}_{2i-1}) + f(\tilde{x}_{2N}) \right), \end{aligned} \quad (5.4.9)$$

wobei $\tilde{h} = \frac{h}{2}$ und $\tilde{x}_{2i} = x_i$ und $\tilde{x}_{2i-1} = \frac{x_{i-1} + x_i}{2}$.

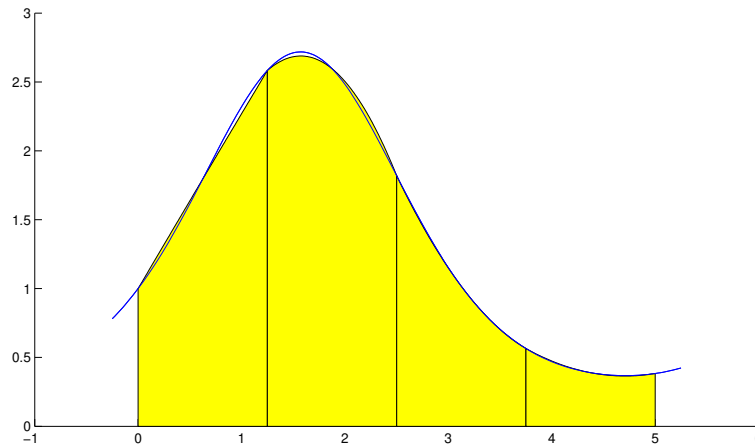


Abb. 5.4.3. Simpson-Regel: die approximierende Funktion ist stückweise quadratisch und global stetig.

Theorem 5.4.4. (Konvergenz der zusammengesetzten Quadraturformeln)

Für eine zusammengesetzte Quadraturformel Q , die auf einer lokalen Quadraturformel der Ordnung $p \in \mathbb{N}$ basiert, gibt es ein $C > 0$, so dass

$$\left| \int_I f(t) dt - Q(f) \right| \leq Ch^p \|f^{(p)}\|_{L^\infty(I)} \quad \text{für alle } f \in C^p(I). \quad (5.4.10)$$

Beispiel 5.4.5. (Konvergenz der zusammengesetzten Quadraturformeln)

Wir untersuchen nun experimentell zusammengesetzte Quadraturformeln basierend auf:

- Trapez-Regel: lokale Ordnung 2 (exakt für lineare Funktionen),
- Simpson-Regel: lokale Ordnung 4

auf einem äquidistanten Gitter $\mathcal{M} := \{jh\}_{j=0}^n$, $h = \frac{1}{n}$, $n \in \mathbb{N}$.

Code 5.4.6: Zusammengesetzte Trapez-Regel

```
from numpy import linspace, sum

def trapezoidal(f, a, b, N):
    r"""
        Trapezoidal quadrature of function f from a to b with N subintervals.

        f:      Function f(x)
        a, b:    Bounds of the integration interval
        N:      Number of subintervals
    """
    x, h = linspace(a, b, int(N)+1, retstep=True)
    # quadrature weights:
    # boundary nodes: w=1/2
    # internal nodes: w=1
```

```

16     I = h/2.0 * (f(x[0]) + 2.0*sum(f(x[1:-1])) + f(x[-1]))
    """#note: this is a compact way of writing. One could also use a for-cycle:
    I = 0.0
18     for k in range(N):
    I = I + h/2.0 * (f(x[k+1]) + f(x[k]))
20     #the two strategies are equivalent
    but MUCH slower, so AVOID loops, if possible!
22     """
    return I

24
25 if __name__ == "__main__":
26     from numpy import arange, size, zeros, log, polyfit
    from scipy import integrate
28
    # Define a function and an interval:
30     f = lambda x: 1.0 / (1.0 + (5*x)**2)
    left = 0.0
32     right = 1.0
34
    # Exact integration with scipy.integrate.quad:
    exact, e = integrate.quad(f, left, right)
36
    # Trapezoidal rule for different number of quadrature points
38     N = arange(2, 101)
    res = zeros(size(N))
40     for i, n in enumerate(N):
    res[i] = trapezoidal(f, left, right, n)
42
    err = abs(res - exact)
44
    # Linear fit to determine convergence order
46     p = polyfit(log(N[-50:]), log(err[-50:]), 1)
    print(("Convergence order: %f" % -p[0]))

```

Code 5.4.7: zusammengesetzte Simpson-Regel

```

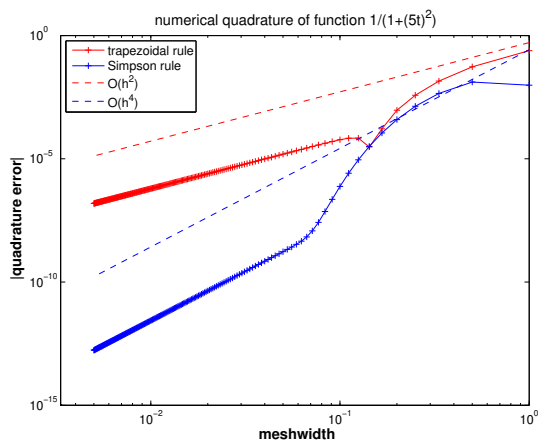
1  from numpy import linspace, sum
3  def simpson(f, a, b, N):
    r"""
5      Simpson quadrature of function f from a to b with N subintervals.
6
7      f:      Function f(x)
    a, b:  Bounds of the integration interval
9      N:      Number of subintervals
    so this method uses 2*N+1 function evaluations
11     """
    x, h = linspace(a, b, 2*int(N)+1, retstep=True)
13     # quadrature weights:
    # boundary nodes:  w=1/3
15     # internal nodes: w=2/3
    # midpoint nodes:  w=4/3
17     I = h/3.0 * sum(f(x[:-2:2]) + 4.0*f(x[1:-1:2]) + f(x[2::2]))

```

```

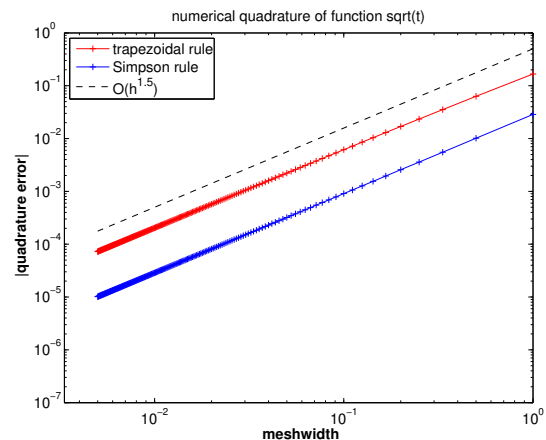
19     """equivalent, less compact way of writing:
        I = 0.0
        for k in range(0,2*N,2):
21         I = I + (2.0*h)/6.0 * (f(x[k]) + 4.0 * f(x[k+1]) + f(x[k+2]))"""
        return I
23
24 if __name__ == "__main__":
25     from numpy import arange, size, zeros, log, polyfit
26     from scipy import integrate
27
28     # Define a function and an interval:
29     f = lambda x: 1.0 / (1.0 + (5*x)**2)
30     left = 0.0
31     right = 1.0
32
33     # Exact integration with scipy.integrate.quad:
34     exact, e = integrate.quad(f, left, right)
35
36     # Simpson rule for different number of quadrature points
37     N = arange(2, 101)
38     res = zeros(size(N))
39     for i, n in enumerate(N):
40         res[i] = simpson(f, left, right, n)
41
42     err = abs(res - exact)
43
44     # Linear fit to determine convergence order
45     p = polyfit(log(N[-50:]), log(err[-50:]), 1)
46     print(("Convergence order: %f" % -p[0]))

```



Quadratur-Fehler für $f_1(t) := \frac{1}{1+(5t)^2}$

Ordnung 2 für die Trapez-Regel und
Ordnung 4 für Simpson-Regel



Quadratur-Fehler für $f_2(t) := \sqrt{t}$

Unglattheit von f beschränkt die
Konvergenzordnung

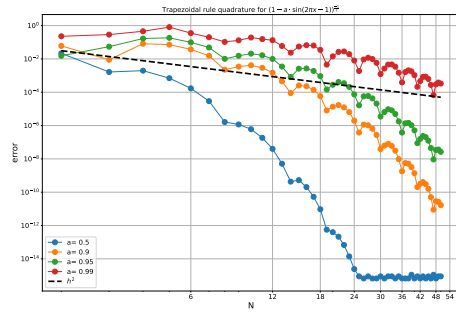
Beispiel 5.4.8. (Konvergenz der Trapez-Regel auf äquidistanten Punkten)

Manchmal gibt es gute Überraschungen: die Konvergenz einer zusammengesetzten Quadraturformel kann mitunter viel besser sein als erwartet von der Ordnung der lokalen Quadraturformel. Die Trapez-Regel auf äquidistanten Punkten hat Ord-

nung 2, aber wenn wir sie auf den 1-periodischen glatten (analytischen) Integranden anwenden

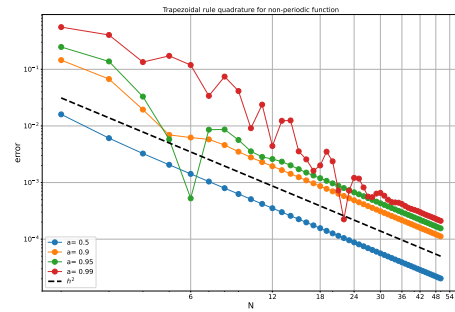
$$f(t) = \frac{1}{\sqrt{1 - a \sin(2\pi t - 1)}} \quad , \quad 0 < a < 1$$

erhalten wir folgende erstaunliche Ergebnisse (als “exakten Wert des Integrals” verwenden wir T_{500}):



Quadratur-Fehler für $T_n(f)$ auf $[0, 1]$

Exponentielle Konvergenz!



Quadratur-Fehler für $T_n(f)$ auf $[0, \frac{1}{2}]$

nur algebraische Konvergenz. . .

Abb. 5.4.9. Die Trapezregel konvergiert exponentiell, wenn der Integrand periodisch und analytisch ist.

Die Erklärung basiert auf der Tatsache, dass T_m exakt für trigonometrische Polynome vom Grad $< 2m$ ist:

$$f(t) = e^{2\pi i k t} \quad \begin{cases} \int_0^1 f(t) dt = \begin{cases} 0, & \text{wenn } k \neq 0, \\ 1, & \text{wenn } k = 0. \end{cases} \\ T_m(f) = \frac{1}{m} \sum_{l=0}^{m-1} e^{\frac{2\pi i}{m} l k} \stackrel{(3.2.15)}{=} \begin{cases} 0, & \text{wenn } k \notin m\mathbb{Z}, \\ 1, & \text{wenn } k \in m\mathbb{Z}. \end{cases} \end{cases}$$

Allerdings braucht man einige Theoreme aus der Funktionentheorie, um die exponentielle Konvergenz für analytische Funktionen in diesem Experiment zu beweisen. In den numerischen Ergebnissen sieht man den Einfluss des Parameters a : die exponentielle Konvergenz verschwindet, wenn a nahe an 1 liegt ($a = 1$ macht die Funktion f unstetig).

Romberg-Schema

Notieren wir mit $T(h)$ das Ergebnis der summierten Trapezregel 5.4.8 für ein äquidistantes Gitter mit Gitterweite h . Nach dem Theorem 5.4.4 verhält sich der Fehler wie h^2 für kleines h .

Bemerkung 5.4.10. Man kann für den Fehler der summierten Trapezregel für äquidistante Gitter

eine genauere Abhängigkeit von der kleinen Gitterweite h beweisen: für glatte f , die $(2p+2)$ -mal stetig differenzierbar in $[a, b]$ sind, gilt:

$$T(h) - \int_a^b f(t)dt = c_1 h^2 + c_2 h^4 + \dots + c_p h^{2p} + o(h^{2p+2}). \quad (5.4.11)$$

Schreiben wir dasselbe für $h/2$:

$$T(h/2) - \int_a^b f(t)dt = c_1 \frac{1}{2^2} h^2 + c_2 \frac{1}{2^4} h^4 + \dots + \tilde{c}_p h^{2p} + o(h^{2p+2}). \quad (5.4.12)$$

Aus den letzten zwei Gleichungen können wir den Term in h^2 nun eliminieren, indem wir eine lineare Kombination mit den Faktoren 2^2 für die erste Gleichung und (-1) für die zweite Gleichung bilden. Die Division mit $2^2 - 1$ liefert dann:

$$\frac{2^2 T(h/2) - T(h)}{2^2 - 1} - \int_a^b f(t)dt = \tilde{c}_2 h^4 + \tilde{c}_3 h^6 + \dots + \tilde{c}_p h^{2p} + o(h^{2p+2}). \quad (5.4.13)$$

Diese lineare Kombination der summierten Trapezregeln mit Gitterweiten h und $h/2$ liefert also eine Quadraturformel, die wie h^4 konvergiert. Besonders günstig ist es, dass wir den Integranden nur einmal in allen Gitterpunkten auswerten müssen. So kostet diese Idee bei geschickter Implementierung nur eine zusätzliche Addition und Multiplikation, keine zusätzlichen Funktionsauswertungen und keinen zusätzlichen Speicher, bei deutlich besserer Genauigkeit. Natürlich bietet es sich an, die Prozedur zu wiederholen, sodass wir die ganze Glattheit von f ausnutzen können. Notieren wir also die neu erhaltene Formel mit T_1 :

$$T_1(h) = \frac{2^2 T(h/2) - T(h)}{2^2 - 1}$$

und schreiben wir die Gleichung (5.4.13) für h und $\frac{1}{2}h$:

$$\begin{aligned} T_1(h) - \int_a^b f(t)dt &= \tilde{c}_2 h^4 + \tilde{c}_3 h^6 + \dots + o(h^{2p+2}), \\ T_1(h/2) - \int_a^b f(t)dt &= \tilde{c}_2 \frac{1}{2^4} h^4 + \tilde{c}_3 \frac{1}{2^8} h^6 + \dots + o(h^{2p+2}). \end{aligned}$$

Die lineare Kombination davon mit Koeffizienten $2^4/(2^4 - 1)$ und $-1/(2^4 - 1)$ eliminiert den h^4 -Term in der Fehlerform:

$$\frac{2^4 T_1(h/2) - T_1(h)}{2^4 - 1} - \int_a^b f(t)dt = \hat{c}_3 h^6 + \hat{c}_4 h^8 + \dots + o(h^{2p+2}). \quad (5.4.14)$$

Der Fehler der neuen Quadraturformel

$$\frac{2^4 T_1(\frac{1}{2}h) - T_1(h)}{2^4 - 1}$$

ist also von der Ordnung h^6 .

Diese Prozedur kann man beliebig weiterverfolgen, solange die Glattheit von f es erlaubt. Notieren wir die summierten Trapezregeln

$$T_{\ell,0} = T(h/2^\ell), \text{ für die Gitterlevels } \ell = 0, 1, 2, \dots,$$

die Quadraturformeln aus einer linearen Kombination von diesen

$$R_{\ell,1} = \frac{2^2 T(\frac{1}{2}h) - T(h)}{2^2 - 1} = \frac{4^1 R_{\ell,0} - R_{\ell-1,0}}{4^1 - 1},$$

die Quadraturformeln aus (noch) einer linearen Kombination von den letzteren

$$R_{\ell,2} = \frac{2^4 T_1(\frac{1}{2}h) - T_1(h)}{2^4 - 1} = \frac{4^2 R_{\ell,1} - R_{\ell-1,1}}{4^2 - 1},$$

und so weiter. Allgemein haben wir also die Definition für $k = 1, 2, \dots, \ell$:

$$R_{\ell,k} = \frac{4^k R_{\ell,k-1} - R_{\ell-1,k-1}}{4^k - 1}, \quad (5.4.15)$$

was dem Romberg-Schema

ℓ	k=0	k=1	k=2	k=3
0	$T(h) = R_{0,0}$			
1	$T(h/2) = R_{1,0}$	$R_{1,1}$		
2	$T(h/2^2) = R_{2,0}$	$R_{2,1}$	$R_{2,2}$	
3	$T(h/2^3) = R_{3,0}$	$R_{3,1}$	$R_{3,2}$	$R_{3,3}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

(5.4.16)

entspricht.

Die erste Spalte beinhaltet die Ergebnisse der summierten Trapezregel für die entsprechenden Gitterweiten. Die anderen Elemente ergeben sich durch die linearen Kombinationen, die die Konvergenzordnung erhöhen. Auf der Diagonale entstehen also die Ergebnisse des Romberg-Schemas mit den verwendeten Gitterweiten, also immer bessere Approximationen.

Der Fehler nimmt mit k und mit ℓ sehr schnell ab:

$$R_{\ell,k} - \int_a^b f(t)dt = C \cdot (h/2^\ell)^{2k+2}.$$

Würde das Ergebnis ganz rechts unten nicht genügend sein, kann man das feinste Gitter noch ein mal verfeinern und nur noch eine Zeile in der oberen Tabelle unten berechnen und hinzufügen.

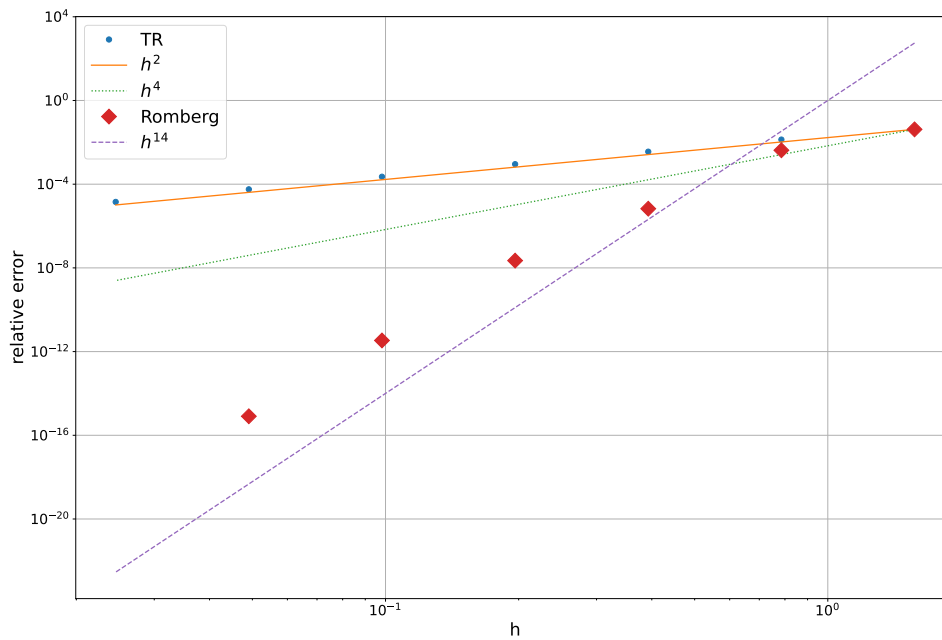
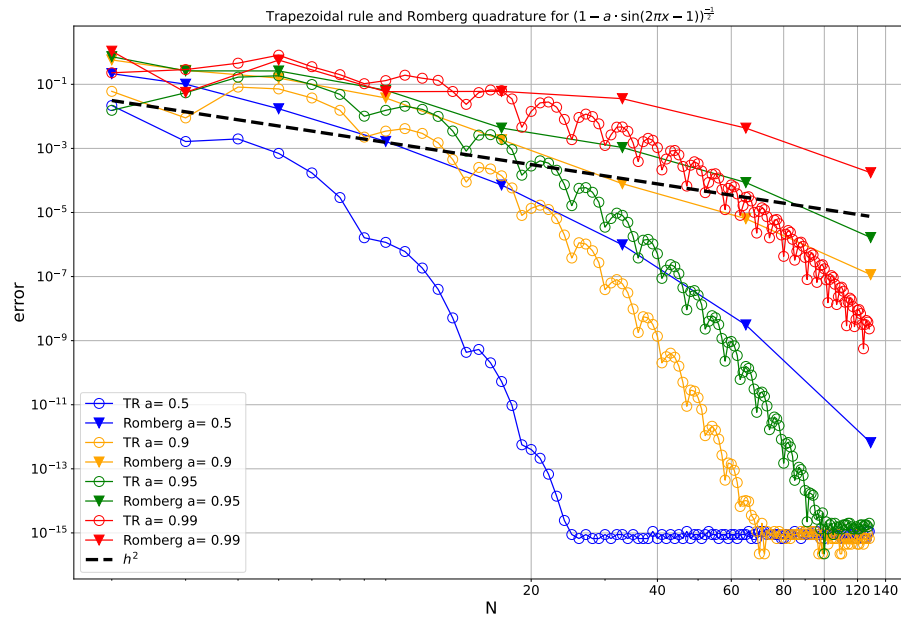


Abb. 5.4.11. Nach nur 6 Iterationen ($\ell = k = 6$) liefert Romberg für ein genügend glattes Integrand ein Fehler im Bereich der Maschinengenauigkeit für grundsätzlich den selben Rechenaufwand, wie die Trapezregel.

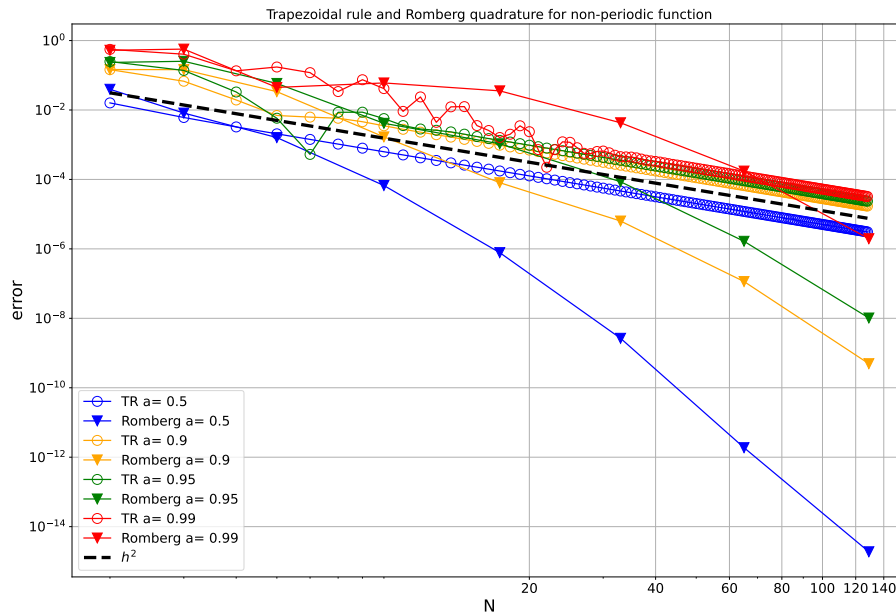
In der Abbildung 5.4.11 sehen wir die rasche Abnahme des Fehlers dank Romberg Schema für eine glatte Funktion: Fehler von Ordnung h^{14} nach 6 Iterationen. So braucht man selten viele Iterationen und man kann die Romberg Schema in einem 2-dimensionalen Array implementieren und Zeile für Zeile bei Bedarf berechnen und hinzufügen. Eine andere Möglichkeit ist, spaltenweise vektorisiert zu arbeiten und bei Bedarf auch jede Spalte im selben Vektor zu schreiben; dann enthält der Vektor am Ende der Iteration die Diagonalelemente $R_{k,k}$ für $k = 0, 1, \dots, \ell$; das erlaubt natürlich nicht mehr das hinzufügen einer neuen Zeile.

Glattheit bleibt die entscheidende Voraussetzungen, jedoch die exponentielle Konvergenz ist mit Romberg quadratur im Allgemein nicht erreichbar:



Quadratur-Fehler für f aus dem Beispiel 5.4.8 auf $[0, 1]$.

Romberg erreicht keine Exponentielle Konvergenz.



Quadratur-Fehler für f aus dem Beispiel 5.4.8 auf $[0, \frac{1}{2}]$.

Die algebraische Konvergenz wird nur bei Glattheit beschleunigt. Die grossartige Effizienz der Romberg Quadratur basiert nicht nur auf die Glattheit des Integrands, sondern auch auf die Tatsache, dass die verwendeten Gitter ver-

schachtelt sind und die Funktionsauswertungen wiederverwendet werden. Bei nicht verschachtelten Gitter ist das nicht mehr der Fall und man muss das Kosten/Nutzen genau überlegen. Bulirsch hat eine optimale Sequenz von Gitterunterteilungen gefunden, die wir hier nicht besprechen.

Bemerkung 5.4.12. Die Idee von Romberg kann man für jede Quantität anwenden, die eine ähnliche Entwicklung hat:

$$N(h) = M + c_1 h^q + c_2 h^{2q} + \dots + c_p h^{pq} + o(h^{(p+1)q}).$$

Sie kommt eigentlich aus der Richardson Extrapolation, die wir bei Interpolation anschauen werden und wo die Schema den Name Aitken-Neville trägt.

Newton-Cotes: nicht verwenden!

Bemerkung 5.4.13. In der Praxis werden **keine äquidistante Quadraturformeln höherer Ordnung** (also keine ausser Mittelpunkt-, Trapez- und Simpson-Formel) verwendet, denn das Kosten/Nutzen Verhalten verschlechtert sich dramatisch.

Das Bild 2.7 zeigt: Interpolation mit Polynome höheres Grades auf äquidistante Knoten ist katastrophal: weite Schwingungen entstehen zwischen den Knoten. Das numerische Experiment mit den Ergebnissen in der Abbildung 5.5.24 bestätigen dies: Newton-Cotes höher Ordnung ist unbrauchbar. Das scheint paradoxal, denn diese Formeln kann man exakt bauen und die Monome werden tatsächlich bis zur gegebenen Grad exakt integriert. Jedoch sind die Monome eine *exponential schlecht konditionierte Basis*: die Rundungsfehler vergrößern sich exponentiell schnell, so dass das Ergebnis der Quadratur für andere Funktionen als die Monomen unbrauchbar sind!

Wenn es Polynome höheres Grades sein müssen, dann muss man auf die Chebyshev-Knoten oder auf die die Chebyshev-Abszissen interpolieren, die nicht äquidistant sind, sondern aus einer trigonometrischer Transformation kommen, was die DFT im Spiel bringt; zu diesem Ansatz entspricht die weiter unten die Clenshaw-Curtis Quadraturformel.

Es ist eventuell interessant, sich die Gewichte der möglichen Quadraturformeln höherer Ordnung auf äquidistante Punkte anzuschauen. Man kann diese via das Lösen des linearen Gleichungssystem, der beim Einsatz der Monome in der Quadraturformel entsteht, berechnen. Man kann sie aber auch exakt mit der Hilfe von **Sympy** berechnen.

Code 5.4.14: exakte Gewichte der Quadraturformeln mit äquidistante Knoten

```

import sympy as sy
import numpy as np

f_ = sy.Function('f')
x_ = sy.symbols('x')

def symbaequi(N=4):
```

```

8      ks = np.arange(N)
      nds = sy.Matrix(
10         N,1, lambda k, i: sy.S(ks[k])/(N-1)
          )
12      fvals = sy.Matrix(
          N,1, lambda k, i: f_( sy.S(ks[k])/(N-1) )
14         )
      qf = sy.integrate(
16         sy.interpolating_poly(
            N,x_, X = nds, Y =fvals
18             ) ,
            (x_, 0, 1)
20         )
      return qf
22
24 a_, b_ = sy.symbols('a b')
26 for N in np.arange(2,12):
      qf = symbaequi(N)
      print('N=', N)
28      #qf = sy.factor(qf)
      sy.pprint(qf)
30      rr = (b_-a_)*qf.replace(f_, lambda x: f_(sy.together(a_+(b_-a_)*x)))
      #rr = sy.factor(rr)
32      sy.pprint(rr)
      sy.print_latex(rr)

```

Für $N = 3$ erhalten wir die Simpson Regel:

$$(b-a) \left(\frac{f(a)}{6} + \frac{f(b)}{6} + \frac{2f(\frac{a+b}{2})}{3} \right),$$

für $N = 4$:

$$(b-a) \left(\frac{f(a)}{8} + \frac{f(b)}{8} + \frac{3f(\frac{a+2b}{3})}{8} + \frac{3f(\frac{2a+b}{3})}{8} \right),$$

für $N = 5$ (Milne-Regel)

$$(b-a) \left(\frac{7f(a)}{90} + \frac{7f(b)}{90} + \frac{2f(\frac{a+b}{2})}{15} + \frac{16f(\frac{a+3b}{4})}{45} + \frac{16f(\frac{3a+b}{4})}{45} \right),$$

für $N = 7$ (Weddle-Regel):

$$(b-a) \left(\frac{41f(a)}{840} + \frac{41f(b)}{840} + \frac{34f(\frac{a+b}{2})}{105} + \frac{9f(\frac{a+2b}{3})}{280} + \frac{9f(\frac{a+5b}{6})}{35} + \frac{9f(\frac{2a+b}{3})}{280} + \frac{9f(\frac{5a+b}{6})}{35} \right).$$

Ab $N = 9$ gibt es auch negative Gewichte:

$$(b-a) \left(\frac{989f(a)}{28350} + \frac{989f(b)}{28350} - \frac{454f(\frac{a+b}{2})}{2835} - \frac{464f(\frac{a+3b}{4})}{14175} + \frac{2944f(\frac{a+7b}{8})}{14175} - \frac{464f(\frac{3a+b}{4})}{14175} \right. \\ \left. + \frac{5248f(\frac{3a+5b}{8})}{14175} + \frac{5248f(\frac{5a+3b}{8})}{14175} + \frac{2944f(\frac{7a+b}{8})}{14175} \right).$$

Negative Gewichte, vor allem in dieser Form, werden auch grosse sich akkumulierende Rundungsfehler produzieren, sind also überhaupt nicht sinnvoll.

Es ist zu betonen, dass nicht die Rundungsfehler selber schuld an das dramatische Scheitern Quadraturformeln höherer Ordnung auf äquidistante Punkte sind, sondern die (hier versteckte) Verwendung von Polynome höherer Grade auf äquidistante Punkte ist eine äusserst schlechte Idee; das katastrophale Verhalten käme selbst bei exakter Arithmetik zum Vorschein: siehe Theorem 2.3.8, Beispiel 2.3.9, Bemerkung 2.3.10 und Beispiel 2.3.11. Der Weg zu höhere Quadraturordnung braucht eine neue Idee.

5.5 Nicht äquidistante Stützstellen

Wir können die Genauigkeit erhöhen, ohne das Intervall zu unterteilen. Dafür müssen wir aber eine flexiblere Wahl der Quadraturpunkte erlauben.

5.5.1 Gauss Quadratur

In diesem Ansatz wird die Genauigkeit erhöht, indem

Gegeben sei eine Funktion $g(t)$. Wir befinden uns auf dem Referenzintervall $[0, 1]$ (kann auch $[-1, 1]$ sein) und haben die Knoten $c_1, c_2, \dots, c_s \in [0, 1]$ und die Gewichte $b_1, b_2, \dots, b_s$. Die Quadraturformel ist somit:

$$\int_0^1 g(t)dt \approx \sum_{j=1}^s b_j \cdot g(c_j) = Q(g; 0, 1) \quad (5.5.17)$$

Der Fehler dieser Quadraturformel beträgt:

$$E(g) = \int_0^1 g(t)dt - \sum_{j=1}^s b_j \cdot g(c_j) = \int_0^1 g(t)dt - Q(g; 0, 1) \quad (5.5.18)$$

Bisher waren die Knoten $c_1, c_2, \dots, c_s$ gegeben. Dabei waren sie stets äquidistant.

Jetzt wollen wir $c_1, c_2, \dots, c_s$ und $b_1, b_2, \dots, b_s$ so bestimmen, dass die Quadraturordnung maximal ist und wir verzichten auf die einschränkende Bedingung äquidistanter Knoten. Genauer gesagt möchten wir die maximal mögliche Ordnung $p = s + m$ erreichen, d.h. das grösstmögliche $m \in \mathbb{N}, m > 0$. Das bedeutet weiter, dass jedes Polynom vom Grad $\leq p - 1 = s + m - 1$ mit der Quadraturformel exakt integriert werden soll.

Sei f ein Polynom vom Grad maximal $p - 1 = s + m - 1$ und für gegebene Knoten $c_1, c_2, \dots, c_s$ sei $M(t) = (t - c_1)(t - c_2)\dots(t - c_s)$. Der Grad von M ist s und $M(c_1) = \dots = M(c_s) = 0$.

Polynomdivision ergibt

$$f(t) = M(t) \cdot g(t) + r(t),$$

mit Grad von r maximal $s - 1$. Durch die Integration beider Seiten erhalten wir nun

$$\int_0^1 f(t)dt = \int_0^1 M(t) \cdot g(t)dt + \int_0^1 r(t)dt.$$

Wenn die Quadraturformel alle Polynome vom Grad maximal $s + m - 1$ exakt integriert, dann gilt

$$\int_0^1 f(t)dt = \sum_{j=1}^s b_j \cdot f(c_j) = \sum_{j=1}^s b_j (M(c_j) \cdot g(c_j) + r(c_j)).$$

Da $M(c_1) = \dots = M(c_s) = 0$, bedeutet dies

$$\int_0^1 f(t)dt = \sum_{j=1}^s b_j \cdot f(c_j) = \sum_{j=1}^s b_j r(c_j) = \int_0^1 r(t)dt,$$

denn die Quadraturformel ist auch für r exakt. Somit erhalten wir:

$$\int_0^1 M(t) \cdot g(t)dt = \int_0^1 f(t)dt - \int_0^1 r(t)dt = 0,$$

für alle Polynome f vom Grad maximal $s + m - 1$, d.h. für alle Polynome g vom Grad maximal $m - 1$, denn M hat Grad s . Mit der Notation des Skalarprodukts für quadratisch integrierbare Funktionen bedeutet dies $\langle M, g \rangle = \int_0^1 M(t)g(t)dt = 0$.

Theorem 5.5.1. Die Ordnung der Quadraturformel ist $s + m$ dann und nur dann, wenn $\langle M, g \rangle = 0$ für alle Polynome g vom Grad maximal $m - 1$, wobei $M(t) = (t - c_1)(t - c_2)\dots(t - c_s)$ (d.h. M steht senkrecht auf allen Polynomen vom Grad maximal $m - 1$).

Theorem 5.5.2. (Maximale Ordnung einer Quadraturformel) Die Ordnung einer Quadraturformel mit s Knoten ist höchstens $2s$.

Proof. Nehmen wir für einen Widerspruch an, es existiere eine Quadraturformel mit s Knoten und Ordnung $p \geq 2s + 1$.

Daraus folgt, dass $\int_0^1 M(t) \cdot g(t)dt = 0$ für alle Polynome g vom Grad $p - s - 1$ also auch für alle Polynome vom Grad $\leq s$. Wir nehmen nun $g = M$ und erhalten

$$\int_0^1 M(t) \cdot M(t)dt = \int_0^1 M(t)^2 dt = 0$$

Zusammen damit, dass M ein Polynom und stetig auf $[0, 1]$ ist, erhalten wir, dass $M(t) = 0$ für alle $t \in [0, 1]$, was im Widerspruch mit der konkreten Formel für M steht. Somit muss die Annahme falsch sein, so dass $p \leq 2s$ sein muss. $\square$

Orthogonale Polynome

Sei $w :]a, b[\rightarrow \mathbb{R}$ eine stetige Gewichtsfunktion mit $w(x) > 0$ für alle $x \in]a, b[$, so dass $\int_a^b |x|^k w(x) dx$ existiert für alle $k = 0, 1, 2, \dots$

Betrachten wir den linearen Raum:

$$V = \{f :]a, b[\rightarrow \mathbb{R}, \text{ stetig, } \int_a^b |f(x)|^2 w(x) dx \text{ existiert}\}$$

Hier können wir als Skalarprodukt $\langle f, g \rangle = \int_a^b f(x)g(x)w(x)dx$ betrachten. Für diesen Raum halten wir das folgende Theorem fest:

Theorem 5.5.3. Es existiert eine eindeutige Folge von Polynomen $p_0, p_1, p_2, \dots$ in V mit $p_k(x) = x^k + \text{Polynom vom Grad} \leq k-1$, wobei $k \geq 0$, sodass

$$p_k \perp \text{span}\{p_0, p_1, \dots, p_{k-1}\}.$$

Diese Polynome kann man mit der sogenannten 3-Term-Rekursion bauen:

$$p_{k+1}(x) = (x - \beta_{k+1}) \cdot p_k(x) - \gamma_k p_{k-1}(x) \quad (5.5.19)$$

Wobei hier $p_0(x) = 1$, $p_{-1}(x) = 0$, $\beta_{k+1} = \frac{\langle x \cdot p_k, p_k \rangle}{\langle p_k, p_k \rangle}$ und $\gamma_k = \frac{\langle p_k, p_k \rangle}{\langle p_{k-1}, p_{k-1} \rangle}$ sind.

Proof. Der Beweis folgt der Konstruktion aus dem Gram-Schmidt Algorithmus, man kann ihn aber auch als mathematische Induktion auffassen. Für die Induktionsverankerung zeigen wir mithilfe der Definition von β , dass p_1 senkrecht auf p_0 steht:

$$\langle p_0, p_1 \rangle = \langle p_0, (x - \beta_1)p_0 \rangle = \langle (x - x)p_0 \rangle = 0$$

Für den Induktionsschritt brauchen wir die Induktionsannahme, dass $p_0, \dots, p_k$ (mit $k \geq 0$) bereits bekannt sind. Wir haben:

$$\begin{aligned} \langle p_k, p_{k+1} \rangle &= \langle p_k, (x - \beta_{k+1})p_k - \gamma_k p_{k-1} \rangle \\ &= \langle p_k, xp_k \rangle - \underbrace{\beta_{k+1} \langle p_k, p_k \rangle}_{\langle xp_k, p_k \rangle} - \underbrace{\beta_k \langle p_k, p_{k-1} \rangle}_0 = 0 \\ \langle p_{k-1}, p_{k+1} \rangle &= \langle p_{k-1}, (x - \beta_{k+1})p_k - \gamma_k p_{k-1} \rangle \\ &= \underbrace{\langle p_{k-1}, xp_k \rangle}_0 - \beta_{k+1} \underbrace{\langle p_{k-1}, p_k \rangle}_0 - \underbrace{\beta_k \langle p_k, p_{k-1} \rangle}_0 = 0 \\ \langle p_j, p_{k+1} \rangle &= \langle p_j, (x - \beta_{k+1})p_k - \gamma_k p_{k-1} \rangle \\ &= \underbrace{\langle p_j, xp_k \rangle}_0 - \beta_{k+1} \underbrace{\langle p_j, p_k \rangle}_0 - \underbrace{\beta_k \langle p_j, p_{k-1} \rangle}_0 = 0, \quad j = 0, \dots, k-2 \end{aligned}$$

Überall, wo ein Term gleich null ist, haben wir die Induktionsannahme verwendet. Somit steht p_{k+1} senkrecht auf allen bisherigen p_i , mit $i = 0, \dots, k$. $\square$

Beispiel 5.5.4. (*Legendre-Polynome*) Für $w(x) = 1$ und $a = -1, b = 1$ erhalten wir durch dieses Theorem orthogonale Polynome, die Legendre-Polynome:

$$\begin{aligned} p_0(x) &= 1 \\ p_1(x) &= x \\ p_2(x) &= \frac{1}{2}(3x^2 - 1) \\ p_3(x) &= \frac{1}{2}(5x^3 - 3x). \end{aligned}$$

Zu bemerken ist, dass es in der Literatur verschiedene Konventionen auf die Normierung der Legendre-Polynome gibt. Darauf muss man besonderes achten, wenn man vorgegebene Software verwendet. Die hier gewählte Normierung sorgt **nicht** für orthonormale Polynome bezüglich des Standardskalarprodukts für quadratisch integrierbare Funktionen auf $[-1, 1]$, sondern stellt sicher, dass $p_n(1) = 1$ für alle n , was zu der Norm $\|p_n\| = \sqrt{2/(2n+1)}$ führt. Schauen Sie sich bitte die Dokumentation von `scipy.special.eval_legendre` und `scipy.special.legendre` an. Verwenden Sie `scipy.special.roots_legendre`, um diese Eigenschaft dieser Implementierung zu überprüfen.

Die Legendre-Polynome können wir für die Berechnung der Gauss-Legendre-Quadraturpunkte und -Gewichte verwenden.

Beispiel 5.5.5. (*Hermite-Polynome*) Für $w(x) = e^{-x^2}$ und $a = -\infty, b = \infty$ erhalten wir durch dieses Theorem die Hermite-Polynome (in der Physik-Konvention). Die ersten Hermite-Polynome sind:

$$\begin{aligned} p_0(x) &= 1 \\ p_1(x) &= 2x \\ p_2(x) &= 4x^2 - 2 \\ p_3(x) &= 8x^3 - 12x \end{aligned}$$

Die Hermite-Polynome können wir für die Berechnung der Gauss-Hermite-Quadraturpunkte und -Gewichte verwenden.

Bemerkung 5.5.6. Oft werden diese Polynome zu $p_k(1) = 1$ "normiert", anstelle zu $p_k(x) = 1 \cdot x^k + \text{Polynom vom Grad} \leq k-1$.

Bemerkung 5.5.7. Aus dem Theorem folgt direkt, dass die Knoten $c_1, c_2, \dots, c_s$ die Nullstellen von p_s sind.

Wir wollen die Berechnungen im Theorem 5.5.3 anhand der folgenden drei Beispiele illustrieren:

Bem.: Wie die Gewichte berechnet werden, wird später in diesem Abschnitt zu sehen sein.

Beispiel 5.5.8. Sei $s = 1$.

Betrachten wir zuerst das Intervall $[0, 1]$. Mit dem Theorem 5.5.3 und $w(x) = 1$ für

das Skalarprodukt erhalten wir, dass $p_1(x) = x - \frac{1}{2}$. Durch die Bemerkung 5.5.7 ergibt sich für die Gauss-Quadratur der Knoten $c_1 = \frac{1}{2}$. Das Gewicht ist $b_1 = 1$.

Betrachten wir nun das Intervall $[-1, 1]$. Wir erhalten $p_1(x) = x$, das zweite Legendre-Polynom. Der Knoten ist $\gamma_1 = 0$ und das Gewicht ist $\omega_1 = 2$.

Wir beobachten dabei, dass dies genau der Mittelpunktsregel entspricht und halten fest, dass die Gauss-Quadratur mit $s=1$ Knoten äquivalent zur Mittelpunktsregel ist. Die Ordnung dieser Gauss-Quadratur ist $2 \cdot s = 2 \cdot 1 = 2$.

Beispiel 5.5.9. Sei $s = 2$.

Wieder betrachten wir zuerst das Intervall $[0, 1]$. p_1 haben wir bereits in Beispiel 5.5.8 berechnet. Mit dem Theorem 5.5.3 und $w(x) = 1$ für das Skalarprodukt erhalten wir, dass $p_2(x) = x^2 - x + \frac{1}{6}$. Durch die Bemerkung 5.5.7 ergeben sich für die Gauss-Quadratur die Knoten $c_1 = \frac{1}{2} - \frac{\sqrt{3}}{6}$ und $c_2 = \frac{1}{2} + \frac{\sqrt{3}}{6}$. Die Gewichte sind $b_1 = b_2 = 1$. Betrachten wir nun das Intervall $[-1, 1]$. Wir erhalten $p_2(x) = \frac{1}{2}(3x^2 - 1)$, das dritte Legendre-Polynom. Die Knoten sind $\gamma_1 = -\frac{1}{\sqrt{3}}$ und $\gamma_2 = \frac{1}{\sqrt{3}}$, die Gewichte sind $\omega_1 = \omega_2 = 1$.

Die Ordnung dieser Gauss-Quadratur ist $2 \cdot s = 2 \cdot 2 = 4$.

Beispiel 5.5.10. Sei $s = 3$.

Hier betrachten wir nur das Intervall $[-1, 1]$. p_1 und p_2 haben wir bereits in Beispiel 5.5.8 und Beispiel 5.5.9 berechnet. Mit dem Theorem 5.5.3 und $w(x) = 1$ für das Skalarprodukt erhalten wir $p_3(x) = \frac{5}{2}x^3 - \frac{3}{2}x$, das vierte Legendre-Polynom. Die Knoten, die sich aus den Nullstellen von p_3 ergeben (siehe Bemerkung 5.5.7), sind $\gamma_1 = -\frac{\sqrt{15}}{5}$, $\gamma_2 = 0$ und $\gamma_3 = \frac{\sqrt{15}}{5}$. Die Gewichte sind $\omega_2 = \frac{8}{9}$ und $\omega_1 = \omega_3 = \frac{5}{9}$. Die Ordnung dieser Gauss-Quadratur ist $2 \cdot s = 2 \cdot 3 = 6$.

Bemerkung 5.5.11. (Knoten und Fehler der Gauss-Quadratur)

- Die Gauss-Knoten sind nicht verschachtelt. Bei einer adaptiven Quadraturformel kann man die vorigen Funktionsauswertungen nicht wiederverwenden. Eine Gauss-Quadratur höherer Ordnung kann also nicht auf die Funktionswerte in den Knotenpunkten der Gauss-Quadratur tieferer Ordnung zurückgreifen.
- Gauss-Knoten sind keine äquidistanten Knoten.
- Die Endpunkte des Intervalls sind keine Knoten. Die Gauss-Quadraturformeln sind somit offen.
- Manchmal wird benötigt, dass einer oder beide Endpunkte Knoten sind. In diesem Falle reduziert sich die maximale Ordnung der Quadratur, aber die Konstruktion ist ähnlich und kann mit Hilfe von `scipy.special.roots_jacobi` gebaut werden.

Bei der *Radau*-Quadratur macht man einen Knoten fest, so hat man Ordnung $2s - 1$. Betrachten wir das Intervall $[-1, 1]$ und setzen wir $c_s = 1$ fest, so ist $M(t) = (1 - t)(t - c_1) \dots (t - c_{s-1})$ und die Orthogonalitätsbedingung wird zu $0 = \int_{-1}^1 M(t)g(t) = \int_{-1}^1 (1 - t)(t - c_1) \dots (t - c_{s-1})g(t)dt$, d.h. wir haben

dasselbe Problem, aber mit $s - 1$ Knoten und Gewichtsfunktion $(1 - t)$. So sind die Knoten $c_1, \dots, c_{s-1}$ die $s - 1$ Knoten der Gauss-Jacobi Quadratur mit dem Gewicht $(1 - t)^1(1 + t)^0$. Diese erhält man z.B. mit

`r,w = scipy.special.roots_jacobi(s-1,alpha=1,beta=0)`. Die Gewichte der Radau-Quadratur ergeben sich aus den $s - 1$ Gewichten w_i der Gauss-Jacobi Quadratur als $b_i = w_i/(1 - c_i)$ für $i = 1, \dots, s - 1$ und $b_s = 2/s^2$ für $c_s = 1$.

Bei der *Lobatto*-Quadratur werden beide Enden des Intervalls als feste Knoten gesetzt, so hat man Ordnung $2s - 2$; auf $[-1, 1]$ sind die Knoten $\{-1, c_2, \dots, c_{s-1}, 1\}$. Die Knoten und Gewichte kann man analog aus der Gauss-Jacobi Quadratur mit $s - 2$ Knoten erhalten, also `alpha=1, beta=1`; die Gewichte kommen als $b_i = C \cdot w_i/(1 - c_i^2)$ für $i = 2, \dots, s - 1$ und die Endgewichte sind $2/(s(s - 1))$. Der Faktor C lässt sich aus der Bedingung über die Summe aller Gewichte ausrechnen.

Die Radau- und die Lobatto-Knoten und -Gewichte haben wichtige Anwendungen bei der numerischen Lösung gewöhnlicher Differentialgleichungen.

- Der Fehler der Gauss-Quadratur beträgt:

$$\int_a^b f(x)dx - \sum_{j=1}^s b_j \cdot f(c_j) = \frac{b-a}{(2s)!} f^{(2s)}(z) \text{ mit } z \in [a, b]$$

Daraus folgt die obere Grenze für den Fehler der zusammengesetzten Gauss-Quadraturformel mit je s Knoten auf einem äquidistanten Gitter mit der Maschenweite h :

$$\left| \int_a^b f(x)dx - \sum_{k=1}^N G_s(f, x_{k-1}, x_k) \right| \leq c \cdot h^{2s} \max_{z \in [a, b]} |f^{(2s)}(z)|,$$

wobei $c \in \mathbb{R}$ eine Konstante und $h = (b - a)/N$ für N Subintervalle ist.

Beispiel 5.5.12. Gegeben seien die Stützstellen x_0, x_1 und x_2 . Die entsprechenden Lagrange-Polynome sind:

$$\ell_0(x) = \frac{x - x_1}{x_0 - x_1} \cdot \frac{x - x_2}{x_0 - x_2}$$

$$\ell_1(x) = \frac{x - x_0}{x_1 - x_0} \cdot \frac{x - x_2}{x_1 - x_2}$$

$$\ell_2(x) = \frac{x - x_0}{x_2 - x_0} \cdot \frac{x - x_1}{x_2 - x_1}$$

Beispiel 5.5.13. Erinnern wir uns zurück an Beispiel 5.5.9. Für $s = 2$ hatten wir auf dem Intervall $[-1, 1]$ die Knoten $\gamma_1 = -\frac{1}{\sqrt{3}}$ und $\gamma_2 = \frac{1}{\sqrt{3}}$. Aus diesen ergeben sich die (Lagrange-)Polynome:

$$\ell_0(x) = \frac{x - \frac{1}{\sqrt{3}}}{-\frac{1}{\sqrt{3}} - \frac{1}{\sqrt{3}}} = \frac{1}{2} - \frac{\sqrt{3}}{2}x$$

$$\ell_1(x) = \frac{x - \frac{-1}{\sqrt{3}}}{\frac{1}{\sqrt{3}} - \frac{-1}{\sqrt{3}}} = \frac{1}{2} + \frac{\sqrt{3}}{2}x$$

Bemerkung 5.5.14. (Gewichte der Gauss-Legendre Quadratur) Gegeben seien die Knoten $c_1, c_2, \dots, c_s$. Daraus erhalten wir die Lagrange-Polynome $\ell_1, \ell_2, \dots, \ell_s$. Der Grad von ℓ_i ist dabei $\text{Grad}(\ell_i) = s - 1$ für $i = 1, 2, \dots, s$. Sei nun $\delta(t) = \ell_i(t)^2$ das Polynom vom Grad $2(s - 1)$. Die zugehörige Quadraturformel ist exakt für Polynome vom Grad $2s - 1$, somit auch für $2(s - 1)$. Daraus folgt:

$$\int_{-1}^1 \ell_i(t)^2 dt = \sum_{j=1}^s b_j \cdot \underbrace{\ell_i(c_j)^2}_{\substack{=0 \text{ für } j \neq i \\ =1 \text{ für } j=i}} = b_i \cdot 1 = b_i$$

Somit folgt, dass die Gewichte der Gauss-Legendre Quadratur wie folgt berechnet werden können:

$$b_i = \int_{-1}^1 \ell_i(t)^2 dt. \quad (5.5.20)$$

Theorem 5.5.15. Die Gewichte der Gauss-Legendre-Quadraturformel sind positiv.

Proof. Dies folgt direkt aus der Berechnung der Gewichte: $b_i = \int_{-1}^1 \ell_i(t)^2 dt > 0$. $\square$

Algorithmen zur Berechnung der Gauss-Knoten

Code 5.5.16: Algorithmus von Golub und Welsch

```

1 from numpy import arange, diag, sqrt
2 from numpy.linalg import eigh
4 def gaussquad(n):
5     r"""
6         Compute nodes and weights for Gauss-Legendre quadrature.
7
8         n: Number of node-weight pairs
9         """
10    i = arange(1,n) #i = array([1,...,n-1])
11    b = i / sqrt(4*i**2 - 1)
12    # now we generate the matrix; it is symmetric
13    J = diag(b, -1) + diag(b, 1)
14    # in order to find the eigenvalues we can use eigh since J is symmetric
15    x, ev = eigh(J)
16    # finally, we apply the formula for the weights
17    w = 2 * ev[0,:]**2
18    return x, w

```

Lange Zeit war das der Standard für die Berechnung der Gauss-Knoten. Für s Knoten kostet das $\mathcal{O}(s^3)$, bestenfalls $\mathcal{O}(s^2)$ Operationen. Die Knoten sind nicht sehr genau und die Gewichte sehr ungenau. Neuere Algorithmen (Bogaert und Townsend,

2012-2014) können die Knoten und Gewichte sehr genau in $\mathcal{O}(s)$ Operationen berechnen. Die unten beschriebene Clenshaw-Curtis Quadratur funktioniert genauso gut für glatte Funktionen und kostet nur $\mathcal{O}(s \log s)$ Operationen für $2s$ Knoten.

Die Komplexität verschiedener Algorithmen wird in der folgenden Tabelle veranschaulicht:

Algorithmus (s Knoten)	Arbeit	Genauigkeit	
		Knoten	Gewichte
GW (1969)	$\mathcal{O}(s^3) / \mathcal{O}(s^2)$	$\mathcal{O}(1)$	$\mathcal{O}(s^2)$
Bogaert-Townsend	$\mathcal{O}(s)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
CC ($2s$ Knoten)	$\mathcal{O}(s \log s)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

Abb. 5.5.17. Komplexität der Algorithmen zur Knoten- und Gewichtsrechnung

Die Gauss'sche Quadraturformel lässt sich allerdings nicht immer anwenden, da bei bestimmten Problemen die Funktionswerte lediglich an bestimmten Punkten gegeben sind und die zugrunde liegende Funktion nicht bekannt ist. Die meisten Messgeräte liefern Messungen zu einer festen Zeitrates, was die Gauss'schen Quadraturformeln ausschliesst. Auch wenn wir frei sind, die Messpunkte zu wählen, bedeutet die Tatsache, dass die Knoten nicht verschachtelt sind, sehr grosse Kosten, wenn wir die Ordnung während der Berechnungen erhöhen müssen. Verschachtelte Quadraturformeln, d.h. bei denen man die Funktionswerte an den Knoten für Formeln niedriger Ordnung wiederverwenden kann, sind in dieser Hinsicht viel günstiger. Daher benutzt man in bestimmten Fällen andere Quadraturformeln. Diese werden am Ende dieses Kapitels kurz besprochen.

5.5.2 Clenshaw-Curtis Quadraturformel

Die Quadraturformeln von Clenshaw-Curtis (1960) bzw. Fejér basieren auf der Idee der Approximation des Integranden mittels Chebyshev-Polynomen. Diese Idee liefert eine (beinahe) ideale Approximation (siehe Theorem 2.4.13 und Beispiel 2.4.14). Wenn wir die (interpolationstheoretisch ideale) Nullstellen der Chebyshev-Polynome erster Art (T) verwenden (Chebyshev-Knoten), d.h. $x_k = \cos((k + \frac{1}{2})\frac{\pi}{n})$ mit $k = 0, \dots, n-1$, erhalten wir die erste Quadraturformel von Fejér. Diese Knoten sind nicht verschachtelt, so schlug Fejér seine zweite Quadraturformel in den Filippi-Knoten $x_k = \cos(k\frac{\pi}{n})$ mit $k = 1, \dots, n-1$ vor. Clenshaw und Curtis nehmen 1960 auch die Enden des Intervalls dazu, also $x_k = \cos(k\frac{\pi}{n})$ mit $k = 0, \dots, n$, und berechnen die entsprechenden Gewichte, so dass die Ordnung der Quadraturformel n ist. Diese Clenshaw-Curtis Knoten sind die Chebyshev-Abszissen, d.h. die Extrema der Chebyshev-Polynome erster Art (T) und die Nullstellen der Chebyshev-Polynome zweiter Art (U), die interpolationstechnisch zwar suboptimal sind, jedoch sehr ähnliche Konvergenzverhältnisse wie die Chebyshev-Knoten liefern. Die Chebyshev-Abszissen zusammen mit den Gewichten von Clenshaw und Curtis liefern ähnliche Genauigkeit wie die Gauss-Quadratur für Integranden f , die nicht analytisch in einer komplexen Nachbarschaft von $[-1, 1]$ sind, d.h. die Formeln von Clenshaw-Curtis und Gauss verhalten sich gleich, wenn die Funktion nicht sehr glatt ist.

Für sehr glatte Funktionen konvergieren sie beide exponentiell, auch wenn Gauss schneller ist (genauer $\mathcal{O}(\rho^{2n})$ versus $\mathcal{O}(\rho^n)$ mit $\rho < 1$), jedoch für kleine n liefern beide sehr ähnliche Ergebnisse. Der Vorteil der Clenshaw-Curtis Quadratur ist die Verschachtelung der Knoten und die sehr schnelle und iterationslose Berechnung der Gewichte. Die Verbindung zur DFT gibt die Möglichkeit, die Clenshaw-Curtis Gewichte sehr schnell und numerisch sehr stabil (ohne Rundungsfehlerakkumulation) zu berechnen. Für die Integration beliebiger Funktionen wird die Variablensubstitution $x = \cos(\theta)$ verwendet. Hieraus folgt:

$$\int_{-1}^1 f(x) dx = \int_0^\pi f(\cos(\theta)) \sin(\theta) d\theta.$$

Der Integrand ist jetzt eine periodische Funktion geworden. Die Glattheit von f überträgt sich auf die Glattheit des neuen Integranden, jedoch wird die Ableitung an 0 und π im Allgemeinen unstetig sein, so liefert weder die Trapezregel noch die Mittelpunktsregel die sehr gerne erwartete exponentielle Konvergenz aus der Abbildung 5.4.9.

Wir definieren $F(\theta) := f(\cos(\theta))$. Da $F(\theta)$ 2π -periodisch und gerade ist, lässt es sich in eine Kosinus-Reihe entwickeln: $F(\theta) = \sum_{k=0}^\infty a_k \cos(k\theta)$, woraus folgt:

$$\begin{aligned} \int_0^\pi F(\theta) \sin(\theta) d\theta &= \int_0^\pi \sum_{k=0}^\infty a_k \cos(k\theta) \sin(\theta) d\theta = \sum_{k=0}^\infty a_k \int_0^\pi \cos(k\theta) \sin(\theta) d\theta \\ &= a_0 + \sum_{2 \leq k \text{ gerade}} \frac{2a_k}{1 - k^2}. \end{aligned}$$

Die Koeffizienten a_k lassen sich mittels FFT ³ oder DCT ⁴ berechnen. Da hinter dieser Quadraturformel die Fourier-Transformation (für die Theorie) mit ihrem schnellen Algorithmus (FFT) steht, ist diese Formel sehr effizient (kurze Rechenzeit und sehr gute Genauigkeit) für glatte Funktionen f .

Die Implementierung nach Nick Trefethen 5.5.18 berechnet direkt das Integral. Die Implementierung nach Greg von Winckel 5.5.19 liefert elegant die Clenshaw-Curtis Gewichte und Knoten.

Die Implementierung nach Jörg Waldvogel (SAM, ETHZ) 5.5.20 ist die effizienteste Art, die Clenshaw-Curtis (und auch die Fejér) Gewichte und Knoten zu bauen.

Code 5.5.18: Clenshaw-Curtis: direkte Implementierung (Trefethen)

```

import numpy as np
def cc2(func,a,b,N):
    """
    Clenshaw-Curtis quadrature rule
    by FFT with the function values
    """

```

³fast Fourier transform

⁴discrete cosine transform


```

# first: change of variable with adaptation of the interval
8  bma = 0.5*(b-a)
x = np.cos(np.pi * np.linspace(0,N,N+1)/N) # Chebyshev-abszissa
10 x *= bma
x += 0.5*(a+b)
12 fx = func(x) # F(theta) = f(cos(theta)) = f(x)
vx = np.hstack((fx,fx[-2:0:-1])) # extend by reflection to 2π-periodic
14 # note: hstack generates a new array consistig of two former arrays put
together.
# the index [-2:0:-1] allows to take all elements of an array, except the
first one, in reverse order
16 # now we perform the FFT to find the coefficients.
g = np.real(np.fft.fft(vx))/(2.0*N) # we have to divide by the length, which
is 2N after reflexion
18 A = np.zeros(N+1) #A is an array with the coeffs of cosines
A[0] = g[0]; A[N] = g[N]
20 A[1:N] = g[1:N] + np.flipud(g[N+1:]) # put together terms to get coeffs of
cosines
w = 0.*x
22 w[:,2] = 2./(1.-np.r_[N+1:2]**2) #directly apply the C.Curtis quadrature
formula
# now we can return the final value of the integral; the dot product is a
clever way of writing; otherwise we could have opted for a for-cycle
24 return np.dot(w,A)*bma

```

Code 5.5.19: Clenshaw-Curtis: Gewichte und Knoten (Greg von Winckel)

```

1 import numpy as np
def cc1(func, a, b, N):
3     """
        Clenshaw-Curtis quadrature rule
        by constructing the points and the weights
        Method of Greg von Winckel
        """
    bma = b-a
    9 c = np.zeros((2,2*(N-1) ))
    c[0,0] = 2.0
    11 c[1,1] = 1.0
    c[1,-1] = 1.0
    13 for i in np.arange(2,N,2):
        val = 2.0/(1-i**2) #compare with C.Curtis formula
    15 c[0,i] = val
        c[0,2*(N-1)-i] = val
    17
    f = np.real(np.fft.ifft(c))
    19 w = f[0,:N]; w[0] *= 0.5; w[-1] *= 0.5 # weights
    x = 0.5*((b+a)+(N-1)*bma*f[1,:N]) # points
    21 return np.dot(w,func(x))*bma

```

Code 5.5.20: Clenshaw-Curtis: Gewichte und Knoten (Jörg Waldvogel, 2012)

```

    """
2  Fejer-1 Fejer-2 and Clenshaw Curtis according to Joerg Wladvogel talk in Oxford
    2012

```

```

4  """
import numpy as np

6  def ccJW(n):
    if n <= 1:
8      raise ValueError("n must be greater than 1")

    n2 = n % 2
    u0 = 1 / (n**2 - 1 + n2) # Boundary weights of CC

12     # Clenshaw-Curtis nodes: k=0,1,...,n; vector of weights wcc,
wcc[0]=wcc[n]=u0
14     K = np.arange(0,n+1)
    xcc = np.cos( K*np.pi/n ) # Chebyshev abszisa / CC points
16     wcc = np.zeros(n+1); wcc[-1] = u0
    L= K[:-1]
18     m = np.minimum(L, n - L)
    r = 2.0 / (1 - 4 * m**2)
20     vc = r - u0
    wcc[:-1] = np.real( np.fft.ifft(vc) )

22
    return xcc, wcc

24
def allfejerJW(n):
26     if n <= 1:
        raise ValueError("n must be greater than 1")

28
    n2 = n % 2
30     u0 = 1 / (n**2 - 1 + n2) # Boundary weights of CC

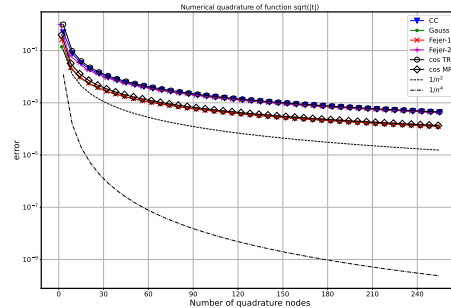
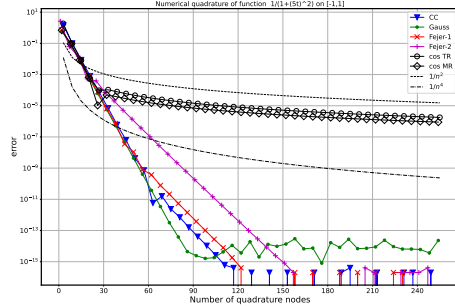
    # Clenshaw-Curtis nodes: k=0,1,...,n; vector of weights wcc
    K = np.arange(0,n+1)
34     xcc = np.cos( K*np.pi/n ) # Chebyshev abszisa / CC points
    wcc = np.zeros(n+1); wcc[-1] = u0
36     L= K[:-1]
    m = np.minimum(L, n - L)
38     r = 2.0 / (1 - 4 * m**2)
    vc = r - u0
40     wcc[:-1] = np.real( np.fft.ifft(vc) )

    # Chebyshev or Fejer-1 nodes: k=1/2,3/2,...,n-1/2; vector of weights wf1
42     xf1 = np.cos( (K[:-1]+0.5)*np.pi/n ) # Chebyshev or Fejer-1 nodes
    s1 = np.sign(n / 2 - L) # auxiliary vector
44     v1 = s1 * r * np.exp(1j * np.pi / n * L) # 1j for imaginary unit
    wf1 = np.real(np.fft.ifft(v1)) # Chebyshev or Fejer-1 weights

    # Filippi or Fejer-2 nodes: k=0,1,...,n; vector of weights wf2,
48     wf2[0]=wf2[n]=0
    xf2 = xcc[1:-1].copy() # Fejer-2 nodes
    # Note: Boolean arrays are treated as 0s and 1s in numerical operations in
50     NumPy
    flag = np.abs(n / 2 - L) < 1
52     s2 = 1 + flag * (n - (n / 2. + 1) * n2) # Ensure n/2 is float division
    v2 = s2 * r
54     wf2 = np.real( np.fft.ifft(v2)[1:] )

```

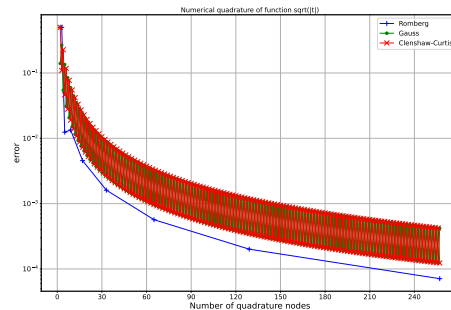
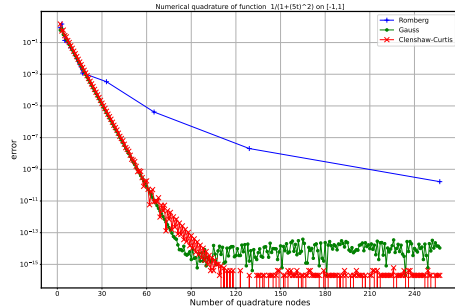
```
return xcc, wcc, xf1, wf1, xf2, wf2
```



Fehler für $f_1(t) := \frac{1}{1+(5t)^2}$ auf $[-1, 1]$

Fehler für $f_2(t) := \sqrt{|t|}$ auf $[-1, 1]$

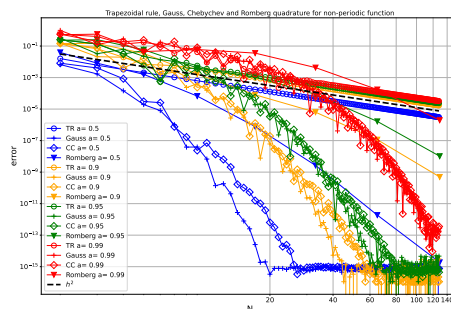
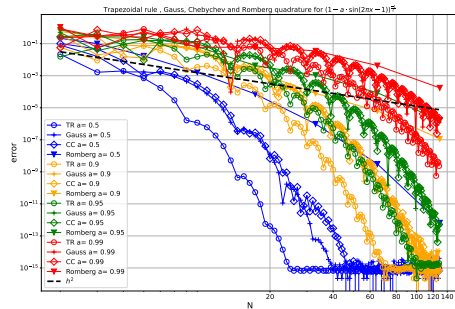
Abb. 5.5.21. Vergleich der Konvergenzen: Gauss, Clenshaw-Curtis, Fejér Methoden und die naive Ideen direkt nach dem Variablenwechsel



Fehler für $f_1(t) := \frac{1}{1+(5t)^2}$ auf $[-1, 1]$

Fehler für $f_2(t) := \sqrt{|t|}$ auf $[-1, 1]$

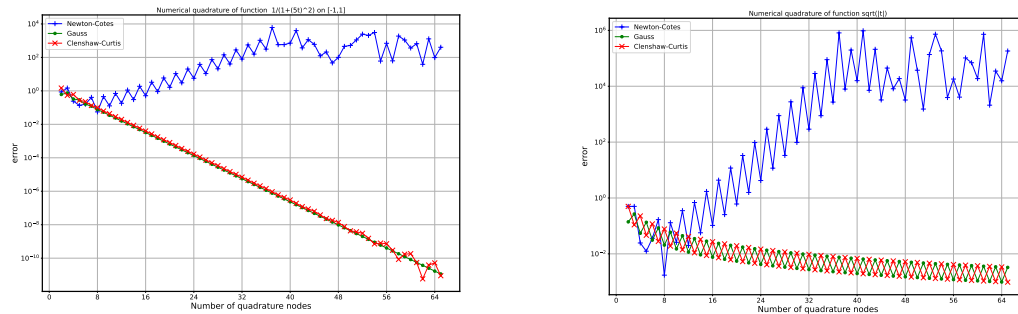
Abb. 5.5.22. Vergleich der Konvergenzen: Clenshaw-Curtis, Gauss und Romberg Quadratur



Fehler für $f_1(t) := \frac{1}{1+(5t)^2}$ auf $[-1, 1]$

Fehler für $f_2(t) := \sqrt{|t|}$ auf $[-1, 1]$

Abb. 5.5.23. Vergleich der Konvergenzen: Clenshaw-Curtis ist ähnlich gut wie Gauss Quadratur



Fehler für $f_1(t) := \frac{1}{1+(5t)^2}$ auf $[-1, 1]$

Fehler für, $f_2(t) := \sqrt{|t|}$ auf $[-1, 1]$

Abb. 5.5.24. Nicht-zusammengesetzte Quadratur:
Newton-Cotes grosser Ordnung ist unbrauchbar.

Für Integrale der Form

$$\int_a^b f(x) \omega(x) dx$$

spielen verschiedene orthogonale Polynome eine wesentliche Rolle:

Quadratur	Intervall	Gewichtsfunktion	Polynom	Not.	scipy.special.
Gauss	$(-1, 1)$	1	Legendre	P_k	roots_legendre
Chebyshev I	$(-1, 1)$	$\frac{1}{\sqrt{1-x^2}}$	Chebyshev I	T_k	roots_chebyt
Chebyshev II	$(-1, 1)$	$\sqrt{1-x^2}$	Chebyshev II	U_k	roots_chebyu
Jacobi $\alpha, \beta > -1$	$(-1, 1)$	$(1-x)^\alpha (1+x)^\beta$	Jacobi	$P_k^{(\alpha, \beta)}$	roots_jacobi
Hermite	$\mathbb{R}$	e^{-x^2}	Hermite	H_k	roots_hermite
Laguerre	$(0, \infty)$	$x^\alpha e^{-x}$	Laguerre	L_k	roots_genlaguerre

Abb. 5.5.25. Gewichtsfunktionen für Quadraturformeln

Die Tabelle soll wie folgt gelesen werden:

- Auf dem Intervall $(-1, 1)$ und mit dem Gewicht $w(t) = 1$ erhalten wir die Legendre-Polynome. Diese werden dann für die Gauss-Legendre Quadratur verwendet.
- Auf $\mathbb{R}$ und mit dem Gewicht $w(t) = e^{-t^2}$ erhalten wir die Hermite-Polynome. Diese werden dann für die Gauss-Hermite Quadratur verwendet.
- Analog für Laguerre, Jacobi, Chebyshev I und II; diese sind jedoch weniger stabil als die oben präsentierten Implementierungen (Trefethen oder von Winkel oder Waldvogel).

5.6 Adaptive Quadratur

Wie können wir Gitterpunkte definieren, so dass wir einen kleinen Fehler erhalten und gleichzeitig nicht allzu viele Quadraturknoten verwenden?

Beispiel 5.6.1. Wir betrachten die zusammengesetzte Quadraturformel auf dem Gitter $\mathcal{M} := \{a = x_0 < x_1 < \dots < x_m = b\}$.

Für den lokalen Fehler gilt (für $f \in C^2([a, b])$):

$$\left| \int_{x_k}^{x_{k+1}} f(t) dt - \frac{f(x_k) + f(x_{k+1})}{2} (x_{k+1} - x_k) \right| \leq (x_{k+1} - x_k)^3 \|f''\|_{L^\infty([x_k, x_{k+1}])}$$

Deswegen ist es sinnvoll, das Gitter nur dort zu verfeinern, wo $|f''|$ gross ist.

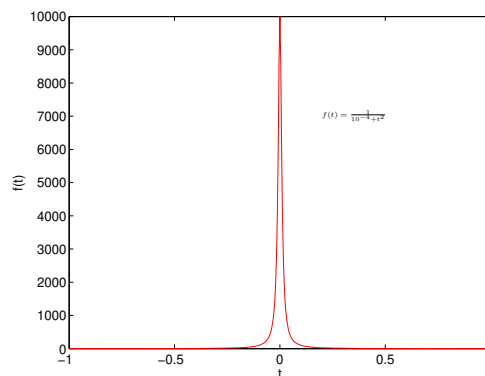


Figure 5.1: Beispiel einer Funktion, deren Integral mit kürzeren Intervallen bei $x = 0$ und mit längeren Intervallen anderswo ausgewertet werden sollte.

Das Ziel der adaptiven Quadratur ist es, eine solche Unterteilung des Integrationsgebietes zu erzielen, dass die Fehler auf den einzelnen Subintervallen gleichermassen zum Gesamtfehler beitragen. Dazu müssen wir den lokalen Fehler schätzen. Dann unterteilen wir nur diejenigen Subintervalle weiter, die einen grossen Fehler aufweisen. Nun: Wie können wir den lokalen Fehler schätzen? Wir wenden zwei Quadraturformeln an, von denen eine kleinere Fehler als die andere liefert. Wir können zum Beispiel die Trapez-Regel und die Simpson-Regel verwenden.

Wenn wir die Trapez-Regel auf das Intervall $[x_j, x_{j+1}]$ der Länge h anwenden, haben wir

$$\int_{x_j}^{x_{j+1}} f(t) dt = Q_1(f, x_j, x_{j+1}) + \mathcal{O}(h^3).$$

Für die Simpson-Regel ergibt sich der genauere Wert:

$$\int_{x_j}^{x_{j+1}} f(t) dt = Q_2(f, x_j, x_{j+1}) + \mathcal{O}(h^5).$$

Für kleines h können wir den exakten Wert des Integrals durch $Q_2(f, x_j, x_{j+1})$ ersetzen, um eine grobe Schätzung des lokalen Fehlers bei der Verwendung der Trapezregel zu erhalten:

$$\left| \int_{x_j}^{x_{j+1}} f(t) dt - Q_1(f, x_j, x_{j+1}) \right| \approx |Q_2(f, x_j, x_{j+1}) - Q_1(f, x_j, x_{j+1})|.$$

Ein grosser Wert hier ist ein Hinweis darauf, dass die Funktion f stark variiert, so dass es sich eventuell lohnt, das Intervall $[x_j, x_{j+1}]$ weiter zu unterteilen. Der Preis für diese Information ist nur die Auswertung von f im Mittelpunkt $(x_j + x_{j+1})/2$, die bei einer weiteren Unterteilung direkt wiederverwertet werden kann. In der Praxis verwenden wir natürlich die genauere Quadraturformel für das Endergebnis; die

ungenauere Formel dient hier nur als Mittel für die Fehlerschätzung. Nach diesem Prinzip sind die professionellen Codes für die Quadratur aufgebaut; der nächste Code zeigt, wie das konkret funktioniert.

Code 5.6.2: h -adaptive Quadratur

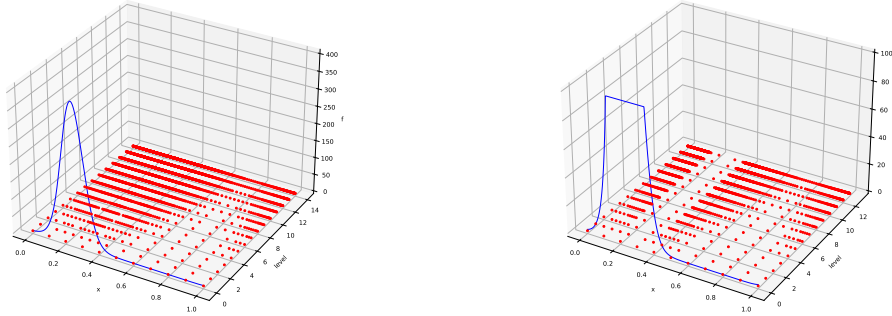
```

1  import numpy as np
   from scipy import integrate
3
   def adaptquad(f,M,rtol,abstol):
       """
       adaptive quadrature using trapezoid and simpson rules
       Arguments:
       f        handle to function f
       M        initial mesh
       rtol     relative tolerance for termination
       abstol   absolute tolerance for termination, necessary in case the exact
11  integral value = 0, which renders a relative tolerance meaningless.
       """
13     h = np.diff(M)                                # compute lengths of mesh
       intervals
       mp = 0.5*( M[:-1]+M[1:] )                    # compute midpoint positions
15     fx = f(M); fm = f(mp)                         # evaluate function at positions and
       midpoints
       trp_loc = h*( fx[:-1]+2*fm+fx[1:] )/4         # local trapezoid rule
17     simp_loc= h*( fx[:-1]+4*fm+fx[1:] )/6         # local simpson rule
       I = simp_loc.sum()                           # use simpson rule value as
       intermediate approximation for integral value
19     est_loc = abs(simp_loc - trp_loc)              # difference of values obtained from
       local composite trapezoidal rule and local simpson rule is used as an estimate
       for the local quadrature error.
       err_tot = est_loc.sum()                       # estimate for global error (sum
       moduli of local error contributions)
21     # if estimated total error not below relative or absolute threshold, refine
       mesh
       if err_tot > rtol*abs(I) and err_tot > abstol:
23         refcells = np.nonzero( est_loc > 0.9*err_tot/len(est_loc) )[0]
         I = adaptquad(f, np.sort(np.append(M,mp[refcells])),rtol,abstol)    #
       add midpoints of intervalls with large error contributions, recurse.
25     return I

27 if __name__ == '__main__':
    f = lambda x: np.exp(6*np.sin(2*np.pi*x))
29    #f = lambda x: 1.0/(1e-4+x*x)
    M = np.arange(11.)/10 # 0, 0.1, ... 0.9, 1
31    rtol = 1e-6; abstol = 1e-10
    I = adaptquad(f,M,rtol,abstol)
33    exact,e = integrate.quad(f,M[0],M[-1])
    print('adaptquad:',I, "exact:",exact)
35    print('error:',abs(I-exact))

```

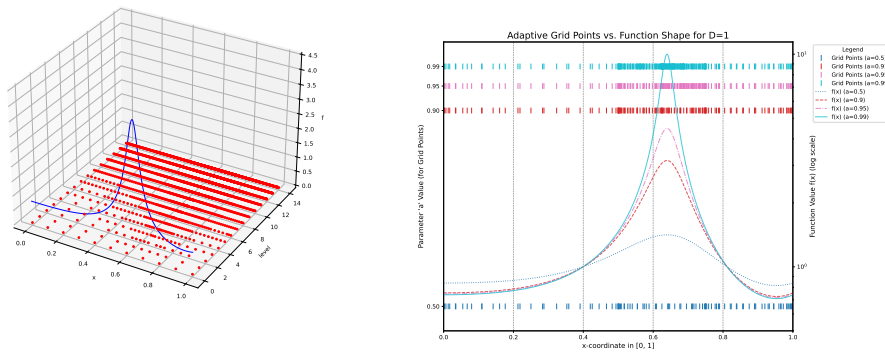
Für die Funktionen $f(x) = \exp(6 * \sin(2 * \pi * x))$ und $\min(f(x), 100)$ ergibt diese Art der Adaptivität die folgenden Gitter:



Bemerkung 5.6.3. (Adaptive Quadratur in Python)

`scipy.integrate.quad` realisiert eine adaptive Quadratur und ist nur ein Wrapper zur Fortran-Bibliothek QUADPACK, siehe `scipy.integrate.explain_quad()` für Details; `scipy.integrate.quadrature` steht für adaptive Gauss-Quadratur.

Für die Funktion $f(x) = 1/\sqrt{1 - a \sin(2\pi x - 1)}$ aus dem Beispiel 5.4.8 ergeben unsere einfache adaptive Trapez-Simpson-Methode (für $a = 0.95$) und `scipy.integrate.quad` folgende Gitter:



Wir erweitern diese Funktion auf $\mathbf{x} \in \mathbb{R}^d$:

$$f(\mathbf{x}) = \frac{1}{\sqrt{1 - a \sin(2\pi \|\mathbf{x}\|_2^2 - 1)}}. \quad (5.6.21)$$

Für den Fall $d = 2$ plotten wir die Niveaulinien und die Punkte der Funktionsauswertungen in der adaptiven Gauss-Quadratur aus `scipy.integrate.nquad` siehe das separate File.

5.7 Quadratur in $\mathbb{R}^d$ und dünne Gitter

Bisher haben wir lediglich eindimensionale Quadraturprobleme betrachtet. Insbesondere in physikalischen Anwendungen sind mehrdimensionale Integrale allgegenwärtig, auch über zwei und drei Dimensionen hinaus. In diesem Abschnitt werden

wir uns mit der Effizienz von solchen Quadraturformeln beschäftigen. Insbesondere wird unser Ziel sein, ein Verfahren zu finden, das auch für grosse Dimensionszahlen noch vernünftige Ergebnisse liefert.

Naive Herangehensweise: Iteration von eindimensionalen Integralen

Betrachte zuerst das zweidimensionale Integral I :

$$I = \int_a^b \int_c^d f(x, y) dx dy = \int_a^b F(y) dy .$$

Für F gilt:

$$F(y) = \int_c^d f(x, y) dx \approx \sum_{j_1=1}^{n_1} \omega_{j_1}^1 f(c_{j_1}^1, y) ,$$

wobei hier die Gewichte und Stützstellen einer geeigneten eindimensionalen Quadraturformel entsprechen. Eben diese verwenden wir für die zweite Integration:

$$I = \sum_{j_2=1}^{n_1} \sum_{j_2=1}^{n_2} \omega_{j_1}^1 \omega_{j_2}^2 f(c_{j_1}^1, c_{j_2}^2) .$$

Wir erhalten die sogenannte Tensor-Produkt-Quadratur:

Gegeben die 1-dimensionalen Quadraturformeln:

$$(w_{j_k}^k, c_{j_k}^k)_{1 \leq j_k \leq n_k} , \quad k = 1, \dots, d ,$$

ist das d -dimensionale Integral approximiert durch

$$I \approx \sum_{j_1=1}^{n_1} \dots \sum_{j_d=1}^{n_d} w_{j_1}^1 \dots w_{j_d}^d f(c_{j_1}^1, \dots, c_{j_d}^d) .$$

Wir bemerken, dass die Funktion in d Dimensionen an n^d Stützstellen ausgewertet werden muss. Dies ist insbesondere problematisch, da f eventuell nur als teure Funktionsroutine verfügbar ist. Es sei ebenfalls angemerkt, dass für die Konvergenzrate gilt: $K = O(N^{-r/d})$, wobei r die Konvergenzrate der eindimensionalen Quadraturformel ist. Deswegen wird die Konvergenz schlechter mit wachsender Anzahl an Dimensionen.

Quadratur auf dünnen Gittern

Betrachten wir zunächst eine eindimensionale Quadraturformel:

$$I = \int_a^b f(x) dx \approx Q_n^1(f) .$$

Durch das Hinzunehmen von Punkten verbessert sich das Resultat zunächst stark, dann jedoch immer weniger, d.h. $Q_{n+1} - Q_n$ wird mit zunehmendem n abnehmen. Hier ein Beispiel mit der Trapezregel:

$$T_1(f, a, b) = \frac{f(a) + f(b)}{2}(b - a).$$

Wir schreiben den Fehler als:

$$S_1(f, a, b) = \int_a^b f(x)dx - T_1(f, a, b).$$

Graphisch lässt sich S_1 anhand von Theorem 5.7.1 veranschaulichen.

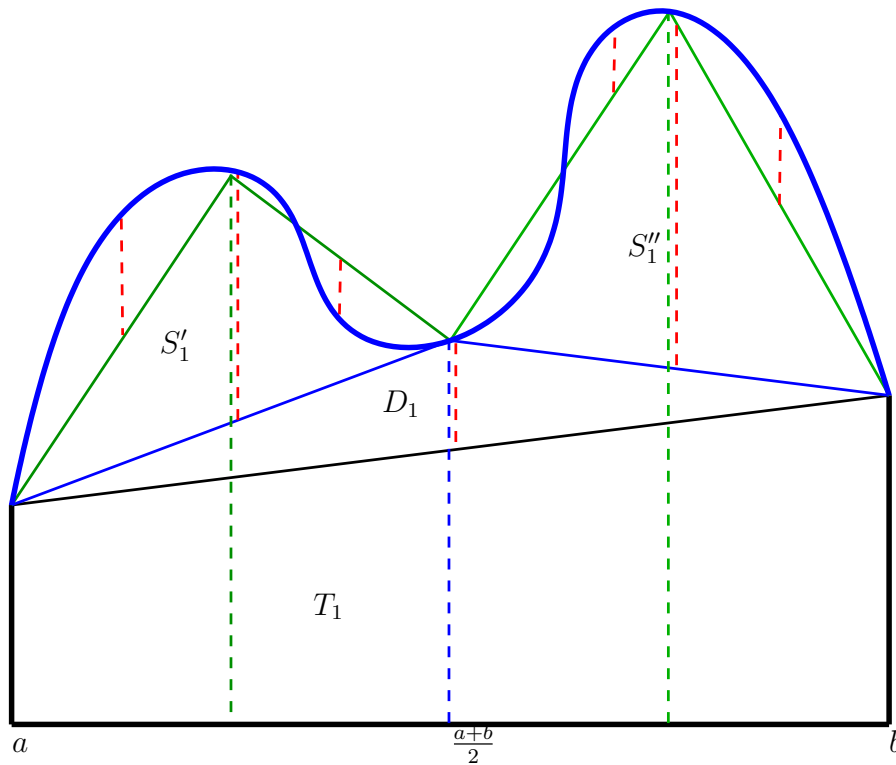


Abb. 5.7.1. Hierarchische Struktur bei der Trapez-Regel

Hier entspricht S_1 der Fläche, die vom Trapez T_1 nicht ausgefüllt wird. Offensichtlich ist diese erste Näherung sehr schlecht, da wir nur die Punkte $f(a)$ und $f(b)$ verwendet haben. Wir erwarten, dass unter Hinzunahme der Information am Mittelpunkt $f((a+b)/2)$ die Approximation besser wird und schreiben:

$$T_2(f, a, b) = \frac{f(a) + f((a+b)/2)}{2} + \frac{f((a+b)/2) + f(b)}{2}.$$

Bemerkung: In der Formel für T_2 verwendet man nicht den bereits berechneten Wert T_1 , man verwirft also bereits gesammelte Information. Das muss nicht sein.

Betrachten wir wieder Theorem 5.7.1. Die Differenz zwischen T_1 und T_2 ist gerade das Dreieck D_1 :

$$T_2(f, a, b) = T_1(f, a, b) + D_1(f, a, b),$$

wobei sich D_1 , wie der Graph zeigt, folgendermassen ergibt:

$$D_1(f, a, b) = \left(f\left(\frac{a+b}{2}\right) - \frac{f(a) + f(b)}{2} \right) \frac{b-a}{2} = g_1(f, a, b) \frac{b-a}{2}.$$

Die Quantität $g_1(f, a, b) = f\left(\frac{a+b}{2}\right) - \frac{f(a)+f(b)}{2}$ heisst *hierarchischer Überschuss* der Funktion f auf dem Intervall $[a, b]$. Der Fehler ist nun die Summe der Flächen S'_1 und S''_1 :

$$S'_1\left(f, a, \frac{a+b}{2}\right) + S''_1\left(f, \frac{a+b}{2}, b\right),$$

und er hat sich durch die Hinzunahme der Auswertung im Mittelpunkt $\frac{a+b}{2}$ genau um die Fläche vom Dreieck D_1 reduziert.

Wir erwarten kleinere Terme S'_1, S''_1 und somit auch hierarchische Überschüsse, wenn diese Prozedur iteriert wird (wie in Theorem 5.7.1 zu sehen ist).

Wir betrachten nun das Intervall $[0, 1]$, welches in 2^ℓ Teilintervalle $(\frac{k}{2^\ell}, \frac{k+1}{2^\ell})$, mit $k = 0, 1, \dots, 2^\ell - 1$ zerfällt. Wir nennen ℓ das *Level* und bezeichnen mit Q_ℓ^1 eine klassische Quadraturformel auf jedem Teilintervall. Typische Beispiele hierfür sind die Trapez- und die Mittelpunktsregel; die Clenshaw-Curtis- und die Gaussformeln haben nicht äquidistante Stützstellenverteilungen, können aber auch verwendet werden. Die Prozedur der hierarchischen Überschüsse lässt sich als Teleskopsumme formulieren:

$$\begin{aligned} Q_\ell^1 f &= Q_0^1 f + (Q_1^1 f - Q_0^1 f) + (Q_2^1 f - Q_1^1 f) + \dots + (Q_\ell^1 f - Q_{\ell-1}^1 f) \\ &= Q_0^1 f + \Delta_1^1 f + \Delta_2^1 f + \dots + \Delta_\ell^1 f. \end{aligned}$$

In einer Dimension vereinfacht diese Umformulierung das Problem keineswegs; in d Dimensionen gibt es jedoch einen fundamentalen Unterschied. Das Tensorprodukt der Level $\ell = (\ell_1, \dots, \ell_d)$ ist:

$$\begin{aligned} Q_\ell^d &= Q_{\ell_1}^1 \otimes \dots \otimes Q_{\ell_d}^1 \\ &= \sum_{j_1=1}^{N_1} \dots \sum_{j_d=1}^{N_d} w_{j_1}^1 \dots w_{j_d}^d f(c_{j_1}^1, \dots, c_{j_d}^d) \\ &= \sum_{j=1}^d \sum_{1 \leq k_j \leq \ell_j} (\Delta_{k_1}^1 \otimes \dots \otimes \Delta_{k_d}^1) f. \end{aligned}$$

Im Falle eines isotropen Gitters $\ell = (\ell, \dots, \ell)$ notieren wir:

$$Q_\ell^d = \sum_{j=1}^d \sum_{1 \leq k_j \leq \ell} (\Delta_{k_1}^1 \otimes \dots \otimes \Delta_{k_d}^1) f = \sum_{|\mathbf{k}|_\infty \leq \ell} (\Delta_{k_1}^1 \otimes \dots \otimes \Delta_{k_d}^1) f.$$

Wenn f glatt ist, sind die Details $(\Delta_{k_1}^1 \otimes \dots \otimes \Delta_{k_d}^1) f$ so klein, dass sie vernachlässigt werden können. Alle Details, die ungefähr gleich gross sind, lassen wir auf einmal weg.

Die klassische Dünngitter-Quadraturformel (Smolyak) ist dann definiert als:

$$S_\ell^d f := \sum_{|\mathbf{k}|_1 \leq \ell + d - 1} (\Delta_{k_1}^1 \otimes \dots \otimes \Delta_{k_d}^1) f.$$

Man kann dazu die Kombinationsformel beweisen:

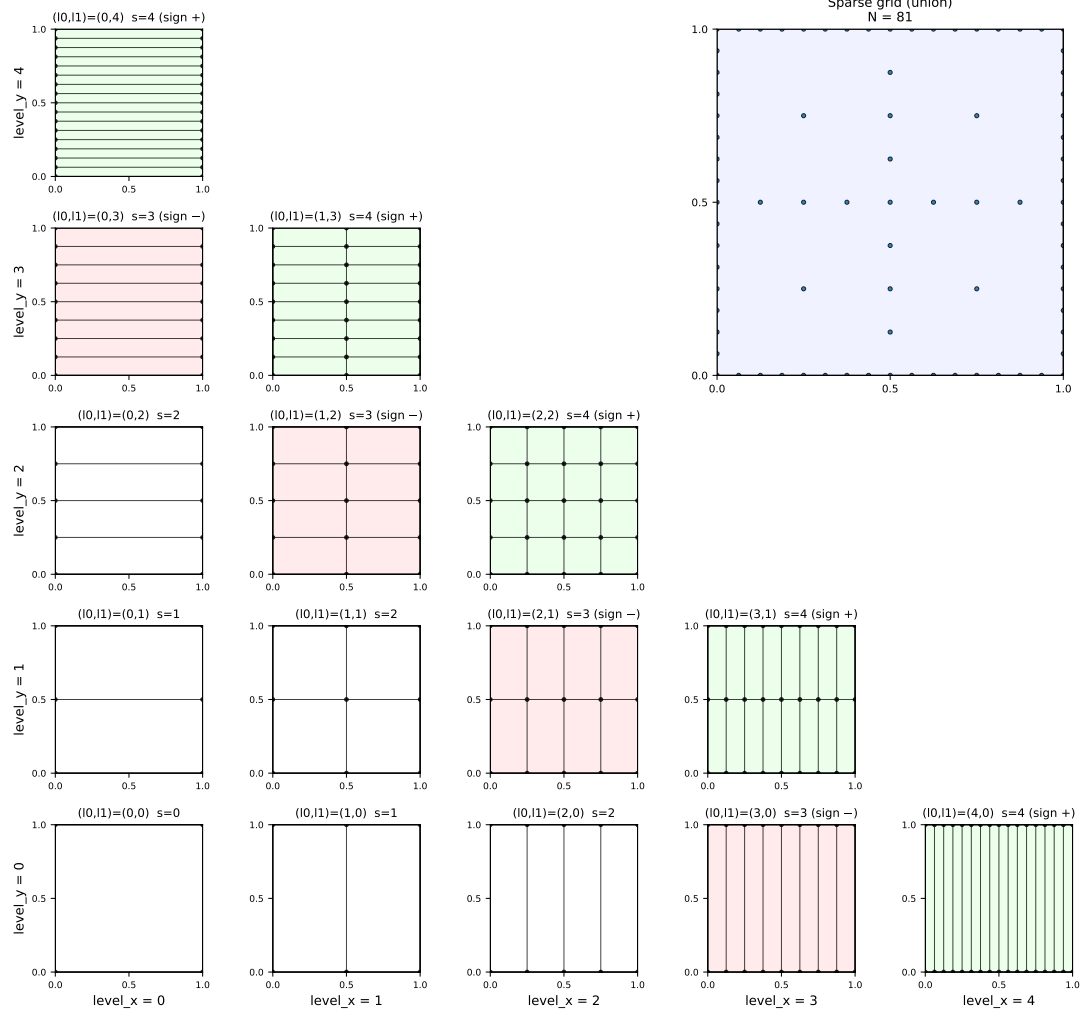
$$S_\ell^d f = \sum_{\ell \leq |\mathbf{k}|_1 \leq \ell + d - 1} (-1)^{\ell + d - |\mathbf{k}|_1 - 1} \binom{d-1}{|\mathbf{k}|_1 - \ell} (Q_{k_1}^1 \otimes \dots \otimes Q_{k_d}^1) f,$$

welche in praktischen Implementierungen verwendet wird.

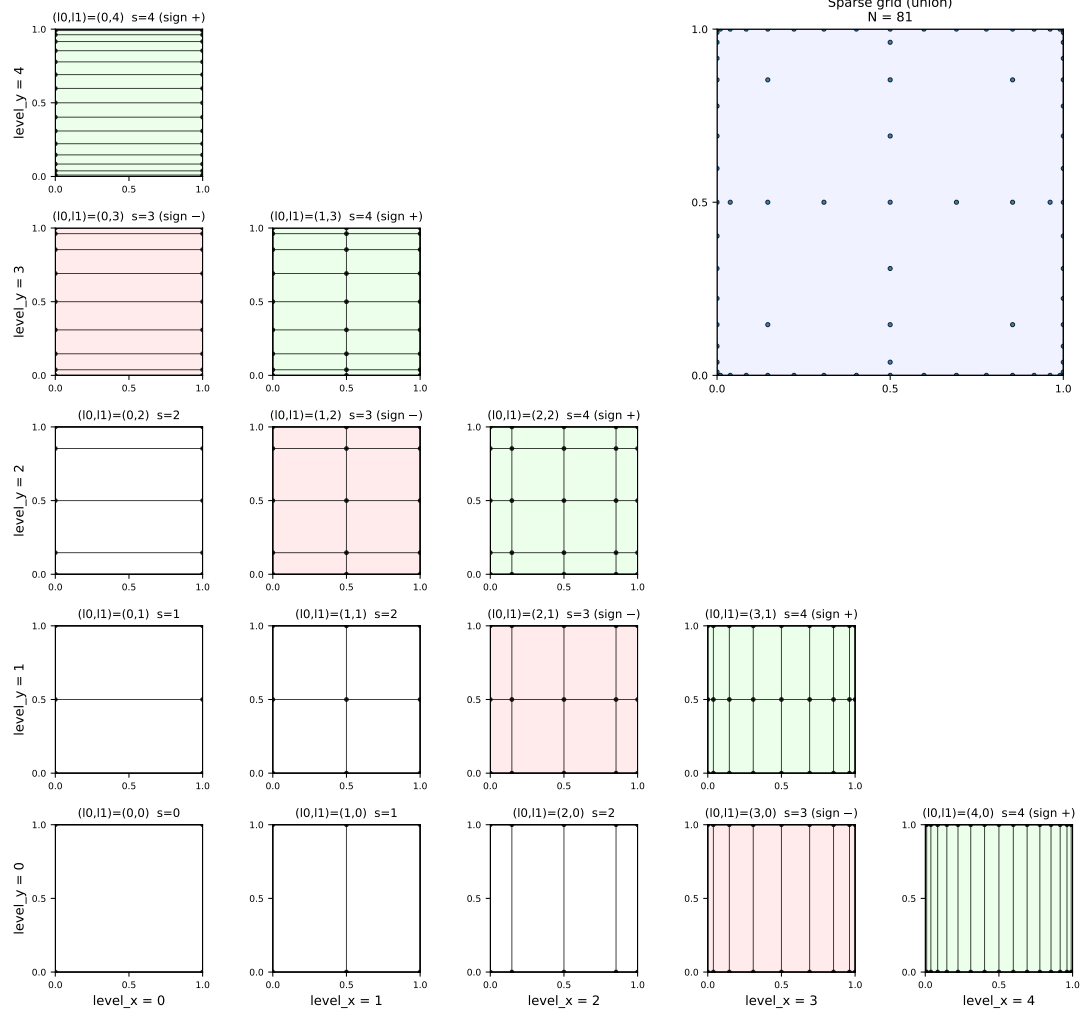
Das dünne Gitter entsteht dann aus der Vereinigung von anisotropen vollen Gittern, die in der Kombinationsformel auftreten. Es sind $N = O(2^\ell \ell^{d-1})$ Punkte, was deutlich weniger als die $O(2^{d\ell})$ Punkte eines vollen Gitters sind. Der Fehler dieser klassischen Dünngitter-Quadratur ist $O(N^{-r} \log^{(d-1)(r+1)}(N))$ für eine Funktion f mit einer gewissen Glattheit r .

In den folgenden Bildern sind mögliche anisotrope volle Gitter bis zum Level 4 für die Mittelpunktsregel, Trapezregel, Clenshaw-Curtis und Gauss-Legendre mit 2 Punkten abgebildet; die farbigen Gitter sind die benötigten Gitter in der Kombinationsformel, die das jeweils rechts oben gezeigte dünne Gitter erzeugen. Der Vorteil verschachtelter Gitter (Trapezregel und Clenshaw-Curtis) ist hier offensichtlich: Man braucht deutlich weniger Funktionsauswertungen.

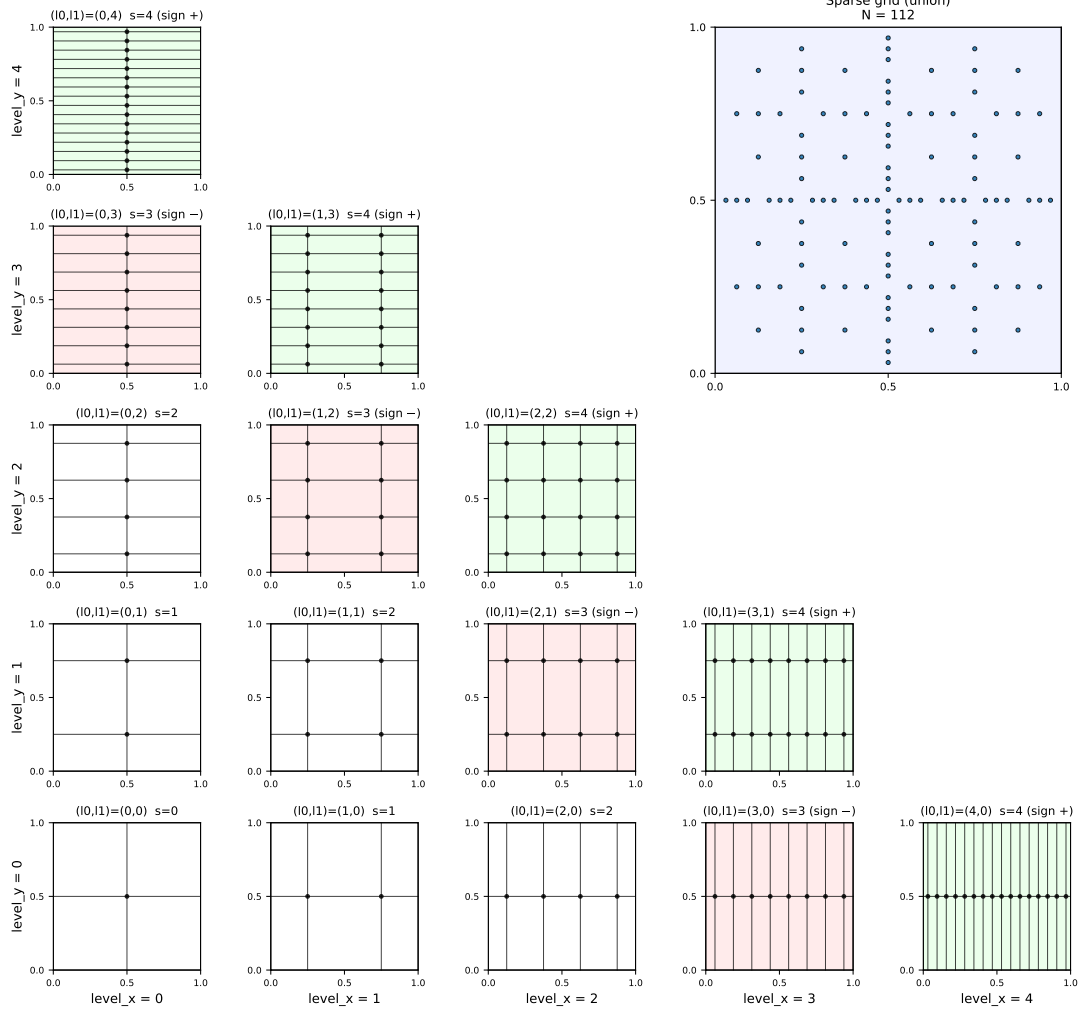
Anisotropic full grids up to $L=4$ — Trapezoid (equispaced, nested)
 (Smolyak diagonals $s=L-1$ (red) and $s=L$ (green) highlighted;



Anisotropic full grids up to $L=4$ — Clenshaw-Curtis (nested)
 (Smolyak diagonals $s=L-1$ (red) and $s=L$ (green) highlighted;



Anisotropic full grids up to $L=4$ — Midpoint (composite)
(Smolyak diagonals $s=L-1$ (red) and $s=L$ (green) highlighted;



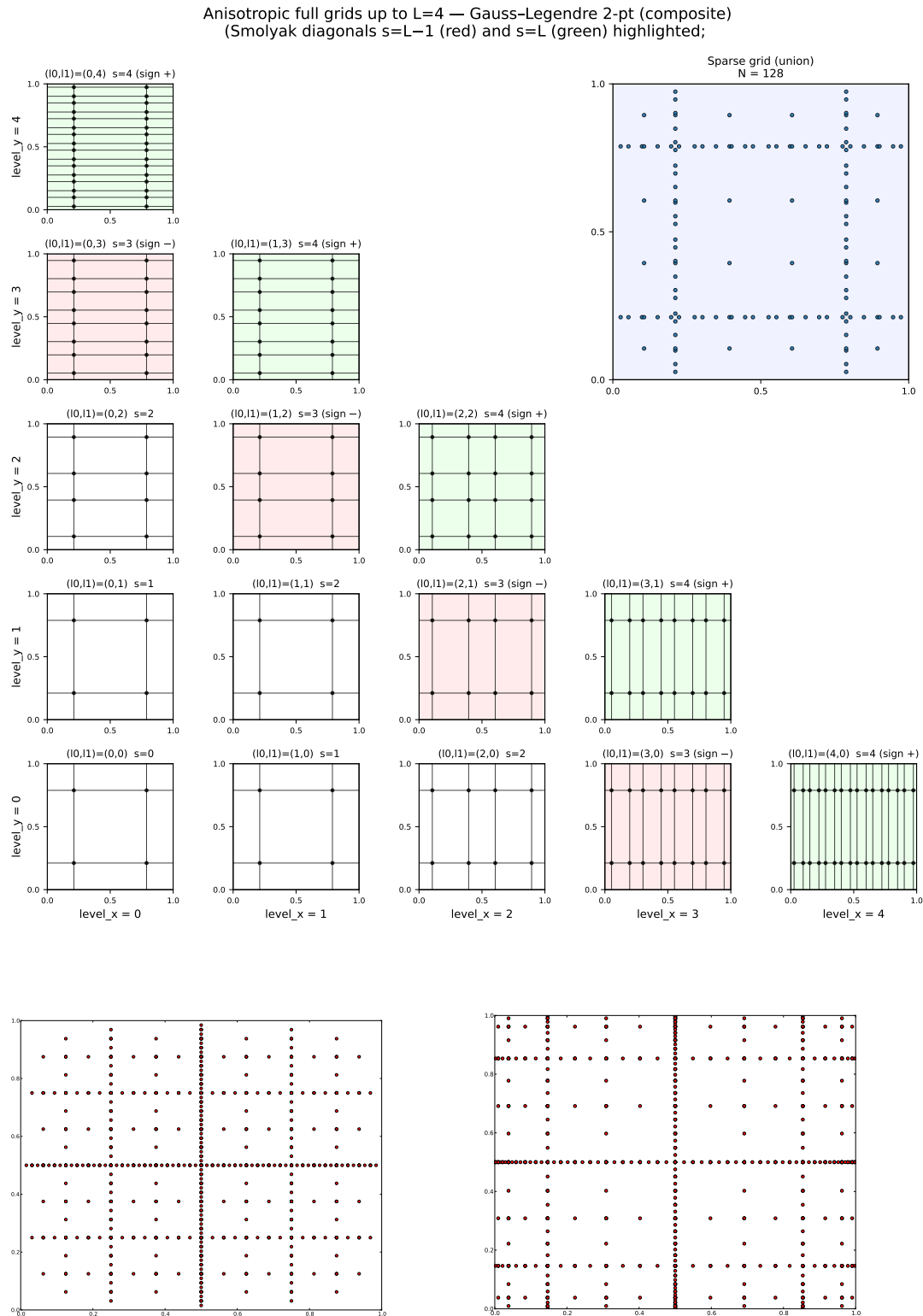
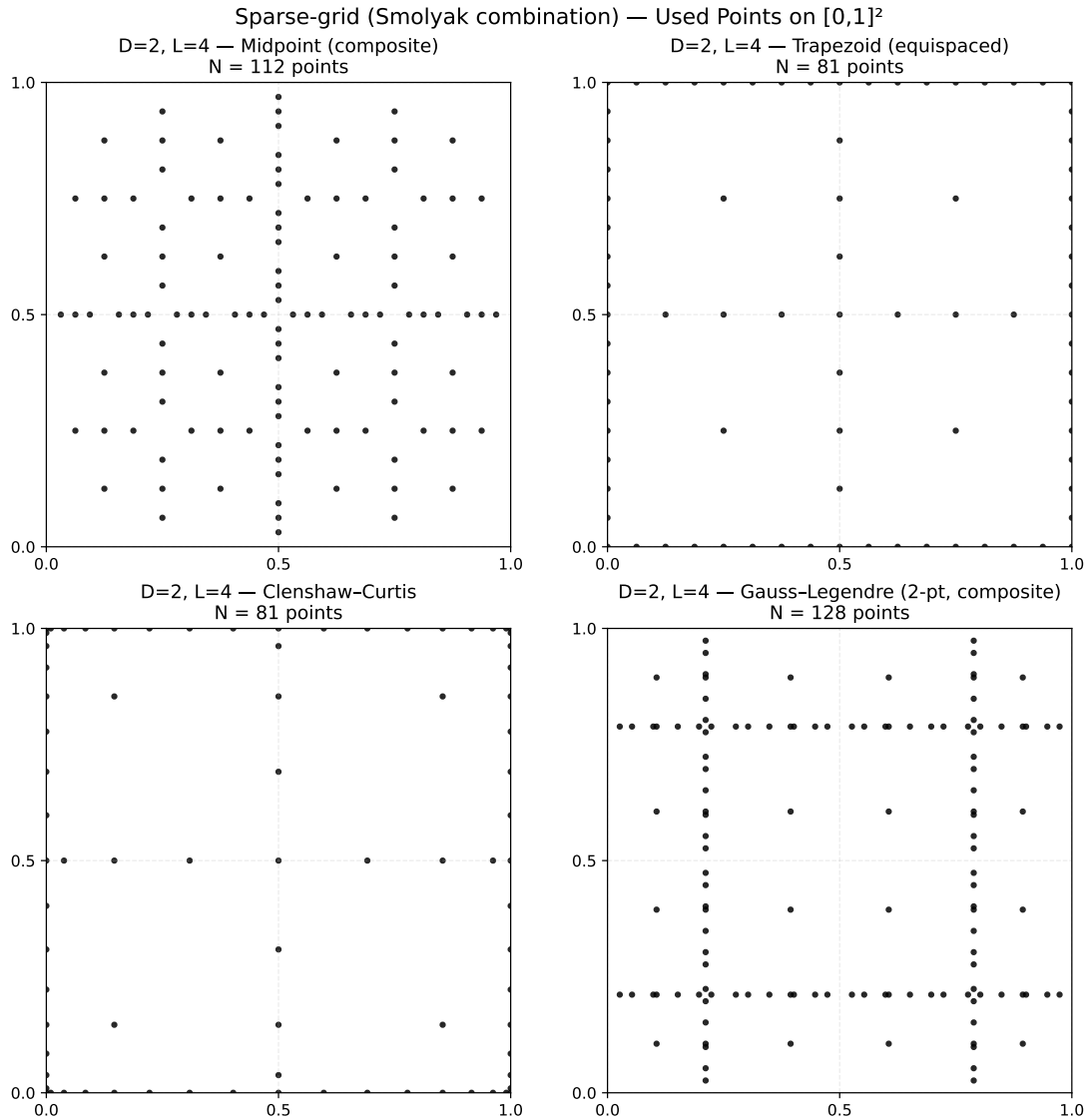


Abb. 5.7.2. Dünne Gitter, die auf der Mittelpunktsregel (links) und der offenen Clenshaw-Curtis-Regel (rechts) basieren.



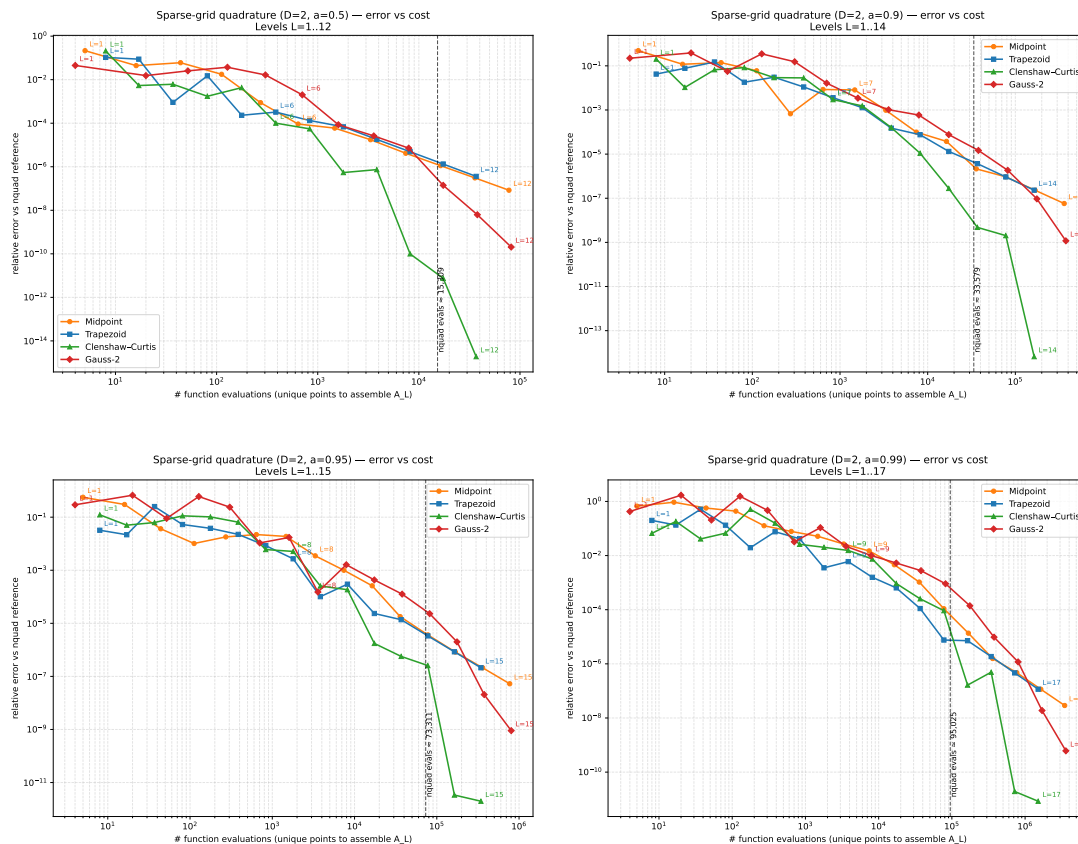
Die Clenshaw-Curtis-Regel auf einem vollen Gitter in $d = 3$ Dimensionen auf dem Level $n = 6$ verwendet 274625 Funktionsauswertungen und braucht zirka 6.6 Sekunden auf meinem Laptop (im Jahre 2011). Dieselbe Quadraturformel, aber auf dem dünnen Gitter auf dem Level $n = 6$, braucht nur 3120 Funktionsauswertungen und 0.1 Sekunden. Der Fehler bei Verwendung des vollen Gitters beträgt $1.5 \cdot 10^{-5}$, während er in der Dünngitter-Quadratur bei $1.5 \cdot 10^{-6}$ liegt, für den Integranden $f(\mathbf{x}) = (1 + \frac{1}{d})^d \sqrt{x_1 \cdots x_d}$.

Dasselbe Beispiel in $d = 4$ Dimensionen führt bei der Verwendung eines vollen Gitters zirka 17.8 Millionen Funktionsauswertungen aus, hat einen Fehler von $3.3 \cdot 10^{-5}$ und braucht 426 Sekunden, während man im Falle eines dünnen Gitters 9065 Funktionsauswertungen in 0.3 Sekunden mit einem Fehler von $8 \cdot 10^{-6}$ braucht.

Die Kombinationsformel lässt sich einfach auf parallelen Rechnerarchitekturen implementieren. Für $d = 10$ Dimensionen auf dem Level $n = 6$ braucht dieser parallele Code nur 33 Sekunden und hat einen Fehler von $3 \cdot 10^{-5}$. Die Kombinationsformel ist in diesem Falle eine Summe mit 3003 Termen.

Die Verallgemeinerung (5.6.21) der Funktion aus dem Beispiel 5.4.8 ist deutlich anspruchsvoller. Das separate File beinhaltet einige Bilder von dünnen Gittern dazu.

Effiziente Algorithmen können die dünnen Gitter sehr kompetitiv für Dimensionen $3 \leq d \leq 12$ machen. Für $d = 2$ ist die Clenshaw-Curtis Quadratur auf dünnen Gittern auch ohne Adaptivität für die schwierige Funktion (5.6.21) relativ zufriedenstellend:



Mehr über die dünnen Gitter findet man in den Büchern:

M. HOLTZ, *Sparse grid quadrature in high dimensions with applications in finance and insurance*, Berlin, Springer 2011, [NEBIS 009878866](#)

J. GARCKE, *Sparse grids and applications*, Heidelberg, Springer 2013, [NEBIS 009895238](#)

5.8 Monte-Carlo Quadratur

Grundrezept.

Manchmal ist es nicht klar, welche Quadratur man anwenden sollte; es kann auch sein, dass eine Quadratur eine zu lange Laufzeit bräuchte. In solchen Fällen verwendet man die Monte-Carlo Quadratur. Oft wird sie benutzt, um eine erste Näherung des Wertes eines Integrals zu bekommen. Diese Technik wird insbesondere für

hochdimensionale Probleme oder bei sehr unglatten Integranden verwendet.

Wir nehmen N Zahlen (N ist normalerweise eine grosse Zahl) $t_i \in [0, 1]$ zufällig aus der uniformen Verteilung auf $[0, 1]$, (wir werden das später genauer sehen; hier können uns vorstellen, dass bei der Ziehung der Zahlen t_i keine Gebiete aus $[0, 1]$ bevorzugt werden) und wir mitteln die Funktionswerte in diesem Intervall:

$$I = \int_0^1 z(t)dt \approx \frac{1}{N} \sum_{i=1}^N z(t_i).$$

Die Knoten t_i werden mithilfe eines Zufallszahlengenerators erzeugt. Wenn wir auf dem Intervall $[a, b]$ statt $[0, 1]$ arbeiten, können wir folgendermassen vorgehen:

$$I = \int_a^b z(s)ds = |b - a| \int_0^1 z(a + (b - a)t)dt \approx |b - a| \frac{1}{N} \sum_{i=1}^N z(s_i) =: I_N, \quad (5.8.22)$$

wobei $s_i = a + (b - a) \cdot t_i$. Der vorherige Wert ist ein Mittelwert. Da wir nun mit Zufallszahlen arbeiten, ist in jedem Experiment ein *verschiedener* Mittelwert zu erwarten. Was hier Konvergenz bedeutet, müssen wir noch klarstellen, siehe dazu Theorem 5.9.8.

Bemerkung 5.8.1. (Eigenschaften der Monte-Carlo-Methode)

- Die Konvergenz vom Monte-Carlo-Verfahren ist sehr langsam, nur $\mathcal{O}(N^{-\frac{1}{2}})$ (siehe ?? und 5.9.8); sie hängt aber nicht (direkt) von der Dimension oder von der Glattheit des Integranden ab!
- Diese Methode ist rein probabilistisch: Das Ergebnis kann völlig falsch sein, siehe dazu Figur 5.8.18.

Das Monte-Carlo-Verfahren wird in der Wahrscheinlichkeitsrechnung und Statistik mathematisch begründet. Wir versuchen hier eine intuitive Herangehensweise, die aber so weit wie möglich auf der mathematischen Sprache basiert.

Wir notieren

$$\tilde{\sigma}_N = \sqrt{\frac{\frac{1}{N} \sum_{i=1}^N z(t_i)^2 - \left(\frac{1}{N} \sum_{i=1}^N z(t_i)\right)^2}{N - 1}} = \sigma_N / \sqrt{N}. \quad (5.8.23)$$

Dann kann man rigoros beweisen (Central Limit Theorem): Wenn wir dieses Experiment sehr oft wiederholen, dann enthält das Intervall

$$[I_N - \tilde{\sigma}_N, I_N + \tilde{\sigma}_N] \quad (5.8.24)$$

den wahren Wert $\int_{[0,1]^d} z(t)dt$ in circa 68.3% der Fälle, falls die N Zahlen t_i aus der uniformen Verteilung auf $[0, 1]^d$ gezogen worden sind (und N selber gross ist).

Jede Monte-Carlo-Methode (nicht nur für Quadratur) braucht:

- ein Gebiet für das “Experiment”: hier $[0, 1]^d$;
- Zahlen, die mittels eines (guten) Zufallszahlengenerators erzeugt worden sind: hier sind sie die t_i ;
- deterministische Berechnungen: hier I_N aus (5.8.22) und $\tilde{\sigma}_N$ aus (5.8.24);
- eine Darstellung des Ergebnisses: hier $P(I \in [I_N - \tilde{\sigma}_N, I_N + \tilde{\sigma}_N]) = 0.683$.

Bemerkung 5.8.2. Die Dimension befindet sich hier nur in der Anzahl N der generierten Zufallszahlen. Die Länge des Intervalls nimmt wie $1/\sqrt{N}$ ab. Eine Verbesserung der Methode kann nur durch eine Verkleinerung der Varianz σ_N^2 erzielt werden. Das wird uns bald zu Techniken zur Reduktion der Varianz führen.

Die Theorie dahinter

Genauer verstanden werden alle diese Ideen erst nach einem Besuch von Vorlesungen in Wahrscheinlichkeit und Statistik. Hier sind nur einige Begriffe, die wir brauchen. Für uns ist eine Zufallsvariable eine Abbildung $X : \mathbb{R} \rightarrow \mathbb{R}$. Im mehrdimensionalen Fall ist $X : \mathbb{R}^d \rightarrow \mathbb{R}$ und die unteren Definitionen lassen sich einfach verallgemeinern. Achtung: die Zufälligkeit liegt im Argument der Funktion X .

Beispiel 5.8.3. Der Wert der Funktion X sei die Anzahl der Punkte, die beim Werfen eines Würfels erscheint. X kann also nur Werte in $\{1, 2, 3, 4, 5, 6\}$ annehmen. Die Wahrscheinlichkeit, dass X einen bestimmten Wert j , sagen wir $j = 5$, annimmt, ist $1/6$. Wir schreiben:

$$P(X = j) = \frac{1}{6}.$$

Definition 5.8.4. Die Verteilungsfunktion F für eine Zufallsvariable X ist definiert durch

$$F(x) = P(X \leq x).$$

Beispiel 5.8.5. Für das Würfelexperiment ist $P(X \leq 2) = 2/6$, also $F(2) = 2/6$.

Definition 5.8.6. Die Zufallsvariable X heisst (absolut) stetig, falls es eine Funktion f gibt, so dass

$$F(x) = \int_{-\infty}^x f(t) dt;$$

f heisst die Dichte von X .

Die Dichte von X zeigt bevorzugte Gebiete für die Werte von X an: $f(t)dt = P[X \in [t, t + dt]]$.

Beispiel 5.8.7. X ist *uniform verteilt* in $[a, b]$ und man schreibt $X \sim \mathcal{U}[0, 1]$, wenn die Dichte von X

$$f(x) = \begin{cases} \frac{1}{b-a}, & \text{wenn } x \in [a, b], \\ 0, & \text{sonst} \end{cases}$$

ist; zu erscheinen. Die entsprechende Verteilungsfunktion ist dann $F(x) = \int_a^x \frac{1}{b-a} dx = \frac{x-a}{b-a}$.

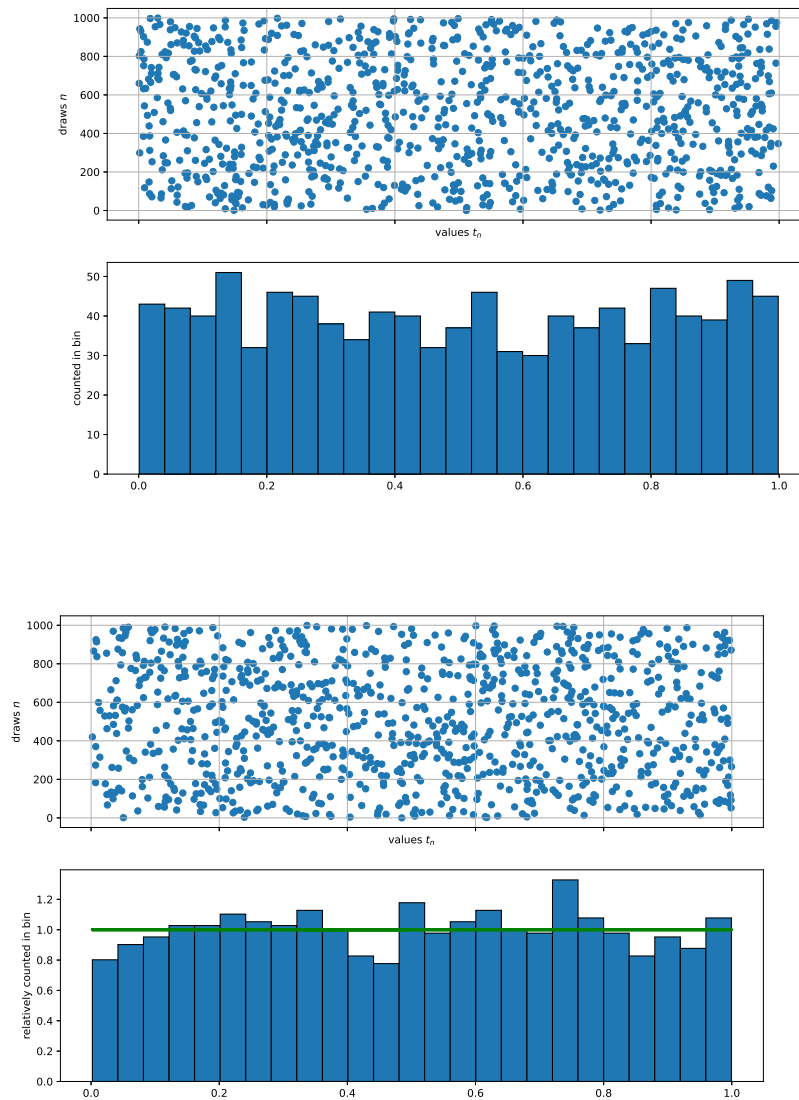


Abb. 5.8.8. Aus der uniformen Verteilung gezogene $N = 1000$ Zufallszahlen; oben: Anzahl im Körbchen, unten: eine andere Ziehung und normiert.

Beispiel 5.8.9. X ist *normalverteilt* mit Erwartungswert μ und Varianz σ^2 und man schreibt $X \sim \mathcal{N}(\mu, \sigma^2)$, wenn die Dichte von X

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

ist. Im Falle, wo $\mu = 0$ und $\sigma^2 = 1$, redet man von Standardnormalverteilung. Bei einer normalverteilten Zufallsvariablen haben die Zahlen rund um den Erwartungswert μ grössere Wahrscheinlichkeit zu erscheinen.

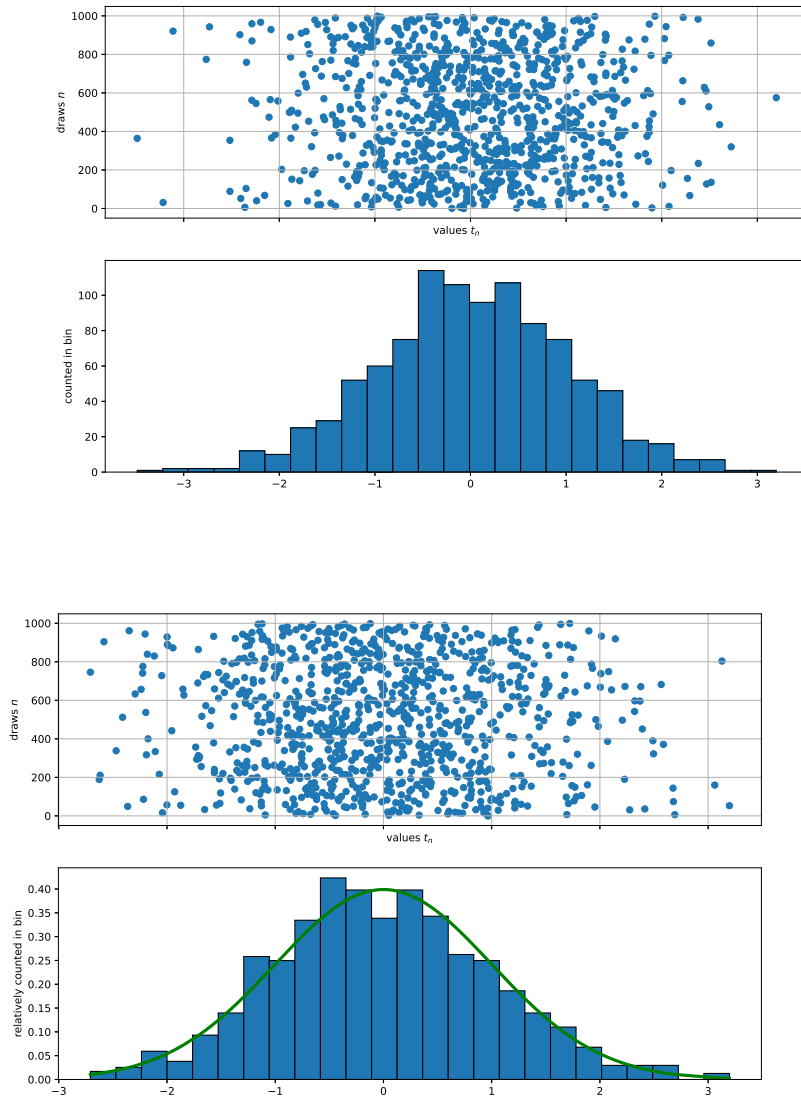


Abb. 5.8.10. Aus der normalen Verteilung gezogene $N = 1000$ Zufallszahlen; oben: Anzahl im Körbchen, unten: eine andere Ziehung und normiert.

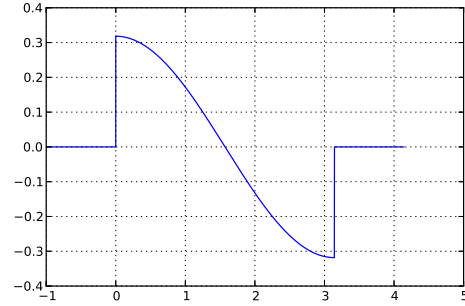
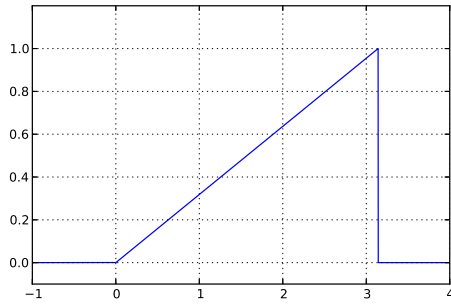
Definition 5.8.11. Für eine Zufallsvariable X mit Dichte f definieren wir den Erwartungswert von X als

$$\mathbb{E}(X) = \int_{\mathbb{R}} x f(x) dx$$

und den Erwartungswert von einer Funktion von X $z(X)$ als

$$\mathbb{E}(z(X)) = \int_{\mathbb{R}} z(x) f(x) dx .$$

Beispiel 5.8.12. Wenn $X \sim \mathcal{U}(0, \pi)$, dann $\mathbb{E}(X) = \int_{\mathbb{R}} x f(x) dx = \int_0^\pi x \frac{1}{\pi} dx = \frac{\pi}{2}$.
 $\mathbb{E}(\cos X) = \int_0^\pi \cos x \frac{1}{\pi} dx = 0$.



Bemerkung 5.8.13. Es gilt:

$$\mathbb{E}(az(X)) = a\mathbb{E}(z(X)) \text{ und } \mathbb{E}(z(X) + f(X)) = \mathbb{E}(z(X)) + \mathbb{E}(f(X)) ,$$

d.h. $\mathbb{E}$ ist ein linearer Operator. Wenn keine zweideutige Bedeutung möglich ist, lassen wir die Klammern weg und schreiben einfach $\mathbb{E}z(X)$.

Definition 5.8.14. Wir nennen die Varianz der Zufallsvariable X die erwartete quadratische Abweichung vom Erwartungswert:

$$\mathbb{V}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}(X^2) - (\mathbb{E}X)^2 .$$

Bemerkung 5.8.15. Die Varianz ist kein linearer Operator, denn

$$\mathbb{V}(az(X)) = a^2\mathbb{V}(z(X)) ,$$

aber wenn X_1 und X_2 zwei unabhängige Zufallsvariablen sind, dann gilt

$$\mathbb{V}(z(X_1) + z(X_2)) = \mathbb{V}(z(X_1)) + \mathbb{V}(z(X_2)) .$$

Das Monte-Carlo-Verfahren beruht auf der einfachen Bemerkung, dass

$$\int_{[0,1]^d} z(x)dx = \mathbb{E}z(X) \text{ mit } X \sim \mathcal{U}([0,1]^d) . \quad (5.8.25)$$

Um das Integral $\int_{[0,1]^d} z(x)dx$ zu approximieren, reicht es also aus, eine Schätzung des Erwartungswertes $\mathbb{E}z(X)$ zu erhalten. Die Monte-Carlo-Methode ist die Kunst, einen Erwartungswert durch den Mittelwert der Funktionswerte simulierter Zufallsvariablen zu approximieren. Es werden stochastische Experimente durchgeführt, die Werte x_i einer Zufallsvariablen X , die uniform verteilt auf $[0,1]^d$ ist, produzieren; man spricht von "Realisierungen" von X . Ein Schätzer ergibt dann aufgrund dieser Werte eine Schätzung von $\mathbb{E}z(X)$. Die Genauigkeit der Approximation wird von der Varianz $\mathbb{V}(X)$ beeinflusst. Wenn wir N Werte verwenden wollen, dann führen wir eben N voneinander unabhängige stochastische Experimente durch, um die N Werte x_i zu produzieren. Wir haben es also eigentlich mit N unabhängigen Zufallsvariablen $X_1, \dots, X_N$ zu tun. Diese Zufallsvariablen sind alle auf $[0,1]^d$ uniform verteilt. Ein guter Schätzer für den Erwartungswert von X ist die Zufallsvariable

$$m_N(z(X)) := \frac{1}{N} \sum_{i=1}^N z(X_i) \quad (5.8.26)$$

und eine Schätzung aufgrund der Realisierung $\mathbf{x} = (x_1, \dots, x_N)$ ist

$$m_N(z(\mathbf{x})) := \frac{1}{N} \sum_{i=1}^N z(x_i) .$$

Bemerkung 5.8.16. Es ist einfach zu sehen, dass

$$\mathbb{E}m_N(z(X)) = N \frac{1}{N} \mathbb{E}z(X) = \int_{[0,1]^d} z(x) dx .$$

Wie gut ist die Approximation von $I = \mathbb{E}z(X)$ durch $m_N(\mathbf{x})$? Das lässt sich nicht so leicht sagen. Ein Vertrauensintervall wird uns zusätzliche Information geben, die uns einen Einblick auf die mögliche (doch nicht tatsächliche) Genauigkeit dieser Approximation ermöglicht. Intuitiv erhält man eine bessere Approximation bei kleinerer Varianz:

$$\mathbb{V}m_N(z(X)) = \mathbb{V}\left(\frac{1}{N} \sum_{i=1}^N z(X_i)\right) = \frac{1}{N^2} N \mathbb{V}(z(X)) = \frac{1}{N} \mathbb{V}(z(X)) \longrightarrow 0 .$$

Die zwei Gesetze der grossen Zahlen beschreiben verschiedene *Konvergenzarten*, die das Monte-Carlo-Verfahren begründen. Der zentrale Grenzwertsatz (Central Limit Theorem) besagt, dass für grosses N die Zufallsvariable

$$\frac{m_N(z(X)) - \mathbb{E}z(X)}{\sqrt{\frac{1}{N} \mathbb{V}(z(X))}}$$

sich fast wie eine normalverteilte $\mathcal{N}(0, 1)$ Zufallsvariable verhält. Mathematisch sagt man, dass die Verteilungsfunktion dieser Zufallsvariable punktweise gegen die Verteilungsfunktion der $\mathcal{N}(0, 1)$ Zufallsvariable konvergiert. Daraus folgt mit der Notation $\sigma(z(X)) = \sqrt{\mathbb{V}(z(X))}$:

$$P\left(|m_N(z(X)) - \mathbb{E}z(X)| \leq \frac{\lambda \sigma(z(X))}{\sqrt{N}}\right) = \frac{1}{\sqrt{2\pi}} \int_{-\lambda}^{\lambda} e^{-x^2/2} dx + \mathcal{O}\left(\frac{1}{\sqrt{N}}\right) .$$

Wir sehen hier bereits die langsame Konvergenz des Monte-Carlo-Verfahrens. Einerseits haben wir eine langsame Konvergenz $\mathcal{O}(\frac{1}{\sqrt{N}})$ für die Verteilungsfunktionen (punktweise). Der Wert des Integrals $p(\lambda) = \frac{1}{\sqrt{2\pi}} \int_{-\lambda}^{\lambda} e^{-x^2/2} dx$ hängt nur von der Länge des Integrationsintervalls 2λ ab und stellt eine $\mathcal{O}(\frac{1}{\sqrt{N}})$ -gute Approximation der Wahrscheinlichkeit, dass der Fehler $|m_N(z(X)) - \mathbb{E}z(X)|$ kleiner als $\frac{\lambda \sigma(z(X))}{\sqrt{N}}$ ist, dar. Eine Wahrscheinlichkeit $p(\lambda) = 0.5$ erhalten wir für $\lambda = 0.6745$, $p(\lambda) = 0.9$ braucht $\lambda = 1.645$ und $p(\lambda) = 0.95$ braucht $\lambda = 1.9605$. Die Wahl $\lambda = 1$ ergibt die am Anfang als “magische Zahl” angegebene Wahrscheinlichkeit $p(1) = 0.683$. Diese Zahlen sehen anders aus, wenn N nicht so gross ist, denn dann müsste man mit der Student t -Verteilung statt mit der normalen Verteilung arbeiten. Hier nehmen

wir an, dass N gross genug ist ($N \geq 30$). Andererseits nimmt die Länge des Vertrauensintervalls nur wie $1/\sqrt{N}$ ab. Ein kürzeres Intervall entspricht einem erwarteten kleineren Fehler, es lässt sich aber nur mit erheblich mehr Zufallszahlen realisieren.

Den Wert $\sigma(z(X)) = \sqrt{V(z(X))}$ haben wir hier als bekannt angenommen. Um für eine Realisierung $\mathbf{x}$ auch ein Vertrauensintervall aufgrund voriger Aussage zu bauen, ohne $\sigma(z(X))$ exakt zu kennen, brauchen wir zunächst einen Schätzer für $V(z(X)) = \mathbb{E}(z(X)^2) - (\mathbb{E}z(X))^2$. Der Schätzer

$$d_2^*(z(X)) := \frac{1}{N-1} \sum_{j=1}^N (z(X_j) - m_N(z(X)))^2 = \frac{N}{N-1} (m_N(z(X)^2) - (m_N(z(X)))^2)$$

hat die gewünschte Eigenschaft $\mathbb{E}d_2^*(z(X)) = V(z(X))$. Eine Schätzung von $\sqrt{V(z(X))}$ ist also

$$\sqrt{\frac{N}{N-1} (m_N(z(\mathbf{x})^2) - (m_N(z(\mathbf{x})))^2)},$$

was die Schätzung (5.8.23)

$$\sqrt{\frac{1}{N-1} (m_N(z(\mathbf{x})^2) - (m_N(z(\mathbf{x})))^2)}$$

für $\sqrt{V(z(X))}/\sqrt{N}$ liefert. Somit erhalten wir

Theorem 5.8.17. Das Intervall

$$[m_N(z(\mathbf{x})) - \sqrt{\lambda d_2^*(z(X))}/\sqrt{N}, m_N(z(\mathbf{x})) + \lambda \sqrt{d_2^*(z(X))}/\sqrt{N}]$$

enthält den wahren Wert $I = \mathbb{E}z(X)$ mit der Wahrscheinlichkeit $p(\lambda) = \frac{1}{\sqrt{2\pi}} \int_{-\lambda}^{\lambda} e^{-x^2/2} dx$ bis auf einen $\mathcal{O}(\frac{1}{\sqrt{N}})$ -Fehler.

Für das berechnete Vertrauensintervall

$$[m_N(z(\mathbf{x})) - \lambda \tilde{\sigma}_N, m_N(z(\mathbf{x})) + \lambda \tilde{\sigma}_N]$$

kann man in einem Experiment nichts sagen. Das Theorem sagt aber, wenn wir dieses Experiment M -mal (mit M gross) wiederholen, dann enthält das Vertrauensintervall den wahren Wert I in circa $100p(\lambda)$ Prozent der M Fälle.

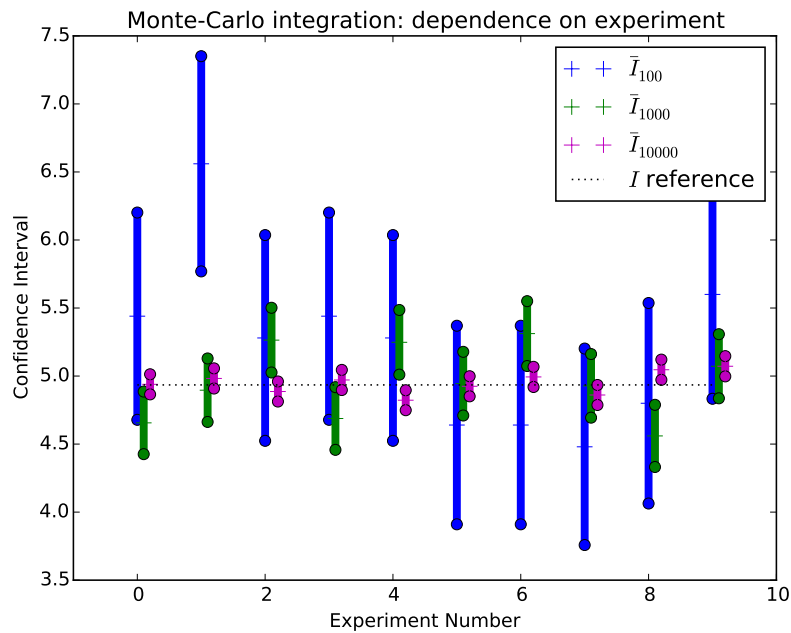


Abb. 5.8.18. MC-Approximation und 10 Konfidenzintervalle aus Code 5.8.21 für verschiedene Anzahl N der Realisierungen.

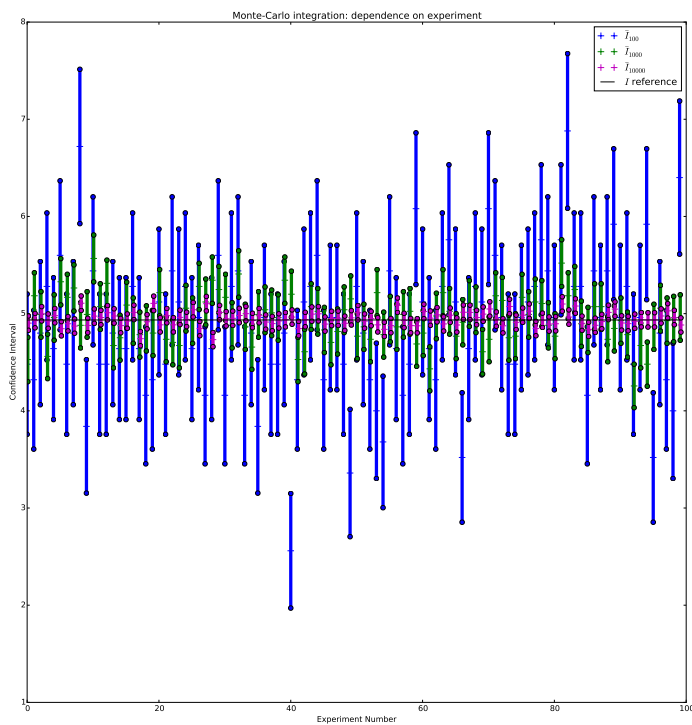


Abb. 5.8.19. MC-Approximation und 100 Konfidenzintervalle aus Code 5.8.21 für verschiedene Anzahl N der Realisierungen.

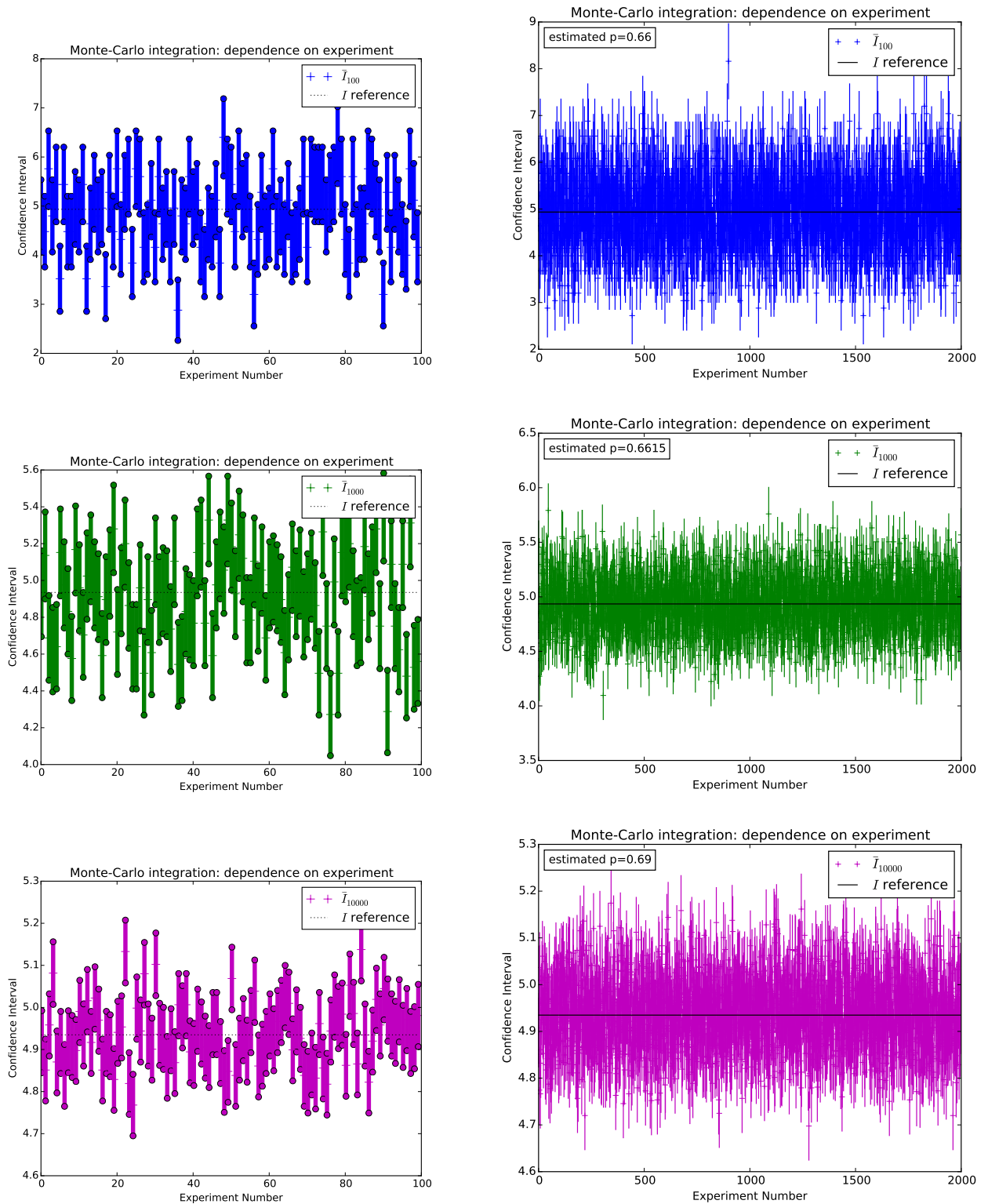


Abb. 5.8.20. MC-Approximationen und $M = 100$ (links) bzw. $M = 2000$ (rechts) Konfidenzintervalle aus Code 5.8.21 für verschiedene Anzahl N der Realisierungen; bitte beachten Sie die Proportion p der Intervalle, die den wahren Wert des Integrals enthalten; diese sollte (für grosses M) nah an 0.683 sein.

Code 5.8.21: Monte-Carlo für das Volumen der Einheitskugel

```

1  import numpy as np
   import matplotlib.pyplot as plt
3  import math

5  def mc_experiments(func,a,b,N,M, rng):

7      """
   Purpose: perform M experiments of Monte-Carlo method

9      Input: func ... function to integrate
11             a ... lower interval bound
13             b ... upper interval bound
15             N ... number of samples
17             M ... number of experiments
19             rng ... random number generator to be used

21      Output: Imean ... mean of Monte-Carlo approx. to integral over
23              experiments
25              Ivar2 ... (empirical) variance of Monte-Carlo approx.
27              to integral over experiments

29      Notes: None

31      """

33      # perform experiments
35      I = np.zeros(M) #this is an array of length M
37      for iexp in np.arange(M):
39          [I[iexp],ivar] = mcquad(func,a,b,N, rng)
41          #we calculate an approximation with N calls, M times and store all the
43          values in I

45      # compute mean of experiments
47      Imean = sum(I)/M

49      # compute square (empirical) variance of experiments
51      Ivar = 0.
   if M>1:
       Ivar = sum(I**2)/(M-1.) - (M/(M-1.))*(Imean**2)
   else:
       Ivar = ivar # this is what mcquad returns, here sigmatilde**2

   # return
   return Imean,Ivar

def mcquad(func,a,b,N, rng):

    """
    Purpose: compute definite integral of func enclosed between a & b
           with the Monte-Carlo method

    Input: func ... function to integrate

```

```

53     a    ... lower interval bound
54     b    ... upper interval bound
55     N    ... number of samples
56     rng  ... random number generator to be used

57     Output: I    ... Monte-Carlo approximation to integral
58             var2 ... (sample) variance/sqrt(N)
59
60     Notes: None
61
62     """
63
64     # dimension of the domain
65     if ( len(a) == len(b) ):
66         dim = len(a)
67     else:
68         return
69
70     # generate N samples in [a,b]
71     x = rng.random((dim,N)) #rand generates a matrix of given diensions with random
numbers
72     #dim is the dimension of the space on which f takes its values, in the simplest
case dim=1
73     #but we often use Monte Carlo for higher-dimensional integrals
74     for idim in np.arange(dim):
75         #we are working on the interval [a,b] but we generate numbers in [0,1], so
we have to change interval
76         x[idim,:] = a[idim] + (b[idim] - a[idim])*x[idim,:]
77
78     # volume of domain [a,b]
79     vol = 1.
80     for idim in np.arange(dim):
81         vol = vol*(b[idim] - a[idim]) #compare with the simple 2-d case of a
rectangular surface
82
83     # evaluate function at sample points
84     fx = func(x)
85
86     # evaluate integral (i.e. compute the expectation value)
87     I = np.sum(fx)/N
88
89     # compute sigmatilde**2 here not further used
90     if ( N > 1 ):
91         #we use the estimator for sigma given in the script
92         var2 = (np.sum(fx**2)/N - I**2)/(N - 1)
93     else:
94         var2 = 0.
95
96     return I*vol, var2*vol**2
97
98     if __name__ == "__main__":
99         # reproductibility: fix, seed, numpy version, avoid parallel randomness
100         seed_value = 1903
101         rng = np.random.default_rng(seed_value)
102
103         # characteristic function of unit sphere in d-dimensions

```

```

def f(x):
105     [dim,n] = np.shape(x)
        r2 = 0.
107     for idim in np.arange(dim):
            r2 = r2 + x[idim,:]**2 #with this recursion we compute  $\langle x, x \rangle^2$ 
109     fx = 1.*(r2 <= 1)  #if  $\langle x, x \rangle^2 < 1$  then x is in the unit sphere, so f(x)=1.
    Otherwise f(x)=0
    #this allows us to calculate the volume of the unit sphere via integration of
    f(x)
111     return fx

    # parameters
    dim = 4 # dimension
115     a = - np.ones(dim) # lower bound
    b = + np.ones(dim) # upper bound
117     Iref = np.pi**(dim/2.)/math.gamma(dim/2.+1) # reference volume of
                                                    # unit sphere in d-dimensions
119     M = 1 # number of experiments
    N = 4*10**np.arange(1,7) # number of samples (list!)
121
    # convergence study
123     Imean_1 = [] # initialize list for experiments means of the integral
    Ivar_1 = [] # initialize list for experiments variance of the integral
125     error_1 = [] # initialize list for error
    for Nk in N:
127         [Imean_k,Ivar_k] = mc_experiments(f,a,b,Nk,M, rng) # here M=1
        Imean_1.append(Imean_k)
129         Ivar_1.append(Ivar_k) # M=1 so this is  $\sigma^2$  for each Nk
        error_1.append(np.abs(Imean_k - Iref)) #the corresponding error
131
    M = 100
    # convergence study
133     Imean = [] # initialize list for experiments means of the integral
    Ivar = [] # initialize list for experiments variance of the integral
135     error = [] # initialize list for error
    for Nk in N:
137         [Imean_k,Ivar_k] = mc_experiments(f,a,b,Nk,M, rng)
        Imean.append(Imean_k)
139         Ivar.append(Ivar_k) # here M>1 hence N-averaged  $\sigma^2$ 
        error.append(np.abs(Imean_k - Iref)) #the corresponding error
141
    #----- plots -----
143     def plot_1():
        plt.figure(1)
        plt.clf()
147         plt.loglog(N,error_1,'b-+',label='Single') #we plot using a logarithmic scala
        plt.axis([N[0],N[-1],1e-5,1e2])
149         plt.title('Monte-Carlo integration')
        plt.xlabel('$N$')
151         plt.ylabel('$Error$')
        plt.legend()
153         plt.grid()

155     def plot_2():
        plt.figure(1)

```

```

157     plt.loglog(N,error,'r-+',label='Average over M='+str(M)+' experiments')
158     plt.legend()
159     plt.grid()

161 def plot_3():
162     plt.figure(1)
163     plt.loglog([1.e0,1.e6],[1.e-2,1.e-5],'k--',label=r'$\propto 1/\sqrt{N}$')
164     plt.legend()
165     plt.grid()

167 def plot_4():
168     plt.figure(2)
169     plt.clf()
170     plt.semilogx([N[0],N[-1]], [Iref,Iref], 'k:', label=r"$I$ reference")
171     plt.semilogx(N,Imean_1, 'r-+', label=r"$\bar{I}_N$")
172     plt.semilogx(N,Imean_1 - np.sqrt(Ivar_1), 'r^--', label=r"$\bar{I}_N - \sqrt{Ivar_1}$")
173     plt.semilogx(N,Imean_1 + np.sqrt(Ivar_1), 'rv--', label=r"$\bar{I}_N + \sqrt{Ivar_1}$")
174     plt.title('Monte-Carlo integration: $I$ run of length $N$')
175     plt.xlabel('$N$')
176     plt.ylabel('$I_N$')
177     plt.legend()
178     plt.grid()

180 def plot_5():
181     plt.figure(2)
182     plt.clf()
183     plt.semilogx([N[0],N[-1]], [Iref,Iref], 'k:', label=r"$I$ reference")
184     plt.semilogx(N,Imean, 'r-+', label=r"$\bar{I}_N$")
185     plt.semilogx(N,Imean - np.sqrt(Ivar), 'r^--', label=r"$\bar{I}_N - \sqrt{Ivar}$")
186     plt.semilogx(N,Imean + np.sqrt(Ivar), 'rv--', label=r"$\bar{I}_N + \sqrt{Ivar}$")
187     plt.title('Monte-Carlo integration: $M$ runs of length $N$')
188     plt.xlabel('$N$')
189     plt.ylabel('$I_N$')
190     plt.legend()
191     plt.grid()

193 plt.figure(3)
194 plt.clf()
195 plt.loglog(N,2*np.sqrt(Ivar), 'rv--', label=r"$2\sqrt{Ivar}$", linewidth=2)
196 plt.loglog(N,2/np.sqrt(N), 'k:', label=r"$2/\sqrt{N}$", linewidth=2)
197 plt.title('Monte-Carlo integration: dependence on $N$')
198 plt.xlabel('$N$')
199 plt.ylabel('Interval length')
200 plt.legend()
201 plt.grid()

203 def runfew(Nk, Mk):
204     Imean_Mk = [] # initialize list for experiments means of the integral
205     Ivar_Mk = [] # initialize list for experiments variance of the integral

```

```

211 error_Mk = [] # initialize list for error
    for m in Mk:
213     [Imean_m,Ivar_m] = mc_experiments(f,a,b,Nk,1, rng)
        Imean_Mk.append(Imean_m)
215     Ivar_Mk.append(Ivar_m)
        error_Mk.append(np.abs(Imean_m - Iref))
217 return Imean_Mk, Ivar_Mk, error_Mk

219 def run_and_plot():
    plt.figure(5)
221     plt.clf()
        Mk = np.arange(10)
223     # a few runs
        Nk = 100
225     Imean_Mk, Ivar_Mk, error_Mk = runfew(Nk,Mk)

227     Ileft_Mk = Imean_Mk - np.sqrt(Ivar_Mk)
        Iright_Mk = Imean_Mk + np.sqrt(Ivar_Mk)
229     for k in Mk:
        z = [k,k], [Ileft_Mk[k],Iright_Mk[k]]
231         plt.plot(z[0], z[1], 'bo-', markersize=10, linewidth=5)
        plt.plot(Mk,Imean_Mk, 'bD', markersize=10,
233 label=r"$\bar{I}_{"+str(Nk)+"}$")

        #
235     Nk = 1000
        Imean_Mk, Ivar_Mk, error_Mk = runfew(Nk,Mk)

237     Ileft_Mk = Imean_Mk - np.sqrt(Ivar_Mk)
        Iright_Mk = Imean_Mk + np.sqrt(Ivar_Mk)
239     for k in Mk:
        z = [k+0.1,k+0.1], [Ileft_Mk[k],Iright_Mk[k]]
241         plt.plot(z[0], z[1], 'go-', markersize=10, linewidth=5)
        plt.plot(Mk+0.1,Imean_Mk, 'gD', markersize=10,
243 label=r"$\bar{I}_{"+str(Nk)+"}$")

245     Nk = 10000
        Imean_Mk, Ivar_Mk, error_Mk = runfew(Nk,Mk)

247     Ileft_Mk = Imean_Mk - np.sqrt(Ivar_Mk)
        Iright_Mk = Imean_Mk + np.sqrt(Ivar_Mk)
249     for k in Mk:
        z = [k+0.2,k+0.2], [Ileft_Mk[k],Iright_Mk[k]]
251         plt.plot(z[0], z[1], 'mo-', markersize=10, linewidth=5)
        plt.plot(Mk+0.2,Imean_Mk, 'mD', markersize=10,
253 label=r"$\bar{I}_{"+str(Nk)+"}$")

255     plt.plot(Mk,Iref*(1+0.*Mk), 'k:', label=r"$I$ reference")
        plt.xlim(-1,10)
257     plt.title('Monte-Carlo integration: dependence on experiment')
        plt.xlabel('Experiment Number')
259     plt.ylabel('Confidence Interval')
        plt.legend()
261     plt.grid()

```

Bemerkung 5.8.22. Ein kurzes Vertrauensintervall entspricht einer niedrigen Wahrscheinlichkeit; grosse Wahrscheinlichkeit, dass das Vertrauensintervall den wahren Wert enthält, ist nur für eine grössere Intervalllänge möglich. Im “Grundrezept” werden $\lambda = 1$ und entsprechend $p(1) = 0.683$ verwendet.

Gegeben eine akzeptable Wahrscheinlichkeit, bekommt man ein kleineres Vertrauensintervall, entweder wenn man ein viel grösseres N wählt (aber nur $1/\sqrt{N}$ kleiner) oder wenn man die Berechnungen so organisiert, dass die Varianz $\mathbb{V}(z(X))$ kleiner wird. Dies werden wir nachher untersuchen, im Kapitel über die Reduktion der Varianz.

Bemerkung 5.8.23. Um das stochastische Experiment korrekt durchführen zu können, brauchen wir gute Zufallszahlengeneratoren. Die generierten Zahlen sollten einerseits statistische Tests für die entsprechende Verteilung erfüllen und gleichzeitig reproduzierbar sein.

Das ist auf den ersten Blick erstaunlich: reproduzierbare Zufallszahlen? Für eine numerische Simulation brauchen wir viele verschiedene komplexe Tools: physikalische Modelle, mathematische Vereinfachungen, Approximationen und numerische Algorithmen, Implementierungen in verschiedenen Codes, etc. Wenn man eine Diskrepanz mit dem physikalischen Experiment oder mit Erwartungen hat, die aus anderen Theorien stammen, und somit einen Fehler vermutet, der eventuell in den Berechnungen versteckt ist, muss man im Stande sein, die Berechnungen zu wiederholen, um eben den Fehler zu suchen. Ernsthafte wissenschaftliche Arbeit muss immer reproduzierbar sein (sonst grenzt es an Schamanismus)⁵. Deshalb werden Pseudo-Zufallszahlen verwendet, die von deterministischen Algorithmen erzeugt werden. Es gibt einige gute und viele schlechte Zufallszahlengeneratoren. Man sollte immer diejenigen verwenden, die sich bereits als gut geeignet für ähnliche Probleme gezeigt haben. Für wichtige Rechnungen sollte man die erzielten Ergebnisse unter Verwendung verschiedener Arten von Zufallsgeneratoren überprüfen. Man sollte immer die neuesten Generatoren verwenden, da ist `numpy` ziemlich gut.

5.9 Methoden zur Reduktion der Varianz

Mit dem Monte-Carlo-Verfahren kann man genauere Ergebnisse bekommen, wenn man die Varianz reduziert. Das Prinzip ist folgendes: Wenn wir den exakten Wert eines Integrals kennen, dann sollten wir ihn auch verwenden⁶. Als erstes Beispiel betrachten wir das folgende Integral:

⁵Die Ergebnisse, die mit teuren Supercomputern erzielt werden, können allerdings aus rein physikalischen Gründen oft nicht reproduzierbar sein. Umdenken und Reformulierungen der Reproduzierbarkeitsanforderungen sind zu erwarten.

⁶und zwar, bis nichts mehr zufällig bleibt, wenn möglich: “You use Monte Carlo until you understand the problem”, Mark Kac.

$$I_0(x) = \frac{1}{\pi} \int_0^\pi e^{-x \cos t} dt .$$

$I_0(x)$ ist eine modifizierte Bessel-Funktion der Ordnung 0, die in der Physik vorkommt. Zuerst approximieren wir $I_0(1)$ mittels eines einfachen Monte-Carlo-Verfahrens.

Code 5.9.1: Plain Monte-Carlo

```

1  """
2  Computes integral
3  I0(1) = (1/pi) int(z=0..pi) exp(-cos(z)) dz by raw MC.
4  Abramowitz and Stegun give I0(1) = 1.266066
5  """
6
7  import numpy as np
8  import time
9
10 # reproductibility: fix, seed, numpy version, avoid parallel randomness
11 seed_value = 1903
12 rng = np.random.default_rng(seed_value)
13
14 t1 = time.time()
15
16 M = 100 # number of times we run our MC inetgration
17 asval = 1.266065878 #exact value
18 ex = np.zeros(M)
19 print('A and S tables: I0(1) = ',asval)
20 print('sample      variance      MC I0 val')
21 print('-----      -')
22 k = 5 # how many experiments
23 N = 10**np.arange(1,k+1) #we take 10,100,...10^k sample points
24 v = []; e = [] #containers for variance and expectation value
25 for n in N:
26     for m in range(M):
27         x = rng.random(n) # sample
28         x = np.exp(np.cos(-np.pi*x))
29         ex[m] = sum(x)/n # quadrature
30     ev = sum(ex)/M #estimate the value of the integral
31     vex = np.dot(ex,ex)/(M-1.)
32     vex -= (M/(M-1.))*ev**2 #estimate the variance
33     v += [vex]; e += [ev] #store the values in the dedicated containers
34     print(n, vex, ev)
35
36 t2 = time.time() #measure the time needed for the operations
37 t = t2-t1
38
39 print("Serial calculation completed, time = %s s" % t)

```

Wir betrachten zwei Techniken zur Reduktion der Varianz: “Control Variates” und “Importance Sampling”.

Control Variates

Das Integral

$$I = \int_0^1 f(t) dt$$

kann man umschreiben als

$$\begin{aligned} I &= \int_0^1 (f(t) - \phi(t)) dt + \int_0^1 \phi(t) dt \\ &\approx \frac{1}{N} \sum_{i=1}^N [f(t_i) - \phi(t_i)] + I_\phi. \end{aligned}$$

Wir nehmen nun ϕ so, dass:

- $\phi(u) \approx f(u)$ und
- $I_\phi = \int_0^1 \phi(u) du$ bekannt ist.

Für unser Beispiel schreiben wir

$$\begin{aligned} f(t) &= e^{-\cos(\pi t)} \\ &= 1 - \cos(\pi t) + \frac{1}{2}(\cos(\pi t))^2 + \dots \end{aligned}$$

und nehmen

$$\phi(t) = \text{die ersten drei Terme.}$$

Wir integrieren $\phi(t)$ analytisch und $g(t) = f(t) - \phi(t)$ mit dem Monte-Carlo-Verfahren.

Code 5.9.2: Control Variates Monte-Carlo

```

1  """
   Computes integral
3  I0(1) = (1/pi) int(z=0..pi) exp(-cos(z)) dz
   by control variate. The splitting is
5  < exp(-cos) > = < exp(-cos) - phi > + int(0..pi) phi dz
   where phi = 1 + cos - (1/2)*cos*cos is the control.
7  The exact integral of the control
   int(z=0..pi) (1 + cos - (1/2)*cos*cos) dz = 1.25.
9  Abramowitz and Stegun give I0(1) = 1.266066
   """
11
12  import numpy as np
13  # reproductibility: fix, seed, numpy version, avoid parallel randomness
   seed_value = 1903
14  rng = np.random.default_rng(seed_value)
15
16  M = 100 # number of times we run our MC inetgration

```

```

19  asval = 1.266065878 #exact value of the integral
    ex = np.zeros(M)
    print('A and S tables:  IO(1) = ',asval)
21  print('sample      variance      MC IO val')
    print('-----      -----      -----')
23  k = 5 # how many experiments
    N = 10**np.arange(1,k+1)
25  v = []; e = []
    for n in N:
27      for m in range(M):
          x = np.pi*rng.random(n) # sample
          ctv = np.exp(-np.cos(x)) - 1. + np.cos(x) - 0.5*np.cos(x)*np.cos(x)
29          ex[m] = 1.25 + sum(ctv)/n # quadrature, 1.25 is the exact value of the
integral of the known part
31      ev = sum(ex)/M
          vex = np.dot(ex,ex)/(M-1.) #dot (x,x) is a clever way for calculating  $\sum x_j^2$ 
33      vex -= (M/(M-1.))*ev**2 #usual routine for estimating the variance
          v += [vex]; e += [ev]
35      print(n, vex, ev)

```

Importance Sampling

Beim “Importance Sampling”-Verfahren wird das Integral als

$$I = \int_{[0,1]^d} z(x) dx = \int_{[0,1]^d} \frac{z(x)}{g(x)} g(x) dx$$

geschrieben, wobei $g(x)$ eine geeignete Dichtefunktion ist. Es werden nach $g(x)$ verteilte Zufallszahlen verwendet, um eine niedrigere Varianz zu erhalten.

Diese Technik zur Reduktion der Varianz beruht auf der Verallgemeinerung der Bemerkung 5.8.16 im folgenden Satz.

Theorem 5.9.3. Seien $X_1, \dots, X_N$ Zufallsvariablen mit der Dichte $f(x)$. Dann ist

$$m_N(z(X)) := \frac{1}{N} \sum_{i=1}^N \frac{z(X_i)}{f(X_i)}$$

eine Zufallsvariable mit

$$\mathbb{E} m_N(z(X)) = \int_{[0,1]^d} z(x) dx .$$

In der Tat,

$$\mathbb{E} m_N(z(X)) = \frac{1}{N} \sum_{i=1}^N \mathbb{E} \frac{z(X_i)}{f(X_i)} = \frac{1}{N} N \mathbb{E} \frac{z(X)}{f(X)} = \int_{[0,1]^d} \frac{z(x)}{f(x)} f(x) dx = \int_{[0,1]^d} z(x) dx .$$

Nun stellt sich die Frage: Welche Dichte g sollen wir wählen? Wir sollten mehr Punkte dort investieren, wo die Funktion grössere Werte annimmt:

$$\theta = \int_0^1 f(x) dx = \int_0^1 \frac{f(x)}{g(x)} g(x) dx = \int_0^1 \frac{f(x)}{g(x)} dG(x),$$

wobei g und G erfüllen

$$G(x) = \int_0^x g(y) dy, \quad G(1) = \int_0^1 g(y) dy = 1,$$

und $G(x)$ eine Verteilungsfunktion ist. Die Varianz ist nun

$$\sigma_{f/g}^2 = \int_0^1 (f(x)/g(x) - \theta)^2 dG(x).$$

Normalerweise nimmt man g so, dass sie analytisch integrierbar ist und $g/f \approx$ konstant ist. Wie wählen wir eine gute Verteilungsfunktion G aus?

Dazu können wir uns zuerst ein Beispiel anschauen.

Beispiel 5.9.4. $\int_0^1 f(x)dx$ mit $f(x) = 1/\sqrt{x(1-x)}$ ist sogar singulär in $x = 0, 1$. Der allgemeine Trick ist hier, die Singularitäten zu isolieren:

$$g(x) = \frac{1}{4\sqrt{x}} + \frac{1}{4\sqrt{1-x}} = \frac{\sqrt{x} + \sqrt{1-x}}{4\sqrt{x(1-x)}} \implies \int_0^1 h(x)dG(x)$$

mit

$$h(x) = \frac{f(x)}{g(x)} = \frac{4}{\sqrt{x} + \sqrt{1-x}}$$

und aus $dG(x)$ wird mit folgendem Trick gezogen (das übliche allgemeine Verfahren werden wir danach sehen; der Trick wird hier nicht bewiesen):

```
u = rand(N)
v = rand(N)
x = u*u
w = where(v>0.5)
x[w] = 1 - x[w]
```

Code 5.9.5: Importance Sampling Monte-Carlo

```
1 from numpy import where, sqrt, arange, array
   import numpy as np
3 from time import time

5 # reproductibility: fix, seed, numpy version, avoid parallel randomness
   seed_value = 1903
7 rng = np.random.default_rng(seed_value)

9 from scipy.integrate import quad
   f = lambda x: 1/sqrt(x*(1-x))
11 print('quad(f,0,1) = ', quad(f,0,1))
```

```

13 func = lambda x: 4./(sqrt(x)+sqrt(1-x))

15 def exotic(N, rng):    #the "trick" given in the script
    u = rng.random(N)
17    v = rng.random(N)
    x = u*u
19    w = where(v>0.5)
    x[w] = 1 - x[w]
21    return x

23 def ismcquad(rng):
    k = 5 # how many experiments
25    N = 10**arange(1,k+1) # 10,100,...10k
    ex = [] #container for results
27    for n in N:
        x = exotic(n, rng) # sample
29        x = func(x) #we use func and not f anymore: importance sampling
integration
        ex += [x.sum()/n] # usual monte carlo quadrature
31    return ex

33 def mcquad(rng):
    k = 5 # how many experiments
35    N = 10**arange(1,k+1) # 10,100,...10k
    ex = [] #container for results
37    for n in N:
        x = rng.random(n) # sample
39        x = f(x)
        ex += [x.sum()/n] # usual monte carlo quadrature
41    return ex

43 M = 100 # number of times we run our MC inetgration

45 t1 = time()
results = []
47 for m in range(M):
    results += [ismcquad(rng)]
49
51 t2 = time()
t = t2-t1 #the time needed for running the simulation

53 #first we adopt importance sampling monte-carlo method
print("ISMC Serial calculation completed, time = %s s" % t)
55
57 ex = array(results)
ev = ex.sum(axis=0)/M
vex = (ex**2).sum(axis=0)/(M-1.)
59 vex = vex - (M/(M-1.))*ev**2 #usual estimateors for expected for expected value
and variance
print(ev[-1]) #value of the integral found with iportance sampling
61 print(vex) #variance

63 #now we uses standard monte-carlo method
t1 = time()
65 results = []

```

```

67     for m in range(M):
        results += [mcquad(rng)]
69
71     t2 = time()
    t = t2-t1
73
75     print("MC Serial calculation completed, time = %s s" % t)
77
79     ex = array(results)
    ev = ex.sum(axis=0)/M
    vex = (ex**2).sum(axis=0)/(M-1.)
    vex = vex - (M/(M-1.))*ev**2
    print(ev[-1])
    print(vex)

```

Beispiel 5.9.6. Wir wollen die zwei Techniken für die Reduktion der Varianz an $\int_0^1 e^x dx$ zeigen. Nun gilt

$$e^x - (1+x) = x^2/2 + \dots$$

und $\int_0^1 (1+x)dx = 3/2$, so dass Control Variates einfach zu implementieren ist. Für Importance Sampling gehen wir folgendermassen vor:

1. Wir definieren $g(x) = \frac{2}{3}(1+x)$.
2. Unser Integral hat nun die Form:

$$I = \int_0^1 \frac{e^x}{\frac{2}{3}(1+x)} \frac{2}{3}(1+x) dx$$

3. Wir suchen $y = y(x)$, so dass $y'(x) = \frac{2}{3}(1+x)$. Es ergibt sich:

$$y(x) = \frac{2}{3} \left(x + \frac{x^2}{2} \right)$$

4. Dann können wir schreiben

$$x(y) = -1 + \sqrt{1+3y}$$

5. Schlussendlich wird unser Integral zu:

$$I = \int_{y(0)=0}^{y(1)=1} \frac{e^{x(y)}}{\frac{2}{3}(1+x(y))} dy$$

6. Wir wenden das Monte-Carlo-Verfahren auf dieses Integral an.

Diese Substitution folgt einem Satz der Wahrscheinlichkeitstheorie, der besagt, dass, wenn x uniform verteilt ist, dann $y = G^{-1}(x)$ die Wahrscheinlichkeitsverteilung G besitzt.

Die beiden Techniken werden im folgenden Code implementiert.

Code 5.9.7: Monte-Carlo mit Reduktion der Varianz

```

1  import numpy as np
3  import matplotlib.pyplot as plt
   import math
5
   def mc1d_experiments(func,a,b,N,M,p=None,prnd=None):
7
9       """
10      Purpose: perform M experiments of Monte-Carlo method
11
12      Input: func ... function to integrate
13             a    ... lower interval bound
14             b    ... upper interval bound
15             N    ... number of samples
16             M    ... number of experiments
17             p    ... probability density p(x), standard None ==> Uniform dist.
18             prnd ... random number generator with probability density p(x),
19                    standard None ==> Uniform dist.
20
21      Output: Imean ... mean of Monte-Carlo approx. to integral over
22              experiments
23              Ivar2 ... (empirical) variance of Monte-Carlo approx.
24                     to integral over experiments
25
26      Notes: None
27
28      """
29
30      # perform experiments
31      I = np.zeros(M)
32      for iexp in np.arange(M):
33          [I[iexp],ivar] = mc1dquad(func,a,b,N,p=p,prnd=prnd)
34
35      # compute mean of experiments
36      Imean = sum(I)/M
37
38      # compute square (empirical) variance of experiments
39      Ivar = sum(I**2)/(M-1.) - (M/(M-1.))*Imean**2
40
41      # return
42      return Imean,Ivar
43
44  def mc1dquad(func,a,b,N,p=None,prnd=None):
45
46      """
47      Purpose: compute definite integral of func enclosed between a & b
48              with the Monte-Carlo method
49
50      Input: func ... function to integrate
51             a    ... lower interval bound
52             b    ... upper interval bound
53             N    ... number of samples

```

```

53     p    ... probability density p(x), standard None ==> Uniform dist.
54     prnd ... random number generator with probability density p(x),
55           standard None ==> Uniform dist.

57     Output: I    ... Monte-Carlo approximation to integral
58             var2 ... (sample) variance/sqrt(N)

59
60     Notes: None
61
62     ""
63
64     # generate N samples in [a,b]
65     if ( p == None ): # uniform distribution
66         x = rng.random(N)
67     else: # probability density p(x)
68         x = prnd(N) #we obtain x(y) as above
69     x[:] = a + (b - a)*x[:]

71     # volume of domain [a,b]
72     vol = b - a

73
74     # evaluate function at sample points
75     fx = func(x)
76     if ( p != None ):
77         px = p(x)

79     # evaluate integral (i.e. compute the expectation value)
80     if ( p == None ):
81         I = np.sum(fx)/N
82     else:
83         I = np.sum(fx/px)/N #division by px: corresponds to point 5) in the
      "kochrezept" given above

85     # compute (sample) variance
86     if ( N > 1 ):
87         var2 = (np.sum(fx**2)/N - I**2)/(N - 1) #usual formula for the variance
88     else:
89         var2 = 0.

91     # return
92     return vol*I,var2*vol**2

93
94     if __name__ == "__main__":
95         # reproductibility: fix, seed, numpy version, avoid parallel randomness
96         seed_value = 1903
97         rng = np.random.default_rng(seed_value)
98         # exp(x)
99         def f(x):
100             fx = np.exp(x)
101             return fx

102         # exp(x) - phi(x)
103         def fmphi(x):
104             fmphix = np.exp(x) - (1. + x + x**2/2. + x**3/3.)

```



```

    return fmphi

107
# probability density p(x) = 2/3*(1 + x)
109 #the denominator in the integral. see its use in the function mclquad
def p(x):
111     px = 2.*(1 + x)/3.
    return px
113
# random numbers with probability density p(x) = 2/3*(1 + x)
115 def prnd(N):
    prnd = -1. + np.sqrt(1. + 3.*rng.random(N)) #in fact, we wrote: x = 1 +
sqrt(1+3y)
117     return prnd

119 # parameters
a = 0. # lower bound
121 b = 1. # upper bound
Iref = np.exp(1.) - 1. # reference
123 Iphi = 1. + 1./2. + 1./6. + 1./12. # in. phi
M = 100 # number of experiments
125
N = np.arange(10,10**4,10**2) # number of samples (list!)
127
# convergence study
129 Imean = [] # initialize list for experiments means of the integral
Imean_cv = [] # initialize list for experiments means of the integral
131 Imean_is = [] # initialize list for experiments means of the integral
Ivar = [] # initialize list for experiments variance of the integral
133 Ivar_cv = [] # initialize list for experiments variance of the integral
Ivar_is = [] # initialize list for experiments variance of the integral
135 error = [] # initialize list for error
error_cv = [] # initialize list for error
137 error_is = [] # initialize list for error
for Nk in N:
    # standard
    [Imean_k,Ivar_k] = mcl_d_experiments(f,a,b,Nk,M)
141     Imean.append(Imean_k)
    Ivar.append(Ivar_k)
143     error.append(np.abs(Imean_k - Iref))
    # controlled variates
145     [Imean_k,Ivar_k] = mcl_d_experiments(fmphi,a,b,Nk,M)
    Imean_k = Imean_k + Iphi
147     Imean_cv.append(Imean_k)
    Ivar_cv.append(Ivar_k)
149     error_cv.append(np.abs(Imean_k - Iref))
    # controlled variates
151     [Imean_k,Ivar_k] = mcl_d_experiments(f,a,b,Nk,M,p=p,prnd=prnd)
    Imean_is.append(Imean_k)
153     Ivar_is.append(Ivar_k)
    error_is.append(np.abs(Imean_k - Iref))
155
def plot_1():
157     plt.figure(1)
    plt.clf()

```

```

159     plt.loglog(N,error    , 'b-+',label='Single'    )
160     plt.loglog(N,error_cv, 'r-+',label='Single CV')
161     plt.loglog(N,error_is, 'g-+',label='Single IS')
162     plt.axis([N[0],N[-1],1e-8,1e2])
163     plt.title('Monte-Carlo integration')
164     plt.xlabel('$N$')
165     plt.ylabel('$Error$')
166     plt.legend()
167
168     def plot_2():
169         plt.figure(1)
170         plt.loglog(N,error    , 'b--+',label='Average over M experiments'    )
171         plt.loglog(N,error_cv, 'r--+',label='Average over M experiments CV')
172         plt.loglog(N,error_is, 'g--+',label='Average over M experiments IS')
173         plt.legend()
174
175     def plot_3():
176         plt.figure(1)
177         plt.loglog([1.e0,1.e6],[1.e-5,1.e-8], 'k--',label=r'$\propto 1/\sqrt{N}$')
178         plt.grid()
179         plt.legend()
180
181     def plot_4():
182         plt.figure(2)
183         plt.clf()
184         plt.semilogx([N[0],N[-1]],[Iref,Iref], 'k:' ,label=r"$I$ reference")
185         plt.semilogx(N,Imean    , 'b-+',label=r"$\bar{I}_N$")
186         plt.semilogx(N,Imean - np.sqrt(Ivar) , 'b^--')
187         plt.semilogx(N,Imean + np.sqrt(Ivar) , 'bv--')
188         plt.semilogx(N,Imean_cv    , 'r-+',label=r"$\bar{I}_N$ CV")
189         plt.semilogx(N,Imean_cv - np.sqrt(Ivar_cv), 'r^--')
190         plt.semilogx(N,Imean_cv + np.sqrt(Ivar_cv), 'rv--')
191         plt.semilogx(N,Imean_is    , 'g-+',label=r"$\bar{I}_N$ IS")
192         plt.semilogx(N,Imean_is - np.sqrt(Ivar_is), 'g^--')
193         plt.semilogx(N,Imean_is + np.sqrt(Ivar_is), 'gv--')
194         plt.title('Monte-Carlo integration')
195         plt.xlabel('$N$')
196         plt.ylabel('$I_N$')
197         plt.grid()
198         plt.legend()

```

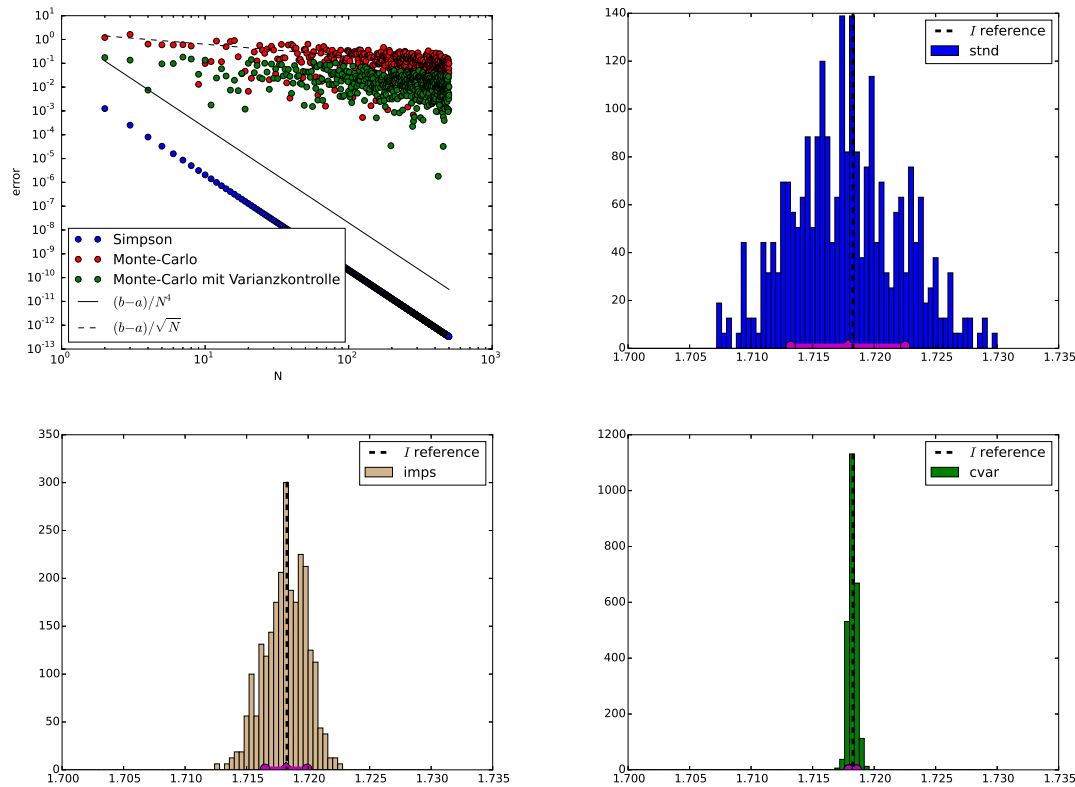


Abb. 5.9.8. Typische Ergebnisse bei der Quadratur mittels verschiedener Methoden; die Histogramme sind für $M = 400$ und $N = 10^4$ Realisierungen im Beispiel 5.9.6.

Bemerkung 5.9.9. Der Fehler bei jeder Monte-Carlo-Methode nimmt wie $N^{-1/2}$ ab, denn der Zufall (das Rauschen) bestimmt die Konvergenz. Die Quasi-Monte-Carlo-Methoden ersetzen die Zufallszahlen durch quasizufällige Sequenzen, die den Integrationsbereich systematisch abdecken (Sobol- und Halton-Sequenzen); das liefert einen deterministischen Fehler, der wie $N^{-1} \cdot (\log(N))^d$ abnimmt. Für moderate Dimensionen d ist das etwa wie N^{-1} , jedoch bei sehr grossem d , z.B. $d = 600$, ist diese Verbesserung nicht mehr sichtbar. Trotzdem sind Sobol-Sequenzen in der Praxis oft deutlich besser als reines MC, weil viele Integranden “effektiv niedrigdimensional” sind.

Beispiel 5.9.10. Die hier besprochenen Fälle sind natürlich zu einfach. Die Situation kann für eine allgemeine Funktion in d Dimensionen deutlich komplizierter sein. Zum Beispiel ist die Funktion (5.6.21) ein recht schwieriger Fall für grosse Dimensionen d , denn sie weist d konzentrierte Schwingungen auf, so dass sie ein gutes Beispiel für den sogenannten “Fluch der Dimensionen” ist: Die Information ist auf eine sehr komplizierte geometrische Art konzentriert, aber der Definitionsbereich hat ein sehr grosses Volumen. Voll besetzte und dünne Gitter sind für grosse Dimensionen d nicht machbar; die Standard-Monte-Carlo-Methode konvergiert sehr langsam und braucht eine enorme Anzahl Punkte: Die meisten landen in Bereichen, in denen die Funktion wenig Information trägt. Wegen der komplizierten Form der Funktion ist

Importance Sampling nicht machbar. Die Schwingungen der Funktion eliminieren auch jeden Ansatz mittels einer Taylor-Approximation für Control Variates. Der Schlüssel ist in diesem speziellen Fall ein Fourier-Ansatz! Wir konstruieren $\varphi(t)$, eine Fourier-Interpolation der Funktion

$$h(t) = \frac{1}{\sqrt{1 - a \sin(2\pi t - 1)}} \approx \varphi(t) = \sum_{-N/2}^{N/2-1} c_k e^{2\pi i k t}$$

auf $[0, 1]$. Hierzu können wir mit vielen Koeffizienten schnell alle Schwingungen perfekt darstellen. So konstruieren wir eine sehr gute Approximation an f :

$$\phi(\mathbf{x}) = \varphi(\|\mathbf{x}\|_2^2) = \sum_{-N/2}^{N/2-1} c_k e^{2\pi i k \|\mathbf{x}\|_2^2}.$$

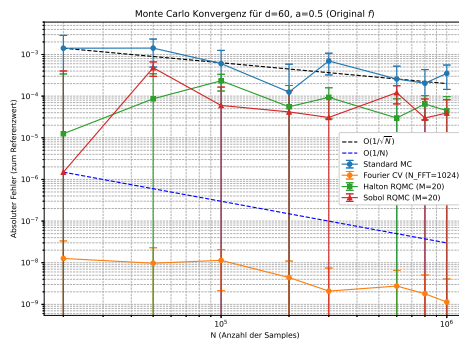
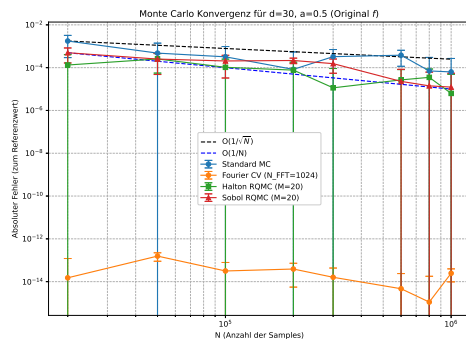
Für Control Variates brauchen wir eine exakte oder zumindest sehr genaue Berechnung von:

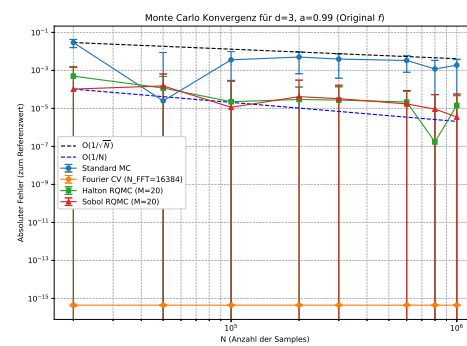
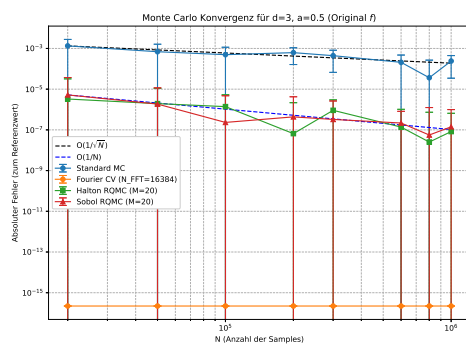
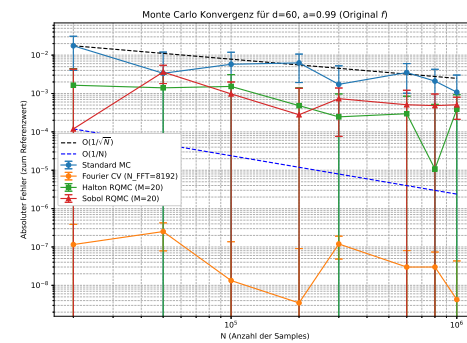
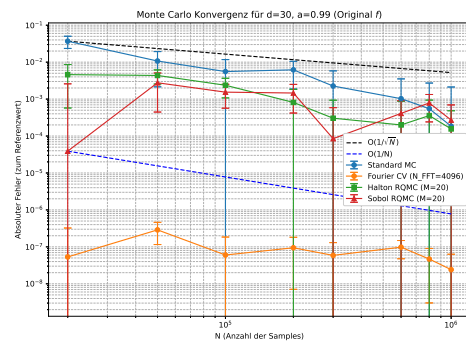
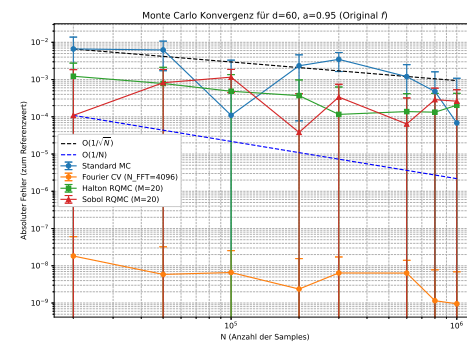
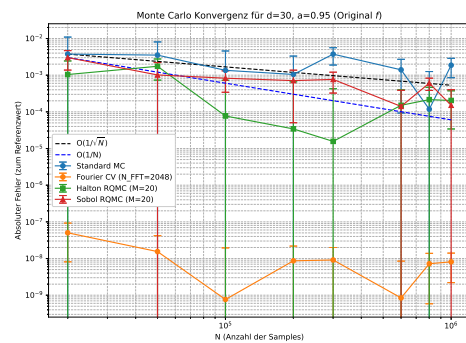
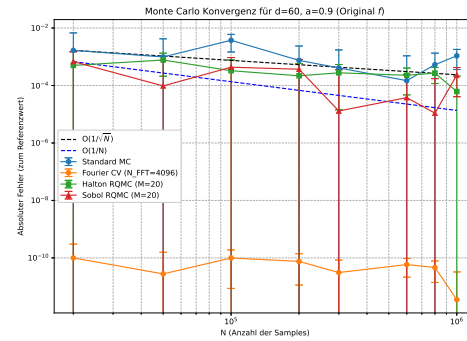
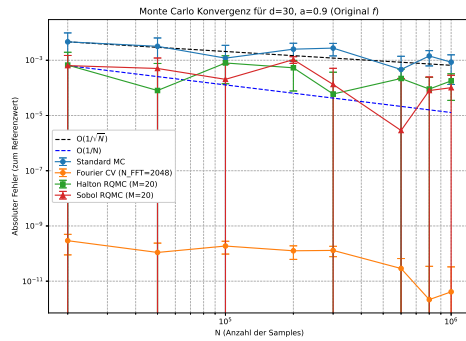
$$\int_{[0,1]^d} \phi(\mathbf{x}) d\mathbf{x} = \sum_{-N/2}^{N/2-1} c_k \int_{[0,1]^d} e^{2\pi i k \|\mathbf{x}\|_2^2} d\mathbf{x}.$$

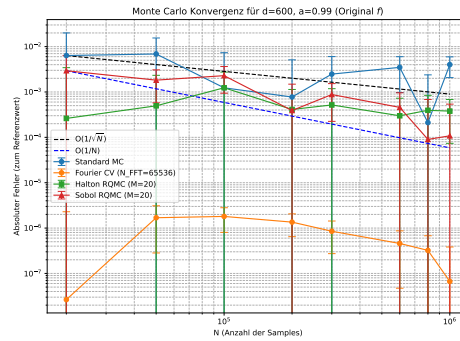
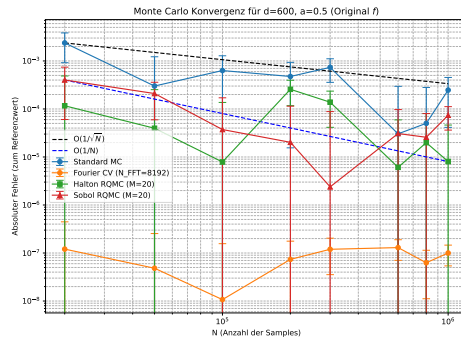
Aber diese komplexe Exponentialfunktion lässt sich separieren:

$$\int_{[0,1]^d} e^{2\pi i k \|\mathbf{x}\|_2^2} d\mathbf{x} = \int_{[0,1]^d} e^{2\pi i k (x_1^2 + \dots + x_d^2)} d\mathbf{x} = \int_{[0,1]^d} \prod_{j=1}^d e^{2\pi i k x_j^2} d\mathbf{x} = \left(\int_0^1 e^{2\pi i k x^2} dx \right)^d.$$

Der Real- und der Imaginärteil des letzten eindimensionalen Integrals sind sehr bekannte Größen, die Fresnel-Integrale; dafür gibt es sehr genaue Approximationen, z.B. in `scipy.special.fresnel`, so dass $\int_{[0,1]^d} \phi(\mathbf{x}) d\mathbf{x}$ fast exakt bekannt ist. Damit erwarten wir, dass Control Variates mit diesem ϕ hervorragend funktioniert. Für $d \geq 200$ sind die Terme $\left(\int_0^1 e^{2\pi i k x^2} dx \right)^d$ für $k \neq 0$ sogar vernachlässigbar, da die Beträge der Fresnel-Integrale kleiner als 1 sind; somit ist der c_0 -Term der wichtigste überhaupt.







Mehr über die Monte-Carlo-Methode findet man in den folgenden Büchern:

W.L. DUNN, J.K. SHULTIS, *Exploring Monte Carlo Methods*, Amsterdam, Elsevier, 2012, [ETHbib](#)

T. MÜLLER-GRONBACH, E. NOVAK, K. RITTER, *Monte Carlo-Algorithmen*, Berlin, Springer, 2012, [ETHbib](#)

Chapter 6

Nullstellensuche

Dieses Kapitel behandelt die Suche nach Lösungen nichtlinearer Gleichungssysteme mittels iterativer Methoden. Das Problem kann man wie folgt ausdrücken:

Gegeben sei die Funktion

$$\mathbf{F}: \Omega \subset \mathbb{R}^d \rightarrow \mathbb{R}^d, \quad d \in \mathbb{N}. \quad (6.0.1)$$

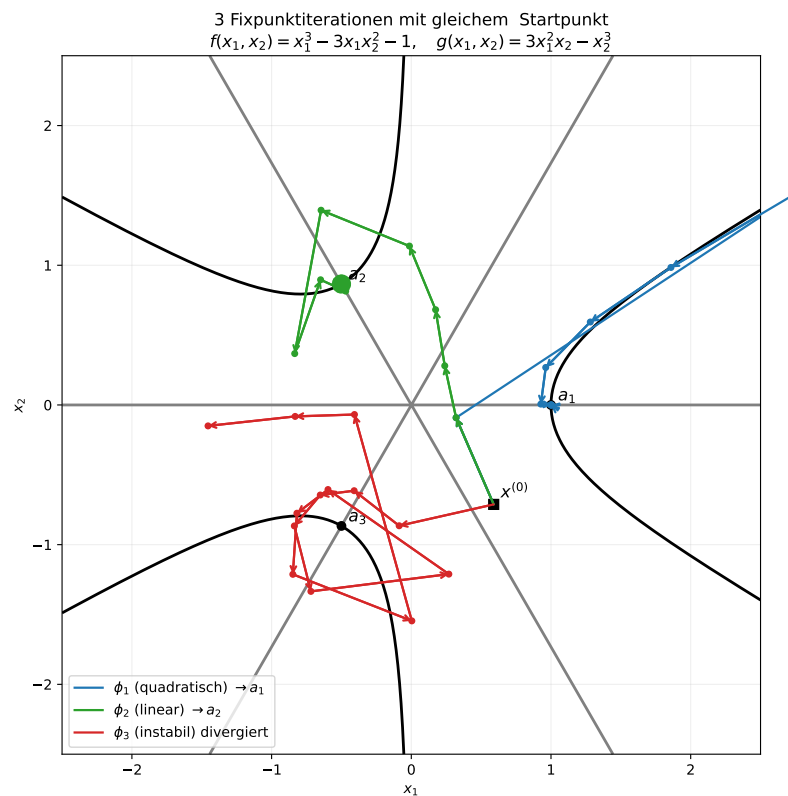
Wir suchen $\mathbf{x}^* \in \Omega$, so dass $\mathbf{F}(\mathbf{x}^*) = 0$.

Solche algebraischen nichtlinearen Probleme kommen in den Anwendungen sehr oft vor. Zum Beispiel ist eine notwendige Bedingung dafür, dass $\mathbf{x}^*$ eine Extremalstelle einer Zielfunktion $f: \Omega \subset \mathbb{R}^d \rightarrow \mathbb{R}$ ist, $\mathbf{D}f(\mathbf{x}^*) = 0$. Im maschinellen Lernen ist d sehr, sehr gross (Millionen). Bei der numerischen Lösung gewöhnlicher Differentialgleichungen mit impliziten Methoden muss man ein solches algebraisches nichtlineares Problem in jedem Zeitschritt approximativ lösen: Hier sind Geschwindigkeit und passende Genauigkeit der Näherung essenziell.

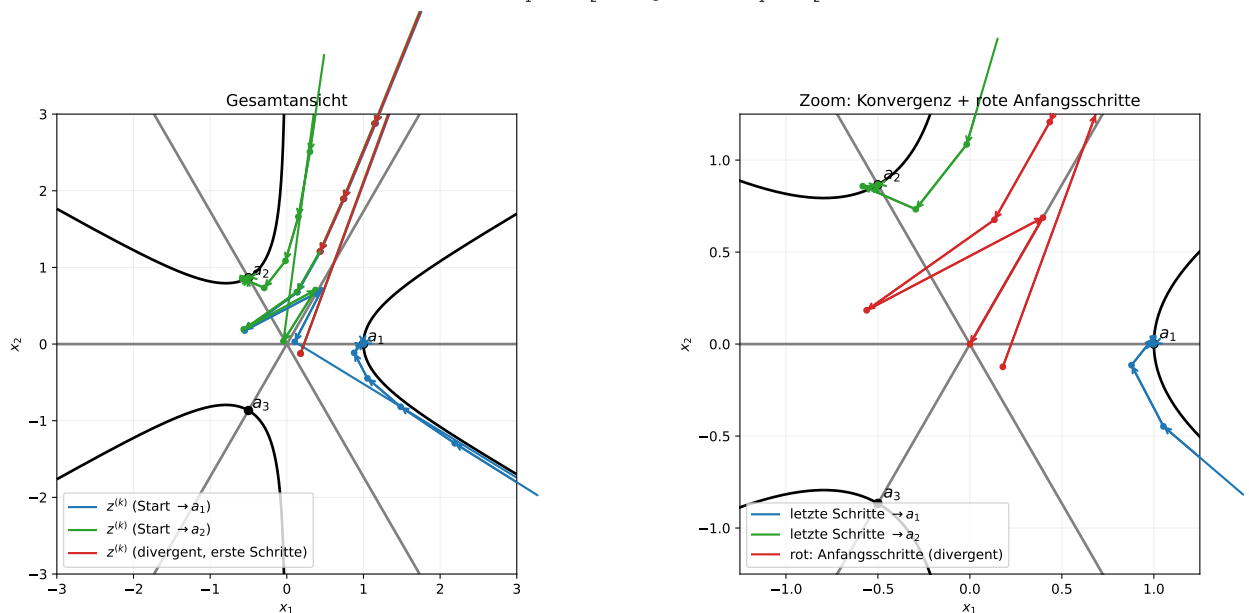
Im Allgemeinen wissen wir nicht, ob eine Lösung existiert und, falls dies der Fall ist, ob sie eindeutig ist. Dieses so einfach formulierte Problem kann recht schwierig zu lösen sein, vor allem für grosse Dimension d , wie Anwendungen in der Quantenmechanik oder im Machine Learning zeigen. Trotzdem besprechen wir allgemeine Verfahren, die aber im Einzelfall noch anzupassen sind, um leistungsfähige Algorithmen zu erhalten. In `scipy.optimize` gibt es dazu die Funktion `root` mit vielen verschiedenen konkreten Lösern für Probleme in einer oder in mehreren Dimensionen (`fsolve` sollte nicht mehr verwendet werden), die manchmal Erfolg haben, manchmal aber auch kläglich scheitern, trotz enormen Rechenaufwands. Verständnis der konkreten Anwendung und der Lösungstechniken ist essenziell für den Erfolg; es gibt keine Methode, die immer funktioniert. Bevor wir aber zur eigentlichen Nullstellensuche kommen, wollen wir zuerst allgemeine iterative Verfahren betrachten und insbesondere die Fixpunktiteration.

6.1 Iterative Verfahren

Definition 6.1.1. Ein iteratives Verfahren ist ein Algorithmus $\phi_{\mathbf{F}}$, der die Folge $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$ approximativer Lösungen $\mathbf{x}^{(j)}$ generiert.



Verschiedene Iterationen mit demselben Startwert
 Niveaulinien $f(x_1, x_2) = 0$, $g(x_1, x_2) = 0$ und Fixpunktiterationen
 $f(x_1, x_2) = x_1^3 - 3x_1x_2^2 - 1$, $g(x_1, x_2) = 3x_1^2x_2 - x_2^3$



Dieselbe Iteration mit sehr nahe gelegenen, aber verschiedenen Startwerten

Abb. 6.1.2. Iteration und Startwert sind **beide** entscheidend

Beispiel 6.1.3. Bei gegebenem Anfangswert $\mathbf{x}^{(0)}$ und gegebener Funktion ϕ ist ein iteratives Verfahren definiert durch $\mathbf{x}^{(k)} := \phi(\mathbf{x}^{(k-1)})$.

Definition 6.1.4. Ein iteratives Verfahren $\phi_{\mathbf{F}}$ zur Lösung von $\mathbf{F}(\mathbf{x}^*) = 0$ konvergiert, wenn $\mathbf{x}^{(k)} \rightarrow \mathbf{x}^* \in \Omega$ und $\mathbf{F}(\mathbf{x}^*) = 0$.

Bemerkung 6.1.5. Normalerweise benötigt man einen Startwert, der nahe beim gesuchten Wert $\mathbf{x}^*$ liegen soll. Wie nahe der Anfangswert an der Lösung liegen muss, hängt von dem Gleichungssystem und dem gewählten Algorithmus ab. In Abbildung 6.1.2 oben sehen wir drei verschiedene Iterationen für ein nichtlineares Problem mit Dimension $d = 2$: Bei demselben Anfangswert konvergiert eine zur Lösung $\mathbf{a}_1$, eine zur Lösung $\mathbf{a}_2$, und eine konvergiert gar nicht. Im unteren Teil der Abbildung sehen wir eine Iteration für dasselbe Problem; diesmal wählen wir drei verschiedene Startwerte, die sehr, sehr nahe beieinander liegen: Je nach Startwert erhalten wir eine zur Lösung $\mathbf{a}_1$ konvergente Folge, eine zur Lösung $\mathbf{a}_2$ konvergente Folge und eine nicht konvergente Folge. Wenn die Anfangswerte aber sehr nahe bei den exakten Lösungen $\mathbf{a}_1$, $\mathbf{a}_2$ oder $\mathbf{a}_3$ liegen, dann liefert die Iteration Folgen, die zur jeweiligen exakten Lösung konvergieren.

Bemerkung 6.1.6. Bei iterativen Verfahren stehen zwei Fragen im Vordergrund: Wie schnell ist die Konvergenz, und wann sollten wir die Iteration beenden? Die Antwort auf die erste Frage erleichtert uns die Wahl eines bestimmten Verfahrens. Eine gute Antwort auf die zweite Frage sichert die Qualität der numerischen Lösung.

Definition 6.1.7. (Norm) Sei X ein linearer Raum über $\mathbb{K}$ (mit $\mathbb{K} = \mathbb{C}$ oder $\mathbb{R}$). Eine Abbildung $\|\cdot\| : X \mapsto \mathbb{R}_0^+$ heisst *Norm* auf X , falls sie erfüllt:

- (i) $\forall \mathbf{x} \in X: \mathbf{x} \neq 0$ dann und nur dann, wenn $\|\mathbf{x}\| > 0$,
- (ii) $\|\lambda \mathbf{x}\| = |\lambda| \|\mathbf{x}\|$ für alle $\mathbf{x} \in X$ und alle $\lambda \in \mathbb{K}$,
- (iii) $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ für alle $\mathbf{x}, \mathbf{y} \in X$ (Dreiecksungleichung).

Beispiel 6.1.8. Wir betrachten Vektoren $\mathbf{x} = (x_1, x_2, \dots, x_d)^T \in \mathbb{K}^d$.

Name	:	Definition	<code>numpy.linalg</code>
Euklidische Norm	:	$\ \mathbf{x}\ _2 := \sqrt{ x_1 ^2 + \dots + x_d ^2}$	<code>norm(x)</code>
1-Norm	:	$\ \mathbf{x}\ _1 := x_1 + \dots + x_d $	<code>norm(x, 1)</code>
∞ -Norm, Max-Norm	:	$\ \mathbf{x}\ _\infty := \max\{ x_1 , \dots, x_d \}$	<code>norm(x, inf)</code>

Definition 6.1.9. Zwei Normen $\|\cdot\|_1$ und $\|\cdot\|_2$ auf einem linearen Raum V sind äquivalent, wenn es Konstanten $\underline{C}$ und $\overline{C}$ gibt, so dass

$$\underline{C} \|v\|_1 \leq \|v\|_2 \leq \overline{C} \|v\|_1$$

für alle $v \in V$.

Theorem 6.1.10. Wenn $\dim V < \infty$, dann sind alle Normen auf V äquivalent.

Beispiel 6.1.11. Einfache explizite Normäquivalenzen: Für alle $\mathbf{x} \in \mathbb{K}^d$ gilt:

$$\|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1 \leq \sqrt{d} \|\mathbf{x}\|_2, \quad (6.1.1)$$

$$\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \sqrt{d} \|\mathbf{x}\|_\infty, \quad (6.1.2)$$

$$\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_1 \leq d \|\mathbf{x}\|_\infty. \quad (6.1.3)$$

Definition 6.1.12. Die Folge $\mathbf{x}^{(k)}$ aus dem linearen Raum mit Norm $\|\cdot\|$ heisst *linear konvergent* gegen $\mathbf{x}^*$, falls:

$$\text{es ein } L < 1 \text{ gibt, so dass } \|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \leq L \|\mathbf{x}^{(k)} - \mathbf{x}^*\| \text{ für alle } k \geq k_0. \quad (6.1.4)$$

Die Konstante L nennt man Konvergenzrate.

Bemerkung 6.1.13. Da alle Normen in einem endlichdimensionalen Raum äquivalent sind, ist die Konvergenz (in $\mathbb{R}^d$) eines Verfahrens *unabhängig* von der Wahl der Norm. Die Konvergenzrate oder sogar die Tatsache, dass die Konvergenz linear ist, sind jedoch von der Normwahl abhängig.

Definition 6.1.14. Eine Folge hat *Konvergenzordnung* p , falls:

$$\text{es ein } C > 0 \text{ gibt, so dass } \|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \leq C \|\mathbf{x}^{(k)} - \mathbf{x}^*\|^p \text{ für alle } k \in \mathbb{N} \quad (6.1.5)$$

mit $C < 1$ für $p = 1$.

Hier wird stillschweigend angenommen, dass der Startwert nahe bei der Lösung $\mathbf{x}^*$ liegt, z.B. $\|\mathbf{x}^0 - \mathbf{x}^*\| < 1$, damit wir eine konvergente Folge erhalten.

Bemerkung 6.1.15. Obwohl wir dasselbe Wort verwenden, ist der Begriff Konvergenzordnung hier sehr unterschiedlich verglichen mit den Kapiteln über Interpolation, Quadratur und Differentialgleichungen. Die Bedeutung des Begriffs (Konvergenz-)Ordnung ist kontextabhängig!

Bemerkung 6.1.16. Im Kontext von Nullstellenproblemen ist man sehr glücklich, wenn quadratische Konvergenz einer Iteration möglich ist. Höhere Konvergenzordnungen sind selten; meistens muss man sich mit superlinearer, linearer oder einfach mit Konvergenz zufriedengeben, vor allem für grosse Dimension d , wo die Kosten für Methoden mit höherer Konvergenzordnung erheblich bis untragbar sein können.

Bemerkung 6.1.17. In einem lin-log Plot des Fehlers $\|\mathbf{e}^{(k)}\|$ für die Iterationszahl k sehen wir im Fall linearer Konvergenz eine Gerade, denn

$$\begin{aligned} \|\mathbf{e}^{(k)}\| &\leq L^k \|\mathbf{e}^{(0)}\|, \\ \log(\|\mathbf{e}^{(k)}\|) &\leq k \log(L) + \log(\|\mathbf{e}^{(0)}\|), \end{aligned}$$

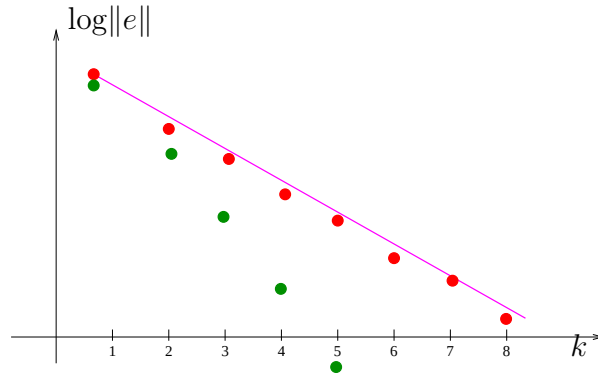


Abb. 6.1.18. Beispiel linearer Konvergenz; jede schnellere Konvergenz ist auch mindestens linear!

Konvergenz höherer Ordnung ist an gekrümmten Konvergenzkurven erkennbar:

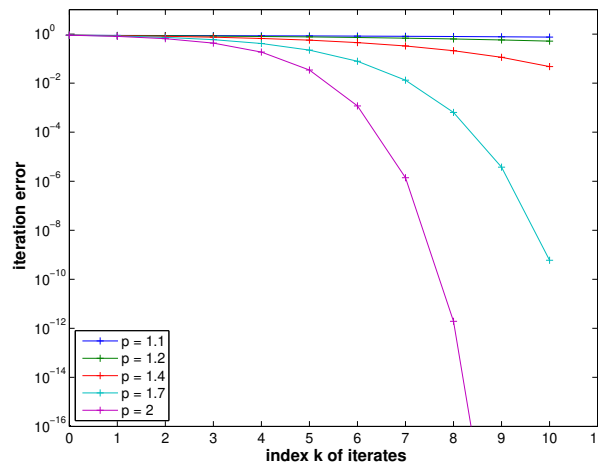


Abb. 6.1.19. Iterationen mit verschiedenen Konvergenzordnungen p .

Bemerkung 6.1.20. Die Konvergenzrate bei linearer Konvergenz lässt sich leicht abschätzen. Sei $\epsilon_k := \|\mathbf{x}^{(k)} - \mathbf{x}^*\|$ die Norm des Fehlers im k -ten Schritt. Bei linearer Konvergenz (Definition 6.1.12) nehmen wir an (hier $0 < L < 1$):

$$\epsilon_{k+1} \approx L\epsilon_k \implies \log \epsilon_{k+1} \approx \log L + \log \epsilon_k \implies \log \epsilon_{k+1} \approx k \log L + \log \epsilon_0. \quad (6.1.6)$$

Somit beschreibt $\log L < 0$ die Steigung der Geraden im lin-log Plot des Fehlers.

Beispiel 6.1.21. Wir beobachten das Verhalten folgender Iteration:

$$x^{(k+1)} = x^{(k)} + \frac{\cos x^{(k)} + 1}{\sin x^{(k)}}.$$

Code 6.1.22: einfache Fixpunktiteration

```

1 from numpy import sin, cos, array
3 #iteration with linear convergence
5 def lincvg(x):
    y = [] #container for the  $x^{(j)}$ 
    for k in range(15):
        x = x + (cos(x)+1)/sin(x) #apply the iteration formula
        y += [x] #store the value in the container
    err = abs(array(y) - x) #estimation for the error
    rate = err[1:]/err[:-1] #estimation for convergence rate
    return err, rate

```

Achtung:

Wir verwenden $x^{(15)}$ statt der exakten Lösung x^* in der Berechnung der Konvergenzrate; im Allgemeinen ist die exakte Lösung ja unbekannt. Wenn aber die Iteration nach 15 Schritten nicht konvergiert hat, ist eine solche Schätzung des Fehlers sinnlos.

k	$x^{(0)} = 0.4$		$x^{(0)} = 0.6$		$x^{(0)} = 1$	
	$x^{(k)}$	$\frac{ x^{(k)} - x^{(15)} }{ x^{(k-1)} - x^{(15)} }$	$x^{(k)}$	$\frac{ x^{(k)} - x^{(15)} }{ x^{(k-1)} - x^{(15)} }$	$x^{(k)}$	$\frac{ x^{(k)} - x^{(15)} }{ x^{(k-1)} - x^{(15)} }$
2	3.3887	0.1128	3.4727	0.4791	2.9873	0.4959
3	3.2645	0.4974	3.3056	0.4953	3.0646	0.4989
4	3.2030	0.4992	3.2234	0.4988	3.1031	0.4996
5	3.1723	0.4996	3.1825	0.4995	3.1224	0.4997
6	3.1569	0.4995	3.1620	0.4994	3.1320	0.4995
7	3.1493	0.4990	3.1518	0.4990	3.1368	0.4990
8	3.1454	0.4980	3.1467	0.4980	3.1392	0.4980

Konvergenzrate $L \approx 0.5$

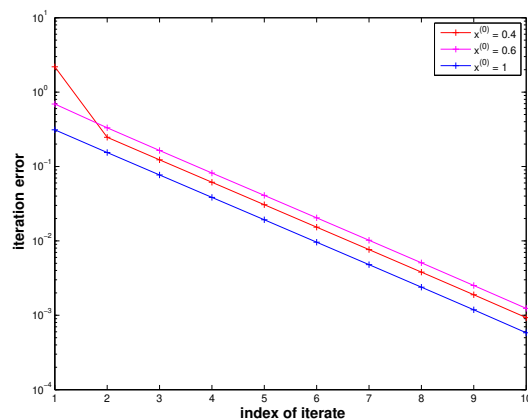


Abb. 6.1.23. Lineare Konvergenz

Beispiel 6.1.24. (Quadratische Konvergenz) Sei die Fixpunktiteration zur Berechnung von $\sqrt{a}$:

$$x^{(k+1)} = \frac{1}{2}\left(x^{(k)} + \frac{a}{x^{(k)}}\right) \implies |x^{(k+1)} - \sqrt{a}| = \frac{1}{2x^{(k)}}|x^{(k)} - \sqrt{a}|^2. \quad (6.1.7)$$

Da $\sqrt{vw} \leq \frac{1}{2}(v+w)$, gilt: $x^{(k+1)} = \frac{1}{2}(v+w) \geq \sqrt{vw} = \sqrt{a}$ für $k \geq 1$ (mit $v = x^{(k)}$ und $w = \frac{a}{x^{(k)}}$). Somit ist die Folge durch $\sqrt{a}$ nach unten beschränkt.

Es gilt ausserdem $x^{(k+1)} < x^{(k)}$ für alle $k \geq 2$. Damit ist die Folge $(x^{(k)})_{k \in \mathbb{N}_0}$ monoton und beschränkt, also konvergent.

Nun wollen wir aus einem numerischen Experiment die Konvergenzordnung dieser Iteration erraten. Wir notieren $\epsilon_k := \|\mathbf{x}^{(k)} - \mathbf{x}^*\|$. Dann:

$$\epsilon_{k+1} \approx C \epsilon_k^p \implies \log \epsilon_{k+1} \approx \log C + p \log \epsilon_k \implies \frac{\log \epsilon_{k+1} - \log \epsilon_k}{\log \epsilon_k - \log \epsilon_{k-1}} \approx p.$$

Das numerische Experiment für $a = 2$ liefert:

k	$x^{(k)}$	$e^{(k)} := x^{(k)} - \sqrt{2}$	$\log \frac{ e^{(k)} }{ e^{(k-1)} } : \log \frac{ e^{(k-1)} }{ e^{(k-2)} }$
0	2.0000000000000000	0.58578643762690485	
1	1.5000000000000000	0.08578643762690485	
2	1.4166666666666665	0.00245310429357137	1.850
3	1.41421568627450966	0.00000212390141452	1.984
4	1.41421356237468987	0.00000000000159472	2.000
5	1.41421356237309492	0.00000000000000022	

Wir beobachten, dass sich die Anzahl der exakten Dezimalstellen im Ergebnis in jedem Schritt verdoppelt. Das gilt für jede konvergente Iteration mit Ordnung 2. Wenn wir den relativen Fehler im Schritt k mit δ_k notieren, haben wir:

$$\begin{aligned} x^{(k)} &= x^*(1 + \delta_k) \implies x^{(k)} - x^* = \delta_k x^*, \\ \implies |x^* \delta_{k+1}| &= |x^{(k+1)} - x^*| \leq C |x^{(k)} - x^*|^2 = C |x^* \delta_k|^2 \\ &\implies |\delta_{k+1}| \leq C |x^*| \delta_k^2. \end{aligned} \quad (6.1.8)$$

Wenn $C \approx 1$, dann $\delta_k = 10^{-\ell}$ und (6.1.8) zeigt $\delta_{k+1} \approx 10^{-2\ell}$.

Code 6.1.25: Beispiel einer stationären Iteration

```

from numpy import sqrt, array

def sqrttit(a,x): #iteration for computing sqrt(a)
    exact = sqrt(a) #exact value found with numpy
    e = [x] #array with the values of the iteration
    x_old = -1.
    while x_old != x:

```

```

8      x_old = x
9      x = 0.5*(x+a/x)
10     e += [x] #store new iteration value
11     e = array(e)
12     e = abs(e-exact) #compute the absolute errors
13     return e
14
15 e = sqrtit(2.,1.)
16 print(e)

```

6.2 Abbruchkriterien

Bei iterativen Verfahren stellt sich die Frage, bei welchem k man die Iteration abbrechen soll oder kann. Hierbei ist es entscheidend, ob die gesetzten Ziele, wie zum Beispiel Genauigkeit, erreicht wurden oder vielleicht keine Verbesserung des Ergebnisses mehr eintreten kann. Einige mögliche Kriterien sind:

- (a) *A priori* Kriterium: Nach einer festen Anzahl k_0 an Schritten wird das Verfahren beendet. Die Qualität des Ergebnisses ist unsicher.
- (b) *A posteriori* Kriterium: Verwende die berechneten Werte. Wenn der Fehler unter einer gewissen Toleranz ($\|\mathbf{x}^{(k)} - \mathbf{x}^*\| < \tau$) liegt, bricht man ab. Man kennt aber den gesuchten Wert $\mathbf{x}^*$ nicht; daher ist diese Strategie nicht anwendbar.
- (c) Führe die Iteration bis $\mathbf{x}^{(k+1)} \approx \mathbf{x}^{(k)}$ durch. Diese Methode ist aber auch sehr ineffizient: Viele Schritte wurden umsonst gemacht, und es ist auch unklar, warum die Iteration keine Fortschritte macht.
- (d) Verwende das Residuum, also Abbruch, wenn $\|\mathbf{F}(\mathbf{x}^{(k)})\| < \tau$. Dies birgt jedoch Probleme, denn $\|\mathbf{F}(\mathbf{x}^{(k)})\|$ kann klein sein, obwohl $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|$ gross ist (siehe Bild).

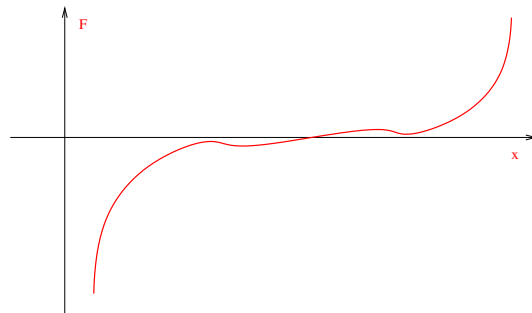


Abb. 6.2.1. Im Falle einer Funktion, die wie im Graphen aussieht, kann das Residuum als Abbruchkriterium problematisch sein.

Folgende Bemerkung zeigt, wie man ein A-posteriori-Abbruchkriterium im Falle einer linear konvergenten Iteration bauen kann.

Bemerkung 6.2.2. Bei linearer Konvergenz und bekanntem L kann man folgende Abschätzung verwenden:

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \leq \frac{L}{1-L} \|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|. \quad (6.2.1)$$

Proof.

$$\begin{aligned} \|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| &\leq L\|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq L\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| + L\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \\ \Rightarrow (1-L)\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| &\leq L\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| \\ \Rightarrow \|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| &\leq \frac{L}{1-L} \|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|. \end{aligned} \quad (6.2.2)$$

□

Somit berechnen wir die Abschätzung

$$\text{errest}[k] = \frac{L}{1-L} \|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|,$$

und wir brechen die Iteration ab, sobald $\text{errest}[k] < \tau$.

Beispiel 6.2.3. Im Beispiel 6.1.21 haben wir eine lineare Konvergenz gegen π mit einer Konvergenzrate 0.5 beobachtet. Wir überprüfen die Qualität unseres Fehlerschätzers für dieses Beispiel. Hier sind die Ergebnisse (absoluter Fehler, Wert der Fehlerschätzung und Fehler in der Abschätzung des Fehlers) für den Startwert $x^0 = 0.4$:

k	$ x^{(k)} - \pi $	$\frac{L}{1-L} x^{(k)} - x^{(k-1)} $	Fehler in der Abschätzung
1	2.191562221997101	4.933154875586894	2.741592653589793
2	0.247139097781070	1.944423124216031	1.697284026434961
3	0.122936737876834	0.124202359904236	0.001265622027401
4	0.061390835206217	0.061545902670618	0.000155067464401
5	0.030685773472263	0.030705061733954	0.000019288261691
6	0.015341682696235	0.015344090776028	0.000002408079792
7	0.007670690889185	0.007670991807050	0.000000300917864
8	0.003835326638666	0.003835364250520	0.000000037611854
9	0.001917660968637	0.001917665670029	0.000000004701392
10	0.000958830190489	0.000958830778147	0.000000000587658
11	0.000479415058549	0.000479415131941	0.000000000073392
12	0.000239707524646	0.000239707533903	0.000000000009257
13	0.000119853761949	0.000119853762696	0.000000000000747
14	0.000059926881308	0.000059926880641	0.000000000000667
15	0.000029963440745	0.000029963440563	0.000000000000181

Wir sehen, dass der A-posteriori-Fehlerschätzer in diesem Fall sehr genau ist.

6.3 Fixpunktiteration

Wir schreiben die Nullstellensuche $\mathbf{F}(\mathbf{x}) = 0$ in ein Fixpunktproblem $\phi(\mathbf{x}) = \mathbf{x}$ um. Die Funktion ϕ liefert nun die Iterationsvorschrift! Ein 1-Punkt-Verfahren benötigt dabei nur den vorhergegangenen Schritt: $\mathbf{x}^{(k+1)} = \phi(\mathbf{x}^{(k)})$.

Definition 6.3.1. Eine Fixpunktiteration heisst konsistent mit $\mathbf{F}(\mathbf{x}) = 0$, falls

$$\mathbf{F}(\mathbf{x}) = 0 \Leftrightarrow \phi(\mathbf{x}) = \mathbf{x}. \quad (6.3.1)$$

Beispiel 6.3.2. Sei $F(x) = xe^x - 1$ mit $x \in [0, 1]$. Dann betrachten wir die folgenden iterativen Verfahren:

- $\phi_1(x) = e^{-x}$ liefert lineare Konvergenz;
- $\phi_2(x) = \frac{1+x}{1+e^x}$ gibt quadratische Konvergenz;
- $\phi_3(x) = x + 1 - xe^x$ erzeugt eine divergente Folge.

Hier sind die Ergebnisse dieser drei Iterationen:

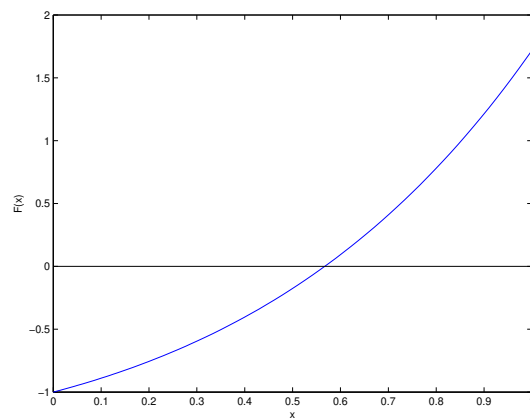


Abb. 6.3.3. Funktion F

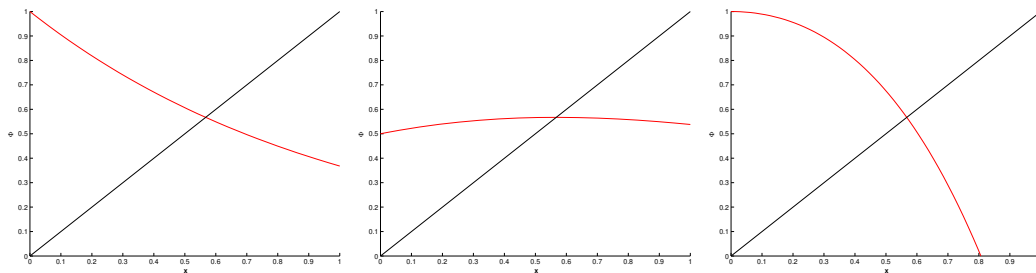


Abb. 6.3.4. Die Funktionen ϕ_1 , ϕ_2 und ϕ_3 .

k	$x^{(k+1)} := \phi_1(x^{(k)})$	$x^{(k+1)} := \phi_2(x^{(k)})$	$x^{(k+1)} := \phi_3(x^{(k)})$
0	0.5000000000000000	0.5000000000000000	0.5000000000000000
1	0.606530659712633	0.566311003197218	0.675639364649936
2	0.545239211892605	0.567143165034862	0.347812678511202
3	0.579703094878068	0.567143290409781	0.855321409174107
4	0.560064627938902	0.567143290409784	-0.156505955383169
5	0.571172148977215	0.567143290409784	0.977326422747719
6	0.564862946980323	0.567143290409784	-0.619764251895580
7	0.568438047570066	0.567143290409784	0.713713087416146
8	0.566409452746921	0.567143290409784	0.256626649129847
9	0.567559634262242	0.567143290409784	0.924920676910549
10	0.566907212935471	0.567143290409784	-0.407422405542253

k	$ x_1^{(k+1)} - x^* $	$ x_2^{(k+1)} - x^* $	$ x_3^{(k+1)} - x^* $
0	0.067143290409784	0.067143290409784	0.067143290409784
1	0.039387369302849	0.000832287212566	0.108496074240152
2	0.021904078517179	0.00000125374922	0.219330611898582
3	0.012559804468284	0.000000000000003	0.288178118764323
4	0.007078662470882	0.000000000000000	0.723649245792953
5	0.004028858567431	0.000000000000000	0.410183132337935
6	0.002280343429460	0.000000000000000	1.186907542305364
7	0.001294757160282	0.000000000000000	0.146569797006362
8	0.000733837662863	0.000000000000000	0.310516641279937
9	0.000416343852458	0.000000000000000	0.357777386500765
10	0.000236077474313	0.000000000000000	0.974565695952037

Wir beobachten lineare Konvergenz für $x_1^{(k)}$, quadratische Konvergenz für $x_2^{(k)}$ und keine Konvergenz für $x_3^{(k)}$; für alle drei Iterationen hat $x_i^{(0)} = 0.5$ als Startwert gedient.

Wir sehen, dass man ein Nullstellenproblem in verschiedene Fixpunktiterationen umschreiben kann, die sich aber sehr unterschiedlich verhalten können. Wann und wie schnell konvergiert eine Fixpunktiteration?

Definition 6.3.5. Eine Funktion ϕ heisst *Kontraktion*, falls

$$\text{es ein } L < 1 \text{ gibt, so dass } \|\phi(\mathbf{x}) - \phi(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\| \text{ für alle } \mathbf{x}, \mathbf{y}. \quad (6.3.2)$$

Bemerkung 6.3.6. Wenn $\mathbf{x}^*$ ein Fixpunkt der Kontraktion ϕ ist, dann ist

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| = \|\phi(\mathbf{x}^{(k)}) - \phi(\mathbf{x}^*)\| \leq L\|\mathbf{x}^{(k)} - \mathbf{x}^*\|,$$

Das heisst, dass iterative Verfahren $\mathbf{x}^{k+1} = \phi(\mathbf{x}^k)$ mindestens linear konvergieren.

Satz 6.3.7. (Banach'scher Fixpunktsatz)

Sei $\Omega \subset \mathbb{K}^d$ ($\mathbb{K} = \mathbb{R}, \mathbb{C}$), mit Ω abgeschlossen, und $\phi: \Omega \rightarrow \Omega$ eine Kontraktion. Dann existiert ein eindeutiger Fixpunkt $\mathbf{x}^*$, also $\phi(\mathbf{x}^*) = \mathbf{x}^*$. Dieser ist der Grenzwert der Folge $\mathbf{x}^{(k+1)} = \phi(\mathbf{x}^{(k)})$.

Proof. Betrachte die Einpunktiteration $\mathbf{x}^{(k)} = \phi(\mathbf{x}^{(k-1)})$ mit dem Startwert $\mathbf{x}^{(0)} \in \Omega$. Die Dreiecksungleichung und die Kontraktionseigenschaft von ϕ ergeben:

$$\begin{aligned} \|\mathbf{x}^{(k+N)} - \mathbf{x}^{(k)}\| &\leq \sum_{j=k}^{k+N-1} \|\mathbf{x}^{(j+1)} - \mathbf{x}^{(j)}\| \leq \sum_{j=k}^{k+N-1} L^j \|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\| \\ &\leq \frac{L^k}{1-L} \|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\| \xrightarrow{k \rightarrow \infty} 0. \end{aligned}$$

Somit ist $(\mathbf{x}^{(k)})_{k \in \mathbb{N}_0}$ eine Cauchy-Folge in $\mathbb{K}^d$ und demnach konvergent. Wir bezeichnen mit $\mathbf{x}^*$ den Grenzwert: $\mathbf{x}^{(k)} \xrightarrow{k \rightarrow \infty} \mathbf{x}^*$. Die Stetigkeit von ϕ ergibt dann $\phi(\mathbf{x}^*) = \mathbf{x}^*$. $\square$

Wir wollen nun eine wichtige Eigenschaft der Fixpunktiteration $\mathbf{x}^{(k+1)} = \phi(\mathbf{x}^{(k)})$ verstehen.

Hilfssatz 6.3.8. (Hinreichende Bedingung für lokale lineare Konvergenz einer Fixpunktiteration) Es sei U konvex und $\phi: U \subset \mathbb{R}^d \mapsto \mathbb{R}^d$ stetig differenzierbar mit $L := \sup_{\mathbf{x} \in U} \|D\phi(\mathbf{x})\| < 1$ (wobei $D\phi(\mathbf{x})$ die Jacobi-Matrix von $\phi(\mathbf{x})$ bezeichnet). Wenn $\phi(\mathbf{x}^*) = \mathbf{x}^*$ für $\mathbf{x}^* \in U$, dann konvergiert die Fixpunktiteration $\mathbf{x}^{(k+1)} = \phi(\mathbf{x}^{(k)})$ gegen $\mathbf{x}^*$ lokal mindestens linear.

Beweis. Die Definition der Ableitung ergibt

$$\|\phi(\mathbf{y}) - \phi(\mathbf{x}^*) - D\phi(\mathbf{x}^*)(\mathbf{y} - \mathbf{x}^*)\| \leq \psi(\|\mathbf{y} - \mathbf{x}^*\|) \|\mathbf{y} - \mathbf{x}^*\|,$$

mit $\psi: \mathbb{R}_0^+ \mapsto \mathbb{R}_0^+$, so dass $\lim_{t \rightarrow 0} \psi(t) = 0$.

Wir wählen ein $\delta > 0$, so dass

$$\psi(t) + \|D\phi(\mathbf{x}^*)\| \leq \frac{1}{2}(1 + \|D\phi(\mathbf{x}^*)\|) \leq L < 1 \quad \forall 0 \leq t < \delta.$$

Die Dreiecksungleichung, zusammen mit $\mathbf{x}^{(k+1)} = \phi(\mathbf{x}^{(k)})$, ergibt dann für die Fixpunktiteration:

$$\begin{aligned} \|\phi(\mathbf{x}) - \mathbf{x}^*\| - \|D\phi(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*)\| &\leq \psi(\|\mathbf{x} - \mathbf{x}^*\|) \|\mathbf{x} - \mathbf{x}^*\| \\ \|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| &\leq (\psi(t) + \|D\phi(\mathbf{x}^*)\|) \|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq L \|\mathbf{x}^{(k)} - \mathbf{x}^*\|, \end{aligned}$$

für $\|\mathbf{x}^{(k)} - \mathbf{x}^*\| < \delta$. $\square$

Beispiel 6.3.9. Wir können das letzte Ergebnis mittels einer eindimensionalen Funktion veranschaulichen: Wenn ϕ um den Fixpunkt $\mathbf{x}^*$ ziemlich flach ist, dann haben wir lokale Konvergenz; wenn ϕ dort aber steil ist, dann haben wir Divergenz.

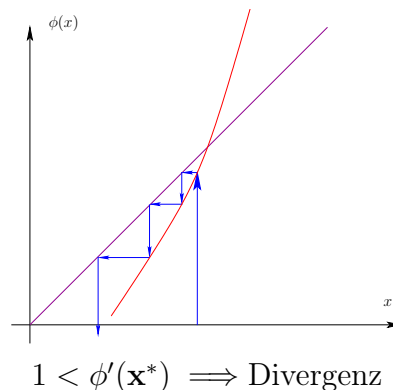
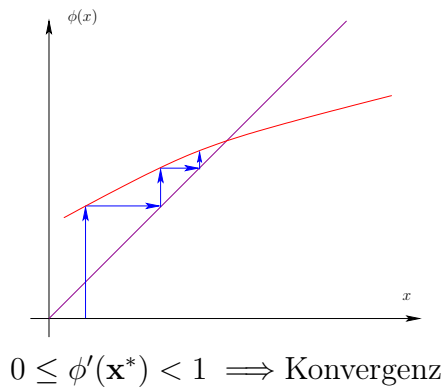
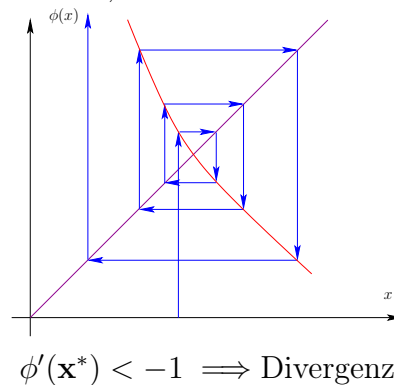
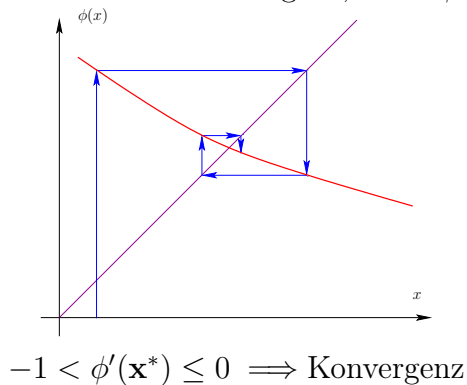


Abb. 6.3.10. Verhalten einiger Iterationen

Hilfssatz 6.3.11. Sei $\phi: \mathbb{R}^n \rightarrow \mathbb{R}^n$ mit $\phi(\mathbf{x}^*) = \mathbf{x}^*$ und ϕ stetig differenzierbar in $\mathbf{x}^*$. Ist $\|D\phi(\mathbf{x}^*)\| < 1$, dann konvergiert die Fixpunktiteration $\mathbf{x}^{(k+1)} = \phi(\mathbf{x}^{(k)})$ lokal und (mindestens) linear, mit $L = \|D\phi(\mathbf{x}^*)\|$.

Beweis. Es gilt:

$$\begin{aligned} \phi(\mathbf{x}) - \phi(\mathbf{y}) &= \int_0^1 D\phi(\mathbf{x} + \tau(\mathbf{y} - \mathbf{x})) (\mathbf{y} - \mathbf{x}) d\tau \implies \\ \|\phi(\mathbf{x}) - \phi(\mathbf{y})\| &\leq L \|\mathbf{y} - \mathbf{x}\|. \end{aligned}$$

Da $\|D\phi(\mathbf{x}^*)\| < 1$ und $D\phi$ stetig ist, gilt $\|D\phi(\mathbf{x}^*)\| < 1$ in einer ganzen Umgebung von $\mathbf{x}^*$. Somit ist $L = \max_{\tau \in [0,1]} \|D\phi(\mathbf{x} + \tau(\mathbf{y} - \mathbf{x}))\|$ strikt kleiner als 1. Damit ist ϕ eine Kontraktion in einer Umgebung von $\mathbf{x}^*$ und hat den Fixpunkt $\mathbf{x}^*$.

Beispiel 6.3.12. (Fixpunktiteration in mehrdimensionalen Räumen)

Wir schreiben das folgende Gleichungssystem in ein Fixpunktproblem um:

$$\begin{cases} x_1 - c(\cos x_1 - \sin x_2) = 0 \\ (x_1 - x_2) - c \sin x_2 = 0 \end{cases} \implies \begin{cases} c(\cos x_1 - \sin x_2) = x_1 \\ c(\cos x_1 - 2 \sin x_2) = x_2. \end{cases}$$

Wir definieren:

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = c \begin{bmatrix} \cos x_1 - \sin x_2 \\ \cos x_1 - 2 \sin x_2 \end{bmatrix} \implies D\phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = -c \begin{bmatrix} \sin x_1 & \cos x_2 \\ \sin x_1 & 2 \cos x_2 \end{bmatrix}.$$

Wir wählen eine *geeignete* Norm: $\|\cdot\| = \infty$ -Norm $\|\cdot\|_\infty$:

$$\text{wenn } c < \frac{1}{3} \implies \|D\phi(\mathbf{x})\|_\infty < 1 \quad \forall \mathbf{x} \in \mathbb{R}^2,$$

und somit ergibt diese Fixpunktiteration mindestens lineare Konvergenz. Die Existenz eines Fixpunktes ist auch gesichert, da ϕ in der abgeschlossenen Menge $[-3, 3]^2$ agiert und der Banach'sche Fixpunktsatz angewendet werden kann.

Satz 6.3.13. (Formel von Taylor)

Sei $U \subseteq \mathbb{R}$ ein Intervall, $\phi: U \rightarrow \mathbb{R}$ $(m+1)$ -mal differenzierbar und $x \in U$. Dann gilt für jedes $y \in U$:

$$\phi(y) - \phi(x) = \sum_{k=1}^m \frac{1}{k!} \phi^{(k)}(x)(y-x)^k + O(|y-x|^{m+1}). \quad (6.3.3)$$

Hilfssatz 6.3.14. Sei $U \subseteq \mathbb{R}$ ein Intervall und $\phi: U \rightarrow \mathbb{R}$ $(m+1)$ -mal differenzierbar mit $\phi(x^*) = x^* \in U$. Sei weiterhin $\phi^{(l)}(x^*) = 0$ für $l = 1, 2, \dots, m$ ($m \geq 1$). Dann konvergiert die Fixpunktiteration $x^{(k+1)} = \phi(x^{(k)})$ gegen x^* lokal mit der Ordnung $p \geq m+1$.

Proof. Setzt man $y = x^{(k)}$ und $x = x^*$, dann ergibt die Formel von Taylor:

$$x^{(k+1)} - x^* = \phi(x^{(k)}) - \phi(x^*) = \sum_{l=1}^m \frac{1}{l!} \underbrace{\phi^{(l)}(x^*)}_{=0} (x^{(k)} - x^*)^l + O(|x^{(k)} - x^*|^{m+1}). \quad (6.3.4)$$

□

Beispiel 6.3.15.

- $\phi_1(x) = e^{-x}$ ist linear konvergent, da $\phi'(x^*) = -e^{-x^*} = -x^* \in [-1, 0]$.
- $\phi_2(x) = \frac{F(x)}{(1+e^x)^2}$ ist quadratisch konvergent, da $\phi'_2(x) = 0$.

Lemma 6.3.16. Konvergiert die Kontraktion ϕ linear mit dem Faktor $L < 1$, dann gilt die folgende Abschätzung:

$$\|\mathbf{x}^* - \mathbf{x}^{(k)}\| \leq \frac{L^{k-l}}{1-L} \|\mathbf{x}^{(l+1)} - \mathbf{x}^{(l)}\|. \quad (6.3.5)$$

Proof.

$$\begin{aligned} \|\mathbf{x}^{k+m} - \mathbf{x}^{(k)}\| &\leq \sum_{j=k}^{k+m-1} \|\mathbf{x}^{(j+1)} - \mathbf{x}^{(j)}\| \\ &\leq \sum_{j=k}^{k+m-1} L^{j-k} \|\mathbf{x}^{k+1} - \mathbf{x}^{(k)}\| \\ &= \frac{1-L^m}{1-L} \|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| \\ &\leq \frac{1-L^m}{1-L} L^{k-l} \|\mathbf{x}^{(l+1)} - \mathbf{x}^{(l)}\|. \end{aligned} \quad (6.3.6)$$

Nimmt man jetzt auf beiden Seiten den Limes $m \rightarrow \infty$, so erhält man die gesuchte Abschätzung. □

Korollar 6.3.17. • $l = 0$ ergibt ein A-priori-Abbruchkriterium:

$$\|\mathbf{x}^* - \mathbf{x}^{(k)}\| \leq \frac{L^k}{1-L} \|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\| \leq \tau \Rightarrow k = \dots;$$

Wir können also das maximale k schätzen.

- $l = k - 1$ ergibt ein A-posteriori-Abbruchkriterium:

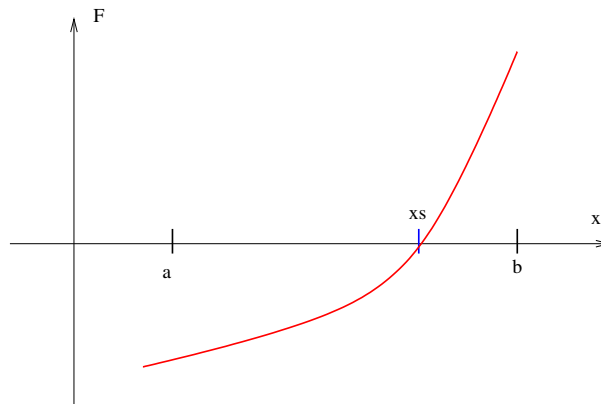
$$\|\mathbf{x}^* - \mathbf{x}^{(k)}\| \leq \frac{L}{1-L} \|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\| \leq \tau.$$

Diese Gleichung wird in jedem Schritt verifiziert. Wenn sie erfüllt ist, wird die Iteration abgebrochen.

6.4 Intervallhalbierungsverfahren (nur für $d = 1$)

Erster Ansatz: Mithilfe des Zwischenwertsatzes.

Ist F stetig auf einem Intervall $[a, b]$ und $F(a)F(b) < 0$ (d.h. F hat verschiedene Vorzeichen an den Endpunkten), so muss es ein $x^* \in [a, b]$ geben mit $F(x^*) = 0$. Man setzt zuerst $c = \frac{a+b}{2}$, betrachtet $F(a)F(c)$ und $F(b)F(c)$ und fährt dann auf dem Intervall $[a, c]$ oder $[c, b]$ fort, je nachdem, in welchem Intervall sich das Vorzeichen geändert hat.



Code 6.4.1: Bisektionsverfahren

```

def mybisect(f,a,b,tol=1e-12):
2
    if a>b:
4        t = a; a = b; b = t #swap a and b

    #evaluate f at two endpoints
6    fa = f(a); fb = f(b)

8    if fa*fb > 0: raise ValueError #fa and fb must have different signs

10    v = 1
12    if fa>0: v = -1
    x = 0.5*(a+b) #middle point between a and b
14    k = 1 #number of iterations
    while (b-a>tol) and (a<x) and (x<b):
16        if v*f(x)>0: b = x #if fa<0 v=1 then we just substitute b with x if
f(x)>0.
        #On the contrary, if fa>0 the function is decreasing (since then fb<0),
18        #so if also f(x) < 0 we substitute b with x
        else: a = x #if the above conditions are not fulfilled with choose a=x.
        #Hint: draw a graph in order to understand this idea.
        x = 0.5*(a+b)
22        k += 1 #update the number of iterations
    return x, k

24
if __name__=='__main__':
26

```

```

28     f = lambda x: x**3 - 2*x**2 + 4.*x/3. - 8./27.
    x, k = mybisection(f,0,1)
    print("Results of mybisection:")
30     print('x_bisection = ', x, ' after k=', k, ' iterations')

32     #fsolve is an old-style solver from scipy (use root; bisection is scipy's
implementation of bisection method
    #we want to compare our result with the one produced by these standard
routines
34     from scipy.optimize import fsolve, bisection
    print(20*" ")
36     print("Results of fsolve:")
    print(fsolve(f,0,full_output=True))
38     print(20*" ")
    print("Results of scipy's bisection")
40     print(bisection(f,0,1))

```

Beachte: Dieses Verfahren hat folgende Eigenschaften:

- Es ist ziemlich einfach, denn es braucht nur Auswertungen von F ;
- im Allgemeinen ist es ein sicheres Verfahren;
- die Konvergenz gegen x^* ist linear, also ziemlich langsam;
- es ist schwierig, es in d Dimensionen zu formulieren.

6.5 Newton-Verfahren in 1 Dimension

In jedem Iterationsschritt verwendet man eine lineare Funktion $\tilde{F}$, die mithilfe der Ableitung von F definiert wird, also $\tilde{F} = F(x^{(k)}) + F'(x^{(k)})(x - x^{(k)})$. Die Nullstelle von $\tilde{F}$ ist dann

$$x^{(k+1)} := x^{(k)} - \frac{F(x^{(k)})}{F'(x^{(k)})}, \quad \text{falls } F'(x^{(k)}) \neq 0. \quad (6.5.1)$$

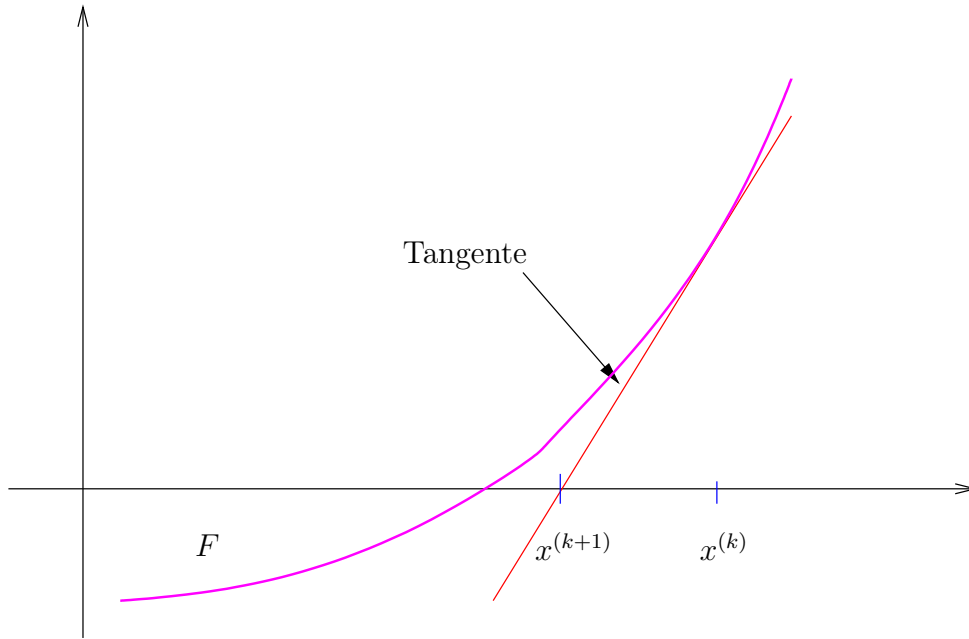


Abb. 6.5.1. Ein Schritt des Newton-Verfahrens

Bemerkung 6.5.2. Die Theorie der Fixpunktiterationen sichert quadratische lokale Konvergenz, falls $F'(x^*) \neq 0$. In der Tat ist eine Newton-Iteration eine Fixpunktiteration mit

$$\phi(x) = x - \frac{F(x)}{F'(x)} \implies \phi'(x) = \frac{F(x)F''(x)}{(F'(x))^2} \implies \phi'(x^*) = 0,$$

falls $F(x^*) = 0$ und $F'(x^*) \neq 0$.

Beispiel 6.5.3. (Newton-Verfahren in 1D)

Newton-Iterationen für zwei verschiedene skalare nichtlineare Gleichungen mit denselben Lösungen:

$$\begin{aligned} F(x) = xe^x - 1 &\implies F'(x) = e^x(1+x) \\ &\implies x^{(k+1)} = x^{(k)} - \frac{x^{(k)}e^{x^{(k)}} - 1}{e^{x^{(k)}}(1+x^{(k)})} = \frac{(x^{(k)})^2 + e^{-x^{(k)}}}{1+x^{(k)}} \end{aligned}$$

$$\begin{aligned} F(x) = x - e^{-x} &\implies F'(x) = 1 + e^{-x} \\ &\implies x^{(k+1)} = x^{(k)} - \frac{x^{(k)} - e^{-x^{(k)}}}{1 + e^{-x^{(k)}}} = \frac{1 + x^{(k)}}{1 + e^{x^{(k)}}}. \end{aligned}$$

Beispiel 6.5.4. (Angepasstes Newton-Verfahren)

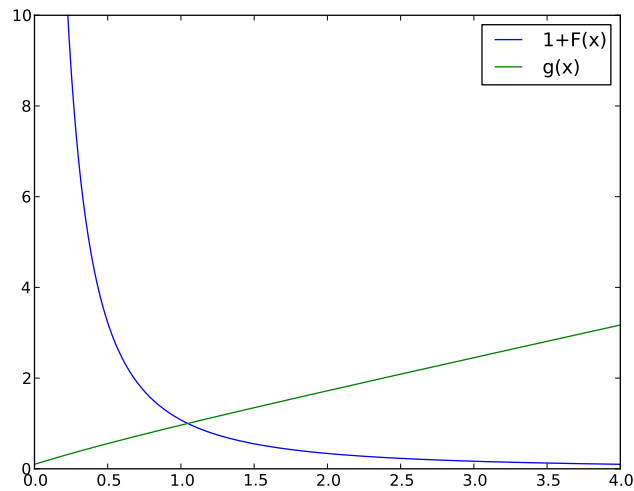
$$F(x) = \frac{1}{(x+1)^2} + \frac{1}{(x+0.1)^2} - 1, \quad x > 0.$$

Bemerkung:

$$F(x) + 1 \approx 2x^{-2} \text{ für } x \gg 1;$$

und somit:

$$g(x) := \frac{1}{\sqrt{F(x) + 1}} \text{ ist "fast linear" für } x \gg 1.$$



Idee:

Statt $F(x)=0$ direkt zu lösen, behandeln wir $g(x)=1$ mit dem Newton-Verfahren (6.5.1). Hier sind die Ergebnisse der Iteration für den Startwert $x^{(0)} = 0$:

k	$x^{(k)}$	$F(x^{(k)})$	$x^{(k)} - x^{(k-1)}$	$x^{(k)} - x^*$
1	0.91312431341979	0.24747993091128	0.91312431341979	-0.13307818147469
2	1.04517022155323	0.00161402574513	0.13204590813344	-0.00103227334125
3	1.04620244004116	0.00000008565847	0.00103221848793	-0.00000005485332
4	1.04620249489448	0.00000000000000	0.00000005485332	-0.00000000000000

6.6 Sekantenverfahren

Wenn die Ableitung $F'(x^{(k)})$ teuer zu berechnen ist oder gar nicht zur Verfügung steht, dann ersetzt man sie durch $q^{(k)} := \frac{F(x^{(k)}) - F(x^{(k-1)})}{x^{(k)} - x^{(k-1)}}$. Dann lautet ein Schritt des Verfahrens:

$$\tilde{F}(x) = F(x^{(k)}) + q^{(k)}(x - x^{(k)}) \Rightarrow x^{(k+1)} := x^{(k)} - \frac{F(x^{(k)})}{q^{(k)}}, \text{ falls } q^{(k)} \neq 0. \quad (6.6.1)$$

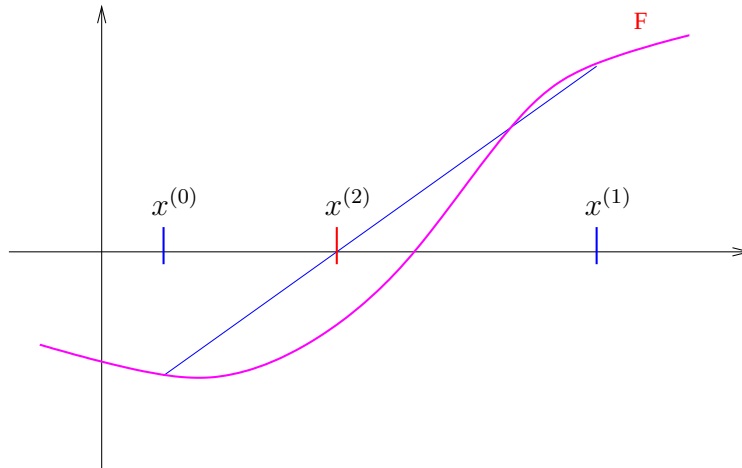


Abb. 6.6.1. Ein Schritt im Sekantenverfahren

Code 6.6.2: Sekantenverfahren

```

1  def secant(f,x0,x1,maxit=50,tol=1e-12):
3
4      fo = f(x0)
5      for k in range(maxit): #maxit: maximal number of iterations
6          fn = f(x1)
7          print('x1=',x1, 'f(x1)=', fn)
8          s = fn*(x1-x0)/(fn-fo) #estimate for derivative
9          x0 = x1; x1 -= s
10         if abs(s)<tol: #we are reasonably near to the solution and can return
11             the value we obtained
12                 x = x1
13                 return x, k #we also return the total number of iterations needed
14         fo = fn
15     x = NaN
16     return x, maxit

```

Beispiel 6.6.3. $F(x) = xe^x - 1$, $x^{(0)} = 0$, $x^{(1)} = 5$.

k	$x^{(k)}$	$F(x^{(k)})$	$e^{(k)} := x^{(k)} - x^*$	$\frac{\log e^{(k+1)} - \log e^{(k)} }{\log e^{(k)} - \log e^{(k-1)} }$
2	0.00673794699909	-0.99321649977589	-0.56040534341070	
3	0.01342122983571	-0.98639742654892	-0.55372206057408	24.43308649757745
4	0.98017620833821	1.61209684919288	0.41303291792843	2.70802321457994
5	0.38040476787948	-0.44351476841567	-0.18673852253030	1.48753625853887
6	0.50981028847430	-0.15117846201565	-0.05733300193548	1.51452723840131
7	0.57673091089295	0.02670169957932	0.00958762048317	1.70075240166256
8	0.56668541543431	-0.00126473620459	-0.00045787497547	1.59458505614449
9	0.56713970649585	-0.00000990312376	-0.00000358391394	1.62641838319117
10	0.56714329175406	0.00000000371452	0.00000000134427	
11	0.56714329040978	-0.000000000000001	-0.000000000000000	

Wir beobachten eine fraktionale Konvergenzordnung p ! Man kann $p = \frac{1}{2}(1 + \sqrt{5}) \approx 1.62$ beweisen.

Beispiel 6.6.4. (Lokale Konvergenz des Sekantenverfahrens)

$$F(x) = \arctan(x)$$

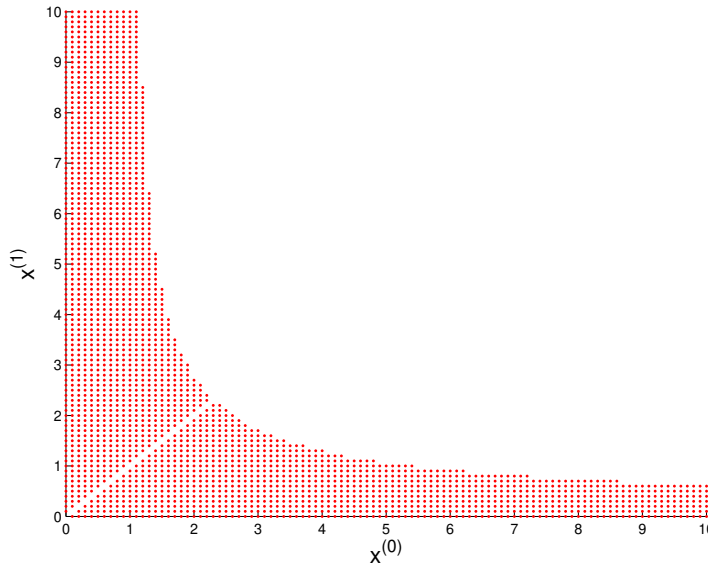


Abb. 6.6.5. $\hat{=}$ Sekantenverfahren konvergiert für ein Paar $(x^{(0)}, x^{(1)})$ als Startwert.

6.7 Newton-Verfahren in d Dimensionen

Sei $\mathbf{F}: \Omega \subseteq \mathbb{R}^d \rightarrow \mathbb{R}^d$ stetig differenzierbar. Gesucht ist $\mathbf{x}^* \in D$ mit $\mathbf{F}(\mathbf{x}^*) = 0$. Die Idee ist dieselbe wie in einer Dimension: Man ersetzt $\mathbf{F}$ durch ein lineares $\tilde{\mathbf{F}}$:

$$\tilde{\mathbf{F}}^{(k)}(\mathbf{x}) = \mathbf{F}(\mathbf{x}^{(k)}) + D\mathbf{F}(\mathbf{x}^{(k)})(\mathbf{x} - \mathbf{x}^{(k)}), \quad (6.7.1)$$

wobei $D\mathbf{F}(\mathbf{x}^{(k)})$ die Jacobi-Matrix von $\mathbf{F}$ ist: $D\mathbf{F}(x) = \left[\frac{\partial F_j}{\partial x_k}(x) \right]_{j,k=1,2,\dots,d}$.

Die Nullstelle von $\tilde{\mathbf{F}}^{(k)}$ ist dann

$$\mathbf{x}^{(k+1)} := \mathbf{x}^{(k)} - D\mathbf{F}(\mathbf{x}^{(k)})^{-1}\mathbf{F}(\mathbf{x}^{(k)}), \quad (6.7.2)$$

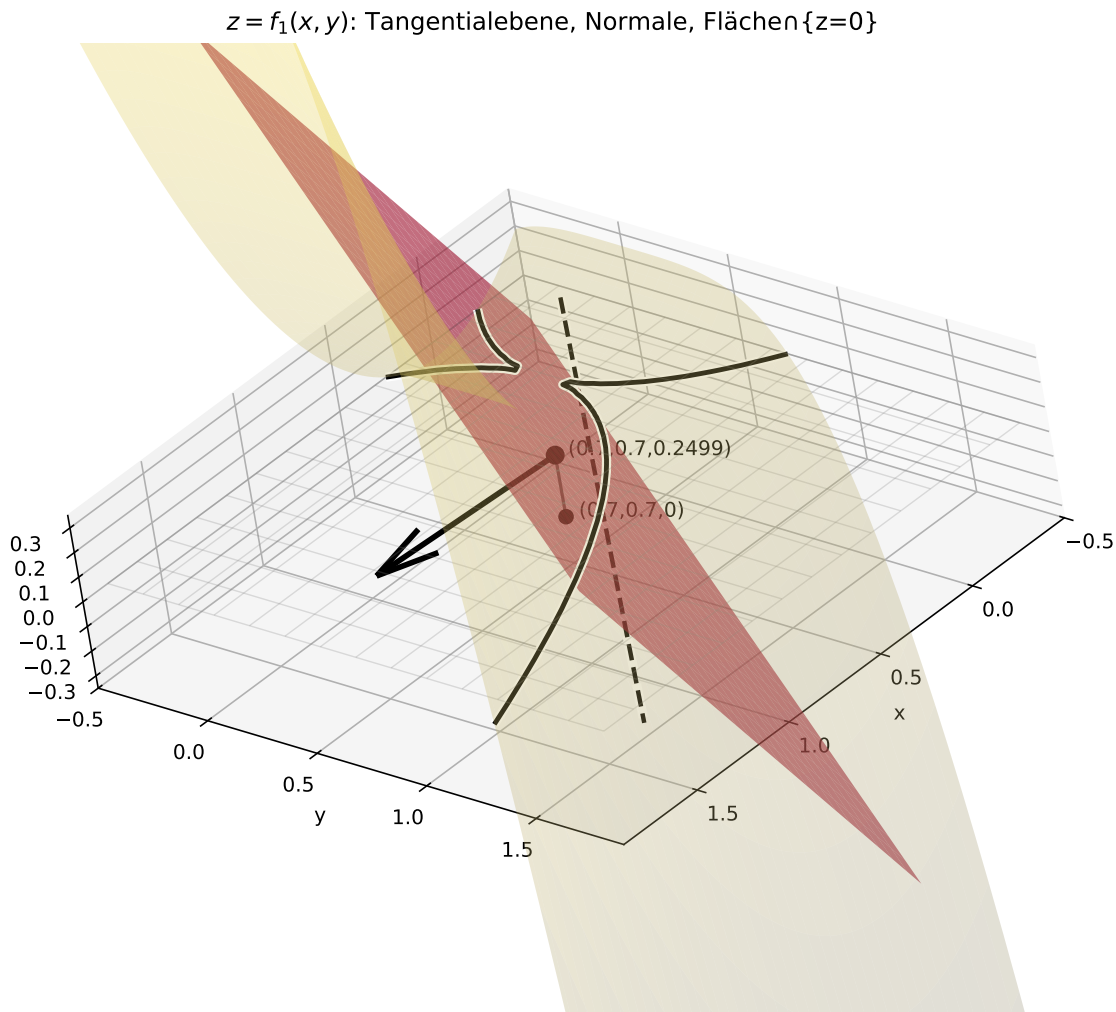
falls $D\mathbf{F}(\mathbf{x}^{(k)})$ invertierbar ist. Der Term $\mathbf{s} = D\mathbf{F}(\mathbf{x}^{(k)})^{-1}\mathbf{F}(\mathbf{x}^{(k)})$ heisst Newton-Korrektur und wird als Lösung des linearen Gleichungssystems $D\mathbf{F}(\mathbf{x}^{(k)})\mathbf{s} = \mathbf{F}(\mathbf{x}^{(k)})$ in jedem Schritt berechnet.

Beachte: Man berechnet **niemals** die Inverse einer Matrix: $\mathbf{s} = \mathbf{A}^{-1}\mathbf{b}$, sondern löst das Gleichungssystem $\mathbf{As} = \mathbf{b}$.

Eine visuelle Interpretation ergibt sich, wenn wir die Linearisierung der einzelnen Komponenten von $\mathbf{F}$ betrachten: Deren Graphen sind die tangentialen Ebenen an die Graphen der Komponentenfunktionen. Z.B. für

$$\mathbf{F}(x, y) = \begin{bmatrix} x^2 - y^4 \\ x - y^3 \end{bmatrix}, \quad \mathbf{x} = [x, y]^T \in \mathbb{R}^2 \quad \text{mit Lösung} \quad F([1, 1]^T) = 0.$$

kann man sich die Graphen der Komponenten von $\mathbf{F}(x, y) = [f_1(x, y), f_2(x, y)]$ zusammen mit den Normalenvektoren und tangentialen Ebenen in $[x_0, y_0, f_1(x_0, y_0)]$ und $[x_0, y_0, f_2(x_0, y_0)]$ anschauen.



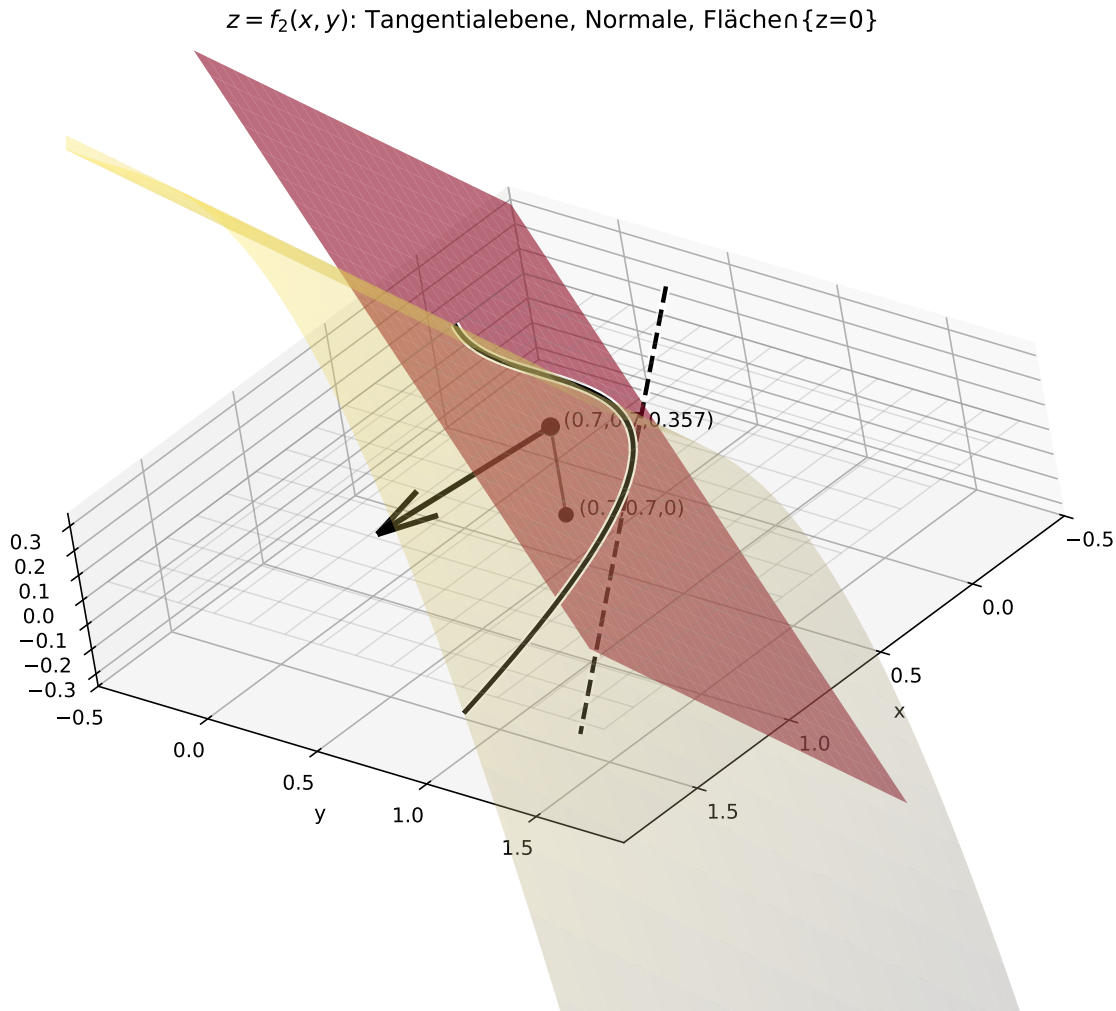


Abb. 6.7.1. Die Graphen der Komponenten von $\mathbf{F}(x, y) = [f_1(x, y), f_2(x, y)]$ zusammen mit den Normalenvektoren und tangentialen Ebenen im Startpunkt.

Der Schnitt jeder dieser Ebenen mit der $z = 0$ -Ebene ist eine Gerade, und der Schnitt dieser Geraden ist die Lösung des linearen Gleichungssystems, das die nicht-lineare Gleichung im Newton-Schritt approximiert. Wenn wir dies nur in der Ebene darstellen wollen, dann sehen wir die Niveau-0-Linien $f_1(x, y) = 0$ und $f_2(x, y) = 0$ mit deren Schnittpunkt(en) (d.h. Lösungen des nichtlinearen Systems) und die Niveau-0-Linien der entsprechenden Linearisierungen, d.h. die Schnittlinien der tangentialen Ebenen mit der $z = 0$ -Ebene von vornhin.

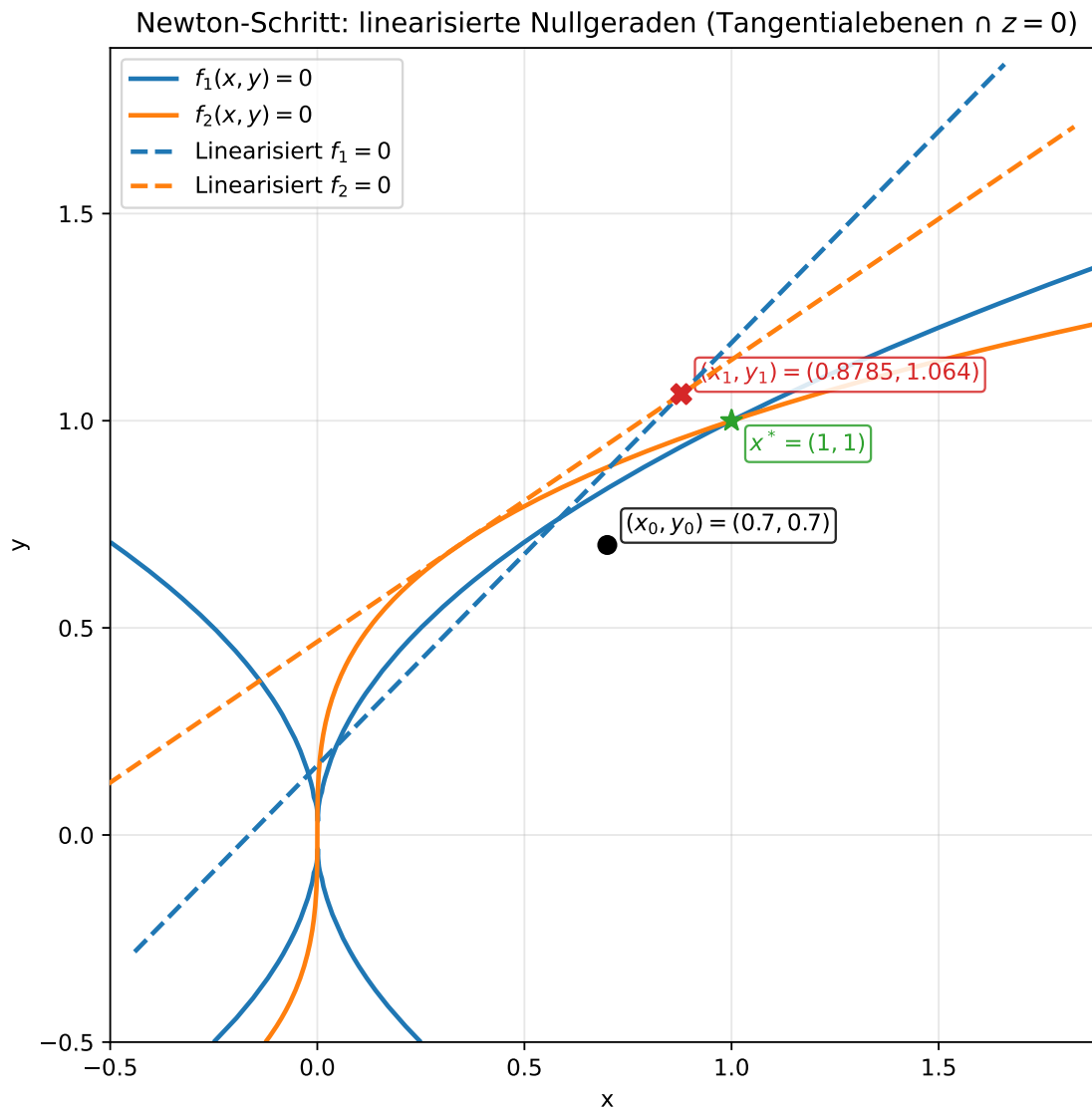


Abb. 6.7.2. Ein Schritt des Newton-Verfahrens in 2D

Code 6.7.3: Newtonverfahren

```

from numpy import atleast_2d
2 from numpy.linalg import solve, norm

4 def newton(x, F, DF, tol=1e-12, maxit=50):
    """Newton Iteration
6     x: Startwert der Iteration
        F: Funktion F(x)
8     DF: Jacobi Matrix von F(x)
    """
10     # 'solve' erwartet x als 2-dimensionaler numpy array
    x = atleast_2d(x)
12     # Newton Iteration
    for i in range(maxit):

```

```

14      s = solve(DF(x), F(x))
      x -= s
16      if norm(s) < tol*norm(x):
          return x

```

Abbruchkriterium:

Liegt $\mathbf{x}^{(k)}$ nahe an der Lösung, so gilt

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\| \approx \|\mathbf{x}^{(k)} - \mathbf{x}^{(k+1)}\| = \|D\mathbf{F}(\mathbf{x}^{(k)})^{-1}\mathbf{F}(\mathbf{x}^{(k)})\| \leq \tau. \quad (6.7.3)$$

Da die Berechnung von $D\mathbf{F}(\mathbf{x}^{(k)})^{-1}\mathbf{F}(\mathbf{x}^{(k)})$ aber teuer ist und wir $D\mathbf{F}(\mathbf{x}^{(k)})^{-1}$ eventuell nicht mehr brauchen (im Falle eines Abbruchs), kann man auch $D\mathbf{F}(\mathbf{x}^{(k)})^{-1} \approx D\mathbf{F}(\mathbf{x}^{(k-1)})^{-1}$ verwenden, da $\mathbf{x}^{(k)}$ ja nahe an der Lösung liegt. Somit erhält man folgendes Abbruchkriterium:

$$\|D\mathbf{F}(\mathbf{x}^{(k-1)})^{-1}\mathbf{F}(\mathbf{x}^{(k)})\| \leq \tau \implies \text{Abbruch}. \quad (6.7.4)$$

$D\mathbf{F}(\mathbf{x}^{(k-1)})^{-1}\mathbf{F}(\mathbf{x}^{(k)})$ heisst die vereinfachte Newton-Korrektur.

Beispiel 6.7.4. (Newton-Verfahren in 2D)

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} x_1^2 - x_2^4 \\ x_1 - x_2^3 \end{bmatrix}, \quad \mathbf{x} = [x_1, x_2]^T \in \mathbb{R}^2 \quad \text{mit Lösung} \quad F([1, 1]^T) = 0.$$

Die Jacobi-Matrix können wir analytisch berechnen:

$$D\mathbf{F}(\mathbf{x}) = \begin{bmatrix} \partial_{x_1} F_1(x) & \partial_{x_2} F_1(x) \\ \partial_{x_1} F_2(x) & \partial_{x_2} F_2(x) \end{bmatrix} = \begin{bmatrix} 2x_1 & -4x_2^3 \\ 1 & -3x_2^2 \end{bmatrix}.$$

Die Newton-Iteration (6.7.2) lautet nun:

1. Löse das lineare Gleichungssystem

$$\begin{bmatrix} 2x_1 & -4x_2^3 \\ 1 & -3x_2^2 \end{bmatrix} \Delta \mathbf{x}^{(k)} = \begin{bmatrix} x_1^2 - x_2^4 \\ x_1 - x_2^3 \end{bmatrix}.$$

wobei $\mathbf{x}^{(k)} = [x_1, x_2]^T$.

2. Setze $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \Delta \mathbf{x}^{(k)}$.

Code 6.7.5: Newton-Iteration in 2D

```

1  from numpy import array, diff, log, zeros, vstack
   from scipy.linalg import norm, solve
3
   F = lambda x: array([x[0]**2 - x[1]**4, x[0]-x[1]**3]) #the system we want to
   solve
5  DF = lambda x: array([[2*x[0], - 4*x[1]**3], [1, -3*x[1]**2]]) #its jacobi
   matrix
7
   #x is the starting vector
   x = array([0.7, 0.7])
9
   #x0 is the exact solution
11  x0 = array([1., 1.])
   tol = 1e-10
13
   res = zeros(4); res[1:3] = x; res[3] = norm(x-x0)
15  #res is a vector containing:
   #in its first entry, k (for the k-th iteration)
17  #in its second and third entry, the approximation of the solution found in the
   k-th step
   #in its last entry, the norm of the difference between the k-th approximation an
   the exact solution
19
   print(DF(x))
21  print(F(x))
   s = solve(DF(x),F(x))
23  #important: we never calculate the inverse of a matrix, we solve a linear system
   instead
   x -= s
25  res1 = zeros(4); res1[0] = 1.; res1[1:3] = x; res1[3] = norm(x-x0)
   res = vstack((res,res1)) #[res,res1]
27  k = 2
29
   #now we perform the newton method until we find a reasonably good solution
   while norm(s) > tol*norm(x):
31       s = solve(DF(x),F(x))
       x -= s
33       res1 = zeros(4); res1[0] = k; res1[1:3] = x; res1[3] = norm(x-x0)
       res = vstack((res,res1)) #[res0,res1,...,resk]
35       k += 1 #update the iteration number
37
   logdiff = diff(log(res[:,3]))
   rates = logdiff[1:]/logdiff[:-1] #usual way of estimating the convergence rate
39
   print(res)
41  print(rates)

```

Die Ergebnisse beim Startpunkt (0.7, 0.7) zeigen offensichtlich quadratische Konvergenz gegen die Lösung (1, 1):

k	$\mathbf{x}^{(k)}$	$\epsilon_k := \ \mathbf{x}^* - \mathbf{x}^{(k)}\ _2$
0	$(0.7, 0.7)^T$	4.24e-01
1	$(0.878500000000000, 1.064285714285714)^T$	1.37e-01
2	$(1.01815943274188, 1.00914882463936)^T$	2.03e-02
3	$(1.00023355916300, 1.00015913936075)^T$	2.83e-04
4	$(1.00000000583852, 1.00000002726552)^T$	2.79e-08
5	$(0.999999999999998, 1.000000000000000)^T$	2.11e-15
6	$(1, 1)^T$	

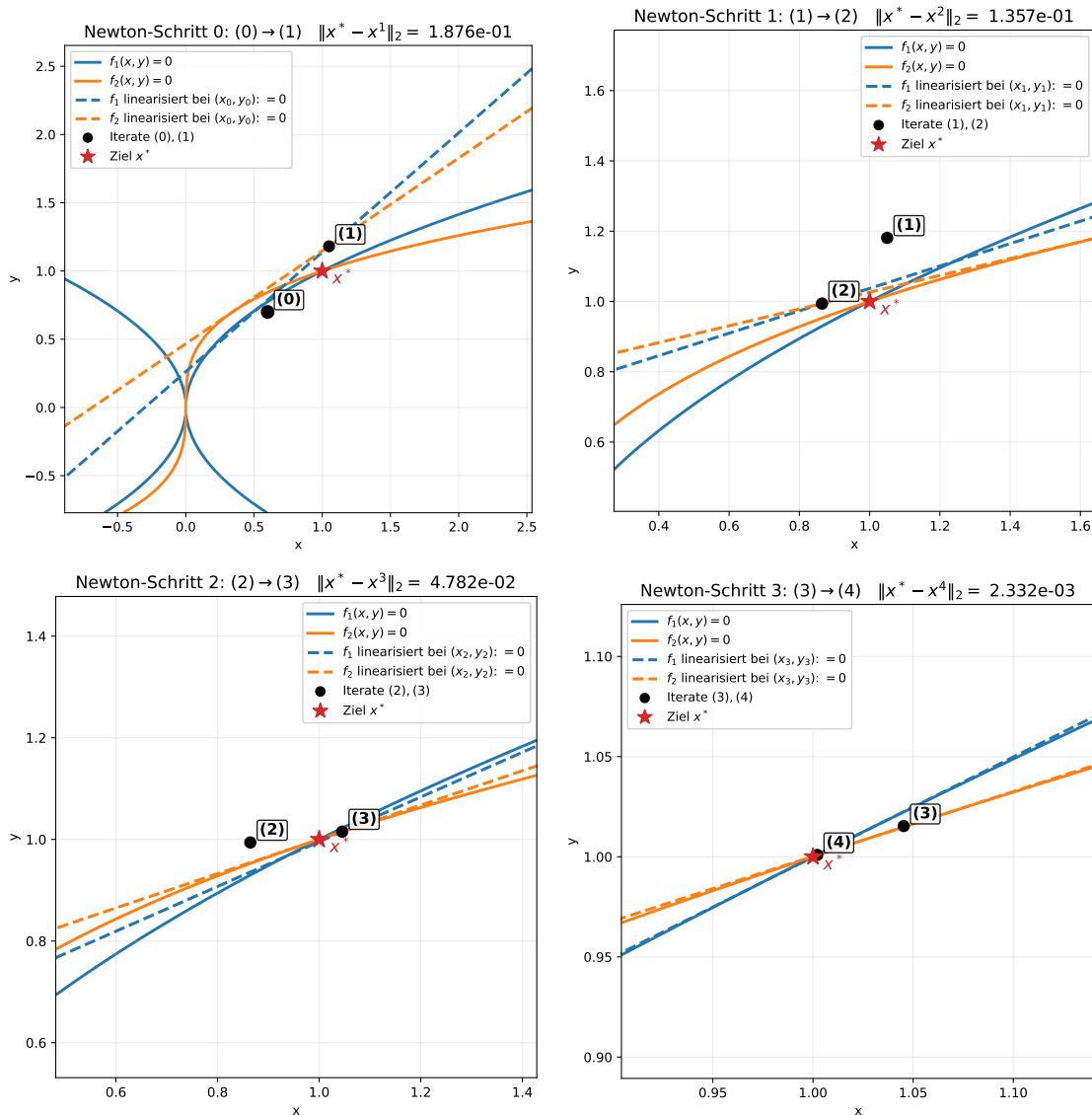


Abb. 6.7.6. Die ersten Schritte des Newton-Verfahrens für den Startwert $(0.6, 0.7)$

Mit dem Startwert $(0.6, 0.7)$ beobachten wir weiterhin quadratische Konvergenz gegen die Lösung $(1, 1)$, obwohl das erste lineare Gleichungssystem als ziemlich schlecht konditioniert erscheint.

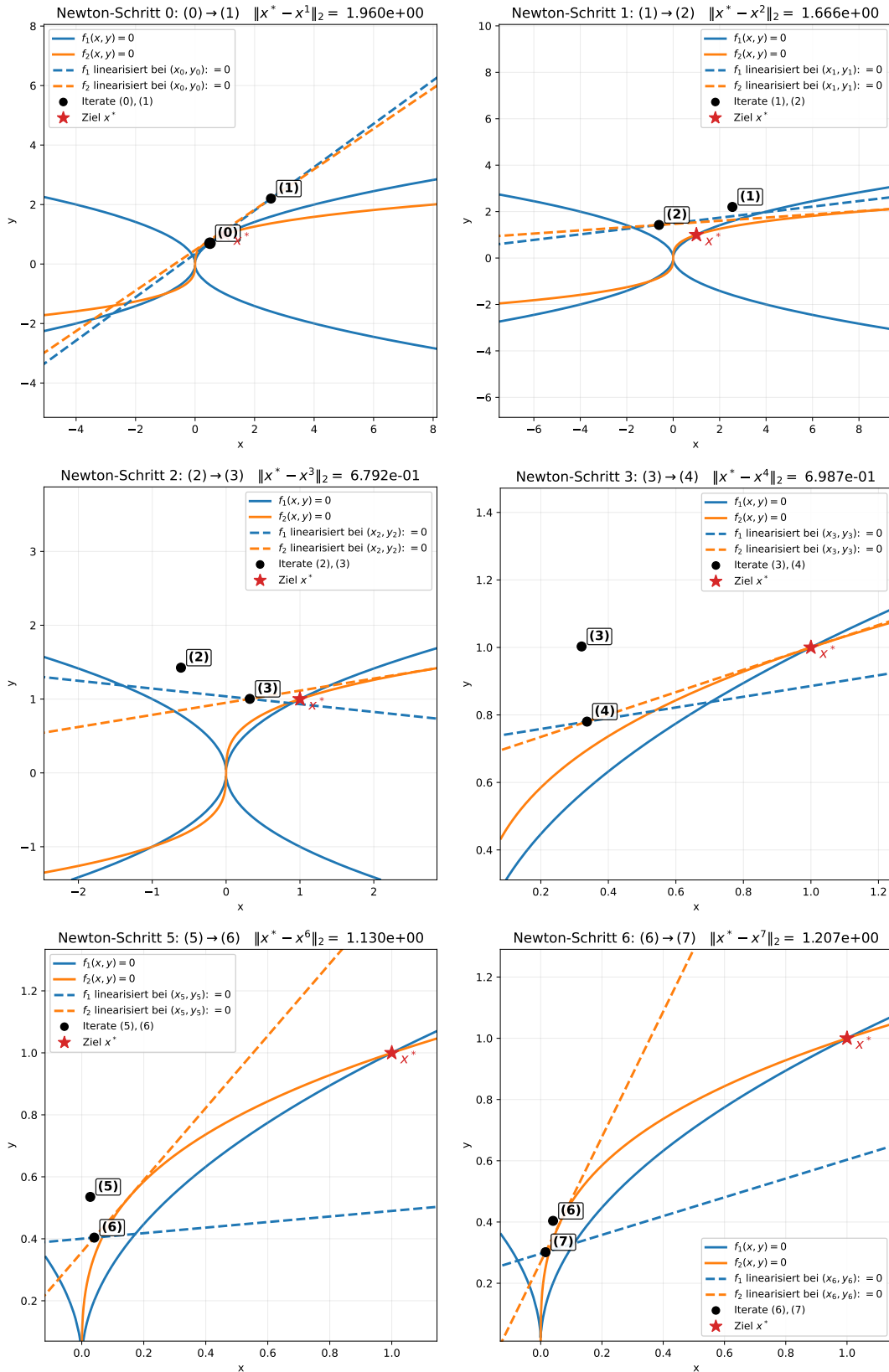


Abb. 6.7.7. Die ersten Schritte des Newton-Verfahrens für den Startwert (0.5, 0.7)

Der Startwert $(0.5, 0.7)$ liegt im Einzugsgebiet der Lösung $(0, 0)$, was eine doppelte Nullstelle ist: Die linearen Gleichungssysteme sind sehr schlecht konditioniert, und die Konvergenz ist nur linear.

Das Paket `sympy` kann bei der Berechnung der Ableitungen hilfreich sein:

Code 6.7.8: Newton-Iteration in 2D mit `sympy`

```

1  import numpy as np
2  import scipy.optimize
3  from sympy import *
4  import matplotlib.pyplot as plt
5
6  x_, y_ = symbols('x y') # define symbols
7
8  f = sin(pi*sqrt(x_**2+y_**2)) # define function f for sympy
9
10 dfdx = diff(f, x_, 1) # its derivative wrt x
11 dfdy = diff(f, y_, 1) # its derivative wrt y
12
13 g = y_-x_**2 # define function g for sympy
14 dgdx = diff(g, x_, 1) # its derivative wrt x
15 dgdy = diff(g, y_, 1) # its derivative wrt y
16
17 # make them useable with numpy:
18 f1 = lambdify((x_, y_), f, "numpy")
19 f2 = lambdify((x_, y_), g, "numpy")
20
21 # prepare 2D grid:
22 N=500
23 x = np.linspace(-3,3,N)
24 X, Y = np.meshgrid(x,x)
25 Z = f1(X, Y)
26 cutoff=5
27 # draw contour lines with value 0
28 CS =plt.contour(X, Y, Z, [0],linestyle='--', colors='b', label='Nullstellen von
29 $f_1$')
30 plt.gca().clabel(CS, inline=1, fontsize=10)
31 plt.contour(X, Y, f2(X,Y), [0], colors='g', label='Nullstellen von $f_2$')
32 plt.legend()
33 plt.grid(True)
34 plt.show()
35 # draw f1
36 plt.pcolormesh(X, Y, f1(X,Y), vmax=20, vmin=-20)
37 plt.colorbar()
38 plt.show()
39
40 # a variant of Newton
41 def newton(f, x0, df, tol=1e-5, N=1000):
42
43     error = np.linalg.norm(f(x0))
44     i=0
45     x=x0
46     while error > tol and i < N:

```

```

    fprime = df(x)
    fx = f(x)
47     x = x - np.linalg.solve(fprime, fx)
49     error = np.linalg.norm(f(x))
    i+=1
51     return x

53 # make derivatives useable in numpy
df1dx = lambdaify((x_, y_), dfdx, 'numpy')
55 df1dy = lambdaify((x_, y_), dfdy, 'numpy')
#
57 df2dx = lambdaify((x_, y_), dgdx, 'numpy')
df2dy = lambdaify((x_, y_), dgdy, 'numpy')
59 # build Jacobi matrix for w= [x,y]
dprime = lambda w: np.array([[df1dx(w[0], w[1]), df1dy(w[0], w[1])],
61                             [df2dx(w[0], w[1]), df2dy(w[0], w[1])]])

# build F for w = [x,y]
63 F = lambda w: np.array([f1(w[0], w[1]), f2(w[0], w[1])])
# failure
65 x = newton(F, np.array([1,1]), dprime)
print(x)
67 print(F(x))
# success
69 s = np.array([0.75,0.75])
x = newton(F,s, dprime)
71 print(x)
print(F(x))

```

Bemerkung 6.7.9. Im Falle vieler Dimensionen ($d \gg 1$) ist die Berechnung einer Newton-Korrektur teuer, denn $D\mathbf{F}(\mathbf{x}^{(k)})$ sind (volle) $d \times d$ -Matrizen.

Bemerkung 6.7.10. Das vereinfachte Newton-Verfahren verwendet $D\mathbf{F}(\mathbf{x}^{(k-1)})$ für mehrere Schritte:

$$\Delta \bar{\mathbf{x}}^{(k)} := D\mathbf{F}(\mathbf{x}^{(k-1)})^{-1} \mathbf{F}(\mathbf{x}^{(k)}).$$

Der Vorteil ist, dass in diesem Fall die LU-Zerlegung von $D\mathbf{F}(\mathbf{x}^{(k-1)})$ wiederverwendet werden kann, so dass die vereinfachte Newton-Korrektur $\Delta \bar{\mathbf{x}}^{(k)}$ in $O(d^2)$ Operationen zur Verfügung steht. Der Preis dafür ist der Verlust der quadratischen Konvergenzordnung: Das vereinfachte Newton-Verfahren konvergiert nur linear.

6.8 Gedämpftes Newton-Verfahren

Das Newton-Verfahren stellt sich oft als gute Methode heraus; allerdings konvergiert es nicht immer. Oft hängt der Erfolg der Iteration von der Wahl des Anfangsparameters ab. Wir wollen dazu zwei Beispiele betrachten:

Beispiel 6.8.1. (Lokale Konvergenz des Newton-Verfahrens)

$$F(x) = xe^x - 1 \implies F'(-1) = 0$$

$$x^{(0)} < -1 \implies x^{(k)} \rightarrow -\infty$$

$$x^{(0)} > -1 \implies x^{(k)} \rightarrow x^*.$$

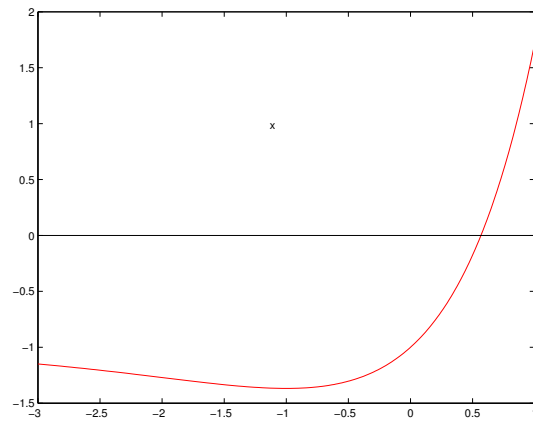


Abb. 6.8.2. Verhalten von $F(x) = xe^x - 1$

Beispiel 6.8.3. (Konvergenzgebiet des Newton-Verfahrens)

$$F(x) = \arctan(ax), \quad a > 0, x \in \mathbb{R} \quad \text{mit der Nullstelle} \quad x^* = 0.$$

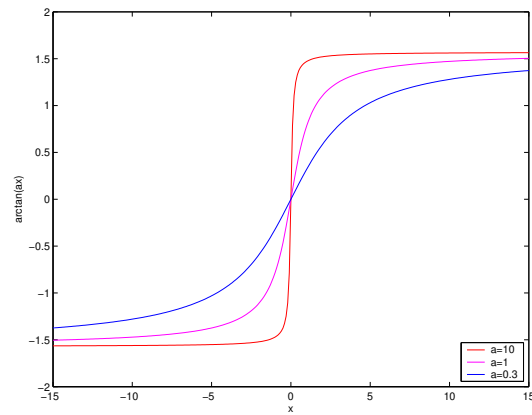


Abb. 6.8.4. Verhalten von $F(x) = \arctan(ax)$ für verschiedene Parameter a .

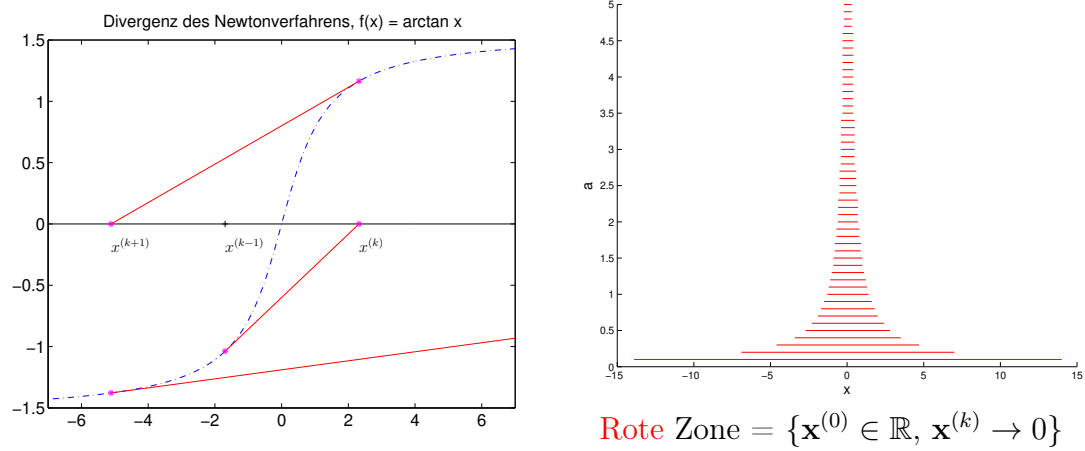


Abb. 6.8.5. Wertebereich für den Startwert $x^{(0)}$, damit das Newton-Verfahren für $F(x) = \arctan(ax)$ konvergiert

Sei $F(x) = \arctan(x)$. Wendet man das normale Newton-Verfahren auf F an, so konvergiert dieses nur für Startwerte im ungefähren Intervall $[-1.39, 1.39]$. Für Werte ausserhalb dieses Bereichs divergiert das Verfahren.

Wir beobachten, dass die Korrekturen zu lang sind. Wir dämpfen sie mit dem schrittabhängigen Faktor $\lambda^{(k)}$. Die Wahl dieses Dämpfungsparameters $\lambda^{(k)}$ ist heuristisch. Wir können zum Beispiel wie im Folgenden vorgehen:

Sei $\Delta \mathbf{x}^{(k)} = D\mathbf{F}(\mathbf{x}^{(k)})^{-1}\mathbf{F}(\mathbf{x}^{(k)})$ die ursprüngliche Newton-Korrektur und $\Delta \mathbf{x}(\lambda^{(k)}) = D\mathbf{F}(\mathbf{x}^{(k)})^{-1}\mathbf{F}(\mathbf{x}^{(k)} - \lambda^{(k)}\Delta \mathbf{x}^{(k)})$ eine provisorische Korrektur mit dem vereinfachten Newton-Verfahren. Wir wählen das grösste $\lambda^{(k)} > 0$, so dass

$$\|\Delta \mathbf{x}(\lambda^{(k)})\|_2 \leq \left(1 - \frac{\lambda^{(k)}}{2}\right) \|\Delta \mathbf{x}^{(k)}\|_2.$$

Dann ist $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \lambda^{(k)} D\mathbf{F}(\mathbf{x}^{(k)})^{-1}\mathbf{F}(\mathbf{x}^{(k)})$.

Code 6.8.6: Gedämpftes Newton-Verfahren

```

1 from scipy.linalg import lu_solve, lu_factor, norm
2 from numpy import array, arctan, exp
4 def dampnewton(x, F, DF, q=0.5, rtol=1e-10, atol=1E-12):
5     # q is the used damping factor
6     cvg = [] #container storing, for each iteration, lambda, xn and F(xn)
7     lup = lu_factor(DF(x)) # first LU decomposition for simplified Newton
8     s = lu_solve(lup,F(x)) # first tentative Newton correction
9     lam = 1 # damping factor, start from maximum (1)
10    xn = x-lam*s # tentative next Newton iteration
11    st = lu_solve(lup,F(xn)) # tentative simplified Newton correction
12
13    while norm(st) > rtol*norm(xn) and norm(st) > atol: # a posteriori criteria
14        # as long as tge tentative correction is larger that the heuristic
15        while norm(st) > (1-lam*q)*norm(s): # natural monotonicity test
16            lam *= q # reduce damping factor
17            if lam < 1e-4:
18                cvg = -1; print('DAMPED NEWTON: Failure of convergence')
19                return x, cvg
20            xn = x-lam*s # tentative next Newton iteration
21            st = lu_solve(lup,F(xn)) # tentative simplified Newton correction
22            cvg += [[lam, norm(xn), norm(F(xn))]]
23            x = xn # accept the proposed the damped Newton iterate
24            lup = lu_factor(DF(x)) # next LU decomposition for simplified Newton
25            s = lu_solve(lup,F(x)) # next tentative Newton correction
26            lam = min(lam/q, 1.) # notice that we must have 1 - qλ > 0
27            xn = x-lam*s # tentative next Newton iteration
28            st = lu_solve(lup,F(xn)) # tentative simplified Newton correction
29
30    x = xn
31    return x, array(cvg)
32
33 #now we test the damped newton method in three different cases
34 if __name__=='__main__':

```

```

36 print('----- 2D F -----')
F = lambda x: array([x[0]**2 - x[1]**4, x[0]-x[1]**3])
38 DF = lambda x: array([[2*x[0], - 4*x[1]**3], [1, -3*x[1]**2]])
x = array([0.7, 0.7]) # exact soluiton: array([1., 1.])
x, cvg = dampnewton(x,F,DF)
40 print('guessed solution:',x, ' in', len(cvg), ' iterations')
42 print(' lambda, |xn|, |F(xn)|\n', cvg)
q = 0.6; print('try damping factor q = ', q)
x = array([0.7, 0.7])
44 x, cvg = dampnewton(x,F,DF,q=q)
46 print('guessed solution:',x, ' in', len(cvg), ' iterations')
print(' lambda, |xn|, |F(xn)|\n', cvg)

48 print('----- arctan -----')
F = lambda x: array([arctan(x[0])])
50 DF = lambda x: array([[1./(1.+x[0]**2)])])
x = array([20.]) # exact soluiton: array([0.])
52 x, cvg = dampnewton(x,F,DF)
54 print('guessed solution:',x, ' in', len(cvg), ' iterations')
print(' lambda, |xn|, |F(xn)|\n', cvg)
q = 0.25; print('try damping factor q = ', q)
56 x = array([20.])
x, cvg = dampnewton(x,F,DF,q=q)
58 print('guessed solution:',x, ' in', len(cvg), ' iterations')
print(' lambda, |xn|, |F(xn)|\n', cvg)

60 print('----- x e^x - 1 -----')
62 F = lambda x: array([x[0]*exp(x[0])-1.])
DF = lambda x: array([[exp(x[0])*(x[0]+1.)]])
64 x = array([-1.5]) # direction of correction is wrong, damping has no chance
x, cvg = dampnewton(x,F,DF)

```

Beispiel 6.8.7. (Gedämpftes Newton-Verfahren)(Siehe Beispiel 6.8.3): $F(x) = \arctan(x)$:

- $x^{(0)} = 20$
- $q = \frac{1}{2}$
- $\text{LMIN} = 0.001$

Beobachtung: asymptotisch quadratische Konvergenz.

k	$\lambda^{(k)}$	$\mathbf{x}^{(k)}$	$\mathbf{F}(\mathbf{x}^{(k)})$
1	0.03125	0.94199967624205	0.75554074974604
2	0.06250	0.85287592931991	0.70616132170387
3	0.12500	0.70039827977515	0.61099321623952
4	0.25000	0.47271811131169	0.44158487422833
5	0.50000	0.20258686348037	0.19988168667351
6	1.00000	-0.00549825489514	-0.00549819949059
7	1.00000	0.00000011081045	0.00000011081045
8	1.00000	-0.00000000000001	-0.00000000000001

Beispiel 6.8.8. (Gedämpftes Newton-Verfahren funktioniert nicht immer)

- Wie im Beispiel 6.8.1: $F(x) = xe^x - 1$,
- wir nehmen als Startwert für das gedämpfte Newton-Verfahren: $x^{(0)} = -1.5$.

k	$\lambda^{(k)}$	$\mathbf{x}^{(k)}$	$\mathbf{F}(\mathbf{x}^{(k)})$
1	0.25000	-4.4908445351690	-1.0503476286303
2	0.06250	-6.1682249558799	-1.0129221310944
3	0.01562	-7.6300006580712	-1.0037055902301
4	0.00390	-8.8476436930246	-1.0012715832278
5	0.00195	-10.5815494437311	-1.0002685596314
Bailed out because of <code>lambda < LMIN</code> !			

Beobachtung: Die Newton-Korrektur zeigt in die “falsche Richtung”, also haben wir keine Konvergenz.

6.9 Das Quasi-Newton Verfahren

Was machen wir, falls $D\mathbf{F}(\mathbf{x})$ zu teuer ist, um es zu berechnen, oder falls es gar nicht zur Verfügung steht? Für eindimensionale Probleme haben wir das Sekantenverfahren eingeführt, bei dem man den Differenzenquotienten statt $F'(x^k)$ verwendet:

$$F'(x^{(k)}) \approx \frac{F(x^{(k)}) - F(x^{(k-1)})}{x^{(k)} - x^{(k-1)}} = J_k. \quad (6.9.1)$$

Dabei wurden $F(x^{(k-1)})$ und $F(x^{(k)})$ bereits berechnet und stehen somit günstig zur Verfügung.

Was sollen wir im mehrdimensionalen Fall tun? Da wir durch einen Vektor nicht teilen können, werden wir die Gleichung (6.9.1) in

$$\mathbf{J}_k \cdot (\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}) = \mathbf{F}(\mathbf{x}^{(k)}) - \mathbf{F}(\mathbf{x}^{(k-1)}) \quad (6.9.2)$$

umschreiben. Im eindimensionalen Fall ($n = 1$) ist J_k eine durch (6.9.2) eindeutig definierte Zahl. Für $d \geq 2$ gibt es aber viele $d \times d$ -Matrizen $\mathbf{J}_k$, die (6.9.2) erfüllen.

Hilfreich ist es, wenn sich $\mathbf{J}_k$ durch eine einfache Änderung aus $\mathbf{J}_{k-1}$ ergibt. Die Broyden-Bedingung dazu ist:

$\mathbf{J}_k \mathbf{z} = \mathbf{J}_{k-1} \mathbf{z}$ für alle $\mathbf{z}$, die orthogonal zu $\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}$ sind, d.h. $\mathbf{J}_k$ und $\mathbf{J}_{k-1}$ wirken in gleicher Weise auf den Raum, der orthogonal zur vorigen Korrektur ist. Somit gilt:

$$\mathbf{J}_k := \mathbf{J}_{k-1} + \frac{\mathbf{F}(\mathbf{x}^{(k)}) - \mathbf{F}(\mathbf{x}^{(k-1)}) (\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)})^T}{\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_2^2}.$$

Das ergibt die Broyden-Quasi-Newton-Methode:

$$\begin{aligned}\Delta \mathbf{x}^{(k)} &:= -\mathbf{J}_k^{-1} \mathbf{F}(\mathbf{x}^{(k)}) \\ \mathbf{x}^{(k+1)} &:= \mathbf{x}^{(k)} + \Delta \mathbf{x}^{(k)} \\ \mathbf{J}_{k+1} &:= \mathbf{J}_k + \frac{\mathbf{F}(\mathbf{x}^{(k+1)})(\Delta \mathbf{x}^{(k)})^T}{\|\Delta \mathbf{x}^{(k)}\|_2^2}.\end{aligned}\tag{6.9.3}$$

Dabei müssen wir $\mathbf{J}_0$ z.B. durch $D\mathbf{F}(\mathbf{x}^{(0)})$ definieren.

Bemerkung 6.9.1. Das Broyden-Update von $\mathbf{J}_k$ (6.9.3) gibt die bezüglich der $\|\cdot\|_2$ -Norm minimale relative Korrektur zu $\mathbf{J}_k$, so dass die Sekantenbedingung (6.9.2) erfüllt ist.

Die Implementierung des Broyden-Verfahrens beruht auf der Sherman-Morrison-Woodbury-Update-Formel (die wir hier nicht beweisen wollen):

$$\mathbf{J}_{k+1}^{-1} = \left(\mathbf{I} - \frac{\mathbf{J}_k^{-1} \mathbf{F}(\mathbf{x}^{(k+1)})(\Delta \mathbf{x}^{(k)})^T}{\|\Delta \mathbf{x}^{(k)}\|_2^2 + (\Delta \mathbf{x}^{(k)})^T \mathbf{J}_k^{-1} \mathbf{F}(\mathbf{x}^{(k+1)})} \right) \mathbf{J}_k^{-1}.$$

Beispiel 6.9.2. (Broyden-Quasi-Newton-Verfahren: Konvergenz)

Wir betrachten das numerische Beispiel 6.7.4, also $d = 2$; wir starten mit $\mathbf{x}^{(0)} = [0.7, 0.7]^T$ und $\mathbf{J}_0 = D\mathbf{F}(\mathbf{x}^{(0)})$.

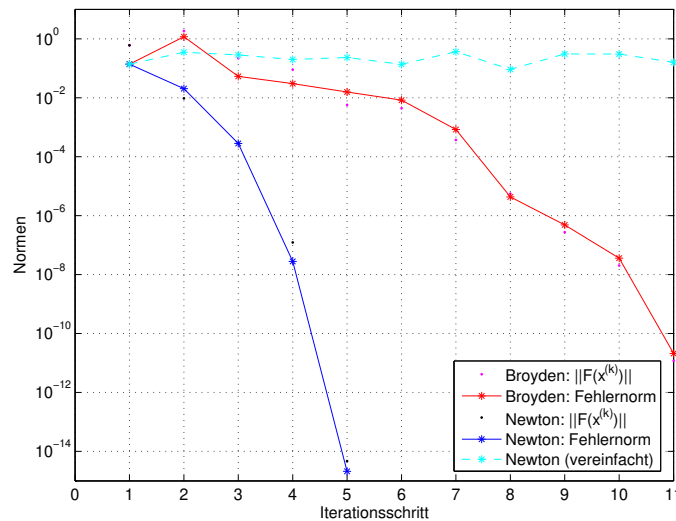


Abb. 6.9.3. Fehler im Broyden-, Newton- und vereinfachten Newton-Verfahren.

Die Ergebnisse zeigen: Diese Methode konvergiert langsamer als das Newton-Verfahren (6.7.2), aber schneller als das vereinfachte Newton-Verfahren.

Code 6.9.4: Broyden-Verfahren

```

1  from scipy.linalg import lu_solve, lu_factor, norm, solve
2  from numpy import dot, zeros, zeros_like
3
4  def fastbroyd(x0, F, J, tol=1e-12, maxit=20):
5
6      x = x0.copy() # make sure we do not change the input
7      lup = lu_factor(J) # LU decomposition of J
8
9      k = 0; s = lu_solve(lup, F(x)) # start with a Newton corection
10     sn = dot(s,s) # squared norm of the correction
11     x -= s; f = F(x) # start with a full Newton step
12
13     #containers for storing corrections s and their sn:
14     dx = zeros((maxit,len(x))); dxn = zeros(maxit)
15     dx[k] = s; dxn[k] = sn
16     k += 1; # the number of the Broyden iteration
17
18     #now we perform the Broyden iteration
19     while sn > tol and k < maxit:
20         w = lu_solve(lup, f) #f = F(actual Broyden iteration x)
21         #now we update the correction for the k-th step
22         #using the Sherman-Morrison-Woodbury formel
23         for r in range(1,k):
24             w += dx[r]*( dot(dx[r-1],w )/dxn[r-1])
25         z = dot(s,w)
26         s = (1+z/(sn-z))*w
27         sn = dot(s,s)
28         dx[k] = s; dxn[k] = sn
29         x -= s; f = F(x); k+=1 #update x and iteration number k
30
31     return x, k #return the final value and the numbers of iterations needed

```

Beispiel 6.9.5. (Broyden-Verfahren für ein grosses, nichtlineares System)

$$\mathbf{F}(\mathbf{x}) = \begin{cases} \mathbb{R}^d & \mapsto \mathbb{R}^d \\ \mathbf{x} & \mapsto \text{diag}(\mathbf{x})\mathbf{A}\mathbf{x} - \mathbf{b} , \end{cases}$$

$$\mathbf{b} = [1, 2, \dots, d]^T \in \mathbb{R}^d ,$$

$$\mathbf{1} = [1, 1, \dots, 1]^T \in \mathbb{R}^d ,$$

$$\mathbf{A} = \mathbf{I} + \mathbf{a}\mathbf{a}^T \in \mathbb{R}^{d,d} ,$$

$$\mathbf{a} = \frac{1}{\sqrt{\mathbf{1} \cdot (\mathbf{b} - \mathbf{1})}}(\mathbf{b} - \mathbf{1}) .$$

Als Startwert nehmen wir den Vektor $\mathbf{x}^0 = [2, 2 + h, \dots, 4 - h]$, wobei $h = 2/d$ ist. Die Ergebnisse sind den Resultaten von Beispiel 6.9.2 ähnlich:

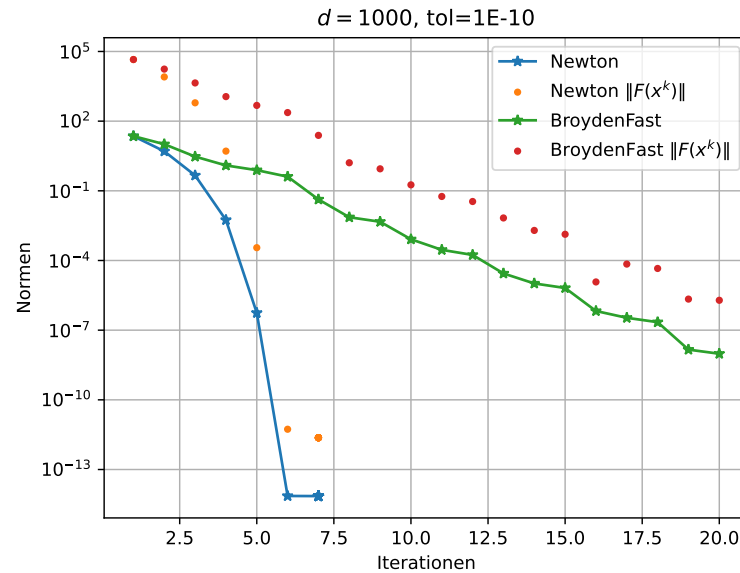


Abb. 6.9.6. Fehler im Broyden-, Newton- und vereinfachten Newton-Verfahren.

Schliesslich können wir die Effizienz der beiden Verfahren vergleichen. Hier haben wir $\text{tol} = 2d \cdot 10^{-5}$ gewählt.

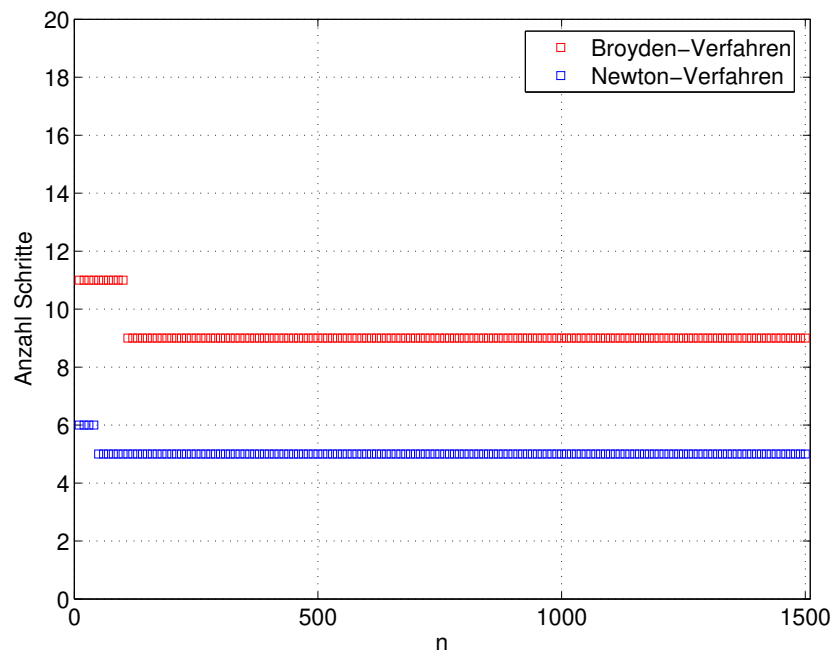


Abb. 6.9.7. Anzahl der nötigen Schritte im Broyden- und Newton-Verfahren.

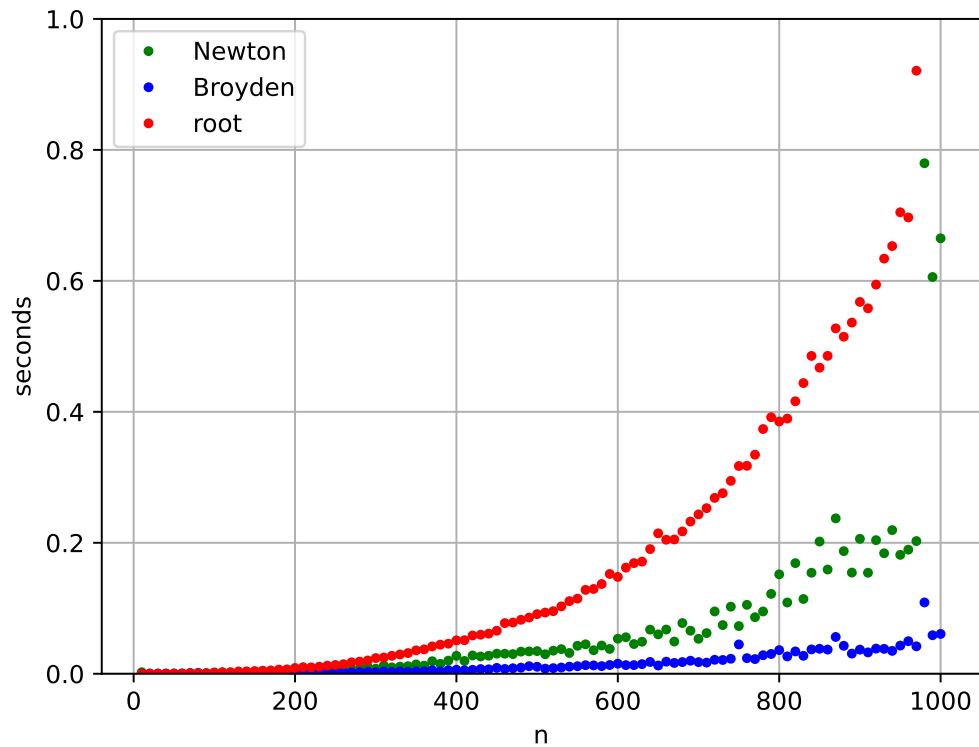


Abb. 6.9.8. Laufzeiten der Broyden- und Newton-Verfahren

Das Broyden-Verfahren lohnt sich für grosse Dimensionen $d \gg 1$ und bescheidene Genauigkeitsanforderungen. Alle Methoden von `scipy.optimize.root` scheitern oder brauchen viel zu viel Zeit bei diesem Problem für grosses d mit diesem Startwert.

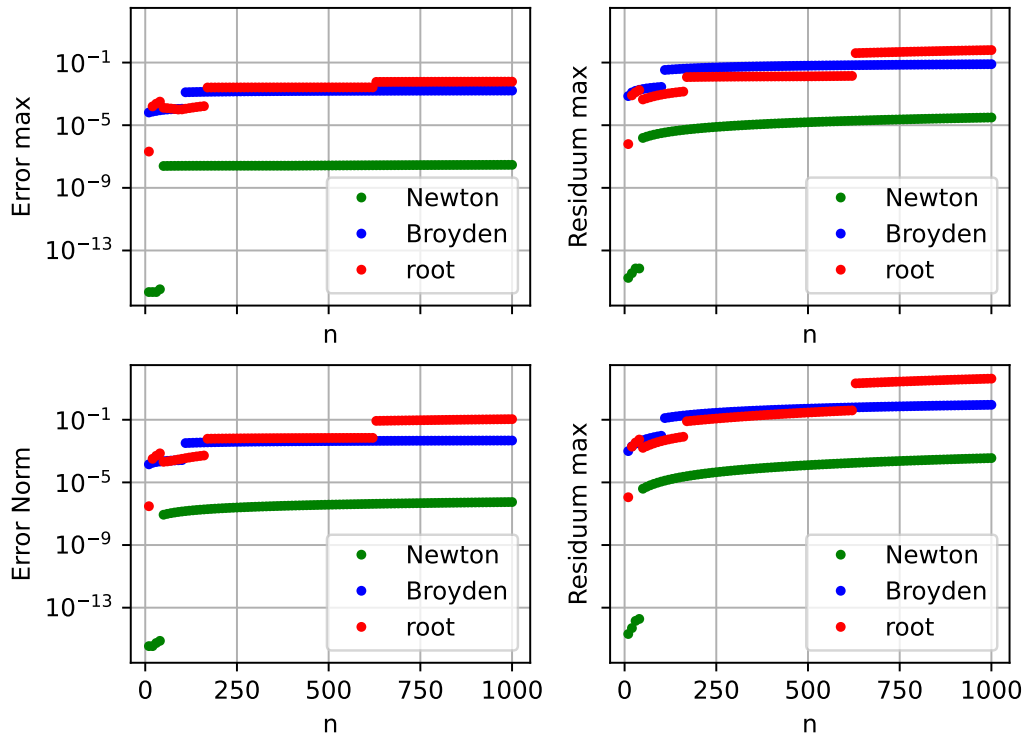


Abb. 6.9.9. Fehler im Broyden- und Newton-Verfahren.

Chapter 7

Intermezzo über (numerische) lineare Algebra

Im Folgenden bezeichnet $\mathbf{A}$ eine $n \times n$ -Matrix mit reellen oder komplexen Einträgen.

7.1.1 Grundlagen

Bemerkung 7.1.1. Das Ergebnis der Multiplikation einer Matrix $\mathbf{A}$ mit einem Vektor $\mathbf{x}$ ist eine lineare Kombination der Spalten von $\mathbf{A}$:

$$\mathbf{A} \mathbf{x} = \sum_{i=1}^n x_i \mathbf{A}_{:,i}.$$

Invertierbar	Nicht invertierbar
A ist regulär	A ist singulär
Spalten sind linear unabhängig	Spalten sind linear abhängig
Zeilen sind linear unabhängig	Zeilen sind linear abhängig
$\det A \neq 0$	$\det A = 0$
$Ax = 0$ hat die Lösung $x = 0$	$Ax = 0$ hat unendlich viele Lösungen
$Ax = b$ hat die Lösung $x = A^{-1}b$	$Ax = b$ hat keine oder unendlich viele Lösungen
A hat vollen Rang	A hat Rang $r < n$
A hat n Nichtnull-Pivote	A hat $r < n$ Pivote
$\text{span}\{A_{:,1}, \dots, A_{:,n}\}$ hat Dimension n	$\text{span}\{A_{:,1}, \dots, A_{:,n}\}$ hat Dimension $r < n$
$\text{span}\{A_{1,:}, \dots, A_{n,:}\}$ hat Dimension n	$\text{span}\{A_{1,:}, \dots, A_{n,:}\}$ hat Dimension $r < n$
Alle Eigenwerte von A sind ungleich null	0 ist Eigenwert von A
$0 \notin \sigma(A) = \text{Spektrum von } A$	$0 \in \sigma(A)$
$A^H A$ ist symmetrisch positiv definit	$A^H A$ ist nur semidefinit
A hat n (positive) Singulärwerte	A hat $r < n$ (positive) Singulärwerte

Definition 7.1.2. Die Vektoren $\mathbf{q}_1, \dots, \mathbf{q}_n$ heissen *orthogonal*, falls

$$\mathbf{q}_i^H \cdot \mathbf{q}_j = 0 \quad \text{für alle } i \neq j;$$

wenn sie ausserdem normiert sind, d.h. $\mathbf{q}_i^H \cdot \mathbf{q}_i = 1$, dann heissen sie *orthonormal*.

Bemerkung 7.1.3. In der vorherigen Definition wird die euklidische Norm verwendet:

$$\|\mathbf{q}\|_2^2 = \mathbf{q}^H \cdot \mathbf{q}.$$

Bemerkung 7.1.4. Wenn $\mathbf{q}_1, \dots, \mathbf{q}_n$ orthonormal sind, dann

$$\mathbf{Q}^H \cdot \mathbf{Q} = \begin{bmatrix} \text{---} & \mathbf{q}_1^H & \text{---} \\ \text{---} & \mathbf{q}_2^H & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{q}_n^H & \text{---} \end{bmatrix} \begin{bmatrix} | & | & & | \\ \mathbf{q}_1 & \mathbf{q}_2 & \dots & \mathbf{q}_n \\ | & | & & | \end{bmatrix} = \begin{bmatrix} 1 & & & 0 \\ & 1 & & \\ & & \ddots & \\ 0 & & & 1 \end{bmatrix} = \mathbf{I}_n.$$

Wenn Q quadratisch ist, dann ist Q eine orthogonale (bzw. unitäre) Matrix, also

$$\mathbf{Q}^{-1} = \mathbf{Q}^H.$$

Bemerkung 7.1.5. Orthogonale/unitäre Transformationen erhalten die euklidische Norm:

$$\|\mathbf{Q}\mathbf{x}\|_2^2 = (\mathbf{Q}\mathbf{x})^H(\mathbf{Q}\mathbf{x}) = \mathbf{x}^H \mathbf{Q}^H \mathbf{Q} \mathbf{x} = \mathbf{x}^H \mathbf{I}_n \mathbf{x} = \|\mathbf{x}\|_2^2.$$

Bemerkung 7.1.6. Permutationsmatrix: Zeilen wie $\mathbf{I}_n$, aber in anderer Reihenfolge. Durch Umordnen der Einträge in einem Vektor wird seine Länge nicht verändert!

$$\mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{P}^H \mathbf{P} = \mathbf{I}.$$

Bemerkung 7.1.7. Drehungen verändern die Länge der Vektoren nicht! Die Drehmatrix mit Winkel θ in der 1-3-Ebene ist:

$$\mathbf{Q} = \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix}$$

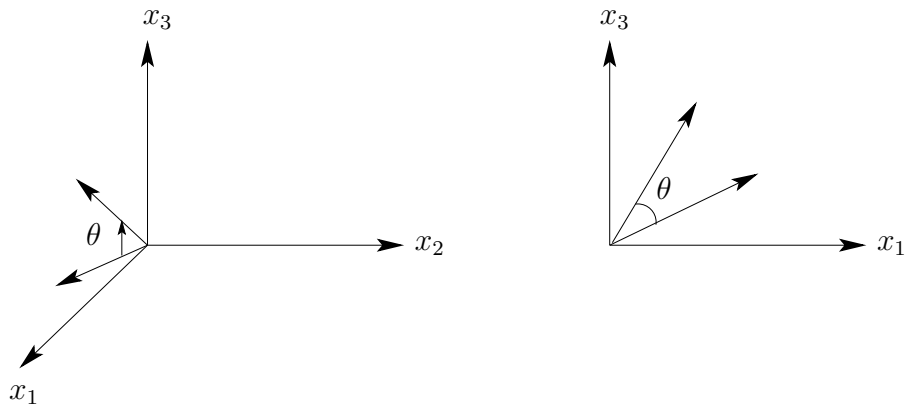


Abb. 7.1.8. Drehung mit Winkel θ in der 1-3-Ebene.

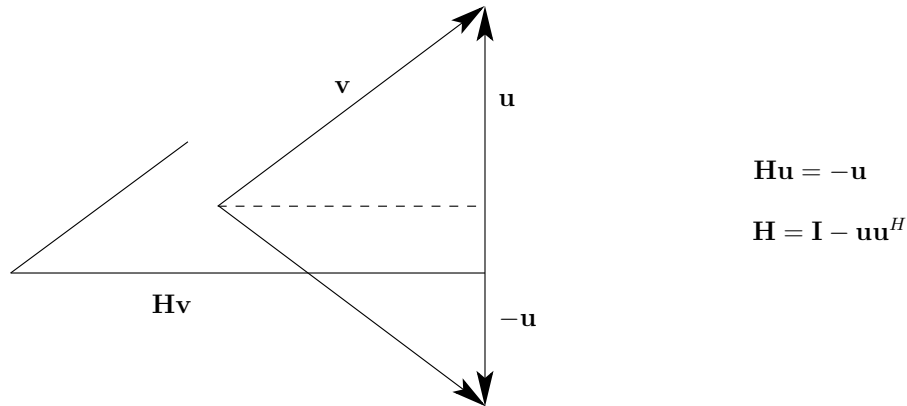


Abb. 7.1.9. Längen werden durch Spiegelungen nicht verändert!

Bemerkung 7.1.10. Gegeben die linear unabhängigen Vektoren $\mathbf{a}_1, \dots, \mathbf{a}_m$, kann man $\mathbf{q}_1, \dots, \mathbf{q}_m$ orthonormale Vektoren finden, die denselben Raum wie $\mathbf{a}_1, \dots, \mathbf{a}_m$ aufspannen. In der Vorlesung über lineare Algebra haben Sie den Gram-Schmidt-Algorithmus kennengelernt:

$$\mathbf{q}_1 := \frac{1}{\|\mathbf{q}_1\|} \mathbf{a}_1$$

$$\mathbf{q}_2 := \mathbf{a}_2 - (\mathbf{q}_1^H \mathbf{a}_2) \mathbf{q}_1; \quad \mathbf{q}_2 := \frac{1}{\|\mathbf{q}_2\|_2} \mathbf{q}_2$$

etc.

Dieser Algorithmus ist aber instabil; deswegen berechnet man $q_1, \dots, q_n$ numerisch mit dem modifizierten Gram-Schmidt-Algorithmus, wie in der entsprechenden Übungsaufgabe beschrieben.

Bemerkung 7.1.11. Das Gram-Schmidt-Verfahren liefert eine QR -Zerlegung der Matrix $\mathbf{A}$:

$$\mathbf{A} = \mathbf{Q}\mathbf{R},$$

mit $\mathbf{Q}$ orthogonal und $\mathbf{R}$ oberer Dreiecksmatrix. Die ersten k Spalten von $\mathbf{Q}$ bilden eine Orthonormalbasis in dem von den ersten k Spalten von $\mathbf{A}$ aufgespannten Raum, und der Basiswechsel erfolgt mit dem entsprechenden Teil der oberen Dreiecksmatrix $\mathbf{R}$.

In jedem Schritt des Gram-Schmidt-Verfahrens wird eine Dreiecksmatrix verwendet. Die QR-Zerlegung kann man sich graphisch im Fall $m > n$ folgendermassen vorstellen:

$$\left[\begin{array}{|c|} \hline Q \\ \hline \end{array} \right] = \left[\begin{array}{|c|} \hline Q \\ \hline \end{array} \right] \left[\begin{array}{|c|} \hline R \\ \hline \end{array} \right], \quad A = QR \quad \begin{array}{l} Q \in \mathbb{K}^{m,n}, \\ R \in \mathbb{K}^{n,n}, \end{array} \quad (7.1.1)$$

wobei $Q^H Q = I$ (da Q orthonormale Spalten hat) und R eine obere Dreiecksmatrix ist. Für $m < n$ sieht die QR-Zerlegung so aus:

$$\left[\begin{array}{|c|} \hline A \\ \hline \end{array} \right] = \left[\begin{array}{|c|} \hline Q \\ \hline \end{array} \right] \left[\begin{array}{|c|} \hline R \\ \hline \end{array} \right], \quad A = QR, \quad Q \in \mathbb{K}^{m,m}, \quad R \in \mathbb{K}^{m,n}.$$

Hier ist Q unitär und R eine obere Dreiecksmatrix.

7.1.2 Wichtige Zerlegungen

1) LU-Zerlegung

$A = LU$ mit

$$L = \left[\begin{array}{|c|} \hline \text{Diagram of L} \\ \hline \end{array} \right] \quad \begin{array}{l} \text{untere Dreiecksmatrix mit 1 auf dem Hauptblock} \\ \text{(lower triangular matrix)} \end{array}$$

und

$$U = \begin{bmatrix} \text{yellow triangle} \end{bmatrix} \quad \begin{array}{l} \text{obere Dreiecksmatrix} \\ \text{(upper triangular matrix)} \end{array}$$

Die Gauss-Elimination ergibt die LU -Zerlegung.

Beispiel 7.1.12. Gauss-Elimination:

$$\begin{bmatrix} 1 & 1 & 0 \\ 2 & 1 & -1 \\ 3 & -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \\ -3 \end{bmatrix} \quad \longleftrightarrow \quad \begin{array}{rrcr} x_1 & + & x_2 & = & 4 \\ 2x_1 & + & x_2 & - & x_3 = & 1 \\ 3x_1 & - & x_2 & - & x_3 = & -3 \end{array}$$

$$\begin{bmatrix} 1 & & \\ & 1 & \\ & & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 \\ 2 & 1 & -1 \\ 3 & -1 & -1 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \\ -3 \end{bmatrix} \Rightarrow$$

$$\begin{bmatrix} 1 & & \\ \textcolor{red}{2} & 1 & \\ & 0 & 1 \end{bmatrix} \begin{bmatrix} \textcolor{blue}{1} & \textcolor{blue}{1} & \textcolor{blue}{0} \\ \textcolor{red}{0} & -1 & -1 \\ 3 & -1 & -1 \end{bmatrix} \begin{bmatrix} 4 \\ -7 \\ -3 \end{bmatrix} \Rightarrow$$

$$\begin{bmatrix} 1 & & \\ 2 & 1 & \\ \textcolor{red}{3} & 0 & 1 \end{bmatrix} \begin{bmatrix} \textcolor{blue}{1} & \textcolor{blue}{1} & \textcolor{blue}{0} \\ 0 & -1 & -1 \\ \textcolor{red}{0} & -4 & -1 \end{bmatrix} \begin{bmatrix} 4 \\ -7 \\ -15 \end{bmatrix} \Rightarrow$$

$$\underbrace{\begin{bmatrix} 1 & & \\ 2 & 1 & \\ 3 & \textcolor{red}{4} & 1 \end{bmatrix}}_{=L} \underbrace{\begin{bmatrix} 1 & 1 & 0 \\ 0 & \textcolor{blue}{-1} & \textcolor{blue}{-1} \\ 0 & \textcolor{red}{0} & 3 \end{bmatrix}}_{=U} \begin{bmatrix} 4 \\ -7 \\ 13 \end{bmatrix}.$$

Blau: **Pivot-Zeile**, blau und fett: **Pivot-Elemente**, rot: **negative Multiplikatoren**

Bemerkung 7.1.13. Numerische Lösung eines linearen Gleichungssystems:

$$Ax = b.$$

1. Zerlege $A = LU$ (Kosten für eine $n \times n$ -Matrix A : $\frac{1}{3} n(n-1)(n-2)$ Operationen¹).

¹Das ist der teure Schritt. Die Gauss-Elimination kann bis $\mathcal{O}(n^{2.373})$ mit sehr komplizierten (also unpraktischen) Algorithmen beschleunigt werden. Die Methode, die ein beliebiges lineares Gleichungssystem in $\mathcal{O}(n^2)$ exakt löst, würde die ganze Wissenschaft revolutionieren, so wie es die FFT nach 1965 tat.

2. Löse $\mathbf{Lz} = \mathbf{b}$ durch Vorwärtssubstitution (Kosten: $\frac{1}{2} n(n-1)$ Operationen).
3. Löse $\mathbf{Ux} = \mathbf{z}$ durch Rückwärtssubstitution (Kosten: $\frac{1}{2} n(n-1)$ Operationen).

Beispiel 7.1.14.

$$\begin{aligned} \mathbf{A} &= \begin{bmatrix} 1 & 2 & 2 \\ 2 & -7 & 2 \\ 1 & 24 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & -7 & 2 \\ 1 & 2 & 2 \\ 1 & 24 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & -7 & 2 \\ 0 & 5.5 & 1 \\ 0 & 27.5 & -1 \end{bmatrix} \rightarrow \\ &\begin{bmatrix} 2 & -7 & 2 \\ 0 & 27.5 & -1 \\ 0 & 5.5 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & -7 & 2 \\ 0 & 27.5 & -1 \\ 0 & 0 & 1.2 \end{bmatrix} \\ \mathbf{U} &= \begin{bmatrix} 2 & -7 & 2 \\ 0 & 27.5 & -1 \\ 0 & 0 & 1.2 \end{bmatrix}, \quad \mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ 0.5 & 0.2 & 1 \end{bmatrix}, \quad \mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}. \end{aligned}$$

Lemma 7.1.15. Für jede $n \times n$ reguläre Matrix A gibt es eine Permutationsmatrix P und eine untere Dreiecksmatrix mit 1 auf der Hauptdiagonalen L zusammen mit einer oberen Dreiecksmatrix U , so dass

$$\mathbf{A} = \mathbf{PLU}.$$

Achtung:

`scipy.linalg.lu` liefert $\mathbf{A} = \mathbf{PLU}$, andere Software eventuell $\mathbf{PA} = \mathbf{LU}$.

Bemerkung 7.1.16. Gauss-Elimination und LU-Zerlegung sind äquivalent. Es ist sinnvoll, die LU-Zerlegung zu verwenden, wenn man die Faktoren $\mathbf{L}$ und $\mathbf{U}$ wiederverwenden möchte, denn der teure Teil des Algorithmus ist die Zerlegung und nicht das Lösen von Dreiecksgleichungssystemen. Man kombiniert dafür $\mathbf{P}, \mathbf{L}, \mathbf{U} = \text{scipy.linalg.lu}(\mathbf{A})$ mit `scipy.linalg.lu_solve(P, L, U)`; dagegen gibt `numpy.linalg.solve` keinen Zugang zu den (intern verwendeten) Faktoren $\mathbf{P}, \mathbf{L}, \mathbf{U}$, sie kann aber parallel auf ganze Batches agieren, d.h. auf ein hochdimensionales `ndarray`, wobei sich die Gauss-Elimination auf die letzten zwei Koordinaten bezieht.

Bemerkung 7.1.17. Jede Vektornorm in $\mathbb{R}^n$ oder $\mathbb{C}^n$ induziert eine Matrixnorm

$$\|\mathbf{A}\| := \sup_{\|\mathbf{x}\| \neq 0} \frac{\|\mathbf{Ax}\|}{\|\mathbf{x}\|} = \sup_{\|\mathbf{x}\|=1} \|\mathbf{Ax}\|.$$

Definition 7.1.18. Die *Konditionszahl* einer Matrix $\mathbf{A}$ ist

$$\text{cond}(\mathbf{A}) := \|\mathbf{A}^{-1}\| \cdot \|\mathbf{A}\|.$$

Bemerkung 7.1.19. $\text{cond}(\mathbf{I}_m) = 1$.

Man kann beweisen: Wenn $\text{cond}(\mathbf{A}) \gg 1$, dann können kleine Änderungen in der Matrix $\mathbf{A}$ sehr grosse Änderungen im Ergebnis $\mathbf{x}$ des linearen Gleichungssystems $\mathbf{Ax} = \mathbf{b}$ produzieren: Man sagt, das mathematische Problem ist *schlecht konditioniert*. Wenn $\text{cond}(\mathbf{A}) \gg 1$, dann können sich die Rundungsfehler propagieren, so dass sehr grosse Fehler in der Gauss-Elimination auftreten².

Beispiel 7.1.20. Wilkinson-Matrix

$$a_{ij} = \begin{cases} 1 & , \text{ wenn } i = j , \\ 1 & , \text{ wenn } j = n , \\ -1 & , \text{ wenn } i > j , \\ 0 & \text{sonst.} \end{cases} \quad \mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ -1 & -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ -1 & -1 & -1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ -1 & -1 & -1 & -1 & 1 & 0 & 0 & 0 & 0 & 1 \\ -1 & -1 & -1 & -1 & -1 & 1 & 0 & 0 & 0 & 1 \\ -1 & -1 & -1 & -1 & -1 & -1 & 1 & 0 & 0 & 1 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & 1 & 0 & 1 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 1 \end{bmatrix}$$

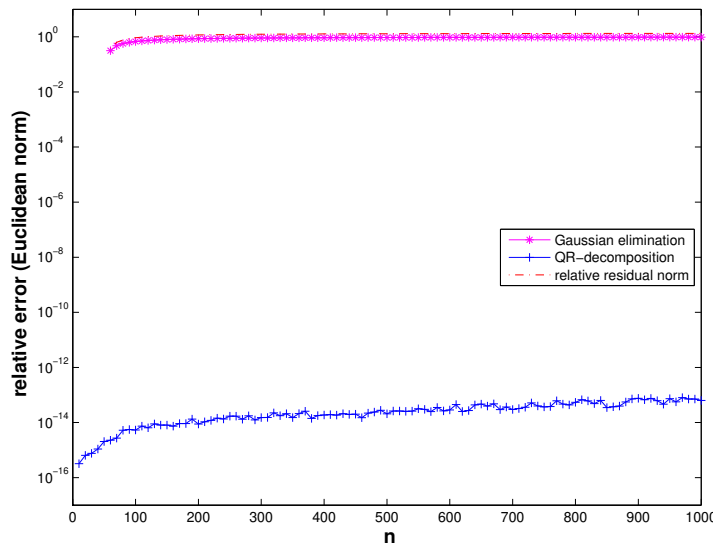


Abb. 7.1.21. Relativer Fehler von LU- und QR-Zerlegungen

Man sieht: Beim Lösen eines linearen Gleichungssystems mit der Wilkinson-Matrix liefert die Gauss-Elimination einen riesigen Rundungsfehler, während die QR-Zerlegung, die wir später im Detail anschauen, ziemlich genaue Ergebnisse liefert.

Ein ähnlicher Graph kann mittels des folgenden Codes erhalten werden:

²Allerdings passiert dies selten in der Praxis, so wie Wilkinson selber meint: *Anyone that unlucky has already been run over by a bus!* Das ist heute noch eine offene Frage: Warum zeigt sich die Gauss-Elimination in allen praktischen Fällen stabil?

Code 7.1.22: Erhaltung der Norm für QR- und LU-Zerlegung

```

1 from numpy import*
2 from numpy.linalg import norm
3 from scipy.linalg import lu, qr
4 from matplotlib.pyplot import figure, semilogy, show, legend, xlabel, ylabel

6 # first we define a function for producing the n × n wilkinson matrix
7 def wil_mat(n):
8     wilm = zeros((n,n))
9     for k in range(n):
10         for j in range(n):
11             if (j==k or j==n-1):
12                 wilm[k,j] = 1.0
13             if (j<k):
14                 wilm[k,j] = -1.0
15     return wilm

16 Ns = arange(5,1000,20)

18 err_lu = []
19 err_qr = []

22 for n in Ns:
23     matrix = wil_mat(n)
24     L, U = lu(matrix, permute_l=True) # P is a permutation matrix
25     Q, R = qr(matrix)
26     # now compute the relative errors. We divide by the norm of LU and QR
27     # because it turns out that in the case of LU it is much bigger than the norm of
28     # the wilkinson matrix
29     err_lu += [norm(dot(L,U)-matrix)/norm(dot(L,U))]
30     err_qr += [norm(dot(Q,R)-matrix)/norm(dot(Q,R))]

32 figure()
33 semilogy(Ns, err_lu, "-x", label = "LU")
34 semilogy(Ns, err_qr, "-*", label = "QR")
35 xlabel("$n$")
36 ylabel("relative error (Euclidean norm)")
37 legend(loc="best")
38 show()

```

Aufgrund der Instabilitäten (und der Rechenzeiten) ist die Berechnung der Inversen einer Matrix immer zu vermeiden; stattdessen sollten nur lineare Gleichungssysteme via LU-Zerlegung gelöst werden. Betrachten wir die für kleines ε sehr schlecht konditionierte Matrix

$$\mathbf{A} = \mathbf{u}\mathbf{v}^T + \varepsilon\mathbf{I}_n,$$

wobei die Einträge der Vektoren $\mathbf{u}$ und $\mathbf{v}$ als $u_k = k/3$ und $v_k = (-1)^k/k$ für $k = 1, 2, \dots, n$ definiert sind. Nehmen wir die exakte Lösung $\mathbf{x}^* = [1, \dots, 1]$ und bauen wir die rechte Seite $\mathbf{b} = \mathbf{A}\mathbf{x}^*$. Für kleines ε liefert die LU-Zerlegung riesige Fehler in der Lösung $\mathbf{x}$, jedoch bleibt das Residuum $\mathbf{A}\mathbf{x} - \mathbf{b}$ bei Maschinengenauigkeit. Dagegen ist das Residuum $\|\mathbf{A}\mathbf{y} - \mathbf{b}\|$ der Lösung mit der Inversen $\mathbf{y} = \mathbf{A}^{-1}\mathbf{b}$ riesig, wie man in Abbildung 7.1.23 für $n = 10$ sehen kann.

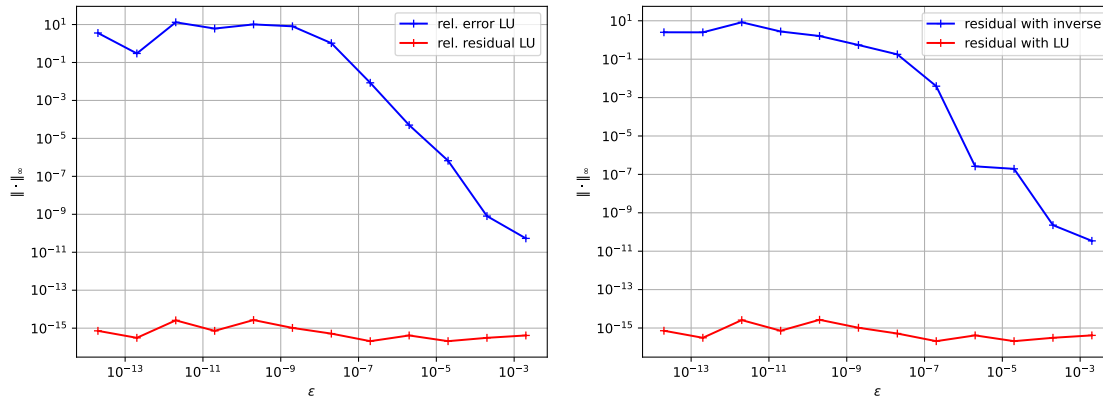


Abb. 7.1.23. Die Lösung mit der LU-Zerlegung für schlecht konditionierte Gleichungssysteme ist sehr ungenau, jedoch ist das Residuum klein; dagegen ist die Berechnung der Inversen nicht nur viel zu teuer, sondern auch nutzlos: Das Residuum der entsprechenden Lösung ist riesig.

1a) Cholesky-Zerlegung

Für eine (hermitesch-)symmetrische Matrix $\mathbf{A}$ vollen Rangs können wir die LU-Zerlegung so umschreiben, dass die Pivote in einer Diagonalmatrix $\mathbf{D}$ isoliert sind:

$$\mathbf{A} = \mathbf{LU} = \mathbf{LDL}^H = \mathbf{R}^H \mathbf{R} \quad \text{mit} \quad \mathbf{R} = \sqrt{\mathbf{D}} \mathbf{L}^H$$

und $\sqrt{\mathbf{D}}$ ist die Diagonalmatrix mit Einträgen $\sqrt{D_{ii}}$, $i = 1, 2, \dots, n$.

Cholesky zeigte: Für eine symmetrisch positiv definite Matrix ist keine Permutation in der Gauss-Elimination nötig, und der Algorithmus lässt sich mit der Hälfte der Anzahl an Operationen (im Vergleich zur LU-Zerlegung) durchführen.

Die entsprechenden Python-Funktionen befinden sich in `scipy.linalg` und heissen `lu`, `lu_factor`, `lu_solve`, `cholesky`, `cho_factor`, `cho_solve`.

`numpy.linalg.cholseky(A)` liefert den Faktor $\mathbf{L}$ ohne weitere Optionen.

`scipy.linalg.cholesky` bietet die Möglichkeit, $\mathbf{L}^T$ zurückzugeben, die Matrix (aus Effizienzgründen) zu überschreiben oder die Einträge nicht zu überprüfen, was für kürzere Laufzeiten nützlich sein kann.

2) QR-Zerlegung

Wir haben gesehen, dass die LU-Zerlegung grosse Rundungsfehler in die Ergebnisse einbringen kann.

Transformationen, die normerhaltend sind, lassen die Konditionszahl einer Matrix unverändert. In der Tat, sei $\mathbf{A} : \mathbb{E}^n \rightarrow \mathbb{E}^m$. Die Kondition der Matrix $\mathbf{A}$ ist $\text{cond}(\mathbf{A}) = \|\mathbf{A}^{-1}\| \cdot \|\mathbf{A}\|$. Seien $\mathbf{T} : \mathbb{E}^n \rightarrow \mathbb{E}^n$ und $\mathbf{S} : \mathbb{E}^m \rightarrow \mathbb{E}^m$ Basistransformationsmatrizen, welche die Norm erhalten.

Wir haben ein kommutierendes Diagramm:

$$\begin{array}{ccccc}
 \mathbf{x} \in \mathbb{E}^n & \xrightarrow{\mathbf{A}} & \mathbb{E}^m & \mathbf{y} & \text{alte Basis } \mathcal{B} \\
 \downarrow \mathbf{T}^{-1} \uparrow \mathbf{T} & & \mathbf{S}^{-1} \downarrow \uparrow \mathbf{S} & \downarrow & \\
 \tilde{\mathbf{x}} \in \mathbb{E}^n & \xrightarrow{\mathbf{B}} & \mathbb{E}^m & \tilde{\mathbf{y}} & \text{neue Basis } \mathcal{B};
 \end{array}$$

$\mathbf{T}$ und $\mathbf{S}^{-1}$ sind normerhaltend, dies bedeutet $\|\mathbf{T}\tilde{\mathbf{x}}\| = \|\tilde{\mathbf{x}}\|$ und $\|\mathbf{S}^{-1}\mathbf{y}\| = \|\mathbf{y}\|$. Daraus folgt:

$$\begin{aligned}
 \|\mathbf{B}\| &= \sup_{\|\tilde{\mathbf{x}}\|=1} \|\mathbf{B}\tilde{\mathbf{x}}\| = \sup_{\|\tilde{\mathbf{x}}\|=1} \|\mathbf{S}^{-1}\mathbf{A}\mathbf{T}\tilde{\mathbf{x}}\| = \\
 &= \sup_{\|\tilde{\mathbf{x}}\|=1} \|\mathbf{A} \underbrace{\mathbf{T}\tilde{\mathbf{x}}}_{\mathbf{x}}\| = \sup_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\| = \|\mathbf{A}\|,
 \end{aligned}$$

und analog für $\|\mathbf{A}^{-1}\|$, wobei $\mathbf{T}^{-1}$ und $\mathbf{S}$ normerhaltend sein sollen.

Wir können also schliessen:

Bemerkung 7.1.24. Unitäre/orthogonale Transformationen lassen die Konditionszahl einer Matrix unverändert.

Den Unterschied zwischen LU- und QR-Zerlegung in Bezug auf die Stabilität der Berechnungen kann man sehr gut bei der Gram-Schmidt-Orthogonalisierung beobachten. Der Gram-Schmidt-Algorithmus verwendet Dreiecksmatrizen, um eine orthogonale Basis zu finden. Dabei kann sich eine schlecht konditionierte Matrix so auswirken, dass das Resultat gar keine orthogonale Basis ist. Nun wollen wir anders vorgehen: Wie können wir eine Matrix $\mathbf{A}$ auf obere Rechtecksform $\mathbf{R}$ bringen, indem wir nur orthogonale/unitäre Transformationen verwenden?

Es gibt zwei Möglichkeiten: Spiegelungen und Drehungen.

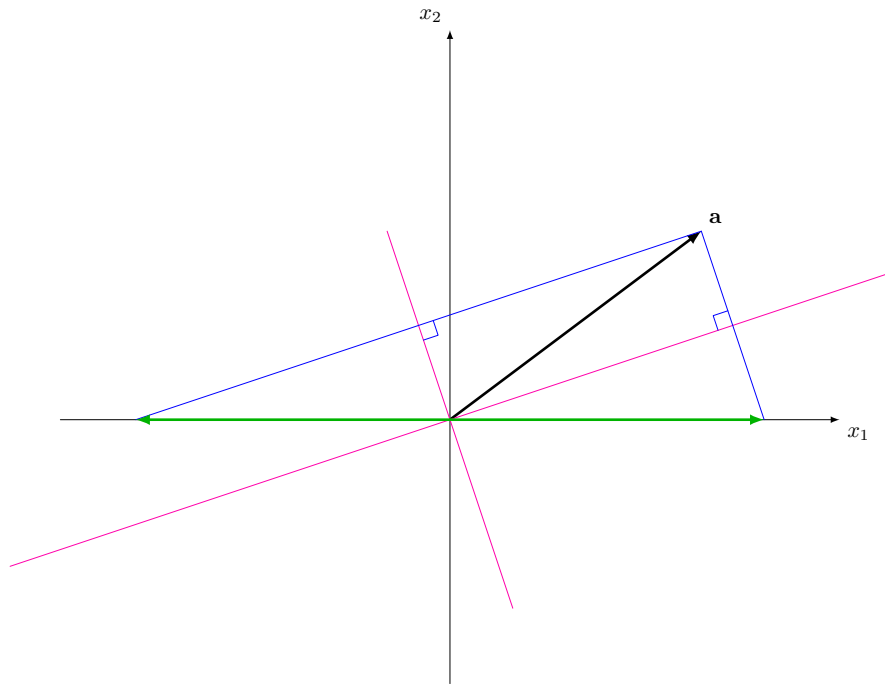


Abb. 7.1.25. Spiegelung von $\mathbf{a}$ um die Winkelhalbierende(n): zwei Möglichkeiten.

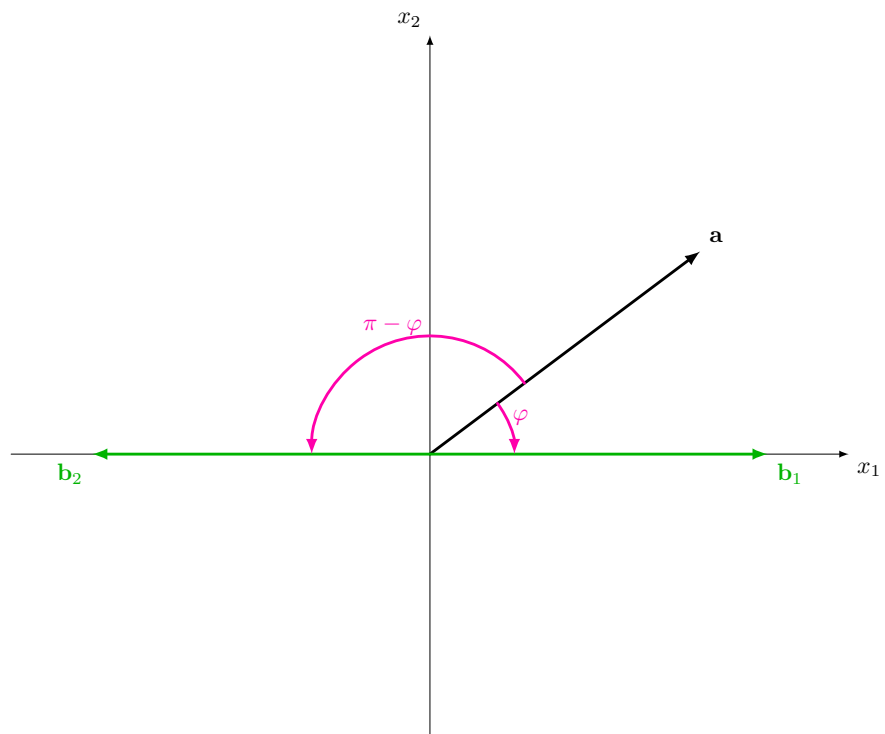


Abb. 7.1.26. Drehung: $\mathbf{a}$ wird über die x_1 -Achse gedreht: zwei Möglichkeiten.

Aufgabe: Gegeben $\mathbf{a} \in \mathbb{R}^n$, finde $\mathbf{Q} \in \mathbb{R}^{n \times n}$ orthogonal, so dass $\mathbf{Q}\mathbf{a} = \|\mathbf{a}\|_2 \mathbf{e}_1$ (wobei $\mathbf{e}_1 = [1 \ 0 \dots 0]^T$ der kanonische Einheitsvektor in $\mathbb{R}^n$ ist). Eine mögliche Lösungsmethode ist die Householder-Spiegelung.

Die Householder-Spiegelung verwendet die Matrix $\mathbf{Q} = \mathbf{H}(\mathbf{v}) := \mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^H}{\mathbf{v}^H\mathbf{v}}$ mit $\mathbf{v} = \frac{1}{2}(\mathbf{a} + \|\mathbf{a}\|_2 \mathbf{e}_1)$; normalerweise nimmt man:

$$\mathbf{v} = \begin{cases} \frac{1}{2}(\mathbf{a} + \|\mathbf{a}\|_2 \mathbf{e}_1), & \text{falls } a_1 > 0 \\ \frac{1}{2}(\mathbf{a} - \|\mathbf{a}\|_2 \mathbf{e}_1), & \text{falls } a_1 \leq 0. \end{cases}$$

Im Fall komplexer Vektoren, $a_1 = |a_1| e^{i\varphi}$ mit der Phase $\varphi \in [0, 2\pi]$ und

$$\mathbf{v} = \frac{1}{2}(\mathbf{a} + \|\mathbf{a}\|_2 \mathbf{e}_1 e^{-i\varphi}).$$

Householder-Schritt: Spiegelung von $\mathbf{a}$ auf die Ox -Achse

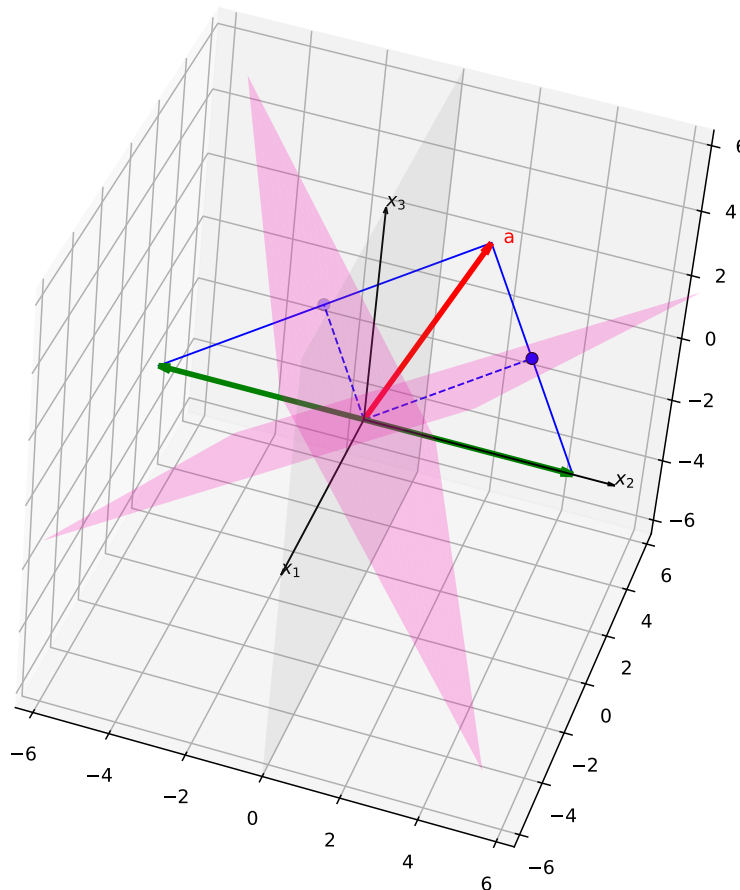


Abb. 7.1.27. Spiegelung von $\mathbf{a}$ um die Winkelhalbierenden (rosa Ebenen) von $\mathbf{a}$ und $\mathbf{e}_2$; es gibt zwei mögliche Ergebnisse auf Ox_2 : Man verwendet die stabilere Version (bezüglich Rundungsfehlern).

Eine Householder-Spiegelung produziert $\|\mathbf{a}\|_2 \mathbf{e}_1$ auf einen Schlag. Für dünn besetzte Matrizen kann es aber günstiger sein, die fehlenden Nullen in der Matrix schrittweise zu erzeugen. Dafür sind Givens-Rotationen geeignet:

$$\mathbf{G}_{1k}(a_1, a_k) := \begin{bmatrix} \bar{\gamma} & \cdots & \bar{\sigma} & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots \\ a_k & \cdots & \gamma & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots \\ a_n & \cdots & 0 & \cdots & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ \vdots \\ a_k \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} a_1^{(1)} \\ \vdots \\ \mathbf{0} \\ \vdots \\ a_n \end{bmatrix}, \quad \begin{aligned} \gamma &= \frac{a_1}{\sqrt{|a_1|^2 + |a_k|^2}}, \\ \sigma &= \frac{a_k}{\sqrt{|a_1|^2 + |a_k|^2}}. \end{aligned} \quad (7.1.2)$$

Nützlich dabei ist, dass die Multiplikation von links mit der Givens-Rotation $\mathbf{G}_{jk}(a_j, a_k)$ an die Matrix $\mathbf{B}$ nur die Elemente aus den Zeilen j und k modifiziert. Für jede Spalte sind so viele Givens-Rotationen nötig, wie es Einträge gibt, die nicht null sind.

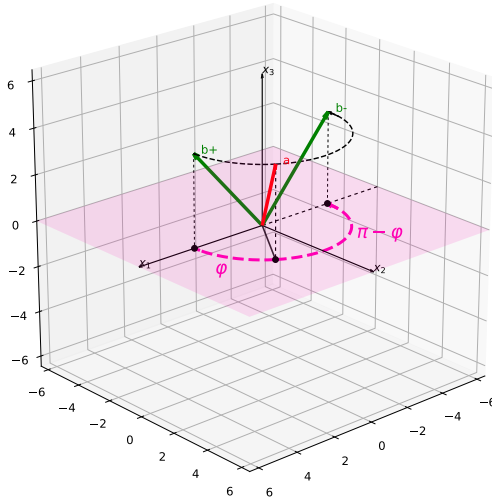
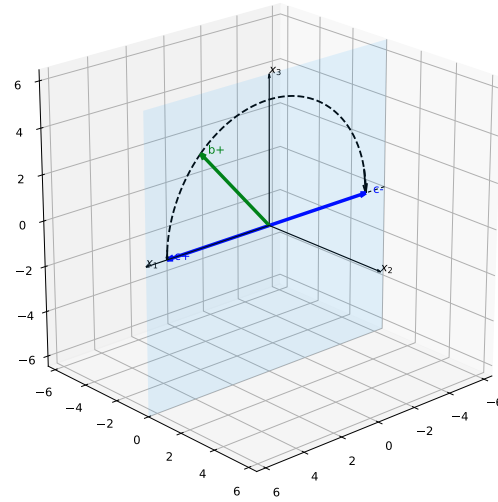
 1. Givens-Rotation um x_3 : x_2 wird eliminiert

 2. Givens-Rotation um x_2 : Ergebnis auf Ox_1


Abb. 7.1.28. Drehung: $\mathbf{a}$ wird in der Ox_1x_2 -Ebene gedreht: $\mathbf{a}$ landet in der Ebene $x_2 = 0$. Man entscheidet sich für das numerisch stabilere Zwischenergebnis (einer der zwei grünen Pfeile). Eine zweite Givens-Drehung rotiert in der Ox_1x_3 -Ebene zum Endergebnis auf der Ox_1 -Achse. Auch hier entscheidet man sich für das numerisch stabilere Ergebnis.

Ob mittels Spiegelungen oder Rotationen: Wir brauchen für jede Spalte unitäre Transformationen und $n - 1$ Schritte:

$$\mathbf{Q}_{n-1} \mathbf{Q}_{n-2} \cdots \mathbf{Q}_1 \mathbf{A} = \mathbf{R}$$

und somit $\mathbf{A} = \mathbf{QR}$ mit $\mathbf{Q} = \mathbf{Q}_1^H \cdots \mathbf{Q}_{n-1}^H$ unitär und $\mathbf{R}$ oberer Dreiecksmatrix.

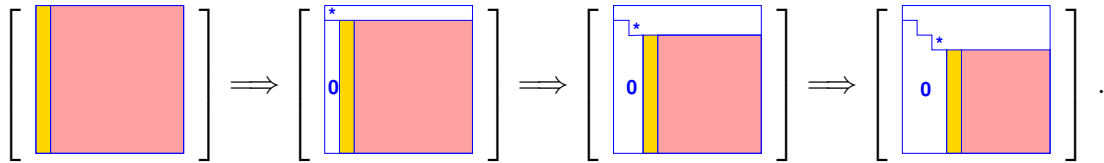


Abb. 7.1.29. Transformation zu einer *oberen Dreiecksmatrix* mittels unitärer Transformationen

Für allgemeine Matrizen geht es genauso:

$$m > n : \begin{bmatrix} \text{Q} \end{bmatrix} = \begin{bmatrix} \text{Q} \end{bmatrix} \begin{bmatrix} \text{R} \end{bmatrix}, \quad (7.1.3)$$

$$\mathbf{A} = \mathbf{QR}, \quad \mathbf{Q} \in \mathbb{K}^{m,n}, \quad \mathbf{R} \in \mathbb{K}^{n,n},$$

$$m < n : \begin{bmatrix} \text{A} \end{bmatrix} = \begin{bmatrix} \text{Q} \end{bmatrix} \begin{bmatrix} \text{R} \end{bmatrix}.$$

$$\mathbf{A} = \mathbf{QR}, \quad \mathbf{Q} \in \mathbb{K}^{m,m}, \quad \mathbf{R} \in \mathbb{K}^{m,n},$$

In Python kann man die folgenden Funktionen verwenden:

`Q, R = scipy.linalg.qr(A)` $\mathbf{Q} \in \mathbb{K}^{m,m}, \mathbf{R} \in \mathbb{K}^{m,n}$ für $\mathbf{A} \in \mathbb{K}^{m,n}$.

`Q, R = scipy.linalg.qr(A, mode='economic')` $\mathbf{Q} \in \mathbb{K}^{m,n}, \mathbf{R} \in \mathbb{K}^{n,n}$ für $\mathbf{A} \in \mathbb{K}^{m,n}, m > n$.
(*sparsame* QR-Zerlegung)

oder auch `numpy.linalg.qr(A, mode='reduced')`.

`scipy.linalg.qr(A, mode=..., pivoting=True)` gibt die Möglichkeit, auch eine Pivotwahl zu treffen, was für schlecht konditionierte Matrizen nützlich ist. Wenn die Spalten von $\mathbf{A}$ fast linear abhängig sind, ist die Pivottechnik numerisch stabiler und verrät auch den Rang der Matrix.

```

import numpy as np
from scipy.linalg import qr
rng = np.random.default_rng()
# Make A with nearly dependent columns
A = np.c_[np.ones(6), np.arange(6), 2*np.arange(6) + 1E-10*rng.normal(0,1,6)]

Q, R, piv = qr(A, mode='economic', pivoting=True)
# piv holds the column permutation: A[:,piv] = Q @ R
print(np.diag(R)) # infer the rank from the diag(R) magnitudes

```

Arbeitsaufwand für die Householder-QR-Zerlegung:

$$\begin{aligned}
 \mathbf{Q}, \mathbf{R} &= \text{qr}(\mathbf{A}) && \implies \text{kostet } O(m^2n) \text{ Operationen,} \\
 \mathbf{Q}, \mathbf{R} &= \text{qr}(\mathbf{A}, \text{mode='economic'}) && \implies \text{kostet } O(mn^2) \text{ Operationen.}
 \end{aligned}$$

Beispiel 7.1.30. (Komplexität der Householder-QR-Zerlegung) Wir können die Komplexität dieser Algorithmen mit einem Experiment überprüfen. Wir verwenden hier die $m \times n$ -Matrix A mit $m = n^2$:

$$A = \begin{bmatrix} 1 \\ \vdots \\ m \end{bmatrix} \begin{bmatrix} 1 & \dots & n \end{bmatrix} + \begin{bmatrix} I_n & 0 \\ 0 & \begin{bmatrix} 1 & \dots & 1 \\ 1 & \dots & 1 \end{bmatrix} \end{bmatrix}.$$

Code 7.1.31: Rechenzeit für QR-Zerlegungen

```

1  #!/usr/bin/env python3
2  """QR-Timing Benchmark (modern NumPy/SciPy style) + Vergleich NumPy vs SciPy in
   zweitem Bild
3
4  Dieses Skript erzeugt ZWEI Plots:
5
6  1) qrtiming_scipy.*
7     Vergleich verschiedener SciPy-QR-Modi:
8     - scipy.linalg.qr(mode='full')
9     - scipy.linalg.qr(mode='economic')
10    - scipy.linalg.qr(mode='economic', overwrite_a=True)
11    - scipy.linalg.qr(mode='r')
12
13  2) qrtiming_numpy_vs_scipy.*
14     Direkter Vergleich numpy.linalg.qr vs scipy.linalg.qr (analoge Modi):
15     - "full/complete":  SciPy full  vs NumPy complete
16     - "econ/reduced":   SciPy economic vs NumPy reduced
17     - "r-only":         SciPy r vs NumPy r
18
19  Design-Ziele:
   - A-Konstruktion via Broadcasting (outer) + in-place Korrektur.

```

```

21 - Saubere Zeitmessung (min ueber Wiederholungen).
    - Fairness: overwrite_a=True bekommt jedes Mal eine frische Kopie.
23
    Outputs:
25 - qrtiming_scipy.png/.pdf/.eps
    - qrtiming_numpy_vs_scipy.png/.pdf/.eps
27 """
29 import time
    from dataclasses import dataclass
31 from pathlib import Path
33
    import numpy as np
    import matplotlib.pyplot as plt
35 from scipy.linalg import qr as scipy_qr
37
    # =====
    # Konfiguration
39 # =====
    NR_EXP = 4
41 SIZES = np.arange(8, 65, 4) # 8,12,...,64
    OUT_SCIPY = Path("qrtiming_scipy")
43 OUT_NVSS = Path("qrtiming_numpy_vs_scipy")
45
    # =====
    # Hilfsfunktionen
47 # =====
49 def make_A(n):
    """Konstruiert A in  $\mathbb{R}^{m \times n}$  mit  $m=n^2$ .
51
    A = outer(1..m, 1..n) + B,
53 wobei B in den ersten n Zeilen  $I_n$  hat und darunter Einsen.
55
    Implementierung:
    - outer via Broadcasting
57 - +1 ueberall
    - Top-Block korrigieren: statt ones(n,n) muss  $I_n$ 
59 """
    m = n * n
61 r = np.arange(1, m + 1, dtype=np.float64)[: , None]
    c = np.arange(1, n + 1, dtype=np.float64)[None, :]
63
    A = r * c
65 A += 1.0
    A[:n, :] += np.eye(n, dtype=A.dtype) - 1.0
67 return A
69
    def time_min(func, repeats= NR_EXP):
    """Minimale Laufzeit ueber `repeats` Wiederholungen."""
71 ts = []
    for _ in range(repeats):
73 t0 = time.perf_counter()
        func()

```

```

75         ts.append(time.perf_counter() - t0)
76     return float(np.min(ts))
77
78 def save_figure(prefix):
79     for ext in ('.png', '.pdf', '.eps'):
80         plt.savefig(prefix.with_suffix(ext), dpi=200, bbox_inches='tight')
81
82 @dataclass
83 class ScipyResult:
84     sizes: np.ndarray
85     full: list[float]
86     econ: list[float]
87     econ_over: list[float]
88     r_only: list[float]
89
90 @dataclass
91 class NumpyVsScipyResult:
92     sizes: np.ndarray
93     scipy_full: list[float]
94     scipy_econ: list[float]
95     scipy_r: list[float]
96     numpy_complete: list[float]
97     numpy_reduced: list[float]
98     numpy_r: list[float]
99
100 # =====
101 # Benchmarks
102 # =====
103
104 def run_scipy_modes():
105     t_full: list[float] = []
106     t_econ: list[float] = []
107     t_over: list[float] = []
108     t_r: list[float] = []
109
110     for n in SIZES:
111         print(f"[SciPy modes] n={n}")
112         A0 = make_A(int(n))
113
114         def A_fortran_copy():
115             # SciPy/LAPACK profitiert oft von Fortran-Order
116             return np.array(A0, order='F', copy=True)
117
118         t_full.append(time_min(lambda: scipy_qr(A_fortran_copy(), mode='full',
119 check_finite=False)))
120         t_econ.append(time_min(lambda: scipy_qr(A_fortran_copy(),
121 mode='economic', check_finite=False)))
122
123         # overwrite a=True: immer frische Kopie
124         t_over.append(time_min(lambda: scipy_qr(A_fortran_copy(),
125 mode='economic', overwrite_a=True, check_finite=False)))
126
127         t_r.append(time_min(lambda: scipy_qr(A_fortran_copy(), mode='r',
128 check_finite=False)))

```

```

125         return ScipyResult(sizes=SIZES.copy(), full=t_full, econ=t_econ,
126                             econ_over=t_over, r_only=t_r)
127
128     def run_numpy_vs_scipy():
129         s_full: list[float] = []
130         s_econ: list[float] = []
131         s_r: list[float] = []
132
133         n_complete: list[float] = []
134         n_reduced: list[float] = []
135         n_r: list[float] = []
136
137         for n in SIZES:
138             print(f"[NumPy vs SciPy] n={n}")
139             AO = make_A(int(n))
140
141             def A_fortran_copy():
142                 return np.array(AO, order='F', copy=True)
143
144             def A_c_copy():
145                 # NumPy nutzt intern ebenfalls LAPACK; C-order reicht meist.
146                 return np.array(AO, order='C', copy=True)
147
148             # SciPy (analog zu NumPy-Modi)
149             s_full.append(time_min(lambda: scipy_qr(A_fortran_copy(), mode='full',
150 check_finite=False)))
151             s_econ.append(time_min(lambda: scipy_qr(A_fortran_copy(),
152 mode='economic', check_finite=False)))
153             s_r.append(time_min(lambda: scipy_qr(A_fortran_copy(), mode='r',
154 check_finite=False)))
155
156             # NumPy
157             n_complete.append(time_min(lambda: np.linalg.qr(A_c_copy(),
158 mode='complete')))
159             n_reduced.append(time_min(lambda: np.linalg.qr(A_c_copy(),
160 mode='reduced')))
161             n_r.append(time_min(lambda: np.linalg.qr(A_c_copy(), mode='r')))
162
163         return NumpyVsScipyResult(
164             sizes=SIZES.copy(),
165             scipy_full=s_full,
166             scipy_econ=s_econ,
167             scipy_r=s_r,
168             numpy_complete=n_complete,
169             numpy_reduced=n_reduced,
170             numpy_r=n_r,
171         )
172
173     # =====
174     # Plots
175     # =====
176
177     def plot_scipy_modes(res):

```



```

173     sizes = res.sizes
174     plt.figure(figsize=(7.5, 5.5))
175
176     plt.loglog(sizes, res.full, 's', label='SciPy qr(full)')
177     plt.loglog(sizes, res.econ, '*', label='SciPy qr(economic)')
178     plt.loglog(sizes, res.econ_over, '.', label='SciPy qr(economic, overwrite)')
179     plt.loglog(sizes, res.r_only, 'o', label='SciPy qr(r-only)')
180
181     # Referenzkurven (qualitativ)
182     if len(sizes) >= 2:
183         i0 = 1
184         v4 = res.econ[i0] * (sizes / sizes[i0]) ** 4
185         v6 = res.full[i0] * (sizes / sizes[i0]) ** 6
186         plt.loglog(sizes, v4, label=r'$O(n^4)$')
187         plt.loglog(sizes, v6 * 1e-1, '--', label=r'$O(n^6)$')
188
189     plt.legend(loc='best')
190     plt.xlabel(r'$n$')
191     plt.ylabel('time [s] (min over repeats)')
192     plt.grid(True, which='both', linestyle=':', linewidth=0.7)
193     plt.title('SciPy QR: Vergleich der Modi')
194
195     save_figure(OUT SCIPY)
196
197 def plot_numpy_vs_scipy(res):
198     sizes = res.sizes
199     plt.figure(figsize=(7.5, 5.5))
200
201     # Full/Complete
202     plt.loglog(sizes, res.scipy_full, 's', label='SciPy qr(full)')
203     plt.loglog(sizes, res.numpy_complete, 's--', label='NumPy qr(complete)')
204
205     # Economic/Reduced
206     plt.loglog(sizes, res.scipy_econ, '*', label='SciPy qr(economic)')
207     plt.loglog(sizes, res.numpy_reduced, '*--', label='NumPy qr(reduced)')
208
209     # R-only
210     plt.loglog(sizes, res.scipy_r, 'o', label='SciPy qr(r)')
211     plt.loglog(sizes, res.numpy_r, 'o--', label='NumPy qr(r)')
212
213     plt.legend(loc='best')
214     plt.xlabel(r'$n$')
215     plt.ylabel('time [s] (min over repeats)')
216     plt.grid(True, which='both', linestyle=':', linewidth=0.7)
217     plt.title('QR: NumPy vs SciPy (analoge Modi)')
218
219     save_figure(OUT_NVSS)
220
221     # =====
222     # Main
223     # =====
224
225 def main():
226     res_scipy = run_scipy_modes()

```

```

227     res_nvss = run_numpy_vs_scipy()

229     print("\n=== Summary SciPy modes (min times) ===")
    for n, tf, te, to, tr in zip(res_scipy.sizes, res_scipy.full,
    res_scipy.econ, res_scipy.econ_over, res_scipy.r_only):
231         print(f"n={int(n):>3d} | full={tf:9.4e}s | econ={te:9.4e}s |
    over={to:9.4e}s | r={tr:9.4e}s")

233     print("\n=== Summary NumPy vs SciPy (min times) ===")
    for n, sf, se, sr, nc, nr, nrr in zip(
235         res_nvss.sizes,
    res_nvss.scipy_full,
237         res_nvss.scipy_econ,
    res_nvss.scipy_r,
239         res_nvss.numpy_complete,
    res_nvss.numpy_reduced,
241         res_nvss.numpy_r,
    ):
243         print(
            f"n={int(n):>3d} | SciPy full={sf:9.4e}s, econ={se:9.4e}s,
245         r={sr:9.4e}s | "
            f"NumPy complete={nc:9.4e}s, reduced={nr:9.4e}s, r={nrr:9.4e}s"
        )

247     plot_scipy_modes(res_scipy)
249     plot_numpy_vs_scipy(res_nvss)

251     plt.show()

253 if __name__ == '__main__':
    main()
    
```

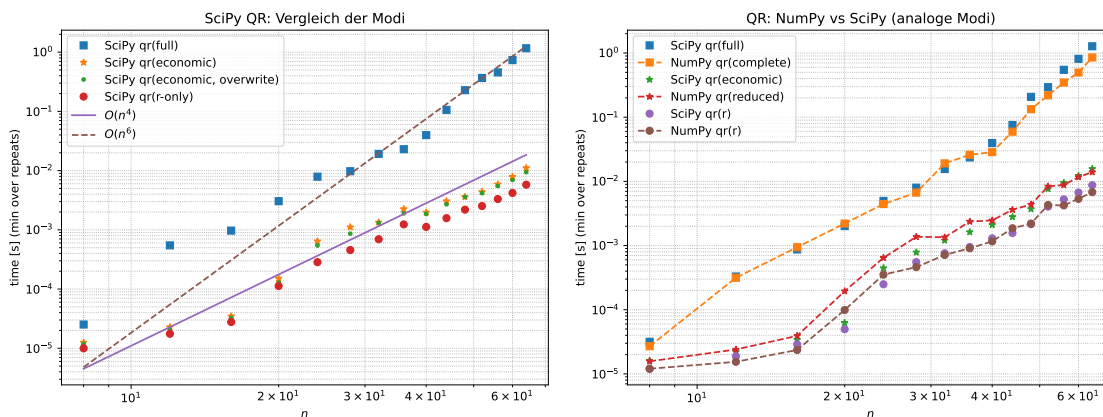
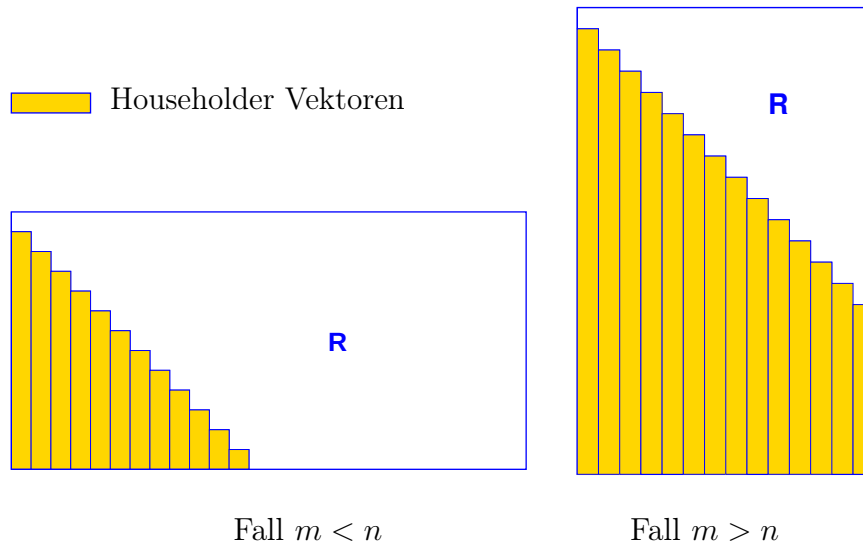


Abb. 7.1.32. Zeitaufwand verschiedener Varianten der QR-Zerlegung

Bemerkung 7.1.33. Wie können wir die verwendeten orthogonalen Transformationen speichern? Die Householder-Vektoren $\mathbf{v}_j$ werden in jedem Schritt kürzer, so dass wir diese im unteren Teil von $\mathbf{A}$ speichern können.



Man verwendet die folgende *Konvention* für die *Givens-Drehungen* ($\mathbb{K} = \mathbb{R}$):

$$\mathbf{G} = \begin{bmatrix} \bar{\gamma} & \bar{\sigma} \\ -\sigma & \gamma \end{bmatrix} \implies \text{speichere } \rho := \begin{cases} 1, & \text{if } \gamma = 0, \\ \frac{1}{2} \operatorname{sign}(\gamma)\sigma, & \text{if } |\sigma| < |\gamma|, \\ 2 \operatorname{sign}(\sigma)/\gamma, & \text{if } |\sigma| \geq |\gamma|. \end{cases}$$

$$\begin{cases} \rho = 1 & \implies \gamma = 0, \quad \sigma = 1 \\ |\rho| < 1 & \implies \sigma = 2\rho, \quad \gamma = \sqrt{1 - \sigma^2} \\ |\rho| > 1 & \implies \gamma = 2/\rho, \quad \sigma = \sqrt{1 - \gamma^2}. \end{cases}$$

Man speichert die Rotationsmatrix $\mathbf{G}_{ij}(ab)$ als (i, j, ρ) , was effizient ist, falls die Matrix $\mathbf{A}$ dünn besetzt ist.

Numerische Stabilität

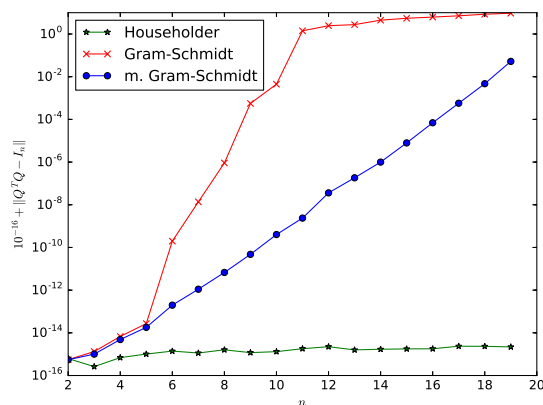


Abb. 7.1.34. Die Qualität verschiedener Algorithmen zur Orthogonalisierung: $\|\mathbf{I} - \mathbf{Q}^t \mathbf{Q}\|_2$ gegen Anzahl Spalten einer Vandermonde-Matrix.

Die Entwicklung mehrerer Methoden zur Lösung eines Problems hat meistens etwas mit der numerischen Stabilität der einzelnen Verfahren zu tun.

Die Abbildung 7.1.34 zeigt den Vergleich der Orthogonalität der Matrix $\mathbf{Q}$ für die QR-Zerlegung einer Vandermonde-Matrix ($m = 20$):

$$\mathbf{A} = \begin{bmatrix} t_0^{n-1} & \cdots & t_0^1 & 1 \\ t_1^{n-1} & \cdots & t_1^1 & 1 \\ \vdots & \vdots & \vdots & \vdots \\ t_{19}^{n-1} & \cdots & t_{19}^1 & 1 \end{bmatrix}, \text{ mit } t_i = \frac{i}{(m-1)}.$$

Bemerkung 7.1.35. Es gibt mehrere Möglichkeiten, die QR-Zerlegung einer Matrix A zu berechnen:

- 1) Der modifizierte Gram-Schmidt-Algorithmus orthogonalisiert A via trianguläre Operationen (obere Dreiecksmatrizen); Vorteil: Wir können früher aufhören und haben schon die ersten orthogonalen Spalten. Nachteil: Die Stabilität ist nicht garantiert.
- 2) Orthogonale Transformationen (Householder-Spiegelungen für eine vollbesetzte Matrix oder Givens-Rotationen für eine dünn besetzte Matrix) triangularisieren A ; Vorteil: $\mathbf{Q}$ ist trotz Rundungsfehlern orthogonal.

3) Singulärwertzerlegung

Beispiel 7.1.36. In der Hauptkomponentenanalyse (Englisch: principal component analysis, PCA) sind n Punkte (Messungen) $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^m$ in einem m -dimensionalen Raum (in dem Grössen gemessen worden sind) gegeben. Die entsprechenden Vektoren stellen wir in einer Matrix dar:

$$\mathbf{A} = [\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_n] \in \mathbb{R}^{m \times n}.$$

Ziel der PCA ist es, nur so viel Information aus den Messungen zu behalten wie notwendig; mathematisch bedeutet dies: Wenn $\mathbf{a}_1, \dots, \mathbf{a}_n$ linear abhängig sind, d.h., wenn sie in einem r -dimensionalen Raum ($r \leq \min\{m, n\}$) liegen, liefert die PCA die Zahl $r = \text{rank}(A)$ und eine orthonormale Basis von $\text{span}\{\mathbf{a}_1, \dots, \mathbf{a}_n\} = \text{Im } A$. Auf diese deterministische Art und Weise werden einige Messfehler eliminiert und alle Daten komprimiert.

Bemerkung 7.1.37. Die PCA ist eine Aufgabe der linearen Algebra; analog wie beim Lösen linearer Gleichungssysteme ist es wichtig, dass wir stabile (das heisst nicht rundungsfehleranfällige) Algorithmen zur Ermittlung von r und der Basis in A verwenden.

Theorem 7.1.38. Sei $\mathbf{A} \in \mathbb{C}^{m \times n}$ beliebig. Dann gibt es unitäre Matrizen $\mathbf{U} \in \mathbb{C}^{m \times m}$ und $\mathbf{V} \in \mathbb{C}^{n \times n}$ und die $m \times n$ -Diagonalmatrix in $\mathbf{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_p)$ mit $p = \min\{m, n\}$ und $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0$, so dass

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^H.$$

Definition 7.1.39. Diese Zerlegung heisst Singulärwertzerlegung (SVD = singular value decomposition) mit den Singulärwerten σ_i . Sie kann wie folgt veranschaulicht werden:

$$\begin{bmatrix} \boxed{\mathbf{A}} \end{bmatrix} = \begin{bmatrix} \boxed{\mathbf{U}} \end{bmatrix} \begin{bmatrix} \boxed{\Sigma} \end{bmatrix} \begin{bmatrix} \boxed{\mathbf{V}^H} \end{bmatrix}$$

$$\begin{bmatrix} \boxed{\mathbf{A}} \end{bmatrix} = \begin{bmatrix} \boxed{\mathbf{U}} \end{bmatrix} \begin{bmatrix} \boxed{\Sigma} \end{bmatrix} \begin{bmatrix} \boxed{\mathbf{V}^H} \end{bmatrix}$$

Bemerkung 7.1.40. Die Singulärwertzerlegung ermöglicht die Darstellung der (eventuell grossen) Matrix $\mathbf{A}$ als Summe von Matrizen vom Rang 1:

$$\mathbf{A} = \sum_{k=1}^p \sigma_k \mathbf{u}_k \mathbf{v}_k^H.$$

$\mathbf{u}_1, \dots, \mathbf{u}_p$ sind die ersten p -Zeilen von $\mathbf{U}$ (die linken Singulärvektoren von $\mathbf{A}$ genannt) und $\mathbf{v}_1, \dots, \mathbf{v}_p$ sind die ersten p -Spalten von $\mathbf{V}$ (auch die rechten Singulärvektoren von $\mathbf{A}$ genannt). Somit lässt sich die Information der $m \times n$ -Einträge aus $\mathbf{A}$ mittels $p + pn + pm = p(1 + n + m) \ll m \cdot n$ Einträgen darstellen. Die SVD wird somit zur Datenkompression verwendet (und war der technologische Schlüssel zum Erfolg von Google vor vielen Jahren).

Bemerkung 7.1.41. Die SVD ist nicht eindeutig, aber die Singulärwerte sind es.

Proof. Angenommen, $\mathbf{A}$ hat zwei Singulärwertzerlegungen:

$$\mathbf{A} = \mathbf{U}_1 \Sigma_1 \mathbf{V}_1^H = \mathbf{U}_2 \Sigma_2 \mathbf{V}_2^H.$$

Dann:

$$\mathbf{A} \mathbf{A}^H = \mathbf{U}_1 \Sigma_1 \mathbf{V}_1^H \mathbf{V}_1 \Sigma_1^H \mathbf{U}_1^H = \mathbf{U}_1 \Sigma_1 \Sigma_1^H \mathbf{U}_1^H$$

und gleichzeitig

$$\mathbf{A} \mathbf{A}^H = \mathbf{U}_2 \Sigma_2 \mathbf{V}_2^H \mathbf{V}_2 \Sigma_2^H \mathbf{U}_2^H = \mathbf{U}_2 \Sigma_2 \Sigma_2^H \mathbf{U}_2^H.$$

Somit sind die Diagonalmatrizen

$$\Sigma_1 \Sigma_1^H = \text{diag}(s_{1,1}^2, \dots, s_{1,p}^2)$$

und

$$\Sigma_2 \Sigma_2^H = \text{diag}(s_{2,1}^2, \dots, s_{2,p}^2),$$

ähnlich via orthogonaler Transformationen und also gleich. $\square$

Bemerkung 7.1.42. $\sigma_1^2 \geq \dots \geq \sigma_r^2 > 0$ sind Eigenwerte von $\mathbf{A}^H \mathbf{A}$, $\mathbf{A} \mathbf{A}^H$ und

$$\begin{bmatrix} 0 & \mathbf{A}^H \\ \mathbf{A} & 0 \end{bmatrix} \text{ zu den Eigenvektoren } \mathbf{v}_1, \dots, \mathbf{v}_r; \mathbf{u}_1, \dots, \mathbf{u}_p; \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{v}_1 \end{bmatrix}, \dots, \begin{bmatrix} \mathbf{u}_p \\ \mathbf{v}_p \end{bmatrix}.$$

Proof. $\mathbf{A} \mathbf{A}^H = \mathbf{U} \Sigma \mathbf{V}^H \mathbf{V} \Sigma^H \mathbf{U}^H = \mathbf{U} \Sigma \Sigma^H \mathbf{U}^H$. Daher sind die Eigenwerte von $\Sigma \Sigma^H$ auch Eigenwerte von $\mathbf{A} \mathbf{A}^H$. $\square$

Die entsprechende Python-Funktion ist: `scipy.linalg.svd`; sie hat Optionen zur Überschreibung der Matrix und für den Verzicht auf die Überprüfung der Einträge, was den Code schneller machen kann. `numpy.linalg.svd(A, full_matrices=True, compute_uv=True)` kann das nicht.

`full_matrices=False`: sparsame SVD:

$$\begin{array}{c} m \\ \boxed{\mathbf{U}^T} \\ r \end{array} \begin{array}{c} n \\ \boxed{\mathbf{A}} \\ m \end{array} \begin{array}{c} r \\ \boxed{\mathbf{V}} \\ n \end{array} = \begin{array}{c} r \\ \boxed{\begin{array}{cc} \sigma_1 & 0 \\ & \sigma_2 \\ & \ddots \\ 0 & \sigma_r \end{array}} \\ r \end{array}$$

Wenn nur die Singulärwerte und nicht auch die Faktoren $\mathbf{U}, \mathbf{V}^H$ gebraucht werden, dann ist es günstiger, `scipy.linalg.svdvals` zu verwenden.

Komplexität:

$$\begin{array}{ll} 2mn^2 + 2n^3 + O(n^2) + O(mn) & \text{für } \mathbf{s} = \text{svdvals}(\mathbf{A}), \\ 4m^2n + 22n^3 + O(mn) + O(n^2) & \text{für } \mathbf{U}, \mathbf{S}, \mathbf{V} = \text{svd}(\mathbf{A}), \\ O(mn^2) + O(n^3) & \text{für } \mathbf{U}, \mathbf{S}, \mathbf{V} = \text{svd}(\mathbf{A}, \text{full_matrices=False}), m \gg n. \end{array}$$

Theorem 7.1.43. Wenn die Singulärwerte von $\mathbf{A}$ erfüllen:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > \sigma_{r+1} = \dots = \sigma_p = 0$$

dann gilt:

$$\begin{aligned} \text{rank } \mathbf{A} &= r, \ker \mathbf{A} = \text{span}\{\mathbf{v}_{r+1}, \mathbf{v}_{r+2}, \dots, \mathbf{v}_n\}, \\ \text{Im } \mathbf{A} &= \text{span}\{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_r\}. \end{aligned}$$

$$\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^H$$

$$\begin{cases} \mathbf{A} \mathbf{v}_j = \sigma_j \mathbf{u}_j & \text{für } j \leq r \\ \mathbf{A} \mathbf{v}_j = 0 & \text{für } j > r \end{cases}$$

und

$$\begin{cases} \mathbf{A}^T u_j = \sigma_j v_j & \text{für } j \leq r \\ \mathbf{A}^T u_j = 0 & \text{für } j > r \end{cases}$$

Bemerkung 7.1.44. Wenn Messfehler oder Rundungsfehler auftreten, dann hat man typischerweise

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \gg \sigma_{r+1} \approx \dots \approx \sigma_p \approx 0.$$

Der *numerische* Rang r ist hier gegeben durch den Sprung

$$\sigma_r \gg \sigma_{r+1} \approx 0$$

und ist in den Anwendungen relevant.

Beispiel 7.1.45. Wir betrachten zwei funktionale Abhängigkeiten, z.B. zwei an $m = 50$ verschiedenen Orten gemessene Signale. Hier haben die zwei Signale die Form $\sin(\pi t)$ und $\cos(\pi t)$ für $t = \frac{1}{50}, \frac{2}{50}, \dots, \frac{50}{50}$. Die Messfehler simulieren wir durch m Zufallszahlen zwischen 0 und 1 (alle Zahlen zwischen 0 und 1 haben die gleiche Wahrscheinlichkeit zu erscheinen), die wir entweder mit dem Signal multiplizieren (für $\sin(\pi t)$) oder ihm mit dem Dämpfungsfaktor 0.1 addieren (für $\cos(\pi t)$).

Wir wiederholen das “Experiment” $n = 10$ -mal.

```

1 from numpy import linspace, sin, cos, pi, zeros
2 from numpy.linalg import norm
3 from numpy.random import rand
4 from matplotlib.pyplot import figure, show, plot, legend, xlabel, ylabel

6 n = 10; m = 50
7 r = linspace(1,m,m)
8 x = sin(pi*r/m)
9 y = cos(pi*r/m)
10 A = zeros((2*n,m))
11 for k in range(n):
12     A[2*k,:] = x*rand(m)
13     A[2*k+1,:] = y + 0.1*rand(m)
14
15 figure()
16 plot(r,A[0,:],"x", label="Measurement 1")
17 plot(r,A[1,:],"o", label="Measurement 2")
18 plot(r,A[2,:],"*", label="Measurement 3")
19 plot(r,A[3,:],"v", label="Measurement 4")
20 legend(loc="best")
    show()
```

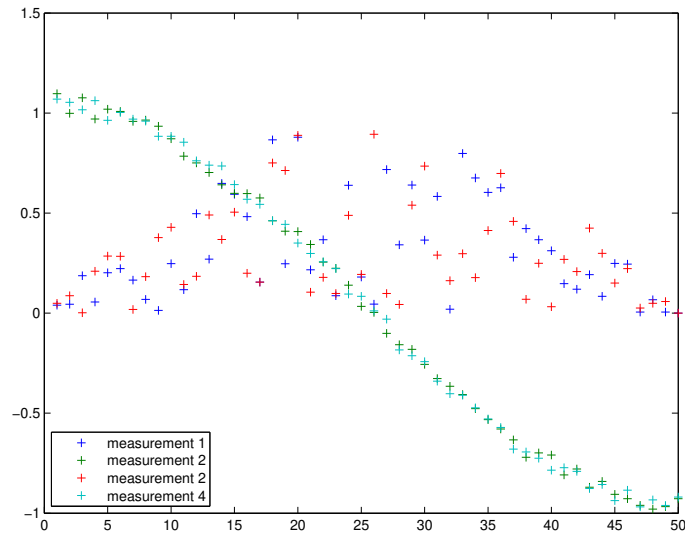


Abb. 7.1.46. Einige rohe Daten aus dem Experiment

Nun haben wir die Ergebnisse von 10 Experimenten, d.h. 20 Vektoren der Länge 50. Mittels PCA wollen wir folgendermassen verfahren:

- 1) Wie viele Abhängigkeiten von t existieren? In der Sprache der linearen Algebra: Wie viele dieser 20 Vektoren sind linear unabhängig?
- 2) Welches sind die wichtigen Komponenten in den Daten und wie wichtig sind sie überhaupt?

Dazu benutzen wir den folgenden Code:


```

from numpy.linalg import svd
2
U, S, Vh = svd(A)
4
figure()
6 I = argsort(1/S) #order the singular values from bigger to smaller
plot(arange(len(S)),S[I],"o")
8 xlabel("Number of singular value"); ylabel("Singular value");
title("Singular values")
show()
10
#now we want to study the principal components (stored in V)
12 figure()
plot(r,-Vh[0,:],"o", label="First principal component")
14 plot(r,y/norm(y),"+", label="First model vector")
plot(r,Vh[1,:],"o", label="Second principal component")
16 plot(r,x/norm(x),"+", label="Second model vector")
legend(loc="best")
18 xlabel("Number of measurement"); ylabel("Principal Component");
title("Principal Components")
show()
20
from numpy.linalg import norm
22
#now we find the contributions of the principal vectors. compare with
formula  $A = \sum \sigma_k u_k v_k^H$ 
24 first_vec_contrib = abs(U[:,0])
second_vec_contrib = abs(U[:,1])
26
figure()
28 plot(arange(2*n),first_vec_contrib,"+",label="First principal component")
plot(arange(2*n),second_vec_contrib,"o",label="Second principal component")
30 xlabel("Measurement"); ylabel("Contributions of principal components")
legend(loc="best")
32 show()

```

Die Singulärwertzerlegung der Matrix A ergibt zwei dominierende Singulärwerte. Die Messungen zeigen sich linear abhängig mit zwei Hauptkomponenten $\mathbf{u}_1$ und $\mathbf{u}_2$.

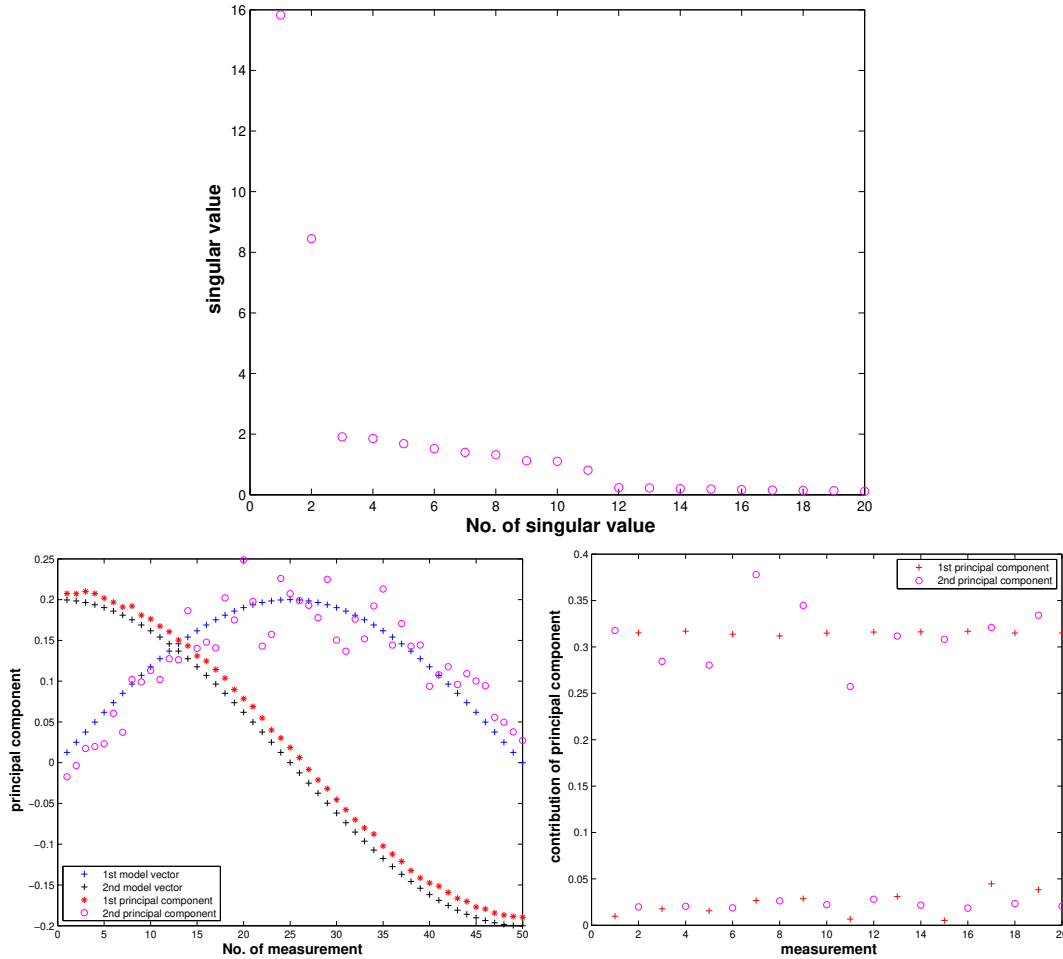


Abb. 7.1.47. Ergebnisse der PCA mittels SVD für die obigen rohen Daten

Bemerkung 7.1.48. Es gibt spezialisierte Module für PCA, z.B. `sklearn.decomposition.PCA` oder den Modular Toolkit für Data Processing; die bessere Vorgehensweise ist jedoch, keine spezialisierten Module nur deswegen zu importieren, sondern zu wissen, was man tut, und dies effizient mit `scipy` zu implementieren.

Bemerkung 7.1.49. Gegeben $A \in \mathbb{C}^{m \times n}$, $m > n$, finde $x \in \mathbb{C}^n$, $\|x\|_2 = 1$, so dass $\|Ax\|_2$ minimal ist. Die SVD hilft, denn unitäre Matrizen erhalten die 2-Norm:

$$\begin{aligned} \min_{\|x\|_2=1} \|Ax\|_2^2 &= \min_{\|x\|_2=1} \|U \Sigma V_x^H\|_2^2 = \min_{\|V_x^H\|_2=1} = \\ &= \min_{\|y\|_2=1} \|\Sigma y\|_2^2 = \min_{\|y\|_2=1} (\sigma_1^2 y_1^2 + \cdots + \sigma_n^2 y_n^2) \geq \sigma_n^2, \end{aligned}$$

denn das Minimum ist genau für $y = (0, \dots, 0, 1)^T$ erreicht und somit $x = v_n$.

Analog kann man zeigen:

$$\sigma_1 = \max_{\|x\|_2=1} \|Ax\|_2 \quad \text{und} \quad v_1 = \arg \max_{\|x\|_2=1} \|Ax\|_2.$$

Somit haben wir

Theorem 7.1.50. Sei $\mathbf{A} \in \mathbb{C}^{m \times n}$. Dann: $\|\mathbf{A}\|_2 = \sigma_1(\mathbf{A})$. Falls $\mathbf{A}$ invertierbar ist, dann

$$\text{cond}_2(\mathbf{A}) = \frac{\sigma_1}{\sigma_m}.$$

Bemerkung 7.1.51. Die Python-Funktionen $\text{norm}(A)$ und $\text{cond}(A)$ verwenden $\text{svd}(A)$.

Definition 7.1.52. Die Frobenius-Norm der $m \times n$ -Matrix $\mathbf{A}$ ist

$$\|\mathbf{A}\|_F^2 := \sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2,$$

Es ist klar, dass die Frobenius-Norm invariant unter unitären/orthogonalen Transformationen ist. Es ist auch einfach zu sehen, dass

$$\|\mathbf{A}\|_F^2 = \sum_{j=1}^p \sigma_j^2. \quad (7.1.4)$$

Theorem 7.1.53. [Bestapproximation durch Niedrigrangmatrizen]

Für die $m \times n$ -Matrix $\mathbf{A}$ mit Rang r gelte die Singulärwertzerlegung $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^H$, mit $m \geq n$ und $\mathbf{U} = [\mathbf{u}_1 \mathbf{u}_2 \dots \mathbf{u}_m]$ und $\mathbf{V} = [\mathbf{v}_1 \mathbf{v}_2 \dots \mathbf{v}_n]$. Für $k \in \{1, 2, \dots, r\}$ sei die $m \times k$ -Matrix $\mathbf{U}_k = [\mathbf{u}_1 \mathbf{u}_2 \dots \mathbf{u}_k]$, die $n \times k$ -Matrix $\mathbf{V}_k = [\mathbf{v}_1 \mathbf{v}_2 \dots \mathbf{v}_k]$ und die $k \times k$ -Matrix $\Sigma_k := \text{diag}(\sigma_1, \dots, \sigma_k)$. Für $\|\cdot\| = \|\cdot\|_F$ und $\|\cdot\| = \|\cdot\|_2$ gilt dann:

$$\|\mathbf{A} - \mathbf{U}_k \Sigma_k \mathbf{V}_k^H\| \leq \|\mathbf{A} - \mathbf{B}\| \quad \text{für alle } m \times n\text{-Matrizen } \mathbf{B} \text{ vom Rang } k.$$

Proof. Sei $\mathbf{A}_k = \mathbf{U}_k \Sigma_k \mathbf{V}_k^H$, das offensichtlich Rang k hat. Dann

$$\|\mathbf{A} - \mathbf{A}_k\| = \|\Sigma - \mathbf{U}^H \mathbf{U}_k \Sigma_k \mathbf{V}_k^H \mathbf{V}\| = \begin{cases} \sigma_{k+1} & , \text{ for } \|\cdot\| = \|\cdot\|_2, \\ \sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2} & , \text{ for } \|\cdot\| = \|\cdot\|_F. \end{cases}$$

Für die euklidische Norm $\|\cdot\|_2$:

Sei die $m \times n$ -Matrix $\mathbf{B}$ vom Rang k , also hat $\text{Ker } \mathbf{B}$ Dimension $n - k$ und enthält somit in der Menge $\text{Ker } \mathbf{B} \cap \text{Span}\{\mathbf{v}_1, \dots, \mathbf{v}_{k+1}\}$ mindestens einen Vektor $\mathbf{x}$ (der Länge n) mit $\|\mathbf{x}\|_2 = 1$. Dann:

$$\|\mathbf{A} - \mathbf{B}\|_2^2 \geq \|(\mathbf{A} - \mathbf{B})\mathbf{x}\|_2^2 = \|\mathbf{A}\mathbf{x}\|_2^2 = \left\| \sum_{j=1}^{k+1} \sigma_j (\mathbf{v}_j^H \mathbf{x}) \mathbf{u}_j \right\|_2^2 = \sum_{j=1}^{k+1} \sigma_j^2 (\mathbf{v}_j^H \mathbf{x})^2 \geq \sigma_{j+1}^2,$$

$$\text{da } \sum_{j=1}^{k+1} (\mathbf{v}_j^H \mathbf{x})^2 = 1.$$

Für die Frobenius-Norm $\|\cdot\|_F$: Sei $\{\mathbf{z}_1, \dots, \mathbf{z}_{n-k}\}$ eine Orthonormalbasis in $\text{Ker } \mathbf{B}$. Wir setzen die $n \times n - k$ -Matrix $\mathbf{Z} = [\mathbf{z}_1 \dots \mathbf{z}_{n-k}]$ zusammen. Dann:

$$\|\mathbf{A} - \mathbf{B}\|_F^2 \geq \|(\mathbf{A} - \mathbf{B})\mathbf{Z}\|_F^2 = \|\mathbf{A}\mathbf{Z}\|_F^2 = \sum_{i=1}^{n-k} \|\mathbf{A}\mathbf{z}_i\|_2^2 = \sum_{i=1}^{n-k} \sum_{j=1}^r \sigma_j^2 (\mathbf{v}_j^H \mathbf{z}_i)^2. \quad \square$$

□

Bemerkung 7.1.54. Die Information, die in einer $m \times n$ -Matrix vom Rang k steckt, kann durch $k(m+n)$ Zahlen dargestellt werden. Die Approximation mittels Matrizen niedrigen Rangs dient zur Datenkomprimierung.

7.1.3 Ausgewählte Funktionen aus numpy, scipy

numpy -> BLAS, LAPACK

dot

vdot: complex scalar product, Frobenius product of two 2D-Arrays

inner

matmult or @

trace

diag

diagonal

sum

transpose

outer

kron

```
# A, B = np.array([[1,2],[3,4]]), np.array([[10,11],[12,13]])
```

```
# print( np.kron(A,B) )
```

```
# print( np.outer(A,B) )
```

```
# np.einsum('ij,kl->ikjl',A,B).reshape(4,4)
```

tensordot

```
# import numpy as np
```

```
# a = np.arange(24).reshape(2,3,4)
```

```
# b = np.arange(30).reshape(3,5,2)
```

```
# print(a)
```

```
# print(b)
```

```
# c = np.tensordot(a, b, axes=([0,1],[2,0]))
```

```
# print(c)
```

```
# #same:
```

```
# c = np.zeros((4,5))
```

```
# for i in range(4):
```

```
#     for j in range(5):
```

```
#         for p in range(2):
```

```
#             for q in range(3):
```

```
#                 c[i,j] += a[p,q,i] * b[q,j,p]
```

```
# print(c)
```

numpy.linalg.

```

cholesky
qr(a, mode='reduced') / 'complete', 'r', 'raw'
svd(a, full_matrices=True, compute_uv=True, hermitian=False)
svdvals(x) is same as np.svd(x, compute_uv=False, hermitian=False)

compare to scipy.linalg.svd and svdvals
svd(a, full_matrices=True, compute_uv=True, overwrite_a=False, check_finite=True, lapack_driver='gelsd')
svdvals(a, overwrite_a=False, check_finite=True)

eig
eigh
eigvals # just a simpler call to eig, EV are simply not returned
eigvalsh
schur is better suited for non-Hermitian normal matrices than eig [stability]
norm
cond # not clear which algorithm is used!
det # LU from LAPACK
matrix_rank(A, tol=None, hermitian=False, *, rtol=None)

solve # matrix must be full rank
lstsq(a, b, rcond=None)
inv # algorithm not mentioned
pinv(a, rcond=None, hermitian=False, *, rtol=<no value>) # via svd

scipy.linalg
-----

numpy.linalg for more linear algebra functions. Note that although scipy.linalg imports
numpy.linalg, it does not use it.

inv # LUP-decomposition LAPACK

solve(a, b, lower=False, overwrite_a=False, overwrite_b=False, check_finite=True, assume_a='gen')

pinv, pinvh

det # via LAPACK LU
norm # Differs from numpy only in non-finite handling and the use of blas

lu(a, permute_l=False, overwrite_a=False, check_finite=True, p_indices=False) # see help

lu_factor # pivots as indices, LU in a single array, see example in help

ldl, cholesky, cholesky_banded, cho_factor, cho_solve, cho_solve_banded, polar

qr(a, overwrite_a=False, lwork=None, mode='full', pivoting=False, check_finite=True) #

```

```

qr_multiply(a, c, mode='right', pivoting=False, conjugate=False, overwrite_a=False, ov

schur

svd

lstsq(a, b, cond=None, overwrite_a=False, overwrite_b=False, check_finite=True, lapack

eig(a, b=None, left=False, right=True, overwrite_a=False, overwrite_b=False, check_fin

eigvals # just a simpler call to eig, EV are simply not returned

eig_banded # LAPACK
eigh_tridiagonal(d, e, eigvals_only=False, select='a', select_range=None, check_finite

expm # plain algorithm
logm, cosm, etc. use expm

# fun:
orth, null_space, bandwidth, issymmetric, ishermitian, subspace_angles
+ a lot of special matrices

# advanced:
solve_banded, solve_circulant, solve_triangular, solve_toeplitz
qr_update(Q, R, u, v, overwrite_qruv=False, check_finite=True)
qr_delete, qr_insert
rq, qz, ordqz, rsf2csf, hessenberg, cdf2rdf, cossin

# nice to know:
Guidelines for importing functions from SciPy
Parallel execution support in SciPy
Thread Safety in SciPy
# style with scientific python:
https://numpydoc.readthedocs.io/en/latest/format.html

```

Chapter 8

Ausgleichsrechnung

In diesem Abschnitt beschäftigen wir uns mit dem Problem der optimalen Anpassung einer gegebenen Funktion an einen Datensatz. Um dieses Thema einzuführen, wollen wir zunächst mit zwei Beispielen aus der Physik beginnen.

Nichtlineare Ausgleichsrechnung

In einem Versuch aus dem Anfängerpraktikum für Physiker soll die Wärmeleitfähigkeit von Nickel bestimmt werden. Die Messung erfolgt mithilfe einer Kompensationsschaltung. Hierbei wirkt ein Stück des Nickelstabes als wärmevariabler Widerstand, der kompensiert werden soll. Das Experiment wird einmal beim Heizvorgang und einmal beim Abkühlvorgang durchgeführt. Die gemessenen Werte sind in der folgenden Tabelle aufgelistet (links: Abkühlvorgang, rechts: Heizvorgang).

<i>Zeit</i> [min]	<i>Stromstärke</i> [mA]	
0	9.66	24.34
5	18.8	18.93
10	22.36	17.09
15	24.07	16.27
20	24.59	15.97
25	24.91	15.91

Temperaturabhängige Stromstärke in Nickel

Es ist bekannt, dass die Korrelation durch eine Exponentialfunktion beschrieben wird: $I(t) = \alpha \pm \beta e^{(-\gamma t)}$. Das Ziel ist es, mithilfe dieser Funktion und der gemessenen Werte die thermische Leitfähigkeit des Nickels zu bestimmen.

Lineare Ausgleichsrechnung

Es soll die Temperatur des absoluten Nullpunktes ohne übermässigen Aufwand bestimmt werden. Um dies zu bewerkstelligen, wird ein Gaskolben mit Helium gefüllt und bei 20°C auf den Luftdruck normiert. Danach misst man die Druckdifferenz am Gefrierpunkt des Wassers und am Siedepunkt von flüssigem Stickstoff.

Unter Verwendung des Gesetzes von Gay-Lussac soll der absolute Nullpunkt abgeschätzt werden. Die einzelnen Messwerte (kochendes Wasser, Gefrierpunkt von Wasser und kochender Stickstoff) sind in der folgenden Tabelle aufgelistet.

<i>Temp</i> [C]	<i>Druck</i> [mmHg]
98.269	852.7
0	624.5
-194.96	172.7

Temperatur-Druck-Relation für ein ideales Gas.

Sei ein mathematisches Modell $y = \Psi(\mathbf{a}, x)$ mit n Parametern $\mathbf{a} = (a_1, a_2, \dots, a_n)$ gegeben. Für gegebene Werte x_i kann man $\Psi(\mathbf{a}, x_i)$ ausrechnen und auch entsprechende physikalische Eigenschaften y_i messen. Das Ziel der Ausgleichsrechnung ist es, einen Wert für den Parameter a zu bestimmen, so dass der Fehler

$$\sum_{i=1}^m \|\Psi(\mathbf{a}, x_i) - y_i\|^2$$

minimiert wird.

Wenn die Funktion Ψ linear vom Parameter $\mathbf{a}$ abhängt, dann haben wir mit *linearer Ausgleichsrechnung* zu tun.

8.1 Lineare Ausgleichsrechnung

Beispiel 8.1.1. (Lineare Ausgleichsrechnung)

Seien $y_i \in \mathbb{R}$ die zu entsprechenden $\mathbf{x}_i \in \mathbb{R}^n$ ($i = 1, 2, \dots, m$) gemessenen Werte. In x_i und y_i befinden sich Messfehler, die im allgemeinsten Fall keine besonderen Bedingungen erfüllen müssen. Sei das zugrunde liegende Modell eine affine Beziehung

$$y = \mathbf{a}^T \mathbf{x} + c,$$

wobei $c \in \mathbb{R}$ und $\mathbf{a} = (a_1, a_2, \dots, a_n)$ die Modellparameter sind, die wir mithilfe des Experiments abschätzen möchten.

Typischerweise führt man mehr Messungen durch, als es Parameter gibt, also $m \geq n + 1$. Da die Messungen fehlerbehaftet sind, können die gemessenen Werte y_i nicht in einer Ebene in $\mathbb{R}^n$ liegen. Wir können aber eine solche Ebene konstruieren, so dass die Summe der Abstände von den gemessenen Werten zu dieser Ebene minimal ist. Diese Methode heisst *Methode der kleinsten Quadrate*:

$$(\mathbf{a}, c) = \underset{\mathbf{p} \in \mathbb{R}^n, q \in \mathbb{R}}{\operatorname{argmin}} \sum_{i=1}^m |y_i - \mathbf{p}^T \mathbf{x}_i - q|^2 \quad (8.1.1)$$

Die Abbildung 8.1.2 zeigt eine solche lineare Regression in der Dimension $n = 1$ mit $m = 8$ Messungen.

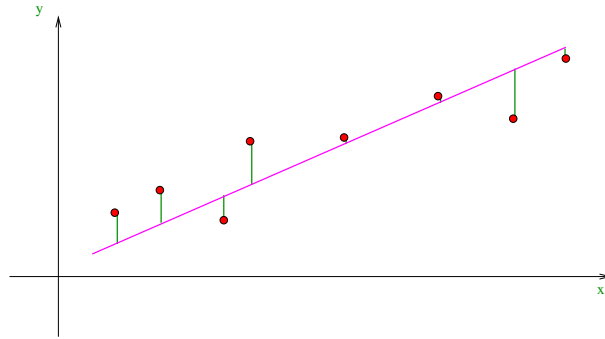


Abb. 8.1.2. Lineare Regression in der Dimension $n = 1$ mit $m = 8$ Messungen

Beispiel 8.1.3. (Daten-Fit)

Die affine Relation aus dem vorigen Beispiel kann zu einer Basis-Darstellung verallgemeinert werden. Nehmen wir an, wir haben m “Knoten” $(t_i, y_i) \in I \times \mathbb{R} \subset \mathbb{R}^2$ und n “Basisfunktionen” $b_j : I \mapsto \mathbb{R}$. Wir suchen dann n Koeffizienten $x_j \in \mathbb{R}$, so dass

$$\sum_{i=1}^m |f(t_i) - y_i|^2 \rightarrow \min, \quad \text{wobei } f(t) := \sum_{j=1}^n x_j b_j(t). \quad (8.1.2)$$

Im Fall der Polynome $b_j(t) = t^{j-1}$ spricht man von *Ausgleichspolynomen*. In `python` verwenden wir die Funktion `numpy.polyfit`.

Natürlich kann man diese Idee sowohl auf $t \in \mathbb{R}^d$ als auch auf $y \in \mathbb{R}^d$ anwenden.

Beispiel 8.1.4. Im Beispiel 8.1.1 könnten wir $\mathbf{a}$ und c finden, indem wir das lineare Gleichungssystem

$$\begin{bmatrix} \mathbf{x}_1^T & 1 \\ \vdots & \vdots \\ \mathbf{x}_m^T & 1 \end{bmatrix} \begin{bmatrix} \mathbf{a} \\ c \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

lösen; wir haben aber mehr Gleichungen als Unbekannte, falls $m > n + 1$. Im Beispiel 8.1.3 liefert dieselbe Idee:

$$\begin{bmatrix} b_1(t_1) & \dots & b_n(t_1) \\ \vdots & & \vdots \\ b_1(t_m) & \dots & b_n(t_m) \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix},$$

mit demselben Schwierigkeitsgrad im Fall $m > n$.

Aufgabe (Lineare Ausgleichsrechnung):

gegeben: $\mathbf{A} \in \mathbb{K}^{m,n}$, $m, n \in \mathbb{N}$, $\mathbf{b} \in \mathbb{K}^m$,
finde: $\mathbf{x} \in \mathbb{K}^n$, so dass

- $$\begin{aligned} \text{(i)} \quad & \|\mathbf{Ax} - \mathbf{b}\|_2 = \inf\{\|\mathbf{Ay} - \mathbf{b}\|_2 : \mathbf{y} \in \mathbb{K}^n\}, \\ \text{(ii)} \quad & \|\mathbf{x}\|_2 \text{ ist minimal.} \end{aligned} \quad (8.1.3)$$

Bemerkung 8.1.5. Lassen Sie uns die ersten zwei Beispiele umformulieren:

$$\text{Beispiel 8.1.1: } \mathbf{A} = \begin{bmatrix} \mathbf{x}_1^T & 1 \\ \vdots & \vdots \\ \mathbf{x}_m^T & 1 \end{bmatrix} \in \mathbb{R}^{m,n+1}, \mathbf{b} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} \in \mathbb{R}^m, \mathbf{x} = \begin{bmatrix} \mathbf{a} \\ c \end{bmatrix} \in \mathbb{R}^{n+1}.$$

$$\text{Beispiel 8.1.3: } \mathbf{A} = \begin{bmatrix} b_1(t_1) & \dots & b_n(t_1) \\ \vdots & & \vdots \\ b_1(t_m) & \dots & b_n(t_m) \end{bmatrix} \in \mathbb{R}^{m,n}, \mathbf{b} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} \in \mathbb{R}^m, \mathbf{x} \in \mathbb{R}^n.$$

Bemerkung 8.1.6. Die Norm des Residuums $\|\mathbf{b} - \mathbf{Ax}\|_2$ erlaubt es uns, die Qualität des Modells zu beurteilen.

Bemerkung 8.1.7. In `python` verwenden wir die Funktion `numpy.linalg.lstsq`.

Bemerkung 8.1.8. Wir werden (durch Konstruktion) sehen, dass das lineare Ausgleichsproblem (8.1.3) eine eindeutige Lösung besitzt. Damit kann man die Pseudoinverse (Moore-Penrose) einer Matrix definieren.

In `python` stehen dafür `numpy.linalg.pinv` und `scipy.linalg.pinv2` zur Verfügung.

8.1.1 Normalengleichungen

Sei

$$\begin{aligned} f(x) &= \|\mathbf{b} - \mathbf{Ax}\|_2^2 = (\mathbf{b} - \mathbf{Ax})^H (\mathbf{b} - \mathbf{Ax}) = \\ &= (\mathbf{b}^H - \mathbf{x}^H \mathbf{A}^H) (\mathbf{b} - \mathbf{Ax}) = \mathbf{b}^H \mathbf{b} - \mathbf{b}^H \mathbf{Ax} - \mathbf{x}^H \mathbf{A}^H \mathbf{b} + \mathbf{x}^H \mathbf{A}^H \mathbf{Ax} = \\ &= \mathbf{b}^H \mathbf{b} - \mathbf{b}^H \mathbf{Ax} - \mathbf{b}^H \mathbf{Ax} + \mathbf{x}^H \mathbf{A}^H \mathbf{Ax} = \mathbf{b}^H \mathbf{b} - 2\mathbf{b}^H \mathbf{Ax} + \mathbf{x}^H \mathbf{A}^H \mathbf{Ax}. \end{aligned}$$

Wenn $f(x)$ minimal ist, dann muss $Df(x) = 0$ gelten, also

$$2(\mathbf{A}^H \mathbf{A})\mathbf{x} - 2\mathbf{A}^H \mathbf{b} = 0,$$

und somit muss

$$\mathbf{A}^H \mathbf{Ax} = \mathbf{A}^H \mathbf{b} \tag{8.1.4}$$

erfüllt werden.

Definition 8.1.9. Die Gleichung (8.1.4) heisst *Normalengleichung*.

Bemerkung 8.1.10. Die Matrix $\mathbf{A}^H \mathbf{A}$ ist hermitesch symmetrisch und, wenn $\mathbf{A}$ vollen Rang hat, dann auch positiv definit; somit besitzt das System $\mathbf{A}^H \mathbf{Ax} = \mathbf{A}^H \mathbf{b}$ eine eindeutige Lösung. Sie lässt sich mithilfe der LU-Zerlegung oder, noch besser, der Cholesky-Zerlegung bestimmen.

Der Nachteil der Normalgleichungen ist vor allem ihre schlechte Konditionierung und die damit verbundene numerische Instabilität. In der Tat gilt für die Kondition von $\mathbf{A}^H \mathbf{A}$:

$$\begin{aligned} \text{cond}(\mathbf{A}^H \mathbf{A}) &= \text{cond}(\mathbf{V} \Sigma \mathbf{U}^H \mathbf{U} \Sigma \mathbf{V}^H) = \text{cond}(\mathbf{V} \Sigma^2 \mathbf{V}^H) = \\ \text{cond}(\Sigma^2) &= \frac{\sigma_1^2}{\sigma_n^2} = \text{cond}(\mathbf{A})^2. \end{aligned}$$

Es ist deshalb nicht empfehlenswert, Normalgleichungen zur Lösung von linearen Ausgleichsproblemen zu verwenden, die durch eine schlecht konditionierte Matrix dargestellt werden. In diesem Fall läuft man Gefahr, im Verlauf der Berechnungen eine enorme Vergrößerung der Rundungsfehler auftreten zu lassen, was zu einer falschen Lösung führen würde.

Ein weiteres Problem der Normalgleichungen tritt auf, wenn die Matrix $\mathbf{A}$ dünn besetzt ist.¹ In der Tat: Wenn $\mathbf{A}$ dünn besetzt ist, so ist es $\mathbf{A}^H \mathbf{A}$ im Allgemeinen nicht. Dies kann bei grossem $\mathbf{A}$ zu einem Speicherüberlauf oder zu sehr langen Rechenzeiten führen. Weiterhin wird so die Möglichkeit der Anwendung effizienter direkter Lösungsmethoden für dünn besetzte Matrizen verschwendet.

Bemerkung 8.1.11. Eine Möglichkeit, die Berechnung von $\mathbf{A}^H \mathbf{A}$ zu umgehen, soll im Folgenden erläutert werden.

Das Residuum $\mathbf{r} = \mathbf{A}\mathbf{x} - \mathbf{b}$ wird als eine weitere Unbekannte behandelt. Man kann die Gleichung (8.1.4) folgendermassen umschreiben:

$$\mathbf{A}^H \mathbf{A} \mathbf{x} = \mathbf{A}^H \mathbf{b} \Leftrightarrow \mathbf{B} \begin{bmatrix} \mathbf{r} \\ \mathbf{x} \end{bmatrix} := \begin{bmatrix} -\mathbf{I} & \mathbf{A} \\ \mathbf{A}^H & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{r} \\ \mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \\ \mathbf{0} \end{bmatrix}. \quad (8.1.5)$$

Allgemeiner definiert man $\mathbf{r} := \frac{1}{a}(\mathbf{A}\mathbf{x} - \mathbf{b})$, wobei $a > 0$ gilt; dann kann man in Gleichung (8.1.5) die Matrix $\mathbf{B}$ durch

$$\mathbf{B}_a := \begin{bmatrix} -a\mathbf{I} & \mathbf{A} \\ \mathbf{A}^H & \mathbf{0} \end{bmatrix}$$

ersetzen. Dabei wählt man a so, dass $\kappa(\mathbf{B}_a)$ minimal wird. Die Umformung (8.1.5) von Gleichung (8.1.4) sowie ihre Variante führen bei dünn besetztem $\mathbf{A}$ wieder zu einer dünn besetzten Matrix $\mathbf{B}_a$ und sind somit mithilfe effizienter Verfahren für dünn besetzte Matrizen lösbar.

¹Englisch: *sparse matrix*

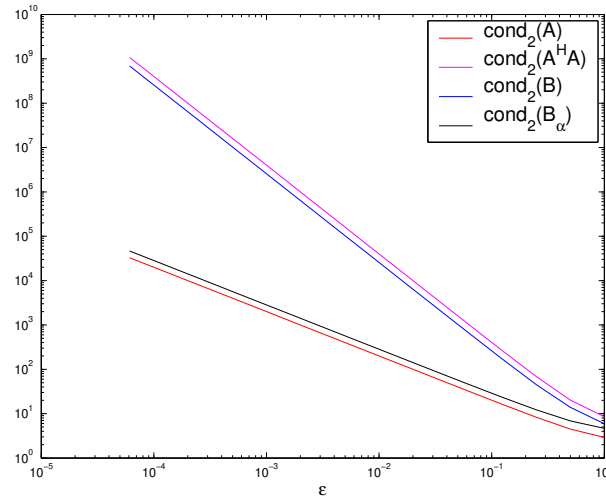


Abb. 8.1.12. Kondition von Normalengleichungen für verschiedene ϵ .

Beispiel 8.1.13. Die bessere Kondition des Systems 8.1.5 lässt sich leicht an einem Beispiel zeigen. Sei

$$\mathbf{A} = \begin{bmatrix} 1 + \epsilon & 1 \\ 1 - \epsilon & 1 \\ \epsilon & \epsilon \end{bmatrix}.$$

Weiterhin wird $\mathbf{B}_a$ mit $a = \epsilon \frac{\|\mathbf{A}\|}{\sqrt{2}}$ verwendet (ohne Beweis). Ein Vergleich der Konditionszahlen für verschiedene ϵ ist in Abbildung 8.1.12 aufgezeichnet.

Der folgende Code wurde verwendet:

Code 8.1.14: Vergleich der Kondition von verschiedenen Matrizen

```

1  from numpy import zeros, sqrt, zeros_like, linspace, shape, dot, conj, eye
2  from numpy.linalg import cond, norm
3  from matplotlib.pyplot import figure, loglog, legend, xlabel, ylabel, show
4  from matplotlib import rcParams #this line only produces bigger and clearer
   images for the notebook
5  rcParams['figure.figsize'] = (20.0, 10.0)

7  #first we define functions for building the two matrices

9  def A(eps):
10     A = zeros((3,2))
11     A[0,0] = 1. + eps; A[0,1] = 1.0
12     A[1,0] = 1. - eps; A[1,1] = 1.0
13     A[2,0] = eps; A[2,1] = eps
14     return A

15
16 def B(eps, adapt): #adapt:  boolean value for choosing a = 1.  or adapting a to
   minimize cond
17     a = 1.0
18     mat = A(eps)
19     if adapt==True:
20         a = eps*norm(mat)/sqrt(2.0)

```

```

21     dimB = shape(mat)[0]+shape(mat)[1]
    B = zeros((dimB,dimB))
23     B[:shape(mat)[0],:shape(mat)[0]] = -a*eye(shape(mat)[0])
    B[:shape(mat)[0],shape(mat)[0]:] = mat
25     B[shape(mat)[0]:,shape(mat)[0]] = conj(mat).T #hermite transpose, .H does
not work with ndarray
    return B
27
    epsilons = linspace(1e-5,1,100)
29
    #containers for condition of the different matrices as a function of epsilon
31     condA = zeros_like(epsilons); condAHA = zeros_like(epsilons)
    condB = zeros_like(epsilons); condBa = zeros_like(epsilons)
33
    #print(B(A(0.04), adapt=True))
35
    for k in range(100):
37         mat = A(epsilons[k])
        condA[k] = cond(mat)
39         condAHA[k] = cond(dot(conj(mat).T,mat))
        condB[k] = cond(B(epsilons[k], adapt = False))
41         condBa[k] = cond(B(epsilons[k], adapt = True))

43     #now we plot the results
    figure()
45     loglog(epsilons,condA,label="cond($A$)")
    loglog(epsilons,condAHA,label="cond($A^HA$)")
47     loglog(epsilons,condB,label="cond($B$)")
    loglog(epsilons,condBa,label="cond($B_a$)")
49     xlabel("$\epsilon$"); ylabel("cond")
    legend(loc="best")
51     show()

```

8.1.2 Lösung mittels orthogonaler Transformationen

Aus Beispiel 8.1.13 sieht man, dass die Normalengleichungen zur numerischen Lösung des Ausgleichsproblems schlecht geeignet sind. Auch das LU-Verfahren kann für eine Matrix mit $m > n$ nicht sinnvoll angewandt werden. Man benötigt also ein Verfahren, das numerisch stabil und gleichzeitig für überbestimmte Systeme geeignet ist.

Wir betrachten wieder

$$\mathbf{r} = \mathbf{Ax} - \mathbf{b}$$

und versuchen, $\|\mathbf{r}\|_2^2$ zu minimieren. Es ist bekannt, dass unitäre/orthonormale Matrizen die Norm unverändert lassen. Betrachtet man die vollständige QR-Zerlegung von $\mathbf{A}$, so lässt sich ein Lösungsansatz herleiten, der stabiler als die Normalengleichungen ist. Wenn

$$\mathbf{A} = \mathbf{QR} = \mathbf{Q} \begin{bmatrix} \tilde{\mathbf{R}} \\ 0 \end{bmatrix}$$

mit der unitären Matrix $\mathbf{Q}$, dann führt dies zu einer Umformulierung von (8.1.3):

$$\begin{aligned}\|\mathbf{r}\|_2^2 &= \|\mathbf{Ax} - \mathbf{b}\|_2^2 = \|\mathbf{QRx} - \mathbf{QQ}^H\mathbf{b}\|_2^2 = \\ &\|\mathbf{Q}(\mathbf{Rx} - \mathbf{Q}^H\mathbf{b})\|_2^2 = \|\mathbf{Rx} - \tilde{\mathbf{b}}\|_2^2,\end{aligned}\tag{8.1.6}$$

wobei hier $\tilde{\mathbf{b}} = \mathbf{Q}^H\mathbf{b}$ ist. Somit gilt

$$\|\mathbf{Ax} - \mathbf{b}\|_2 \rightarrow \min \Leftrightarrow \left\| \begin{pmatrix} \begin{matrix} \text{yellow staircase} \\ \mathbf{R} \\ \mathbf{0} \end{matrix} \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} - \begin{pmatrix} \tilde{b}_1 \\ \vdots \\ \tilde{b}_m \end{pmatrix} \right\|_2 \rightarrow \min$$

und eine explizite Lösung des Ausgleichsproblems ist

$$\mathbf{Rx} - \tilde{\mathbf{b}} = \begin{bmatrix} \tilde{\mathbf{R}} \\ 0 \end{bmatrix} \mathbf{x} - \begin{bmatrix} \tilde{\mathbf{b}}_1 \\ \tilde{\mathbf{b}}_2 \end{bmatrix} \Rightarrow \tilde{\mathbf{R}}\mathbf{x} = \tilde{\mathbf{b}}_1 \Leftrightarrow \mathbf{x} = \tilde{\mathbf{R}}^{-1}\tilde{\mathbf{b}}_1.$$

$$\mathbf{x} = \left(\begin{pmatrix} \text{yellow staircase} \\ \tilde{\mathbf{R}} \end{pmatrix} \right)^{-1} \begin{bmatrix} \tilde{b}_1 \\ \vdots \\ \tilde{b}_n \end{bmatrix}, \quad \text{Residuum } \mathbf{r} = \mathbf{Q} \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \tilde{b}_{n+1} \\ \vdots \\ \tilde{b}_m \end{bmatrix}.$$

Weiterhin folgt für die Norm des Residuums $\|\mathbf{r}\|_2 = \|\mathbf{Q}\tilde{\mathbf{b}}_2\|_2 = \|\tilde{\mathbf{b}}_2\|_2 = \sqrt{\tilde{b}_{n+1}^2 + \cdots + \tilde{b}_m^2}$.

Da die QR-Zerlegung relativ teuer ist und die Matrix $\mathbf{Q}$ nicht benötigt wird, muss sie nicht gespeichert werden. Führt man die QR-Zerlegung der Matrix $\mathbf{A}$ durch (z.B. mittels Householder-Spiegelungen) und wendet die Transformationen in jedem Schritt auch auf $\mathbf{b}$ an, erhält man die rechte Seite von Gleichung (8.1.6) ohne die Matrix $\mathbf{Q}$ je vollständig im Speicher zu haben, was bei grossen Matrizen einen Geschwindigkeitsvorteil bringt.

Wenn die Matrix $\mathbf{A}$ nicht vollen Rang hat (was in konkreten Fällen vorkommen kann), ist es besser, die Singulärwertzerlegung zu verwenden. Diese steckt

auch hinter professionellen Lösern wie `numpy.linalg.lstsq`. Die Normerhaltung unitärer/orthogonaler Transformationen liefert wie vorher:

$$\|\mathbf{Ax} - \mathbf{b}\|_2 = \|\mathbf{U}\Sigma\mathbf{V}^H\mathbf{x} - \mathbf{b}\|_2 = \|\Sigma\mathbf{V}^H\mathbf{x} - \mathbf{U}^H\mathbf{b}\|_2.$$

Wenn $\text{rank}(\mathbf{A}) = r < \min(m, n)$, dann

$$\Sigma = \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix},$$

und alle Zeilenvektoren $\mathbf{v}_j$ der Matrix $\mathbf{V}$, für die gilt: $j > r$, liegen im Kern von $\mathbf{A}$. Ähnliches gilt auch für die Matrix $\mathbf{U}$. Beide werden folgendermassen unterteilt:

$$\begin{aligned} \mathbf{A} &= [\mathbf{U}_1 \quad \mathbf{U}_2] \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{V}_1^H \\ \mathbf{V}_2^H \end{bmatrix} \\ \left[\begin{array}{|c|} \hline \mathbf{A} \\ \hline \end{array} \right] &= \left[\begin{array}{|c|c|} \hline \mathbf{U}_1 & \mathbf{U}_2 \\ \hline \end{array} \right] \left[\begin{array}{|c|c|} \hline \Sigma_r & 0 \\ \hline 0 & 0 \end{array} \right] \left[\begin{array}{|c|c|} \hline \mathbf{V}_1^H \\ \hline \mathbf{V}_2^H \end{array} \right], \end{aligned} \quad (8.1.7)$$

mit je orthonormalen Spalten in den Matrizen $\mathbf{U}_1$, $\mathbf{U}_2$, $\mathbf{V}_1$ und $\mathbf{V}_2$:

$$\mathbf{U}_1 \in \mathbb{K}^{m \times r}, \mathbf{U}_2 \in \mathbb{K}^{m \times n-r}, \mathbf{V}_1 \in \mathbb{K}^{n \times r}, \mathbf{V}_2 \in \mathbb{K}^{n \times n-r}.$$

Damit gilt für das Residuum:

$$\|\mathbf{Ax} - \mathbf{b}\|_2 = \left\| [\mathbf{U}_1 \quad \mathbf{U}_2] \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{V}_1^H \\ \mathbf{V}_2^H \end{bmatrix} \mathbf{x} - \mathbf{b} \right\|_2 = \left\| \begin{bmatrix} \Sigma_r \mathbf{V}_1^H \mathbf{x} \\ 0 \end{bmatrix} - \begin{bmatrix} \mathbf{U}_1^H \mathbf{b} \\ \mathbf{U}_2^H \mathbf{b} \end{bmatrix} \right\|_2.$$

Somit wird das Residuum genau dann minimal, wenn

$$\Sigma_r \mathbf{V}_1^H \mathbf{x} = \mathbf{U}_1^H \mathbf{b} \Leftrightarrow \mathbf{V}_1^H \mathbf{x} = \Sigma_r^+ \mathbf{U}_1^H \mathbf{b}, \quad (8.1.8)$$

wobei $\Sigma_r^+ = \text{diag}(\frac{1}{\sigma_1}, \dots, \frac{1}{\sigma_r})$ die “Inverse” von Σ_r ist. Das Residuum dazu ist $\|\mathbf{U}_2^H \mathbf{b}_2\|_2$. Wir definieren nun

$$\mathbf{x} = \mathbf{V}_1 \Sigma_r^+ \mathbf{U}_1^H \mathbf{b}, \quad (8.1.9)$$

das offensichtlich die Gleichung (8.1.8) erfüllt ($\mathbf{V}_1^H \mathbf{V}_1 = \mathbf{I}_r$). Das so definierte x liegt in $\text{Bild}(\mathbf{A}^H) = \text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_r\}$ und steht somit orthogonal auf $\text{Ker}(\mathbf{A}) = \text{span}\{v_{r+1}, \dots, v_n\}$. Dass die Lösung des Minimierungsproblems (i) von (8.1.3) nicht eindeutig ist, sieht man bereits im Fundamentalsatz der Linearen Algebra: Jedes $x_g = x + x_n$ mit beliebigem x_n in $\text{Ker}(\mathbf{A})$ löst das Ausgleichsproblem. Die in (8.1.9) definierte Lösung steht orthogonal auf $\text{Ker}(\mathbf{A}) = \text{span}\{v_{r+1}, \dots, v_n\}$ und minimiert somit auch die euklidische Norm $\|x\|_2$ und löst die Lineare Ausgleichsrechnung (8.1.3) vollständig. Die Bedingung (ii) legt also genau eine spezielle Lösung des einfachen Ausgleichsproblems fest.

Definition 8.1.15. Die Matrix $\mathbf{V}_1 \Sigma_r^+ \mathbf{U}_1^H$ in Gleichung (8.1.9) wird als *Pseudoinverse* der Matrix $\mathbf{A}$ bezeichnet und formal definiert durch:

$$\mathbf{A}^+ = (\mathbf{A}^H \mathbf{A})^{-1} \mathbf{A}^H = \mathbf{V}_1 \Sigma_r^+ \mathbf{U}_1^H.$$

Die Lösung von (8.1.3) mittels Singulärwertzerlegung wird also auf die Bestimmung der Pseudoinversen von $\mathbf{A}$ und die Berechnung von $\mathbf{x}$ mittels dieser zurückgeführt. Eine mögliche Implementierung, die auf dem *numerischen Rang*

$$r = \max\{i : \sigma_i / \sigma_1 > \text{tol}\}$$

einer Matrix beruht, ist:

Code 8.1.16: Lösung des linearen Ausgleichsproblems mit Singulärwertzerlegung

```

1 from numpy.linalg import svd
2 from numpy import dot, where
3
4 def lsqsvd(A,b,eps=1e-6):
5     U,s,Vh = svd(A) # python routine for singular value decomposition
6     r = 1+where(s/s[0]>eps)[0].max() # numerical rank
7     y = dot(Vh[:,r:].T, dot(U[:,r:].T,b)/s[r:])
8     return y

```

Im Allgemeinen lassen sich die drei Verfahren in zwei Anwendungskategorien einordnen.

1. $\mathbf{A} \in \mathbb{K}^{m \times n}$ ist voll besetzt und n ist klein ($m \gg n$).
2. $\mathbf{A} \in \mathbb{K}^{m \times n}$ ist dünn besetzt und m, n sind gross.

Im ersten Fall bieten sich QR-Zerlegung und Singulärwertzerlegung aufgrund ihrer numerischen Stabilität an. Da beide jedoch die spezifische Struktur dünn besetzter Matrizen nicht ausnutzen können, wendet man auf Probleme der zweiten Art Normalengleichungen an.

Beispiel 8.1.17. Wir kehren hier zum zweiten Beispiel zurück. Aus den Werten der Tabelle erhalten wir die Matrix $\mathbf{A}$ und den Vektor $\mathbf{b}$ als:

$$\mathbf{A} = \begin{bmatrix} 98.269 & 1 \\ 0 & 1 \\ -194.96 & 1 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 852.7 \\ 624.5 \\ 172.7 \end{bmatrix}.$$

Die QR-Zerlegung ergibt:

$$\mathbf{Q} = \begin{bmatrix} 0.45010 & 0.71625 \\ 0.00000 & 0.59720 \\ -0.89298 & 0.36102 \end{bmatrix} \quad \mathbf{R} = \begin{bmatrix} 218.32590 & -0.44287 \\ 0.00000 & 1.67447 \end{bmatrix}.$$

Durch Auflösen nach $\tilde{\mathbf{b}} = \mathbf{Q}^T \mathbf{b}$ und Rückwärtseinsetzen von $\mathbf{R}\mathbf{x} = \tilde{\mathbf{b}}$ erhalten wir somit schliesslich für die Gleichung $f(x) = mx + c$ die Koeffizienten

$$\begin{aligned} m &= \mathbf{x}_1 = 2.3188 \\ c &= \mathbf{x}_2 = 624.7017. \end{aligned}$$

Löst man dann noch $f(t) = 0$ nach t auf, erhält man als Näherung für den absoluten Nullpunkt $t = -269.41$, was eine relativ gute Approximation ist ($T_0 = -273.15$).

Code 8.1.18: Lösung eines linearen Ausgleichsproblems mittels verschiedener Methoden

```

1 from numpy.linalg import qr, solve, lstsq
2 from numpy import array, dot
3
4 A = array([[98.269,1.0],[0.0,1.0],[-194.96,1.0]])
5 b = array([852.7,624.5,172.7])
6
7 #first: with qr-decomposition:
8 Q, R = qr(A)
9 b_tilde = dot(Q.T,b)
10 x_qr = solve(R,b_tilde)
11
12 print("QR-decomposition:")
13 print("m = x_1 = ", x_qr[0])
14 print("c = x_2 = ", x_qr[1])
15
16 #now with the pseudo inverse method, using the function that we defined in the
17 previous code:
18 x_lsqsvd = lsqsvd(A,b)
19
20 print("Pseudo inverse:")
21 print("m = x_1 = ", x_lsqsvd[0])
22 print("c = x_2 = ", x_lsqsvd[1])
23
24 #now with Numpy's function (least squares):
25 x_lstsq = lstsq(A,b)
26
27 print("Numpy - lstsq:")
28 print("m = x_1 = ", x_lstsq[0][0])
29 print("c = x_2 = ", x_lstsq[0][1])

```

8.1.3 Totale Ausgleichsrechnung

Was soll man tun, wenn auch die Matrix und die rechte Seite in $\mathbf{Ax} = \mathbf{b}$ fehlerbehaftet sind? Eine Lösung existiert, auch im Sinne der kleinsten Quadrate, nur wenn $\mathbf{b}$ in Bild $\mathbf{A}$ liegt. Die Messfehler in $\mathbf{A}$ und $\mathbf{b}$ verhindern dann eine Lösung. In diesem Fall ersetzt man das System $\mathbf{Ax} = \mathbf{b}$ durch ein neues System $\hat{\mathbf{A}}\hat{\mathbf{x}} = \hat{\mathbf{b}}$, das *so nah wie möglich* am ursprünglichen System liegt, so dass $\hat{\mathbf{b}} \in \text{Bild } \hat{\mathbf{A}}$. Die mathematische Formulierung lautet:

Gegeben die $m \times n$ -Matrix $\mathbf{A}$ mit Rang n und der Vektor $\mathbf{b}$ der Länge m , finde $\hat{\mathbf{A}}$ und $\hat{\mathbf{b}} \in \text{Bild } \hat{\mathbf{A}}$, so dass folgende Norm minimal wird

$$\left\| [\mathbf{A} \ \mathbf{b}] - [\hat{\mathbf{A}} \ \hat{\mathbf{b}}] \right\|_F.$$

Wir notieren $\mathbf{C} = [\mathbf{A} \ \mathbf{b}]$ und $\hat{\mathbf{C}} = [\hat{\mathbf{A}} \ \hat{\mathbf{b}}]$. Die Bedingung $\hat{\mathbf{b}} \in \text{Bild } \hat{\mathbf{A}}$ bedeutet dann, dass $\hat{\mathbf{C}}$ Rang n hat, so dass sich das Problem umschreiben lässt zu

$$\min_{\text{Rang } \hat{\mathbf{C}}=n} \left\| \mathbf{C} - \hat{\mathbf{C}} \right\|_F.$$

Das Theorem 7.1.53 weist uns den Weg zur Lösung. Die Singulärwertzerlegung

$$\mathbf{C} = \mathbf{U} \Sigma \mathbf{V}^H = \sum_{j=1}^{n+1} \sigma_j (\mathbf{u})_j (\mathbf{v})_j^H$$

gibt das Optimum

$$\hat{\mathbf{C}} = \sum_{j=1}^n \sigma_j (\mathbf{u})_j (\mathbf{v})_j^H.$$

Die Matrix $\hat{\mathbf{C}}$ ist $m \text{ times } (n+1)$ und hat Rang n , so ist die Dimension seines Kerns 1. Die Orthogonalität der Vektoren $\mathbf{v}_j$ ergibt $\hat{\mathbf{C}} \mathbf{v}_{n+1} = 0$, d.h. $\mathbf{v}_{n+1}$ liegt im Kern von $\hat{\mathbf{C}}$, der Dimension 1 hat, so ist $\{\mathbf{v}_{n+1}\}$ eine Basis im Kern von $\hat{\mathbf{C}}$.

Das lineare Gleichungssystem $\hat{\mathbf{A}} \hat{\mathbf{x}} = \hat{\mathbf{b}}$ schreiben wir in $\hat{\mathbf{C}} [\hat{\mathbf{x}}^T, -1]^T = \mathbf{0}$, so muss $[\hat{\mathbf{x}}^T, -1]^T$ ein vielfaches vom einzigen Basiselement des Kerns von $\hat{\mathbf{C}}$, nämlich $\mathbf{v}_{n+1}$ sein. Falls $v_{n+1, n+1} \neq 0$, dann haben wir auch die Lösung des Systems $\hat{\mathbf{A}} \hat{\mathbf{x}} = \hat{\mathbf{b}}$ als $\hat{\mathbf{x}} = -\mathbf{v}_{n+1} / v_{n+1, n+1}$.

8.2 Nichtlineare Ausgleichsrechnung

Wenn das zugrunde liegende Modell für das Ausgleichsproblem nicht linear in den Parametern ist, dann sprechen wir von *nichtlinearer Ausgleichsrechnung*. Im Folgenden notieren wir die Modellfunktion mit $f : \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}$, die gesuchten Parameter mit $\mathbf{x} \in \mathbb{R}^n$, die m Messpunkte mit $t_i \in \mathbb{R}$ und die entsprechenden Messungen mit $y_i \in \mathbb{R}$.

Aufgabe (nichtlineare Ausgleichsrechnung): Finde $\mathbf{x} \in \mathbb{R}^n$, so dass

$$\mathbf{x} = \underset{\mathbf{u} \in \mathbb{R}^n}{\operatorname{argmin}} \left(\sum_{i=1}^m |f(t_i, \mathbf{u}) - y_i|^2 \right) = \underset{\mathbf{u} \in \mathbb{R}^n}{\operatorname{argmin}} \frac{1}{2} \|\mathbf{F}(\mathbf{u})\|_2^2. \quad (8.2.1)$$

Hier ist die Residuumfunktion $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ von der Form:

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} f(t_1, \mathbf{x}) - y_1 \\ \vdots \\ f(t_m, \mathbf{x}) - y_m \end{bmatrix}.$$

Für das weitere Vorgehen definieren wir $\Phi(\mathbf{x}) := \frac{1}{2} \|\mathbf{F}(\mathbf{x})\|_2^2$.

Bemerkung 8.2.1. Die Existenz und Eindeutigkeit einer Lösung von (8.2.1) ist im Allgemeinen nicht klar. Aus der Analysis ist bekannt (Satz über implizite Funktionen), dass es eine Umgebung $U_\epsilon(\mathbf{x})$ gibt, in welcher $\text{rang}(D\Phi(\mathbf{x})) = n$ ist, $D\Phi(\mathbf{x})$ also in $U_\epsilon(\mathbf{x})$ invertierbar ist. Da dies aber eine lokale Eigenschaft von $\Phi(\mathbf{x})$ ist, lässt sich lediglich folgern, dass eine Lösung lokal eindeutig ist, wenn sie existiert.

8.2.1 Newton-Verfahren

Gesucht wird $\mathbf{x} \in \mathbb{R}^n$, so dass gilt: $\Phi(\mathbf{x}) = \min$. Aus der Analysis ist bekannt, dass eine notwendige Bedingung dafür $\text{grad } \Phi(\mathbf{x}) = 0$ ist. Wir haben das Problem (8.2.1) somit auf eine Nullstellensuche im $\mathbb{R}^n$ reduziert. Diese lässt sich mithilfe des Newton-Verfahrens lösen:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - (D \text{grad } \Phi(\mathbf{x}))^{-1} \text{grad } \Phi(\mathbf{x}). \quad (8.2.2)$$

$$(8.2.3)$$

Wir rechnen dann aus:

$$\text{grad } \Phi(\mathbf{x}) = (D\mathbf{F}(\mathbf{x}))^T \mathbf{F}(\mathbf{x}) \quad (8.2.4)$$

$$D(\text{grad } \Phi)(\mathbf{x}) = (D\mathbf{F}(\mathbf{x}))^T D\mathbf{F}(\mathbf{x}) + \sum_{j=1}^m F_j(\mathbf{x}) D^2 F_j(\mathbf{x}). \quad (8.2.5)$$

In der Gleichung (8.2.5) taucht die Hesse-Matrix der Funktion Φ auf:

$$H_\Phi(\mathbf{x}) := D(\text{grad } \Phi)(\mathbf{x}). \quad (8.2.6)$$

Aus (8.2.2), (8.2.3) und (8.2.5) mit der Notation (8.2.6) können wir die Iteration umschreiben in:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - (H_\Phi(\mathbf{x}))^{-1} \text{grad } \Phi(\mathbf{x}). \quad (8.2.7)$$

Andererseits kann (8.2.2) auch als Minimierung der quadratischen Form $Q(\mathbf{x}) := \Phi(\mathbf{x}^{(k)}) + (\text{grad } \Phi(\mathbf{x}^{(k)}))^T \mathbf{s} + \frac{1}{2} \mathbf{s}^T H_\Phi(\mathbf{x}^{(k)}) \mathbf{s}$, die Φ lokal approximiert (Taylor 2. Ordnung), betrachtet werden:

$$\text{grad } Q(\mathbf{x}) = 0 \Leftrightarrow H_\Phi(\mathbf{x}^{(k)}) \mathbf{s} + \text{grad } \Phi(\mathbf{x}^{(k)}) = 0,$$

woraus wiederum (8.2.7) folgt.

Die Newton-Iteration kann man auch so schreiben:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{s} \quad \text{mit} \quad D^2 \Phi(\mathbf{x}^{(k)}) \mathbf{s} = (D\mathbf{F}(\mathbf{x}^{(k)}))^T \mathbf{F}(\mathbf{x}^{(k)}) \quad (8.2.8)$$

8.2.2 Gauss-Newton Verfahren

Das reine Newton-Verfahren für (8.2.2) ist aufwendig, selbst wenn man die Matrixinversion durch das Lösen eines LGS ersetzt. Zudem ist $F(\mathbf{x})$ nicht immer zweimal stetig differenzierbar auf $U_\epsilon(\mathbf{x}_0)$, was das Aufstellen der Hesse-Matrix unmöglich macht. Um beide Probleme zu umgehen, bedient man sich einer wohlbekannten Idee aus der Analysis.

Die komplizierte Funktion $F(\mathbf{x})$ wird lokal (auf einer Umgebung der Lösung $\mathbf{x}_0$) durch eine lineare Funktion approximiert, das heisst linearisiert:

$$F(\mathbf{x}) \approx F(\mathbf{y}) + DF(\mathbf{y})(\mathbf{x} - \mathbf{y}) = F(\mathbf{y}) + D\mathbf{F}(\mathbf{y})\mathbf{x} - D\mathbf{F}(\mathbf{y})\mathbf{y}. \quad (8.2.9)$$

Notiert man $\mathbf{A} := D\mathbf{F}(\mathbf{y})$ und $\mathbf{b} := D\mathbf{F}(\mathbf{y})\mathbf{y} - \mathbf{F}(\mathbf{y})$ so erhält man für (8.2.1) unter Zuhilfenahme von (8.2.9) ein lineares Ausgleichsproblem:

$$\operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \|F(\mathbf{x})\|_2^2 \approx \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \|F(\mathbf{y}) + D\mathbf{F}(\mathbf{y})\mathbf{x} - \mathbf{F}(\mathbf{y})\|_2^2 = \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2,$$

wobei $\mathbf{y}$ eine Näherung der Lösung $\mathbf{x}$ von (8.2.1) ist.

Setzt man $\mathbf{x} = \mathbf{x}^{k+1} = \mathbf{x}^k - \mathbf{s}$ und $\mathbf{y} = \mathbf{x}^k$, dann erhält man

$$\operatorname{argmin}_{\mathbf{z} \in \mathbb{R}^n} \frac{1}{2} \|F(\mathbf{x}^k - \mathbf{z})\| \approx \operatorname{argmin}_{\mathbf{z} \in \mathbb{R}^n} \frac{1}{2} \|F(\mathbf{x}^k) + D\mathbf{F}(\mathbf{x}^k)(\mathbf{x}^k - \mathbf{z} - \mathbf{x}^k)\| \quad (8.2.10)$$

$$= \operatorname{argmin}_{\mathbf{z} \in \mathbb{R}^n} \frac{1}{2} \|F(\mathbf{x}^k) - D\mathbf{F}(\mathbf{x}^k)\mathbf{z}\| \quad (8.2.11)$$

Damit ergibt sich die Vorschrift für die Gauss-Newton-Iteration zu:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{s} \quad \text{mit} \quad \mathbf{s} := \operatorname{argmin}_{\mathbf{z} \in \mathbb{R}^n} \|F(\mathbf{x}^{(k)}) - D\mathbf{F}(\mathbf{x}^{(k)})\mathbf{z}\|_2^2. \quad (8.2.12)$$

Würden wir die Normalengleichung für das Lösen des linearen Problems verwenden, würde die Iteration so aussehen:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{s} \quad \text{mit} \quad \left[(D\mathbf{F}(\mathbf{x}^{(k)}))^T D\mathbf{F}(\mathbf{x}^{(k)}) \right] \mathbf{s} = (D\mathbf{F}(\mathbf{x}^{(k)}))^T \mathbf{F}(\mathbf{x}^{(k)}). \quad (8.2.13)$$

Diese Iteration ist mit der Newton-Iteration (8.2.8) zu vergleichen. Der Unterschied ist: Die Hesse-Matrix (8.2.5) wurde durch die Matrix $\left[(D\mathbf{F}(\mathbf{x}^{(k)}))^T D\mathbf{F}(\mathbf{x}^{(k)}) \right]$ ersetzt, was dem Weglassen der letzten Summe in (8.2.5) entspricht.

Wie auch beim Newton-Verfahren muss dem Gauss-Newton-Verfahren ein guter Startwert $\mathbf{x}^{(0)} \in \mathbb{R}^n$ übergeben werden.

Code 8.2.2: Gauss-Newton Verfahren

```
1 from numpy.linalg import lstsq, norm
3 def gn(x,F,J,tol):
```

```

#x is the starting vector that has to be intelligently chosen
#F: function
#J: Jacobi-Matrix
#tol: tolerance
s = lstsq(J(x),F(x))[0]
x = x-s
#now we perform the iteration
while norm(s) > tol*norm(x):
    #every time we update x by subtracting s, found with the least square
method
    s = lstsq(J(x),F(x))[0]
    x = x-s
return x

```

Der Code 8.2.2 stellt eine beispielhafte Implementierung des Gauss-Newton-Verfahrens dar. Dabei wird der Funktion mit $\mathbf{x}$ der Anfangswert $\mathbf{x}^{(0)}$ übergeben, F ist eine Referenz auf die zu minimierende Funktion F , J ist die Jakobi-Matrix von F und tol ist der Grenzwert für den Fehler.

Die zweite Ableitung von $F(\mathbf{x})$ wird nicht benötigt, aber die Konvergenzordnung ist niedriger als die des Newton-Verfahrens ($p \leq 2$).

Beispiel 8.2.3. Wir kehren zurück zum ersten Beispiel. Die Modellfunktion für den Heizvorgang ist $I_1(t) = a_1 + b_1 e^{-c_1 t}$ und für den Abkühlvorgang $I_2(t) = a_2 - b_2 e^{-c_2 t}$. Wir stellen die jeweiligen Funktionen F_1, F_2 nach (8.2.1) auf:

$$F_1(\mathbf{a}_1) = \begin{bmatrix} a_1 + b_1 - 24.34 \\ a_1 + b_1 e^{-5c_1} - 18.93 \\ \vdots \\ a_1 + b_1 e^{-25c_1} - 15.91 \end{bmatrix}, \quad F_2(\mathbf{a}_2) = \begin{bmatrix} a_2 - b_2 - 9.66 \\ a_2 - b_2 e^{-5c_2} - 18.8 \\ \vdots \\ a_2 - b_2 e^{-25c_2} - 24.91 \end{bmatrix},$$

wobei $\mathbf{a}_1 = (a_1, b_1, c_1)$, $\mathbf{a}_2 = (a_2, b_2, c_2)$ sind. Die entsprechenden Jakobi-Matrizen sind dann

$$J_{F_1}(\mathbf{a}_1) = \begin{bmatrix} 1 & 1 & 0 \\ 1 & e^{-5c_1} & -5b_1 e^{-5c_1} \\ \vdots & \vdots & \vdots \\ 1 & e^{-25c_1} & -25b_1 e^{-25c_1} \end{bmatrix}, \quad J_{F_2}(\mathbf{a}_2) = \begin{bmatrix} 1 & 1 & 0 \\ 1 & -e^{-5c_2} & 5b_2 e^{-5c_2} \\ \vdots & \vdots & \vdots \\ 1 & -e^{-25c_2} & 25b_2 e^{-25c_2} \end{bmatrix}.$$

Verwendet man diese Matrizen und Vektoren als Eingabedaten für die Funktion aus Code 8.2.2, mit den Startvektoren $x_1 = (10, 5, 0)$ für F_1 und $x_2 = (30, 10, 0)$ für F_2 , so erhält man folgende Parameter:

$$a_1 = 15.8489, b_1 = 8.4822, c_1 = 0.1991 \\ a_2 = 25.0657, b_2 = 15.3954, c_2 = 0.1779.$$

Das Ergebnis ist in Abbildung 8.2.5 dargestellt. Die Messwerte werden durch I_1, I_2 mit den entsprechenden Parametern gut approximiert.

Der folgende Code wurde verwendet:

Code 8.2.4: Beispiel: Anwendung des Gauss-Newton Algorithmus

```

1  from numpy import arange, array, zeros, exp
3  t = arange(0,30,5)
   curr_heating = array([24.34,18.93,17.09,16.27,15.97,15.91])
5  curr_cooling = array([9.66,18.8,22.36,24.07,24.59,24.91])

7  n =len(t)

9  #define the functions that have to be minimized
   F_1 = lambda a: a[0] + a[1]*exp(-a[2]*t) - curr_heating
11  F_2 = lambda a: a[0] - a[1]*exp(-a[2]*t) - curr_cooling

13  #define the corresponding Jacobi matrices
   def J_1(a):
15       mat = zeros((n,3))
       for k in range(n):
17           mat[k,0] = 1.0
           mat[k,1] = exp(-t[k]*a[2])
19           mat[k,2] = -t[k]*a[1]*exp(-t[k]*a[2])
       return mat

21   def J_2(a):
23       mat = zeros((n,3))
       for k in range(n):
25           mat[k,0] = 1.0
           mat[k,1] = -exp(-t[k]*a[2])
27           mat[k,2] = t[k]*a[1]*exp(-t[k]*a[2])
       return mat

29   #guess starting vector
31   x_1 = array([10.,5.,0.])
   x_2 = array([30.,10.,0.])

33   #use the Gauss-Newton algorithm declared above
35   a_1 = gn(x_1,F_1,J_1,tol=10e-6)
   a_2 = gn(x_2,F_2,J_2,tol=10e-6)
37   print("Heating ", a_1)
   print("Cooling ", a_2)

39   #plot the data and the fit
41   ts = linspace(0,25,750) #for plotting
   from matplotlib.pyplot import figure, show, plot, xlabel, ylabel, legend, grid
43   figure(); xlabel("Zeit $t$"); ylabel("Strom $I(t)$"); grid(True)
   plot(t, curr_heating, "bv", label="Messdaten")
45   plot(t, curr_cooling, "rv", label="Messdaten")
   plot(ts,a_1[0] + a_1[1]*exp(-a_1[2]*ts), "b", label="$I_1(t)$")
47   plot(ts,a_2[0] - a_2[1]*exp(-a_2[2]*ts), "r", label="$I_2(t)$")
   legend(loc="best")
49   show()

```

In der gestellten Aufgabe ist die Spannung für $t \rightarrow \infty$ gesucht, welche den

konstanten Termen a_1, a_2 entspricht. Mithilfe dieser beiden Spannungswerte ist es möglich, einen Näherungswert für die Wärmeleitfähigkeit von Nickel zu bestimmen.

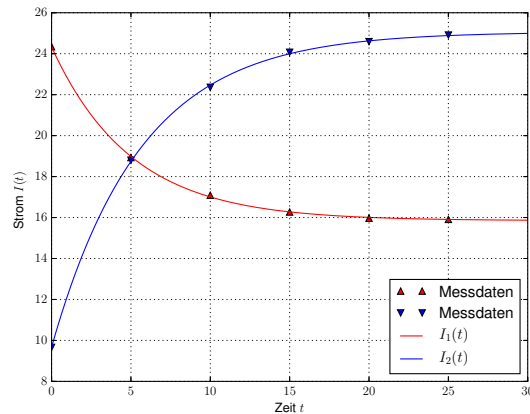


Abb. 8.2.5. Nichtlineare Funktionsanpassung und Messwerte

Beispiel 8.2.6. Für dasselbe nichtlineare Ausgleichsproblem wie vorhin:

$$f(t) = x_1 + x_2 \exp(-x_3 t),$$

untersuchen wir in einem numerischen Experiment die Konvergenz beider Verfahren für zwei verschiedene Startwerte. Hier ist der Simulationscode:

Code 8.2.7: Experiment mit Gauss-Newton und Newton

```

from numpy.linalg import lstsq, norm, solve
2 from scipy.linalg import lu_solve, lu_factor
from numpy import arange, array, zeros, exp, ones, linspace, dot
4 from matplotlib.pyplot import figure, show, plot, xlabel, ylabel, legend, grid,
semilogy, subplots, twinx, title, grid
from numpy.random import default_rng

6
seed = 1903 # reproducibility
8 rng = default_rng(seed)

10 x = array([1.,2.,0.15]) # reality
t = arange(1.,7.,0.3) # where we measure (training points)
12 f = lambda x,t: x[0] + x[1]*exp(-x[2]*t) # model
y = f(x,t) # exact measurements
14 y += 0.1*(rng.random(len(t))-0.5) # perturbed measurements
# Residuum function len(F) = len(t)
16 F = lambda x: f(x,t) - y
#Jacobi-Matrix: DF(x).shape = ( len(t), len(x) )
18 DF = lambda x: array([ones(len(t)), exp(-x[2]*t), -t*x[1]*exp(-x[2]*t)]).T

20 # Hesse-Matrix of  $\Phi(x) = 1/2 ||F(x)||_2^2$  involves the sum over all measurements
def HessPhi(x):

```

```

22     f = F(x)
23     df = DF(x)
24     tmp = t*exp(-x[2]*t) # used the global t = measurement points
25     # dot(f, tmp) sums over measurements
26     H = array([\
27         [0., 0., 0.],\
28         [0., 0., -dot(f,tmp)],\
29         [0, -dot(f,tmp), x[1]*dot(f,t*tmp)]\
30     ])
31     H += dot(df.T,df) # don't forget to add the 'easy' part in the Hesse matrix
32     return H

33
34 GradPhi = lambda x: dot( DF(x).T, F(x)) #gradient of  $\Phi(x) = 1/2||F(x)||_2^2$ 

35
36 x_0s = [[1.5,1.5,0.1],[1.8,1.8,0.1]] # two possible initial values

37
38 #gauss-newton, we adapt the function to return also the corrections
39 def gn(x,F,J,tol):
40     #x is the starting vector that has to be intelligently chosen
41     #F: function
42     #J: Jacobi-Matrix
43     #tol: tolerance
44     s = lstsq(J(x),F(x),rcond=False)[0]
45     S = []; X = [] #container for corrections and temporary x values
46     S += [norm(s)]
47     X += [x]
48     x = x-s
49     #now we perform the iteration
50     while norm(s) > tol*norm(x):
51         #every time we update x by subtracting s, found with the least square
52         method
53         s = lstsq(J(x),F(x),rcond=False)[0]
54         S += [norm(s)]
55         X += [x]
56         x = x-s
57         #print(x)
58         return x, X, S

59
60 #now we plot the magnitude of the corrections
61 fig, ax1 = subplots()
62 ax2 = ax1.twinx()
63
64 for x_0 in x_0s:
65     x, X, S = gn(x_0, F, DF, 10**-12)
66     #print(S)
67     ax1.semilogy(arange(len(S)),[norm(F(x))**2 for x in X], "-x",
68     label='Residuum '+str(x_0))
69     ax2.semilogy(arange(len(S)),S, "--x", label= 'Correction' + str(x_0))
70     ax1.set_xlabel("No of step of Gauss-Newton method")
71     ax1.set_ylabel("$||F(x^{\{k\}})||_2^2$")
72     ax1.legend(loc="best") # ignored
73     ax1.grid(axis='both') # need to call on ax1 to see a grid in both directions
74
75     ax2.set_ylabel("Norm of the correction")
76     ax2.grid(linestyle=':') # different linestyle for the right grid

```



```

ax2.legend(loc="best")
76 show()

78 #now we finally plot the data together with the function that we obtained via
interpolation
ts = linspace(1.,7.,500) #for plot
80 figure()
xlabel("$t$"); ylabel("$y$"); title("Gauss-Newton fit")
82 plot(t,y,"ro")
plot(ts, f(x,ts), "g")
84 grid()
show()

86
88 #newton-method

89 def newton(x,GP,HP,tol):
90     #GP: gradient of  $\Phi$ . HP: Hesse Matrix of  $\Phi$ 
s = solve(HP(x),GP(x)) # Newton correction
92     S = []; X = [] #container for corrections and temporary x values
S += [norm(s)]
94     X += [x]
x -= s
96     while norm(s) > tol*norm(x):
s = solve(HP(x),GP(x)) # Newton correction
98     x -= s
S += [norm(s)]
100    X += [x]
return x, X, S

102
fig, ax1 = subplots()
104 ax2 = ax1.twinx()

106 for x_0 in x_0s:
x, X, S = newton(x_0, GradPhi, HessPhi, 10**-12)
108     #print(S)
ax1.semilogy(arange(len(S)), [norm(F(x))*2 for x in X], "-x",
label=str(x_0))
110     ax2.semilogy(arange(len(S)), S, "--x", label= 'Correction' + str(x_0))
ax1.set_xlabel("No of step of Newton method")
112 ax1.set_ylabel("$||F(x^{\{k\}})||_2^2$")
ax2.set_ylabel("Norm of the correction")
114 legend(loc="best")
ax1.grid(axis='both') # need to call on ax1 to see a grid in both directions
116 ax2.grid(linestyle=':')
show()

118
ts = linspace(1.,7.,500) #for plot
120 figure()
xlabel("$t$"); ylabel("$y$"); title("Newton fit")
122 plot(t,y,"ro")
plot(ts, f(x,ts), "g")
124 grid()
show()

126

```

```

#finally, damped newton
128 def dampnewton(x,GP,HP,q=0.5,tol=1e-10):

130     lup = lu_factor(HP(x)) #LU decomposition
    s = lu_solve(lup,GP(x)) #lu_solve is more efficient
132     xn = x-s
    lam = 1
134     st = lu_solve(lup,GP(xn))
    S = [] #container for corrections
136     X = []
    S += [norm(s)]
138     X += [xn]
    for k in range(15): #otherwise too many steps, d. newt. isn't a good
        technique for this problem...
140         #while norm(st) > tol*norm(xn):
        #(a posteriori termination criteria)
142         while norm(st) > (1-lam*0.5)*norm(s): #natural monotonicity test
            lam *= 0.5 # reduce damping factor
144             if lam < 1e-10:
                print('DAMPED NEWTON: Failure of convergence')
                return x, X, S
            xn = x-lam*s #simplified Newton
148             st = lu_solve(lup,GP(xn))
            x = xn
150             lup = lu_factor(HP(x))
            s = lu_solve(lup,GP(x))
152             lam = min(lam/q, 1.) #notice that we must have  $1 - \lambda/2 > 0$ 
            xn = x-lam*s #damped newton verfahren
154             S += [lam*norm(s)]
            X += [xn]
156             st = lu_solve(lup,GP(xn)) #simplified Newton, prepare st for the next
iteration

158     x = xn
    return x, X, S

160
fig, ax1 = subplots()
162 ax2 = ax1.twinx()

164 for x_0 in x_0s:
    x, X, S = dampnewton(x_0, GradPhi, HessPhi,0.5, 10**-12)
    #print(cvg)
    #print(S)
168     ax1.semilogy(arange(len(S)),[norm(F(x))*2 for x in X], "-x",
label=str(x_0))
    ax2.semilogy(arange(len(S)),S, "--x", label= 'Correction'+str(x_0))
170 ax1.set_xlabel("No of step of Newton method")
ax1.set_ylabel("$||F(x^{\{k\}})||_2^2$")
172 ax2.set_ylabel("Norm of the correction")
legend(loc="best")
174 ax1.grid(axis='both') # need to call on ax1 to see a grid in both directions
ax2.grid(linestyle=':')
176 show()

178 ts = linspace(1.,7.,500) #for plot

```

```

figure()
xlabel("$t$"); ylabel("$y$"); title("Damped Newton fit (with second starting
vector, iteration with first one does not converge)")
plot(t,y,"ro")
plot(ts, f(x,ts), "g")
grid()
show()

```

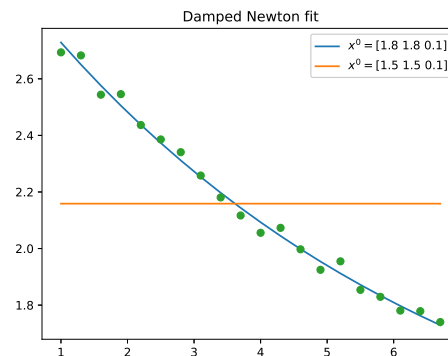
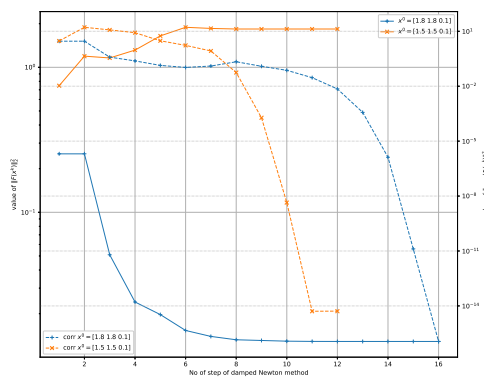
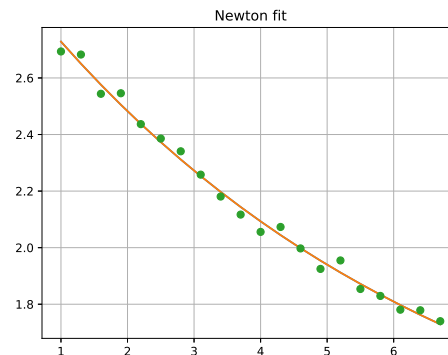
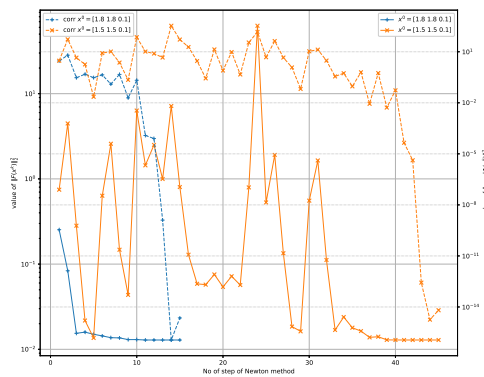
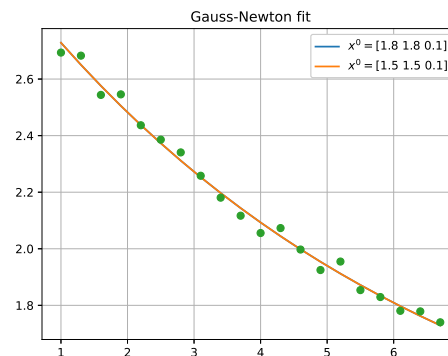
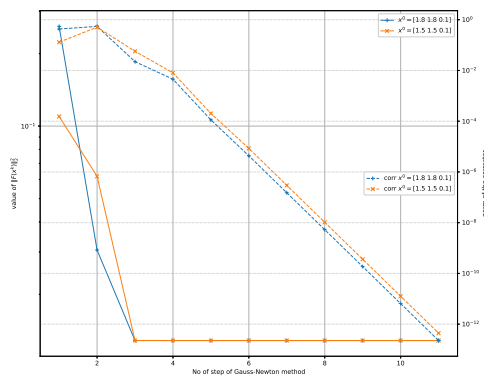


Abb. 8.2.8. Für den Anfangswert $(1.8, 1.8, 0.1)^T$ bleibt die Iteration des gedämpften Newton-Verfahrens in einem lokalen Minimum gefangen, so dass die gelieferte Lö-

sung weit weg von den wahren Parametern liegt; für den Anfangswert $(1.5, 1.5, 0.1)^T$ scheitert das gedämpfte Newton-Verfahren. Das ungedämpfte Newton-Verfahren konvergiert quadratisch. Das Gauss-Newton-Verfahren konvergiert linear in beiden Fällen.

Beispiel 8.2.9. Global Positioning System ist ein globales Navigationssatellitensystem zur Positionsbestimmung und Zeitmessung. GPS basiert auf Satelliten, die mit codierten Radiosignalen ständig ihre aktuelle Position und die genaue Uhrzeit ausstrahlen. Aus den Signallaufzeiten können GPS-Empfänger dann ihre eigene Position und Geschwindigkeit berechnen. Theoretisch reichen dazu die Signale von drei Satelliten aus, welche sich oberhalb ihres Abschaltwinkels befinden müssen, da daraus die genaue Position und Höhe bestimmt werden kann. In der Praxis haben aber GPS-Empfänger keine Uhr, die genau genug ist, um die Laufzeiten korrekt zu messen. Deshalb wird das Signal eines vierten Satelliten benötigt, mit dem dann auch die genaue Zeit im Empfänger bestimmt werden kann. ²

Das Signal des Satelliten i enthält seine Zeit und Position:

$$[t_{s,i}, x_{s,i}, y_{s,i}, z_{s,i}]$$

Wir können N Satelliten empfangen und speichern die Messwerte zeilenweise in $\mathbf{X} \in \mathbb{R}^{N \times 4}$. Die Zeit des Empfängers t_r ist im Vergleich zur Signallaufzeit nur unzureichend bekannt und muss ebenso wie die Position bestimmt werden, d.h. wir suchen:

$$\mathbf{x} := [t_r, x_r, y_r, z_r].$$

Dazu minimieren wir die Residuen r_i , $i = 1 \dots N$, die durch folgende Gleichung gegeben sind:

$$r_i := -c^2(t_r - t_{s,i})^2 + (x_r - x_{s,i})^2 + (y_r - y_{s,i})^2 + (z_r - z_{s,i})^2$$

wobei c die Lichtgeschwindigkeit im Vakuum darstellt.

Das entsprechende nichtlineare Ausgleichsproblem kann man mit der Gauss-Newton-Methode lösen. Dabei merkt man, dass die Genauigkeit der Positionsbestimmung vom Messfehler in der Signallaufzeit, der Anzahl sichtbarer Satelliten und ihrer Verteilung abhängt.

²Aus Wikipedia: <http://de.wikipedia.org/wiki/GPS>

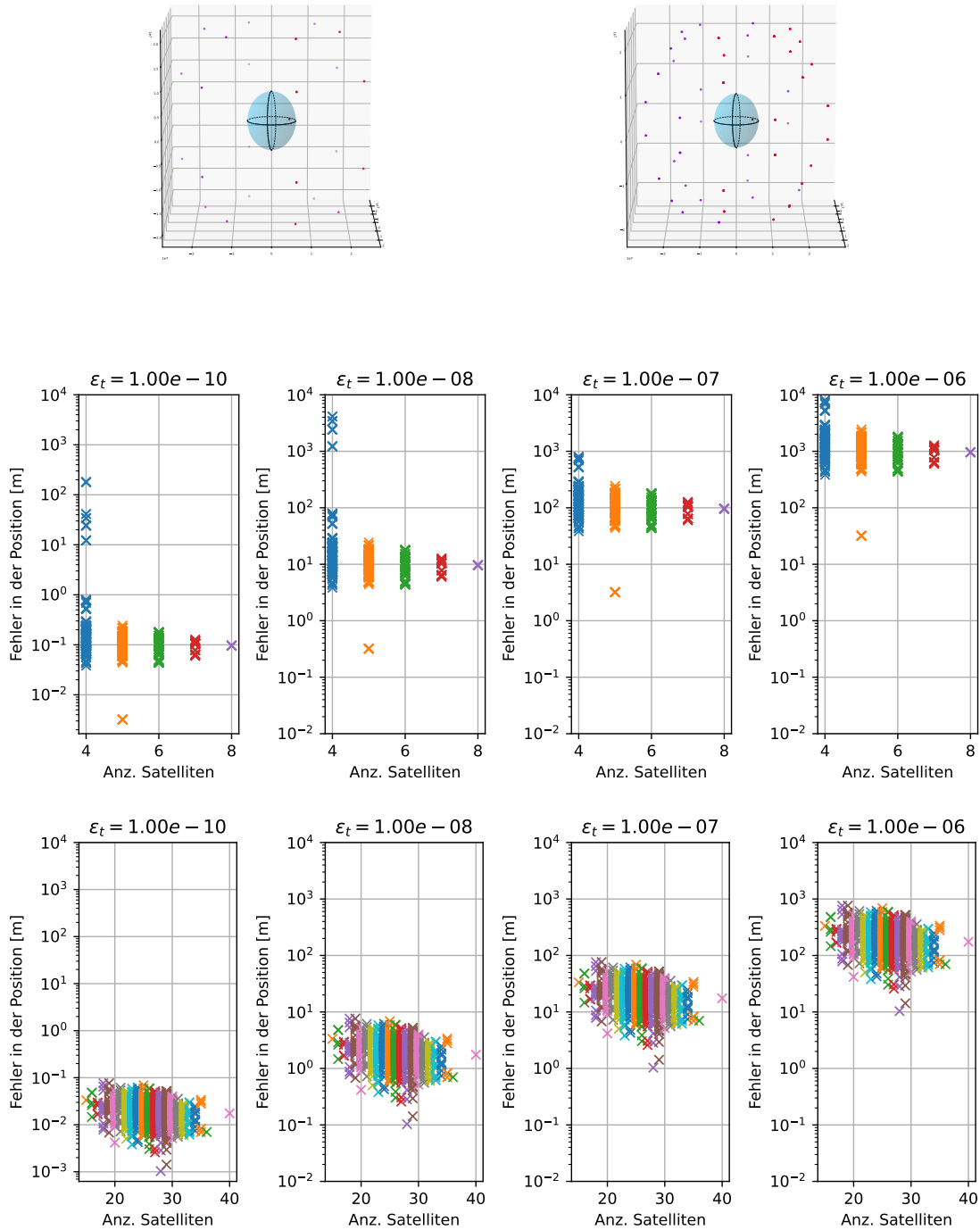


Abb. 8.2.10. Verschiedene Anzahlen von Satelliten und Lösungen des GPS-Problems.

8.2.3 Weitere Methoden: BFGS, GD, SGD, CG, LM, ADAM

Wir haben gesehen, dass die nichtlineare Ausgleichsrechnung ein nichtlineares Optimierungsproblem ist:

$$\operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^n} \Phi(\mathbf{x})$$

für $\Phi(\mathbf{x}) := \frac{1}{2} \|F(\mathbf{x})\|_2^2$. Alle Methoden bauen Iterationen der Form

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{s}$$

und unterscheiden sich nur durch die Wahl und die Berechnungsart des Updates $\mathbf{s}$. Das optimale $\mathbf{x}$ erfüllt die Bedingung $D\Phi(\mathbf{x}) = 0$, so dass alle Methoden für das Nullstellenproblem zur Anwendung kommen können. Das Newton-Verfahren schlägt die Korrektur $\mathbf{s}$ aus dem linearen Gleichungssystem mit der Matrix $D^2\Phi$ vor:

$$D^2\Phi(\mathbf{x}^{(k)})\mathbf{s} = D\Phi(\mathbf{x}^{(k)}).$$

Die Berechnung der zweiten Ableitung von Φ (und das anschliessende Lösen des linearen Gleichungssystems) kann sehr teuer bis unmöglich sein. Darum wurde die Gauss-Newton-Methode als Alternative eingeführt. Nun wollen wir nach weiteren Alternativen suchen.

Statt einer Linearisierung des Modells in jedem Schritt (Gauss-Newton) können wir auch eine schnelle Variante der Broyden-Methode verwenden, z.B. die Erweiterung des Broyden-Algorithmus mit den SMW-Formeln und Rändern, den **BFGS**-Algorithmus. Die rechte Seite ist selbst nicht ganz günstig zu berechnen: $D\Phi(\mathbf{x}^{(k)}) = D\mathbf{F}(\mathbf{x}^{(k)})^T \mathbf{F}(\mathbf{x}^{(k)})$.

Noch langsamer in der Konvergenz, aber etwas kostengünstiger, ist die Gauss-Newton-Methode: Das Update $\mathbf{s}$ wird als Lösung des linearen Ausgleichsproblems

$$\operatorname{argmin}_{\mathbf{s} \in \mathbb{R}^n} \frac{1}{2} \|\mathbf{F}(\mathbf{x}^{(k)}) - D\mathbf{F}(\mathbf{x}^{(k)})\mathbf{s}\|^2$$

konstruiert. Die Lösung mit der Normalengleichung liefert ein Update in Form der Lösung des folgenden linearen Gleichungssystems:

$$D\mathbf{F}(\mathbf{x}^{(k)})^T D\mathbf{F}(\mathbf{x}^{(k)})\mathbf{s} = D\mathbf{F}(\mathbf{x}^{(k)})^T \mathbf{F}(\mathbf{x}^{(k)}).$$

Die rechte Seite ist immer noch $D\Phi(\mathbf{x}^{(k)})$, aber die Systemmatrix ist nun $D\mathbf{F}(\mathbf{x}^{(k)})^T D\mathbf{F}(\mathbf{x}^{(k)})$ statt $D^2\Phi(\mathbf{x}^{(k)})$.

Da es sich um ein Optimierungsproblem handelt, kann man noch rechengünstigere Iterationen auf Kosten der Konvergenzgeschwindigkeit konstruieren, z.B. indem man das Update $\mathbf{s}$ in Richtung des steilsten Abstiegs nimmt. Diese Idee liefert die Klasse der Gradientenverfahren (**Gradient Descent**), die billiger und langsamer in der Konvergenz als die vorigen Methoden sind:

$$\mathbf{s} = \lambda_k D\Phi(\mathbf{x}^{(k)}).$$

In Machine Learning wird λ_k “learning rate” genannt; sie kann konstant für alle Schritte gewählt oder in jedem Schritt angepasst werden. Man kann z.B. in jedem Schritt λ_k so wählen, dass das eindimensionale Optimierungsproblem

$$\operatorname{argmin}_{\lambda \in \mathbb{R}} \Phi(\mathbf{x}^{(k)} - \lambda D\Phi(\mathbf{x}^{(k)}))$$

gelöst wird. Die Dimensionalität des Problems in Machine Learning kann so gross sein, dass auch dies viel zu teuer ist, so dass dort oft ein festes λ verwendet wird. Ein in jedem Schritt optimiertes λ kann unter Umständen sogar kontraproduktiv sein: In einem engen steilen Tal liefert Gradient Descent eine sehr langsam konvergente Folge, die viele winzige Zickzacks macht. Z.B. liefert für $\Phi(\mathbf{x}) = \frac{1}{2}(x_1^2 + bx_2^2)$ Gradient Descent $\mathbf{x}^{(k+1)} = [(1 - \lambda)x_1^{(k)}, (1 - b\lambda)x_2^{(k)}]^T$. Die optimale Wahl von λ in jedem Schritt liefert den Zickzack $\mathbf{x}^{(k)} = r^k[(-1)^k b, 1]^T$ mit $0 < r = \frac{1-b}{1+b} < 1$ und $\Phi(x^{(k)}) = r^{2k}\Phi(x^{(0)})$, was umso langsamer konvergiert, je kleiner b ist. In Abbildung 8.2.12 sind die Iterationen und die Ergebnisse verschiedener Verfahren für die Wahl $b = 0.02$ dargestellt. Die gelbe Zickzack-Linie zeigt die Schwäche dieser Methode. Beim Betrachten der Trajektorien in den Niveaulinien sollte man auch die unterschiedliche Skalierung in der x_1 - und x_2 -Richtung beachten: Wir haben mit einem sehr engen Tal zu tun.

Es gibt mehrere Auswege aus dem Dilemma des Zickzacks im Gradient Descent. Falls man sie sich leisten kann, ist die Methode der konjugierten Gradienten (**CG**) deutlich besser, aber unter Umständen auch zu teuer. **CG** ist speziell dafür konstruiert, die lokale Geometrie der Minimierungsfunktion bestmöglich auszunutzen (wie bei dem obigen engen Tal).

Eine andere Möglichkeit ist es, Gradient Descent mit der Gauss-Newton-Methode zu kombinieren. Die ersten Schritte (vor dem Zickzack) von Gradient Descent können die Iteration in die Nähe eines (lokalen) Minimums bringen, so dass Gauss-Newton einen guten Startpunkt bekommt. Zusammen mit einer heuristischen Dämpfung führt diese Überlegung zur **Levenberg-Marquardt-Methode**: Anstatt

$\|F(\mathbf{x}^{(k)}) + DF(\mathbf{x}^{(k)})\mathbf{s}\|^2$ zu minimieren, wird

$$\|F(\mathbf{x}^{(k)}) - DF(\mathbf{x}^{(k)})\mathbf{s}\|^2 + \lambda \|\mathbf{s}\|_2^2$$

minimiert. Der Dämpfungsparameter λ bestraft also grosse Schritte: Wenn diese in die falsche Richtung gehen, sind sie zumindest nicht lang; so besteht die Hoffnung, noch im Konvergenzbereich zu bleiben. Die modifizierte Gleichung für die Korrektur lautet bei dieser Methode:

$$(DF(\mathbf{x}^{(k)})^T DF(\mathbf{x}^{(k)}) + \lambda \mathbf{I}) \mathbf{s} = DF(\mathbf{x}^{(k)})^T F(\mathbf{x}^{(k)}) . \quad (8.2.14)$$

Eine andere Methode, die bei sehr grossen Dimensionen (wie in Machine Learning) meistens verwendet wird, besteht darin, zufällig Teile von $D\Phi(\mathbf{x}^k)$ zu wählen und zu verwenden (stochastic gradient, stochastischer Gradientenabstieg). Dies ist die extrem billigste und, wenn überhaupt, am langsamsten konvergierende Methode.

Ein guter Kompromiss, der in Machine Learning sehr viel Erfolg hat, befindet sich in der Klasse von Methoden, die Momentum verwenden. Die Idee kommt aus der

Mechanik: Eine sehr schwere Kugel wird in einem engen steilen Tal weniger Zickzacks hinterlassen. So entsteht die Idee, eine künstliche Trägheit in die Iteration einzubauen. Man läuft also nicht mehr in Richtung des steilsten Abstiegs, sondern wählt

$$\mathbf{s} = \lambda \mathbf{z}^k \text{ mit } \mathbf{z}^k = D\Phi(\mathbf{x}^{(k)}) + \beta \mathbf{z}^{(k-1)}.$$

Die berühmte **ADAM**-Methode wählt spezielle λ und β und liefert die einfache Iteration

$$\begin{aligned} \mathbf{x}^{(k+1)} &= \mathbf{x}^k - \lambda \mathbf{z}^k \\ \mathbf{z}^{(k+1)} &= D\Phi(\mathbf{x}^{(k)}) + \beta \mathbf{z}^{(k)}, \end{aligned}$$

was eine verblüffende Ähnlichkeit zur Verlet-Methode für die Newtonsche Gleichung in der Mechanik aufweist.

Die Iterationen und Ergebnisse einiger dieser Verfahren für die Minimierung sehen wir in der Abbildung 8.2.12 für die quadratische Funktion mit engem Tal ($b = 0.02$) und in der Abbildung 8.2.13 für eine nichttriviale Funktion in 2 Dimensionen: $\Phi(x, y) = (1 + (3x - 2.3)^3)^2 + (1 + (0.7 - 1.5y)^3)^2$. Im quadratischen Fall sehen wir keinen wesentlichen Unterschied in der Konvergenzgeschwindigkeit zwischen ADAM und Gradient Descent mit Line Search, aber ADAM braucht viel weniger Zeit: 1.5 ms gegenüber 32 ms (GDls). Gradient Descent mit Momentum und Line Search (GDM) braucht auch 32 ms bis zur selben Genauigkeit $1e - 12$, konvergiert jedoch viel schneller, d.h. bei Genauigkeit $2e - 7$ ist es dank weniger Iterationen schneller als GDls: 10 ms (GDM) versus 15 ms (GDls). Das ist auch mit 0.6 ms von ADAM zu vergleichen, der bei dieser geringeren Genauigkeit fast an die Rechenzeit der `optimize.minimize` von 0.5 ms herankommt; das ist beachtlich, denn unsere naive Implementierung von ADAM ist gar nicht auf Rechenzeit oder Rechenstrategien optimiert, im Vergleich zu den professionellen `scipy`-Methoden (hier BFGS in FORTRAN). Das sieht man noch deutlicher in der Abbildung 8.2.13 für $\Phi(x, y) = (1 + (3x - 2.3)^3)^2 + (1 + (0.7 - 1.5y)^3)^2$; `optimize.minimize` braucht 0.84 ms (dank analytisch gegebener Ableitungen), ADAM braucht 0.88 ms und GDM 22.7 ms, alles bei der Toleranz $1e - 6$. GDls brauchte 75.8 ms für 1000 Iterationen ohne zu konvergieren: Es bleibt in einem lokalen Minimum gefangen, auch wenn man mehr Iterationen zulässt.

In mehreren Dimensionen und wenn die Ableitungen nicht mehr so günstig zur Verfügung stehen, gewinnt ADAM dank deutlich kürzerer Rechenzeiten gegen BFGS. Das macht ADAM zur am häufigsten verwendeten Methode in Machine Learning.

In `python` verwendet man die Funktion `scipy.optimize.leastsq`, wo man auch unter einigen Iterationsmethoden auswählen kann. Mehrere Wahlmöglichkeiten für die Iteration entstehen aber, wenn man das Problem als allgemeines Optimierungsproblem schreibt und `scipy.optimize.minimize` verwendet.

Code 8.2.11: Einige Minimierungsfunktionen

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize

verbose = False
#verbose = True
NMAX = 1000
TOL = 1e-12 # try 1e-6 too
# -----
# define function and compute its derivatives

import sympy
x, y = sympy.symbols("x, y")
# actual function to minimize
b = 2*1e-2
F_mat = sympy.Matrix([0.5*(x**2 + b*y**2)])
# analytic computations
J_mat = F_mat.jacobian(sympy.Matrix([x, y]))
H_mat = J_mat.jacobian(sympy.Matrix([x, y]))
print('F(x,y)=\n', F_mat)
print('DF(x,y)=\n', J_mat)
print('---')
print('DDF(x,y)=\n', H_mat)
F = sympy.lambdify((x,y),F_mat, 'numpy')
J = sympy.lambdify((x,y),J_mat, 'numpy')
H = sympy.lambdify((x,y),H_mat, 'numpy')
def f(x):
    return(F(*x).ravel())
def df(x):
    return(J(*x).ravel())
def ddf(x):
    return(H(*x))

# aux to keep the evolution of the iteration
class FollowEv:
    def __init__(self, nomgv):
        self.nomgv = nomgv
    def __call__(self, xk):
        self.nomgv.append(xk.copy())
    def getdata(self):
        return self.nomgv

# aux to easily build the function needed for line search
class LocFunc:
    def __init__(self, f, x, dfx):
        self.f = f
        self.x = x
        self.dfx = dfx
    def __call__(self, s):
        return self.f(self.x - s*self.dfx)
    def set(self, x, dfx):
        self.x = x

```

```

        self.dfx = dfx
54
# gradient descent with line search
56 def gdls(f, x0, jac, Nmax=NMAX, callback=None, tol=TOL):
    df = jac
58     step0 = 1e-4
    xn = x0.copy()
60     fx = f(xn)
    lf = LocFunc(f, xn, df(xn))
62     k = 0
    while( k < Nmax and abs(fx[0]) > tol):
64         dfxn = df(xn)
        lf.set(xn, dfxn)
66         lsres = optimize.minimize_scalar(lf)
        if verbose: print('line search gives:', lsres[-1])
68         if lsres.success:
            step = lsres.x
70             if verbose: print('line_search worked, step=', step)
        else:
72             if verbose: print('line search did not converged')
            step = step0
74         if verbose: print('k=', k, 'step=', step)
        xn -= step*dfxn
76         fx = f(xn)
        if callback != None: callback(xn)
78         if verbose: print(k,xn, fx)
        k += 1
80     return xn

82 # -----
# gradient descend with momentum (and line search)
84 def gdm(f, x0, jac, beta=0.9, lam = 1e-4, Nmax=NMAX, callback=None, tol=TOL):
    df = jac
86     step0 = lam
    xn = x0.copy()
88     fx = f(xn)
    dfxn = df(xn)
90     zn = 0.*xn # initialize momentum with zeros
    dof1s = dfxn
92     lf = LocFunc(f, xn, dfxn)

94     k = 0
    while( k < Nmax and abs(fx[0]) > tol):
96         lf.set(xn, dof1s)
        lsres = optimize.minimize_scalar(lf)
98         if verbose: print('line search gives:', lsres[-1])
        if lsres.success:
100             step = lsres.x
            if verbose: print('line_search worked, step=', step)
102         else:
            if verbose: print('line search did not converged')
            step = step0
104         if verbose: print('k=', k, 'step=', step)
        xn -= step*dof1s # new point
106

```

```

108     dfxn = df(xn) # gradient at new point
109     zn = (1-beta)*dfxn + beta*zn # new momentum
110     dof1s = 1.*zn # direction of next line search
111     fx = f(xn) # new func value
112     if callback != None: callback(xn)
113     if verbose: print(k,xn, fx)
114     k += 1
115     return xn
116
117 # -----
118 # adam: no line search
119 def adam(f, x0, jac, beta1=0.9, beta2=0.999, alpha = 1e-1, Nmax=NMAX,
120         callback=None, tol=TOL):
121     df = jac
122     xn = x0.copy()
123     fx = f(xn)
124     dfxn = df(xn)
125     m = 0.*xn; v = 0.*xn # initialize moments with 0s
126     k = 0
127     while( k < Nmax and abs(fx[0]) > tol):
128         m = beta1*m + (1-beta1)*dfxn
129         v = beta2*v + (1-beta2)*dfxn**2
130         mh = m/( 1-beta1**(k+1) ) # eliminate bias
131         vh = v/( 1-beta2**(k+1) )
132         xn -= alpha*mh/(np.sqrt(vh) + 1e-8)
133         fx = f(xn)
134         dfxn = df(xn)
135         if callback != None: callback(xn)
136         if verbose: print(k,xn, fx)
137         k += 1
138     return xn
139
140 # -----
141 def runmin(f, x0, df=None, method=optimize.minimize, tol=TOL):
142     print('runmin using method =',method.__name__)
143     EvMin = [x0]
144     followev = FollowEv(EvMin)
145     # call a solver with this callback
146     method(f, x0, jac = df, callback=followev, tol=tol)
147     EvMin = np.array(followev.getdata())
148     return(EvMin)
149
150 # -----
151 # plot the function
152 def func_X_Y_to_XY(f, X, Y):
153     """
154     Wrapper for f(X, Y) -> f([X, Y])
155     """
156     s = np.shape(X)
157     return f(np.vstack([X.ravel(), Y.ravel()])).reshape(*s)
158
159 # prepare grid to plot
160 x_ = np.linspace(-5*b, 5*b, 100)
161 y_ = np.linspace(-0.4, 1+2*b, 500)

```

```

160 X, Y = np.meshgrid(x_, y_)

162 # plot f as contour on the grid X, Y
fig, ax = plt.subplots(figsize=(12, 8))
164 #ax.axis('equal')
c = ax.contour(X, Y, func_X_Y_to_XY(f, X, Y), 100)
166 from matplotlib import cm, ticker
print('-----\ncontourf on so many points may take a while, please wait...\n')
168 c = ax.contourf(X, Y, func_X_Y_to_XY(f, X, Y), locator=ticker.LogLocator())
ax.set_xlabel(r"$x_1$", fontsize=18)
170 ax.set_ylabel(r"$x_2$", fontsize=18)
plt.colorbar(c, ax=ax)

172 # set a starting point
174 x_start = np.array([2*b, 1.])

176 # optimize with BFGS
EvOpt = runmin(f, x_start, df = df, tol=1e-6)
178 vals = func_X_Y_to_XY(f,EvOpt[:,0], EvOpt[:,1])
errX01 = np.linalg.norm(EvOpt,axis = 1)
180 vals01 = vals
# plot the optimization path
182 ax.plot(EvOpt[:,0], EvOpt[:,1],'+-', color='red', label='BFGS')

184 # optimise with GDls
EvOpt = runmin(f, x_start, df = df, method=gdls)
186 #print(EvOpt)
vals = func_X_Y_to_XY(f,EvOpt[:,0], EvOpt[:,1])
188 errX02 = np.linalg.norm(EvOpt,axis = 1)
# plot the optimization path
190 ax.plot(EvOpt[:,0], EvOpt[:,1],'x-', color='orange', label='GDls')

192 # optimise with GDMls
EvOpt = runmin(f, x_start, df = df, method=gdm)
194 #print(EvOpt)
valsGD = func_X_Y_to_XY(f,EvOpt[:,0], EvOpt[:,1])
196 errX03 = np.linalg.norm(EvOpt,axis = 1)
# plot the optimization path
198 ax.plot(EvOpt[:,0], EvOpt[:,1],'x-', color='magenta', label='GDMls')

200 # optimise with ADAM
EvOpt = runmin(f, x_start, df = df, method=adam)
202 #print(EvOpt)
valsADAM = func_X_Y_to_XY(f,EvOpt[:,0], EvOpt[:,1])
204 errXadam = np.linalg.norm(EvOpt,axis = 1)
# plot the optimization path
206 ax.plot(EvOpt[:,0], EvOpt[:,1],'x-', color='black', label='ADAM')
plt.legend()

208 plt.savefig('zickzackNLsq.eps'); plt.savefig('zickzackNLsq.pdf');
plt.savefig('zickzackNLsq.png')
210 plt.show()

212 plt.semilogy(vals01,'+-',color='red',label='BFGS')

```

```
plt.semilogy(vals, 'x-', color='orange', label='GDls')
214 plt.semilogy(valsGD, 'x-', color='magenta', label='GDMls')
plt.semilogy(valsADAM, 'x-', color='black', label='ADAM')
216 plt.legend()
plt.xlabel('Iteration number')
218 plt.ylabel('Residuum')
plt.grid()
220
plt.savefig('zickzackNLsqRes.eps'); plt.savefig('zickzackNLsqRes.pdf');
plt.savefig('zickzackNLsqRes.png')
222 plt.show()
```

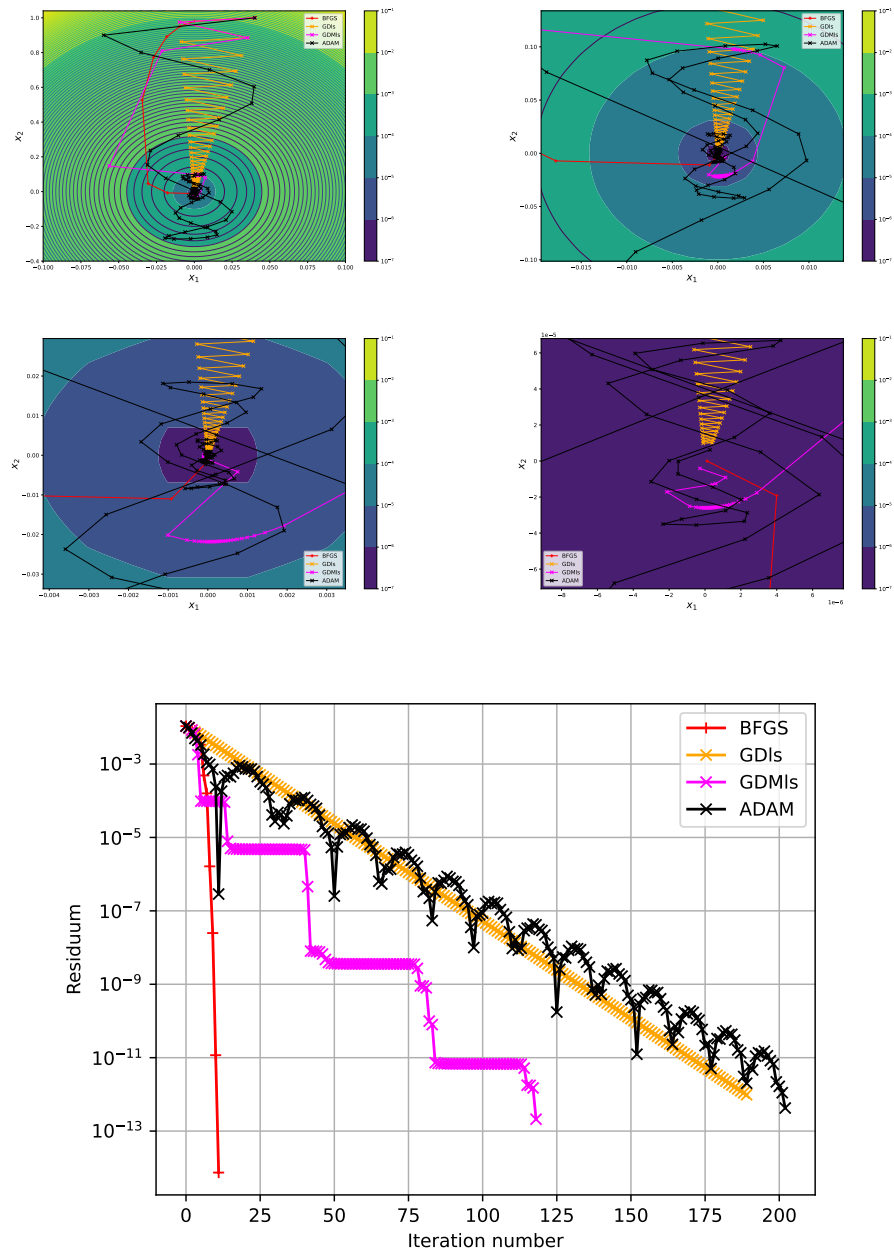


Abb. 8.2.12. Konvergenzverhalten in einer quadratischen Zielfunktion.

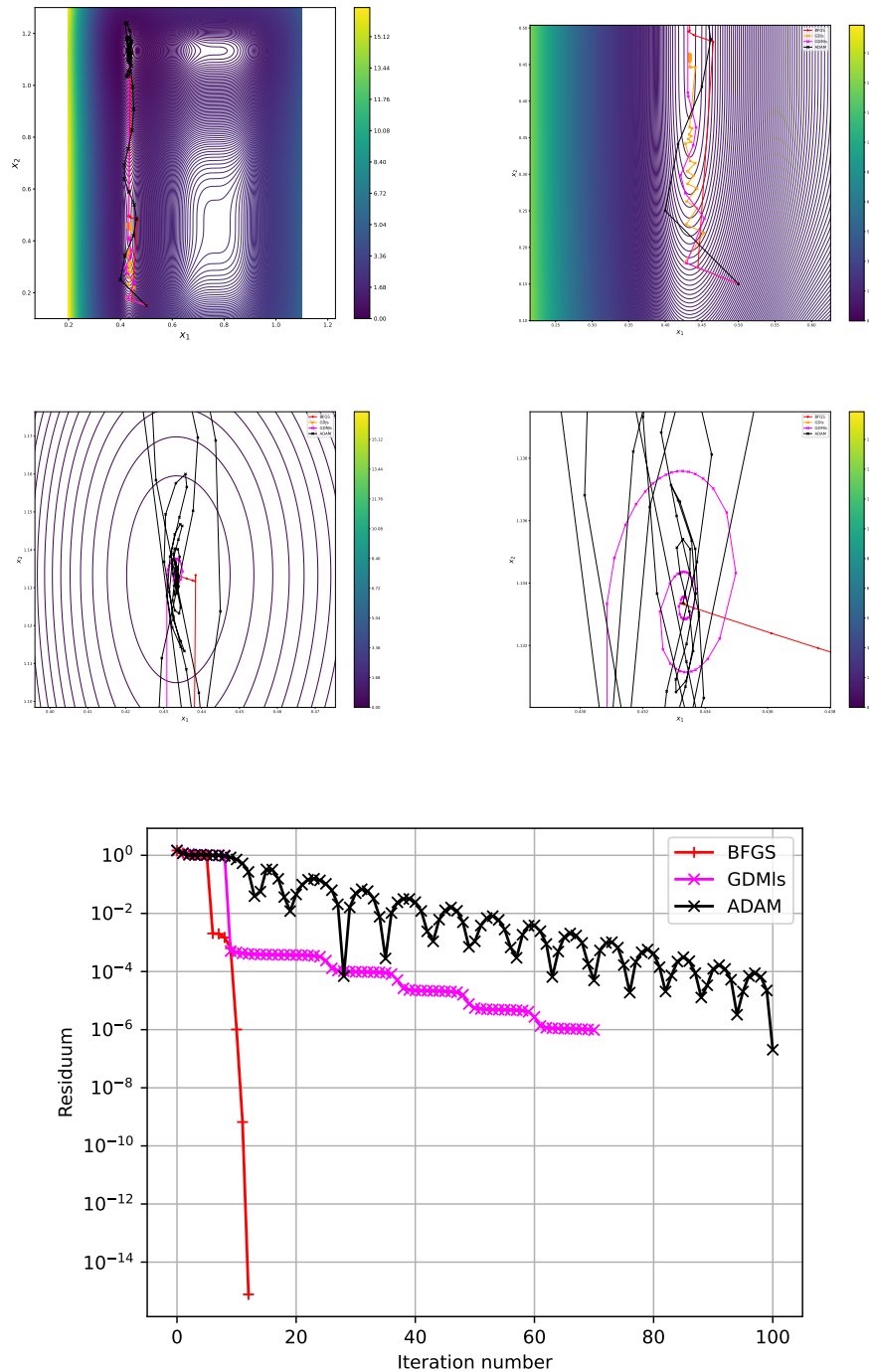


Abb. 8.2.13. Konvergenzverhalten in $\Phi(x, y) = (1 + (3x - 2.3)^3)^2 + (1 + (0.7 - 1.5y)^3)^2$.

8.3 DNNs

Deep Neuronal Networks (DNNs) sind ein Konstrukt, das im Machine Learning verwendet wird und einen Spezialfall der Ausgleichsrechnung darstellt.

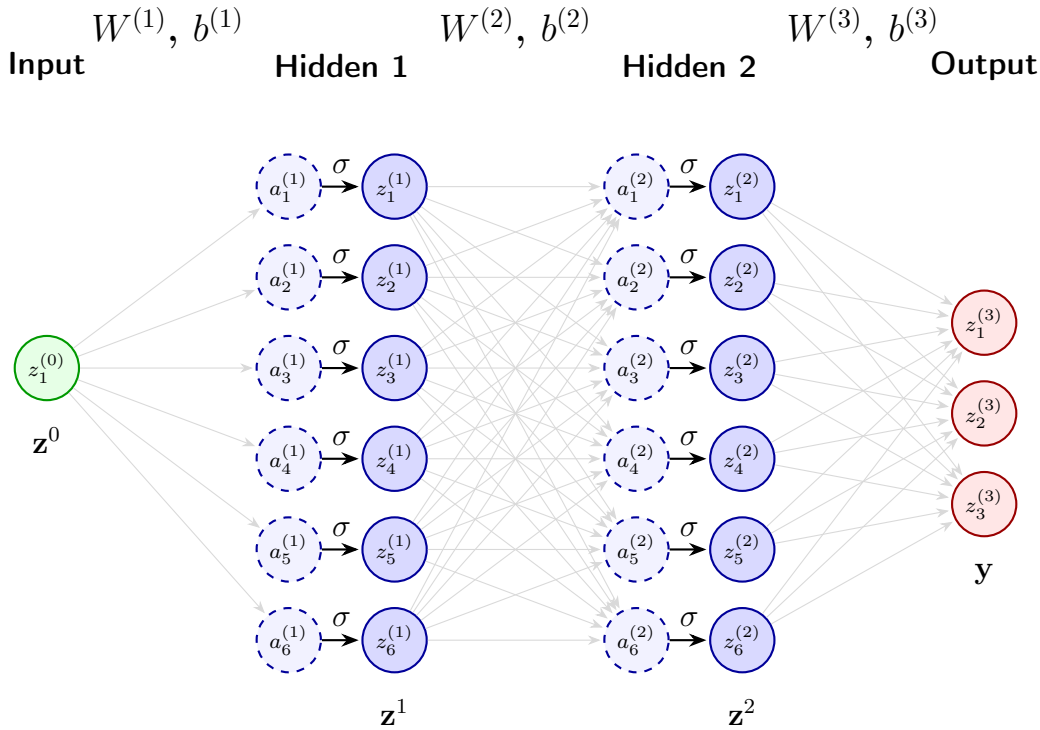


Abb. 8.3.1. Standarddarstellung eines DNNs mit Gestalt [1, 6, 6, 3]

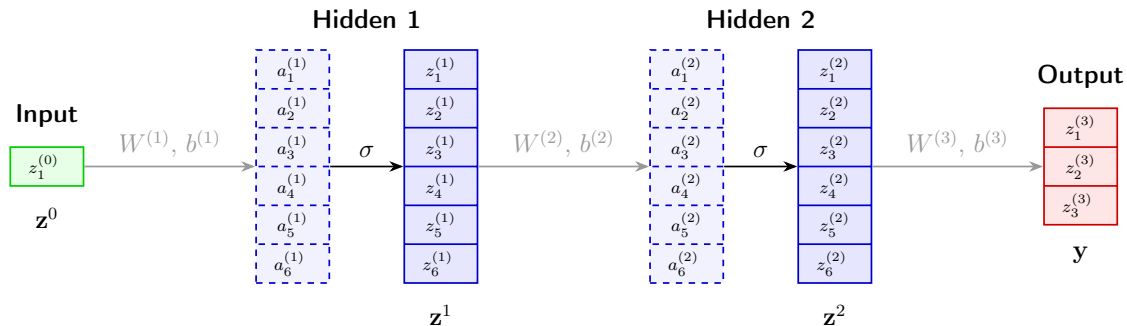


Abb. 8.3.2. Kompakte Darstellung eines DNNs mit Gestalt [1, 6, 6, 3]

Ein Neuronales Netz besteht aus mehreren Schichten (Layers) von Neuronen. Ein typisches Bild wie Abbildung 8.3.1 zeigt ein solches Netz: Die Eingangsschicht (Input) hat nur ein Neuron, die Ausgangsschicht (Output) hat 3 Neuronen, und dazwischen liegen zwei Zwischenschichten (Hidden Layers) mit je 6 Neuronen. Im Allgemeinen hat ein DNN mehrere Zwischenschichten, und die Anzahl Neuronen in einer Schicht kann ebenfalls unterschiedlich sein. Es ist sinnvoll, mathematische Notation zu verwenden, damit die Komplexität des Konstrukts reduziert erscheint: Alle Werte der Neuronen in einer Schicht bilden einen Vektor $\mathbf{z}$; von einer Schicht zur nächsten fließt die Information mittels einer affinen Abbildung

$$\mathbf{a}^{(j)}(\mathbf{z}^{(j-1)}) = \mathbf{a}^{(j)} := \mathbf{b}^{(j)} + \mathbf{W}^{(j)}\mathbf{z}^{(j-1)}$$

gefolgt von einer nichtlinearen Aktivierungsfunktion

$$\mathbf{z}^{(j)} = \sigma(\mathbf{a}^j).$$

Ein Vektor $\mathbf{b}^{(j)}$ heisst “Bias”, die Matrizen $\mathbf{W}^{(j)}$ heissen Gewichte, und alle müssen die entsprechenden Dimensionen d_j bzw. $d_{j-1} \times d_j$ haben. Die Aktivierungsfunktion σ ist eine einfache nichtlineare Funktion von $\mathbb{R}$ nach $\mathbb{R}$; Wikipedia gibt viele Beispiele für Funktionen, die im Machine Learning an dieser Stelle eingesetzt werden. Mit diesen Notationen vereinfacht sich die schematische Darstellung eines DNNs wesentlich, siehe Abbildung 8.3.2. Die Tiefe eines DNNs ist die Anzahl der Zwischenschichten. Das hier abgebildete DNN hat also Tiefe 2 und die Gestalt $[1, 6, 6, 3]$.

Die Menge aller Werte aus den Biases und aus den Gewichten ist die Menge der Parameter $\Theta \subset \mathbb{R}^D$ mit der Dimension $D = \sum_{j=1}^{K+1} d_j(1 + d_{j-1})$ für ein DNN mit K Zwischenschichten. Das ist die Dimension des Raumes, in dem wir die optimale Lösung suchen. DNNs sind erfolgreicher, wenn tiefe und breite Netze eingesetzt werden. Dies führt zu riesigen Dimensionen D und verlangt nach approximativen Lösungen sehr schwieriger Optimierungsprobleme. In unserem Beispiel haben wir $D = (6 * 1 + 6) + (6 * 6 + 6) + (6 * 3 + 3) = 12 + 42 + 21 = 75$.

Die mathematische Darstellung eines DNNs ist die Hintereinanderausführung affiner Operationen $a^{(k)}$ und punktwiser Aktivierungsfunktionen σ :

$$f^*(t; \theta) = a^{(K+1)} \circ \sigma \circ \dots \circ \sigma \circ a^{(2)} \circ \sigma \circ a^{(1)}(t).$$

Nehmen wir an, unsere Zielfunktion ist $f : \mathbb{R} \rightarrow \mathbb{R}^3$. Zu bestimmten $t_m \in \mathbb{R}$ (M Elemente der Trainingsmenge) haben wir Messungen $y_m \approx f(t_m)$. Das Ziel ist, ein Parameterset $\theta \in \Theta$ zu finden, so dass das entsprechende DNN die gemessenen Werte für die Trainingsmenge gut approximiert:

$$f^*(t_m; \theta) \approx y_m \text{ für } m = 1, \dots, M.$$

Wenn wir ein solches optimales θ gefunden haben, dann können wir das DNN $f^*(t; \theta)$ für beliebige t auswerten, um günstig Voraussagen für die sonst unbekannten oder nur schwer erhältlichen Werte von $f(t)$ zu erhalten. Abhängig von der Problemstellung und den Zielen gibt es verschiedene Zielfunktionen, die zu minimieren sind. Die naheliegende Wahl für ein einfaches Darstellungsproblem ist die euklidische Norm des Residuums:

$$J(\theta) = \frac{1}{M} \sum_{m=1}^M (\|f^*(t_m; \theta) - y_m\|^2)^p.$$

Für $p = 2$ erhalten wir die Standardmethode der kleinsten Quadrate; $p = 1$ favorisiert dünn besetzte Lösungen. Die zuvor diskutierten Methoden kommen zum Einsatz, wobei ADAM die erfolgreichste Methode ist.

Es ist wichtig zu wissen, dass DNNs am besten für stark unterdeterminierte Systeme funktionieren, d.h. $D \gg M$, und gar nicht für überdeterminierte Systeme ($D \leq M$). Die Kunst der “künstlichen Intelligenz” besteht darin, aus der grossen

Menge möglicher Lösungen eine zu finden, die sich auf neue Daten oder Messungen verallgemeinern lässt.

Die Schlüsselideen der DNNs sind:

- Nacheinanderausführung sehr vieler sehr einfacher Operationen (“go deep”).
- Nichtlinearität der Aktivierungsfunktion, die jedoch einfach genug ist, dass man sie und ihre Ableitung sehr günstig auswerten kann.
- Verwende die Kettenregel, um Ableitungen nach den Parametern exakt automatisch (nicht analytisch!) zu berechnen; so stehen mögliche “Abstiegsrichtungen” zur Verfügung.
- optimal schnelle Anwendung der Kettenregel: “backpropagation” ist **linear** in der Anzahl Gewichte $\sum_{j=1}^{K+1} d_j^2$.

Alle diese Eigenschaften sind notwendig, doch die *Backpropagation* ist entscheidend: Einerseits ist die Berechnung der Ableitungen nach den Parametern dank Kettenregel und exakter Ableitung der einfachen Aktivierungsfunktion ganz genau, andererseits ist sie schnell. Finite Differenzen wären ungenau und numerisch sehr instabil und zudem quadratisch in der Anzahl Gewichte, was die Verwendung tiefer Netze verunmöglichen würde.

Beispiel 8.3.3. (Konkrete *Backpropagation* für das DNN $[1, 6, 6, 3]$)

Wir betrachten ein tiefes neuronales Netzwerk (DNN) mit der spezifischen Architektur $[1, 6, 6, 3]$. Dies bedeutet:

- **Eingabeschicht:** 1 Neuron
- **Zwischenschicht 1:** 6 Neuronen
- **Zwischenschicht 2:** 6 Neuronen
- **Ausgabeschicht:** 3 Neuronen

Das Ziel der Backpropagation ist es, die partiellen Ableitungen der Zielfunktion J bezüglich aller Gewichte und Bias-Terme effizient zu berechnen, indem die Kettenregel der Differentialrechnung angewendet wird.

Wir verwenden die Notation von Zeilenvektoren (analog zur Implementierung in numpy), wobei gilt: $\mathbf{x}W + \mathbf{b}$.

- **Eingabe:** $\mathbf{z}^{(0)} \in \mathbb{R}^{1 \times 1}$
- **Schicht 1 (Hidden):**
 - Gewichtsmatrix $W^{(1)} \in \mathbb{R}^{1 \times 6}$
 - Bias-Vektor $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times 6}$
- **Schicht 2 (Hidden):**

- Gewichtsmatrix $W^{(2)} \in \mathbb{R}^{6 \times 6}$
- Bias-Vektor $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times 6}$

• **Schicht 3 (Ausgabe):**

- Gewichtsmatrix $W^{(3)} \in \mathbb{R}^{6 \times 3}$
- Bias-Vektor $\mathbf{b}^{(3)} \in \mathbb{R}^{1 \times 3}$

Der Vorwärtsthroughlauf (Forward Pass)

Das Netzwerk wird als Verkettung von Funktionen modelliert. Sei $\sigma(\cdot)$ die nichtlineare Aktivierungsfunktion (elementweise angewendet).

Die lineare Prä-Aktivierung $\mathbf{a}^{(1)}$ und die Aktivierung $\mathbf{z}^{(1)}$:

$$\mathbf{a}^{(1)} = \mathbf{z}^{(0)}W^{(1)} + \mathbf{b}^{(1)} \quad (8.3.1)$$

$$\mathbf{z}^{(1)} = \sigma(\mathbf{a}^{(1)}) \quad (8.3.2)$$

$$\mathbf{a}^{(2)} = \mathbf{z}^{(1)}W^{(2)} + \mathbf{b}^{(2)} \quad (8.3.3)$$

$$\mathbf{z}^{(2)} = \sigma(\mathbf{a}^{(2)}) \quad (8.3.4)$$

Wir nehmen für die Ausgabeschicht eine lineare Aktivierung an (für Interpolationsaufgaben):

$$\mathbf{a}^{(3)} = \mathbf{z}^{(2)}W^{(3)} + \mathbf{b}^{(3)} \quad (8.3.5)$$

$$\hat{\mathbf{y}} = \mathbf{a}^{(3)} \quad (8.3.6)$$

Wir verwenden den mittleren quadratischen Fehler (MSE):

$$J = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2 = \frac{1}{2} \sum_{k=1}^3 (\hat{y}_k - y_k)^2 \quad (8.3.7)$$

Der Rückwärtsthroughlauf (Backward Pass)

Wir wenden die Kettenregel an, um die Ableitung nach den Parametern zu berechnen. Dies entspricht einer Propagation der Abweichung in den Ergebnissen, d.h. des Fehlers, vom Ende des Netzwerks zum Anfang; darum heisst dies *backpropagation*. Wir definieren den Fehlerterm $\delta^{(l)}$ als den Gradienten des Fehlers bezüglich $\mathbf{a}^{(l)}$.

Wir beginnen mit der Ableitung nach der Ausgabe $\hat{\mathbf{y}}$:

$$\frac{\partial J}{\partial \hat{\mathbf{y}}} = (\hat{\mathbf{y}} - \mathbf{y}) \quad (8.3.8)$$

Da die Ausgabe linear ist ($\hat{\mathbf{y}} = \mathbf{a}^{(3)}$), ist der Fehlerterm $\delta^{(3)}$ identisch:

$$\delta^{(3)} = \frac{\partial J}{\partial \mathbf{a}^{(3)}} = (\hat{\mathbf{y}} - \mathbf{y}) \in \mathbb{R}^{1 \times 3} \quad (8.3.9)$$

Die Gradienten für die Parameter von Schicht 3 sind:

$$\frac{\partial J}{\partial W^{(3)}} = (\mathbf{z}^{(2)})^T \cdot \boldsymbol{\delta}^{(3)} \in \mathbb{R}^{6 \times 3} \quad (8.3.10)$$

$$\frac{\partial J}{\partial \mathbf{b}^{(3)}} = \boldsymbol{\delta}^{(3)} \quad (8.3.11)$$

Wir propagieren den Fehler $\boldsymbol{\delta}^{(3)}$ zurück durch die Gewichtsmatrix $W^{(3)}$. Dabei müssen wir die Ableitung der Aktivierungsfunktion $\sigma'(\mathbf{a}^{(2)})$ berücksichtigen. Wir verwenden die Notation Hadamard-Produkt ($\odot$) für die elementweise Multiplikation.

$$\boldsymbol{\delta}^{(2)} = \left(\boldsymbol{\delta}^{(3)} (W^{(3)})^T \right) \odot \sigma'(\mathbf{a}^{(2)}) \in \mathbb{R}^{1 \times 6} \quad (8.3.12)$$

Die Gradienten für die Parameter von Schicht 2 sind:

$$\frac{\partial J}{\partial W^{(2)}} = (\mathbf{z}^{(1)})^T \cdot \boldsymbol{\delta}^{(2)} \in \mathbb{R}^{6 \times 6} \quad (8.3.13)$$

$$\frac{\partial J}{\partial \mathbf{b}^{(2)}} = \boldsymbol{\delta}^{(2)} \quad (8.3.14)$$

Analog propagieren wir den Fehler $\boldsymbol{\delta}^{(2)}$ zurück durch $W^{(2)}$:

$$\boldsymbol{\delta}^{(1)} = \left(\boldsymbol{\delta}^{(2)} (W^{(2)})^T \right) \odot \sigma'(\mathbf{a}^{(1)}) \in \mathbb{R}^{1 \times 6} \quad (8.3.15)$$

Die Gradienten für die Parameter von Schicht 1 sind:

$$\frac{\partial J}{\partial W^{(1)}} = (\mathbf{z}^{(0)})^T \cdot \boldsymbol{\delta}^{(1)} \in \mathbb{R}^{1 \times 6} \quad (8.3.16)$$

$$\frac{\partial J}{\partial \mathbf{b}^{(1)}} = \boldsymbol{\delta}^{(1)} \quad (8.3.17)$$

Zusammenfassung der Berechnungsschritte

Für eine einzelne Iteration des Gradientenabstiegs führt der Algorithmus Folgendes aus:

1. **Forward Pass:** Berechne $\mathbf{z}^{(0)}, \mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \hat{\mathbf{y}}$.
2. **Fehlerberechnung:** $\boldsymbol{\delta}^{(3)} = \hat{\mathbf{y}} - \mathbf{y}$.
3. **Backward Pass (Hidden 2):** $\boldsymbol{\delta}^{(2)} = (\boldsymbol{\delta}^{(3)} W^{(3)T}) \odot \sigma'(\mathbf{a}^{(2)})$.
4. **Backward Pass (Hidden 1):** $\boldsymbol{\delta}^{(1)} = (\boldsymbol{\delta}^{(2)} W^{(2)T}) \odot \sigma'(\mathbf{a}^{(1)})$.
5. **Gradienten-Update:**
 - $W^{(l)} \leftarrow W^{(l)} - \eta \cdot (\mathbf{z}^{(l-1)T} \boldsymbol{\delta}^{(l)})$
 - $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \cdot \boldsymbol{\delta}^{(l)}$

Bei sehr tiefen Netzwerken (viele Schichten) greift das Prinzip der **dynamischen Programmierung**. Anstatt die Kettenregel für die erste Schicht vollständig neu vom Ausgang her zu berechnen, nutzen wir die Zwischenergebnisse. Der Fehlerterm $\delta^{(2)}$ enthält bereits alle Informationen über die nachfolgenden Schichten (hier Schicht 3). Zur Berechnung von $\delta^{(1)}$ ist lediglich eine Matrixmultiplikation mit $W^{(2)}$ nötig. Der Rechenaufwand wächst somit nur linear mit der Tiefe des Netzwerks.

In der Praxis, insbesondere bei der Interpolation vieler Messpunkte, verarbeiten wir nicht einen einzelnen Vektor $\mathbf{z}^{(0)}$, sondern eine Matrix $\mathbf{Z}^{(0)} \in \mathbb{R}^{N \times 1}$, wobei N die Anzahl der Messungen ist. Die Operationen ändern sich von Vektor-Matrix-Produkten zu **Matrix-Matrix-Produkten**:

$$\mathbf{Z}^{(1)} = \sigma(\mathbf{Z}^{(0)}W^{(1)} + \mathbf{B}^{(1)}) \quad (8.3.18)$$

Moderne Hardware (GPUs) ist hochgradig auf parallele Matrixoperationen optimiert. Der Aufwand, den Gradienten für N Messungen gleichzeitig zu berechnen, ist pro Messpunkt um Größenordnungen geringer als bei der sequentiellen Berechnung. Der Gewinn an Geschwindigkeit ist umso grösser, je weniger Genauigkeit verwendet wird. Da Machine Learning keine grosse Genauigkeit braucht (Messfehler und Zufall dominieren), werden zurzeit GPUs verwendet, die sogar mit weniger als einfacher Genauigkeit rechnen.

Hier ist der Simulationscode (DNN+ADAM) für ein Interpolationsproblem; die Ergebnisse sind in Abbildung 8.3.5 dargestellt:

Code 8.3.4: Interpolation mit dem DNN+ADAM

```

1  import numpy as np
   import matplotlib.pyplot as plt
3  from scipy.interpolate import make_interp_spline
   from scipy.optimize import minimize_scalar
5  from numpy.random import default_rng

7  seed = 1903
   rng = default_rng(seed)
9  # =====
   # CONFIGURATION
11 # =====
   # Choose Optimization Algorithm: 'SD', 'LineSearch', 'SGD', 'Adam'
13 #OPTIMIZER = 'SD'
   #OPTIMIZER = 'LineSearch'
15 #OPTIMIZER = 'SGD'
   OPTIMIZER = 'Adam'

17 # Choose Activation Function: 'Sigmoid', 'ReLU', 'Tanh'
19 #ACTIVATION = 'Sigmoid'
   #ACTIVATION = 'ReLU'
21 ACTIVATION = 'Tanh'

23 # Data Generation Settings

```

```

NUM_MEASUREMENTS = 15 # Number of points available for training
25 NOISE_LEVEL = 0.0    # Set > 0 to simulate noisy measurements
EPOCHS = 5000         # Number of training iterations
27 BATCH_SIZE = 16     # Only used for SGD

29 # AUTO-TUNE LEARNING RATE
# Adam needs a much smaller LR than Steepest Descent
31 if OPTIMIZER == 'Adam':
    LEARNING_RATE = 0.01 # Safe default for Adam
33 elif ACTIVATION == 'ReLU':
    LEARNING_RATE = 0.05 # ReLU can be unstable with high LR
35 else:
    LEARNING_RATE = 0.5  # Aggressive default for Sigmoid/SD
37
39 print(f"--- Configuration ---")
41 print(f"Optimizer:    {OPTIMIZER}")
43 print(f"Activation:     {ACTIVATION}")
45 print(f"Learning Rate: {LEARNING_RATE}")
47 print(f"-----")
49
51 # =====
53 # 1. DATA GENERATION
55 # =====
57
59 def true_function(t):
    """
    Function f: [0,1] -> R3
    t is expected to be shape (N, 1)
    """
    # Component 1: Sine wave
    y1 = np.sin(4 * np.pi * t)
    # Component 2: Decaying Cosine
    y2 = np.cos(4 * np.pi * t) * np.exp(-2 * t)
    # Component 3: Quadratic curve
    y3 = 2 * (t - 0.5)**2
    y3 = 1 / (1 + 5 * (2 * t - 1)**2) # Runge
    """y3 = 1 / (1 + 10 * 5 * (2 * t - 1)**2) # steeper"""

    return np.column_stack((y1, y2, y3))

63 # Generate "Ground Truth" for plotting (dense grid)
t_dense = np.linspace(0, 1, 200).reshape(-1, 1)
65 y_dense = true_function(t_dense)

67 # Generate "Measurements" for training (sparse points)
"""t_train = np.sort(rng.random( (NUM_MEASUREMENTS, 1)), axis=0)"""
69 t_train = np.linspace(0, 1, NUM_MEASUREMENTS, endpoint=True).reshape(
    (NUM_MEASUREMENTS, 1) )
y_train_clean = true_function(t_train)
71 y_train = y_train_clean + rng.normal(0, NOISE_LEVEL, y_train_clean.shape)

73 # =====
75 # 2. NEURAL NETWORK CLASS [1 -> 6 -> 6 -> 3]
# =====
class DNN_Interpolator:

```

```

77     def __init__(self, activation_type='Sigmoid'):
78         self.act_type = activation_type
79
80         # Architecture: Input(1) -> Hidden(6) -> Hidden(6) -> Output(3)
81         # Initialization logic (He/Xavier)
82         if activation_type == 'ReLU':
83             scale = np.sqrt(2.0)
84         else:
85             scale = 1.0
86
87         # Weights and Biases
88         # Layer 1: 1 -> 6
89         self.W1 = np.random.randn(1, 6) * scale
90         self.b1 = np.zeros((1, 6))
91
92         # Layer 2: 6 -> 6
93         self.W2 = np.random.randn(6, 6) * scale * np.sqrt(1/6)
94         self.b2 = np.zeros((1, 6))
95
96         # Layer 3: 6 -> 3
97         self.W3 = np.random.randn(6, 3) * scale * np.sqrt(1/6)
98         self.b3 = np.zeros((1, 3))
99
100        # Adam specific memory
101        self.m = None
102        self.v = None
103
104        def activate(self, x):
105            if self.act_type == 'Sigmoid': return 1 / (1 + np.exp(-x))
106            if self.act_type == 'ReLU': return np.maximum(0, x)
107            if self.act_type == 'Tanh': return np.tanh(x)
108
109        def activate_deriv(self, x):
110            if self.act_type == 'Sigmoid':
111                s = 1 / (1 + np.exp(-x))
112                return s * (1 - s)
113            if self.act_type == 'ReLU': return (x > 0).astype(float)
114            if self.act_type == 'Tanh': return 1 - np.tanh(x)**2
115
116        def forward(self, t, params=None):
117            """
118            Forward pass. Supports temporary parameters for Line Search.
119            """
120            if params is None:
121                W1, b1, W2, b2, W3, b3 = self.W1, self.b1, self.W2, self.b2,
122                self.W3, self.b3
123            else:
124                W1, b1, W2, b2, W3, b3 = params
125
126            # Layer 1
127            a1 = np.dot(t, W1) + b1
128            z1 = self.activate(a1)
129
130            # Layer 2

```

```

131     a2 = np.dot(z1, W2) + b2
132     z2 = self.activate(a2)
133
134     # Layer 3 (Linear Output for Interpolation)
135     a3 = np.dot(z2, W3) + b3
136     y_pred = a3
137
138     return y_pred, z1, z2, a1, a2
139
140 def get_gradients(self, t, y_true):
141     """Standard Backpropagation"""
142     m = t.shape[0]
143     y_pred, z1, z2, a1, a2 = self.forward(t)
144
145     # MSE Derivative: (ypred - ytrue)
146     dz3 = (y_pred - y_true)
147
148     # Layer 3 Gradients
149     dW3 = np.dot(z2.T, dz3) / m
150     db3 = np.sum(dz3, axis=0, keepdims=True) / m
151
152     # Backprop to Layer 2
153     da2 = np.dot(dz3, self.W3.T) * self.activate_deriv(a2)
154     dW2 = np.dot(z1.T, da2) / m
155     db2 = np.sum(da2, axis=0, keepdims=True) / m
156
157     # Backprop to Layer 1
158     da1 = np.dot(da2, self.W2.T) * self.activate_deriv(a1)
159     dW1 = np.dot(t.T, da1) / m
160     db1 = np.sum(da1, axis=0, keepdims=True) / m
161
162     return [dW1, db1, dW2, db2, dW3, db3]
163
164 # -- OPTIMIZATION ALGORITHMS --
165
166 def train_step_SD(self, t, y, lr):
167     """Steepest Descent with fixed step"""
168     grads = self.get_gradients(t, y)
169     self.W1 -= lr * grads[0]
170     self.b1 -= lr * grads[1]
171     self.W2 -= lr * grads[2]
172     self.b2 -= lr * grads[3]
173     self.W3 -= lr * grads[4]
174     self.b3 -= lr * grads[5]
175     return np.mean((self.forward(t)[0] - y)**2)
176
177 def train_step_LineSearch(self, t, y):
178     """Steepest Descent with Scipy Line Search"""
179     grads = self.get_gradients(t, y)
180
181     def objective(alpha):
182         p = [
183             self.W1 - alpha*grads[0], self.b1 - alpha*grads[1],
184             self.W2 - alpha*grads[2], self.b2 - alpha*grads[3],

```



```

        self.W3 -= alpha*grads[4], self.b3 -= alpha*grads[5]
    ]
    return np.mean((self.forward(t, params=p)[0] - y)**2)

res = minimize_scalar(objective, bounds=(0.0, 10.0), method='bounded')
best_alpha = res.x

self.W1 -= best_alpha * grads[0]
self.b1 -= best_alpha * grads[1]
self.W2 -= best_alpha * grads[2]
self.b2 -= best_alpha * grads[3]
self.W3 -= best_alpha * grads[4]
self.b3 -= best_alpha * grads[5]
return objective(best_alpha)

def train_step_SGD(self, t, y, lr, batch_size):
    """Stochastic Gradient Descent"""
    indices = np.random.permutation(t.shape[0])
    t_shuff = t[indices]
    y_shuff = y[indices]

    losses = []
    for i in range(0, t.shape[0], batch_size):
        t_batch = t_shuff[i:i+batch_size]
        y_batch = y_shuff[i:i+batch_size]

        grads = self.get_gradients(t_batch, y_batch)
        self.W1 -= lr * grads[0]
        self.b1 -= lr * grads[1]
        self.W2 -= lr * grads[2]
        self.b2 -= lr * grads[3]
        self.W3 -= lr * grads[4]
        self.b3 -= lr * grads[5]

        losses.append(np.mean((self.forward(t_batch)[0] - y_batch)**2))
    return np.mean(losses)

def train_step_Adam(self, t, y, lr, beta1=0.9, beta2=0.999, eps=1e-8):
    """Adam Optimizer"""
    grads = self.get_gradients(t, y) # List of 6 gradients (W1, b1, W2, b2,
W3, b3)

    # List of references to the actual parameters for easy looping
    params_list = [self.W1, self.b1, self.W2, self.b2, self.W3, self.b3]

    # Initialize Adam memory (Momentum m and Velocity v)
    if self.m is None:
        self.m = [np.zeros_like(p) for p in params_list]
        self.v = [np.zeros_like(p) for p in params_list]
        self.t_adam = 0

    self.t_adam += 1

    # Update loop

```

```

237     for i in range(6):
238         g = grads[i]
239
240         # 1. Update biased first moment estimate (Momentum)
241         self.m[i] = beta1 * self.m[i] + (1 - beta1) * g
242
243         # 2. Update biased second raw moment estimate (Velocity)
244         self.v[i] = beta2 * self.v[i] + (1 - beta2) * (g**2)
245
246         # 3. Compute bias-corrected first moment estimate
247         m_hat = self.m[i] / (1 - beta1**self.t_adam)
248
249         # 4. Compute bias-corrected second raw moment estimate
250         v_hat = self.v[i] / (1 - beta2**self.t_adam)
251
252         # 5. Update parameters (In-place modification of the array)
253         params_list[i] -= lr * m_hat / (np.sqrt(v_hat) + eps)
254
255     return np.mean((self.forward(t)[0] - y)**2)
256
257 # =====
258 # 3. TRAINING LOOP
259 # =====
260 model = DNN_Interpolator(activation_type=ACTIVATION)
261 loss_history = []
262
263 print(f"Starting Training...")
264
265 for epoch in range(EPOCHS):
266     if OPTIMIZER == 'SD':
267         loss = model.train_step_SD(t_train, y_train, LEARNING_RATE)
268     elif OPTIMIZER == 'LineSearch':
269         loss = model.train_step_LineSearch(t_train, y_train)
270     elif OPTIMIZER == 'SGD':
271         loss = model.train_step_SGD(t_train, y_train, LEARNING_RATE, BATCH_SIZE)
272     elif OPTIMIZER == 'Adam':
273         loss = model.train_step_Adam(t_train, y_train, LEARNING_RATE)
274
275     loss_history.append(loss)
276     if epoch % (EPOCHS // 10) == 0:
277         print(f"Epoch {epoch}: Loss = {loss:.6f}")
278
279 print(f"Final Loss: {loss_history[-1]:.6f}")
280
281 # Predict on dense grid for visualization
282 y_pred_dense = model.forward(t_dense)[0]
283
284 # =====
285 # 4. SPLINE INTERPOLATION (Comparison)
286 # =====
287 # We interpolate each dimension separately
288 # k=3 means Cubic Spline, k=2 is Quadratic
289 # If few points, we might need lower order or B-splines
290 k_order = 3 if NUM_MEASUREMENTS > 3 else 1

```

```

291 # Spline for dim 1
293 spl1 = make_interp_spline(t_train.flatten(), y_train[:, 0], k=k_order)
    y_spline1 = spl1(t_dense.flatten())
295
297 # Spline for dim 2
    spl2 = make_interp_spline(t_train.flatten(), y_train[:, 1], k=k_order)
    y_spline2 = spl2(t_dense.flatten())
299
301 # Spline for dim 3
    spl3 = make_interp_spline(t_train.flatten(), y_train[:, 2], k=k_order)
    y_spline3 = spl3(t_dense.flatten())
303
    y_spline_dense = np.column_stack((y_spline1, y_spline2, y_spline3))
305
    # =====
307 # 5. PLOTTING (The 3 Subplots)
    # =====
309 fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 15), sharex=True)
    colors = ['red', 'green', 'blue']
311 labels = ['y1', 'y2', 'y3']
313
313 # -- Plot 1: Ground Truth + Measurements --
    ax1.set_title("1. Ground Truth Function & Measurements")
315 for i in range(3):
        ax1.plot(t_dense, y_dense[:, i], color=colors[i], label=f'True {labels[i]}',
            linestyle='--')
317         ax1.scatter(t_train, y_train[:, i], color=colors[i], s=40,
            edgecolors='black', label=f'Meas {labels[i]}')
        ax1.legend(loc='upper right', fontsize='small', ncol=3)
319 ax1.grid(True, alpha=0.3)
321
321 # -- Plot 2: Measurements + DNN Interpolation --
    ax2.set_title(f"2. DNN Interpolation ({OPTIMIZER}, {ACTIVATION})")
323 for i in range(3):
        # Plot measurements again for context
325         ax2.scatter(t_train, y_train[:, i], color=colors[i], s=40, alpha=0.5)
        # Plot DNN result
327         ax2.plot(t_dense, y_pred_dense[:, i], color=colors[i], linewidth=2,
            label=f'DNN {labels[i]}')
        ax2.legend(loc='upper right', fontsize='small', ncol=3)
329 ax2.grid(True, alpha=0.3)
        ax2.text(0.02, 0.95, f"Loss: {loss_history[-1]:.1e}", transform=ax2.transAxes,
            bbox=dict(facecolor='white', alpha=0.8))
331
331 # -- Plot 3: Measurements + Spline Interpolation --
333 ax3.set_title("3. Spline Interpolation (Scipy)")
    for i in range(3):
        # Plot measurements again
335         ax3.scatter(t_train, y_train[:, i], color=colors[i], s=40, alpha=0.5)
        # Plot Spline result
337         ax3.plot(t_dense, y_spline_dense[:, i], color=colors[i], linewidth=2,
            linestyle='-.', label=f'Spline {labels[i]}')
339

```

```
ax3.set_xlabel("Input t")
341 ax3.legend(loc='upper right', fontsize='small', ncol=3)
ax3.grid(True, alpha=0.3)
343
plt.tight_layout()
345 plt.savefig('dnn_interpolation_result.png')
plt.savefig('dnn_interpolation_result.eps')
347 plt.show()
```

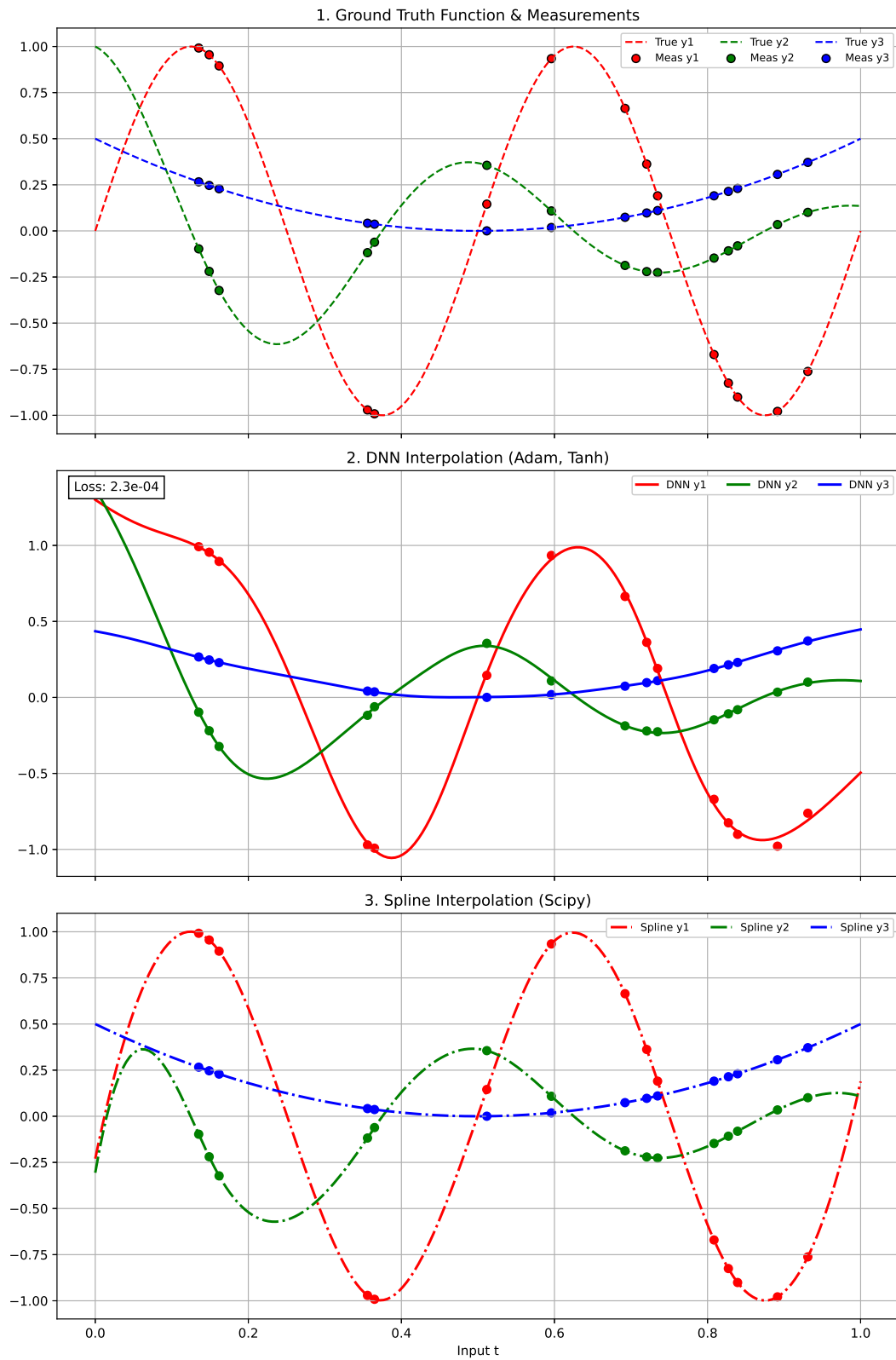


Abb. 8.3.5. Ergebnisse der Interpolation mit DNNs und ADAM

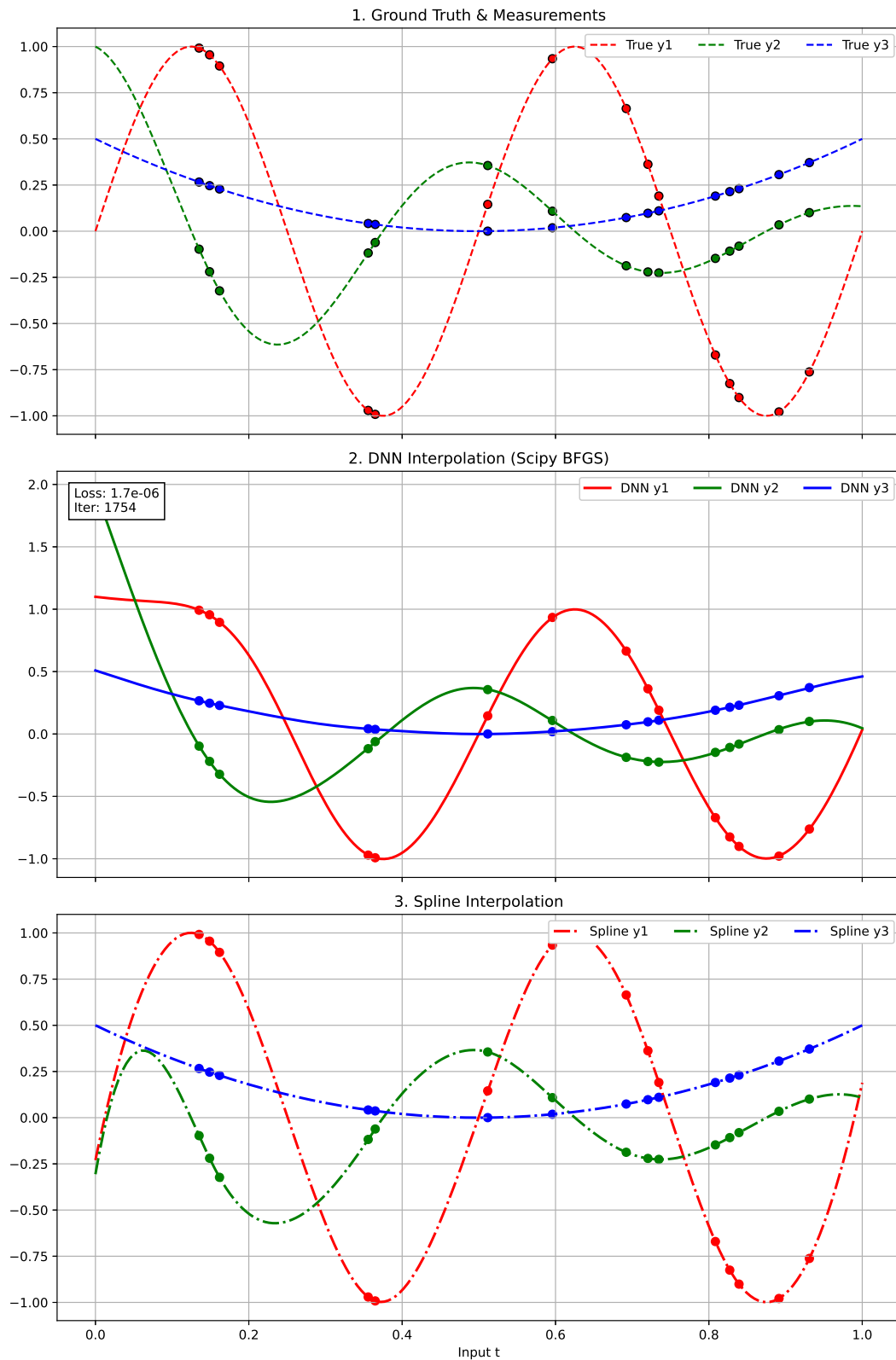


Abb. 8.3.6. Ergebnisse der Interpolation mit DNNs und BFGS

Hier ist der Simulationscode (DNN+BFGS) für ein Interpolationsproblem; die

Ergebnisse sind in Abbildung 8.3.6 dargestellt:

Code 8.3.7: Interpolation mit dem DNN+BFGS

```

1  import numpy as np
   import matplotlib.pyplot as plt
3  from scipy.optimize import minimize
   from scipy.interpolate import make_interp_spline
5  from numpy.random import default_rng

7  seed = 1903
   rng = default_rng(seed)

9  # =====
11 # CONFIGURATION
   # =====
13 # Options: 'BFGS', 'L-BFGS-B', 'CG', 'Newton-CG'
   SCIPY_METHOD = 'BFGS'

15 # Options: 'Sigmoid', 'Tanh' (ReLU is not recommended for BFGS due to
   non-smoothness)
17 ACTIVATION = 'Tanh'

19 # Data Settings
   NUM_MEASUREMENTS = 15 # OK
21 NOISE_LEVEL = 0.0

23 # =====
   # 1. DATA GENERATION
25 # =====

   def true_function(t):
27     # Component 1: Sine wave
       y1 = np.sin(4 * np.pi * t)
29     # Component 2: Decaying Cosine
       y2 = np.cos(4 * np.pi * t) * np.exp(-2 * t)
31     # Component 3: Quadratic curve
       #y3 = 2 * (t - 0.5)**2
33     y3 = 1 / (1 + 5 * (2 * t - 1)**2) # Runge
       """y3 = 1 / (1 + 10 * 5 * (2 * t - 1)**2) # steeper"""
35     return np.column_stack((y1, y2, y3))

37 # Dense grid for ground truth plotting
   t_dense = np.linspace(0, 1, 200).reshape(-1, 1)
39 y_dense = true_function(t_dense)

41 # Sparse training measurements
   #np.random.seed(42)
43 t_train = np.sort(rng.uniform(0, 1, (NUM_MEASUREMENTS, 1)), axis=0)
       """t_train = np.linspace(0,1,NUM_MEASUREMENTS, endpoint=True).reshape(
   (NUM_MEASUREMENTS, 1) )"""
45 y_train_clean = true_function(t_train)
   y_train = y_train_clean + rng.normal(0, NOISE_LEVEL, y_train_clean.shape)
47

```

```

# =====
49 # 2. NEURAL NETWORK CLASS (Scipy Compatible)
# =====
51 class ScipyDNN:
    def __init__(self, activation='Tanh'):
53         self.act_type = activation

55         # Architecture Definition [1, 6, 6, 3]
        self.shapes = [
57             (1, 6), # W1
59             (1, 6), # b1
61             (6, 6), # W2
63             (1, 6), # b2
65             (6, 3), # W3
67             (1, 3)  # b3
69         ]

71         """# More neurons allow the network to fit more complex shapes
73         # but make optimization harder and slower.
75         self.shapes = [
77             (1, 10), # Increase neurons in hidden layer 1
79             (1, 10),
81             (10, 10), # Increase neurons in hidden layer 2
83             (1, 10),
85             (10, 3),
87             (1, 3)
89         ]

91         """
93         # Calculate total number of parameters
95         self.num_params = sum(r*c for r, c in self.shapes)

97         # Initialize parameters randomly
99         scale = np.sqrt(2.0) if activation == 'ReLU' else 1.0
101        self.params = rng.standard_normal(self.num_params) * scale * 0.5

    def activate(self, x):
        if self.act_type == 'Sigmoid': return 1 / (1 + np.exp(-x))
        if self.act_type == 'Tanh': return np.tanh(x)
        if self.act_type == 'ReLU': return np.maximum(0, x)

    def activate_deriv(self, x):
        if self.act_type == 'Sigmoid':
            s = 1 / (1 + np.exp(-x))
            return s * (1 - s)
        if self.act_type == 'Tanh': return 1 - np.tanh(x)**2
        if self.act_type == 'ReLU': return (x > 0).astype(float)

    def unpack_params(self, theta):
        """Reconstruct weight matrices from the 1D parameter vector."""
        weights = []
        idx = 0
        for rows, cols in self.shapes:
            size = rows * cols
            w = theta[idx : idx + size].reshape(rows, cols)

```



```

        weights.append(w)
        idx += size
    return weights # [W1, b1, W2, b2, W3, b3]

def forward(self, t, theta):
    W1, b1, W2, b2, W3, b3 = self.unpack_params(theta)

    a1 = np.dot(t, W1) + b1
    z1 = self.activate(a1)

    a2 = np.dot(z1, W2) + b2
    z2 = self.activate(a2)

    a3 = np.dot(z2, W3) + b3
    y_pred = a3 # Linear output

    return y_pred, z1, z2, a1, a2

def objective_function(self, theta, t, y_true):
    """
    The function passed to scipy.optimize.minimize.
    Returns: (Loss, Gradient)
    """
    m = t.shape[0]
    W1, b1, W2, b2, W3, b3 = self.unpack_params(theta)

    # -- Forward --
    y_pred, z1, z2, a1, a2 = self.forward(t, theta)

    # MSE Loss
    loss = 0.5 * np.mean((y_pred - y_true)**2) # 0.5 factor for cleaner
derivatives

    # -- Backward (Gradient Computation) --
    # Note: We divide by m here to match the mean in the loss

    dz3 = (y_pred - y_true) / m

    dW3 = np.dot(z2.T, dz3)
    db3 = np.sum(dz3, axis=0, keepdims=True)

    da2 = np.dot(dz3, W3.T) * self.activate_deriv(a2)
    dW2 = np.dot(z1.T, da2)
    db2 = np.sum(da2, axis=0, keepdims=True)

    da1 = np.dot(da2, W2.T) * self.activate_deriv(a1)
    dW1 = np.dot(t.T, da1)
    db1 = np.sum(da1, axis=0, keepdims=True)

    # Flatten gradients into a single 1D vector
    grads = [dW1, db1, dW2, db2, dW3, db3]
    grad_flat = np.concatenate([g.flatten() for g in grads])

    return loss, grad_flat

```

```

# =====
157 # 3. OPTIMIZATION
# =====
159 model = ScipyDNN(activation=ACTIVATION)

161 print(f"Optimizing using Scipy {SCIPY_METHOD} with {ACTIVATION}...")
161 print(f"Total Parameters: {model.num_params}")
163
# We define a lambda or partial function to fix the data arguments
165 fun = lambda theta: model.objective_function(theta, t_train, y_train)

167 # Run Optimization
# jac=True tells Scipy that our function returns (value, gradient)
169 res = minimize(fun, model.params, method=SCIPY_METHOD, jac=True,
options={'disp': True, 'maxiter': 2000})

171 print(f"Optimization Success: {res.success}")
171 print(f"Final Loss: {res.fun:.6f}")
173 print(f"Iterations: {res.nit}")

175 # Update model with optimized parameters
best_theta = res.x
177 y_pred_dense, _, _, _ = model.forward(t_dense, best_theta)

179 # =====
# 4. SPLINE INTERPOLATION (Comparison)
# =====
181
k_order = 3 if NUM_MEASUREMENTS > 3 else 1
183 y_spline_dense = np.zeros_like(y_dense)

185 for i in range(3):
    spl = make_interp_spline(t_train.flatten(), y_train[:, i], k=k_order)
187 y_spline_dense[:, i] = spl(t_dense.flatten())

189 # =====
# 5. PLOTTING
# =====
191
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 15), sharex=True)
193 colors = ['red', 'green', 'blue']
labels = ['y1', 'y2', 'y3']
195
# Plot 1: Ground Truth
197 ax1.set_title("1. Ground Truth & Measurements")
for i in range(3):
199 ax1.plot(t_dense, y_dense[:, i], color=colors[i], linestyle='--',
label=f'True {labels[i]}')
    ax1.scatter(t_train, y_train[:, i], color=colors[i], s=40, edgecolors='k')
201 ax1.legend(ncol=3)
ax1.grid(True, alpha=0.3)
203
# Plot 2: DNN Result
205 ax2.set_title(f"2. DNN Interpolation (Scipy {SCIPY_METHOD})")
for i in range(3):
207 ax2.scatter(t_train, y_train[:, i], color=colors[i], s=40, alpha=0.3)

```

```

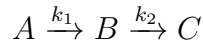
    ax2.plot(t_dense, y_pred_dense[:, i], color=colors[i], linewidth=2,
label=f'DNN {labels[i]}')
209 ax2.text(0.02, 0.9, f"Loss: {res.fun:.1e}\nIter: {res.nit}",
transform=ax2.transAxes, bbox=dict(facecolor='white', alpha=0.8))
ax2.legend(ncol=3)
211 ax2.grid(True, alpha=0.3)

# Plot 3: Spline Result
ax3.set_title("3. Spline Interpolation")
213
215 for i in range(3):
    ax3.scatter(t_train, y_train[:, i], color=colors[i], s=40, alpha=0.3)
    ax3.plot(t_dense, y_spline_dense[:, i], color=colors[i], linewidth=2,
217 linestyle='-.', label=f'Spline {labels[i]}')
ax3.legend(ncol=3)
219 ax3.grid(True, alpha=0.3)
ax3.set_xlabel("Input t")
221

plt.tight_layout()
223 plt.savefig('dnn_scipy_bfgs.png')
plt.savefig('dnn_scipy_bfgs.eps')
225 plt.show()

```

Beispiel 8.3.8. (Reaktionskinetik mit PINNs = Physically Inspired Neuronal Networks) Wir betrachten ein System gewöhnlicher Differentialgleichungen (ODE), welches eine chemische Reaktionskette erster Ordnung beschreibt. Eine Substanz A wandelt sich in Substanz B um, welche sich wiederum in Substanz C umwandelt:



Das mathematische Modell beschreibt die Konzentrationen $C_a(t), C_b(t), C_c(t)$ über die Zeit $t \in [0, 10]$:

$$\frac{dC_a}{dt} = -k_1 C_a \quad (8.3.19)$$

$$\frac{dC_b}{dt} = k_1 C_a - k_2 C_b \quad (8.3.20)$$

$$\frac{dC_c}{dt} = k_2 C_b \quad (8.3.21)$$

Mit den Parametern $k_1 = 1, k_2 = 1$ und den Anfangswerten zum Zeitpunkt $t = 0$:

$$C(0) = [1, 0, 0]^T$$

Ziel ist es, ein DNN $f^*(t; \theta)$ zu trainieren, welches die Funktion $\mathbf{C}(t) : \mathbb{R} \rightarrow \mathbb{R}^3$ approximiert.

Das neuronale Netz fungiert als universeller Funktionsapproximator. Anstatt Messungen zu verwenden (was die Lösung der ODE bereits voraussetzen würde), nutzen wir die Differentialgleichung selbst als Zielfunktion (Loss Function). Diese setzt sich aus zwei Termen zusammen:

$$J(\theta) = J_{\text{Residuum}} + J_{\text{Anfangswert}} \quad (8.3.22)$$

- **Residuum:** $f^*(t; \theta)$ muss die ODE erfüllen.

$$J_{\text{Residuum}} = \frac{1}{N} \sum_{i=1}^N \left\| \frac{df^*(t_i)}{dt} - \text{RHS}(f^*(t_i)) \right\|^2$$

- **Anfangswert:** Das Netz muss bei $t = 0$ starten.

$$J_{\text{Anfangswert}} = \|f^*(0) - [1, 0, 0]^T\|^2$$

Wenn die Zielfunktion das mathematische Modell des Phänomens verwendet, in diesem Fall die Differentialgleichung der Reaktionskinetik, sprechen wir von PINNs = Physically Inspired Neuronal Networks. Die kürzere Rechenzeit mit PINNs im Vergleich zu Standard-Diskretisierungsmethoden für partielle Differentialgleichungen kann helfen, sehr komplizierte Probleme zu lösen, die sonst sehr viele Ressourcen bräuchten oder gar unmöglich zu behandeln wären. Jedoch ist das Design der Netze samt spezifischen Aktivierungsfunktionen und sonstigen Parametern, damit PINNs belastbare Resultate liefern, zurzeit eine schwierige Aufgabe, da es dazu noch ungenügend Theorie gibt. Im Folgenden beschreiben wir die Algorithmen in diesem sehr einfachen Fall.

Da das DNN eine Hintereinanderausführung differenzierbarer Funktionen ist (Matrizenmultiplikationen und Aktivierungsfunktionen σ), können wir die Ableitung nach der Zeit t exakt mittels der Kettenregel berechnen. Sei das Netzwerk wie vorher definiert als $[1, 6, 6, 3]$. Da die Eingabe t ein Skalar ist, vereinfacht sich die Berechnung.

Schicht 1:

$$\mathbf{a}^{(1)} = tW^{(1)} + \mathbf{b}^{(1)} \quad (8.3.23)$$

$$\frac{d\mathbf{a}^{(1)}}{dt} = W^{(1)} \quad (\text{Da } \frac{dt}{dt} = 1) \quad (8.3.24)$$

$$\frac{d\mathbf{z}^{(1)}}{dt} = \sigma'(\mathbf{a}^{(1)}) \odot \frac{d\mathbf{a}^{(1)}}{dt} = \sigma'(\mathbf{a}^{(1)}) \odot W^{(1)} \quad (8.3.25)$$

Schicht 2:

$$\mathbf{a}^{(2)} = \mathbf{z}^{(1)}W^{(2)} + \mathbf{b}^{(2)} \quad (8.3.26)$$

$$\frac{d\mathbf{a}^{(2)}}{dt} = \frac{d\mathbf{z}^{(1)}}{dt}W^{(2)} \quad (8.3.27)$$

$$\frac{d\mathbf{z}^{(2)}}{dt} = \sigma'(\mathbf{a}^{(2)}) \odot \frac{d\mathbf{a}^{(2)}}{dt} \quad (8.3.28)$$

Ausgabeschicht:

$$\frac{df^*}{dt} = \frac{d\mathbf{a}^{(3)}}{dt} = \frac{d\mathbf{z}^{(2)}}{dt}W^{(3)} \quad (8.3.29)$$

Da die Zielfunktion nun von $\frac{df^*}{dt}$ abhängt, müssen wir für den Gradientenabstieg die Ableitung der Ableitung berechnen. Dies erfordert die zweite Ableitung der Aktivierungsfunktionen (σ'').

Wenn die Zielfunktion J von der Steigung abhängt, dann gilt für ein Gewicht w in der Kette:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial(\frac{dy}{dt})} \cdot \frac{\partial(\frac{dy}{dt})}{\partial w}$$

Dies führt zu einer komplexeren Backpropagation, da w sowohl den Wert $y(t)$ als auch dessen Steigung beeinflusst. Zu bemerken ist auch, dass die ReLU-Aktivierungsfunktion hierfür ungeeignet ist, da ihre zweite Ableitung überall 0 ist; daher werden glatte Funktionen wie Sigmoid oder Tanh als Aktivierungsfunktion verwendet.

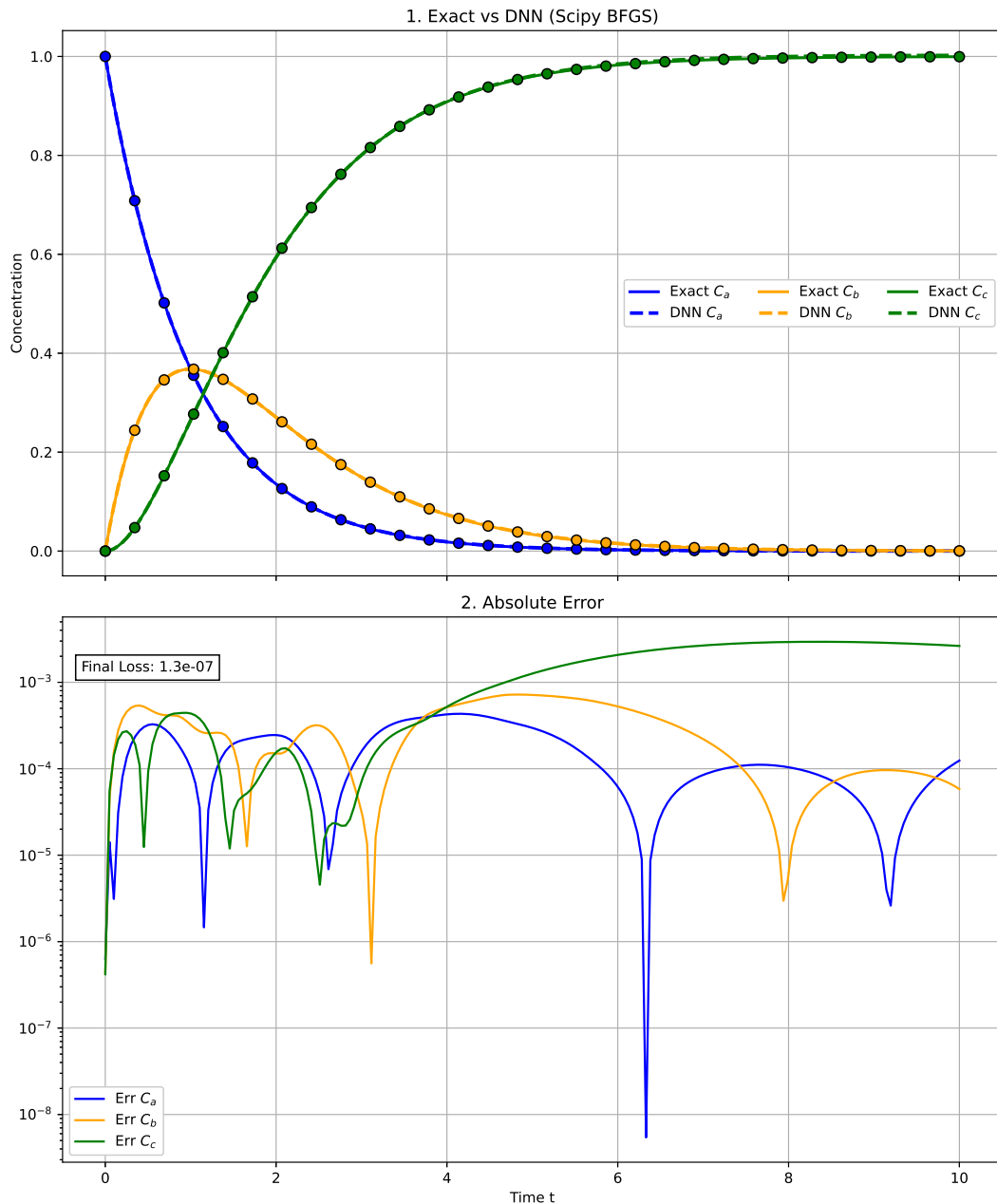


Abb. 8.3.9. ODE-Löser mit DNNs und BFGS

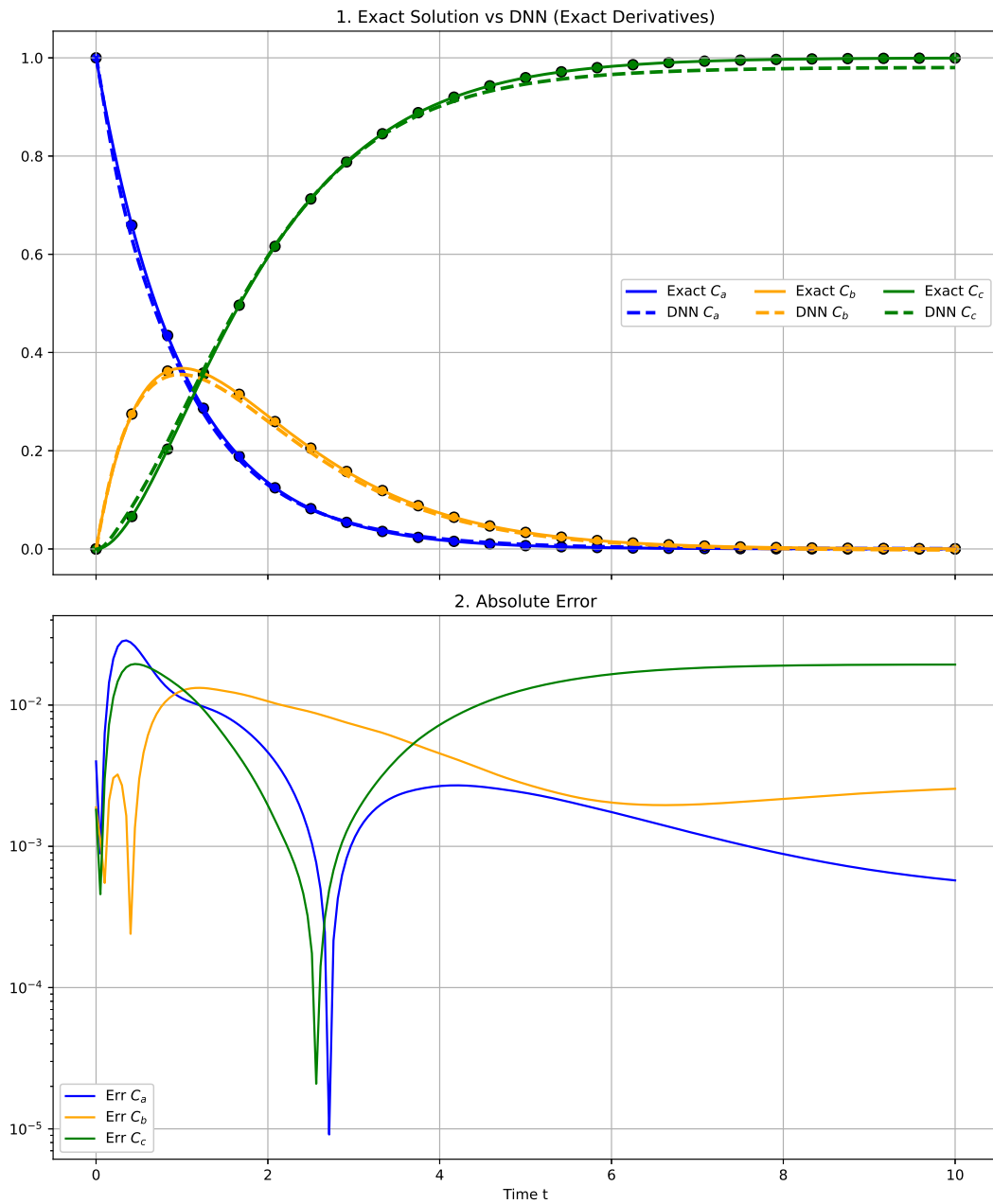


Abb. 8.3.10. ODE-Löser mit DNNs + ADAM - FD

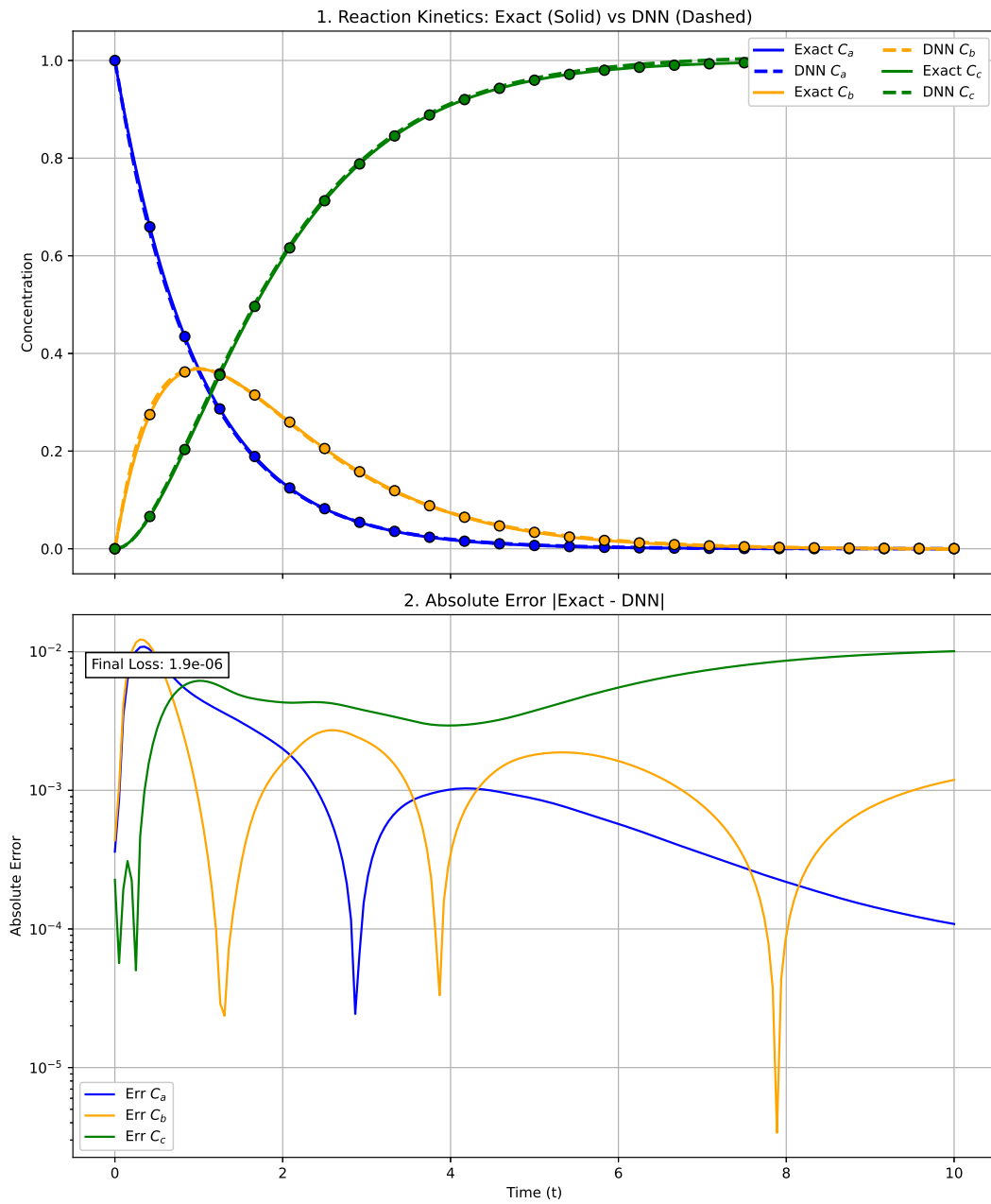


Abb. 8.3.11. ODE-Löser mit DNNs + ADAM + FD